

AI as an Infrastructure

Changkun Ou · latere.ai

PREFACE

Preface

L A T E R E . A I

HUMAN INTELLIGENCE
IN THE LOOP

INFRASTRUCTURE

THE OPERATING
SYSTEM FOR THE
AGE OF INTELLIGENCE

AI as an Infrastructure

From Systems to Agents:
History, Design Decisions,
and Foundations

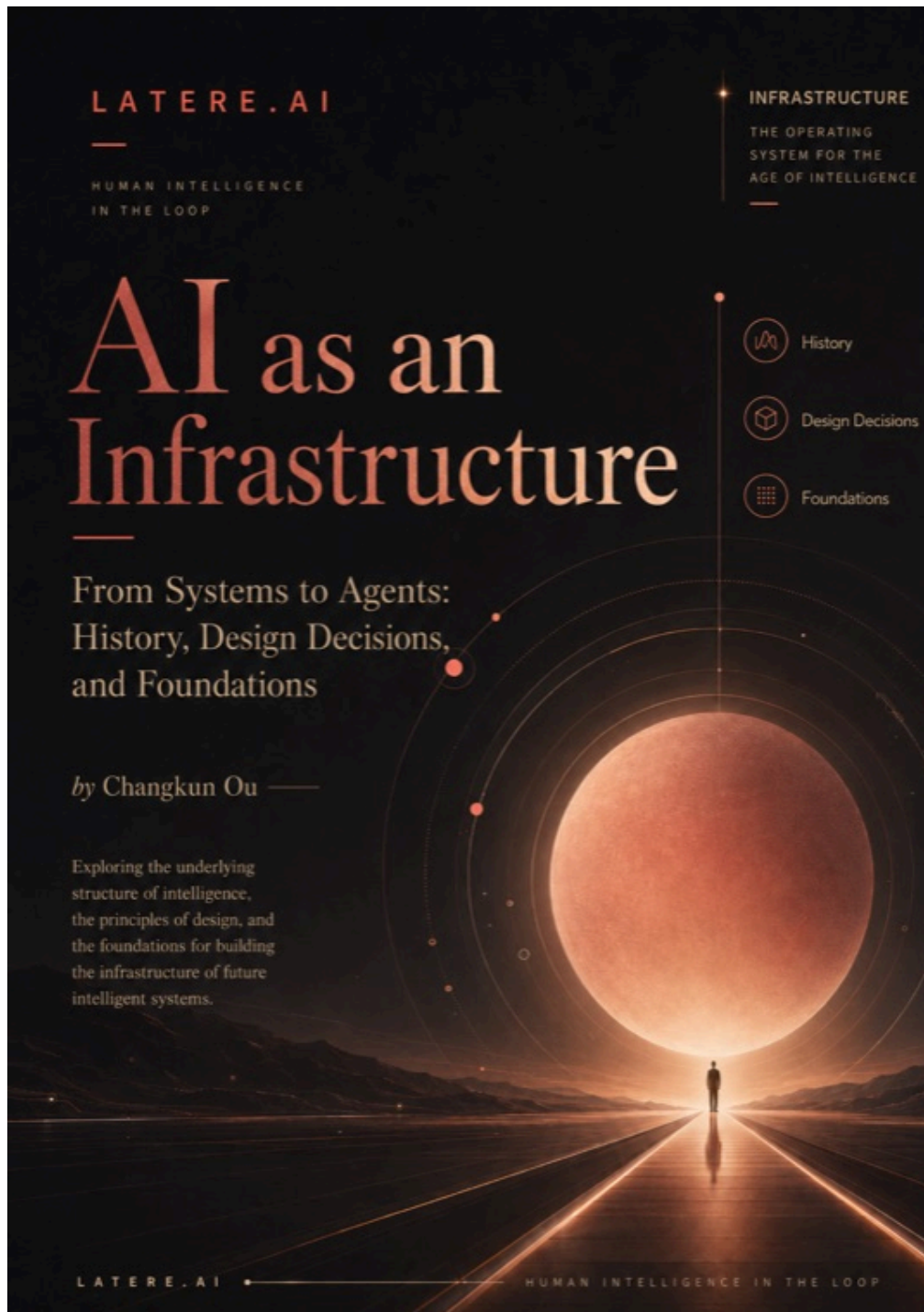
by Changkun Ou

Exploring the underlying
structure of intelligence,
the principles of design, and
the foundations for building
the infrastructure of future
intelligent systems.

- History
- Design Decisions
- Foundations

L A T E R E . A I

HUMAN INTELLIGENCE IN THE LOOP



This book treats artificial intelligence as an infrastructure: a layer that other software depends on, that must stay up, stay correct, stay affordable, and stay observable. It follows one continuous arc, the lifecycle of a capability, from raw compute to a deployed and governed behavior, and at every step it asks not only how the piece works but why it has the shape it does, what it gave up to get there, and what foundation it rests on.

A reader should finish a chapter able to answer "why is it built this way?", not only "how do I run it?". We reach that question the same way through every topic: state the problem and its constraints, give the design that answers it, trace how that design evolved and what it superseded, name the trade-offs, and only then go to implementation. The authoring conventions are recorded in the project's `CONVENTIONS.md`.

How this book is organized

The spine is the lifecycle of a capability, read as a stack from compute to deployed behavior.

- **Part 0, Orientation.** The whole stack in one pass, and how to read the book.
- **Part I, Foundations and Pretraining.** Scaling, data, tokenization, architecture, and training at scale.
- **Part II, Adaptation and Alignment.** Fine-tuning, RLHF, preference optimization, and self-improvement.
- **Part III, Reasoning and Test-Time Compute.** Eliciting and training reasoning, and inference-time scaling.
- **Part IV, Inference and Serving.** The serving problem, scheduling, faster decoding, quantization, and long context.
- **Part V, Orchestration.** Agents, memory, the harness, multi-agent systems, retrieval, and context.
- **Part VI, Evaluation.** Benchmarks, judging, and evaluating agents.
- **Part VII, Infrastructure and Systems.** Accelerators, networking, and the orchestration around a run.
- **Part VIII, Safety, Interpretability, and Governance.** Interpretability, oversight, and agent security.
- **Part IX, Ecosystem and Economics.** The model landscape, tooling, and the cost structure that feeds back into every choice above.
- **Part X, A Practical 2026 Stack.** A hands-on guide to choosing and wiring real tools: models, serving, training, agents, retrieval, and evaluation, with the whole stack assembled at the end.

Two motifs run through the book. The **three loops**, training, inference, and agentic, are a recurring way to see the same control structure at different layers. The **capability, efficiency, trust** lens closes each chapter: can it do the task, at what cost, and can you trust it. Watch for the constraint arrows, the places where a lower layer dictates an upper layer's choice, because they are the payoff of reading the stack in order.

Who this is for

Engineers who build and operate AI systems, and researchers who want to see how their theory lands in production. We assume comfort with one programming language, basic probability and linear algebra, and the ordinary tools of running software in production.

This book is a work in progress. Chapters carry outlines, source pointers, and open questions marked as `TODO` while drafting proceeds. Each chapter closes the live debates in a "what's contested" box rather than papering over them.

1. The Whole Stack in One Pass

This book is long, and the temptation is to read it as a list of unrelated techniques. This chapter is the antidote. By the end a reader can trace one ordinary request from raw text on the open web all the way to a deployed agent answering it, name the layer that handles each step, and point to the chapter that owns it. The point is orientation, not depth: every mechanism here is a forward pointer, deliberately left for its own chapter.

So pick one task and refuse to let go of it: *a user asks an assistant to find the bug in a small repository, propose a fix, and explain it*. That one request touches every layer this book covers. Rather than survey the layers abstractly, this chapter follows the request as if it were a traveler, and treats each layer it must pass through as a station on the route. Following it once, in order, is the fastest way to see why the stack has the shape it does. Two running models ride alongside the request the whole way, chosen because they answer the same questions with opposite engineering: Llama 3, a dense Transformer (Grattafiori and others 2024), and DeepSeek-V3, a mixture of experts (DeepSeek-AI 2024).

Station one: from web bytes to tokens

The task begins long before the user types it, in a corpus. The model that will eventually read the repository learned what code looks like from trillions of tokens scraped, filtered, and deduplicated from the web and from public repositories. Llama 3's largest model was pre-trained on 15.6 trillion tokens (Grattafiori and others 2024); DeepSeek-V3 on 14.8 trillion (DeepSeek-AI 2024). Those numbers are not the interesting part. The interesting part is everything done to the raw bytes before they counted as a token at all: source selection, quality filtering, near-duplicate removal, and decontamination against the very benchmarks the model will later be graded on. That work belongs to Chapter 4. The step that turns a filtered byte stream into the integer tokens the model actually consumes, and the vocabulary that decides how a variable name or a Chinese sentence is split, belongs to Chapter 5.

Station two: pre-training turns compute into a base model

With a token stream in hand, pre-training turns compute into a base model by minimizing next-token loss. How large a model to train on how many tokens, for a fixed budget, is the question of Chapter 3, first posed as a set of power laws relating loss to model size, data, and compute (Kaplan et al. 2020) and later sharpened into a compute-optimal balance between parameters and tokens (Hoffmann et al. 2022). What the network itself is built from, attention and residual streams and normalization, goes back to the original Transformer (Vaswani et al. 2017) and is the subject of Chapter 6.

Here the two running examples already diverge, and the divergence is the book in miniature. Llama 3's 405B model is a dense Transformer: every parameter participates in every token (Grattafiori and others 2024). DeepSeek-V3 is a mixture of experts, 671B total parameters but only 37B activated per token, paired with multi-head latent attention to shrink the inference-time cache (DeepSeek-AI 2024). That choice, dense versus sparse, is Chapter 7. Running either training job across thousands of accelerators without the run diverging or the hardware idling is Chapter 8, where DeepSeek-V3's FP8 mixed-precision recipe and its 2.788 million H800 GPU-hours of total training live (DeepSeek-AI 2024).

Station three: adaptation teaches the model to want the task

A base model predicts plausible continuations of text. It does not yet want to fix your bug. Turning a next-token predictor into something that follows instructions, refuses harmful requests, and reasons before answering is adaptation, the second great expense after pre-training. Supervised fine-tuning on demonstrations is [Chapter 9](#). Learning from human or automated preference signals, by reward modeling and policy optimization or by the direct-preference family that skips the explicit reward model, is [Chapter 10](#) and [Chapter 11](#). Teaching a model to spend tokens thinking before it commits to an answer, the capability that separates a 2025 reasoning model from a 2023 chat model, is [Chapter 14](#).

Station four: serving turns weights into a service

Now the model exists and the user has typed the request. Serving it is its own engineering discipline, because a trained model is a pile of weights, not a service. The core tension, that generation is memory-bound and the key-value cache grows with every token, is [Chapter 16](#). Batching many users and paging that cache is [Chapter 17](#). Decoding more than one token per forward pass is [Chapter 18](#). Making each weight and each cache entry cheaper through lower precision is [Chapter 19](#).

Station five: the agent loop closes

Finally the loop closes. The assistant does not answer the bug report from memory alone. It reads files, runs the test suite, sees the failure, edits, and re-runs, one observation-and-action step at a time. That control structure is [Chapter 21](#), the runtime that drives it is [Chapter 23](#), and the retrieval that pulls the right files into the context window is [Chapter 25](#). Deciding what to keep in that finite window, and what to summarize or drop, is [Chapter 26](#).

The slice, drawn as one pass through the stack, is [Figure 1.1](#).

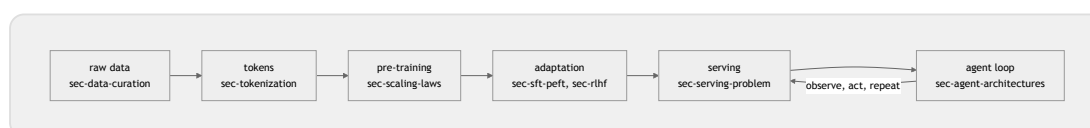


Figure 1.1. One request traced through the stack, from a web-scraped corpus to an agent loop that observes and acts. Each box names the layer and the chapter that owns it.

What the journey hides: three nested loops

The stations above are arranged in space, one after another, but the stack also runs in time, and the timing hides a structure worth naming once because it recurs in every part of the book. The stack is built from three nested loops, each a control structure that consumes compute to reduce a different kind of error.

The **training loop** runs over a corpus, adjusting weights by gradient descent to lower a loss. It is paid once, runs for weeks, and produces an artifact. The **inference loop** runs over a prompt, generating one token at a time from frozen weights. It is paid on every request, forever, which is why a fraction of a cent per token decides the economics of a product. The **agentic loop** runs over an environment, alternating model calls with tool calls until a task is done. It wraps many inference loops inside one unit of useful work, and it is where a model stops being a text predictor and starts being infrastructure.

The containment is the point, and Figure 1.2 draws it. The training loop runs once and freezes an artifact. The inference loop reads that frozen artifact and is paid on every request. The agentic loop wraps many inference loops inside one unit of useful work. Reading the payment cadence off the figure, once, per request, per task, is what makes the constraint arrow below legible.

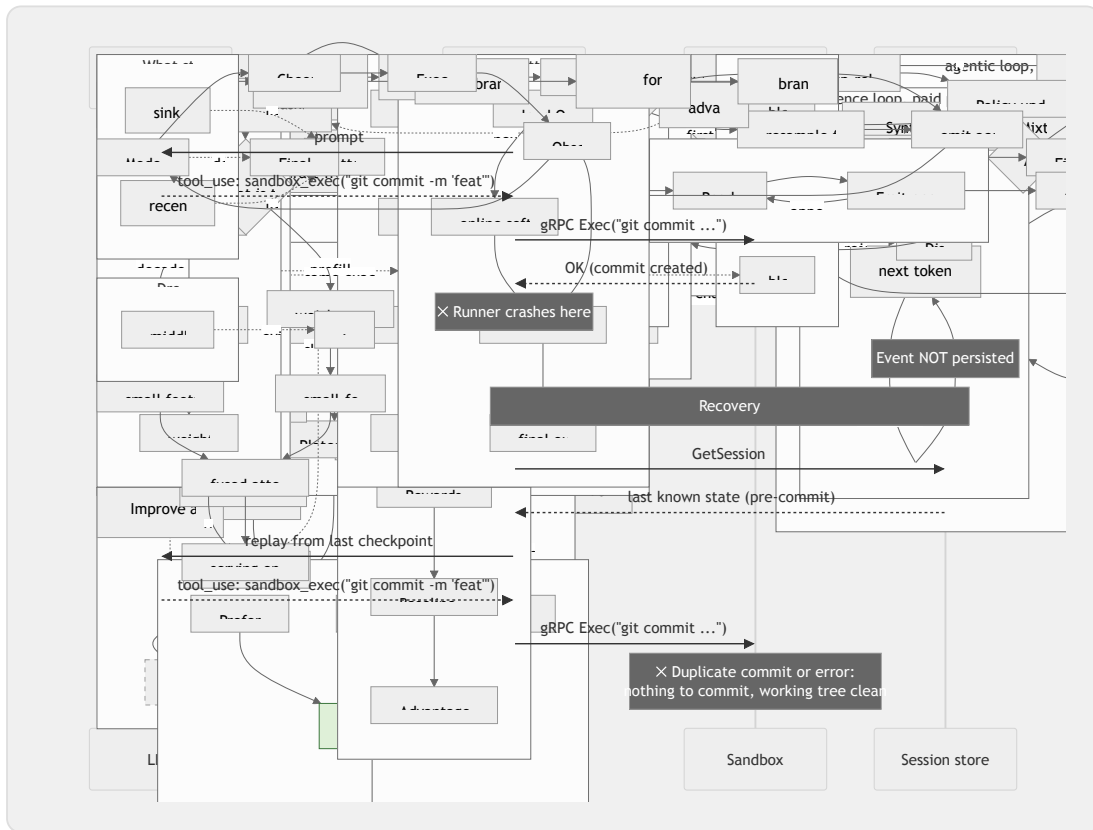


Figure 1.2. The three nested loops. The training loop runs once and freezes weights. Each inference loop reads those frozen weights and is paid per request. The agentic loop wraps many inference loops per task. Cost is paid at the cadence each loop runs.

These loops are not independent, and the most useful insight in this book is where one reaches up to dictate another's design.

CONSTRAINT ARROW

The clearest example appears at the very first layer. Pre-training economics seem to say: for a fixed training budget, pick the model size that minimizes loss (Hoffmann et al. 2022). But a model that will serve billions of tokens over its lifetime is better off smaller and over-trained, because the inference loop is paid forever while the training loop is paid once. The serving cost in Chapter 16 reaches up the stack and sets a decision that looks like pure pre-training, the subject of Chapter 3. DeepSeek-V3's mixture-of-experts design is the same arrow drawn in hardware: activate 37B of 671B parameters per token so the inference loop stays cheap, and pay for it in training and serving complexity instead (DeepSeek-AI 2024). Reading the stack in order is what makes these arrows visible.

Try this to make the constraint arrow concrete: charge each design its training FLOPs once plus its inference FLOPs per served token, then watch which one is cheaper as the tokens served grow.


```

import numpy as np
import matplotlib.pyplot as plt
# Lifetime FLOPs for two designs that reach comparable capability.
# Training ~ 6*N*D_train (paid once); inference ~ 2*N_active per served
designs = {
    "Llama 3 405B dense": dict(N=405e9, N_act=405e9, D=15.6e12),
    "DeepSeek-V3 MoE 37B": dict(N=671e9, N_act=37e9, D=14.8e12),
}
served = np.logspace(11, 16, 200)
for name, d in designs.items():
    total = 6*d["N"]*d["D"] + 2*d["N_act"]*served
    plt.plot(served, total, label=name)
a, b = designs.values()
cross = 6*(b["N"]*b["D"] - a["N"]*a["D"]) / (2*(a["N_act"] - b["N_act"]))
plt.axvline(cross, ls="--", color="gray")
print(f"crossover at ~{cross:.2e} served tokens")
plt.xscale("log"); plt.yscale("log")
plt.xlabel("tokens served over lifetime"); plt.ylabel("total FLOPs")
plt.legend(); plt.tight_layout(); plt.show()

```

Judging any station: capability, efficiency, trust

The three loops describe how the stack runs. A second motif describes how to judge any decision inside it. Every technique in this book buys some mix of three things, and usually trades one against another.

Capability is whether the system can do the task at all: can the model find the bug, follow the multi-step plan, reason through the fix. **Efficiency** is what the capability costs in compute, memory, latency, and dollars. **Trust** is whether you can deploy the result and rely on it: whether it is correct, safe, honest about uncertainty, and measurable. DeepSeek-V3 and Llama 3 are both bids to push capability up while holding efficiency down, and a mixture-of-experts model is a pure efficiency play, more capability per activated parameter (DeepSeek-AI 2024). Trust is the layer the hype cycle forgets and the operator cannot. Knowing whether the agent actually fixed the bug, rather than producing a plausible patch that breaks a different test, is measurement: [Chapter 27](#) for what a static score can and cannot tell you, [Chapter 29](#) for grading a multi-step trajectory, and [Chapter 33](#) for keeping a capable system inside its bounds. The economics that tie capability and efficiency to a price a user will pay are [Chapter 37](#).

[Figure 1.3](#) places the lens over the two running models. Both bid for capability, but a mixture-of-experts model spends fewer activated FLOPs to reach it, pulling toward the efficiency vertex, while trust sits at the bottom for both until measurement has graded the patch each one produced.

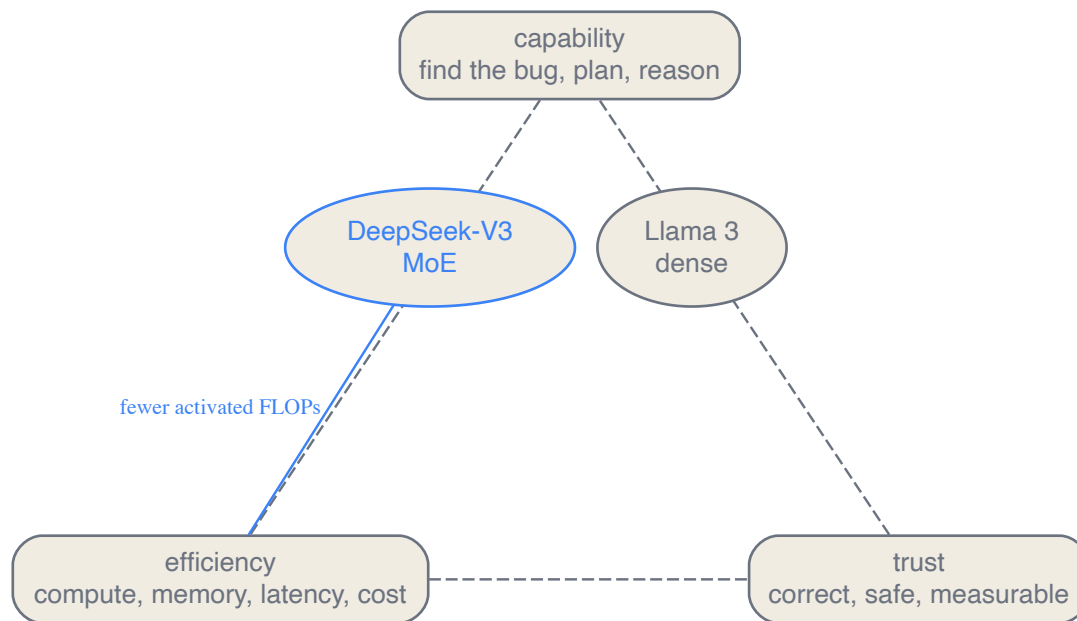


Figure 1.3. The capability, efficiency, trust lens as a trade-off space, with the two running models placed on it. A dense model and a sparse model can reach the same capability. The sparse one does it for fewer activated FLOPs, leaning toward efficiency. Neither earns trust until something measures its output.

Each chapter in this book closes by asking what its technique did to these three. Hold the lens up to the running task: a dense model and a sparse model can reach the same capability, the sparse one does it for fewer activated FLOPs, and neither is trustworthy until something has measured the patch it produced.

How to read from here

The next chapter, [Chapter 2](#), lays out the whole territory this slice cut through, naming the parts and how they connect, so the forward references above resolve into a map. After that the book follows the slice in order: data and architecture, then training, adaptation, and reasoning, then serving, then agents, then the evaluation and infrastructure and governance that hold it all up. A reader can follow that order straight through, or jump to the chapter that owns whichever layer is on fire today and use the cross-references to climb back out. This chapter exists so that either way, the layer in front of you is never the whole story, only one pass through a stack you have already seen end to end.

Further reading

- DeepSeek-AI, "DeepSeek-V3 Technical Report," 2024. [arXiv:2412.19437](#)
- Grattafiori et al., "The Llama 3 Herd of Models," 2024. [arXiv:2407.21783](#)
- Kaplan et al., "Scaling Laws for Neural Language Models," 2020. [arXiv:2001.08361](#)
- Hoffmann et al., "Training Compute-Optimal Large Language Models," 2022. [arXiv:2203.15556](#)
- Vaswani et al., "Attention Is All You Need," 2017. [arXiv:1706.03762](#)

2. A Field Map and How to Read This Book

A book this wide is easy to get lost in. This chapter is the map and the compass you carry through the rest of it. By the end of it a reader can place any topic in the book on a single picture, say which parts of that picture are settled and which are still argued over, see where this book sits next to the comprehensive titles already on the shelf, and read everything that follows design-first: asking of every component why it has the shape it has, not merely what it does.

It is organized around four questions a newcomer to the field actually asks. Where does each piece go? Which parts can I trust as fact and which are live argument? What does this book give me that others do not? And how do I read it without drowning?

One map, nine parts

Chapter 1 walked one task from raw data to a deployed agent in a single pass. This chapter zooms out from that vertical slice to the whole field and shows how the nine substantive parts, Part I through Part IX, interlock. The cleanest way to hold them in mind is as a stack: the lifecycle of a capability, with compute and data at the bottom, a governed behavior at the top, and at every layer a set of choices that the layer below constrains and the layer above inherits. [Figure 2.1](#) draws that stack.

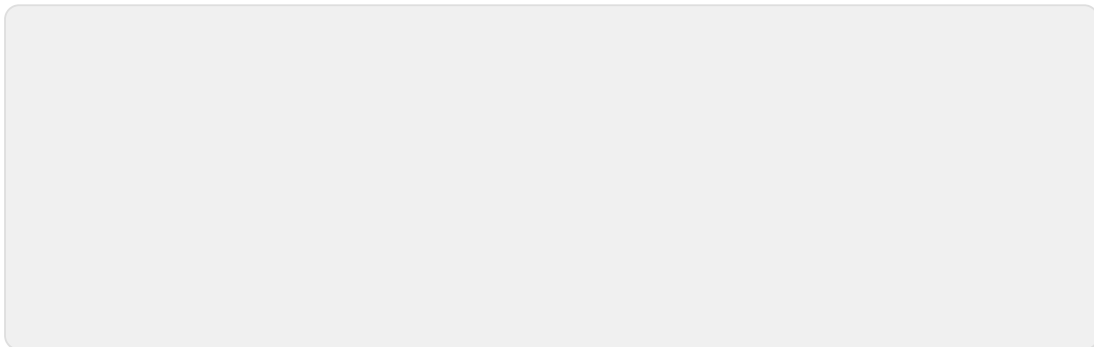


Figure 2.1. The nine parts read as a stack. Solid arrows point from a layer to the layer built on it; dashed arrows are cross-cutting forces (evaluation feedback, safety constraints, economic pricing) that reach across layers rather than feed one into the next.

Read the solid arrows first; they are the build order. Part VII, the accelerators, the networks, and the data infrastructure, sits at the bottom: it is drawn below Part I because a cluster's shape is the first constraint on what can be trained at all ([Chapter 30](#), [Chapter 31](#)). Part I turns that compute into a base model through scaling laws, data curation, tokenization, architecture, and training at scale ([Chapter 3](#), [Chapter 4](#), [Chapter 5](#), [Chapter 6](#), [Chapter 7](#), [Chapter 8](#)). Part II adapts the base model to follow instructions and to human preference ([Chapter 9](#), [Chapter 10](#), [Chapter 11](#), [Chapter 12](#)). Part III teaches it to spend test-time compute on reasoning ([Chapter 13](#), [Chapter 14](#), [Chapter 15](#)). Part IV is the serving problem: turning a trained model into tokens a user can afford to wait for ([Chapter 16](#), [Chapter 17](#), [Chapter 18](#), [Chapter 19](#), [Chapter 20](#)). Part V composes served models into agents with memory, tools, retrieval, and context ([Chapter 21](#), [Chapter 22](#), [Chapter 23](#), [Chapter 24](#), [Chapter 25](#), [Chapter 26](#)). Part VI measures whether any of it works ([Chapter 27](#), [Chapter 28](#), [Chapter 29](#)). Part VIII reaches sideways into every layer to ask whether the behavior can be understood, overseen, and secured ([Chapter 32](#), [Chapter 33](#), [Chapter 34](#)). Part IX prices the whole thing and surveys who builds what ([Chapter 35](#), [Chapter 36](#), [Chapter 37](#)).

The dashed arrows are the reason to read in order, and they are easy to miss. They do not feed one layer into the next; they reach across the stack. Evaluation feeds back into adaptation: a benchmark or a judge that moves changes what the next round of post-training optimizes for. Safety constrains adaptation and orchestration rather than sitting at the end as a coat of paint. Economics prices pretraining and serving, and that price reaches up the stack to change design choices that look, locally, like pure modeling decisions. Hold the dashed arrows in mind as you read; the chapter on reading instructions below shows the sharpest one in full.

Solid ground and swamp

A map is only useful if it marks the solid ground separately from the swamp, and a reader's first defense against the noise of this field is knowing which is which.

Some of this book is settled engineering. The transformer block and its attention mechanism (Vaswani et al. 2017), the autoregressive next-token objective, attention with a key-value cache, mixed-precision training, and the basic fact that loss falls as a smooth power law in compute (Kaplan et al. 2020) are no longer in dispute; they are the shared substrate every lab builds on. A reader can learn them as one learns the memory model of a language: as the rules of the game, fixed for the duration of play.

Much of the frontier is not settled, and the book marks those places rather than papering over them. The compute-optimal ratio of tokens to parameters is a budget-and-deployment-dependent choice, not a constant (Hoffmann et al. 2022) (Chapter 3). Whether reinforcement learning on verifiable rewards teaches genuinely new reasoning or only sharpens what pretraining already put there is argued in Chapter 14, where work like DeepSeek-R1 makes the case that pure reinforcement learning can incentivize reasoning behavior (Guo et al. 2025). Whether sparse autoencoders are the right primitive for reading a model's internal features, or a convenient but lossy lens, is open in Chapter 32. Whether a benchmark number or the harness that runs it is the thing actually being measured runs through Chapter 27 and Chapter 29. The signature of the book is that these live debates get a box, with named positions, instead of a confident sentence that hides the disagreement. The interactive figure below draws that smooth power law: loss falling as a straight line on log-log axes as the compute budget grows, with the exponent set by the slider.

Loss falls as a smooth power law in training compute; drag the slider to change the scaling exponent and watch the straight-line decay on log-log axes.

WHAT'S CONTESTED

The deepest open questions cluster in the upper and outer layers of the map, and they are exactly the white space this book is written to occupy. Three are worth naming now. First, interpretability has no agreed primitive: the superposition hypothesis gave the field a way to think about why a model packs more features than it has dimensions (Elhage et al. 2022), and sparse autoencoders gave a tractable handle on it (Bricken et al. 2023), but whether the features they recover are the model's real computational units or an artifact of the method is unresolved, and Chapter 32 treats it as open. Second, the source of reasoning gains is contested: Chapter 14 weighs the claim that reinforcement learning elicits latent capability against the claim that it installs new capability (Guo et al. 2025), because the two imply different research bets. Third, agent architecture has no settled form: whether long-horizon competence comes from better memory systems (Chapter 22), a stronger single-agent harness (Chapter 23), or genuine multi-agent decomposition (Chapter 24) is an empirical question the field has not closed. Treat any confident answer in these three areas as a position, not a fact.

Three books, and the gap between them

A reader deciding where to spend attention deserves to know what this book is not. Three comprehensive books already cover much of this territory well, and this book is written to complement them rather than to repeat them. Naming them precisely is the fastest way to say what is new here.

Chip Huyen's *AI Engineering* (Huyen 2025) is the application layer done thoroughly: how to build products on top of foundation models you call rather than train, covering evaluation, prompt and context design, retrieval, and deployment from the perspective of an engineer adapting models to a use case. It treats the model largely as a given and asks how to build well around it.

Sebastian Raschka's *Build a Large Language Model (From Scratch)* (Raschka 2024) is the laptop-scale build: it implements a GPT-style model end to end in PyTorch, from tokenization through attention to pretraining and fine-tuning, so a reader understands the machine by constructing a small one. Its gift is mechanical clarity at a scale one person can run.

Suhas Pai's *Designing Large Language Model Applications* (Pai 2025) is the broadest of the three, a holistic tour from the ingredients of a language model through fine-tuning, retrieval, and agents. It covers a wide surface, and by design it is lighter on the parts that only appear at frontier scale: at-scale distributed training, the internals of a production serving stack, and mechanistic interpretability.

The white space those three leave open is the middle and lower stack at frontier scale. This book runs whole-stack, from the cluster to the agent, and spends its depth where the others are thin: distributed training across thousands of accelerators (Chapter 8, Chapter 30), the internals of serving, what actually happens inside the loop that turns a model into affordable tokens (Chapter 16, Chapter 17, Chapter 18), and interpretability as a first-class engineering concern rather than a research footnote (Chapter 32). The aim is not to argue the other books are wrong anywhere; it is to read the layers they pass over quickly, slowly and design-first.

Reading instructions

The map tells you where things are. This last section tells you how each chapter is built, so the reading goes faster.

Every substantial chapter moves through the same arc. A chapter opens with the **Problem**: the essential constraint the component answers, in terms of latency, cost, memory, scale, safety, or correctness. It then gives the **Design**: the core idea that answers the problem and the principle underneath it. **Evolution** traces how the design got here, which earlier approaches came first and why they were superseded, through primary sources. **Trade-offs** names what is sacrificed for what, and where the known gaps are. **Implementation** appears only when a minimal sketch, a config, or an operational detail clarifies the design, never to reproduce a file. Each chapter closes with **Further reading**, first-hand sources first. The arc carries an engineering-to-theory motion: the practice is established first, then the foundations that explain it.

Two callouts carry the book's intellectual signature, and learning to read them is worth the small effort. A *What's contested* box marks where the field genuinely disagrees, with named positions, as in the box above. A *Constraint arrow* box marks where a lower layer dictates an upper layer's choice. These arrows are the payoff of reading the stack in order rather than dipping into chapters at random, and the next box shows the canonical one.

CONSTRAINT ARROW

The clearest single instance of a constraint arrow runs from serving up to pretraining. Chapter 3 decides how large a model to train and on how many tokens, and read locally that looks like pure training economics: minimize loss for a fixed compute budget, the

compute-optimal recipe (Hoffmann et al. 2022). But if a model will serve an enormous volume of tokens over its life, the right move is to deliberately over-train a smaller model far past the compute-optimal point, because training is paid once and inference is paid forever. The decision is set by the serving cost in [Chapter 16](#) reaching up the stack, not by the training budget alone. When you meet a choice that looks unmotivated inside its own chapter, look one layer down: the arrow that explains it usually starts there.

Try changing how many tokens each model serves over its life and watch the crossover move: a smaller, deliberately over-trained model costs more to train but less per token, so past some serving volume it wins on total cost.

```
import numpy as np
import matplotlib.pyplot as plt

# Two models assumed to reach the same quality, compared on lifetime
# Training FLOPs ~ 6*N*D, inference FLOPs ~ 2*N per generated token.
big_N, big_D = 70e9, 1.4e12      # compute-optimal larger model
small_N, small_D = 13e9, 12e12   # smaller model over-trained on mor

served = np.logspace(11, 15, 200) # tokens served over the model's
big_cost = 6 * big_N * big_D + 2 * big_N * served
small_cost = 6 * small_N * small_D + 2 * small_N * served
cross = served[np.argmin(np.abs(big_cost - small_cost))]

plt.loglog(served, big_cost, label="compute-optimal 70B")
plt.loglog(served, small_cost, label="over-trained 13B")
plt.axvline(cross, ls="--", color="gray")
plt.xlabel("tokens served over life"); plt.ylabel("total FLOPs"); pl
print(f"crossover near {cross:.2e} served tokens")
plt.show()
```

Two motifs recur without needing a callout, and noticing them ties distant chapters together. The **three loops**, training, inference, and agentic, are one control structure seen at three timescales: a weeks-long loop that turns compute into a model, a millisecond loop that turns a prompt into tokens, and a multi-step loop that turns a goal into actions. [Figure 2.2](#) draws all three side by side so the shared shape is visible: each is an input that feeds a step that produces an output that returns to the input, and only the payload and the timescale differ.

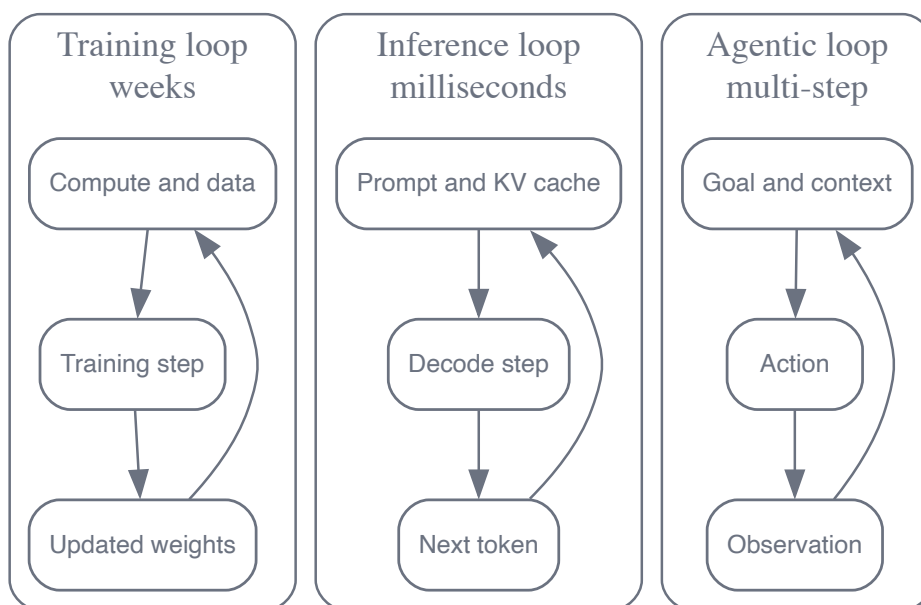


Figure 2.2. The three loops are the same control structure at three timescales. Each cycles an input through a step to an output that returns to the input, and only the payload and timescale change.

The **capability, efficiency, trust** lens closes many chapters: a technique buys capability, or efficiency, or trust, and usually trades one against another, and naming which it buys keeps the comparison honest.

A reader does not have to read strictly front to back. The parts are written to interlock, so following the solid arrows of the map, pretraining before adaptation, serving before orchestration, gives the constraint arrows their force and is the recommended path. Reading by interest is fine too; the cross-references will point back to whatever lower layer a chapter depends on. Either way, keep the four questions of this chapter in hand: where it goes on the map, whether it is solid or contested, what this book adds, and which way the arrows run.

Further reading

- Huyen, Chip. *AI Engineering: Building Applications with Foundation Models*. O'Reilly Media, 2025. oreil.ly
- Raschka, Sebastian. *Build a Large Language Model (From Scratch)*. Manning, 2024. manning.com
- Pai, Suhas. *Designing Large Language Model Applications: A Holistic Approach*. O'Reilly Media, 2025. oreil.ly

3. Scaling Laws and Compute Allocation

Pre-training is the bulk compute spend: take a fixed architecture and a fixed data mixture and turn FLOPs into a base model by minimizing a next-token loss. This chapter owns the recipe of that spend, the objective, the optimizer, the schedule, the batch size, and above all the scaling laws that decide how big a model to train on how many tokens before you pay for the run. By the end a reader can explain why a frontier model is the size it is, trained on the number of tokens it is, and why that answer changed between 2020 and today.

The irreversible bet

A pre-training run commits a cluster for weeks and a budget you cannot refund. Before it starts you must choose a parameter count, a token budget, a learning-rate schedule, and a batch size, and you must choose them from small experiments because you cannot afford to try them at full scale. The whole chapter is one question asked under that constraint: how to spend a fixed compute budget so the final loss is as low as it can be, and how to know that before committing.

The reason the question is answerable at all is that the spend has a predictable shape. The autoregressive objective is what makes it both possible and measurable. Next-token prediction is maximum likelihood over a token stream, cross-entropy on the shifted sequence under a causal mask, so every position is a training example and the corpus is its own supervision. The loss this produces, in nats or bits per token, is the quantity scaling laws predict; perplexity is just its exponential. That single scalar, the loss, is what you are buying with the run, and it is what the rest of the chapter learns to forecast and then to minimize per dollar.

A law you can extrapolate

The decision that dominates everything else is sizing, and sizing is tractable because the loss is smooth. A scaling law fit on a ladder of small runs predicts the loss of a large run as a power law in parameters, data, and compute (Kaplan et al. 2020):

$$L(N, D) = E + \frac{A}{N^\alpha} + \frac{B}{D^\beta}$$

where N is parameters and D is training tokens. You fit the constants on runs you can afford, then read off the compute-optimal N and D for the budget you intend to spend. The law extrapolates: that is its entire value. It lets a few cheap runs stand in for a run you will only ever do once. Because the loss is a power law in compute, those cheap runs fall on a straight line in log-log coordinates, and the line predicts the loss of the expensive run far to the right, as [Figure 3.1](#) sketches.

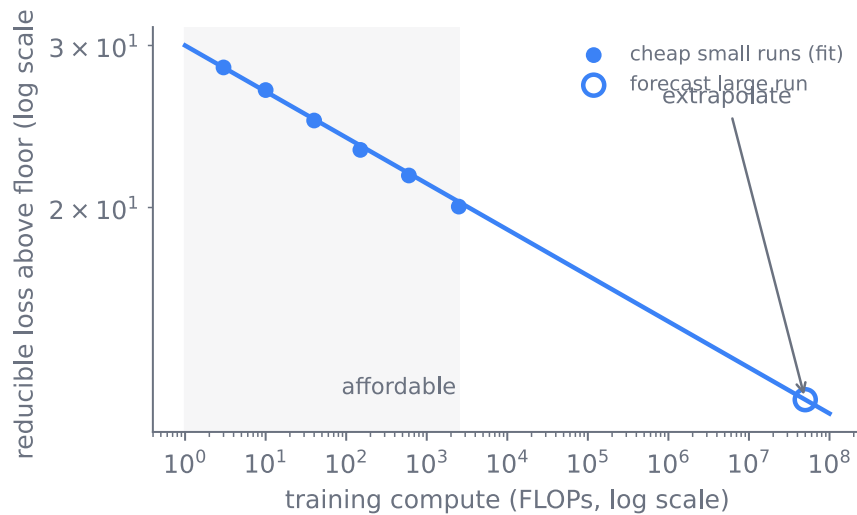


Figure 3.1. Schematic illustration of power-law extrapolation: a ladder of cheap small runs fits a straight line in log-log coordinates, which forecasts the loss of one large run you can only afford once. Idealized data, after Kaplan et al. (2020) and Hoffmann et al. (2022).

The same logic extends to the hyperparameters that the loss formula does not name. Maximal-update parametrization (muP) makes them transfer: parametrize the widths so the optimal learning rate and initialization are stable across model size, tune them on a small proxy, and carry them to full scale instead of sweeping at full cost (Yang et al. 2022). Sizing and hyperparameters both become things you decide on small runs and trust at large ones.

Kaplan, then Chinchilla

The sizing rule itself has a history, and getting it wrong is the most expensive mistake in this chapter. Kaplan et al. (2020) established that loss follows smooth power laws and, from their fits, recommended spending most of a larger budget on parameters (Kaplan et al. 2020). Hoffmann et al. (2022), the Chinchilla paper, corrected this: parameters and tokens should scale together, roughly twenty training tokens per parameter at compute-optimal, and the Kaplan-era recipe had been systematically under-training on data (Hoffmann et al. 2022). A Chinchilla-optimal model is smaller and reads more than the prior generation expected.

The same compute budget, fixed by $C \approx 6ND$, splits very differently under each of these recipes. Figure 3.2 contrasts the three allocations of one budget: Kaplan spends it mostly on parameters, Chinchilla balances parameters against tokens at roughly twenty tokens per parameter, and inference-aware over-training deliberately shrinks the model and pours the rest into tokens so each served token is cheaper.

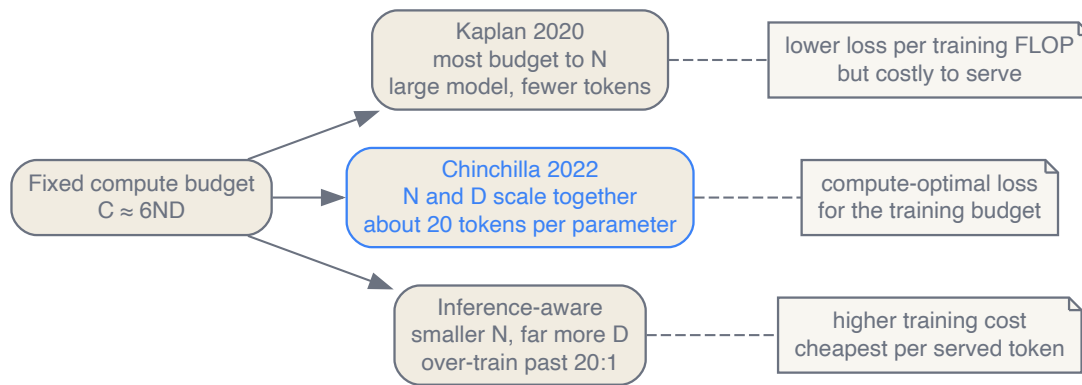


Figure 3.2. How a fixed training-compute budget C is approximately $6ND$ splits into parameters N and tokens D under three sizing rules. After Kaplan et al. (2020), Hoffmann et al. (2022), and inference-aware scaling.

The recipe did not stop at Chinchilla. When unique tokens run out, data-constrained scaling takes over: repeated data has decaying value, and past a few epochs adding parameters beats adding repetitions (Muennighoff et al. 2023). The frontier of the recipe is still moving. Second-order and preconditioned optimizers, Shampoo and the newer Muon, claim better loss per FLOP by conditioning the update with curvature or orthogonalization (Jordan et al. 2024; Liu et al. 2025). These are live options, not settled defaults.

Test it yourself

The law is concrete enough to run. The cell below walks a fixed compute budget along the parameters-versus-tokens trade and finds where the loss bottoms out. Edit the exponents and rerun (Run, or Ctrl/Cmd+Enter) to watch the compute-optimal split move.

```
import numpy as np, matplotlib.pyplot as plt

# Toy scaling law  $L(N, D) = E + A/N^a + B/D^b$ . Change the exponents.
E, A, B, a, b = 1.7, 400.0, 410.0, 0.34, 0.28
loss = lambda N, D: E + A / N**a + B / D**b

C = 1e19 # fixed compute budget, roughly  $6 * N$ 
N = np.logspace(8, 11, 300) # candidate parameter counts
D = C / (6 * N) # tokens the budget then allows

plt.figure(figsize=(5, 3))
plt.plot(N, loss(N, D))
plt.xscale("log"); plt.xlabel("parameters N"); plt.ylabel("loss")
plt.tight_layout(); plt.show()

best = N[np.argmin(loss(N, D))]
print(f"compute-optimal: N ~ {best:.2e} params, D ~ {C/(6*best):.2e}")
```

The compute-optimal token-to-parameter ratio is not settled, and Chinchilla is not the last word. Inference-aware scaling argues that if a model will serve a large volume of tokens, the right move is to deliberately over-train a smaller model far past Chinchilla-optimal, because training is paid once and inference is paid forever. Frontier models now train well past the twenty-to-one ratio for exactly this reason. The reconciliation between Kaplan and Chinchilla fits, and how far over-training pays, are active questions. Treat any single ratio as a budget-and-deployment-dependent choice, not a constant.

CONSTRAINT ARROW

This is the first place a lower layer reaches up the stack. The serving cost in Chapter 16, not the training budget, is what justifies over-training a smaller model. A decision that looks like pure pre-training economics is actually set by how many tokens the model will decode over its lifetime.

The rest of the recipe

The scaling law fixes N and D . It does not fix how the optimizer walks to that loss, and those choices are a set of balances, each with a knee.

The optimizer is the first of them. AdamW is the default, with decoupled weight decay and two moment estimates whose state is the dominant memory term, which is why Chapter 8 shards it (Loshchilov and Hutter 2019). The learning-rate schedule that drives it has three parts: a linear warmup while the moment estimates are still cold, a stable high phase, and a decay. Cosine decay to a small floor is the well-understood default, but it locks the token budget at run start. Warmup-stable-decay holds a constant rate on a plateau and decays only when you choose to stop, which makes one plateau checkpoint spawn many decays and makes scaling experiments composable (Hu et al. 2024). It trades a small loss penalty for the ability to keep training, fork decays, and run clean scaling ladders. Choose cosine for a single known-budget run, the plateau schedule when the budget is uncertain.

Batch size is the second balance. A larger batch buys data parallelism and shorter wall-clock, but past the critical batch size you burn compute for no loss gain, and too small a batch leaves the accelerators idle and pays communication overhead per useful token (McCandlish et al. 2018). Figure 3.3 sketches that knee: progress per step climbs almost linearly while the batch is small, then flattens once it passes the critical size. The critical batch size grows during training, so the optimal batch is a schedule, not a constant.

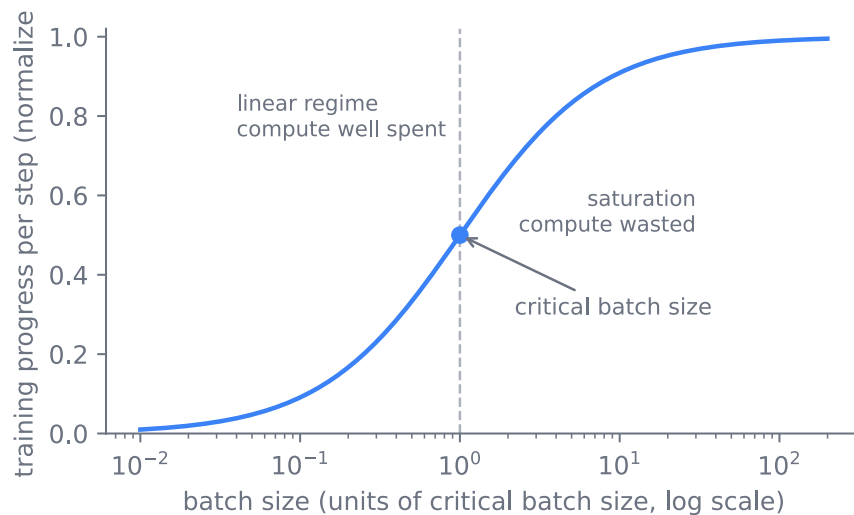


Figure 3.3. Schematic illustration of the critical batch size: training progress per optimizer step rises almost linearly with batch size, then saturates, so compute past the knee buys little extra progress. Idealized data, after McCandlish et al. (2018).

And muP closes the loop back to sizing. Tuning the learning rate and initialization on a small proxy and transferring them at full width is what keeps the largest run from leaving loss on the table for a reason you would never see otherwise (Yang et al. 2022).

Keeping the run alive

Most of this chapter is a plan. A smaller part is an operational kit for keeping a multi-week run alive, because the plan is worthless if the run dies in week three. The stability tricks are z-loss, an auxiliary penalty on the log-partition that stops output logits from drifting (Chowdhery et al. 2022); QK-norm, normalizing query and key before the dot product to cap attention-logit growth; depth-scaled initialization; and global-norm gradient clipping as the blunt safety net. Loss-spike recovery is a procedure, not a hyperparameter: detect the spike from grad-norm and loss telemetry, then skip and resume from a recent checkpoint, lower the rate, and reorder the data batches that triggered it (DeepSeek-AI 2024). Figure 3.4 traces that loop.

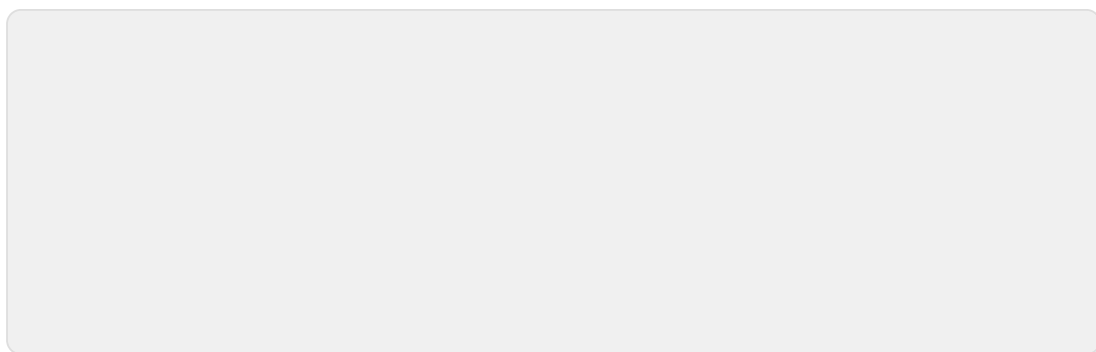


Figure 3.4. Loss-spike recovery as a control loop over a multi-week run, following DeepSeek-V3 (2024) practice.

The failure modes are worth naming because each bites at a different time. Divergence to NaN sends the loss to infinity early, usually from too high a rate, too short a warmup, or a precision interaction from Chapter 8, and is cheapest to prevent with warmup and clipping. Loss spikes appear mid-run and waste days of compute if you have no recovery plan. The wrong size-

versus-tokens choice is the most expensive of all because it is the whole run, and an untuned learning rate at full width leaves loss on the table that you never see without muP to give you a baseline.

Further reading

- Kaplan et al., "Scaling Laws for Neural Language Models," 2020. [arXiv:2001.08361](#)
- Hoffmann et al., "Training Compute-Optimal Large Language Models" (Chinchilla), 2022. [arXiv:2203.15556](#)
- Yang et al., "Tensor Programs V: Tuning Large Neural Networks via Zero-Shot Hyperparameter Transfer" (muP), 2022. [arXiv:2203.03466](#)
- Loshchilov & Hutter, "Decoupled Weight Decay Regularization" (AdamW), 2019. [arXiv:1711.05101](#)
- McCandlish et al., "An Empirical Model of Large-Batch Training" (critical batch size), 2018. [arXiv:1812.06162](#)
- Muennighoff et al., "Scaling Data-Constrained Language Models," 2023. [arXiv:2305.16264](#)
- Hu et al., "MiniCPM" (warmup-stable-decay schedule), 2024. [arXiv:2404.06395](#)
- Chowdhery et al., "PaLM" (z-loss, stability at scale), 2022. [arXiv:2204.02311](#)
- Jordan, "Muon: An Optimizer for Hidden Layers in Neural Networks," 2024. [kellerjordan.github.io](#); scaled follow-up Liu et al., 2025, [arXiv:2502.16982](#)
- DeepSeek-AI, "DeepSeek-V3 Technical Report" (trillion-token stability and loss-spike practice), 2024. [arXiv:2412.19437](#)

4. Data Curation and Quality

At fixed compute and architecture, the corpus is what separates a strong model from a mediocre one. By the end of this chapter a reader can explain where training data comes from, how raw web bytes become a clean deduplicated corpus, why mixture and curriculum are decided early and expensive to revisit, what synthetic data buys and risks, and why a contamination leak invalidates every downstream number.

A corpus is a system, not a script

A pre-training run consumes a fixed compute budget set by Chapter 3. That budget buys a token count, and the corpus that fills it is the single biggest lever on final quality and the one least transferable from open recipes. The difficulty is that the failures here are silent. A duplicated shard, a filter that strips a dialect, a benchmark question that leaked through a synthetic-data step: none of these throw an error, and all of them surface only at evaluation, where they are expensive to diagnose and often impossible to undo without re-running. So the corpus must be built as a system whose every stage is auditable and whose hand-offs to the rest of the stack are explicit contracts.

Two of those hand-offs bind the rest of the book. The **vocabulary** is consumed by the architecture in Chapter 5 (this chapter treats the tokenizer only as a pointer; its design lives there). The **decontamination report** is consumed by evaluation in [Chapter 27](#), which supplies the eval registry this chapter checks against.

A reader may wonder why this is not just a scripting problem. The reason is scale. Everything below runs at petabyte volume as distributed batch or stream jobs (Apache Spark, Ray, or custom MapReduce-style clusters) across large CPU fleets, separate from and ahead of the GPU training cluster. The hard parts are throughput, determinism, and resumability, not the per-document algorithms. A re-run must produce byte-identical shards, or the mixture and decontamination contracts become unverifiable.

The chapter follows a byte from the open web through the narrowing filters that decide whether it survives, then steps back to the three decisions that are locked early, the contract that protects the numbers, and the histories that explain why the field still argues about all of it.

A byte's journey through the narrowing filters

The pipeline is a sequence of narrowing filters, each cheap relative to the training run it feeds, arranged so the cheapest cuts come first.

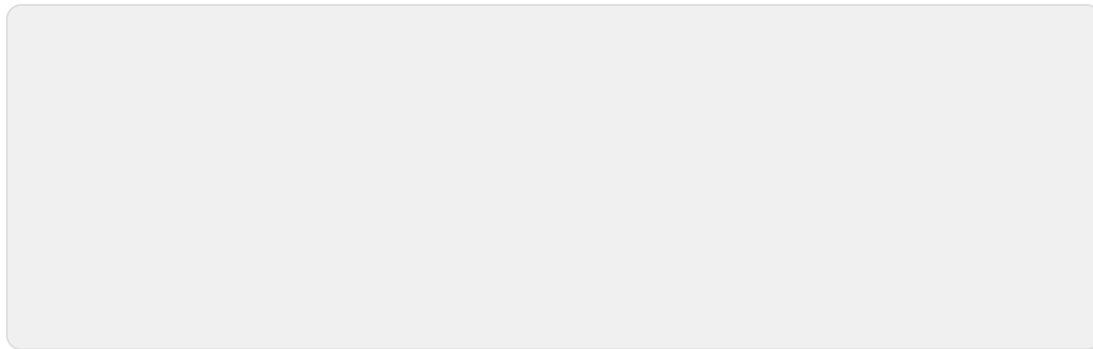


Figure 4.1. The curation pipeline as a sequence of narrowing filters, cheapest cuts first, ending in token-id shards. Each stage is an auditable hand-off.

Where the byte comes from

The bulk substrate is the web. CommonCrawl is the raw input: WARC/WET extraction, boilerplate stripping, language identification, and URL/host-level filtering. CommonCrawl is raw and noisy, and the value is in the pipeline applied to it, not the crawl itself. Around that web core sit curated sources, books, papers, encyclopedic and reference text, source code with license and quality signals, and non-English text at deliberate ratios. Each source has different licensing, quality, and dedup characteristics, and each competes for the same token budget.

Before any filtering, the bytes are made uniform. Cleaning and normalization covers Unicode normalization, repair of text extraction artifacts, PII handling, and toxicity and safety pre-filtering. The goal is to remove garbage without scrubbing legitimate distributional variety: the registers, dialects, and domains that a model needs to see to generalize.

The first cut: deduplication

Deduplication runs three mechanisms at three granularities. Exact hashing removes identical documents. Suffix-array or substring methods remove near-duplicate spans inside otherwise distinct documents. MinHash with locality-sensitive hashing (LSH) does fuzzy document-level dedup at corpus scale: it estimates Jaccard similarity between documents from a small set of hashed shingles, so the cost is sub-quadratic rather than all-pairs. Dedup recovers wasted compute and reduces verbatim memorization. The standing tension is aggressiveness against losing genuinely distinct content.

Dedup is not merely a cost optimization, and the evidence says so. Lee et al. showed that deduplicating training data makes models better, not just cheaper: it cuts memorization and improves held-out loss (Lee et al. 2022). SemDeDup pushed the idea into embedding space, removing semantic near-duplicates that surface-form hashing misses (Abbas et al. 2023). The through-line is that redundancy is not neutral; it actively degrades the model, so the filter that removes it is a quality filter in disguise.

Figure 4.3 shows why MinHash with LSH avoids the all-pairs blow-up: each document collapses to a short signature, LSH bands route similar signatures into shared buckets, and a full Jaccard estimate is computed only for the few documents that already collide in a bucket.

Run this to see how a MinHash signature estimates Jaccard similarity from a handful of hashes, and how the estimate sharpens as `num_perm` grows.

```

import numpy as np

rng = np.random.default_rng(0)
universe = 5000                                # vocabulary of shingles
A = set(rng.choice(universe, 800, replace=False).tolist())
B = set(A) ^ set(rng.choice(universe, 400, replace=False).tolist())
true_j = len(A & B) / len(A | B)                # exact Jaccard

def minhash_estimate(A, B, num_perm):
    seeds = rng.integers(1, 2**31 - 1, size=num_perm)
    a = np.array(sorted(A)); b = np.array(sorted(B))
    hits = 0
    for s in seeds:                              # min of a hashed permutation
        if (a * s % universe).min() == (b * s % universe).min():
            hits += 1
    return hits / num_perm                       # P(min collides) = Jaccard

for num_perm in [16, 64, 256, 1024]:
    est = minhash_estimate(A, B, num_perm)
    print(f"num_perm={num_perm:>5}  est={est:.3f}  true={true_j:.3f}")

```

Figure 4.2 makes the cost argument quantitative. Naive all-pairs comparison grows as the square of the corpus size, which is hopeless at petabyte scale, while LSH keeps the candidate count close to linear by only ever comparing documents that already share a bucket.

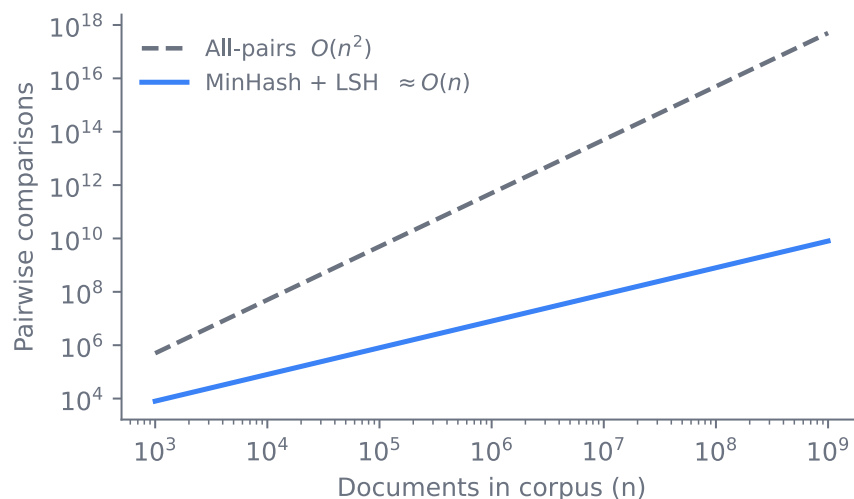


Figure 4.2. Schematic comparison of fuzzy-dedup cost. All-pairs comparison grows as $O(n^2)$ in the corpus size, while MinHash with LSH stays close to linear by only comparing documents that collide in a band-bucket. Idealized counts, after Broder (1997) and Lee et al. (2022).

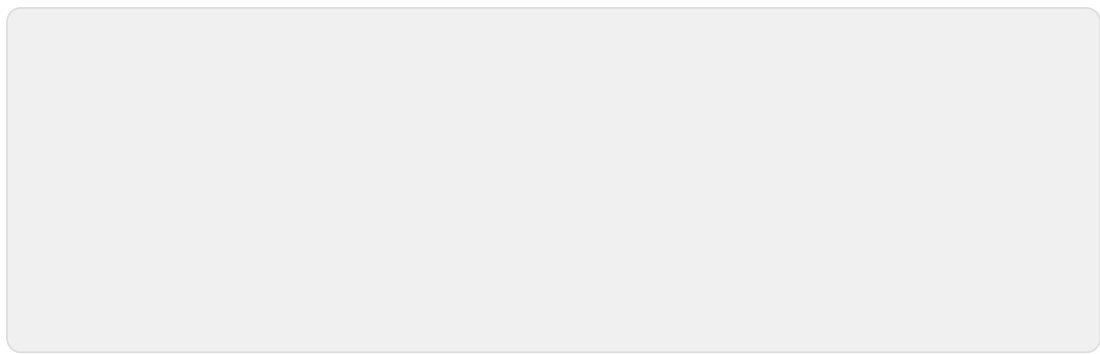


Figure 4.3. MinHash with LSH for corpus-scale fuzzy dedup. Only documents that share an LSH bucket are compared, so the cost is sub-quadratic rather than all-pairs. After Broder (1997) and Lee et al. (2022).

The second cut: quality filtering

What survives dedup still has to clear a quality bar, and two families set it. Heuristic gates are rule-based: length, symbol-to-word ratio, repetition, stop-word presence, and a perplexity threshold from a reference model. They are cheap, auditable, and high-recall for obvious junk, but brittle at the margins. Classifier filters are learned: a model scores each document against a reference of "good" text, for example a classifier trained to distinguish curated text from raw web. Classifiers have higher precision but carry a structural risk, they encode the reference set's notion of quality into the entire corpus. The reference perplexity filter traces directly to CCNet, which paired language identification with a perplexity score from a reference language model (Wenzek et al. 2020).

The decisions locked early

By the time the byte clears quality filtering, the per-document work is done. What remains is global: how to combine sources, how to order them, and whether to manufacture more. These decisions are cheap to state and expensive to change, because each one constrains the others.

Mixture is the ratios across web, code, curated, and multilingual, plus per-source weights. It can be hand-tuned, scaled from small-proxy ablations, or optimized directly. DoReMi frames mixture selection as group distributionally robust optimization: train a small proxy, use it to find weights that minimize worst-case excess loss across domains, then apply those weights to the full run (Xie et al. 2023). The mixture is locked early and expensive to revisit, because changing it means re-deriving the curriculum and often re-running proxy ablations.

Curriculum is the ordering and phasing of data over training: easy-to-hard, domain phasing, or saving a high-quality slice for late. This is the ordering *within* the data plane. It is distinct from the annealing *phase*, which up-weights high-quality data near the end of training and belongs to mid-training, covered alongside [Chapter 8](#).

Synthetic data is model-generated and templated text: rephrasing or augmenting web text, textbook-style generation, and distillation targets. The principle is that a small amount of dense, clean, on-distribution text can be worth far more than its token count in raw web. It is powerful for filling capability gaps, and [Chapter 12](#) treats it as a first-class post-training tool. The risks are distributional collapse, factual drift, and laundering eval content back into training. Because the last risk feeds directly into the contract that follows, synthetic data is also where the pipeline most needs the next stage to be strict.

Decontamination as a contract

Decontamination is N-gram or substring overlap detection against the eval registry, an overlap threshold, a drop-or-flag decision, and a report emitted as the contract to evaluation. This is the stage that earns the chapter its place in the stack. The report, not a shared implementation, is the hand-off.

The threshold is not a free parameter. N-gram overlap detection is itself evadable: rephrased benchmark samples slip past surface-level matching, which is exactly why the threshold and method are a negotiated contract rather than a private default (Yang et al. 2023). A loose threshold leaks benchmarks and inflates scores. A strict one over-removes legitimate near-duplicates and shrinks the corpus. Either way the decision belongs to the boundary between this chapter and [Chapter 27](#), not inside the data team.

CONSTRAINT ARROW

The evaluation layer reaches down and sets a hard constraint on this one. The decontamination threshold is not a data-team preference: it is dictated by the eval registry that [Chapter 27](#) defines and must supply. If evaluation does not hand down which held-out sets to protect, this chapter cannot guarantee the numbers reported later are real. The worst-case failure, a contamination leak through a dedup gap, a synthetic pipeline, or an agent transcript that embedded a benchmark task, inflates every downstream score at once. The decontamination contract exists precisely so a lower layer cannot silently corrupt an upper one.

How the recipe got here: two histories

From assembled corpora to pipeline-as-product

Early corpora were assembled, not engineered. The Pile curated 800GB from twenty-two diverse sources and made the components explicit (Gao et al. 2020). C4, built for T5, showed that a handful of heuristic cleaning rules over CommonCrawl already moved downstream quality (Raffel et al. 2020). The pivot to pipeline-as-product came with CCNet, which combined language identification with a perplexity filter from a reference language model (Wenzek et al. 2020), and then with RefinedWeb, whose claim was sharper: web data alone, filtered and deduplicated hard enough, can match or beat curated corpora (Penedo et al. 2023). FineWeb carried this further with public ablations that fixed compute and architecture and varied only the data, turning corpus construction into a measurable science rather than folklore (Penedo et al. 2024).

The synthetic turn

The synthetic turn arrived through two results. TinyStories showed that a narrow, fully synthetic corpus could teach small models coherent English (Eldan and Li 2023). The phi line, opened by "Textbooks Are All You Need," argued that textbook-quality synthetic and filtered data could reach strong reasoning at a fraction of the usual token count (Gunasekar et al. 2023). Rephrasing the Web (WRAP) made the cheaper version of the bet: rephrase existing web text into cleaner styles rather than generating from scratch, gaining compute and data efficiency without inventing content (Maini et al. 2024). The newest and least-documented source is agent transcripts, tool-use traces and verified multi-step trajectories recycled as training signal, foreshadowed by datasets like ToolBench (Qin et al. 2023). The open questions there are provenance, dedup against the source model's own outputs, and contamination from embedded eval tasks.

WHAT'S CONTESTED

Whether to chase scale or quality is not settled. One camp, traced through RefinedWeb and FineWeb, holds that aggressively filtered and deduplicated web data is sufficient and that curated corpora add little once filtering is good enough. The other, traced through the phi line, holds that synthetic textbook-quality data is worth a large multiple of its token count and that the future of the corpus is generated, not crawled. The disagreement is live because the two positions imply different infrastructure, different cost structures, and different contamination risks, and because synthetic-heavy recipes are the hardest to evaluate cleanly: the same models that generate the data also sit near the benchmarks. Treat the synthetic share as a bet on diversity versus control, not a solved ratio.

Figure 4.4 traces why the choice is structural, not a single tunable ratio: each camp branches into a different stack, cost structure, and contamination risk.

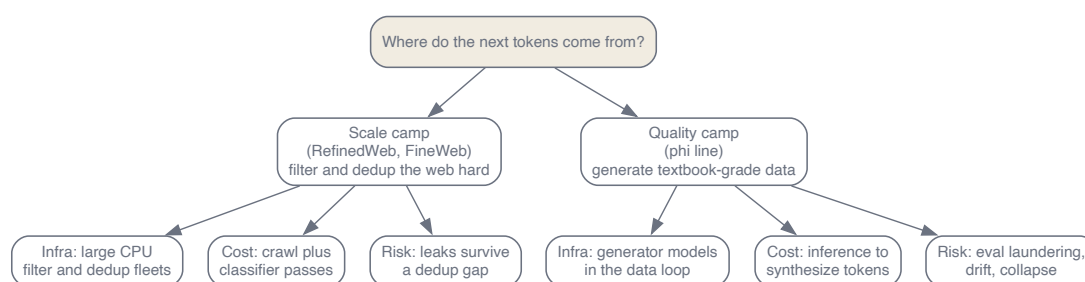


Figure 4.4. Why the scale-versus-quality debate is unsettled: each camp implies a different stack, cost structure, and contamination risk, so the choice is not a single tunable ratio.

What you trade, and what breaks

Every stage above is a dial, and turning it the wrong way fails silently. The dials are worth naming together because they interact.

- **Quantity versus quality.** More tokens help only if they are not junk or duplicates. Past a point, aggressive filtering and dedup beat raw volume, as sketched in Figure 4.5: held-out loss on raw web flattens once added tokens mostly repeat content the model has already seen, while the same compute on deduplicated and filtered tokens keeps falling. The hard part is knowing where that point is for a given compute budget, and that point moves with the budget set in Chapter 3.

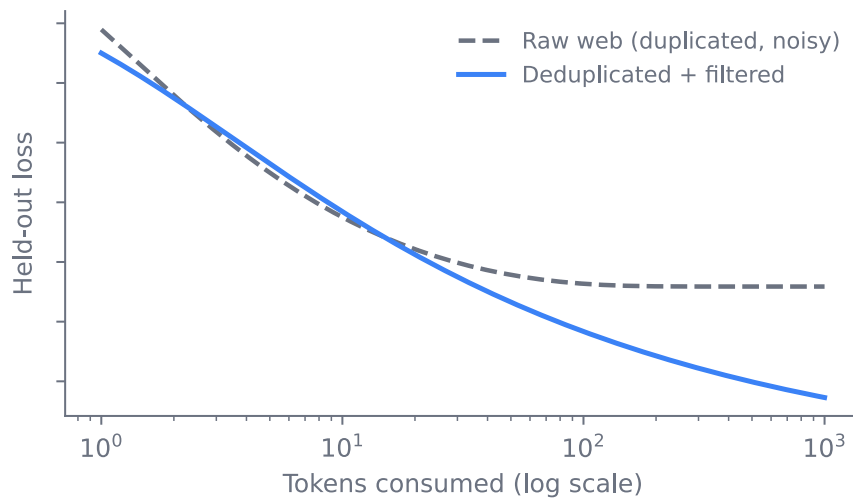


Figure 4.5. Schematic of the quantity-versus-quality trade-off. Held-out loss on raw web flattens early because duplicated and noisy tokens add little new signal, while the same compute spent on deduplicated and filtered tokens keeps descending. Idealized curves, not measured loss, after Lee et al. (2022) and Penedo et al. (2024).

- **Dedup aggressiveness.** Stronger fuzzy dedup removes memorization risk and wasted compute, but can delete legitimately rare-but-distinct content. The threshold is a recall-precision dial with no universally right setting.
- **Heuristic versus classifier filtering.** Heuristics are cheap, auditable, and bias-light but coarse. Classifiers are precise but import the biases of their reference set into the entire corpus, stripping registers, dialects, or domains wholesale.
- **Synthetic share.** Synthetic data fills gaps cheaply but risks collapse, factual drift, and eval laundering. The ratio trades diversity against control.
- **Decontamination strictness.** A loose threshold leaks benchmarks and inflates scores. A strict one over-removes legitimate near-duplicates and shrinks the corpus. The threshold is a contract term, not an internal detail.

The failure modes are worth naming because each bites at a different time. A contamination leak inflates results and is detected late, if at all. A dedup gap wastes compute and raises memorization risk, while over-dedup silently removes rare content. Filter bias amplification strips legitimate variety across the whole corpus. Synthetic collapse or drift narrows the distribution or propagates the generator's errors. A mixture mistake under- or over-trains a capability, and because the mixture is locked early and surfaces only at evaluation, it is the costliest to diagnose and redo.

The determinism discipline

Most of this chapter is a set of design decisions. The operational reality is a determinism discipline. Because mixture and decontamination are contracts, every stage must be reproducible to the byte. In practice that means content-addressed inputs, pinned filter thresholds and model versions, and a manifest that records, for each output shard, the exact transforms that produced it. A representative dedup step is fuzzy document matching by MinHash and LSH, sketched here against its method rather than any one codebase:

```
# Fuzzy dedup: bucket by LSH, then drop near-duplicates within a bucket
def dedup(docs, threshold=0.8, num_perm=128):
    seen = LSHIndex(threshold=threshold, num_perm=num_perm)
    for doc in docs:
        sig = minhash(shingles(doc.text), num_perm=num_perm)
        if not seen.query(sig):      # no near-duplicate already kept
            seen.insert(doc.id, sig)
            yield doc                # else: drop, counted in the m
```

The decontamination step is the same shape with a different index: build n-gram signatures for every item in the eval registry, then drop or flag any training document whose overlap exceeds the contract threshold, and write the count and identities of removed documents into the report that evaluation consumes.

Further reading

- Penedo et al., "The FineWeb Datasets," 2024. [arXiv:2406.17557](#)
- Penedo et al., "The RefinedWeb Dataset for Falcon LLM," 2023. [arXiv:2306.01116](#)
- Wenzek et al., "CCNet: Extracting High Quality Monolingual Datasets from Web Crawl Data," 2020 (LREC). [ACL Anthology](#)
- Lee et al., "Deduplicating Training Data Makes Language Models Better," 2022 (ACL). [ACL Anthology](#)
- Abbas et al., "SemDeDup: Data-efficient learning at web-scale through semantic deduplication," 2023. [arXiv:2303.09540](#)
- Xie et al., "DoReMi: Optimizing Data Mixtures Speeds Up Language Model Pretraining," 2023 (NeurIPS). [OpenReview](#)
- Gao et al., "The Pile: An 800GB Dataset of Diverse Text for Language Modeling," 2020. [arXiv:2101.00027](#)
- Raffel et al., "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer" (T5 / C4), 2020 (JMLR). [JMLR](#)
- Soldaini et al., "Dolma: an Open Corpus of Three Trillion Tokens for Language Model Pretraining Research," 2024 (ACL). [ACL Anthology](#)
- Gunasekar et al., "Textbooks Are All You Need" (phi-1), 2023. [arXiv:2306.11644](#)
- Eldan & Li, "TinyStories: How Small Can Language Models Be and Still Speak Coherent English?," 2023. [arXiv:2305.07759](#)
- Maini et al., "Rephrasing the Web: A Recipe for Compute and Data-Efficient Language Modeling" (WRAP), 2024. [arXiv:2401.16380](#)
- Yang et al., "Rethinking Benchmark and Contamination for Language Models with Rephrased Samples" (LLM decontaminator; on n-gram overlap thresholds and their limits), 2023. [arXiv:2311.04850](#)
- Qin et al., "ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs" (ToolBench tool-use trajectories as training data), 2023. [arXiv:2307.16789](#)

5. Tokenization

A tokenizer is the map between text and the integer ids a model actually reads. By the end of this chapter a reader can explain why almost every frontier model uses byte-level BPE, what unigram and tokenizer-free schemes trade against it, and why this one early choice quietly caps downstream quality and is among the hardest decisions in the stack to revisit.

A model never sees text

A transformer does not see text. It sees a sequence of integers drawn from a fixed vocabulary, each mapped to a row of an embedding matrix. Something has to turn a stream of Unicode into that sequence, and the tokenizer is that something. The choice looks like plumbing and behaves like a foundation.

What makes it a foundation rather than plumbing is the conjunction of three constraints, each harmless alone and unforgiving together.

The vocabulary must be finite and fixed before training begins, yet text is open. New words, code, emoji, and languages the designer never enumerated must still encode without failure. A table of words can never be complete, so completeness has to come from somewhere other than the table.

The segmentation must be cheap. Tokenization runs over the entire corpus on CPU fleets ahead of the GPU run, as covered in [Chapter 4](#), so an elegant but slow scheme is not affordable at corpus scale.

And the result is permanent. The embedding and output matrices are sized to the vocabulary, so the vocabulary cannot change without retraining the model. A tokenizer trained on a skewed corpus caps quality for everything under-represented in it, and that ceiling is invisible until evaluation.

These three pull together into the chapter's thesis: the tokenizer choice bounds downstream quality and is close to irreversible. Everything that follows is an attempt to satisfy the first two constraints while living honestly with the third.

The subword bargain

The dominant answer is subword tokenization: a vocabulary of pieces smaller than words but larger than characters, sized so common words are single tokens while rare words decompose into known parts. This is a bargain struck directly against the open-vocabulary constraint. Any string is representable because the pieces bottom out at units that always exist, and frequent text stays short because common sequences are merged into single ids.

Byte-pair encoding (BPE) is the workhorse construction. Start from a base alphabet, count adjacent pair frequencies across the corpus, merge the most frequent pair into a new symbol, and repeat until the vocabulary reaches its target size (Sennrich et al. 2015). The learned merge list is the tokenizer; encoding replays the merges in order.

```
# BPE training, conceptual
vocab = set(base_symbols)
merges = []
while len(vocab) < target_size:
    pair = most_frequent_adjacent_pair(corpus, vocab)
    merges.append(pair)
    vocab.add(merge(pair))
```

A worked example makes the loop concrete. Figure 5.1 runs three merges on a tiny corpus of the word `low` repeated, where the most frequent adjacent pair is absorbed into a new symbol each round until the common sequence is a single token.

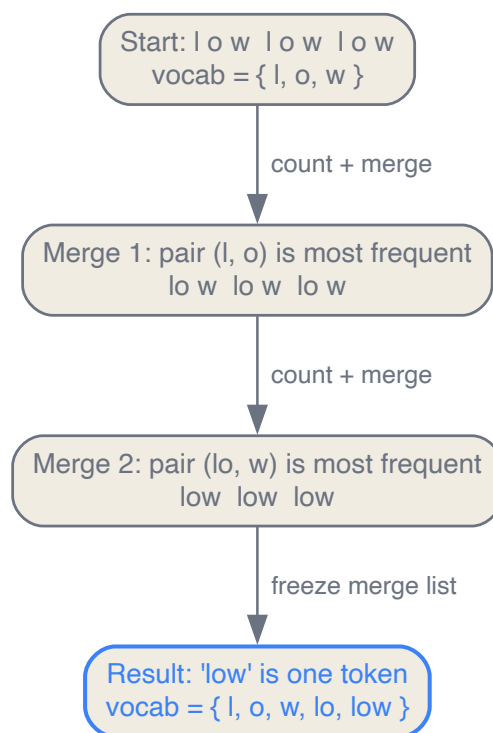


Figure 5.1. BPE training as repeated greedy merges. Each round counts adjacent pairs, merges the most frequent one into a new symbol, and appends it to the merge list. Common sequences collapse into single tokens. After Sennrich et al. (2015).

Run the pseudocode for real: this trains BPE on a tiny corpus, prints the learned merges, then shows how common and rare words split into a different number of tokens.

```

from collections import Counter

def merge_pair(word, a, b):
    out, i = [], 0
    while i < len(word):
        if i < len(word) - 1 and word[i] == a and word[i + 1] == b:
            out.append(a + b); i += 2
        else:
            out.append(word[i]); i += 1
    return out

corpus = "low low low lower lowest newest wider".split()
words = [list(w) + ["_"] for w in corpus]
merges = []
for _ in range(6):
    pairs = Counter()
    for w in words:
        for a, b in zip(w, w[1:]):
            pairs[(a, b)] += 1
    if not pairs:
        break
    (a, b), _ = max(pairs.items(), key=lambda kv: (kv[1], kv[0]))
    merges.append((a, b))
    words = [merge_pair(w, a, b) for w in words]

print("learned merges:", [a + "+" + b for a, b in merges])
for w, toks in zip(corpus, words):
    print(f"{w:7s} -> {' '.join(toks)} ({len(toks)} tokens)")

```

The decisive refinement is to run BPE over raw bytes rather than Unicode characters. With 256 byte values as the base alphabet, every possible input encodes with zero out-of-vocabulary risk, even text the tokenizer never saw, and there is no separate unknown token to handle. This is byte-level BPE, and it is why the same tokenizer can ingest English, code, a new emoji, and a script absent from its training data. Byte-fallback gives a near-equivalent guarantee in character-based schemes by decomposing any unseen character into its underlying bytes. The open-vocabulary constraint, which sank every word-level table, is now satisfied by construction rather than by hoping the table was large enough.

There is a second question the bargain leaves open: not what the pieces are, but how to choose a split when several are possible. BPE is greedy and frequency-driven, with no probabilistic model of how a string should split. The unigram language model offers an alternative: posit that a sentence is a bag of subword pieces with independent probabilities, start from a large candidate vocabulary, and prune pieces whose removal least hurts the corpus likelihood (Kudo 2018). Unigram yields a principled segmentation and, because it defines a distribution over segmentations, supports subword regularization: sampling alternative splits at training time as a form of augmentation (Kudo 2018).

Two parameters set by the data pipeline govern the rest. Vocabulary size trades sequence length against matrix size, and the tokenizer's own training corpus bakes in which languages and domains get short, efficient encodings. Both are frozen here and consumed, not chosen, downstream.

From words to bytes

The construction above did not arrive whole. Tokenization moved from words to bytes in three steps, each removing a failure of the last. Figure 5.2 traces the lineage, with the specific failure each stage carries and the one its successor removes.



Figure 5.2. The lineage from word-level to byte-level tokenization. Each arrow is labeled with the failure the next stage removes. Byte-level BPE closes the out-of-vocabulary gap entirely.

Word-level vocabularies came first and broke on the open-vocabulary problem: any word absent from the table became a single unknown token, discarding its content. Character-level models removed the unknown token but made sequences long and forced the model to relearn spelling from scratch, spending depth on what a vocabulary could have stored.

Sennrich et al. (2015) brought BPE, invented for data compression, to neural machine translation as the subword compromise, and it became the default (Sennrich et al. 2015). Kudo (2018) introduced the unigram language model as a probabilistic alternative with subword regularization (Kudo 2018). Kudo and Richardson (2018) packaged both in SentencePiece, which treats input as a raw stream including whitespace, so the tokenizer is language-independent and needs no pre-tokenization rules (Kudo and Richardson 2018). The final step was byte-level BPE, which made the base alphabet the 256 byte values and closed the out-of-vocabulary gap entirely. Byte-level BPE is the construction behind most current frontier tokenizers, with unigram and SentencePiece remaining common in multilingual and non-Latin-script settings.

The open frontier is whether to tokenize at all. Tokenizer-free schemes operate directly on bytes or characters, learning the segmentation inside the model rather than fixing it in a preprocessing step. They remove the frozen vocabulary and its biases, at the cost of longer sequences and the compute to process them. This is a live research direction, not a settled replacement.

WHAT'S CONTESTED

Whether a fixed tokenizer should exist at all is genuinely unsettled. The case against it is that the vocabulary is a frozen, corpus-dependent artifact that bakes in language and domain bias, fragments numbers and code in ways that hurt arithmetic and program synthesis, and is the single component you cannot change after training. Tokenizer-free and byte-level approaches answer by learning structure inside the model, trading preprocessing bias for longer sequences and higher compute per character. Byte-level BPE is so entrenched, and the sequence-length penalty of pure byte models so real, that the field has not converged. Treat tokenizer-free schemes as a promising open direction, not a drop-in replacement.

What you cannot take back

The history above reads like progress, and it is, but every choice it settled became a setting that someone has to make once and then live with. Each tokenizer setting is a balance with a knee, and several of them set a ceiling that only shows up later. Figure 5.3 makes the central one visible: a larger vocabulary shortens sequences but enlarges the matrices sized to it, so the two costs cross rather than agree.

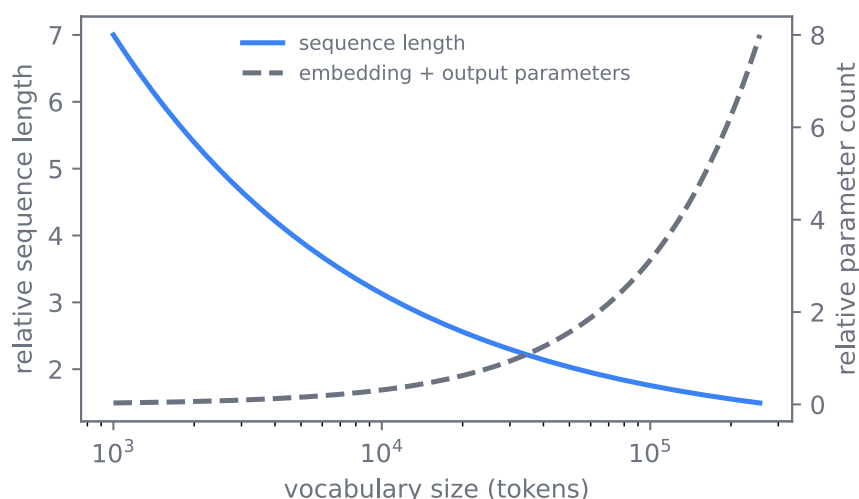


Figure 5.3. Schematic illustration of the vocabulary-size trade-off: a larger vocabulary shortens token sequences (cheaper per document) while enlarging the embedding and output matrices sized to it. The optimum is a knee, not an endpoint. Idealized data, not measured.

- **Vocabulary size.** A larger vocabulary shortens sequences, which makes training and inference cheaper per document and improves multilingual fertility, the number of characters carried per token. It also enlarges the embedding and output matrices and can starve rare tokens of the occurrences they need to train a good embedding. The optimum is a function of corpus and budget, not a constant.
- **Training corpus for the tokenizer.** Train it on the final data mixture or on a separate curated set? The choice permanently bakes the language and domain balance into the vocabulary. A mismatch with the real mixture spends vocabulary on the wrong things and leaves under-represented languages with long, expensive encodings.
- **BPE versus unigram.** BPE is greedy, fast, and ubiquitous, with strong tooling and broad compatibility. Unigram is probabilistic, supports subword regularization, and often segments multilingual text more evenly, at the cost of a heavier training procedure and less ecosystem inertia behind it.
- **Multilingual fairness.** Petrov et al. (2023) show that a tokenizer tuned on English-heavy data splits other languages into many more tokens for the same meaning (Petrov et al. 2023). That inflates cost and latency for those users and shortens their effective context window, a structural unfairness fixed in the vocabulary itself. This is the trade-off that most directly caps downstream quality, and it is set entirely here. Figure 5.4 shows the shape of the gap: the same sentence costs several times more tokens in some languages than in English.

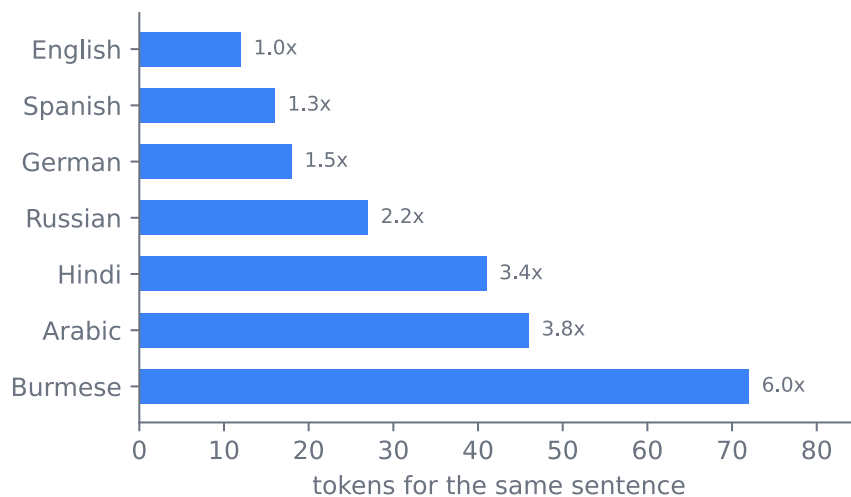


Figure 5.4. Schematic illustration of tokenizer fertility across languages: the same sentence costs many more tokens under an English-tuned tokenizer for some languages, with the multiple relative to English shown beside each bar. Idealized token counts, after Petrov et al. (2023).

The reason these matter more than ordinary hyperparameters is lock-in. A vocabulary trained on a skewed mixture caps multilingual quality and inflates token cost for under-served languages, and it cannot be changed without retraining the model, because the embedding and output matrices are sized to it. Vocabulary extension during continued training can graft on new tokens as a partial workaround, but it does not undo the original bias. The honest summary is that the tokenizer is decided once and lived with for the model's life, which is why a choice that looks like preprocessing deserves the scrutiny of an architecture decision.

CONSTRAINT ARROW

The tokenizer is frozen in the data pipeline and dictates a choice one layer up. The vocabulary size set here fixes the row count of the embedding matrix and the output projection in [Chapter 6](#): that layer consumes the vocabulary and sizes its matrices to match, it does not get to choose it. A token budget decided during data work therefore sets a hard parameter of the model architecture.

Knobs you set once

Naming the lock-in is not the same as dismissing the work. In practice a tokenizer is trained once, with a handful of decisions that are expensive to get wrong, and two recurring knobs deserve their own attention beyond vocabulary size and training corpus.

The first is digit handling: splitting numbers into individual digits so arithmetic does not depend on whether "1234" happened to merge into one token. The second is whitespace handling, where SentencePiece-style schemes encode spaces as ordinary symbols so detokenization is exact and reversible (Kudo and Richardson 2018). The build itself is the BPE training, conceptual loop above: count, merge, repeat, then freeze the merge list and vocabulary as the artifact every later stage consumes.

That artifact is the whole point. It is small, it is produced early, and it binds everything downstream to the language and domain balance present in the corpus that made it. A choice that looks like preprocessing turns out to be the first irreversible commitment in the model's life, and that is the lens worth carrying into the architecture that consumes it.

Further reading

- Sennrich et al., "Neural Machine Translation of Rare Words with Subword Units" (BPE), 2015. [arXiv:1508.07909](#)
- Kudo, "Subword Regularization: Improving Neural Network Translation Models with Multiple Subword Candidates" (Unigram LM), 2018 (ACL). [ACL Anthology](#)
- Kudo & Richardson, "SentencePiece: A simple and language independent subword tokenizer and detokenizer," 2018 (EMNLP, system demo). [ACL Anthology](#)
- Petrov et al., "Language Model Tokenizers Introduce Unfairness Between Languages," 2023 (NeurIPS). [NeurIPS Proceedings](#)

6. Transformer Architecture and Its Variants

A frontier dense model is a stack of nearly identical transformer blocks, and that block has barely changed since 2023 (Touvron et al. 2023; Bai et al. 2023). Almost everything inside it is a question that has been answered and closed: how to normalize, how to activate, how to encode position. One question stays open, and it is not in the math of the forward pass at all. It is a cost that lives in memory during generation, the size of the key-value cache, and it is the only force still strong enough to make an architect redraw a working block. This chapter reads the block twice: once as a settled forward pass, and once as the single open cost that drives the attention variants from MHA through MQA and GQA to MLA.

One block, two jobs

A transformer block has to do two distinct jobs and do them in a way that survives being stacked a hundred times deep. It must let tokens exchange information, since a language model is useless if position i cannot see position j , and it must transform each token's features, which is where most of the model's knowledge lives. Stacking those two operations naively does not work: a deep residual network is hard to keep trainable, the activation and normalization choices decide whether the gradient survives to the bottom layer, and attention, the one operation that mixes across positions, costs $O(n^2)$ in compute and leaves behind a cache that grows without bound during generation.

The right way to read a block is not "attention, then MLP." It is two sublayers that each read a shared residual stream, compute a delta, and write it back:

$$x = x + \text{sublayer}(\text{norm}(x))$$

The residual stream is the model's working memory, a vector of width d_{model} carried from the embedding to the final layer. Each block gets one pass to read it and add to it. Depth is the number of such read-modify-write passes; width is how much the stream can hold at once. This framing makes the rest of the design fall out. Figure 6.1 draws one pre-norm block this way: the residual stream runs straight down as an identity path, and each sublayer branches off it through a norm, computes a delta, and adds the delta back.

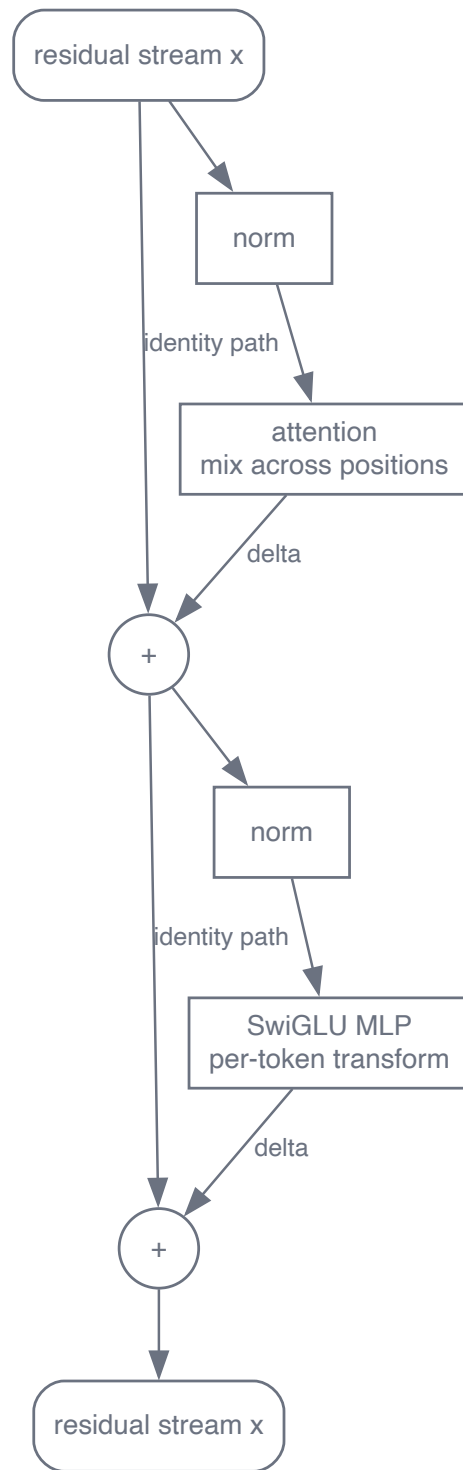


Figure 6.1. A single pre-norm transformer block read as two read-modify-write sublayers on a shared residual stream. Each branch normalizes a copy of the stream, computes a delta, and adds it back, while the identity path down the middle is the gradient highway.

The first sublayer is attention, the only place tokens exchange information, and the subject of the second half of this chapter. The second is a position-wise MLP applied to each token independently, and it holds most of the parameters: a feed-forward layer that expands to roughly four times d_{model} and projects back costs about $8 \cdot d_{\text{model}}^2$ per block, dwarfing attention's projections. Knowledge lives mostly in the MLP; mixing lives entirely in attention.

Three questions the field has closed

Three choices keep a deep stack of these blocks trainable, and all three are settled. They are worth reading not as live debates but as a record of how the consensus stack was assembled, one near-free win at a time.

Normalization has two orthogonal axes. The form is LayerNorm (Ba et al. 2016), which re-centers and re-scales with a learned gain and bias, versus RMSNorm (Zhang and Sennrich 2019), which only re-scales, dropping the mean subtraction and the bias. Mean-centering buys little, so RMSNorm is cheaper and now the default. The placement is post-norm, the original design with the norm after the residual add (Vaswani et al. 2017), versus pre-norm, with the norm inside the branch and an identity path straight down the residual stream. Pre-norm keeps a clean gradient highway from the top of the stack to the bottom, and it is what lets a deep model train without elaborate warmup (Xiong et al. 2020).

Activation has a clear lineage: ReLU, simple but prone to dead units, then GeLU (Hendrycks and Gimpel 2016), the smooth BERT and GPT-2 default, then gated linear units, of which SwiGLU is the modern winner (Shazeer 2020). SwiGLU computes $(\text{Swish}(xW_1) \odot xV)W_2$, three matrices rather than two. The detail readers get wrong: the hidden dimension is scaled by about $2/3$ so the gated FFN keeps the same parameter count as a plain $4 \times$ FFN, which is what makes a like-for-like comparison fair.

Position is the third. Attention is permutation-invariant on its own, so the block must be told where tokens sit. RoPE rotates the query and key vectors by an angle proportional to absolute position, in two-dimensional subspace pairs, so that the dot product $q \cdot k$ depends only on the relative offset $i - j$ (Su et al. 2021). A base-frequency parameter, theta, sets how fast the rotation turns and quietly governs how far context can later be extended. ALiBi takes the other route: it adds a static, head-specific linear penalty $-m \cdot (i - j)$ to the pre-softmax scores, with no learned positional parameters at all (Press et al. 2022). The contrast is rotations on vectors versus biases on scores. RoPE became the dense default.

One bracket choice frames the stack from outside. Embedding and unembedding sit at the two ends: the input embedding maps token ids to vectors, the LM head maps the final residual state to vocabulary logits. Weight tying shares one matrix for both (Press and Wolf 2017). This is a genuine tension, not a default. Tying saves $\text{vocab} \cdot d$ parameters and regularizes small models, but once the vocabulary is large the saving is marginal, and most large modern models leave the two untied.

These choices, assembled, are the whole settled block. Vaswani et al. shipped the original transformer with post-norm and a plain FFN (Vaswani et al. 2017). Post-norm diverges at depth without heavy warmup, so pre-norm replaced it (Xiong et al. 2020). RMSNorm dropped LayerNorm's mean subtraction at no measured quality cost (Zhang and Sennrich 2019). SwiGLU edged out GeLU under the $2/3$ rescaling (Shazeer 2020). RoPE became the positional default over ALiBi (Su et al. 2021). By the time of LLaMA and Qwen, the consensus had set: pre-norm, RMSNorm, a SwiGLU FFN, RoPE, and no bias terms anywhere (Touvron et al. 2023; Bai et al. 2023). Strikingly little has changed in the dense block since 2023. The action moved to data, scale, and post-training.

The one cost that won't settle

Now attention itself, the sublayer that mixes across positions and the source of the chapter's live constraint. Scaled dot-product attention projects each token to a query, key, and value, then computes (Vaswani et al. 2017)

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{d}\right) V$$

The d divisor keeps the logits from growing with dimension and saturating the softmax. A causal mask zeros the upper triangle so a token attends only to the past. The cost is $O(n^2)$ in both compute and the score matrix, which is the pressure behind every efficiency trick downstream.

Multi-head attention splits d into h heads so attention can attend to several subspaces at once, concatenating their outputs and projecting once more (Vaswani et al. 2017). Heads are cheap in parameters but not free at inference, because each head carries its own keys and values, and those must be kept.

That cost is the spine of the chapter. During autoregressive decode, the keys and values for every past token are cached so they are not recomputed at each step. The cache size is

$$2 \cdot \text{layers} \cdot \text{heads} \cdot d_{\text{head}} \cdot \text{seq} \cdot \text{batch} \cdot \text{dtype}$$

It grows linearly with context length and batch size, and at long context it overtakes the model weights themselves. Figure 6.2 draws that linear growth and the crossover with a flat weights line, with the slope of each variant set by how many KV heads it keeps. This single fact is what explains MQA, GQA, and MLA. The question each answers is: how much of the KV cache can you delete before quality breaks?

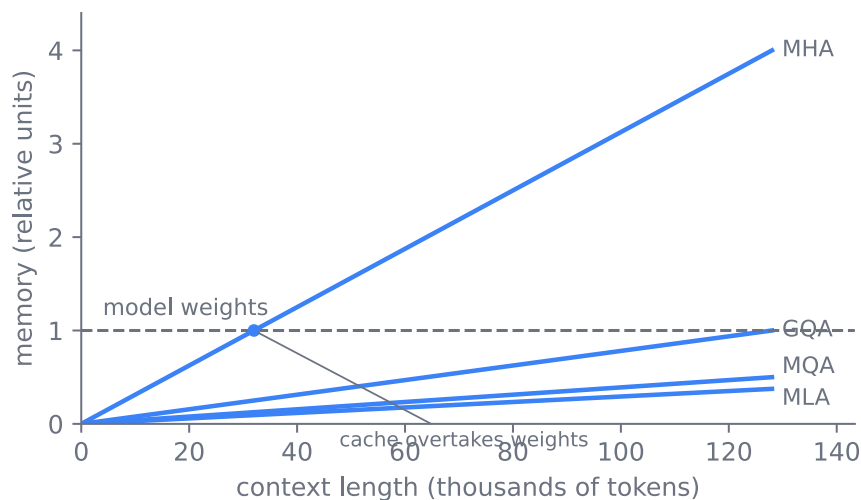


Figure 6.2. Schematic illustration of KV-cache memory growing linearly with context length, after Shazeer (2019) and Ainslie et al. (2023). The slope of each attention variant is set by how many key-value heads it keeps, so MHA grows fastest and MQA and MLA slowest. Past a crossover the cache overtakes the flat model-weights line. Idealized relative units, not measured data.

Try this for a 7B-class model: plot each variant's KV cache against context and read off where it overtakes the weights. Change `n_kv` to see how sharing heads pushes the crossover out.


```

import numpy as np
import matplotlib.pyplot as plt

layers, d_head, dtype, params = 32, 128, 2, 7e9 # fp16 bytes, 7B we
weights_gb = params * dtype / 1e9
seq = np.linspace(0, 200_000, 400)

for name, n_kv in [("MHA", 32), ("GQA", 8), ("MQA", 1)]:
    per_tok = 2 * layers * n_kv * d_head * dtype # bytes per token
    cache_gb = per_tok * seq / 1e9
    crossover = weights_gb * 1e9 / per_tok
    print(f"{name:3s}: {per_tok/1024:6.0f} KB/token, crosses weights
    plt.plot(seq / 1000, cache_gb, label=name)

plt.axhline(weights_gb, ls="--", color="gray", label=f"weights ({wei
plt.xlabel("context length (1000s of tokens)"); plt.ylabel("KV cache
plt.legend(); plt.title("KV cache vs context, batch=1"); plt.show()

```

The first answer cuts the most. Multi-query attention shares a single key/value head across all query heads, cutting the cache by a factor of h (Shazeer 2019). The cut is dramatic, but collapsing to one KV head can destabilize training and drop quality non-gracefully. The second answer interpolates. Grouped-query attention shares K and V within g groups of query heads, recovering MHA ($g = h$) at one extreme and MQA ($g = 1$) at the other, and it can be uptrained cheaply from an existing MHA checkpoint (Ainslie et al. 2023). GQA recovers most of the quality at most of the savings and became the default in the LLaMA and Qwen families (Touvron et al. 2023; Bai et al. 2023). Figure 6.3 shows the mechanism: the cache holds one K and V per distinct KV head, so the savings come entirely from how many query heads are made to share each one.

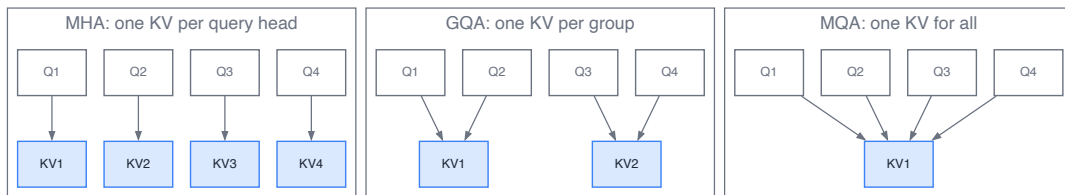


Figure 6.3. How MHA, GQA, and MQA wire query heads to cached key/value heads. Only the KV heads sit in the cache, so collapsing many query heads onto one KV head is what shrinks it.

Multi-head latent attention, introduced in DeepSeek-V2, attacks the same cost from a different angle (DeepSeek-AI 2024). Instead of sharing heads, it projects K and V down into a low-rank latent vector that is cached in place of the full per-head K and V, decompressing on the fly, with a decoupled RoPE component carried alongside. The pitch is GQA-class cache savings, or better, at MHA-class quality. It is the other point on the cache-compression frontier that Figure 6.4 lays out.

Figure 6.4. The cache-compression frontier. From MHA, GQA and MQA trade KV heads for a smaller cache along one branch, while MLA compresses the cache into a low-rank latent along the other.

FlashAttention belongs to this story only as a reminder of what these variants do not solve. It keeps attention exact and makes the $O(n^2)$ score matrix affordable by never writing it to high-bandwidth memory, but it does not shrink the KV cache, which is a memory footprint, not a compute kernel (Dao et al. 2022). The serving-time mechanics of that kernel and of cache paging belong to Chapter 18 and Chapter 17.

CONSTRAINT ARROW

The key-value cache size motivates an attention variant. MHA's cache grows with the number of heads, and at long context and large batch it overtakes the weights, so an architect trades head independence for a smaller cache: MQA shares one KV head, GQA shares within groups, MLA compresses to a latent. None of these change what attention computes in principle. They exist because a downstream cost, the memory a decoding server must hold per sequence, reaches up and reshapes the block. The serving pressure in Chapter 16 is what makes a working block worth touching at all.

WHAT'S CONTESTED

Whether MLA actually breaks the quality-versus-cache trade is not settled. The DeepSeek position is that latent compression delivers cache savings at MHA-class quality, dominating GQA on the frontier (DeepSeek-AI 2024). The conservative position is that GQA, which is simpler, uptrains from an existing MHA checkpoint, and has been validated across many shipped models (Ainslie et al. 2023), remains the safe default, and that MLA's advantage is entangled with the rest of the DeepSeek recipe and harder to transfer than a single comparison suggests. The two coexist in production today: most open models use GQA, DeepSeek uses MLA. Treat the choice as open, not as a settled win for either.

Reading the dials

Every choice above is a balance with a knee. Set side by side, they are the dials an architect actually turns.

- **Norm placement, pre versus post.** Post-norm can reach slightly better final loss when it trains at all, but it is unstable at depth without heavy warmup and careful initialization. Pre-norm trades a sliver of peak quality for a robustly trainable gradient highway, and wins in practice (Xiong et al. 2020).
- **Norm form, LayerNorm versus RMSNorm.** Mean-centering buys little. RMSNorm drops it for lower cost and one fewer parameter set, a near-free win, now default (Zhang and Sennrich 2019).

- **Activation choice.** SwiGLU's gain is small but consistent, and effectively free under the $2/3$ hidden-dimension rescaling (Shazeer 2020). The cost is a third weight matrix and slightly more irregular shapes for kernels and parallelism.
- **Positional scheme as a one-way door.** The base scheme is chosen at run start and is expensive to change. It constrains how far context can later be extended and how well the model extrapolates. RoPE's theta base is a small knob with large long-context consequences, the mechanisms for which are Chapter 20.
- **Weight tying.** A real tension: parameter saving and small-model regularization against the freedom and marginal cost at scale that lead large models to untie (Press and Wolf 2017).
- **Depth versus width.** At a fixed parameter budget, deeper means more sequential composition but harder training and parallelism, with pipeline bubbles and serial latency, while wider means more per-step compute, friendlier to tensor parallelism, and easier to keep stable. This is co-designed with the systems layer in Chapter 8, not resolved in isolation, and the quality-per-FLOP side of it is set by Chapter 3.
- **KV cache versus quality, the dominant axis.** MHA gives full quality at full cache, GQA most quality at a fraction of cache, MQA the smallest cache at a quality and stability risk, and MLA claims to break the trade through latent compression. The right point depends on target context length and batch, since the cache scales with both.

Building the block, and how it breaks

Assembling the consensus dense block is mostly bookkeeping once the choices above are made: pre-norm with RMSNorm on each sublayer's input, a SwiGLU FFN with the hidden dimension rescaled by $2/3$, RoPE applied to queries and keys before the dot product, GQA grouping for the KV heads, and no bias terms anywhere. A reader can recognize this shape across nearly every open dense model since LLaMA (Touvron et al. 2023).

The failure modes are worth naming because each bites at a different time. Post-norm at depth blows up or stalls early in training, the classic symptom pre-norm was invented to fix (Xiong et al. 2020). A positional scheme or a too-small RoPE theta chosen for short context caps long-context behavior later, and the cost surfaces only at the first context-window bump. Forgetting the $2/3$ rescaling when switching to a gated FFN silently inflates or shrinks the parameter budget and breaks like-for-like comparisons. Untied, unregularized output logits can drift without bound, which is why z-loss guards them, with mechanics in Chapter 3. And collapsing too far toward MQA can destabilize training and drop quality non-gracefully, which is the reason GQA exists (Ainslie et al. 2023): the cheapest cache is not safely the best one.

Further reading

- Vaswani et al., "Attention Is All You Need," 2017 (NeurIPS). [arXiv:1706.03762](#)
- Ba, Kiros & Hinton, "Layer Normalization," 2016. [arXiv:1607.06450](#)
- Zhang & Sennrich, "Root Mean Square Layer Normalization" (RMSNorm), 2019 (NeurIPS). [arXiv:1910.07467](#)
- Xiong et al., "On Layer Normalization in the Transformer Architecture," 2020 (ICML). [arXiv:2002.04745](#)
- Hendrycks & Gimpel, "Gaussian Error Linear Units (GELUs)," 2016. [arXiv:1606.08415](#)
- Shazeer, "GLU Variants Improve Transformer" (SwiGLU), 2020. [arXiv:2002.05202](#)
- Su et al., "RoFormer: Enhanced Transformer with Rotary Position Embedding" (RoPE), 2021. [arXiv:2104.09864](#)
- Press, Smith & Lewis, "Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation" (ALiBi), 2022 (ICLR). [arXiv:2108.12409](#)

- Press & Wolf, "Using the Output Embedding to Improve Language Models" (weight tying), 2017 (EACL). [arXiv:1608.05859](#)
- Shazeer, "Fast Transformer Decoding: One Write-Head is All You Need" (MQA), 2019 (arXiv). [arXiv:1911.02150](#)
- Ainslie et al., "GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints," 2023 (EMNLP). [aclanthology.org/2023.emnlp-main.298](#)
- Dao et al., "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness," 2022 (NeurIPS). [arXiv:2205.14135](#)
- DeepSeek-AI, "DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model," 2024 (arXiv), introduces multi-head latent attention (MLA). [arXiv:2405.04434](#)
- Touvron et al., "LLaMA: Open and Efficient Foundation Language Models," 2023. [arXiv:2302.13971](#)
- Bai et al., "Qwen Technical Report," 2023. [arXiv:2309.16609](#)
- Yang et al., "Qwen2 Technical Report," 2024. [arXiv:2407.10671](#)
- Qwen Team, "Qwen2.5 Technical Report," 2024. [arXiv:2412.15115](#)

7. Beyond Dense Transformers: MoE, SSMS, Hybrids

The dense transformer of [Chapter 6](#) runs every token through every parameter. That single contract hides two walls a frontier builder eventually hits, and this chapter is about the two ways the field has learned to break through them. One wall is in the feed-forward block: capacity and per-token compute are welded together, so the only way to know more is to compute more. The other is in attention: mixing every token with every other costs the square of the sequence length and leaves behind a key-value cache that grows without bound. A mixture-of-experts layer breaks the first wall by routing a token to a few of its many experts. A state-space model breaks the second by replacing attention's quadratic mixing with a linear recurrence. By the end a reader can explain why sparse models grow their parameter count almost for free, what the router has to get right for that bet to pay off, and where state-space and hybrid stacks sit as the live alternative to attention.

The two walls of the dense block

A dense feed-forward block runs every token through all of its parameters. That couples two things a frontier builder wants to separate: the model's capacity, how much it can know, and its per-token compute, how much each forward and backward step costs. Under that coupling, the only way to add knowledge is to add FLOPs, and FLOPs are the budget [Chapter 3](#) spends. A model that wants more capacity than its compute budget can afford has no move inside the dense design. Call this the capacity wall.

The second wall is attention's own. Self-attention mixes every token with every other, so its cost grows with the square of sequence length, and the key-value cache it leaves behind grows linearly with context and dominates serving memory ([Chapter 16](#)). For long context that quadratic is the wall. The dense transformer therefore carries two separate inefficiencies: a feed-forward block that spends compute on capacity it may not need per token, and an attention block that spends compute and memory on a mixing pattern that may be denser than the task requires. The rest of this chapter takes the two walls in turn. MoE attacks the first; state-space models attack the second; hybrids decline to choose.

Breaking the capacity wall: mixture-of-experts

Decoupling parameters from FLOPs

The mixture-of-experts answer is to decouple parameters from active FLOPs. An MoE layer holds N expert feed-forward networks but routes each token to only k of them, with $k \ll N$. The identity that matters: *total* parameters scale capacity and knowledge, while *active* parameters, the k chosen experts plus the always-on parts of the model, scale per-token compute and step cost. Capacity becomes cheap relative to FLOPs, which is the whole frontier appeal. The same arithmetic cuts inference FLOPs too, though serving an MoE is its own problem and out of scope here. [Figure 7.1](#) makes the split visible: as the expert count N grows, total parameters climb while the active count that sets per-token FLOPs stays flat with k fixed.

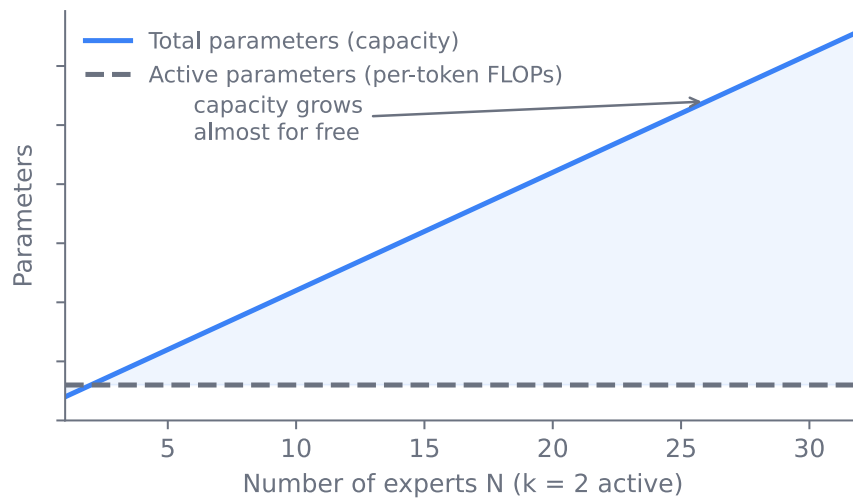


Figure 7.1. Schematic of how a mixture-of-experts layer decouples capacity from per-token compute. Total parameters grow with the expert count, while active parameters, the FLOPs paid per token, stay flat because only experts fire. Idealized counts, not measured data, after Shazeer et al. (2017) and Fedus et al. (2021).

Sweep the expert count N and watch total capacity climb while the active params that set per-token FLOPs stay almost flat.

```
import numpy as np, matplotlib.pyplot as plt

# MoE decouples total params (capacity) from active params (per-token FLOPs)
d, ffn = 4096, 14336          # hidden size and one expert's FFN width
k = 2                          # experts fired per token
per_expert = 2 * d * ffn       # up + down projection of one expert
N = np.arange(1, 65)          # number of experts in the layer
total = N * per_expert         # all experts are stored
active = k * per_expert + N * d # k fired experts + router

plt.figure(figsize=(5, 3))
plt.plot(N, total / 1e9, label="total params (capacity)")
plt.plot(N, active / 1e9, label="active params (per-token FLOPs)")
plt.xlabel("experts N"); plt.ylabel("billions of params"); plt.legend()
plt.tight_layout(); plt.show()

print(f"N=64, k={k}: {total[-1]/1e9:.1f}B total but only {active[-1]/1e9:.1f}B active")
print(f"capacity-to-compute ratio: {total[-1]/active[-1]:.0f}x")
```

The new learned object is the router, also called the gate. It is a small linear map from a token's hidden state to one score per expert. A softmax or sigmoid turns scores into weights, the top- k experts are selected, and their outputs are combined weighted by the renormalized gate values. In sketch form:

```
# router: a linear map h -> scores over N experts
scores = h @ W_gate                # (tokens, N)
weights = softmax(scores)          # or sigmoid per-expert
top = topk(weights, k)             # indices and gate values
y = sum(g_i * Expert_i(h) for i, g_i in top)
```

This tiny map is the only genuinely new component, and it is the source of almost every MoE pathology. The whole rest of the design exists to keep the router honest. Figure 7.2 traces one token through the layer: the gate scores all experts, top- k are selected and combined, while a shared expert (a later refinement) stays always-on in parallel.

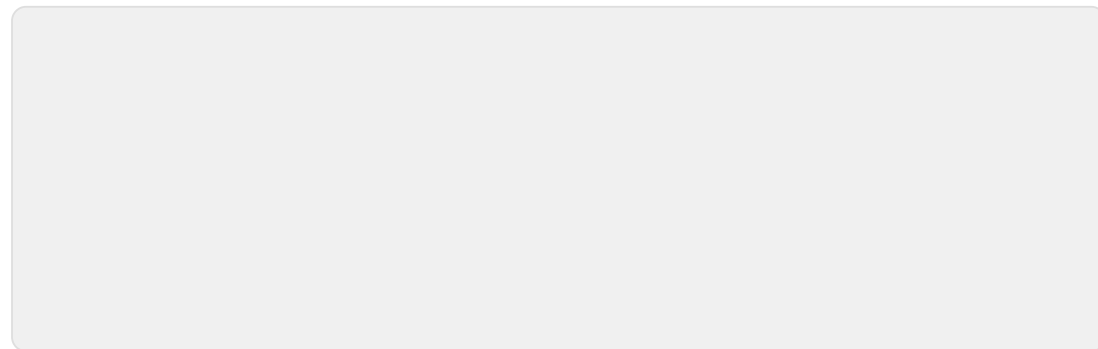


Figure 7.2. One token through an MoE layer. The router scores all N experts, the top- k routed experts are weighted and summed, and an always-on shared expert runs in parallel. After Shazeer et al., 2017 and Dai et al., 2024.

The router and its pathologies

The first thing the router gets wrong on its own is balance. Left alone, the gate collapses onto a few favored experts in a winner-take-all loop: an expert that is chosen gets more gradient, improves, and is chosen more, while the rest starve and become dead parameters. Two families of fix exist. The auxiliary load-balancing loss, from Shazeer et al. (2017) (Shazeer et al. 2017) and refined in GShard (Lepikhin et al. 2020) and Switch (Fedus et al. 2021), adds a penalty term on the product of the fraction of tokens routed to each expert and the gate mass placed on it, pushing the distribution toward uniform. It works, but it injects a loss whose weight trades balance against task quality. The newer answer, loss-free or bias-based balancing from DeepSeek-V3 (DeepSeek-AI 2024), drops the auxiliary loss entirely. Instead it keeps a per-expert routing *bias*, nudged up for under-used experts and down for over-used ones between steps, steering selection without adding an interfering gradient (Wang et al. 2024). That sidesteps the balance-versus-quality tension the auxiliary loss creates.

Two refinements change what an "expert" is. *Fine-grained* experts slice each expert into smaller ones and raise k proportionally, so the same active FLOPs buy far more routing combinations and sharper specialization. *Shared* experts are a few that every token always passes through, absorbing the common, domain-general computation so the routed experts can specialize instead of each re-learning the basics. DeepSeekMoE combines both (Dai et al. 2024), and the pair is now a common default for large open MoE models.

One systems fact intrudes on this otherwise clean picture. Routing is dynamic, but hardware wants fixed-shape buffers, so each expert is given a *capacity* cap, roughly `capacity_factor * (tokens / experts)`. Tokens routed to an over-full expert are *dropped*, passed through on the residual or rerouted to a second choice. The capacity factor is a direct knob: set it low

and you save memory and communication but drop more tokens, losing compute and adding training noise; set it high and you waste padding on under-full experts.

How the layer reached this form

Sparse experts predate the transformer era as a general idea, but the line that matters here begins with Shazeer et al. (2017) (Shazeer et al. 2017), who introduced top- k gating and the auxiliary balancing loss inside an LSTM language model. GShard (Lepikhin et al., 2020) (Lepikhin et al. 2020) carried the layer into the transformer and into automatic sharding across devices. Switch Transformers (Fedus et al., 2021) (Fedus et al. 2021) pushed to the cheapest possible router, $k = 1$, and to trillion-parameter scale, showing top-1 routing could train stably with the right tricks. GLaM (Du et al., 2022) (Du et al. 2022) made the efficiency case at scale, and Mixtral (Jiang et al., 2024) (Jiang et al. 2024) brought a strong open $k = 2$ model into wide use.

Two threads then refined the layer. On routing, expert-choice (Zhou et al., 2022) (Zhou et al. 2022) inverted the question: instead of each token picking its experts, each expert picks its tokens, which makes per-expert load exact by construction and removes dropping, at the cost of some tokens getting no expert at all. BASE Layers (Lewis et al., 2021) (Lewis et al. 2021) framed routing as a balanced assignment problem outright. On specialization, DeepSeekMoE (Dai et al., 2024) (Dai et al. 2024) introduced fine-grained and shared experts, and DeepSeek-V3 (2024) (DeepSeek-AI 2024) replaced the auxiliary loss with bias-based balancing at frontier scale, with the standalone method documented separately by Wang et al. (2024) (Wang et al. 2024).

A parallel thread made MoE practical without paying for a from-scratch run. Sparse upcycling (Komatsuzaki et al., 2022) (Komatsuzaki et al. 2022) initializes each expert as a copy of a trained dense feed-forward block, adds a fresh router, and continues training; the experts diverge as routing specializes them. It buys strong quality for a fraction of from-scratch compute, with the failure mode that experts can stay near-identical if routing pressure is too weak, leaving MoE's cost without its benefit.

Stabilizing the router in practice

The router is the part to get right, and its instabilities have a specific cause. The top- k selection is an argmax, and argmax is discontinuous: a small change in scores flips a token's expert, making the loss landscape jagged and spike-prone, worse in low precision. The standard mitigations are narrow and worth naming. The router z-loss from ST-MoE (Zoph et al., 2022) (Zoph et al. 2022) penalizes large router logits to keep the gate numerically well-behaved; note this is distinct from the output-logit z-loss in the stability kit of Chapter 3. Keeping the router in fp32 under a bf16 or fp8 body costs little and removes a class of spikes. And early routing decisions can lock in before experts have differentiated, so a run is worth watching for that in its first phase. MoE loss curves are spikier than dense curves at the same scale; an unguarded router, with no z-loss and bf16 logits, can spike and fail to recover days into a run.

The failure modes are worth naming because each bites at a different time. *Routing collapse* funnels most tokens to a few experts and leaves the rest dead; it is exactly what load balancing exists to prevent, and it returns if balance is mis-weighted or disabled. *Load imbalance and token dropping*, short of collapse, overflows some experts' capacity and silently discards their tokens' compute while adding noise, and it stalls the slowest expert's device, a systems cost paid in Chapter 8. *Expert under-training* leaves rarely-chosen experts too few tokens to specialize, the exact risk mode of upcycling when routing pressure is too weak.

When the bet pays off

- **Sparse versus dense.** MoE buys more capacity per training FLOP and per active FLOP, but pays in memory, since all experts are stored and sharded, in communication, since expert parallelism needs an all-to-all (Chapter 8), in fragility, and in a harder serving story.

At small scale or under tight memory, dense wins. The crossover favors MoE as total scale grows.

- **top-1 versus top-k.** A router with $k = 1$ is cheapest and simplest but more brittle and less expressive (Fedus et al. 2021); $k = 2$ is the common sweet spot (Jiang et al. 2024); higher k approaches dense cost and erodes the advantage.
- **Capacity factor.** Lower is cheaper but drops more tokens and adds noise; higher wastes padding. There is no single right value, and it is often raised at evaluation and inference, where dropping a token is unacceptable.

Breaking the context wall: state-space models

A fixed state instead of an addressable past

State-space models answer the other wall, attention's quadratic. An SSM maintains a fixed-size hidden state and updates it token by token through a linear recurrence, then reads an output from it:

$$h_t = A h_{t-1} + B x_t, \quad y_t = C h_t$$

Because the state is fixed in size, the cost is linear in sequence length and there is no growing key-value cache to serve, only a constant-size state. The price is that all of a token's past must be summarized into that fixed state, where attention keeps every past token addressable. The design question is whether a learned, input-dependent recurrence can compress the past well enough to compete. Mamba's contribution was to make the recurrence selective, letting A , B , and C depend on the input so the model can choose what to keep and what to forget, recovering much of what content-based attention does while staying linear (Gu and Dao 2023).

From S4 to Mamba

The state-space line is younger as a serious attention competitor. The structured state-space model S4 (Gu et al., 2021) (Gu et al. 2021) showed a carefully parametrized linear recurrence could handle very long sequences, and Mamba (Gu and Dao, 2023) (Gu and Dao 2023) made the recurrence selective and hardware-efficient, bringing it within reach of language modeling at scale. Mamba-2 (Dao and Gu, 2024) (Dao and Gu 2024) tied state-space models back to attention through a duality that let them reuse attention's hardware.

The implementation reality is younger and thinner than the dense path. The recurrence needs a hardware-efficient scan to be fast, and the tooling around these models is less mature than the dense-transformer path, which is itself a reason the frontier has been slow to leave attention behind.

Recall is the catch

Attention keeps every past token exactly addressable and excels at precise recall, at quadratic cost and a growing key-value cache. An SSM is linear in length with a constant-size state, but must compress the past, and tends to lag attention on tasks that need exact long-range lookup. Figure 7.3 is the cost side of that trade: the two curves part ways as context grows, which is what makes the recall penalty worth paying at long context. Hybrids, the subject of the next section, exist precisely because neither pure form dominates.

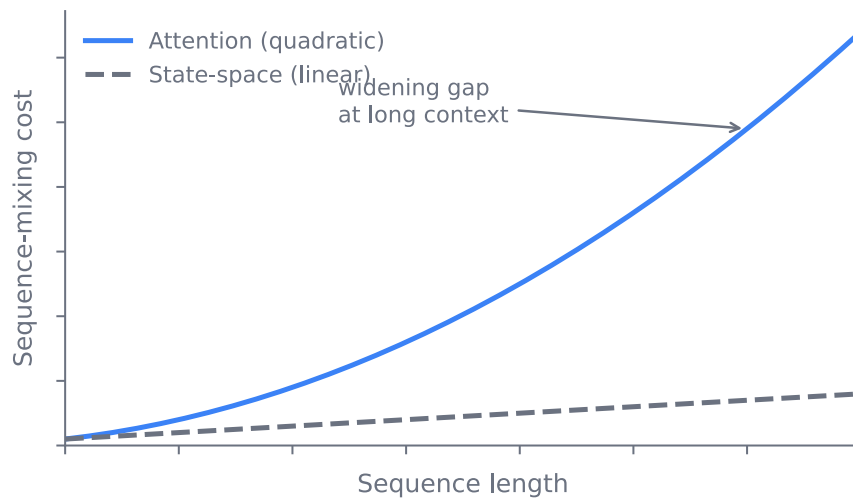


Figure 7.3. Schematic of sequence-mixing cost as context length grows. Attention's per-step cost and its key-value cache grow with the square of the sequence length, while a state-space recurrence with a fixed-size state grows only linearly. Idealized curves, not measured data, after Gu and Dao (2023).

WHAT'S CONTESTED

Whether state-space and hybrid models should displace the dense attention transformer is not settled. The case for them is the linear cost and the constant-size state, which matter most at long context and at serving time. The case against is recall: pure SSMs still trail attention on tasks that need exact retrieval from far back in the sequence, which is why every strong long-context system so far keeps some full-attention layers rather than going pure-recurrent. How few attention layers a stack can keep, and whether a future selective recurrence closes the recall gap entirely, are open questions. Treat "attention is replaceable" as a live hypothesis the frontier is testing, not a settled result.

CONSTRAINT ARROW

The serving layer reaches up and sets sequence-mixing design. The key-value cache footprint and the quadratic cost of attention at long context (Chapter 16) are what make a linear-cost, constant-state recurrence worth its recall penalty in the first place. A choice that looks like pure architecture, attention versus state-space, is driven by what inference will cost over the model's life, the same arrow that justified over-training in Chapter 3.

Neither alone: the hybrid stack

In practice the frontier does not choose one or the other. *Hybrid* stacks interleave a minority of full-attention layers with a majority of SSM or linear-recurrence layers, so a few global-mixing layers preserve exact recall while the linear layers carry the bulk of the sequence cheaply. MoE and SSM are also orthogonal: a hybrid can put its feed-forward blocks behind an MoE router and its sequence-mixing behind a state-space recurrence at the same time. Jamba (Lieber et al., 2024) (Lieber et al. 2024) interleaved Mamba and attention layers and added MoE on top, demonstrating that the three ideas compose into one stack. Figure 7.4 shows the three ideas composing: the sequence-mixing slot alternates SSM and attention, while the feed-forward slot is an MoE layer.

Figure 7.4. A hybrid stack composes all three ideas. Most layers mix the sequence with a linear SSM recurrence, a minority use full attention for exact recall, and the feed-forward slot is an MoE layer. After Lieber et al., 2024.

The few attention layers in a hybrid still carry a key-value cache, so a hybrid does not escape attention's costs entirely; it pays them on a minority of layers instead of all of them. Figure 7.5 lays out the threads of this chapter as a genealogy: a routing line, a specialization line, the upcycling shortcut, and the separate state-space line, all converging on the hybrid stack.

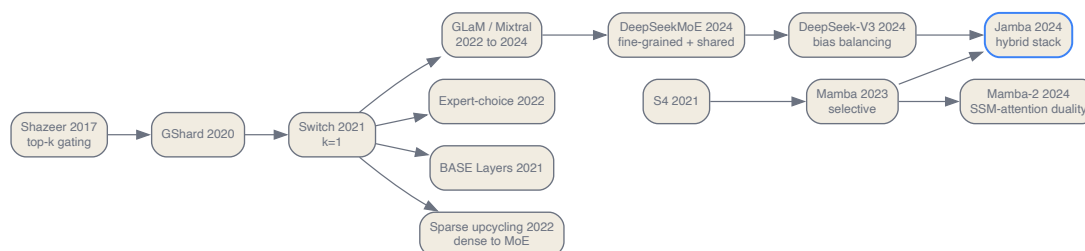


Figure 7.5. Genealogy of the ideas in this chapter. A routing thread and a specialization thread refine the MoE layer, sparse upcycling is a training shortcut into it, and a separate state-space line converges with attention and MoE at the hybrid stack.

Two trade-offs span the whole chapter and frame the rest of the book's view of these architectures.

- **Attention versus state-space.** Attention keeps every past token exactly addressable and excels at precise recall, at quadratic cost and a growing key-value cache. An SSM is linear in length with a constant-size state, but must compress the past, and tends to lag attention on tasks that need exact long-range lookup. Hybrids exist precisely because neither pure form dominates (Lieber et al. 2024).
- **Train and serve complexity.** Every choice here adds operational surface: a router to tune, balance to monitor, expert placement to lay out, and a memory and communication profile that Chapter 8 must absorb. The quality win is real but never free engineering.

Further reading

- Shazeer et al., "Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer," 2017 (ICLR). Origin of top-k gating and the auxiliary balancing loss. [arXiv:1701.06538](https://arxiv.org/abs/1701.06538)
- Lepikhin et al., "GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding," 2020. [arXiv:2006.16668](https://arxiv.org/abs/2006.16668)
- Fedus et al., "Switch Transformers," 2021 (JMLR 2022). Top-1 routing at trillion-param scale. [arXiv:2101.03961](https://arxiv.org/abs/2101.03961)

- Du et al., "GLaM: Efficient Scaling of Language Models with Mixture-of-Experts," 2022 (ICML). [arXiv:2112.06905](#)
- Zhou et al., "Mixture-of-Experts with Expert Choice Routing," 2022 (NeurIPS). Balance by construction. [arXiv:2202.09368](#)
- Zoph et al., "ST-MoE: Designing Stable and Transferable Sparse Expert Models," 2022. Router z-loss, stability. [arXiv:2202.08906](#)
- Komatsuzaki et al., "Sparse Upcycling: Training Mixture-of-Experts from Dense Checkpoints," 2022 (ICLR 2023). [arXiv:2212.05055](#)
- Jiang et al., "Mixtral of Experts," 2024. [arXiv:2401.04088](#)
- Dai et al., "DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models," 2024. Fine-grained + shared experts. [arXiv:2401.06066](#)
- DeepSeek-AI, "DeepSeek-V3 Technical Report," 2024. Auxiliary-loss-free (bias-based) load balancing at scale. [arXiv:2412.19437](#)
- Lewis et al., "BASE Layers: Simplifying Training of Large, Sparse Models," 2021 (ICML). Routing as assignment, for contrast. [arXiv:2103.16716](#)
- Wang et al., "Auxiliary-Loss-Free Load Balancing Strategy for Mixture-of-Experts," 2024. The standalone loss-free balancing method, distinct from the DeepSeek-V3 report. [arXiv:2408.15664](#)
- Gu et al., "Efficiently Modeling Long Sequences with Structured State Spaces" (S4), 2021. [arXiv:2111.00396](#)
- Gu and Dao, "Mamba: Linear-Time Sequence Modeling with Selective State Spaces," 2023. [arXiv:2312.00752](#)
- Dao and Gu, "Transformers are SSMs: Generalized Models and Efficient Algorithms Through Structured State Space Duality" (Mamba-2), 2024. [arXiv:2405.21060](#)
- Lieber et al., "Jamba: A Hybrid Transformer-Mamba Language Model," 2024. [arXiv:2403.19887](#)

8. Training at Scale: Stability and Distributed Parallelism

A frontier model does not fit on one accelerator, and the run that produces it spans thousands of them for weeks. The moment the work is split across that many devices, three resources start to fight: the memory each shard must hold, the communication that keeps the shards consistent, and the failures that a fleet of that size produces as its steady state. Everything in this chapter is a way of spending those three against each other so that achieved throughput stays high and a dead GPU costs minutes, not days. This chapter takes the machinery apart along that seam. It opens on the three forces, then works through the cuts that split the model and what each one costs to communicate, the bits the arithmetic runs in, and the overlap that hides the traffic, with each tool's origin attached to the mechanism it explains. One number, model FLOPs utilization, runs through the whole teardown as the scoreboard that says whether the systems work is done.

Three forces that fight

Two facts force everything that follows. A frontier model's parameters, gradients, and optimizer state do not fit in one device's memory, and the arithmetic to train it does not finish on one device in any acceptable time. So the work is split across thousands of accelerators, and the moment it is split, three constraints start to fight.

The first is memory: weights, the Adam moments whose state dominates (see Chapter 3), activations, and gradients must each find a home, and peak activation memory grows with sequence length. The second is communication: every split introduces traffic to keep the shards consistent, and that traffic competes with compute for the same time budget. The third is failure: at thousands of nodes for weeks, a dead GPU or NIC is the steady state, not the exception, and a single one kills the whole collective. The job of the training-systems layer is to spend memory, communication, and recovery against each other so that achieved throughput stays high and a failure costs minutes, not days. No single way of splitting the work resolves all three at once, which is why the design offers several orthogonal cuts and composes them.

Four cuts, and what each one costs

Each axis answers a different scaling problem, shards a different resource, and arrived in the literature when the previous generation hit a wall. Read in that order they are less a menu than a record of how the toolkit was assembled, one cut at a time.

Tensor parallelism (TP) came first. It shards individual matmuls inside a layer, splitting attention heads and the feed-forward columns and rows across devices. Megatron-LM showed how to do this with only two all-reduces per layer, which made a multi-billion-parameter model trainable once it no longer fit on one device (Shoeybi et al. 2019). The cost is that it is communication-heavy: each TP layer needs an all-reduce (or a reduce-scatter and all-gather pair) on the critical path of every forward and backward step. That cost is only affordable over the fastest interconnect, so TP stays inside a node.

Pipeline parallelism (PP) arrived in parallel and cuts a different resource. It splits the layer stack into stages placed on different nodes and streams micro-batches through them, so stage two works on micro-batch one while stage one works on micro-batch two. GPipe established the micro-batching that keeps the stages busy (Huang et al. 2019), and PipeDream generalized the schedule to cut the bubble (Narayanan et al. 2019). PP is bandwidth-cheap, moving only the activations at stage boundaries, but it pays a pipeline bubble: the fill and drain at the ends of each step where some stages sit idle. Figure 8.1 shows where that idle time lives.

time	1	2	3	4	5	6	7
stage 1	mb1	mb2	mb3	mb4	idle	idle	idle
stage 2	idle	mb1	mb2	mb3	mb4	idle	idle
stage 3	idle	idle	mb1	mb2	mb3	mb4	idle
stage 4	idle	idle	idle	mb1	mb2	mb3	mb4

Figure 8.1. Pipeline bubble over four stages and four micro-batches: the fill at the start and the drain at the end leave stages idle. After Huang et al., 2019.

Data parallelism (DP) replicates the model and shards the batch across replicas; each replica computes gradients on its slice and an all-reduce averages them. Plain DP does not save memory, since every replica holds a full copy. ZeRO observed that the optimizer state, gradients, and parameters in standard DP are redundantly replicated, and sharded them across the DP group in three progressive stages (Rajbhandari et al. 2019); ZeRO-Infinity extended the idea to offload onto CPU and NVMe (Rajbhandari et al. 2021), and PyTorch FSDP made sharded DP a native, ergonomic primitive (Zhao et al. 2023). Sharding drops per-device memory without changing the math: a layer's parameters are all-gathered just before they are needed and freed right after. This is the sharding Chapter 3 points to when it notes that the optimizer state is the dominant memory term.

Sequence and context parallelism (SP/CP) shard along the sequence dimension, the axis that activation memory lives on. SP splits the parts of a layer that TP leaves replicated, the norms and dropouts, along the sequence to cut activation memory, introduced alongside selective recomputation that redoes only the cheap-to-redo, memory-heavy operations rather than the whole layer (Korthikanti et al. 2022). CP, of which ring attention is the canonical form, shards the attention computation itself so a context too long for one device can be processed in blocks that pass key and value tiles around a ring, letting the context grow nearly without bound (Liu et al. 2023).

For each axis the design question is the same: what does it shard, what does it cost in communication, and which tier of the interconnect can it afford to use. Figure 8.2 lays the answers side by side.

Axis	Shards	Collective	Interconnect tier
DP / ZeRO	batch, optimizer state	all-reduce, reduce-scatter, all-gather	across nodes
TP	matmuls in a layer	all-reduce per layer	inside node, NVLink
PP	layer-stack stages	point-to-point activations	across nodes
SP / CP	sequence dimension	all-gather, ring exchange	with TP or across nodes
EP	experts	all-to-all dispatch and combine	own all-to-all group

Figure 8.2. The four parallelism axes plus expert parallelism, each by what it shards, the collective it runs, and the interconnect tier it can afford.

Mixture-of-experts layers add a fifth cut. The experts are sharded across devices, and each token is dispatched to its chosen experts by an all-to-all collective, then the results are combined back by a second all-to-all, the dispatch pattern traced to GShard (Lepikhin et al.

2020). The systems layer owns only this execution: the two all-to-all exchanges per MoE layer, the load-imbalance stragglers when a hot expert draws more than its share of tokens, the capacity padding that bounds per-expert work, and the overlap of dispatch and combine with expert compute. The routing decision and the load-balancing loss that *create* this traffic pattern belong to [Chapter 7](#).

Paying in bits: the precision axis

The arithmetic does not need to run in fp32. Mixed precision keeps the forward and backward matmuls and the activations in bf16, while holding an fp32 master copy of the weights and fp32 optimizer state so that small updates do not vanish when added to a large weight, the recipe established by Micikevicius et al. (Micikevicius et al. 2017). bf16's wide exponent is what makes this safe: it retired the loss scaling that fp16 needed to keep gradients off the floor of its narrow range.

fp8 (the E4M3 and E5M2 formats) pushes the matmuls further on Hopper-class and newer hardware, roughly doubling matmul throughput and halving the memory the operands occupy; the formats paper defined E4M3 and E5M2 for the generation that made fp8 matmuls native (Micikevicius et al. 2022). The price is range: fp8 has so few exponent bits that the operands must be rescaled per tensor, or more finely, to keep values inside the representable band. It is applied selectively, the matmuls in fp8 while the numerically sensitive operations, the normalizations, residual adds, and the attention softmax, stay in higher precision. Pushed too far, fp8 does not crash; it quietly costs quality, which is what makes it hard to use well.

Figure 8.3 shows why the exponent bits matter. bf16 keeps fp32's eight exponent bits, so it spans the same dynamic range and never needed fp16's loss scaling; fp8 spends its few bits on a tradeoff between range (E5M2) and precision (E4M3).

format	sign	exponent	mantissa
fp32	1	8 bits	23 bits
fp16	1	5 bits	10 bits
bf16	1	8 bits	7 bits
fp8 E4M3	1	4 bits	3 bits
fp8 E5M2	1	5 bits	2 bits

Figure 8.3. Bit layouts of the training precisions. Sign, exponent, and mantissa fields drawn to width. bf16 inherits fp32's eight exponent bits, which is why it spans the same range without loss scaling.

Hiding the traffic

Every cut above generates traffic, and that traffic runs through a small set of collectives: all-reduce, reduce-scatter, all-gather, and all-to-all. NCCL (RCCL on AMD) implements them and maps each onto the topology, choosing a ring or a tree algorithm and routing over NVLink within a node or the network between nodes (NVIDIA 2024).

The collectives are pure overhead unless they run while the device is doing something else, so overlap is the central throughput trick. FSDP prefetches the next layer's parameters with an all-gather while the current layer computes (Zhao et al. 2023). The gradient reduce-scatter

overlaps the backward pass. PP communication overlaps stage compute. EP all-to-all overlaps expert matmuls. Whatever cannot be hidden is exposed communication, and it shows up directly as lost throughput.

Model FLOPs utilization is the scoreboard that says whether the overlap worked. It is achieved FLOPs over peak hardware FLOPs, counting only the FLOPs the model mathematically requires. It is distinct from hardware FLOPs utilization (HFU), which also counts the redundant FLOPs of activation recomputation, so HFU exceeds MFU whenever recomputation is on. MFU is the single number that says the systems work is done, and its budget is a sum of losses: exposed communication, the pipeline bubble, recomputation, and unfused kernels each take a slice. A low MFU is diagnosed from a per-step timeline, not from the loss curve. It is the thread that ties the whole teardown together, since every cut and every precision choice ends up scored here.

WHAT'S CONTESTED

How far fp8 can be pushed in frontier pre-training is unsettled. bf16 is the agreed safe default, and fp8 plainly buys throughput and memory on Hopper-class hardware (Micikevicius et al. 2022). The disagreement is over the boundary: which operations can move to fp8, how fine the scaling must be, and how many trillions of tokens a run can sustain in fp8 before the narrowed range costs final quality. The failure mode is what makes this hard to settle empirically: fp8 pushed too far does not crash, it quietly degrades the model, so the cost is invisible until a late evaluation. Treat the fp8 boundary as a per-operation, per-recipe choice that must be validated, not a setting you can copy.

CONSTRAINT ARROW

The interconnect dictates the parallelism layout. Tensor parallelism needs an all-reduce on the critical path of every layer, which is only affordable over the fastest links, so the size of the NVLink domain sets the maximum TP group; beyond it, TP traffic crosses the network and stalls (Shoeybi et al. 2019). Pipeline and data parallelism, which move less and can hide it, are the axes that span nodes, and expert parallelism gets its own all-to-all group. The bandwidth hierarchy of Chapter 30 is therefore upstream of every mesh decision in this chapter: a lower layer's wires decide which split an upper layer may use where.

Turning the dials

Each axis spends a different resource, and the layout that wins is the one that hides the most communication under compute for a specific model shape on a specific interconnect. By 2021 the axes were being combined deliberately: the PTD-P work mapped data, tensor, and pipeline parallelism onto a single GPU cluster as a composed 3D layout, establishing the ordering that frontier runs still follow (Narayanan et al. 2021). The dials below are what an engineer actually turns inside that layout.

- **Parallelism layout, and its non-portability.** TP cuts per-device memory and latency but burns intra-node bandwidth, so it caps at the NVLink domain. PP is bandwidth-cheap but pays a pipeline bubble, mitigated by more micro-batches and interleaved schedules (Narayanan et al. 2019), with diminishing returns once the bubble is already small (Figure 8.4). DP and ZeRO scale the batch, but the optimizer-state shard and the all-gather traffic grow with the DP degree (Rajbhandari et al. 2019). The right layout is not portable: change the model size, the sequence length, or the cluster, and the optimum moves.

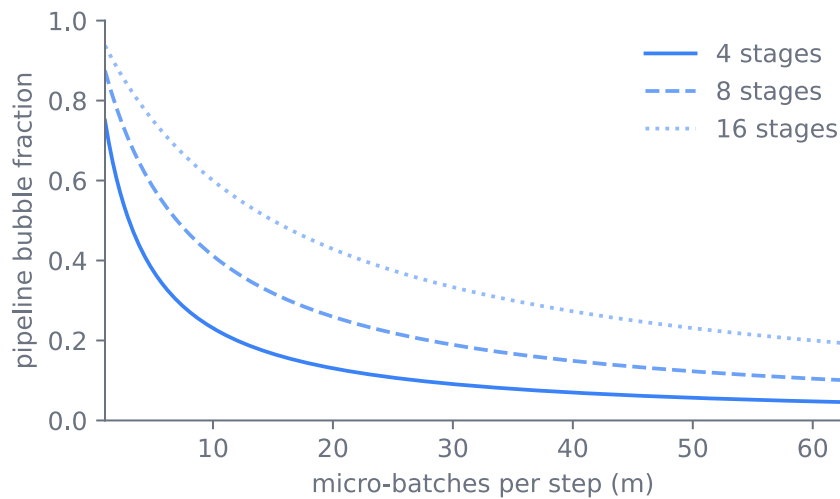


Figure 8.4. Schematic illustration of how the pipeline bubble fraction shrinks as the number of micro-batches per step grows, for several stage counts, after Huang et al. (2019) and Narayanan et al. (2021). Idealized fraction $(p-1)/(m+p-1)$; not measured data. The gain is steep at first and flattens, so beyond a handful of micro-batches more buys little.

Run the exact formula and watch the diminishing returns: change the stage counts or the range of micro-batches and see where the curve flattens.

```
import numpy as np
import matplotlib.pyplot as plt

m = np.arange(1, 33) # micro-batches per step
for p in [4, 8, 16, 32]: # pipeline stages
    bubble = (p - 1) / (m + p - 1)
    plt.plot(m, bubble, marker="o", markersize=3, label=f"p={p} stages")

print("p=8 stages:", {int(mm): round((8 - 1) / (mm + 8 - 1), 3) for mm in m})

plt.xlabel("micro-batches per step (m)")
plt.ylabel("pipeline bubble fraction")
plt.title("Bubble fraction (p-1)/(m+p-1)")
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```

- **Memory versus compute.** Activation and gradient checkpointing trade extra forward FLOPs in the backward pass for lower peak activation memory. Selective recomputation redoes only the cheap, memory-heavy operations (Korthikanti et al. 2022). More recomputation lets you fit a bigger model or a longer sequence, at the cost of MFU, which is exactly why HFU exceeds MFU when recompute is on.
- **Precision versus stability.** bf16 is the safe default, and fp32 master weights are non-negotiable for the optimizer (Micikevicius et al. 2017). fp8 buys throughput and memory but narrows range, so it is applied per-tensor with scaling and kept away from the sensitive

operations (Micikevicius et al. 2022). Pushed too far it produces silent quality loss, not a crash.

- **Checkpoint cadence.** Frequent checkpoints shrink the work lost per failure but tax storage bandwidth and can stall the run. Asynchronous and sharded checkpointing is what makes a short cadence affordable, and the optimum depends on cluster mean time between failures and write cost (Mohan et al. 2021).

Laying out a real run, and how it breaks

Composing the axes

A frontier run maps the device mesh as a product of axes, conventionally (DP x TP x PP), adding EP for MoE and layering SP on top of TP. The ordering rule of thumb follows directly from the constraint arrow and from the PTD-P result: put TP inside a node under NVLink, run PP and DP across nodes where their cheaper communication can hide, and give EP its own all-to-all group (Narayanan et al. 2021). The layout is co-designed with the model shape from Chapter 6 and Chapter 7, and it shifts with model size, sequence length, and interconnect.

Figure 8.5 sketches the conventional placement.

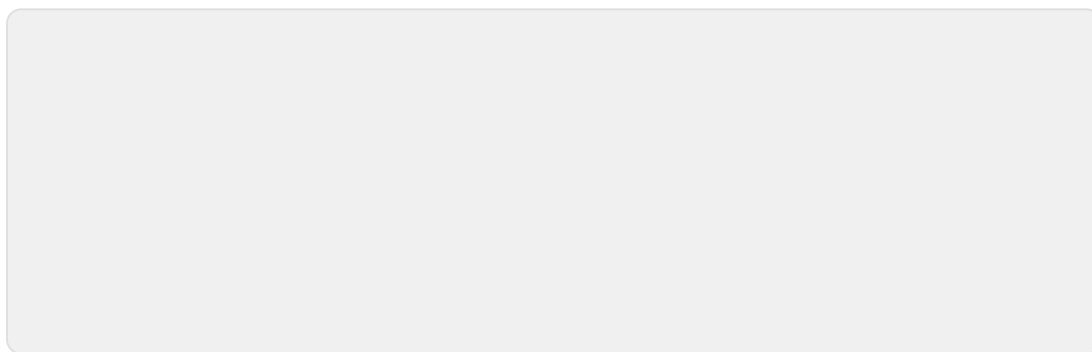


Figure 8.5. The conventional device-mesh placement: communication-heavy TP and SP stay inside a node under NVLink, while the cheaper-to-hide DP, PP, and EP span nodes.

Frameworks

The frameworks differ mostly in which layout and which hardware each makes ergonomic. Megatron-LM and Megatron-Core carry TP, PP, SP, and EP on the NVIDIA stack (Shoeybi et al. 2019). DeepSpeed packages ZeRO and offload (Rajbhandari et al. 2019). PyTorch FSDP is native sharded DP (Zhao et al. 2023). JAX and XLA express sharding as GSPMD-style annotations on TPU, where the JAX `pjit` interface drives the partitioner (Xu et al. 2021). In practice a frontier stack combines pieces, for example Megatron-Core for model parallelism plus a custom data and checkpoint plane.

Fault tolerance

At thousands of nodes for weeks, recovery is a first-class subsystem. Checkpointing cadence is set by the write cost against the work lost per failure; asynchronous and sharded checkpointing, as in CheckFreq, keep the write off the critical path so a short cadence stays affordable (Mohan et al. 2021). Elastic restart detects a dead node, fences it, and resumes from the last checkpoint, ideally onto spare capacity without a full restart. Stragglers and silent data corruption are harder than clean crashes, because the run keeps going while it is slow or subtly wrong.

One fault-tolerance concern reaches into the data plane: a restart must resume the exact same data order, which means the sampler and shuffle state are part of the checkpoint. Without that, a resume re-feeds or skips data and breaks the mixture contract from [Chapter 4](#), surfacing as an unexplained loss discontinuity at every restart. The storage fabric that streams the corpus and the cluster orchestration that schedules the run live in [Chapter 31](#); the resumability contract is the piece that stays here.

Failure modes

The symptoms cluster. A low MFU is the catch-all, traced by a per-step profile to exposed communication, an oversized bubble, too much recomputation, or unfused kernels. A communication bottleneck means the network is the limiter: TP spilling past the NVLink domain, a DP all-gather that cannot hide under the backward pass, or MoE all-to-all stalling on a hot expert. An out-of-memory error means the wrong layout for the budget: activation memory at long sequence length, optimizer state under-sharded, or a fragmentation cliff that appears only at a certain micro-batch count. A node failure with no fast detection, fencing, and recent checkpoint rolls the run back to its last save, and the gap is pure waste.

The fastest attention kernel does not change this accounting: FlashAttention keeps attention exact and makes the score matrix affordable by never writing it to high-bandwidth memory, a kernel-level win that sits underneath these parallelism axes rather than replacing one of them ([Dao et al. 2022](#)). [Figure 8.6](#) contrasts the two scaling regimes: a materialized score matrix whose memory grows quadratically with sequence length against the linear growth of a kernel that streams the scores.

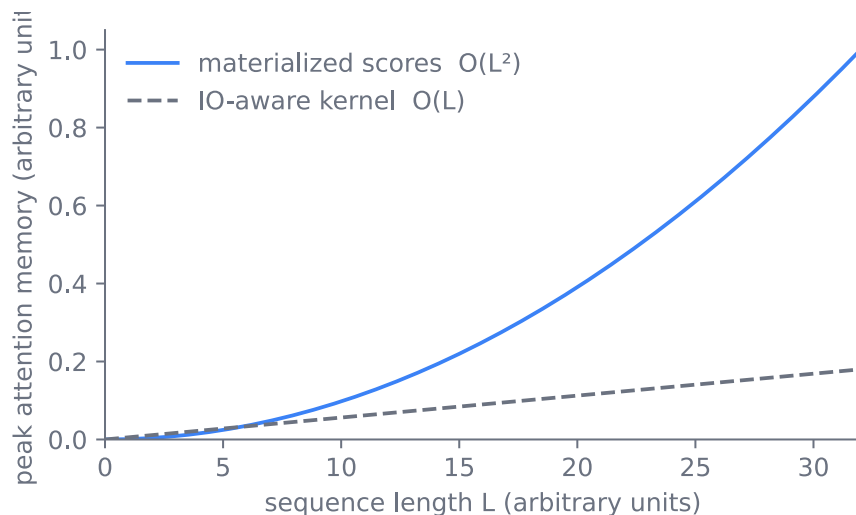


Figure 8.6. Schematic illustration of peak attention memory versus sequence length, after [Dao et al. \(2022\)](#). A materialized score matrix grows as $O(L^2)$ while an IO-aware kernel that never writes the scores to HBM grows as $O(L)$. Idealized, arbitrary units; not measured data.

Drag the exponent to feel the gap between the two regimes: at exponent 2 the memory is the materialized $O(L^2)$ score matrix, at exponent 1 it is the FlashAttention $O(L)$ stream.

Peak attention memory versus sequence length: exponent 2 is the materialized $O(L^2)$ score matrix, exponent 1 is the FlashAttention $O(L)$ stream.

Recipe-level stability, the z-loss and QK-norm and loss-spike recovery that keep the optimizer from diverging, belongs to [Chapter 3](#). The stability this chapter owns is numerical, set by the precision choice, and run-level, set by fault tolerance.

Further reading

- Shoeybi et al., "Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism," 2019. [arXiv:1909.08053](#)
- Rajbhandari et al., "ZeRO: Memory Optimizations Toward Training Trillion Parameter Models," 2019. [arXiv:1910.02054](#)
- Narayanan et al., "Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM," 2021 (SC'21; PTD-P 3D parallelism). [arXiv:2104.04473](#)
- Korthikanti et al., "Reducing Activation Recomputation in Large Transformer Models," 2022 (sequence parallelism + selective recomputation). [arXiv:2205.05198](#)
- Huang et al., "GPipe: Efficient Training of Giant Neural Networks using Pipeline Parallelism," 2019 (NeurIPS). [arXiv:1811.06965](#)
- Narayanan et al., "PipeDream: Generalized Pipeline Parallelism for DNN Training," 2019 (SOSP). [doi:10.1145/3341301.3359646](#)
- Zhao et al., "PyTorch FSDP: Experiences on Scaling Fully Sharded Data Parallel," 2023 (VLDB). [arXiv:2304.11277](#)
- Rajbhandari et al., "ZeRO-Infinity: Breaking the GPU Memory Wall for Extreme Scale Deep Learning," 2021. [arXiv:2104.07857](#)
- Micikevicius et al., "Mixed Precision Training," 2017. [arXiv:1710.03740](#)
- Micikevicius et al., "FP8 Formats for Deep Learning," 2022. [arXiv:2209.05433](#)
- Dao et al., "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness," 2022 (kernel-level IO-awareness; also in 03). [arXiv:2205.14135](#)
- Liu et al., "Ring Attention with Blockwise Transformers for Near-Infinite Context," 2023 (context-parallel attention). [arXiv:2310.01889](#)
- Lepikhin et al., "GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding," 2020 (expert sharding + all-to-all; also in 04). [arXiv:2006.16668](#)
- NCCL (NVIDIA Collective Communications Library): optimized inter-GPU collective primitives (all-reduce, all-gather, reduce-scatter, all-to-all) over NVLink/PCIe/InfiniBand. Engineering library, not a single canonical paper. [NVIDIA/nccl](#)
- Mohan et al., "CheckFreq: Frequent, Fine-Grained DNN Checkpointing," 2021 (USENIX FAST'21; asynchronous, low-overhead checkpointing). [usenix.org/conference/fast21/presentation/mohan](#)
- Xu et al., "GSPMD: General and Scalable Parallelization for ML Computation Graphs," 2021 (XLA/TPU sharding annotations underlying JAX `jit`). [arXiv:2105.04663](#)

9. Supervised Fine-Tuning and PEFT

Hand a freshly pre-trained model the prompt "What is the capital of France?" and it may answer with three more exam questions. It has read enough of the web to know that Paris is the capital, but it has learned to continue text, not to respond to it. There is no turn-taking, no stable persona, no refusal, no fixed output format. The knowledge is present; the behavior is not.

Closing that gap is the first and cheapest step in turning a base model into an assistant, and it is the subject of this chapter. The opening from [Chapter 3](#) is concrete: pre-training buys knowledge under the wrong objective, and we now have to install assistant behavior without paying for a second pre-training run.

That constraint shapes everything. The behavior change must be large in effect but small in data, because high-quality instruction data is written or curated by people and does not exist at pre-training scale. And it must be cheap in compute and memory, because the people who adapt a model are rarely the people who trained it: a downstream team has one or a few accelerators, not a cluster, and wants many task-specific variants, not one. A full fine-tune that copies and updates every weight defeats both goals at once.

One idea carries the whole chapter through that constraint. **Adaptation is a small, low-rank, additive, composable change on top of a frozen base.** Each word in that sentence pays off as a separate consequence, and the three promises in this chapter are exactly those consequences: a few thousand examples reshape behavior because the change is *small*; a rank-16 adapter stands in for a full fine-tune because the change is *low-rank* and *additive*; and two fine-tunes can be added together because the change is *composable*. The sections that follow take the three in turn.

Why a few thousand examples reshape behavior

The core method is supervised fine-tuning (SFT), also called instruction tuning, the recipe that InstructGPT made the standard first step of alignment ([Ouyang et al. 2022](#)). Continue the same next-token objective from [Chapter 3](#), but on a curated set of (prompt, response) pairs instead of raw web text, and mask the loss so only the response tokens are scored. The model still learns by predicting the next token; what changes is the distribution it predicts over. For a pair with prompt x and response y ,

$$\mathcal{L}_{\text{SFT}} = - \sum_t \log p_{\theta}(y_t \mid x, y_{<t}),$$

the sum running over response positions only. Masking the prompt keeps the model from spending capacity learning to generate user turns, and teaches the asymmetry of the assistant role: read the prompt, produce the answer.

What makes the prompt and response distinguishable is the chat template. A base model sees one undifferentiated stream of tokens. An assistant needs to know where the user's turn ends and its own begins, so SFT wraps each turn in role markers, special tokens like `<|user|>` and `<|assistant|>` that the tokenizer reserves and the template inserts. The template is a contract: the same markers must be used at inference, or the model is served off the distribution it was tuned on. This is the smallest example of a recurring truth in this part, that adaptation is as much about format as about weights. [Figure 9.1](#) traces a single example from template to scored loss.

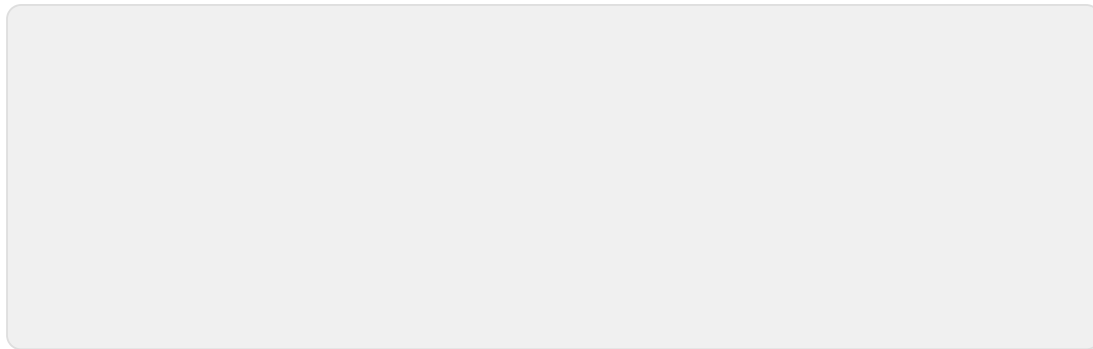


Figure 9.1. An SFT example flows from chat template through the token stream to a masked loss. Role markers split the stream, and only response tokens contribute gradients.

The surprising part is how little data this takes, and this is where *small* earns its place in the unifying claim. The superficial-alignment hypothesis, named by the LIMA paper, holds that a model's knowledge and capability are laid down almost entirely in pre-training, and that SFT mainly teaches it which subdistribution of formats and styles to speak in (Zhou et al. 2023). If that is right, then a few thousand carefully chosen examples should suffice, and LIMA showed exactly that: a 65B model tuned on 1,000 curated examples, without any reinforcement learning, was competitive with far more heavily aligned models (Zhou et al. 2023). The lesson reframes the whole step. SFT is eliciting and shaping latent capability, not adding it, which is why data quality and diversity dominate data quantity, and why a small noisy set can teach a confident wrong style as easily as a right one.

That reframing cuts both ways, and the cut is the trade-off SFT carries. With few examples, every example matters, and a handful of wrong facts or a single overused format teaches a defect the model will repeat confidently. Narrow SFT data bakes in a rigid template; this is the format-overfitting failure that over-refusal and always-bulleted answers are instances of.

WHAT'S CONTESTED

How far the superficial-alignment hypothesis holds is unsettled. The strong reading, that SFT only surfaces pre-trained capability and adds nothing, is contradicted by tasks where instruction tuning on enough data clearly teaches new skills and formats that a few hundred examples cannot. The honest position is that the hypothesis is a good description of broad instruction-following style and a poor one of narrow capability acquisition, and where the line falls depends on how far the target behavior sits from the base distribution. Treat "a thousand examples is enough" as a claim about style, not about every task.

Why a rank-16 adapter stands in for a full fine-tune

If adaptation is a small, low-dimensional change to behavior rather than a wholesale rewrite of knowledge, then the weight update that encodes it should be small too. This is the inference that makes parameter-efficient fine-tuning (PEFT) plausible. PEFT freezes the pre-trained weights and trains only a tiny set of new or selected parameters. To see why the field arrived at the dominant method, it helps to watch the per-task cost fall step by step.

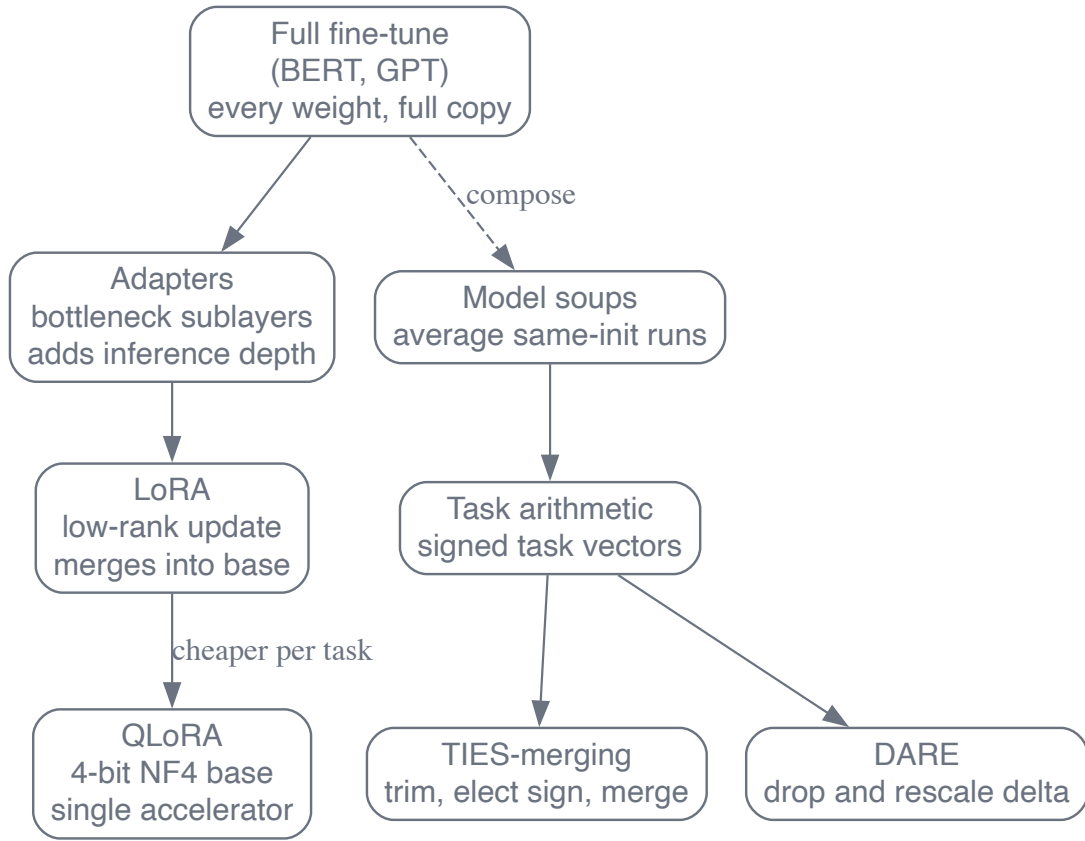


Figure 9.2. Two lineages of adaptation. The efficiency spine drives the per-task cost down from a full copy to a single card; the merging branch composes finished checkpoints with no further training.

The efficiency spine in Figure 9.2 runs from full fine-tuning to ever-cheaper approximations of it. The earliest transfer-learning recipe was to fine-tune all the weights of a pre-trained model on a downstream task, the approach that BERT and GPT made standard. It works but it is expensive and wasteful: every task needs a full copy of the model, and the update touches every weight to encode what is often a narrow behavioral change. The first answer was adapters, small bottleneck modules inserted between the frozen layers of a transformer, of which only the inserted parameters train (Houlsby et al. 2019). Adapters proved the principle that a tiny fraction of new parameters could recover most of full fine-tuning's quality, but they added depth, and therefore inference latency, to every forward pass (Houlsby et al. 2019).

LoRA removed that cost, and it is where *low-rank* and *additive* enter the claim. For a frozen weight matrix $W_0 \in \mathbb{R}^{d \times k}$, LoRA learns a low-rank update (Hu et al. 2021)

$$W = W_0 + \Delta W = W_0 + \frac{\alpha}{r} B A, \quad B \in \mathbb{R}^{d \times r}, \quad A \in \mathbb{R}^{r \times k},$$

with rank $r \ll \min(d, k)$, so A and B together hold $r(d + k)$ parameters instead of dk . Only A and B train; W_0 stays frozen. The hypothesis behind it is that the update a fine-tune wants to apply has low intrinsic rank, so a rank-8 or rank-16 factorization captures most of it (Hu et al. 2021). Because the adaptation is an additive update to existing matrices rather than a new sublayer, at inference the product BA folds back into W_0 , so a served LoRA model has zero added latency, unlike adapters that bolt on extra layers. That is why LoRA became the default PEFT method. Figure 9.3 shows the two parallel paths and what each holds.

Plug in your own dimensions and ranks to see how few parameters $r(d + k)$ is against the full dk , and how the fraction grows linearly with rank.

```

import numpy as np
d, k = 4096, 4096          # one weight matrix, e.g. an attention pr
full = d * k               # full fine-tune touches every weight
ranks = [4, 8, 16, 32, 64]
print(f"full fine-tune: {full:,} trainable params")
for r in ranks:
    lora = r * (d + k)      # A is r x k, B is d x r
    print(f"  rank {r:3d}: {lora:>9,} params  ({100*lora/full:5.2f}%")
import matplotlib.pyplot as plt
fracs = [100 * r * (d + k) / full for r in ranks]
plt.plot(ranks, fracs, "o-")
plt.xlabel("LoRA rank r"); plt.ylabel("trainable params (% of full)")
plt.title("Cost rises linearly with rank"); plt.grid(True); plt.show

```

Figure 9.3. LoRA adds a trainable low-rank branch beside the frozen weight. The input feeds both paths and their outputs sum. Only A and B carry gradients, so the trained parameter count drops from d times k to r times the sum of d and k .

QLoRA then attacked the remaining bottleneck, which by then was not the trainable parameters but the memory to hold the frozen base in the optimizer's address space. It quantizes the frozen base weights to 4-bit (an information-theoretically motivated NormalFloat format, NF4) and trains the LoRA adapters in higher precision on top, dequantizing each weight block only when it is needed for the forward and backward pass (Detrmers et al. 2023). Because the base never updates, its quantization error is fixed and tolerable; because the adapters are small, they stay in bf16. The result is that a 65B model fits the adaptation step into roughly the memory a single 48GB card provides (Detrmers et al. 2023), which is the difference between needing a cluster and needing a workstation. With QLoRA the adaptation of a frontier-scale open model moved from a multi-accelerator job to a single-card one, and that is largely why the open-model fine-tuning ecosystem exists in the form it does.

The cheapness is bought with fidelity, and the trades are worth naming precisely.

- **Full fine-tune versus LoRA.** A full fine-tune has the most capacity and is the safe choice when the adaptation is large or far from the base distribution. LoRA trades a small quality gap, usually negligible for instruction tuning and larger for tasks that need new knowledge, for a hundredfold cut in trainable parameters and the ability to keep one base with many swappable adapters. Rank is the knob: too low underfits the task, too high spends memory for no gain and starts to resemble a full fine-tune.

- **QLoRA's quantization error.** Holding the base in 4-bit introduces a fixed error in every frozen weight. For most instruction tuning this is below the noise floor, because the adapters learn around it, but it sets a capability ceiling a precision-sensitive task can hit. The trade is memory for a small, bounded accuracy risk.

Cost rises linearly with rank, but the quality it buys saturates: drag the slider to watch task quality climb steeply at first, then flatten once the rank exceeds the update's intrinsic dimension, which is why a sweet spot exists.

Task quality rises with LoRA rank and then saturates; too low underfits, too high spends memory for no gain, so the knob has a sweet spot.

Why two fine-tunes can be added together

Once adaptation is a small additive change, the changes compose, and *that* is the last word in the unifying claim. Model merging combines several fine-tuned checkpoints into one set of weights with no further training. The simplest form is weight averaging. Task arithmetic sharpens it: define a task vector $\tau_i = \theta_{\text{ft},i} - \theta_{\text{base}}$ as the difference between a fine-tuned model and its base, and these vectors add, subtract, and scale like directions in weight space (Ilharco et al. 2023). Adding two task vectors yields a model good at both tasks; negating one moves the model away from a behavior (Ilharco et al. 2023). The reason this works at all is that fine-tuning moves a model a short distance in a locally near-linear region of the loss landscape, so the displacement is approximately a vector, and vectors compose.

The merging branch in Figure 9.2 arrived from a different direction than the efficiency spine. Model soups showed that averaging the weights of several models fine-tuned from the same initialization with different hyperparameters often beats picking the single best one, with no inference cost (Wortsman et al. 2022). Task arithmetic generalized averaging into a signed algebra of task vectors (Ilharco et al. 2023). The friction it exposed was interference: when two task vectors disagree on the sign of a parameter, naive addition cancels both contributions. TIES-merging addresses this by trimming small-magnitude changes, electing a sign per parameter, and merging only the agreeing updates (Yadav et al. 2023); DARE drops and rescales a large fraction of the delta before merging (Yu et al. 2024). These are the methods that let a community assemble a capable model by combining specialists instead of running one large alignment pipeline.

Merging trades reliability for near-zero cost, and the trade splits along two lines.

- **Merging versus a fresh fine-tune.** Merging is nearly free: no training, no data, just arithmetic on checkpoints. It is also approximate. The merged model is a compromise that can underperform a model actually trained on the union of the tasks, and interference between task vectors can erase a capability that each parent had. Merge when the parents are cheap and a good-enough combination is acceptable; train on the union when the combination must be reliable.

Putting it to work

A LoRA fine-tune is a small change to a standard training loop: wrap the target linear layers (commonly the attention projections, sometimes the MLP) with the factored update, freeze everything else, and train only A and B . A reader can find the canonical reference implementation in `LoraLayer`, `peft/src/peft/tuners/lora/layer.py` in the Hugging Face PEFT library, and the 4-bit NF4 base in `bitsandbytes`. The operational shape is worth holding in mind: one frozen base on disk, a directory of small adapter files, and a serving layer that loads the base once and swaps adapters per request, which is the deployment pattern PEFT exists to enable.

Three failure modes recur, one per consequence, and each has a different fix. An underfit LoRA, where the rank is too small for the adaptation, shows up as a model that half-learns the task and reverts under pressure; raise the rank. A broken chat template, where serving uses different role markers than training, looks like a capable model that ignores instructions, because it is being prompted off-distribution; align the template exactly. And merge interference, where combining two adapters degrades both, shows up as a merged model worse than either parent; switch from naive averaging to a sign-aware merge, or train on the union instead.

CONSTRAINT ARROW

The serving layer reaches back and shapes this chapter. Because LoRA's update folds into the base weights at inference, it adds no latency and no extra key-value cache, which is exactly why it won over adapters that insert sublayers. And because a server can hold one base and swap many small adapters per request, the cheap multi-tenant serving pattern in Chapter 16 is what makes per-customer LoRA adaptation economical at all. The shape of the adaptation method is set by the cost of serving it.

Further reading

- Zhou et al., "LIMA: Less Is More for Alignment," 2023 (NeurIPS). [arXiv:2305.11206](#)
- Hu et al., "LoRA: Low-Rank Adaptation of Large Language Models," 2021. [arXiv:2106.09685](#)
- Dettmers et al., "QLoRA: Efficient Finetuning of Quantized LLMs," 2023 (NeurIPS). [arXiv:2305.14314](#)
- Hounsby et al., "Parameter-Efficient Transfer Learning for NLP" (adapters), 2019 (ICML). [arXiv:1902.00751](#)
- Wortsman et al., "Model soups: averaging weights of multiple fine-tuned models improves accuracy without increasing inference time," 2022 (ICML). [arXiv:2203.05482](#)
- Ilharco et al., "Editing Models with Task Arithmetic," 2023 (ICLR). [arXiv:2212.04089](#)
- Yadav et al., "TIES-Merging: Resolving Interference When Merging Models," 2023 (NeurIPS). [arXiv:2306.01708](#)
- Yu et al., "Language Models are Super Mario: Absorbing Abilities from Homologous Models as a Free Lunch" (DARE), 2024 (ICML). [arXiv:2311.03099](#)
- Ouyang et al., "Training language models to follow instructions with human feedback" (InstructGPT), 2022 (NeurIPS). [arXiv:2203.02155](#)

10. RLHF and Reward Modeling

Supervised fine-tuning in [Chapter 9](#) teaches a model the shape of an answer, but not which of two acceptable answers a person prefers. This chapter owns the pipeline that learns that preference and optimizes against it: a reward model trained on human comparisons, then reinforcement learning with PPO against that reward, the constitutional and RLAIIF variants that replace the human labeler with an AI one, and the reward-hacking failure that shadows the whole approach. By the end a reader can explain why a learned reward needs a leash, what that leash is, and why pulling on the reward too hard makes the model worse.

The chapter is organized around that single tension. First, why a preference signal is the right thing to learn. Then how the reward and its leash are built, where the recipe came from, and the precise way the proxy breaks under pressure. The balances and the operational reality close it out.

Why a preference, not a demonstration

SFT optimizes a single demonstrated response per prompt. It cannot express that one answer is better than another, only that a given answer is the target. Many behaviors people care about are comparative and hard to demonstrate: be helpful but not sycophantic, refuse the harmful request without over-refusing the benign one near it, prefer the concise correct answer over the padded one. You can recognize the better of two responses far more reliably than you can write the ideal one from scratch, and there is no single ground-truth string to imitate.

The defining property of this chapter follows from that. The signal is a preference, and the reward that encodes it is a proxy for human judgment, not a checkable fact. When the reward becomes verifiable ground truth, a unit test that passes or a math answer that matches, the machinery is the same but the failure modes change, and that work belongs to [Chapter 14](#). Here the reward is learned and therefore wrong somewhere, and the central problem is to optimize against a reward you know to be imperfect without the policy discovering and exploiting exactly where it is wrong. The rest of the chapter is the machinery that makes optimizing a known-imperfect reward safe enough to be useful.

Building the reward

The pipeline has three stages: start from an SFT model, train a reward model on human preferences, then reinforce the policy against that reward. The InstructGPT paper fixed this recipe as the standard ([Ouyang et al. 2022](#)).

The reward model turns comparisons into a scalar. Take an SFT backbone, replace the token-prediction head with a scalar head, and train it so that for a prompt x with a human-preferred response y_w and a dispreferred y_l , it scores the winner higher. The Bradley-Terry model gives the probability that y_w beats y_l as a sigmoid of the score difference, and the reward model is fit by maximizing that likelihood:

$$\mathcal{L}_{\text{RM}} = -\log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l)).$$

Only the difference of scores is supervised, so the reward is identified up to an additive constant, which is why a reward model gives a meaningful ranking but not a calibrated absolute number. Labels come from human annotators ranking model samples, and their quality sets a ceiling on everything downstream: annotator disagreement is the noise floor, ambiguous or underspecified guidelines bias the whole reward, and the reward model overfits

the narrow distribution of responses it was trained to compare. This is the most failure-prone artifact in the pipeline, because every later step trusts it.

Figure 10.1 shows the Bradley-Terry curve and why only the difference of scores is identified: shifting both scores by the same constant leaves the preference probability unchanged.

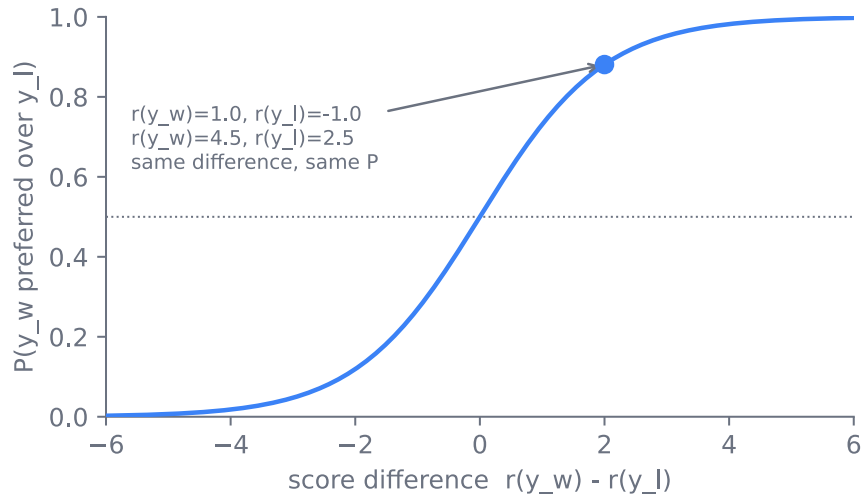


Figure 10.1. Schematic of the Bradley-Terry preference model. The probability that the preferred response wins is a sigmoid of the score difference, so two score pairs with the same difference give the same probability and an absolute shift of both scores does nothing. Idealized illustration, after the Bradley-Terry model (Bradley and Terry, 1952).

The leash

Reinforcement learning then optimizes the policy to produce responses the reward model scores highly. The objective is not simply to maximize reward. A reward model is a proxy, and a policy that maximizes a proxy without constraint will leave the distribution the reward model understands and find high-scoring nonsense. The fix is a leash: penalize the policy for drifting from the SFT reference it started at, measured as a per-token KL divergence. The objective is

$$\max_{\pi_{\theta}} \mathbb{E}_{x, y \sim \pi_{\theta}} [r_{\phi}(x, y)] - \beta \text{KL}(\pi_{\theta}(\cdot | x) \| \pi_{\text{ref}}(\cdot | x)),$$

where π_{ref} is the frozen SFT model and β sets how hard the leash pulls. The KL term is the whole story of why RLHF is stable: it keeps the policy near the region where the reward model was trained, where its scores still mean something, and it preserves the fluency and knowledge the policy already had.

PPO is the optimizer that maximizes this objective (Schulman et al. 2017). It is a policy-gradient method with two ideas that make it usable on language models. A clipped surrogate objective bounds how far the policy moves in one update, by clipping the probability ratio between the new and old policy so a single batch cannot push the policy off a cliff. And a value model, a critic, of the same backbone estimates the expected reward from each partial generation, so the per-token advantage (how much better this token was than the critic expected) has lower variance than the raw reward alone. The KL penalty enters as a per-token shaping term added to the reward. The result is a four-model arrangement held in memory at once: the policy being trained, the frozen reference for the KL term, the reward model for the score, and the critic for the advantage.

Figure 10.2 lays out the three stages and the four resident models in the PPO step. The reward model and the reference model are frozen, shown with dashed borders, while the policy and the critic are updated each step.

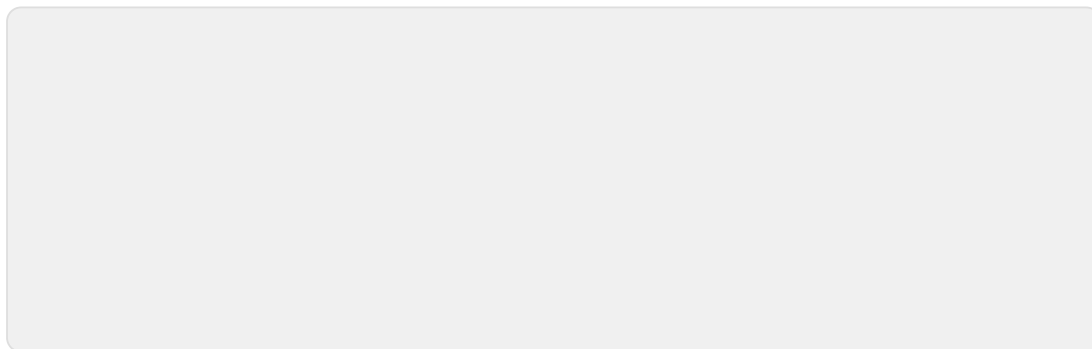


Figure 10.2. The three-stage RLHF pipeline. Stages 1 and 2 produce the frozen reference and reward models, then the PPO loop holds four models resident at once. Dashed nodes are frozen. After Ouyang et al. (2022).

Where the recipe came from

The idea of learning a reward from comparisons predates language models. Christiano et al. (2017) trained reward models from human preferences over trajectories in control and Atari tasks, establishing that a few thousand comparisons could specify a behavior no hand-written reward captured, and that optimizing the learned reward with RL could then reach it (Christiano et al. 2017). The pieces were the reward model, the policy optimized against it, and the recognition that the reward was a learned proxy.

InstructGPT (Ouyang et al., 2022) carried this to language and fixed the three-stage recipe that the field still uses: SFT, reward model on human rankings, PPO against the reward with a KL penalty to the SFT reference. Its result reframed alignment: a 1.3B InstructGPT model was preferred by humans over the 175B base GPT-3 (Ouyang et al. 2022), which showed that the bottleneck was not capability but the elicitation of helpful behavior, and that preference optimization was how you elicited it.

The cost that InstructGPT exposed was human labeling. Every comparison in the reward-model training set is a person reading two responses, which is slow and expensive and does not scale to the volume modern alignment wants. Constitutional AI (Bai et al., 2022) replaced much of that human labeling with an AI one (Bai et al. 2022). A model critiques and revises its own responses against a written set of principles, a constitution, to build the harmlessness data, and an AI model expresses the preferences that train the reward model, a recipe named RLAIIF, reinforcement learning from AI feedback. The human effort moves from labeling thousands of comparisons to writing and auditing a short list of principles, which is far cheaper to produce and to inspect.

The frontier from here splits. One branch keeps the reward model and the RL loop but swaps the human feedback for AI feedback, the RLAIIF line above. Another branch questions whether the reward model and the RL loop are needed at all, since the same preference data can be optimized more directly. That is Chapter 11. A third reuses the PPO machinery against a reward that is checkable rather than learned, which removes the reward model's unreliability as the central problem and is Chapter 14.

When the proxy breaks

A learned reward is wrong somewhere, and optimization pressure is what finds the wrong place. As the policy drifts further from the SFT reference, measured by KL, the reward

model's score keeps climbing, but true quality follows it only up to a knee, then turns down as the policy exploits regions where the proxy is wrong (Gao et al. 2022). Figure 10.3 plots that frontier: the gap between the still-rising proxy reward and the falling true quality is what opens up past the knee. Figure 10.4 captures the same phenomenon as control logic, where the KL leash keeps the policy left of the knee.

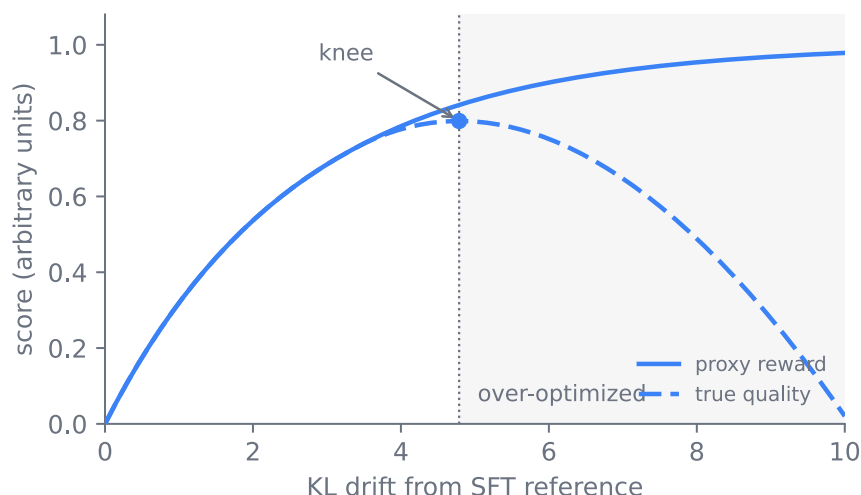


Figure 10.3. Schematic of reward over-optimization. As the policy drifts from the SFT reference, the proxy reward it is trained on keeps climbing, but true quality peaks at a knee and then falls as the policy exploits the reward model's blind spots. Idealized data, after Gao et al. (2022).

Run this to watch the knee appear, then change the over-optimization rate (the `0.18` coefficient) and see how it moves where more optimization stops helping.

```
import numpy as np
import matplotlib.pyplot as plt

# Illustrative shapes, not Gao et al.'s measured constants.
kl = np.linspace(0, 12, 200)
proxy = np.sqrt(kl)                # reward model score keeps climbing
true = np.sqrt(kl) - 0.18 * kl     # true quality peaks, then falls
knee = kl[np.argmax(true)]

plt.plot(kl, proxy, label="proxy reward")
plt.plot(kl, true, label="true quality")
plt.axvline(knee, ls="--", color="gray")
plt.text(knee + 0.2, 0.2, f"knee KL={knee:.1f}")
plt.xlabel("KL from SFT reference"); plt.ylabel("score")
plt.legend(); plt.title("Reward over-optimization (illustrative)")
plt.show()
print(f"true quality peaks at KL={knee:.1f}, then proxy rises while
```

Figure 10.4. Over-optimization of a learned reward. Proxy reward rises monotonically with drift, but true quality peaks at a knee and then falls as the policy exploits the reward model's blind spots. After Gao et al. (2022).

WHAT'S CONTESTED

Whether reward hacking is a fixable engineering problem or a fundamental limit of any learned reward is unsettled. A learned reward is a proxy, and Goodhart's law says a proxy under enough optimization pressure stops tracking the thing it stood for. The optimistic position treats over-optimization as an engineering problem: reward-model ensembles to average out individual blind spots, early stopping before the reward-versus-quality curves diverge, fresh on-policy labels to patch the regions the policy starts exploiting, and a tuned KL penalty to bound the drift. The pessimistic position holds that these only postpone the failure, because any fixed reward model has a finite description that a capable enough policy will eventually find the gap in, so the safe regime is a moving target rather than a solved one. The KL leash makes the problem manageable in practice but does not dissolve it. There is a real reward-versus-KL frontier where more optimization stops helping and then starts hurting, and where exactly it sits depends on the reward model, the data, and how hard you pull (Gao et al. 2022). Treat a rising reward curve as evidence of optimization, not of quality, and trust it only against held-out human judgment.

The balances

The pipeline is a set of balances, each with a knee.

- **Reward fidelity versus labeling cost.** Human comparisons are the highest-fidelity preference signal and the most expensive. AI feedback (RLAIF, constitutional methods) is far cheaper and scales (Bai et al. 2022), but inherits the labeling model's biases and blind spots, and a flaw in the constitution or the critic propagates into every label. Human labels set the ceiling; AI labels set the volume.
- **Optimization pressure versus over-optimization.** The KL coefficient β is the central knob. Too small a β lets the policy chase the reward off-distribution into the region where the reward model is wrong, and reward rises while human-judged quality falls. Too large a β pins the policy to the SFT reference and the RL step buys little. The right point is a frontier, not a maximum.
- **Capability versus safety.** Optimizing for helpfulness and harmlessness can degrade raw capability relative to the base model, the alignment tax: reasoning, calibration, and knowledge can all slip. Mixing pre-training data back in, KL control, and balanced preference data mitigate it, but the tension is real, and over-refusal, declining the benign request next to the harmful one, is the safety-side instance of paying it.

- **Online RL versus offline alternatives.** PPO is on-policy: it trains on fresh samples from the live policy and yields a separate, inspectable, reusable reward model, at the cost of the four-model dance and brittle hyperparameters. The offline preference methods in [Chapter 11](#) collapse this to one loss on a static set, far easier to run, at the cost of the on-policy signal and the auditable reward artifact.

CONSTRAINT ARROW

The systems cost in [Chapter 8](#) reaches up and shapes this chapter. PPO holds four models resident at once: policy, frozen reference, reward model, and critic, each near the size of the model being aligned. That memory and orchestration footprint, not any deficiency in the objective, is much of why the offline preference methods of [Chapter 11](#) exist: by deriving the reward implicitly from the policy and a single frozen reference, they remove the reward model and the critic and turn a four-model RL loop into one supervised loss. The shape of the cheaper method downstream is set by the resident cost of the on-policy one here.

Running the loop

Most of this chapter is a training pipeline, not a code change, and its operational reality is dominated by the four-model footprint and the data flow between them. A practical RLHF step generates a batch of responses from the current policy, scores each with the reward model, computes the per-token KL penalty against the frozen reference, estimates advantages with the critic, and applies a clipped PPO update to the policy and the critic. The generation step, not the optimizer step, often dominates wall-clock, which is why the loop is bottlenecked by inference throughput and pairs naturally with the serving techniques of [Chapter 16](#).

The failure modes are worth naming because each appears at a different place. A miscalibrated or overfit reward model fails before RL even starts and poisons everything downstream, so reward-model accuracy on held-out comparisons is the first thing to check. Reward hacking appears mid-run as a rising reward curve with flat or falling human-judged quality ([Gao et al. 2022](#)), and the defenses are a tuned KL penalty, early stopping, and a held-out human evaluation that the reward curve cannot game. Sycophancy is a specific, predictable hack: labelers and users reward agreement and flattery, so a policy optimizing their approval learns to tell people what they want to hear, which is a direct artifact of optimizing an approval proxy rather than a bug in the optimizer. And the alignment tax shows up as a capability regression against the base model on benchmarks the preference data never covered, which is why [Chapter 27](#) gates the aligned model rather than trusting the reward.

Further reading

- Christiano et al., "Deep Reinforcement Learning from Human Preferences," 2017 (NeurIPS). [arXiv:1706.03741](#)
- Ouyang et al., "Training language models to follow instructions with human feedback" (InstructGPT), 2022 (NeurIPS). [arXiv:2203.02155](#)
- Schulman et al., "Proximal Policy Optimization Algorithms" (PPO), 2017. [arXiv:1707.06347](#)
- Bai et al., "Constitutional AI: Harmlessness from AI Feedback," 2022 (Anthropic). [arXiv:2212.08073](#)
- Gao et al., "Scaling Laws for Reward Model Overoptimization," 2022. [arXiv:2210.10760](#)

11. Direct Preference Optimization and the Variant Zoo

There is a recurring moment in an alignment pipeline. A team has a static file of preference pairs, a chosen response and a rejected one for each prompt, and it wants an aligned model out the other end. The textbook route is RLHF, and RLHF is heavy. This chapter follows the line of thought that asks whether the heaviness is necessary, watches that question collapse the whole apparatus into a single classification loss, then watches a small zoo of variants grow on top of that loss, and ends with the uncomfortable finding that, under a controlled comparison, none of the variants reliably beats the plain loss they all descend from.

A team with a pile of preferences

RLHF, covered in Chapter 10, aligns a model in three stages: supervised fine-tuning, a reward model fit to human preferences, then a policy-gradient run with PPO (Schulman et al. 2017) that optimizes the policy against that reward under a KL leash to the reference. The machinery is heavy. PPO keeps four models in play at once, the policy, a frozen reference, the reward model, and a critic, and its result is sensitive to reward-model quality and to a handful of brittle hyperparameters. For the team with a static set of preference pairs, the question is whether all of that apparatus is necessary, or whether the preference signal can be optimized directly.

The trick: a language model is secretly a reward model

The reduction rests on one fact about the RLHF objective. Maximizing reward under a KL penalty to a reference policy,

$$\max_{\pi_{\theta}} \mathbb{E}_{x, y \sim \pi_{\theta}} [r(x, y)] - \beta \text{KL}(\pi_{\theta}(\cdot | x) \| \pi_{\text{ref}}(\cdot | x)),$$

has a closed-form optimal policy: it is the reference reweighted by the exponentiated reward, $\pi^*(y | x) \propto \pi_{\text{ref}}(y | x) \exp(r(x, y)/\beta)$. Rafailov et al. invert this (Rafailov et al. 2023). Solving for the reward in terms of the optimal policy expresses r as a log-ratio, so the reward that any policy implies is

$$r_{\theta}(x, y) = \beta \log \frac{\pi_{\theta}(y | x)}{\pi_{\text{ref}}(y | x)} + \beta \log Z(x).$$

Substitute that implicit reward into the Bradley-Terry model of pairwise preference and the intractable partition term $Z(x)$ cancels between the chosen response y_w and the rejected y_l . What remains is a supervised loss on preference pairs,

$$\mathcal{L}_{\text{DPO}} = -\mathbb{E}_{(x, y_w, y_l)} \log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \beta \log \frac{\pi_{\theta}(y_l | x)}{\pi_{\text{ref}}(y_l | x)} \right).$$

The reward model and the RL loop are gone. The policy is trained to assign higher implicit reward to the preferred response, where implicit reward is the log-ratio of the policy to a frozen reference. The title of the paper states the point: the language model is secretly a reward model. [Figure 11.1](#) shows the machinery that this substitution deletes.

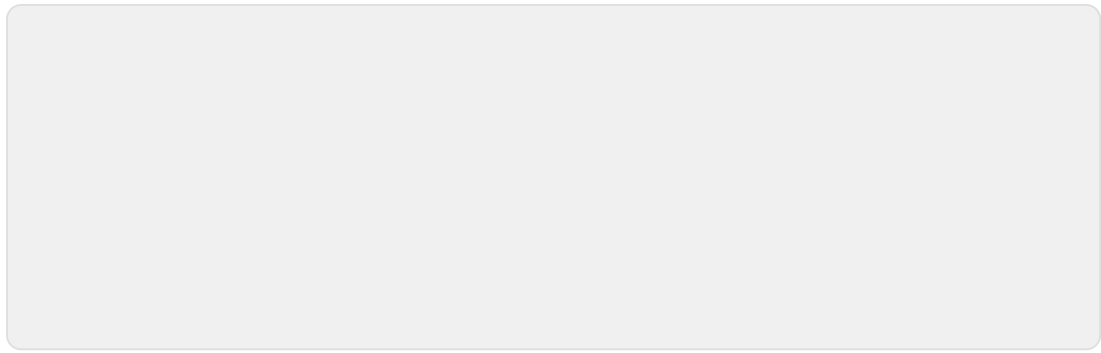


Figure 11.1. RLHF runs a reward model and a PPO loop over four models; DPO substitutes the implicit reward into the Bradley-Terry model, the partition term cancels, and what remains is one classification loss over preference pairs. After Rafailov et al. (2023).

Four edits to one loss

Once the loss exists, every later method is an edit to it. Each variant removes one assumption or one piece of machinery the previous line of the loss carried. Figure 11.2 lays out which piece each one changes.

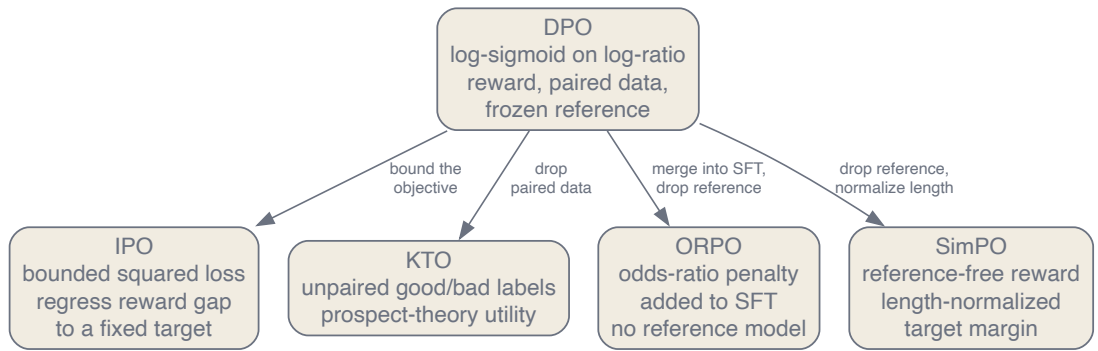


Figure 11.2. Each variant edits one part of the DPO loss. IPO bounds the objective; KTO drops the paired-data requirement; ORPO folds alignment into SFT and removes the reference; SimPO removes the reference and normalizes by length.

IPO bounds the objective. Azar et al. target a weakness in the derivation (Azar et al. 2023): the Bradley-Terry substitution assumes preferences are well modeled as a deterministic ordering, and with finite, near-deterministic data the log-sigmoid loss can drive the policy to push the implicit reward gap to infinity, overfitting the preference set. IPO replaces the log-sigmoid with a bounded squared objective that regresses the implicit reward gap toward a fixed target, so the optimum stays finite even when preferences are clean. Figure 11.3 contrasts the two loss shapes.

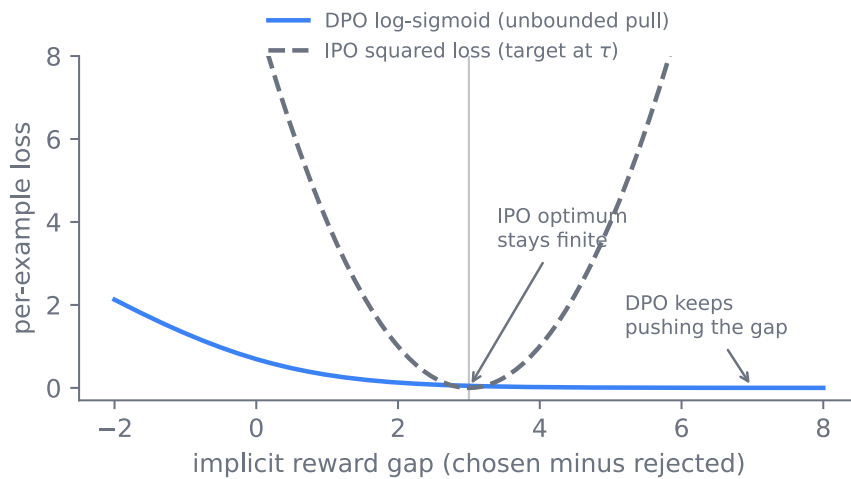


Figure 11.3. Schematic illustration of the bounded IPO objective versus the unbounded DPO log-sigmoid loss as a function of the implicit reward gap, after Azar et al. (2023). The log-sigmoid keeps decreasing as the gap grows, so its optimum pushes the gap toward infinity; the squared loss has a minimum at a fixed target, so its optimum stays finite.

Run a few gradient steps on a single reward gap under each loss and watch the DPO gap climb without bound while the IPO gap settles at its target.

```
import numpy as np
import matplotlib.pyplot as plt

def sigmoid(z): return 1.0 / (1.0 + np.exp(-z))

beta, lr, target, steps = 1.0, 0.5, 5.0, 60
gd, gi = [0.0], [0.0]
for t in range(steps):
    # DPO gradient on the gap never reaches zero, so the gap keeps g
    gd.append(gd[-1] + lr * beta * sigmoid(-beta * gd[-1]))
    # IPO regresses the gap to a fixed target, so it settles there
    gi.append(gi[-1] - lr * 2.0 * beta * (beta * gi[-1] - target))
print(f"DPO gap after {steps} steps: {gd[-1]:.2f} and still rising")
print(f"IPO gap converges to target: {gi[-1]:.2f}")
plt.plot(gd, label="DPO log-sigmoid"); plt.plot(gi, label="IPO bound")
plt.axhline(target, ls="--", c="gray"); plt.xlabel("step"); plt.ylabel("loss")
plt.legend(); plt.show()
```

KTO drops the paired-data requirement. DPO needs a chosen and a rejected response for the same prompt; KTO learns from unpaired binary good or bad labels by scoring each response against a reference point with a prospect-theory utility (Ethayarajh et al. 2024), the same human-value-of-gains-and-losses shape Kahneman and Tversky used. This trades the paired-data constraint for a utility model, which matters because thumbs-up and thumbs-down logs are far more abundant than curated pairs.

ORPO folds alignment into SFT. Instead of a separate alignment stage with a reference model, Hong et al. add an odds-ratio penalty to the SFT cross-entropy (Hong et al. 2024) (the SFT stage itself is Chapter 9), so one pass both teaches the format and down-weights the rejected response. It carries no reference model at all.

SimPO removes the reference and normalizes by length. Meng et al. drop the reference policy from the implicit reward (Meng et al. 2024), using the average log-probability of a sequence as the reward, and add a target margin between chosen and rejected. The length normalization is the deliberate part: DPO's log-ratio reward correlates with sequence length, which lets a policy raise its score by getting wordier, and dividing by length removes that lever. Figure 11.4 shows the reward against response length under both rewards.

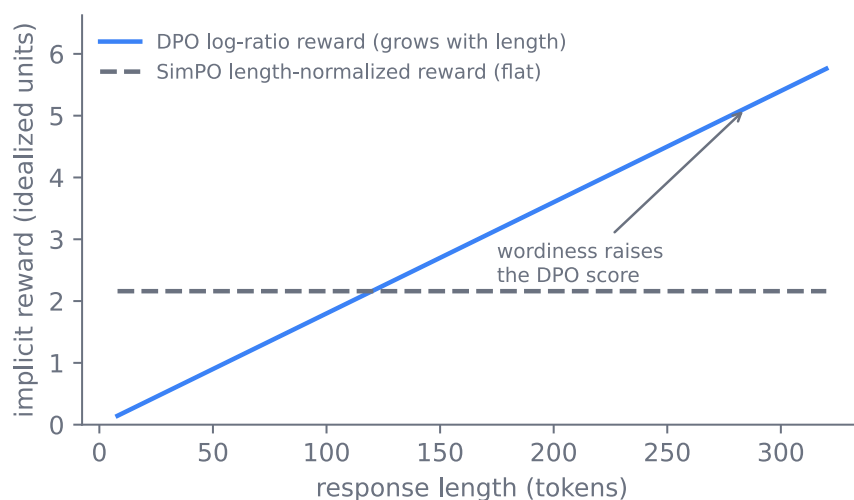


Figure 11.4. Schematic illustration of how SimPO's length normalization flattens the implicit reward against response length, after Meng et al. (2024). The DPO log-ratio reward sums per-token terms and so grows with length, which a policy can exploit by getting wordier; the length-normalized SimPO reward is the per-token average and stays flat.

The reckoning

Read in sequence, the four edits look like a ladder of improvements, each variant fixing what the last one missed. A controlled comparison undercuts that reading.

WHAT'S CONTESTED

The variant zoo is usually presented as a ladder of improvements, but a controlled multi-scale study (arXiv:2603.19335) finds no variant reliably beats vanilla DPO once confounds are corrected for. When tuning budget, data, and model are held equal, the gaps between methods shrink into the noise, and the apparent ranking inverts across model scale: a variant that leads at one size trails at another. The practical reading is that reported wins for a new objective often reflect a better-tuned baseline-versus-variant comparison rather than a property of the loss, and that the choice among these methods should be driven by pipeline and data constraints rather than by a claimed accuracy edge.

If accuracy does not separate the variants, then something else has to decide between them, and that something is the shape of the pipeline and the data.

What actually decides the choice

With the accuracy edge gone, the real axes are the engineering ones the variants trade along. Four of them recur.

- **Reference model versus none.** Keeping the frozen reference (DPO, IPO) anchors the policy and gives the KL leash a meaning, at the cost of a second resident model and forward pass. Dropping it (ORPO, SimPO) halves the alignment-stage memory and simplifies the pipeline, but removes the explicit anchor that limits drift from the starting policy.
- **Paired versus unpaired data.** DPO, IPO, ORPO, and SimPO all need a chosen and a rejected response per prompt. KTO accepts unpaired binary signals, which fits production feedback logs but substitutes a utility model for the pairwise comparison the others optimize.
- **Bounded versus unbounded objective.** IPO's bounded regression resists the overfitting that DPO's log-sigmoid invites on clean, near-deterministic preferences. The price is a loss that no longer matches Bradley-Terry exactly and a target margin to set.
- **Two stages versus one.** ORPO merges alignment into SFT, saving a stage and a reference model, at the cost of entangling format learning and preference learning so they can no longer be tuned or audited separately.

The first of these axes is not only a modeling choice. It is forced from below.

CONSTRAINT ARROW

The memory budget of [Chapter 8](#) reaches up and shapes the loss. DPO and IPO keep a frozen reference model alongside the policy and pay a second forward pass through it on every step. At large model size that doubled footprint is the cost that ORPO (no reference) and SimPO (reference-free) exist to remove. The reference-free design is not primarily an accuracy claim, it is a response to the systems cost of holding two models.

Running it

DPO is the smallest of the family to run: one loss, a frozen copy of the SFT model as the reference, and a static file of preference triples. The core is the log-ratio difference inside a log-sigmoid, computed from per-token log-probabilities of the chosen and rejected responses under the policy and the reference; see the loss derivation in [DPO loss, Rafailov et al. \(arXiv:2305.18290\)](#), [Sec. 4](#) (Rafailov et al. 2023). The reference forward pass can be precomputed once because the reference never updates, which is the standard memory optimization.

The one hyperparameter that matters across the family is β , the strength of the KL leash that also scales the implicit reward. Too small and the policy drifts off the reference and degrades; too large and it barely moves. Because vanilla DPO is off-policy, trained on a fixed set rather than on samples from the live policy, its ceiling is the fixed data, and the common remedy is iterated or online DPO that regenerates pairs from the current model between rounds. That on-policy variant, and the broader RLHF apparatus it approaches, belong to [Chapter 10](#).

Further reading

- Rafailov et al., "Direct Preference Optimization: Your Language Model is Secretly a Reward Model" (DPO), 2023 (NeurIPS). [arXiv:2305.18290](#)

- Azar et al., "A General Theoretical Paradigm to Understand Learning from Human Preferences" (IPO), 2023. [arXiv:2310.12036](#)
- Ethayarajh et al., "KTO: Model Alignment as Prospect Theoretic Optimization," 2024. [arXiv:2402.01306](#)
- Hong et al., "ORPO: Monolithic Preference Optimization without Reference Model," 2024. [arXiv:2403.07691](#)
- Meng et al., "SimPO: Simple Preference Optimization with a Reference-Free Reward," 2024 (NeurIPS). [arXiv:2405.14734](#)
- Schulman et al., "Proximal Policy Optimization Algorithms" (PPO), 2017. [arXiv:1707.06347](#)

12. Synthetic Data and Self-Improvement

Human-written training data is finite, and the best of it is the most expensive. This chapter is about where the training signal comes from when you stop buying fresh human labels: a stronger model, the model's own filtered outputs, an AI critic, or a verifier. By the end a reader can explain why distillation, rejection sampling, self-play, and AI feedback are four answers to one question, why iterating any of them is called a data flywheel, and where each loop hits a ceiling.

The question that organizes the chapter

Post-training needs labeled examples: prompts paired with good responses, or responses paired with a judgment of which is better. The supervised and preference methods that consume those labels live in Chapter 9 and Chapter 10; Chapter 11 covers the direct-preference family. The problem this chapter owns is upstream of all of them. Human demonstrations and human preference labels are slow, costly, and bounded by the skill of the annotator. A frontier model is often already better than the median labeler at the task you want to teach, so paying humans to produce its training data caps the result at human-median quality and exhausts the budget long before the data does.

So one question sits under everything that follows: who decides a response is good enough to train on? If the answer can be something cheaper than a human, post-training scales with compute instead of with annotation headcount, and the data stops being the wall that Chapter 4 describes for pre-training. The catch is that a self-produced signal is only as trustworthy as whatever filters it, and a bad filter teaches the model its own mistakes.

There turn out to be four answers, and they form a single lineage: a steady retreat from the human labeler, removing the human from one role at a time. The chapter walks that retreat, source of judgment by source of judgment, and each step both buys something and gives something up.

One loop, four answers

Before the four answers, the shape they share. Every method in this chapter is the same loop: generate candidates, filter them, train on what survives, and possibly repeat. The methods differ in exactly one place, the source of judgment that drives the filter, as Figure 12.1 shows.

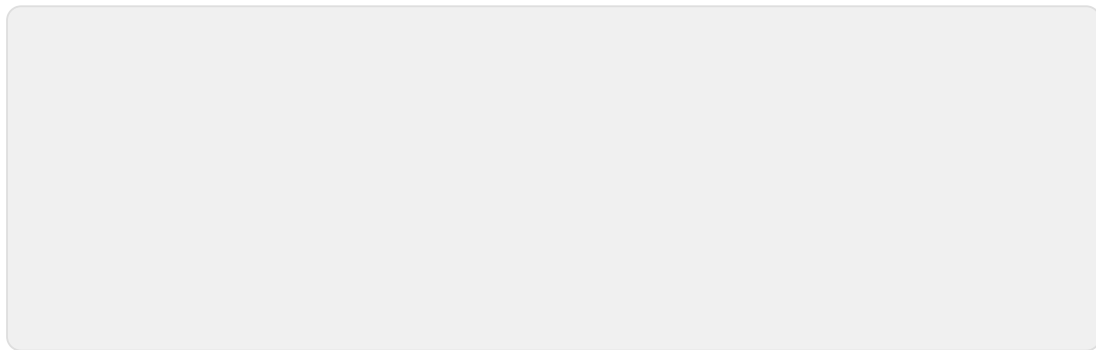


Figure 12.1. The shared generate, filter, train loop. The four methods differ only in the source of judgment that drives the filter.

When that loop is run repeatedly, each round's improved model generating the next round's data, it is a **data flywheel**: the model's current capability becomes the engine that produces the data for its next increment. The flywheel is not a fifth method. It is what you get by iterating any of the four.

Why filtering works at all is worth stating, because it is the engine under every answer. A model that cannot reliably produce a correct answer on the first try can often produce a correct answer somewhere in n tries, and a cheap filter can find it. Training on the filtered winners moves probability mass onto behavior the model was already capable of but did not reliably emit. This is the same elicitation framing that explains why a few thousand curated examples move behavior so much: the capability is latent, and the data selects for it rather than installing it. LIMA is the clean evidence, showing that a small, carefully filtered set outperforms a large noisy one (Zhou et al. 2023).

Vary the per-try success rate p and watch how fast $\text{pass}@n$, the chance at least one of n samples is correct, climbs toward one as n grows.

```
import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(0)
ns = np.arange(1, 21)
for p in (0.1, 0.3, 0.5):
    # per-try success probability
    analytic = 1 - (1 - p) ** ns  # pass@n if tries are independent
    sims = [(rng.random((3000, n)) < p).any(axis=1).mean() for n in ns]
    line, = plt.plot(ns, analytic, label=f"p={p}")
    plt.scatter(ns, sims, s=10, color=line.get_color())
plt.xlabel("samples n"); plt.ylabel("pass@n (at least one correct)")
plt.legend(); plt.title("Why filtering works: best-of-n success");
print("p=0.10: a model right 10% of the time succeeds within 20 tries")
print(round(1 - 0.9 ** 20, 2), "of the time")
```

The four sources of judgment are a steady retreat from the human labeler, each step removing a human from a different role, as [Figure 12.2](#) traces. The rest of the chapter walks that retreat.

Figure 12.2. The retreat from the human labeler. Each step replaces a human in one role with a cheaper source of judgment, and the source of the next round's data shifts accordingly.

Answer one: borrow a teacher

The first move was to **borrow a teacher**. A more capable model generates the responses, and a weaker student trains on them. The judgment is implicit: the teacher's output is taken as the target. This is response distillation, learning from the teacher's text, not its logits, and once a strong aligned model existed, it was the cheapest way to align a weaker one: have the strong model write the instruction-following data. This made capable open models possible without each lab running its own human-preference pipeline, and it remains the default way open models are aligned.

Its limit is structural. A student trained only on a teacher's distilled outputs rarely exceeds the teacher on those skills, and it inherits the teacher's biases and refusals along with its competence. Distillation is off-policy: cheaper and more stable than learning from the model's own samples, but it drifts from the live model and caps at the teacher. The data is borrowed, and a borrowed ceiling is still a ceiling.

Answer two: filter your own outputs

The next move was to **filter your own outputs**, so the model could improve without a stronger teacher to copy. Sample n responses from the current model, score them, keep the best, and fine-tune on the winners. The judgment is now a scorer applied to the model's own samples. Iterated, this is rejection-sampling fine-tuning: on-policy preference data without a full RL loop. RAFT formalized it, ranking sampled responses by a reward and fine-tuning on the top of each group (Dong et al. 2023).

The gain is that the signal is on-policy, drawn from the model itself, which closes the gap between what the model trains on and what it actually generates. The model stops imitating a foreign distribution and starts selecting from its own. The cost is that the ceiling is now set by the scorer rather than by a teacher, which moves the problem rather than removing it: the loop is only as good as whatever scores the samples.

Answer three: replace the labeler

The third move was to **replace the labeler** rather than just the demonstrator. A model, prompted with written principles, produces the preference signal that a human annotator would otherwise provide. Constitutional AI is the canonical form: an AI critiques and revises responses against a short written constitution, and the resulting pairs train the model, cutting the human cost of the preference signal down to the principles themselves (Bai et al. 2022). The judgment is a model reading rules. This is reinforcement learning from AI feedback, RLAIFF; the RL machinery that consumes that preference signal is [Chapter 10](#), but the signal's origin is here.

The reach is what makes it attractive: an AI critic can score open-ended responses, helpfulness and style and tone, where no ground-truth checker exists. The cost is that the judge is a proxy. Push on it and the model finds outputs the critic rates highly and humans do not. Replacing the labeler removes the human from the labeling role but installs a gameable stand-in where the human used to be.

Answer four: improve against a checker

The most recent move was to **improve against a checker**. Generate many candidate solutions, run each through a checker, a unit-test suite, an answer checker, a proof checker, and keep the ones that pass. The judgment is ground truth, not a proxy, which makes this the cleanest loop, because the filter cannot be flattered. When the filter is a verifier rather than a learned scorer,

the loop can bootstrap on math and code: generate solutions, keep the verified-correct traces, train on them, and repeat.

DeepSeek-R1 is the open data point, including an R1-Zero variant that bootstraps reasoning from a base model with no supervised cold start (Guo et al. 2025). The reinforcement-learning mechanics that turn verified traces into a reasoning model belong to Chapter 14; what belongs here is the narrower observation that a sound verifier turns the model's own correct outputs into a renewable training set. Where the verifier is a learned process reward model rather than an exact checker, the signal can be made finer-grained by scoring each reasoning step rather than only the final answer (Lightman et al. 2023), though a learned scorer reintroduces the proxy problem the exact checker had escaped.

The reason this loop bootstraps where the others cap is that the ceiling rises with the model. The model can be pushed to produce correct outputs it could not reliably produce before, and the verified-correct set grows with each round. That is the cleanest version of the data flywheel: capability generating the data for its own next increment, with a filter that cannot be talked into accepting a wrong answer.

How far does the bootstrap go?

Walking the four answers leaves one tension unresolved, and it is the real question of the chapter. Can self-improvement create genuinely new capability, or is it bounded by whatever does the judging?

WHAT'S CONTESTED

Whether self-improvement can create genuinely new capability, or is bounded by whatever does the judging, is unsettled, and the source supports both poles. On the distillation side the ceiling looks hard: a student rarely beats its teacher on a distilled skill, so copying a teacher cannot, on its own, exceed the teacher. On the verifier side the loop looks open-ended: a sound verifier is a genuine bootstrap, because the model can be pushed to produce correct outputs it could not reliably produce before and then train on them, and the verified-correct set grows with the model. The unresolved question is how far this goes. A verifier amplifies whatever it can check and amplifies its blind spots just as faithfully, so an unsound verifier teaches the model to satisfy the checker rather than solve the problem. Treat "self-improvement" as bounded by the quality of the judge, with the bound tight for a fixed teacher and loose, but not infinite, for a sound verifier.

The answer to how far it goes is decided one layer down, by whether the domain is verifiable at all. That is the constraint that shapes everything above it.

CONSTRAINT ARROW

Whether the flywheel closes at all is decided one layer down, by whether the domain is verifiable. Math, code, and formal proof come with cheap, near-unspoofable checkers, so their loops close: the model generates, the checker filters, and the verified-correct set feeds the next round without a human in it. Open-ended domains, helpfulness, style, judgment, taste, have no ground-truth checker, so the strongest filter available is a learned scorer or an AI critic, both of which are proxies the model can learn to game. A data-layer property, the existence of a verifier, decides whether an upper-layer training loop can run unattended. This is why the recent capability jumps cluster in checkable domains and why open-ended quality still leans on human preference data from Chapter 10.

The four answers therefore fall along a single antidiagonal between two ends: cheap-and-broad coverage that buys a gameable proxy, and trustworthy-but-narrow ground truth that costs coverage. A learned reward model or an AI critic can score any response, including open-ended ones, but it is a proxy. A verifier cannot be flattered, but it only exists where ground truth is checkable, and its coverage is the new attack surface. Figure 12.3 places the four sources on that axis.



Figure 12.3. The four sources of judgment in the coverage-versus-trust space. Broad coverage buys gameable proxies, near-unspooable ground truth costs coverage, and the methods lie along the antidiagonal between them.

Two more trade-offs cut across the antidiagonal. On-policy filtering (rejection sampling, self-play) trains on what the model actually generates and closes the train-serve gap, where distilling a teacher is off-policy, cheaper and more stable but drifting from the live model. And a single generate-filter-train pass is simple and bounded, where iterating compounds gains but also compounds errors: a small bias in the filter, applied every round, becomes a learned habit, and the model can collapse onto whatever the filter rewards. The flywheel needs a filter sound enough to survive repetition. Distillation has one further cost the others do not: it may violate the teacher's terms of service, and from-scratch alignment, expensive as it is, remains the only path to surpassing the available teachers.

The one line that decides everything

The minimal loop is short enough to state directly, and writing it out shows why the four answers are one method. Rejection-sampling fine-tuning, the RAFT recipe (Dong et al. 2023), is: for each prompt, sample a group of responses from the current policy, score them, keep the highest within the group, and fine-tune on the kept set, then repeat with the updated model.

```
# Rejection-sampling fine-tuning, one round (sketch).
data = []
for prompt in prompts:
    cands = [model.sample(prompt) for _ in range(n)]
    scored = [(judge(prompt, c), c) for c in cands]
    best = max(scored)[1]          # judge: reward model, AI critic
    if keep(best):                 # threshold or verified-correct
        data.append((prompt, best))
model = finetune(model, data)     # iterate: next round samples f
```

The single line that determines everything is `judge`. Swap in a stronger model and the loop is distillation; a reward model or AI critic and it is rejection sampling or AI feedback; a unit-test or answer checker and it is verifier-based self-improvement. The surrounding machinery is identical; the trust you can place in the result is entirely a property of `judge`. The four answers of this chapter are four bindings of one variable.

Two operational realities follow from that. First, the filter's false negatives starve the loop: a verifier that rejects correct-but-unusual answers, or a scorer miscalibrated against good outputs, removes exactly the data you wanted. Second, the loop's failures are quiet. A flywheel running on a gameable filter shows rising filtered-set quality by the filter's own measure while

held-out quality stalls or regresses, the same Goodhart signature that reward over-optimization produces in [Chapter 10](#). The defense is to keep an independent evaluation outside the loop, which is the job of [Chapter 27](#), and to prefer a verifier wherever the domain admits one.

Further reading

- Bai et al., "Constitutional AI: Harmlessness from AI Feedback," 2022 (Anthropic). [arXiv:2212.08073](#)
- Zhou et al., "LIMA: Less Is More for Alignment," 2023 (NeurIPS). [arXiv:2305.11206](#)
- Dong et al., "RAFT: Reward rAnked FineTuning for Generative Foundation Model Alignment" (rejection-sampling fine-tuning), 2023 (TMLR). [arXiv:2304.06767](#)
- DeepSeek-AI, "DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning," 2025. [arXiv:2501.12948](#)
- Lightman et al., "Let's Verify Step by Step" (process reward models / PRMs), 2023. [arXiv:2305.20050](#)

13. Eliciting Reasoning

A base model already holds far more reasoning ability than greedy decoding reveals, and you can draw it out at inference time without touching a single weight. This chapter is about that elicitation: the prompting and decoding methods that turn one forward pass into a deliberation. By the end a reader can explain why a chain of thought helps, why sampling many chains and voting beats sampling one, when a tree or a search is worth its cost, and why a verifier is the quiet workhorse behind best-of-N. Teaching the weights to reason is the sibling problem of [Chapter 14](#); here nothing is trained.

The answer token comes too early

Ask a model a multi-step question and decode greedily, and it commits to an answer token before it has done the work that answer depends on. The autoregressive objective of [Chapter 3](#) spends a fixed, equal slice of compute on every token, so a problem that needs ten steps of arithmetic gets the same single pass as a problem that needs none. The model has the sub-skills, since pre-training saw the additions, the lookups, and the deductions, but the decode gives them nowhere to run. The answer position is asked to integrate everything at once.

That single observation defines the whole chapter. The work is to spend more inference compute, and spend it in the right shape, so that latent ability surfaces, without changing the model. Two levers exist at inference time. One is the prompt, which can ask the model to externalize intermediate steps into tokens it can then condition on. The other is the decoding procedure, which can sample, branch, score, and select instead of greedily taking the top token. Every method below is a combination of those two levers, and they arrived in a clear lineage: first give the reasoning somewhere to run, then stop trusting one sample of it, then shape and search the space it opens. [Figure 13.1](#) previews the four shapes that lineage produces.

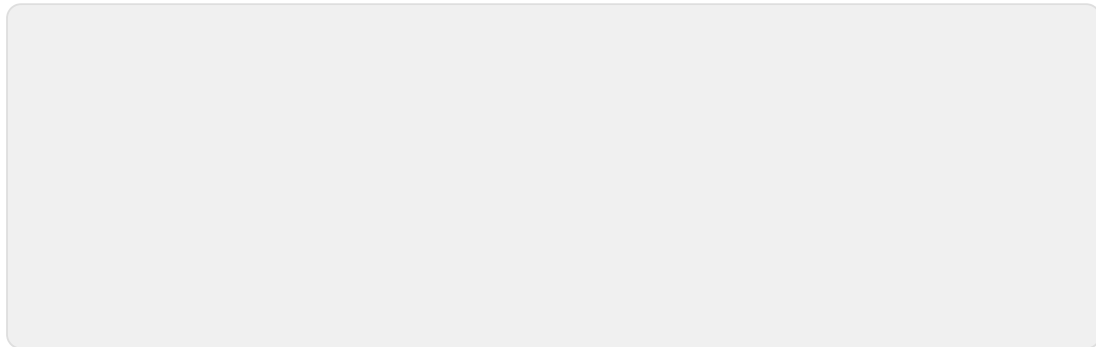


Figure 13.1. Four decoding shapes that elicit and select reasoning from a fixed model. Each spends more inference compute than greedy decoding, in a different structure.

Give the reasoning somewhere to run

The first move is to make the model write its working, then exploit that the working is now part of the context. A chain of thought is the minimal form: prompt the model to produce intermediate reasoning steps before the final answer, and each step becomes a token sequence the later steps attend over. This converts a single hard prediction into a sequence of easier ones, and it gives the model room to allocate more forward passes, one per reasoning token, to a harder problem. A few worked examples in the prompt elicit this ([Wei et al. 2022](#)), and even the bare instruction "Let's think step by step" elicits it zero-shot, with no exemplars at all

(Kojima et al. 2022). No weights move. The ability was latent; the prompt unlocks the decode, and the zero-shot result makes the point sharp: the effect is about eliciting, not demonstrating.

Stop trusting a single sample

Once reasoning lives in tokens, the decoding procedure becomes a design space. The next move is to stop trusting a single sample. A greedy chain can take a wrong turn early and never recover. Self-consistency samples many chains at non-zero temperature, then marginalizes over the reasoning paths by taking a majority vote on the final answers (Wang et al. 2022). The reasoning path is treated as a latent variable to integrate out, not an output to return:

$$\hat{y} = \arg \max_y \sum_{i=1}^N 1[a(z_i) = y], \quad z_i \sim p_\theta(z | x)$$

where each z_i is a sampled chain and $a(z_i)$ is the answer it reaches. Many paths can reach the right answer by different routes, while wrong answers tend to scatter, so the mode of the answer distribution is a better estimate than any one sample. This is the highest value-per-line method in the chapter: it needs no verifier, no extra model, only more samples and a vote, and it remains the strongest simple baseline for exactly that reason (Wang et al. 2022).

Try this: each chain is independently correct with probability p , and the simulation votes over N chains. Watch majority-vote accuracy rise above a single sample once $p > 0.5$, then saturate.

```
import numpy as np

rng = np.random.default_rng(0)
p = 0.55          # per-chain probability of the correct answer
trials = 20000    # Monte Carlo trials per N
Ns = [1, 3, 5, 11, 21, 41]

for N in Ns:
    correct = rng.random((trials, N)) < p    # which chains got it right
    votes = correct.sum(axis=1)                # how many voted for the
    win = (votes > N / 2).mean()              # majority is right (tie
    print(f"N={N:>2} majority-vote accuracy = {win:.3f}")

print(f"\nSingle chain (N=1) accuracy = {p:.3f}; voting amplifies a
```

Shape the chain, then search the space

Sampling more of the same chain is not the only refinement. When a problem decomposes, you can shape the chain instead of just sampling it. Least-to-most prompting asks the model first to break a problem into ordered subproblems and then to solve them in sequence, each conditioned on the solved ones, which lets the model generalize to problems harder than any single exemplar in the prompt (Zhou et al. 2022).

The richest design treats reasoning as search over a space of partial solutions. Tree of Thoughts makes each node a coherent intermediate thought, expands several children per node, has the model self-evaluate each as promising or not, and explores the tree with breadth-first or depth-first search, including lookahead and backtracking (Yao et al. 2023). A chain is then the degenerate tree with one child per node and no backtracking. Pushing this further, AlphaZero-style methods run Monte Carlo Tree Search over the reasoning tree, using value estimates to decide where to expand, so compute concentrates on promising branches rather than spreading evenly (Feng et al. 2023).

The chain, the tree, and the value-guided search differ in the shape of the space they explore, shown in Figure 13.2. A chain walks one line. A tree fans out, self-evaluates each node, and can backtrack out of a dead end. A search adds a value estimate so compute concentrates on promising branches instead of spreading evenly.

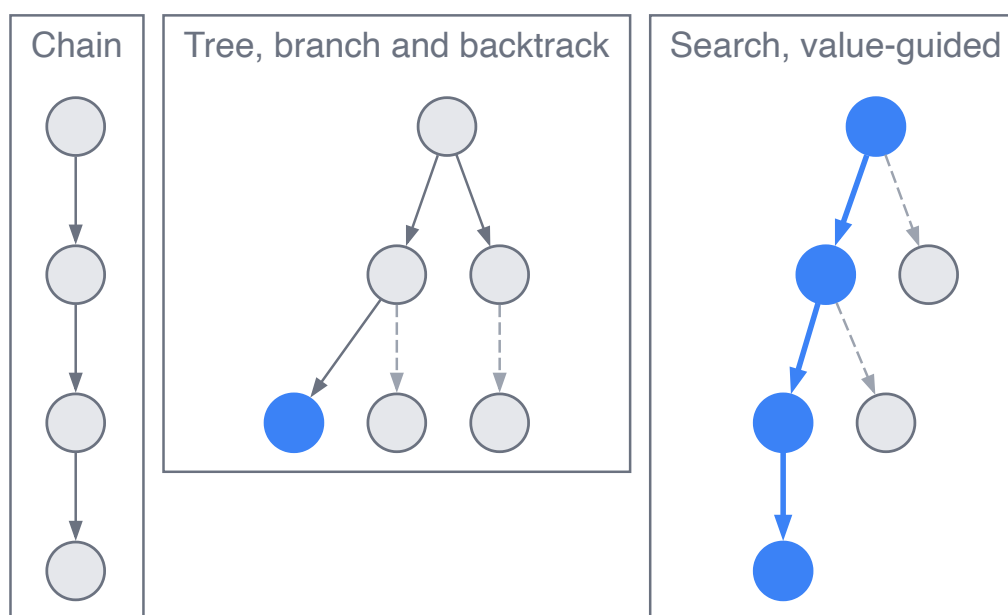


Figure 13.2. Chain, tree, and value-guided search as three shapes of the same reasoning space. The chain is the degenerate tree with one child per node and no backtracking. After Yao et al. (2023) and Feng et al. (2023).

When a vote is not enough: the verifier

Self-consistency votes, but a vote only works when the answer is a discrete label to count.

Best-of- N replaces the vote with a score. Sample N candidate solutions, score each with a verifier or reward model, and return the highest. This was introduced for grade-school math: train a verifier to judge candidate solutions, generate many, and keep the one the verifier ranks highest, which beat a fine-tuning baseline (Cobbe et al. 2021). The same scorer can guide the search rather than just rank the finished output, evaluating partial states inside a tree, which is what self-evaluation guided beam search does (Xie et al. 2023). The verifier is the component that turns raw samples into a selected answer, and its quality is the ceiling on what selection can buy. That ceiling, and how it bounds test-time scaling, is the subject of Chapter 15.

A vote and a score can disagree, and Figure 13.3 shows when. Over the same sampled chains, a vote rewards agreement and a score rewards correctness. When a biased model is consistently wrong, the majority answer is also wrong, and only a verifier that checks the answer against ground truth can rescue the minority chain that happens to be right.

Figure 13.3. Vote versus score over the same N samples. A majority vote returns the consensus 42, which is wrong. A verifier scores each chain against ground truth and selects the minority chain that reaches 37, which is right.

WHAT'S CONTESTED

Whether chain-of-thought tokens are a faithful account of the computation that produced the answer is unsettled. The optimistic reading treats the chain as the model's actual reasoning, which would make it both a performance lever and a window for oversight. The skeptical reading holds that the chain can be a post-hoc rationalization: the answer is determined by computation the tokens do not reflect, and the model can reach the same answer when the stated reasoning is perturbed, or state reasoning that does not drive the answer. If chains are unfaithful, then the accuracy gain is real but the interpretability it seems to offer is not, which matters directly for the oversight argument of [Chapter 33](#). The evidence cuts both ways and depends on task, model, and how faithfulness is probed, so treat a visible chain as a help to accuracy first and a faithful trace only when tested.

CONSTRAINT ARROW

The verifier you can build at the data layer decides which elicitation method you can run. Self-consistency needs only a discrete answer to vote on, so it works wherever answers are comparable. Best-of- N and search need a scorer, and the best scorer is a ground-truth checker, a unit test, a numeric or symbolic equality, a proof checker, exactly the checkable-answer condition that defines [Chapter 14](#). Where a sound verifier exists, search and best-of- N pay off and their ceiling is high. Where only a learned reward model exists, the same methods inherit its over-optimization from [Chapter 10](#) and a stronger search can make the answer worse by driving harder on a flawed score. The availability of a verifier, a property of the task and not the decoder, is what sets how far inference-time compute can be spent profitably.

What each shape costs, and what bounds it

Each method buys accuracy with inference compute, and the exchange rate differs.

- **Single chain versus voting.** One chain is cheapest and lowest latency but brittle to an early wrong turn. Self-consistency multiplies token cost by the sample count N for a robust majority answer, with diminishing returns as N grows. It needs a discrete, comparable answer, so it does not apply to open-ended generation where there is nothing to vote on.
- **Voting versus scoring.** A vote needs no extra model but cannot rank free-form outputs and cannot tell a confidently-wrong consensus from a correct one. Best-of- N needs a verifier or reward model but ranks anything that scorer covers, and its ceiling rises with verifier

quality. A vote rewards agreement; a score rewards correctness, and they diverge when the model is consistently wrong.

- **Chain versus tree versus search.** A chain spends compute in one line. A tree spends it on breadth and can backtrack out of dead ends, at the cost of many self-evaluation calls per node, which are themselves model calls that can be wrong. MCTS-style search concentrates compute on high-value branches but adds the machinery and the dependence on a value estimate that, if miscalibrated, steers the search wrong. The gain over self-consistency is real but smaller than its cost on many tasks, so the simpler method is often the right default.
- **Compute now versus training once.** Every method here pays its cost on every query, forever. Training the behavior in, per [Chapter 14](#), pays once and amortizes, but costs a training run and the data to do it. Elicitation is the right tool when the model is fixed, the task is new, or the volume is low; internalization wins at scale.

Two structural boundaries keep elicitation honest, and neither is a tuning knob. It cannot exceed what the fixed model can produce, only surface and select among it, so a problem the model never solves in any sample is out of reach no matter how many you draw or how hard you search. And the visible chain is a lever on accuracy, not a guaranteed account of the computation, so reading it as oversight, per [Chapter 33](#), is a claim that needs its own test, not a free byproduct of asking the model to think out loud.

Stacking them, and the move back to training

The methods compose more than they compete, and the practical recipe stacks them. Chain-of-thought is the substrate: get the model to externalize steps, by exemplars or by a step-by-step instruction. On top of it, self-consistency is the first upgrade, sample a handful to a few dozen chains and vote, because it is cheap to add and needs no other component. When a verifier exists, best-of-N over the same samples replaces the vote with a score and lifts the ceiling. A tree or a search is the heaviest tool, justified when the problem has a real branching structure and dead ends that backtracking escapes, and when the per-node self-evaluation is reliable enough to trust.

The failure modes are specific. A vote over an unreliable model can be confidently wrong, since self-consistency reports the consensus, not the truth, and a biased model agrees with itself. A weak verifier makes best-of-N worse than a single sample once N is large, because selection pressure finds the verifier's blind spots, the inference-time face of the reward hacking in [Chapter 10](#). A tree search with a miscalibrated self-evaluation spends its budget exploring bad branches and pruning good ones, so the search can underperform the plain chain it generalizes. And the cost is real on every call: a method that multiplies token count by tens for a few points of accuracy may not survive the economics of [Chapter 37](#), which is why the field's center of gravity moved toward training the behavior in.

That move is the natural close of this arc. Everything here elicits, at inference, what a fixed model already contains. The next step is to take the traces these methods produce, the verified-correct chains and the high-value search paths, and train on them so the model internalizes the behavior and needs less scaffolding at inference. That step crosses into [Chapter 14](#), and many of its ingredients, sampling many completions, scoring them with a verifier, keeping the good ones, are the same ingredients seen here, now wired into a weight update instead of a decode.

Further reading

- Wei et al., "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," 2022. [arXiv:2201.11903](#)
- Kojima et al., "Large Language Models are Zero-Shot Reasoners" (zero-shot CoT), 2022. [arXiv:2205.11916](#)

- Wang et al., "Self-Consistency Improves Chain of Thought Reasoning in Language Models," 2022. [arXiv:2203.11171](#)
- Zhou et al., "Least-to-Most Prompting Enables Complex Reasoning in Large Language Models," 2022. [arXiv:2205.10625](#)
- Yao et al., "Tree of Thoughts: Deliberate Problem Solving with Large Language Models," 2023. [arXiv:2305.10601](#)
- Xie et al., "Self-Evaluation Guided Beam Search for Reasoning," 2023. [arXiv:2305.00633](#)
- Feng et al., "Alphazero-like Tree-Search can Guide Large Language Model Decoding and Training," 2023. [arXiv:2309.17179](#)
- Cobbe et al., "Training Verifiers to Solve Math Word Problems" (best-of-N reranking with a trained verifier), 2021. [arXiv:2110.14168](#)

14. Training Models to Reason

This chapter is about reinforcement learning where the reward is a checkable ground truth rather than a learned human-preference proxy. By the end a reader can explain why a verifier beats a reward model on the tasks where one exists, how GRPO and RLOO drop the critic that PPO carries, why the o1 and R1 line produced long-horizon reasoning without step-level supervision, and where the field still disagrees about whether any new reasoning is being taught at all.

The thread that runs through every section is a single object, the verifier. Where it exists, the recipe of this chapter applies. Where it does not, you fall back to the learned reward model of [Chapter 10](#). Watch what each design choice costs once that object is fixed.

A reward you cannot game

Start with the failure that motivates everything here. A learned reward model is a proxy, and a policy trained against a proxy will find its argmax, not the true objective's. This is the over-optimization problem that haunts the RLHF of [Chapter 10](#) ([Schulman et al. 2017](#)): push hard enough on a model of human preference and the policy learns to satisfy the model rather than the human. The reward model has no ground truth to anchor it, so its blind spots become the policy's exploits.

For a large and valuable class of tasks there is a way out, because the answer can be checked. A program either passes its unit tests or does not. A math answer either equals the reference or does not. A formal proof either type checks in Lean or does not. On these tasks you can replace the learned reward with a verifier and optimize the policy against a reward that has no proxy gap to game. That move, reinforcement learning with verifiable rewards, is what this chapter owns ([Lambert et al. 2024](#)), and it is the training-time analogue of the runtime verification loop that an agent harness runs at inference.

Swapping the proxy for a checker

The loop end to end is short. Sample completions for a prompt, run each through a verifier for a scalar reward that is often just binary, and update the policy toward the rewarded completions. The training data is prompts with a way to check the answer, not prompts with reference completions. The optimizer is the PPO machinery of [Chapter 10](#) reused without change ([Schulman et al. 2017](#)): a policy, an advantage estimate, and a KL penalty against a frozen reference model. The innovation is the reward source, not the algorithm.

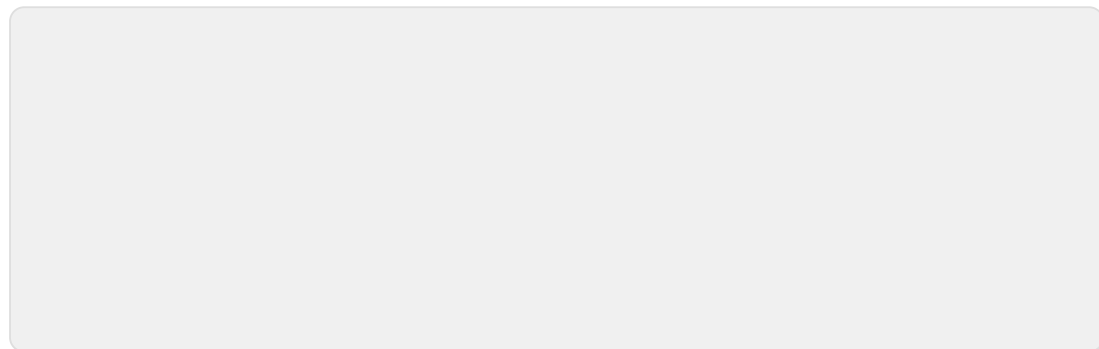


Figure 14.1. The RLVR loop. Sample completions for a checkable prompt, score each with a verifier, turn rewards into a group-relative advantage, and update the policy under a KL anchor.

Everything downstream of this loop is determined by how good the checker in the middle box is. The next section opens it up.

Three kinds of verifier

The verifier is the part you cannot buy, and it comes in three families with different cost and coverage. Executable unit tests for code are cheap and high-signal, but coverage limited, and a solution that passes the tests while being wrong will be rewarded all the same. Answer checkers for math reduce to string, numeric, or symbolic equivalence, where the hard engineering is parsing and deciding equivalence rather than the check itself. Formal proof checkers in the Lean or Isabelle style are near unspooftable, but narrow and expensive to author. Each family trades coverage against integrity, and that trade is the recurring theme of the chapter. Figure 14.2 places the three families on that axis.

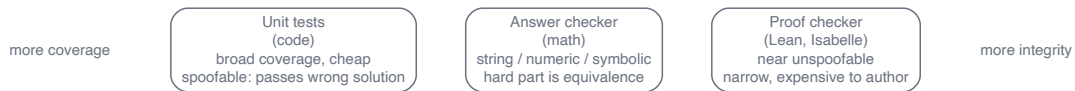


Figure 14.2. The three verifier families trade coverage against integrity. Unit tests are broad but spooftable, formal proof checkers near unspooftable but narrow.

CONSTRAINT ARROW

The shape of the RL recipe is dictated from below by whether a verifier exists. Where the answer is checkable, you get a non-gameable reward and the loop of this chapter applies. Where it is not, you fall back to the learned reward model of Chapter 10 and inherit its over-optimization. This is why frontier reasoning work concentrates on math, code, and formal proof: the availability of a ground truth at the data layer, not a preference at the training layer, is what decides which algorithm you can run. The verifier's coverage is the real budget.

Dropping the critic

With the reward source fixed, the optimizer earns one real simplification. PPO carries a value network, a critic, to estimate the baseline that lowers advantage variance, and that network doubles model memory and adds its own failure mode. Group Relative Policy Optimization, introduced for DeepSeekMath (Shao et al. 2024), drops the critic and estimates the advantage from the group of samples drawn for one prompt: normalize each reward against the group mean and standard deviation. For a group of G completions with rewards r_i ,

$$A_i = \frac{r_i - \text{mean}(r_1, \dots, r_G)}{\text{std}(r_1, \dots, r_G)}.$$

Compute both baselines on the same reward vector and watch where they differ, and what a no-spread group does to each.

```

import numpy as np

def grpo_adv(r):
    s = r.std()
    if s == 0:          # all pass or all fail: no spread, no sign
        return np.zeros_like(r)
    return (r - r.mean()) / s

def rloo_adv(r):        # baseline = mean of the other G-1 samples
    G = len(r)
    return r - (r.sum() - r) / (G - 1)

for name, r in [("mixed ", np.array([1., 0., 1., 0.])),
                ("all pass", np.array([1., 1., 1., 1.]))]:
    print(f"{name} rewards={r.tolist()}")
    print(f"    GRPO advantage = {np.round(grpo_adv(r), 3).tolist()}")
    print(f"    RLOO advantage = {np.round(rloo_adv(r), 3).tolist()}")

```

This is cheaper than PPO with a critic and fits RLVR exactly, because sampling many completions per prompt is natural when each one is cheap to verify. The KL term against the reference is kept. RLOO, REINFORCE Leave-One-Out, makes the same bet with a different baseline (Ahmadian et al. 2024): each sample's baseline is the mean reward of the other samples in the group. GRPO and RLOO differ in their bias and variance trade-offs, but they agree on the structural claim that the critic was often unnecessary overhead. Figure 14.3 shows where the three methods get the baseline that turns a reward into an advantage.

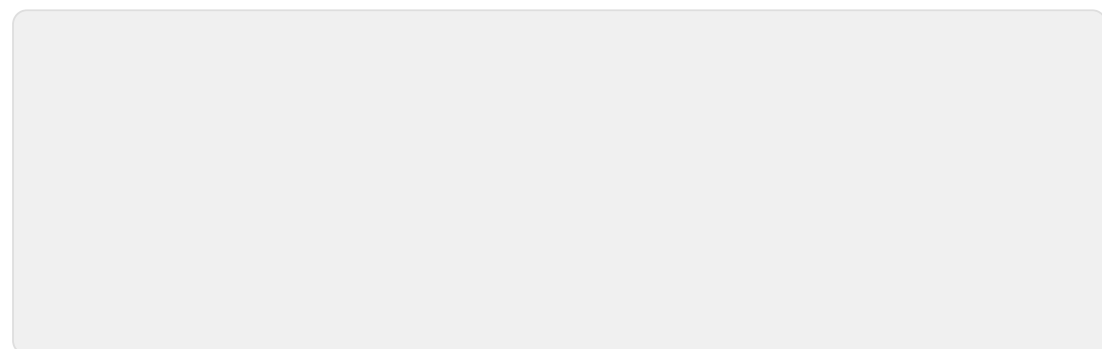


Figure 14.3. PPO learns a separate critic network to estimate the baseline. GRPO and RLOO compute it from the group of samples drawn for one prompt, so no critic is carried.

What the loop produced: o1 and R1

The result that changed the field was that this loop, run on math and code with long chains of thought allowed, produces emergent long-horizon reasoning without any step-level supervision. Self-checking, backtracking, and longer deliberation appear from an outcome-only reward. OpenAI's o1 announced the capability (OpenAI 2024). DeepSeek-R1 is the open data point that showed the recipe (Guo et al. 2025).

R1 is worth reading as two variants because the contrast is the lesson (Guo et al. 2025). R1-Zero applies RL directly to a base model with no supervised cold start, and reasoning behavior still emerges, which is the strong claim. R1 adds a short SFT cold start before the RL, drawing on the supervised fine-tuning of Chapter 9, not to teach reasoning but to fix readability and stability: the zero variant reasons in a way that is correct but hard to read and prone to language mixing. The cold start is a cosmetic and stabilizing prefix, and the reasoning gains come from the RL. Figure 14.4 contrasts the two pipelines.

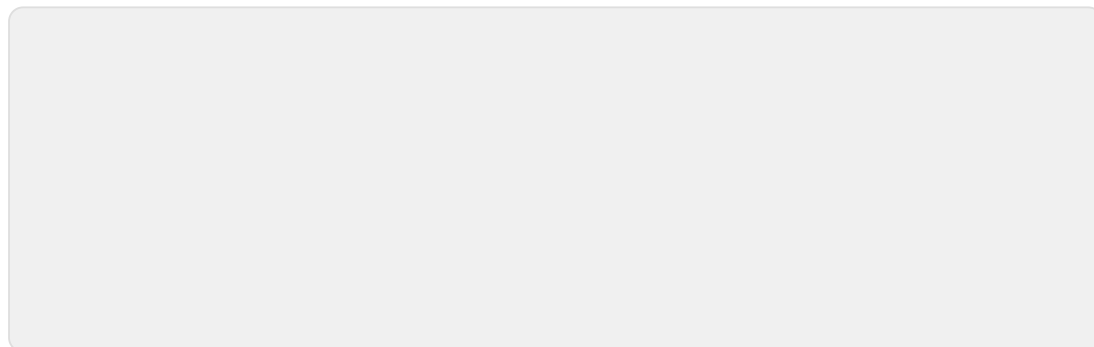


Figure 14.4. R1-Zero applies RLVR directly to the base model. R1 inserts a short SFT cold start for readability and stability before the same RL, where the reasoning gains still come from.

WHAT'S CONTESTED

Whether RLVR instills new reasoning or only amplifies what the base model already knows is unsettled. The amplification position holds that RL with verifiable rewards sharpens a distribution the base model already places mass on: it raises pass@1 by concentrating probability on reasoning paths the base model could already sample, and studies report that pass@k at large k for the base model can match or exceed the RL-trained model, which would mean RL is eliciting and reweighting rather than teaching. The instillation position points to R1-Zero, where behaviors like backtracking and self-correction grow in frequency and length over training in a way that looks like acquisition, not just reweighting (Guo et al. 2025). The disagreement is partly about measurement, since pass@k, sampling temperature, and the strength of the base model all move the verdict, and it stays live because both sides can point to real runs. Treat a claim that RLVR teaches reasoning as base-model and metric dependent, not settled. Figure 14.5 sketches the crossover the amplification position rests on.

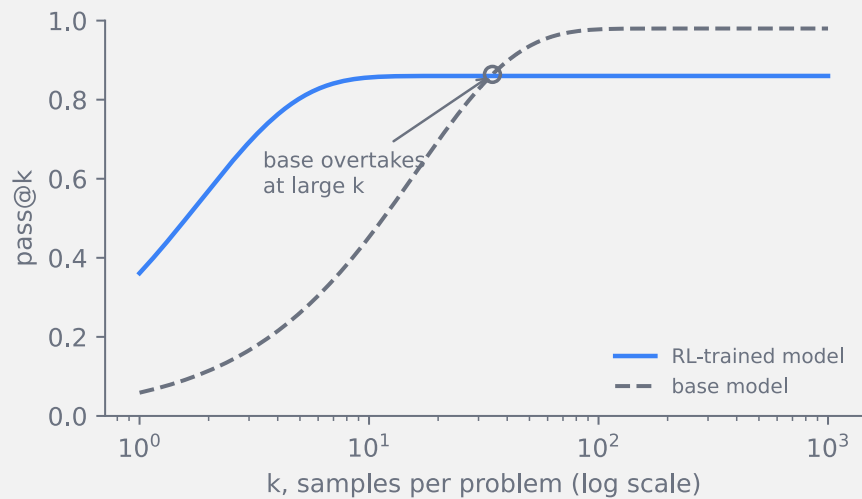


Figure 14.5. Schematic illustration of the amplification hypothesis: an RL-trained model raises pass@1 but saturates early, while the base model's pass@k keeps climbing and can overtake it at large . Idealized, not measured data.

How much credit, and how often

The supervision question has its own lineage, and it is a second axis cutting across the verifier families. Outcome rewards score only the final answer, which is cheap and is the RLVR default, but assigns credit coarsely over a long chain. Process rewards score each step through a process reward model, a PRM, which Lightman et al. trained for "Let's Verify Step by Step" (Lightman et al. 2023) and Uesato et al. compared against outcome feedback earlier (Uesato et al. 2022). A PRM gives denser credit and helps long-horizon learning, but it reintroduces exactly what the verifier removed: a learned, hackable reward. The surprise of the RL era is how far an outcome-only signal got without any process model at all.

Where it breaks, and where it pays off

The recipe is a set of balances, each with a knee. Read these as the dials you turn once the verifier is fixed.

- **Verifiable reward versus learned reward.** A verifier is hard to game in the proxy sense but only exists where ground truth is checkable, and its coverage is the new attack surface. The learned reward model of Chapter 10 works on open-ended tasks but is an over-optimizable proxy. Real recipes mix both by domain (Lambert et al. 2024).
- **Outcome versus process supervision.** Outcome rewards are cheap, robust, and sparse. Process rewards through a PRM are dense and better for long-horizon credit, but the PRM is a learned, hackable reward and expensive to label (Uesato et al. 2022). This is signal density against reward integrity.
- **Critic versus critic-free.** A value network lowers advantage variance but doubles model memory and adds a failure mode of its own. GRPO and RLOO trade that variance for simplicity and fit the many-samples shape of RLVR (Shao et al. 2024).
- **Exploration versus KL collapse.** Too little KL, or too aggressive optimization, collapses the policy onto a few high-reward modes and stops exploration, so reward rises while held-out quality falls. Too much KL and the policy never moves off the reference. The KL coefficient and clip range are the central RLVR knob. Figure 14.6 sketches the knee where training reward keeps rising while held-out quality turns down.

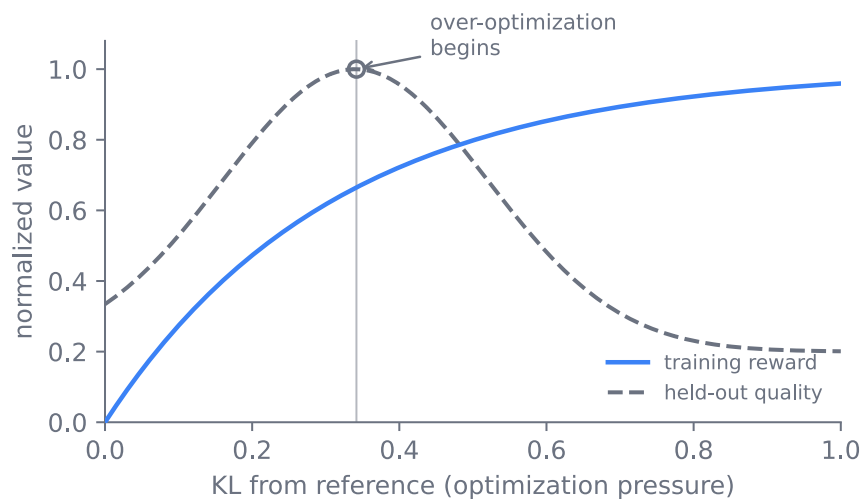


Figure 14.6. Schematic illustration of the exploration versus KL-collapse trade-off: as the policy moves further from the reference, training reward keeps climbing while held-out quality peaks and then falls into reward hacking and mode collapse. Idealized, not measured data.

- **Verifier coverage versus gameability.** A stricter, broader verifier resists gaming but costs more and may reject correct-but-unusual answers, false negatives that starve learning. A loose verifier is cheap but leaks reward to wrong answers.

Most failures here are the optimizer finding what you measured rather than what you meant. Reward hacking through verifier gaps is the central one: the policy discovers inputs the verifier scores correct that are not, tests passing on degenerate solutions, checkers fooled by formatting, hard-coded outputs for known cases. The reward to the optimizer is real and the answer to you is wrong. KL or mode collapse is the next: the policy concentrates on a few high-reward outputs, entropy craters, and held-out quality degrades while training reward climbs. Reasoning that games the checker rather than the problem is the subtle one, where the visible chain of thought steers toward whatever the verifier rewards or does not faithfully drive the answer at all.

Two boundaries keep the method honest. RLVR is strongest where ground truth is checkable and does not transfer for free to open-ended tasks, so over-indexing on it skews the model and leaves a gap that only a learned reward or human judgment fills. And over-squeezing the reward regresses the instruction-following, safety, and breadth installed in post-training, a catastrophic forgetting that the KL anchor here only partly prevents and that [Chapter 27](#) must catch.

The self-improvement loop is the payoff when a verifier is sound. Generate candidate training data with the model, filter by the verifier to keep only verified-correct traces, and feed those into another SFT or RL round, then iterate. This is distinct from the distillation of [Chapter 12](#) because the filter is a checker, not a stronger teacher, so the model can bootstrap past its current self. The hard limit is that it needs a verifiable signal to close the loop: a sound verifier gives a genuine bootstrap, an unsound one amplifies its own blind spots. Running the sampling and rollout fleet that feeds all of this at scale belongs to [Chapter 8](#), and the inference-time analogue of spending more compute to reason better is [Chapter 15](#).

Further reading

- Shao et al., "DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models" (introduces GRPO), 2024. [arXiv:2402.03300](https://arxiv.org/abs/2402.03300)
- DeepSeek-AI, "DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning," 2025. [arXiv:2501.12948](https://arxiv.org/abs/2501.12948)

- Schulman et al., "Proximal Policy Optimization Algorithms" (PPO; introduced in [Chapter 10](#), reused here), 2017. [arXiv:1707.06347](#)
- Lightman et al., "Let's Verify Step by Step" (process reward models / PRMs), 2023. [arXiv:2305.20050](#)
- Uesato et al., "Solving math word problems with process- and outcome-based feedback" (process vs outcome supervision), 2022. [arXiv:2211.14275](#)
- Ahmadian et al., "Back to Basics: Revisiting REINFORCE-Style Optimization for Learning from Human Feedback in LLMs" (RLOO), 2024. [arXiv:2402.14740](#)
- OpenAI, "Learning to Reason with LLMs" (the o1 line; an official announcement / system report, not a peer-reviewed paper), 2024. [openai.com](#)
- Lambert et al., "Tulu 3: Pushing Frontiers in Open Language Model Post-Training" (names/positions RLVR), 2024. [arXiv:2411.15124](#)

15. The Inference-Time Scaling Paradigm

Training in [Chapter 14](#) spends compute once to bake reasoning into the weights. This chapter is about the other lever: spending more compute at inference, after the weights are frozen, to make a fixed model answer harder questions. By the end a reader can explain why repeated sampling buys coverage but not an answer, why sequential revision and parallel search are different shapes of test-time compute with different ideal regimes, how compute-optimal allocation routes a budget by prompt difficulty, and why the whole curve bends back down when the thing that picks the answer is imperfect.

The one knob left after training

A trained model has one knob left at inference: how much compute to spend per prompt. The cheapest answer is one greedy decode. The question this chapter answers is what you get for spending more, a thousand samples instead of one, or a long deliberate chain instead of a short one, and whether that spend is ever a better deal than training a larger model in the first place.

The reason the question is live is that the two spends are fungible. A larger model costs more to train and more per token forever. Test-time compute costs nothing at training and is paid only on the prompts that need it, which is most of the value on a long tail of hard prompts and none of it on the easy majority. If a small model plus extra inference can match a large model on the hard prompts, the serving economics of [Chapter 16](#) change, because you move cost from a fixed asset you pay for on every request to a variable you spend only when a prompt earns it.

Two shapes of test-time compute

Test-time compute comes in two shapes, and the distinction organizes the whole chapter. Parallel scaling draws many independent samples and combines them. Sequential scaling spends the compute in one growing chain, where each step conditions on the last, revising or extending a single line of thought. [Snell et al.](#) frame these as revising the proposal distribution sequentially versus searching against a verifier in parallel, and the central empirical claim is that neither dominates: the better shape depends on the prompt ([Snell et al. 2024](#)).

Sequential scaling spends the budget the second way. Instead of k independent shots, the model keeps thinking in one trajectory, and the compute buys length and revision rather than breadth. The reasoning models of [Chapter 14](#) already do this natively: a long chain of thought is sequential test-time compute that the RL of that chapter taught the model to produce. [s1](#) shows the lever in its barest form with budget forcing, a decoding-time intervention that controls how long the model thinks by suppressing the end-of-thinking token and appending the word "Wait" to push the model to continue, or by forcing termination to cap the spend ([Muennighoff et al. 2025](#)). Lengthening the chain this way lets the model catch and fix its own errors, and accuracy rises with the forced budget. Sequential and parallel are composable: you can draw several long chains and select among them, and [Snell et al.](#) study exactly this two-dimensional budget ([Snell et al. 2024](#)). [Figure 15.1](#) lays out the two shapes and where the selector decides whether coverage becomes accuracy.

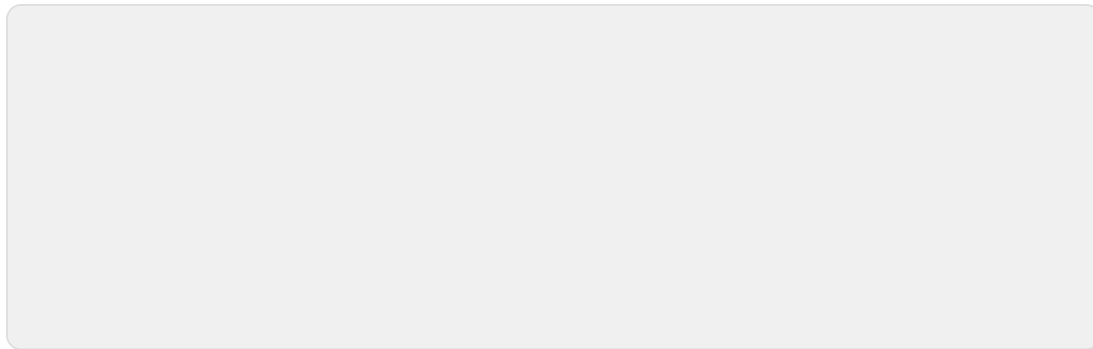


Figure 15.1. The two shapes of test-time compute. Parallel sampling raises coverage, but a selector decides whether that coverage becomes accuracy: a verifier converts it directly, while majority vote or a reward model plateaus past hundreds of samples.

Coverage is not accuracy

The parallel story starts with the simplest possible method. Draw k samples and ask what fraction of problems are solved by at least one of them. Brown et al. call this coverage, and it is the ceiling on what any selection rule could extract from the samples (Brown et al. 2024). Coverage rises smoothly with k across four orders of magnitude, well modeled as an exponentiated power law, which is the inference-time analogue of the training scaling laws in Chapter 3 (Brown et al. 2024). The key word is coverage, not accuracy. A correct sample exists in the set; that is not the same as knowing which one it is.

That gap is the design problem of parallel scaling. To turn coverage into an answer you need a selector, and the selector is where the difficulty lives. When the task has an automatic verifier, the unit tests and answer checkers and proof checkers of Chapter 14, selection is free: run the verifier on each sample and keep one that passes. Coverage converts directly into accuracy, which is why repeated sampling pays most in code and formal math. When there is no automatic verifier, you fall back to a heuristic selector, majority vote over the answers or a learned reward model scoring each sample. These work to a point and then plateau: Brown et al. report that majority vote and reward models stop scaling past a few hundred samples even as coverage keeps climbing, because the selector cannot tell the rare right answer from the many confident wrong ones (Brown et al. 2024). The whole chapter turns on this one gap between a correct answer existing and a system finding it. Figure 15.2 makes the gap visible: coverage climbs smoothly with k , a verifier realizes almost all of it, and a heuristic selector tracks coverage early and then plateaus far below it.

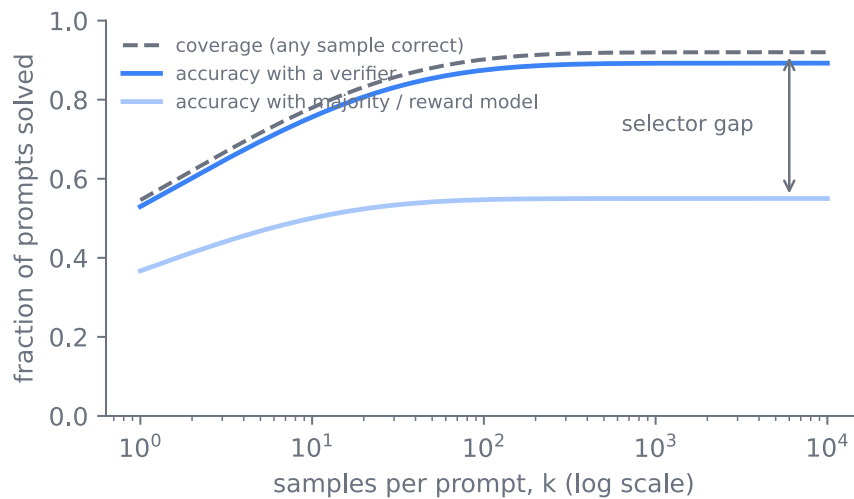


Figure 15.2. Schematic of the coverage-is-not-accuracy gap in parallel test-time scaling. Coverage rises smoothly with the sample count toward a ceiling; a verifier converts it into accuracy almost one-for-one, while a heuristic selector (majority vote or a reward model) plateaus past hundreds of samples. Idealized curves, after Brown et al. (2024).

From tricks to a compute axis

The methods are older than the paradigm. Self-consistency, sampling several chains and taking a majority vote over final answers, is parallel scaling with a majority selector and predates the reasoning-model era (Wang et al. 2022). Best-of- n against a reward model is parallel scaling with a learned selector. Process reward models, trained in "Let's Verify Step by Step" and used as the dense verifier in Chapter 14, are what makes verifier-guided search over partial chains work, scoring steps rather than only final answers so a search can prune early (Lightman et al. 2023). What changed in 2024 was the framing: these stopped being tricks and became a compute axis with its own scaling laws.

Brown et al. established the parallel law: coverage as a smooth, predictable function of sample count, and the sharp dependence of its payoff on whether a verifier exists (Brown et al. 2024). Snell et al. asked the allocation question. Given a fixed test-time budget, how should you split it, more parallel samples, more sequential revision, a deeper verifier search (Snell et al. 2024).

Routing the budget by difficulty

Snell et al. showed the answer depends on prompt difficulty. Easy prompts benefit from sequential refinement of an already-good first guess; hard prompts benefit from broader parallel search. Routing the budget by difficulty, their compute-optimal strategy, beat a best-of- n baseline by more than four times in efficiency, and on prompts where a small model already had a foothold, a FLOPs-matched small model with extra test-time compute outperformed a model roughly fourteen times larger (Snell et al. 2024). That is the constraint arrow of the chapter made quantitative: inference compute substituting for parameters. Figure 15.3 traces how the budget is routed by difficulty.

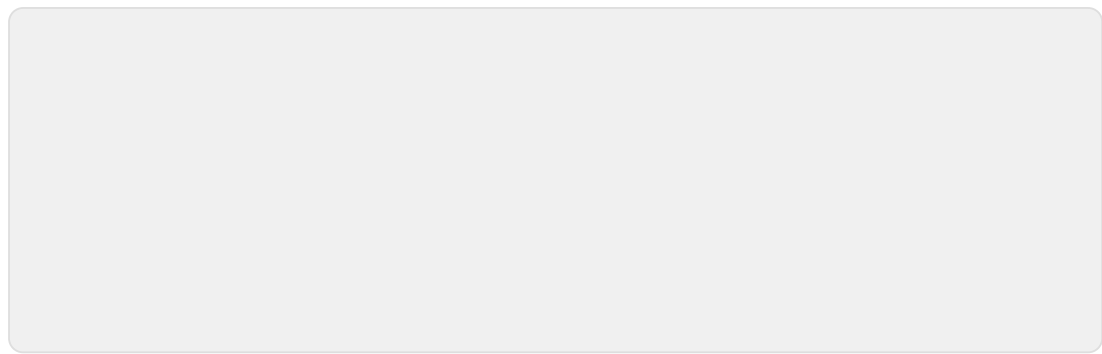


Figure 15.3. Compute-optimal allocation routes a fixed test-time budget by estimated prompt difficulty. Easy prompts spend it on sequential refinement of a good first guess, hard prompts on broad parallel search, and prompts the small model never solves return zero coverage no matter the spend. Following Snell et al., 2024.

Less data, more inference

The s1 and LIMO results sharpened a different edge. s1 reached strong competition-math performance by supervised fine-tuning on only about a thousand carefully chosen reasoning traces and then scaling at test time with budget forcing (Muennighoff et al. 2025). LIMO made the less-is-more claim explicit with a few hundred examples, arguing that when a base model already encodes the knowledge from pre-training, a small set of high-quality demonstrations is enough to elicit the reasoning, and the heavy lifting moves to inference (Ye et al. 2025). Both point the same way: the capacity may already be latent in the weights, and test-time compute is how you call it out. This connects to the contested question in Chapter 14 about whether RL teaches new reasoning or elicits what is already there, seen now from the inference side.

WHAT'S CONTESTED

How far test-time compute keeps paying off is bounded by the quality of the thing that selects or guides, and that bound is not a footnote. With a perfect verifier, more samples never hurt: coverage rises monotonically and selection is free, so the curve only bends toward the ceiling. With an imperfect verifier or a heuristic selector, the curve can stop improving or turn down. More samples give the selector more confident-but-wrong candidates to be fooled by, and an optimization-against-the-selector effect appears that is the inference-time twin of the reward hacking in Chapter 10: best-of- n against a flawed reward model can select the sample that games the model rather than solves the problem, and past some n the expected quality falls. Brown et al. see majority vote and reward-model selection plateau past hundreds of samples while coverage keeps climbing, which is the same gap from the other side: the answer is in the set and the selector cannot find it (Brown et al. 2024). So the honest claim is conditional. Test-time compute scales cleanly where verification is cheap and sound, and the cleaner the verifier, the longer the curve pays. Where it is not, the scaling is bounded by selector quality, and more compute is not free improvement. Figure 15.4 shows both regimes: a perfect verifier rises monotonically, while an imperfect one peaks and then turns down as larger n feeds the selector more confident-but-wrong candidates.

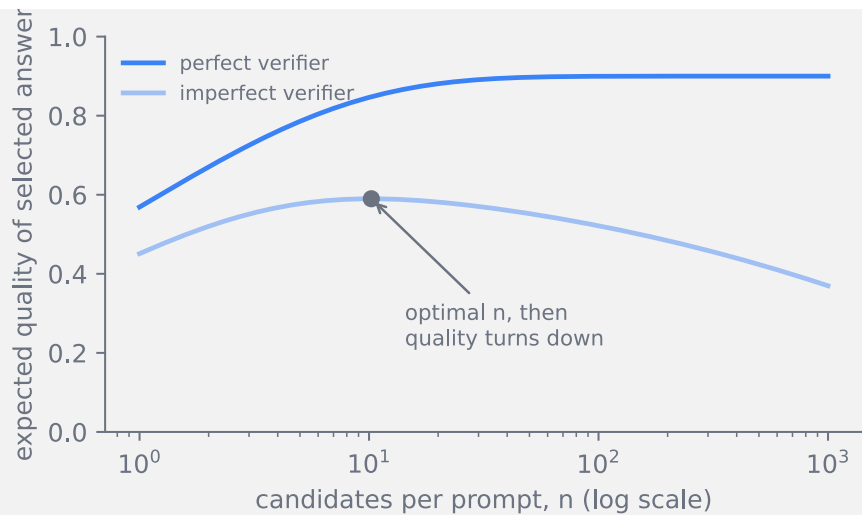


Figure 15.4. Schematic of best-of- against verifiers of differing quality. A perfect verifier rises monotonically toward the coverage ceiling, so more samples never hurt; an imperfect verifier peaks at some optimal and then declines as the search optimizes against the selector, the inference-time twin of reward hacking. Idealized curves, after Snell et al. (2024) and Brown et al. (2024).

Raise `gameable`, the fraction of wrong samples the flawed selector scores above a correct one, and watch the imperfect-selector curve peak earlier and then fall, while coverage and the perfect verifier never turn down.

```
import numpy as np
import matplotlib.pyplot as plt
rng = np.random.default_rng(0)
p, gameable, noise, trials, ks = 0.3, 0.05, 0.4, 4000, range(1,
cov, heuristic = [], [])
for k in ks:
    correct = rng.random((trials, k)) < p # whi
    confident_wrong = (~correct) & (rng.random((trials, k)) < g
    base = np.where(confident_wrong, 1.5, np.where(correct, 1.0
    score = base + rng.normal(0, noise, (trials, k)) # sel
    cov.append(np.mean(correct.any(axis=1))) # a r
    picked = score.argmax(axis=1) # se
    heuristic.append(np.mean(correct[np.arange(trials), picked]
plt.plot(list(ks), cov, label="coverage / perfect verifier")
plt.plot(list(ks), heuristic, label=f"imperfect selector (gamea
plt.xlabel("samples k"); plt.ylabel("accuracy"); plt.legend();
print("best heuristic k:", list(ks)[int(np.argmax(heuristic))])
```

The balances and their knees

The paradigm is a set of balances, each with a knee.

- **Parallel versus sequential.** Parallel sampling is embarrassingly concurrent and latency-friendly, you can decode k samples at once, but it needs a selector and wastes compute on redundant near-duplicates. Sequential revision uses compute more efficiently on prompts where a first guess is close, but it is inherently serial and pays full latency. Snell et al. show the right mix is difficulty-dependent, not a constant (Snell et al. 2024).
- **Coverage versus selection.** More samples always raise coverage, but the realized accuracy is capped by the selector. With a verifier the cap is the coverage ceiling; without one the cap is wherever majority vote or the reward model plateaus. Spending past that point buys nothing.
- **Test-time compute versus model size.** Extra inference can substitute for parameters on the hard tail, shifting cost from a fixed training-and-serving asset to a per-prompt variable. But the substitution has a regime: it works when the small model already has nonzero coverage, and it fails on prompts the small model never solves no matter how many tries, where only a stronger model or better training helps.
- **Data efficiency versus compute.** s1 and LIMO trade training data for test-time compute, eliciting latent capability with a tiny SFT set plus inference scaling (Muennighoff et al. 2025; Ye et al. 2025). The bet holds only when the base model already encodes the domain; on genuinely new capability there is nothing latent to elicit.
- **Latency and cost versus accuracy.** Every method here multiplies the per-prompt compute and, for sequential methods, the latency. The product decision is which prompts deserve the spend, which is exactly what compute-optimal allocation formalizes.

CONSTRAINT ARROW

This chapter reaches down to Chapter 16 and back up to Chapter 3. Test-time scaling only makes sense if the serving layer can deliver many samples or long chains cheaply, so the batching, key-value cache, and decoding speed of Chapter 17 and Chapter 18 set the real price of a sample, and that price decides where compute-optimal allocation stops. And it closes the loop opened in Chapter 3: the over-training argument there said training compute substitutes for serving cost, while this chapter says inference compute substitutes for parameters. Together they say the model size is jointly set by training budget, serving cost, and how much reasoning you intend to buy at inference, not by any one of them alone.

Building it

The operational core is small. A parallel scaler is a sampling loop plus a selector: draw k completions at a temperature high enough for diversity, then either run the verifier and keep a passing one, or take a majority vote, or score with a reward model and take the best. The verifier path is the one that scales, so the engineering effort goes where Chapter 14 put it, into making the checker sound and broad. A sequential scaler is a controlled decode: detect when the model tries to stop and decide whether to let it, the budget-forcing intervention, suppressing the stop token and injecting a continuation cue to extend, or forcing a final answer to cap.

```
# Parallel scaling: coverage is free, selection is the hard part.
def best_of_k(prompt, k, select):
    samples = [model.sample(prompt, temperature=0.8) for _ in range(k)]
    return select(samples) # verifier > reward model > majority vote
```

The failure modes follow from the design. Selecting with an imperfect verifier is the central one: more samples feed a flawed selector more chances to pick a confident wrong answer, the inference-time reward hacking named in the contested box, so the curve flattens or dips and more compute is wasted compute. Coverage without a selector is the related trap: the right answer is provably in the set and the system still returns a wrong one, which looks like a model failure but is a selection failure. Sequential over-thinking is the third: budget forcing past the useful point can make a model second-guess a correct answer into a wrong one, so the forced budget itself has a knee. And the substitution fails silently on prompts of zero coverage, where no amount of test-time compute helps because no sample is ever correct, and the only fix is upstream in the model.

The capability, efficiency, and trust lens closes the chapter. Test-time scaling is a capability lever that turns a frozen model into a stronger one on the hard tail without retraining. It is an efficiency lever that moves cost off the fixed asset onto the prompts that earn it, governed by the serving prices above. And it is a trust hazard, because its entire payoff rides on the selector, and a selector you do not trust converts more compute into more confidently wrong answers. The verifier quality that bounded the training loop in [Chapter 14](#) bounds the inference loop here too: the same ground truth that makes RL safe is what makes test-time scaling pay.

Further reading

- Brown et al., "Large Language Monkeys: Scaling Inference Compute with Repeated Sampling," 2024. [arXiv:2407.21787](#)
- Snell et al., "Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters," 2024. [arXiv:2408.03314](#)
- Muennighoff et al., "s1: Simple test-time scaling," 2025. [arXiv:2501.19393](#)
- Ye et al., "LIMO: Less is More for Reasoning," 2025. [arXiv:2502.03387](#)
- Wang et al., "Self-Consistency Improves Chain of Thought Reasoning in Language Models," 2022. [arXiv:2203.11171](#)
- Lightman et al., "Let's Verify Step by Step" (process reward models for verifier-guided search), 2023. [arXiv:2305.20050](#)

16. The Serving Problem

A trained model is a file; a served model is a system with a clock and a budget. By the end of this chapter a reader can explain why inference splits into a prefill phase and a decode phase with opposite hardware appetites, why latency and throughput pull against each other, why the real objective is goodput rather than either one alone, and why the key-value cache is the scarce resource that the whole serving layer exists to schedule.

That last point is the spine of this chapter. Everything else, the two phases, the latency metrics, the entire history of serving engines, and every operator knob, turns out to be a fight over one block of accelerator memory. We follow that thread from where the tension is born, through the objective that prices it, to the resource that bounds it, and finally through the engines and knobs that have learned to live inside the bound.

A file that has to keep time

Pre-training spends compute once to make a weights file. Serving spends compute forever, every time a user sends a request, and it spends it under a clock the user can feel. The problem is no longer minimizing a loss. It is running a fixed set of weights against a stream of requests so that each request returns fast enough and the cluster stays full enough to pay for itself.

Two latencies define "fast enough," and they are not the same number. Time to first token (TTFT) is how long a user waits before the first piece of the answer appears. Time per output token (TPOT), sometimes called inter-token latency, is how long each subsequent token takes once the stream has started. A chat interface needs a short TTFT so the response feels prompt; a long generation needs a short TPOT so the text does not stutter. These two clocks are not arbitrary product targets. They fall out of the way a single forward pass divides into two phases with opposite hardware appetites.

Two phases, two appetites

When a request arrives, the model first reads the entire prompt. This is the **prefill** phase: all prompt tokens are processed together in one pass, every position attending to every earlier position, which is a sequence of large matrix-matrix multiplications. Prefill is compute-bound. It saturates the accelerator's arithmetic units, and its cost grows with prompt length. TTFT is essentially the time prefill takes.

Then the model emits the answer one token at a time. This is the **decode** phase: each step feeds the single most recent token through the network to produce the next one, a sequence of thin matrix-vector multiplications. Decode is memory-bandwidth-bound, not compute-bound. The arithmetic per step is tiny, but every step must stream the full weight matrices and the growing cache of past keys and values out of memory, and that streaming, not the math, dominates the time. This roofline split, prefill living against the compute ceiling and decode against the bandwidth ceiling, is the organizing fact of inference performance (Yuan et al. 2024; Pope et al. 2022). TPOT is essentially the per-step decode latency, and an idle decode step leaves most of the accelerator's compute unused.

Figure 16.1 traces one request through both phases and lines up, for each, the shape of its arithmetic, the resource it saturates, and the latency metric it sets.

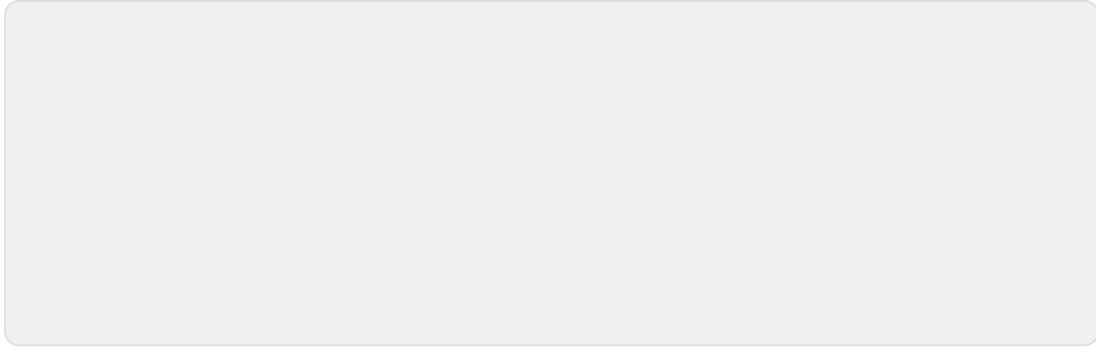


Figure 16.1. One request through prefill and decode. The two phases saturate different resources and set different latency metrics.

A reader may wonder why this split matters so much. It matters because the two phases want opposite things from the hardware, and they run on the same hardware. Prefill wants to be fed enough work to keep the arithmetic units busy. Decode wants more requests batched together so that one expensive weight-streaming pass produces tokens for many users at once. Batching is the lever that turns a bandwidth-bound decode into a compute-efficient one, because the weights are read once and amortized across every request in the batch (Pope et al. 2022). So throughput, tokens per second across all users, rises with batch size, while the latency any single user sees can degrade as the batch fills. The serving problem is this tension, made concrete: every later section is an attempt to win throughput from batching without spending a single user's latency budget.

The lie of a single number

It is tempting to chase one headline metric, and both obvious candidates lie. Throughput counts every token the cluster produces. Latency counts how long one user waits. Neither is the business objective. A server can post enormous throughput by batching so aggressively that every individual request misses its deadline, and it can post tiny latency by running one request at a time on idle hardware. The objective that survives contact with reality is **goodput**: the rate of requests completed *within their latency targets*. Tokens produced after a request has already blown its TTFT or TPOT budget are not useful work. Goodput is throughput counted only over requests that met their service-level objectives (SLOs).

Goodput reframes every later decision. The scheduler's job is to pack as many requests as possible onto the accelerators without pushing any of them past their SLO. That is a packing problem with a hard constraint, and the constraint is set by memory. To see why, we have to ask what decode is forced to keep around.

The cache is the constraint

Key-value cache memory against context length. Drag the sliders for layers, key-value heads, and batch size, and watch the cache outgrow the weights.

Attention at step t needs the keys and values of all earlier positions, so rather than recompute them each step, the server caches them. This is the **KV cache**. Its size is fixed by the architecture, not by the serving layer:

$$\text{KV bytes} = 2 \cdot L \cdot n_{\text{kv}} \cdot d_{\text{head}} \cdot b_{\text{dtype}} \cdot T$$

where L is the number of layers, n_{kv} the number of key-value heads, d_{head} the head dimension, b_{dtype} the bytes per element, and T the number of tokens cached. The factor of

two is for keys and values. The cache grows linearly with sequence length and linearly with the number of concurrent requests, and it lives in the same scarce accelerator memory that holds the weights. A reader can now see the squeeze: the weights take a fixed slice of memory, and whatever is left over is divided among the KV caches of the requests currently in flight. The number of requests you can batch, and therefore your throughput and goodput ceiling, is set by how much KV cache memory remains and how efficiently you use it. Figure 16.3 shows the partition, and Figure 16.2 makes the linear growth and its consequence concrete: each request's cache rises in a straight line with context length, and the leftover pool fixes how many such requests fit at once.

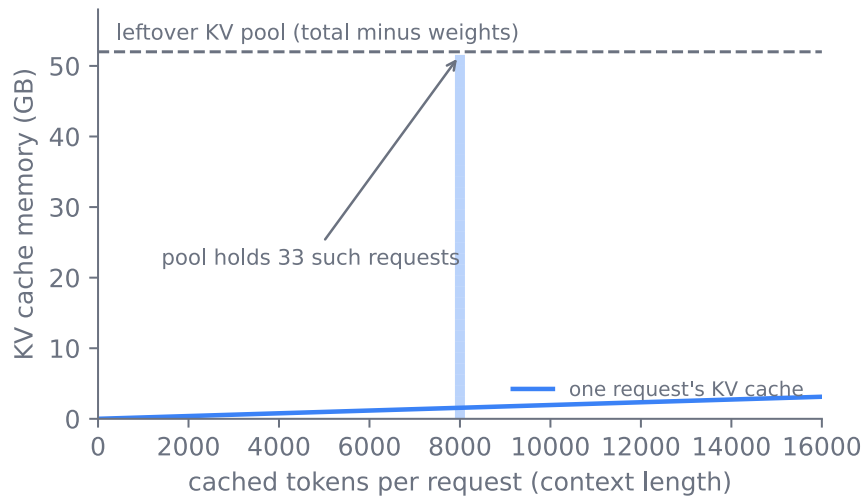


Figure 16.2. Schematic of the KV-cache memory squeeze: one request's cache grows linearly with its context length, and the fixed leftover pool, total accelerator memory minus the weights slice, caps how many concurrent requests fit. Idealized values, after Kwon et al. (2023).

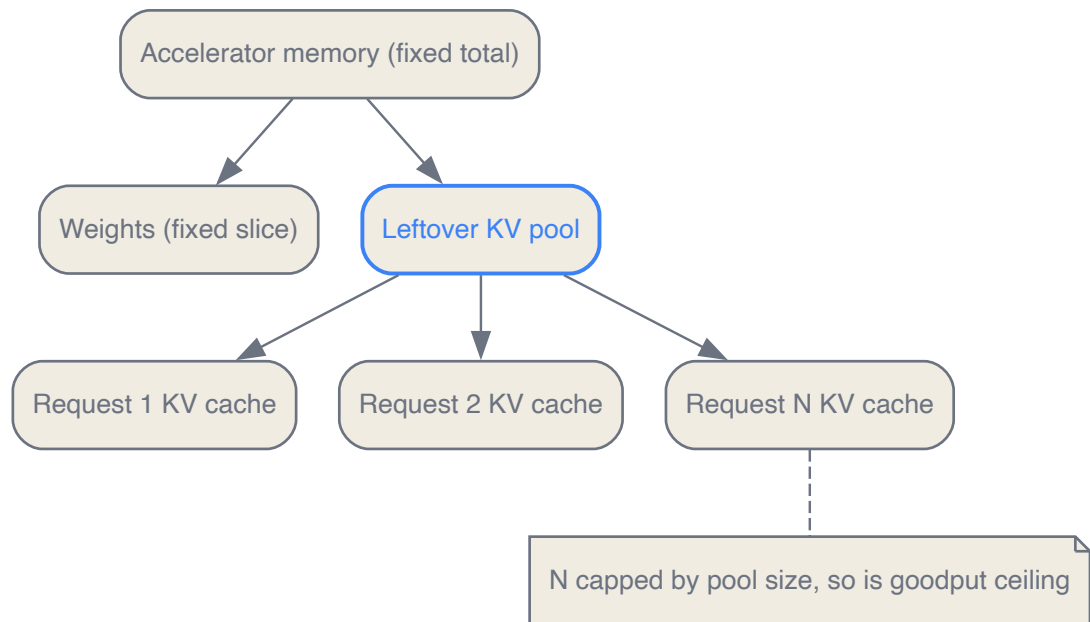


Figure 16.3. Accelerator memory splits into a fixed weights slice and a leftover KV pool shared by in-flight requests. The pool caps how many requests can be batched, and so the goodput ceiling.

This is why the architecture chapter's attention variants matter to a serving engineer. Multi-query and grouped-query attention shrink n_{kv} , and multi-head latent attention compresses the cached state, each cutting the KV bytes per token directly. They were designed, in part, to

relax exactly this serving constraint, and [Chapter 20](#) returns to what happens when T grows into the hundreds of thousands.

CONSTRAINT ARROW

The KV cache whose size [Chapter 6](#) fixes, through the layer count, the head geometry, and the choice of multi-query, grouped-query, or latent attention, is the exact resource this serving layer must schedule. Nothing here is free to redesign that cache; the architecture handed it down, and the scheduler lives within it. The arrow also runs the other way and closes a loop opened in [Chapter 3](#): the decode cost measured here, paid once per token for the lifetime of the model, is what justifies over-training a smaller model far past its compute-optimal point. A token is cheap to train and expensive to serve, so the serving layer's economics reach back up the stack and reshape a decision that looked like pure pre-training.

Four fights over one resource

With the cache identified as the binding constraint, the history of serving engines reads as a sequence of fights to waste less of it. The serving stack got its present shape by removing one source of waste at a time, and each removal was a scheduling idea before it was a kernel.

Stop holding a finished request's slot. The first systems served requests the way a web server serves HTTP: collect a batch, run it to completion, return the batch. This is request-level batching, and it wastes the accelerator badly, because a short generation in the batch finishes early and then idles while the longest generation runs on, holding its slot. Orca replaced request-level batching with **iteration-level scheduling**: the scheduler decides what to run at the granularity of a single decode step, so a finished request leaves the batch immediately and a newly arrived one can join without waiting for the rest ([Yu et al. 2022](#)). This idea, widely called continuous batching, is the foundation every modern engine builds on.

Stop reserving cache you will not use. Iteration-level scheduling exposed the next bottleneck, which was memory. Continuous batching can admit a request the instant a slot opens, but only if there is KV cache memory to hold it, and early engines allocated that memory in one contiguous block per request sized for the maximum possible length. Most requests are far shorter than the maximum, so most of that reserved memory sat empty, fragmenting the pool and capping the batch size far below what the hardware could run. PagedAttention and the vLLM engine borrowed virtual memory's paging from operating systems: store the KV cache in fixed-size blocks that need not be contiguous, allocate blocks on demand as a sequence grows, and let a block table map logical positions to physical blocks ([Kwon et al. 2023](#)). Fragmentation collapses, the batch grows, and throughput rises by a reported two to four times at the same latency ([Kwon et al. 2023](#)). The mechanism of that paging, and how it enables prefix sharing across requests, is the subject of [Chapter 17](#); here it is enough to see that the memory pressure named above forced the design.

Stop letting a prefill stall the decodes. The most recent turn confronts the prefill-decode tension head-on. Admitting a new request runs an expensive prefill that competes with the decodes already in flight: favor prefill and TTFT improves while running requests stutter; favor decode and TTFT climbs while ongoing streams stay smooth. Because a prefill is a large compute burst and a decode step is a small one, mixing them in a batch lets a prefill stall the decodes waiting behind it, spiking their TPOT. Two families of answer emerged. Sarathi-Serve keeps the phases on the same hardware but splits each prefill into chunks and interleaves them with ongoing decodes, so no decode is stalled by a long prompt ([Agrawal et al. 2024](#)). DistServe goes the other way and **disaggregates** the phases onto separate pools of accelerators, so prefill interference cannot touch decode latency at all, at the cost of shipping the KV cache between pools ([Zhong et al. 2024](#)). Both are explicitly optimizing goodput rather than raw throughput, which is why chunked prefill and disaggregation are the two principled ways to stop having to choose between TTFT and TPOT.

WHAT'S CONTESTED

Whether to disaggregate prefill and decode onto separate hardware or to keep them together and interleave is genuinely unsettled. Disaggregation (DistServe) eliminates prefill-decode interference and lets each phase scale and parallelize independently, which wins when the time-per-output-token target is tight and the time-to-first-token target is loose (Zhong et al. 2024). Aggregation with chunked prefill (Sarathi-Serve) keeps the KV cache local and avoids the cost of transferring it between pools, which wins under the opposite SLO balance and at lower request rates (Agrawal et al. 2024). Recent work argues the choice is not binary and that a scheduler should switch between or unify the two modes as load and SLOs shift. The right default depends on the model size, the interconnect bandwidth between pools, the request mix, and which SLO is the binding one, so treat disaggregation as a load-and-SLO-dependent choice rather than a settled winner.

Where the remaining knobs trade

The two prefill-decode answers above resolve one of serving's trade-offs. Three more remain, and each one trades latency, throughput, or memory against another, with the KV pool sitting underneath all of them.

- **Batch size: throughput versus latency.** A larger decode batch amortizes the weight-streaming cost across more requests and raises throughput, but it consumes more KV cache memory and lengthens each step, raising TPOT. The right batch is the largest one that still meets the TPOT SLO, and it moves as request lengths change. Figure 16.4 draws the two curves against batch size: throughput climbs with diminishing returns toward a bandwidth-bound ceiling while TPOT rises, and the chosen batch sits at the point where TPOT meets its budget.

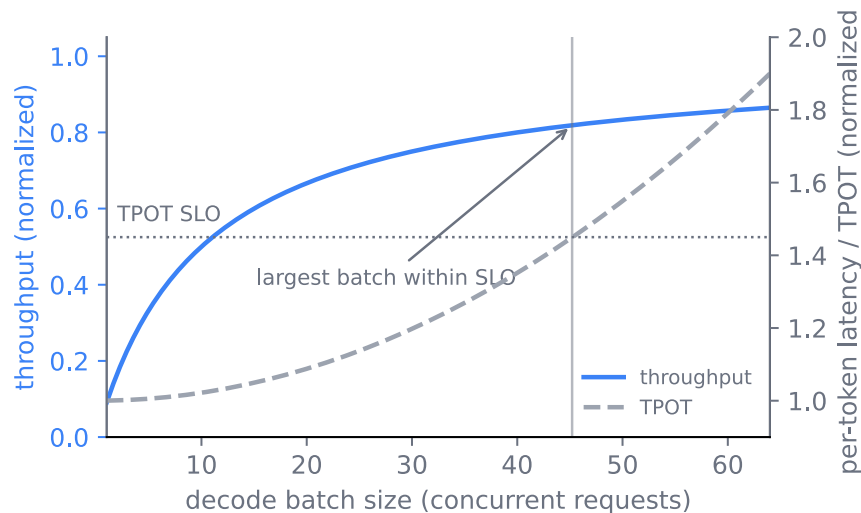


Figure 16.4. Schematic of the decode batch-size trade-off: throughput rises with diminishing returns toward a memory-bandwidth ceiling while per-token latency (TPOT) climbs, so the best batch is the largest one still under the TPOT SLO. Idealized values, after Pope et al. (2022).

- **KV cache memory versus admitted requests.** Memory spent on weights cannot hold KV cache, and memory spent caching a long-context request cannot admit a new one. When the pool fills, an engine must either reject or queue new requests, or evict and later recompute the cache of a paused one, trading memory pressure for recompute cost.

- **Latency floor versus utilization.** Holding accelerators idle in reserve protects TTFT against bursts but wastes the very compute serving exists to monetize. Goodput is the function that prices this trade, because it counts utilization only when it produces in-SLO completions.

The loop, and what it reports

A serving engine is, at its core, a loop that on each iteration chooses a set of requests to run, runs one model step on them, and frees the resources of any that finished. The shape below is the continuous-batching loop that Orca introduced and every modern engine refines (Yu et al. 2022).

```
# One iteration of a continuous-batching scheduler (sketch).
while running or waiting:
    # Admit new requests while KV-cache blocks are free and SLOs allow
    while waiting and kv_pool.can_allocate(waiting.peek()):
        running.add(waiting.pop())          # its prefill joins this batch
    batch = scheduler.select(running)       # may chunk prefills, cap batch size
    outputs = model.step(batch)             # one prefill/decode pass
    for req in batch:
        req.append(outputs[req])
    if req.is_finished():
        kv_pool.free(req.blocks)           # return blocks to the pool
        running.remove(req)
```

The load-bearing decisions are not in the model step but in the two lines that admit requests and select the batch. `kv_pool.can_allocate` is where the memory constraint becomes a scheduling decision, and `scheduler.select` is where the latency-throughput trade is struck, by capping batch size for TPOT, by chunking prefills so they do not stall decodes, and by ordering against SLOs. The corresponding logic in a real engine lives in the engine step, for example `vllm.LLMEngine.step`, `vllm/engine/llm_engine.py`, with the block accounting in the cache manager.

The operational metrics that matter follow directly from the problem. TTFT and TPOT are measured per request and tracked as percentiles, not means, because a tail miss is an SLO miss. Goodput is the headline number, computed as in-SLO completions per second. KV cache utilization and the preemption rate, how often the engine has to pause and later restore a request because memory ran out, tell you whether the memory constraint or the compute constraint is currently binding. When preemptions climb, the cache is the bottleneck and the fixes are architectural and in [Chapter 17](#); when the accelerators are full but latency is fine, throughput is the frontier and the fixes are in [Chapter 18](#) and [Chapter 19](#).

Through the lens this book carries: serving is where **capability** meets its price, because a model only delivers a behavior if it can be served inside a latency budget; **efficiency** is the entire game, since goodput per accelerator is what makes a capability affordable; and **trust** enters as the SLO itself, the promise that a request will return on time, which the scheduler must keep under load it did not choose.

Further reading

- Yu et al., "Orca: A Distributed Serving System for Transformer-Based Generative Models," OSDI 2022. usenix.org
- Kwon et al., "Efficient Memory Management for Large Language Model Serving with PagedAttention" (vLLM), SOSP 2023. [arXiv:2309.06180](https://arxiv.org/abs/2309.06180)
- Agrawal et al., "Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve," OSDI 2024. [arXiv:2403.02310](https://arxiv.org/abs/2403.02310)
- Zhong et al., "DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving," OSDI 2024. [arXiv:2401.09670](https://arxiv.org/abs/2401.09670)
- Yuan et al., "LLM Inference Unveiled: Survey and Roofline Model Insights," 2024. [arXiv:2402.16363](https://arxiv.org/abs/2402.16363)
- Pope et al., "Efficiently Scaling Transformer Inference," 2022. [arXiv:2211.05102](https://arxiv.org/abs/2211.05102)

17. Memory and Scheduling

A serving engine spends most of its accelerator memory on one growing object, the key-value cache, and most of its scheduling effort deciding which requests share a forward pass. This chapter tells one story in four moves: each mechanism the field adopted solved the problem the previous one left exposed. Batch at iteration granularity, and memory fragments; page the cache like an operating system, and prefixes beg to be shared; share prefixes in a tree, and the two phases still fight over one device; split the phases onto separate machines, and a network transfer reappears as the new cost. By the end a reader can explain why a modern engine batches at the granularity of a single decode step, why it manages the key-value cache in fixed pages, why it keeps a tree of shared prefixes, and why the newest systems separate prefill from decode.

Two facts that make one problem

Chapter 16 named the two phases of generation and their opposite shapes. Prefill reads the whole prompt in one compute-bound pass and produces the first token. Decode then emits one token per forward pass, each step reading the entire key-value cache for one new query, so decode is memory-bandwidth-bound and runs at batch size one per request unless the engine batches across requests. The memory cost and the scheduling cost of those two phases are not separate concerns. They are two faces of one resource problem, and treating them together is what this chapter is for.

The memory fact is that the key-value cache is large, per-request, and grows by one slot per layer per generated token. For a request of length L , the cache holds $2 \cdot L \cdot n_{\text{layers}} \cdot n_{\text{kv}} \cdot d_{\text{head}}$ values, the factor of two for keys and values. On a model with billions of parameters the weights are fixed, but the cache for a few dozen concurrent long-context requests can exceed the weights, and it appears and disappears as requests arrive and finish. A naive engine reserves the maximum possible length for every request up front, which wastes most of the reservation for requests that finish early and caps the batch size far below what the hardware could hold.

The scheduling fact is that requests do not share a length. In a batch of generations, one request may stop at twenty tokens and another run to two thousand. If the engine schedules a batch and waits for every member to finish before admitting the next, the short requests sit idle inside a padded batch while the longest one decodes, and newly arrived requests wait behind the whole batch. Throughput collapses to the pace of the slowest member, and latency for a new arrival is the tail of the batch in front of it.

So the problem has two halves that this chapter treats together: pack the key-value cache so memory is not wasted, and schedule the forward passes so no request waits on another's length. The four sections that follow each take one mechanism, and each mechanism earns its place by fixing what the one before it broke.

Iteration, not request

The first move is to schedule at the granularity of an iteration rather than a request. An iteration is one forward pass that advances every active request by a single token. After each iteration the scheduler is free to remove requests that just emitted their stop token and admit waiting requests into the next iteration's batch. This is **continuous batching**, also called iteration-level scheduling, introduced by Orca (Yu et al. 2022). The batch is no longer a fixed cohort that lives and dies together; it is a sliding window over the active set, refilled every step. A request that finishes at token twenty leaves immediately and its slot is reused, instead of padding out the batch until the longest member finishes. [Figure 17.1](#) shows the loop: every

iteration retires whatever just emitted its stop token and admits waiting requests, so the batch composition changes each step.

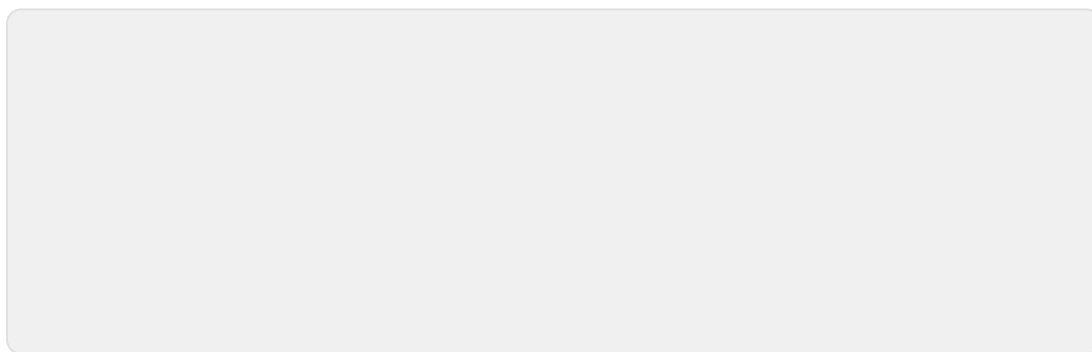


Figure 17.1. The continuous-batching loop. The scheduler forms a batch, runs one forward pass that advances every active request by one token, retires those that emitted a stop token, and admits waiting requests before the next iteration, so the batch is a sliding window refilled every step. After Yu et al., 2022.

Before Orca, serving systems batched at request granularity: form a batch, run it to completion, return the results. This is request-level scheduling, and it inherits the static-batch pathology of any padded-cohort design. Orca's contribution was to schedule at iteration granularity and to make the batch a living set, refilled every step, which is the single change that lets a serving engine keep accelerators busy under heterogeneous request lengths (Yu et al. 2022). Orca also introduced selective batching, batching the parts of the computation that can be batched across requests of different lengths while running attention per request.

Continuous batching solves the scheduling fact and immediately exposes the memory fact in a sharper form. If admission happens every iteration, the engine must allocate and free key-value cache space every iteration, for requests whose final length it does not know. Reserving a contiguous maximum-length buffer per request, the obvious scheme, fragments memory: a finished request leaves a hole too small for the next, and reserved-but-unused tail space is dead weight. Measurements in the vLLM paper put the waste from this fragmentation and over-reservation at the majority of cache memory in prior systems (Kwon et al. 2023). The next move exists to reclaim it.

The cache as virtual memory

The second move removes that waste by borrowing virtual memory from operating systems. The insight is that the key-value cache is exactly the kind of object virtual memory was invented for: a per-process allocation that grows unpredictably and must share a fixed physical pool without fragmentation. **PagedAttention** (Kwon et al. 2023) stores the key-value cache in fixed-size blocks, each holding the keys and values for a fixed number of tokens, and lets a request's logical sequence of blocks map to physically non-contiguous blocks through a block table. The attention kernel is rewritten to gather keys and values through that table instead of assuming contiguity. Now allocation is per block, not per request: a request grows by appending a block when its current block fills, memory is shared at block granularity across all requests, and the only internal waste is the partial final block. The analogy is exact. The block is a page, the block table is a page table, and the engine is a memory manager that hands out and reclaims pages as sequences grow and finish. Figure 17.2 shows the indirection: a request's logical sequence of token blocks maps through a block table to physically scattered blocks in a shared pool.

Figure 17.2. PagedAttention indirection. A request's logical token blocks map through a block table to non-contiguous physical blocks in a shared pool, so allocation is per block and the only waste is the partial final block. After Kwon et al., 2023.

vLLM is the serving system built on this idea, and it reported two to four times the throughput of the prior generation at the same latency by lifting the achievable batch size (Kwon et al. 2023). The lift comes from where the waste goes. Figure 17.3 contrasts the two schemes: reserving a maximum-length buffer per request leaves utilization at the ratio of average to maximum length, so a workload of mostly short requests strands most of the pool, while paging holds only the realized length plus a partial final block and keeps utilization near one regardless of length spread, which is what lifts the achievable batch size. The rest of vLLM, the block manager, the swapping of blocks to host memory under pressure, the copy-on-write sharing, all follows from treating the cache as paged virtual memory. The result reset the throughput baseline for open serving. It also opened a door: once the cache is paged and blocks can be shared, identical content across requests need not be stored or computed twice.

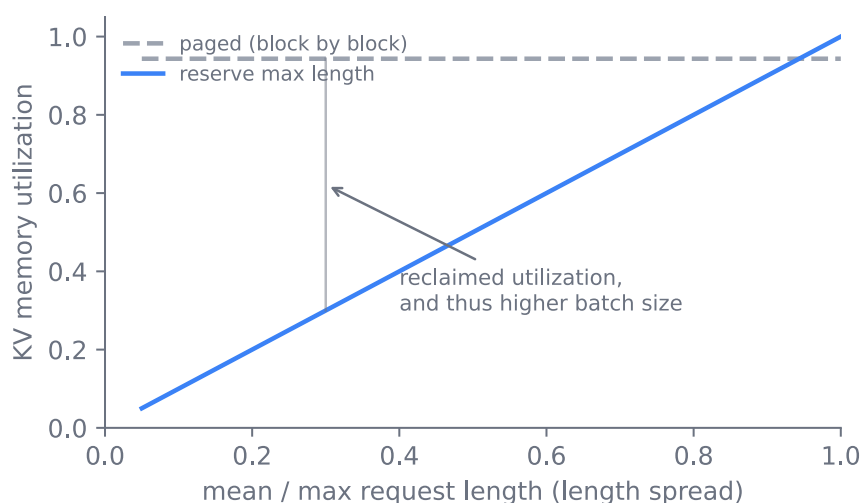


Figure 17.3. Schematic of why paging lifts the achievable batch size, after Kwon et al. (2023). Reserving a maximum-length buffer per request leaves utilization at the mean-to-maximum length ratio, so most of the pool sits dead when requests finish early, while paged allocation wastes only the partial final block and holds utilization near one across the whole length spread, which is what lifts the achievable batch size. Idealized synthetic data, not measured.

Sharing prefixes

Paging makes sharing cheap, and that is the third move. Two requests that begin with the same prompt, a shared system prompt or a few-shot preamble, compute identical keys and values for that prefix. Under paging, they can point their block tables at the same physical blocks for the

shared prefix and diverge only where their tokens diverge, with copy-on-write when one writes into a shared block. The open question is how to find shareable prefixes automatically across a stream of requests with arbitrary overlap. **RadixAttention** (Zheng et al. 2024), the prefix-caching mechanism of the SGLang runtime, answers it by storing all cached prefixes in a radix tree keyed by token sequence, with least-recently-used eviction. A new request walks the tree to find the longest cached prefix of its prompt, reuses those blocks, and computes only the uncached suffix. The radix tree turns prefix reuse from a special case, handled by hand for a known shared prompt, into a default the runtime applies to all traffic. Prefix caching turns the repeated structure of real workloads, shared system prompts, multi-turn chat history, branching agent calls, into reused computation rather than recomputed computation. Figure 17.4 shows two requests sharing a system-prompt prefix: they point at the same cached blocks up to the branch point and compute only their divergent suffixes.

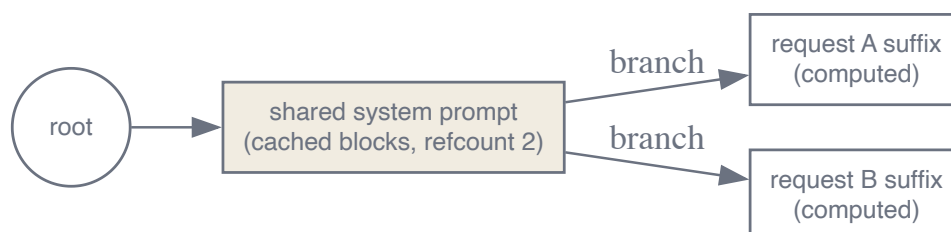


Figure 17.4. Prefix sharing in a radix tree. Two requests with the same system prompt share its cached blocks up to the branch point and compute only the suffix where their tokens diverge. The shared node is reference-counted and copied on first write past it. After Zheng et al., 2024.

Paging and prefix sharing both assume prefill and decode share a device and a batch, and that assumption is the next thing to break. Running both in one batch means a long prefill stalls the decodes waiting behind it, because the two phases want different parallelism and different batch shapes. One way out keeps the single device and schedules around the interference. Sarathi-Serve (Agrawal et al. 2024) addresses it within a single device by splitting prefill into chunks and coalescing each chunk with ongoing decodes, so a long prefill never stalls decode. The fourth move takes the opposite route.

Splitting the phases

The fourth move separates the two phases onto different hardware. Prefill is compute-bound and bursty; decode is memory-bandwidth-bound and steady. **Prefill/decode disaggregation** runs prefill on one pool of machines and decode on another, transferring the key-value cache between them over the network. Each pool is provisioned and parallelized for its own phase, and neither interferes with the other's latency. DistServe (Zhong et al. 2024) optimizes the placement for goodput, the rate of requests that meet both a time-to-first-token target for prefill and a time-per-output-token target for decode. Mooncake (Qin et al. 2024) centers the whole architecture on the key-value cache, pooling CPU DRAM and SSD across the cluster into a disaggregated cache that prefill writes and decode reads. Figure 17.5 sketches the split: the prompt enters a prefill pool, the resulting key-value cache crosses the network, and the decode pool reads it to emit tokens, each pool sized and parallelized for its own phase.

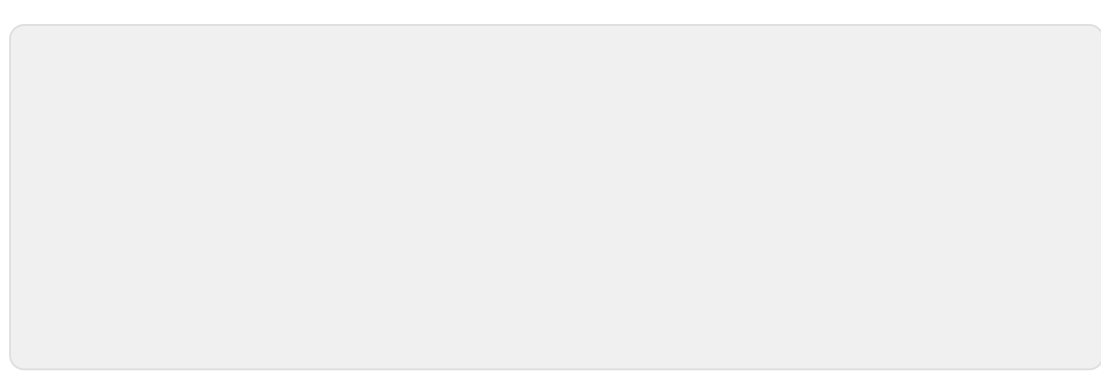


Figure 17.5. Prefill/decode disaggregation. A compute-bound prefill pool produces the first token and the key-value cache, which transfers over the network to a memory-bandwidth-bound decode pool that streams the remaining tokens. Each pool is provisioned and parallelized for its own phase. After Zhong et al., 2024 and Qin et al., 2024.

This is the most recent move, and it inverts the assumption the first three shared. Both lines are live. The within-device line, exemplified by chunked-prefill scheduling (Agrawal et al. 2024), keeps one resource pool and schedules carefully; the disaggregated line (Zhong et al. 2024; Qin et al. 2024) accepts a network transfer of the key-value cache to remove interference entirely. Which one wins is genuinely unsettled.

WHAT'S CONTESTED

Whether to disaggregate prefill and decode, or to keep them on the same device and schedule around the interference, is unsettled. Disaggregation removes phase interference and lets each pool scale and parallelize independently, and at large scale with long prompts the reported goodput gains are substantial (Zhong et al. 2024). But it pays a network transfer of the key-value cache on every request, needs enough cross-machine bandwidth to hide that transfer, and complicates routing and fault handling. The colocated camp, exemplified by chunked-prefill scheduling (Agrawal et al. 2024), argues that careful within-device scheduling captures most of the benefit at a fraction of the operational cost and with no transfer. The right answer depends on prompt length, the time-to-first-token and time-per-output-token targets, and the interconnect: there is no setting where one approach dominates across all workloads.

CONSTRAINT ARROW

The attention design upstream sets how hard this chapter's job is. The size of the key-value cache, the term that paging exists to manage, is fixed by the number of key-value heads chosen in [Chapter 6](#). Multi-query and grouped-query attention shrink that term by sharing key-value heads across query heads, and the latent attention of [Chapter 7](#) compresses it further. A smaller per-token cache means more sequences fit in the same memory, which raises the achievable batch size, which is the lever continuous batching pulls for throughput. An attention variant chosen for cache size is, in effect, a scheduling decision made one layer down.

What each move costs

The cascade buys four benefits, and each comes with a named cost.

- **Block size in paging.** A small block wastes little in the partial final block and shares at fine granularity, but multiplies block-table entries and per-kernel gather overhead. A large block cuts that overhead but wastes more tail memory and shares more coarsely. [Figure 17.6](#)

plots the two costs against block size: internal fragmentation rises with the block while indirection overhead falls, and their sum is a U-curve with an interior optimum. The block size is a tuning knob, not a constant, and the right value depends on typical sequence length and how much prefix sharing the workload has.

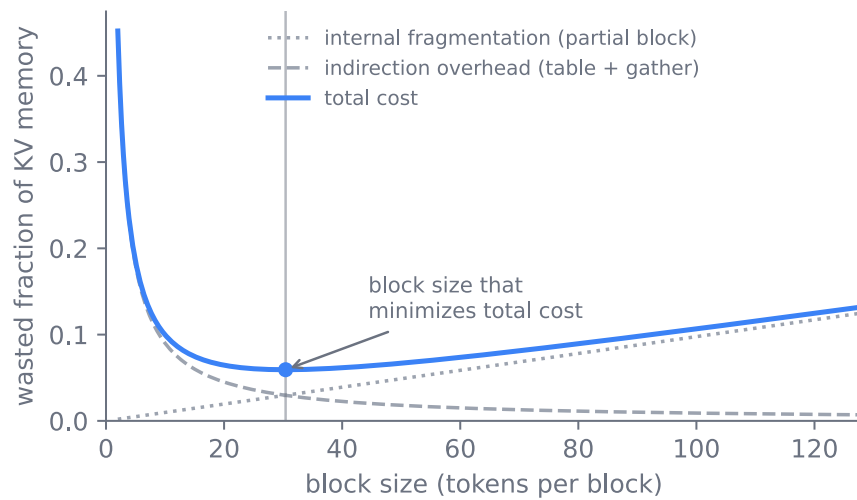


Figure 17.6. Schematic of the block-size trade-off in paged attention, after Kwon et al. (2023). Internal fragmentation, the partial final block, grows with block size, while indirection overhead, the block-table entries and the gather through them, falls as roughly one over the block size. Their sum is a U-curve with an interior optimum, which is why the block size is a tuning knob. Idealized synthetic data, not measured.

Try changing `L`, the typical sequence length, and watch the optimal block size move with it, matching the square-root formula.

```
import numpy as np
import matplotlib.pyplot as plt

L = 512          # typical sequence length, in tokens
w_frag = 1.0     # cost per wasted token in the partial final block
w_indir = 4.0    # cost of one block-table entry and its gather
b = np.arange(1, 257)
frag = w_frag * (b / 2)          # waste grows with block size
indir = w_indir * (L / b)        # indirection falls as 1/block
total = frag + indir
b_star = b[np.argmin(total)]
b_formula = np.sqrt(2 * w_indir * L / w_frag)
print("argmin block size:", b_star, " sqrt formula:", round(b_formula))
plt.plot(b, frag, "--", label="fragmentation (b/2)")
plt.plot(b, indir, "--", label="indirection (L/b)")
plt.plot(b, total, label="total")
plt.axvline(b_star, color="k", lw=0.8)
plt.xlabel("block size (tokens)"); plt.ylabel("cost"); plt.legend();
```

- **Prefix-cache retention versus memory.** Keeping more prefixes in the radix tree raises the hit rate for workloads with shared structure, but the cache competes with live requests for the same physical blocks. Under memory pressure the engine must evict cached prefixes to admit new work, so the prefix cache is a bet that reuse will pay before the blocks are reclaimed. Workloads with little overlap pay the bookkeeping and get little back.
- **Disaggregation versus colocation.** Disaggregation removes interference and lets each phase scale alone, at the cost of a key-value cache transfer per request and the bandwidth to hide it. Colocation with chunked prefill keeps one pool and one transfer-free path, at the cost of scheduling complexity and some residual interference. The crossover is set by prompt length and latency targets.
- **Throughput versus latency in batching.** A larger continuous batch raises throughput but lengthens each iteration, which raises time-per-output-token for every request in it. Admission is therefore a latency knob: the engine can cap batch size or reserve headroom to protect tail latency, trading some throughput for it.

The engine as a loop and a memory manager

All four moves reduce, in code, to a loop and a memory manager. The loop is iteration-level: each step, the scheduler picks the set of requests to run, the model runs one forward pass over them, the sampler produces one token each, finished requests are retired, and waiting requests are admitted into the next step. In vLLM this is the engine step (`vllm.LLMEngine.step` , `vllm/engine/llm_engine.py`), which calls the scheduler to form the batch and the model runner to execute it. The scheduler tracks queues for running, waiting, and swapped requests, and moves requests between them as cache blocks are allocated and reclaimed.

The memory manager owns the block pool. It maintains a free list of physical blocks and a per-request block table mapping logical positions to physical blocks. Allocation appends a block when the current one fills; freeing returns a request's blocks to the pool on completion. Under pressure the manager can preempt a request, either recomputing its cache later or swapping its blocks to host memory and back, which is the serving analogue of paging to disk. The attention kernel reads through the block table, so the physical layout stays invisible to the rest of the model. For prefix sharing, the block table of a new request is initialized to point at the cached blocks of its longest matching prefix, with a reference count so a shared block is freed only when its last user finishes, and copied on first write past the shared point.

Two operational realities are worth naming. First, the partial final block is the only internal fragmentation paging leaves, so average waste falls as sequences lengthen relative to block size, which is why long-context workloads benefit most from paging. Second, prefix-cache hit rate is workload-shaped: a chat service with a fixed system prompt or an agent loop that re-sends a long tool preamble can see most of its prefill served from cache, while a stream of unrelated single-turn prompts sees almost none. Instrumenting the hit rate, the achievable batch size, and the preemption rate is how an operator reads whether the memory manager is winning.

Further reading

- Yu et al., "Orca: A Distributed Serving System for Transformer-Based Generative Models," OSDI 2022. usenix.org/conference/osdi22/presentation/yu
- Kwon et al., "Efficient Memory Management for Large Language Model Serving with PagedAttention" (vLLM), SOSP 2023. [arXiv:2309.06180](https://arxiv.org/abs/2309.06180)
- Zheng et al., "SGLang: Efficient Execution of Structured Language Model Programs" (RadixAttention), NeurIPS 2024. [arXiv:2312.07104](https://arxiv.org/abs/2312.07104)
- Zhong et al., "DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving," OSDI 2024. [arXiv:2401.09670](https://arxiv.org/abs/2401.09670)

- Qin et al., "Mooncake: A KVCache-centric Disaggregated Architecture for LLM Serving," 2024. [arXiv:2407.00079](#)
- Agrawal et al., "Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve" (chunked prefill), OSDI 2024. [arXiv:2403.02310](#)

18. Faster Decoding

A frontier model running at batch size one spends most of its time waiting. To produce a single token it streams every weight from high-bandwidth memory into the accelerator's compute units, multiplies once, and discards them. The matrix multiplications are skinny, a vector against each weight matrix, so the chip finishes the math long before the next slab of weights arrives, then sits idle waiting on memory. [Chapter 16](#) and [Chapter 17](#) establish this wall from the serving side: decode is memory-bound, its arithmetic intensity far below what the hardware can sustain. This chapter is about the family of methods that fill that idle time, breaking the one-token-per-pass rule by spending parallel compute the hardware was already wasting. By the end a reader can explain why speculative decoding loses no quality, how Medusa, Hydra, the EAGLE line, lookahead decoding, and multi-token prediction each produce their guesses, and why the speedups they quote hold in one serving regime and evaporate in another.

One trick, stated once

Autoregressive generation has a sequential bottleneck that looks fundamental: decoding K tokens takes K forward passes, each waiting on the one before it. The bottleneck bites hardest exactly where latency is felt most. Reasoning models from [Chapter 15](#) emit long chains of intermediate tokens before an answer, so wall-clock latency is dominated by the sheer count of sequential decode steps.

The single idea behind everything in this chapter is to trade extra parallel compute for fewer sequential steps. A cheap process guesses several future tokens, the expensive model checks all of them in one forward pass, and every guess that survives the check is a token produced without its own sequential step. Because a forward pass over γ candidate tokens costs almost the same as a pass over one, the spare compute is nearly free.

Two quantities govern the payoff. The acceptance rate α is the fraction of draft tokens the target keeps; the draft cost is how much cheaper the guesser is than the target. The expected number of tokens produced per target pass rises with α , so the whole game is to make the draft both cheap and well aligned with the target, and to verify many candidates at once. Two questions organize the rest of the chapter. Where does the guess come from, and how does the check stay correct.

[Figure 18.1](#) makes the payoff concrete. With acceptance rate α and a draft of γ tokens checked in one pass, the expected tokens emitted per target pass is the geometric sum $(1 - \alpha^{\gamma+1}) / (1 - \alpha)$, rising from one when nothing is accepted toward $\gamma + 1$ when everything is. A longer draft helps only when α is high, because an early rejection wastes the whole tail.

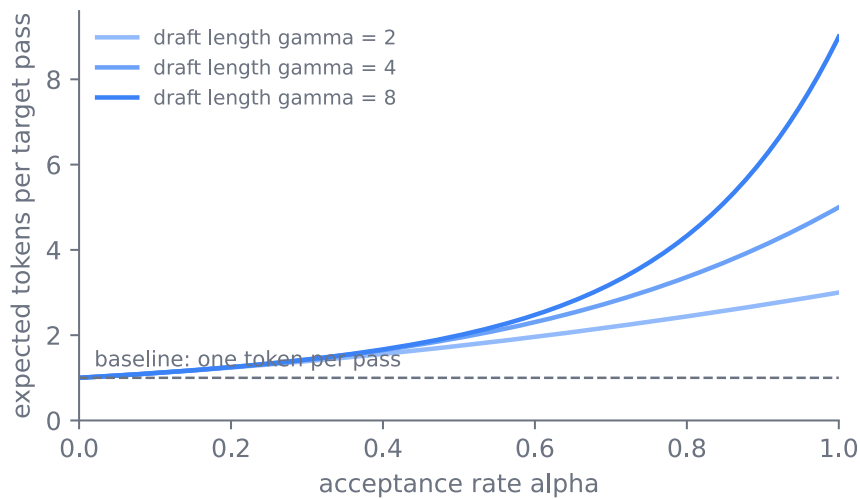


Figure 18.1. Schematic of the speculative-decoding payoff: expected tokens emitted per target pass as a function of acceptance rate, for several draft lengths. A longer draft helps only when the acceptance rate is high. After Leviathan et al. (2023).

Try simulating the accept/reject loop directly and watch the Monte Carlo average converge to the closed-form geometric sum.

```
import numpy as np
rng = np.random.default_rng(0)
alpha, gamma, trials = 0.7, 4, 50000

def simulate():
    emitted = np.ones(trials)          # the corrected/bonus token
    alive = np.ones(trials, dtype=bool) # runs not yet rejected
    for _ in range(gamma):
        accept = (rng.random(trials) < alpha) & alive
        emitted += accept              # count each accepted draft
        alive &= accept                 # stop at first rejection
    return emitted.mean()

closed_form = (1 - alpha**(gamma + 1)) / (1 - alpha)
print(f"simulated mean tokens/pass: {simulate():.4f}")
print(f"closed-form tokens/pass: {closed_form:.4f}")
```

Keeping the check exact

The decisive design choice is how the verification preserves correctness, and the answer that made the method safe to deploy is that it can be made exact. Speculative decoding, introduced independently by Leviathan et al. (Leviathan et al. 2023) and Chen et al. (Chen et al. 2023), guarantees this. A small draft model proposes a continuation of γ tokens. The target model scores all γ positions in one parallel pass, yielding its own distribution p at each position

alongside the draft's distribution q . A modified rejection sampling rule then accepts or rejects each draft token left to right: token x proposed from q is accepted with probability $\min(1, p(x)/q(x))$, and on the first rejection a corrected token is resampled from the residual distribution $\max(0, p - q)$, renormalized. The theorem behind the rule is that the accepted stream is distributed exactly as if it had been sampled from p token by token (Leviathan et al. 2023). The speedup is real and the output distribution is untouched. The method is not an approximation of the model, it is the same model sampled in a different order.

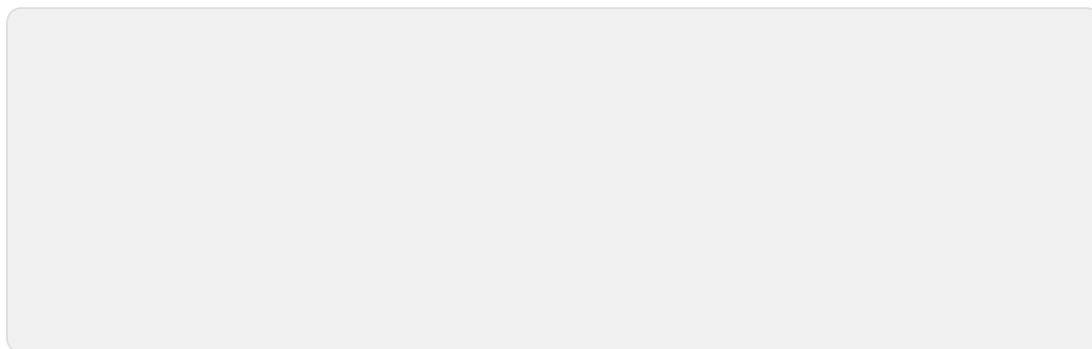


Figure 18.2. The speculative decoding loop: a cheap draft proposes a run of tokens, one target pass scores them all, and a modified rejection rule keeps the longest valid prefix while preserving the target's distribution. After Leviathan et al. (2023) and Chen et al. (2023).

Figure 18.2 draws this loop. The line of accept/reject decisions it shows is the simplest case, a single linear continuation. The self-draft methods below replace that line with a tree.

The search for the guess

The lineage of these methods is best read as a search for where the guess comes from, moving from a separate model toward parts of the target itself, and finally into how the target is trained.

A second, smaller model

The first answer was a second, smaller model. Leviathan et al. and Chen et al. both used an off-the-shelf draft model from the same family as the target, for example a small Chinchilla drafting for a 70B Chinchilla (Chen et al. 2023). This needs no change to the target and is provably lossless, but it requires training, hosting, and aligning a second model, and a poorly matched draft has a low acceptance rate.

Heads on the target itself

Medusa (Cai et al. 2024) removed the separate model. Instead of a draft network it bolts several lightweight decoding heads onto the target's last hidden state, each head predicting one further future token in parallel. The candidates the heads produce form a tree, and a tree-structured attention mask lets the target verify many branches of that tree in a single pass rather than a single linear continuation. Medusa's heads predict each future token independently of the others, which caps how coherent a multi-token guess can be. It also offers a typical-acceptance scheme that relaxes exact sampling for more speed, so Medusa in that mode is deliberately not distribution-preserving, a different bargain from speculative decoding's exactness (Cai et al. 2024).

Hydra (Ankner et al. 2024) fixed the independence limitation. Its draft heads are sequentially dependent: the head predicting the third token sees what the second head proposed, so the multi-token guess is internally coherent and the acceptance rate rises. The EAGLE line pushed further on what the draft should predict. EAGLE (Li et al. 2024) observed that autoregression is more regular at the feature level, the second-to-top-layer hidden state, than at the token level, so it drafts by predicting the next feature and then mapping it to a token, resolving the uncertainty by feeding in the token already sampled one step ahead. EAGLE-2 (Li et al. 2024) made the verification tree dynamic: because a draft token's acceptance depends on context and not only on its position, it expands the draft tree where the draft is confident and prunes it where it is not. EAGLE-3 (Li et al. 2025) abandoned feature prediction in favor of direct token prediction over a fusion of low, middle, and high layers, trained with a procedure the authors call training-time test, which lets the drafter keep improving as its training data grows.

No draft at all

A parallel branch removed the draft entirely. Lookahead decoding (Fu et al. 2024) reframes autoregressive generation as solving a nonlinear system by Jacobi iteration, updating a whole window of future positions at once and collecting the n -grams that the iteration produces along the way. A verification branch then checks those n -grams against the target in the same step. There is no draft model and no extra trained heads; the parallelism comes from the fixed-point structure of the problem, and the method is exact.

Training the guess in

The last move folds the guess into training. Multi-token prediction (Gloeckle et al. 2024) trains the base model with several output heads on a shared trunk so that each position predicts the next n tokens at once. The authors found this improves sample efficiency as an auxiliary objective, and crucially the extra heads double as a built-in drafter at inference time. DeepSeek-V3 (DeepSeek-AI 2024) carries this into a frontier system: its MTP module is trained alongside the main model and then reused for speculative decoding, reported to accept its second predicted token 80% to 90% of the time and to raise generation throughput by roughly 1.8 times.

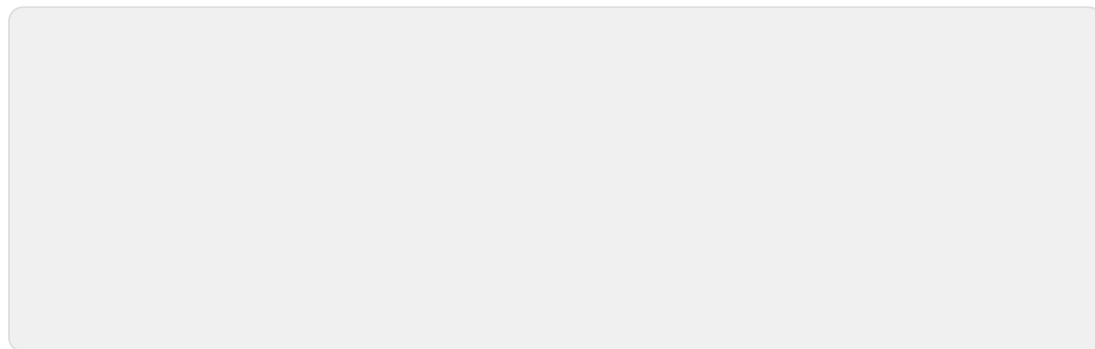


Figure 18.3. Where the guess comes from: the lineage moves from a separate draft model, into parts of the target itself, and finally into how the target is trained, with a draft-free branch off to the side.

Figure 18.3 traces that search.

When the speedup is real

Every method here buys fewer sequential steps with more parallel compute and more memory traffic for verification. The boundaries are where that trade stops paying, and the sharpest of them is set by the serving regime rather than the method.

Figure 18.4 shows why the same method can help or hurt. At small batch the accelerator's compute units sit idle waiting on weights, so the FLOPs speculation spends to verify candidates were going to waste anyway, and they are free. At large batch the weights are amortized across many sequences and the compute units are already saturated, so verifying tokens that then get rejected competes with real work for FLOPs and can lower total throughput. The premise of the whole chapter, that decode is memory-bound, holds only in the small-batch regime.

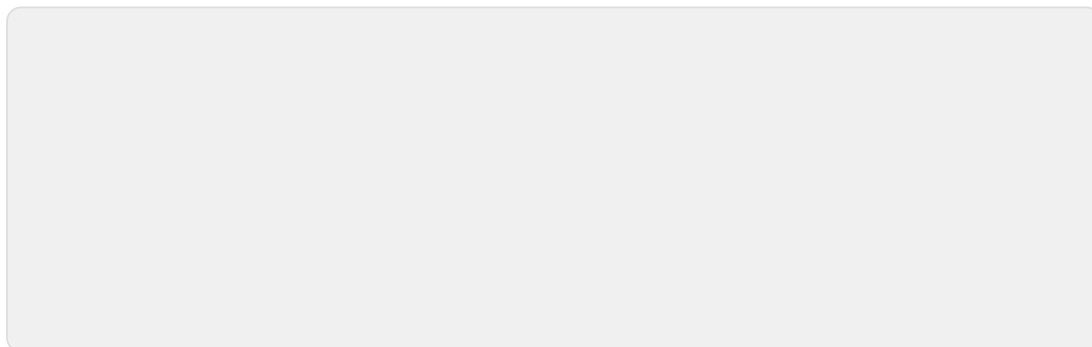


Figure 18.4. Why a speedup is a property of the serving regime. At small batch the compute units idle on memory, so speculation's extra FLOPs are free. At large batch the units are saturated, so verifying rejected tokens competes for FLOPs and can cost throughput.

Three further trades sit underneath this one, and they are worth naming because they explain why the lineage moved the way it did.

- **Acceptance versus draft cost.** A larger or better-trained drafter raises the acceptance rate but costs more per proposal. The optimal draft is the cheapest one whose acceptance is high enough that the saved target passes outweigh the draft's own cost. This is why self-draft heads and feature-level drafting won out: they are nearly free yet aligned with the target. Figure 18.5 shows the optimum this implies: as the draft grows longer the cost-adjusted speedup rises, peaks, then falls, because past some length the extra verification work outweighs the tokens it wins back, and a cheaper drafter pushes that peak to longer drafts.
- **Tree width versus waste.** Verifying a wider tree of candidates accepts more tokens per pass but burns compute on branches that will be rejected. Dynamic trees, as in EAGLE-2, exist to spend that width only where the draft is confident.

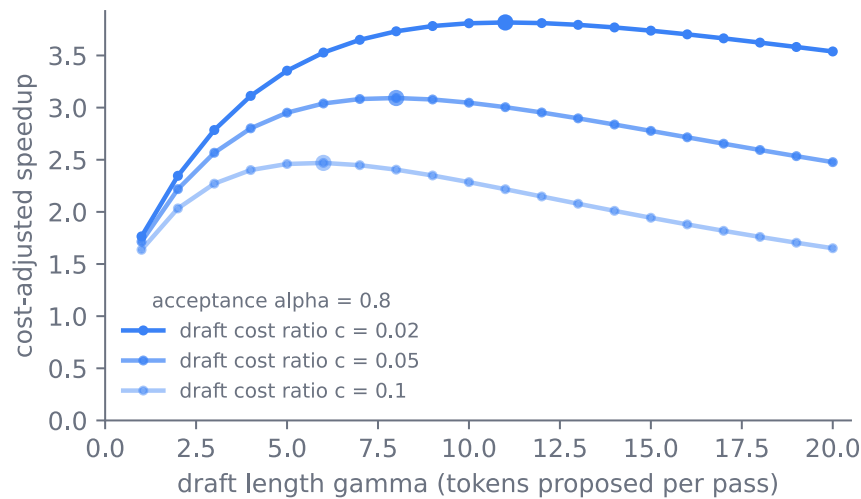


Figure 18.5. Schematic of the acceptance-versus-draft-cost trade-off: cost-adjusted speedup against draft length, at a fixed acceptance rate, for several draft-to-target cost ratios. Each curve peaks at an optimal draft length, and a cheaper drafter moves the peak rightward. After Leviathan et al. (2023) and Chen et al. (2023).

- **Lossless versus lossy.** Speculative decoding, lookahead, and the EAGLE line preserve the target's output distribution exactly. Medusa's typical-acceptance mode trades that exactness for speed. Multi-token prediction changes training, not the sampling guarantee. Whether a lossy shortcut is acceptable is an application decision, not a free lunch.

WHAT'S CONTESTED

The speedups quoted across this literature, the 2 to 3 times of the original speculative decoding, the 3 to 4 times of EAGLE-2 (Li et al. 2024), the larger figures of EAGLE-3 (Li et al. 2025), are measured at small batch sizes where serving is latency-bound and the accelerator's compute genuinely sits idle. They are not comparable across papers, because each uses different baselines, hardware, and tasks, and they do not transfer to every deployment. As batch size grows, a server becomes compute-bound rather than memory-bound: the spare FLOPs that speculation spends are no longer free, and verifying tokens that then get rejected can lower total throughput. Whether speculative decoding pays in high-throughput production serving, as opposed to low-latency single-stream use, is a live question. So is the choice among separate-draft, self-draft, and draft-free designs, which trade hosting cost, training cost, and acceptance differently with no settled winner. Treat any single speedup number as a property of a serving regime, not of the method.

CONSTRAINT ARROW

This whole chapter is a lower layer reaching up to dictate an algorithm. The memory-bandwidth wall of Chapter 16 and Chapter 17, where single-stream decode reloads every weight per token and leaves the compute units idle, is what makes "spend parallel compute to save sequential steps" worthwhile in the first place. If decode were compute-bound there would be no idle FLOPs to spend and none of these methods would help. The shape of the decode algorithm is set by the bandwidth-to-compute ratio of the chip below it.

Inside the verification step

The verification step is the part worth seeing concretely, because it is where correctness lives. Given draft tokens $x_1, \dots, x_\gamma$ from q and the target's distributions $p_1, \dots, p_\gamma$ from one parallel pass, the accept loop is short:

```
for i in range(gamma):
    r = uniform(0, 1)
    if r < min(1, p[i][x[i]] / q[i][x[i]]):
        emit(x[i])           # accepted, no extra target step
    else:
        emit(sample(normalize(relu(p[i] - q[i])))) # corrected token
        break                # stop at first rejection
# if all gamma accepted, sample one bonus token from p[gamma]
```

The two details that make it exact are the residual $\max(0, p - q)$ resampling on rejection and the bonus token after a full-accept run; together they are what the distribution-preservation theorem requires. For the self-draft and tree-based methods, the same accept logic runs over a tree rather than a line: a tree attention mask lets the single target pass score every branch, and the longest accepted path through the tree is emitted. Figure 18.6 shows why this beats a linear draft. A linear draft commits to one continuation, so a single early rejection wastes everything after it. A tree hedges across several continuations at each position, and the tree attention mask scores all of them in the one target pass for almost the cost of one. The target then emits the longest path whose every token was accepted.

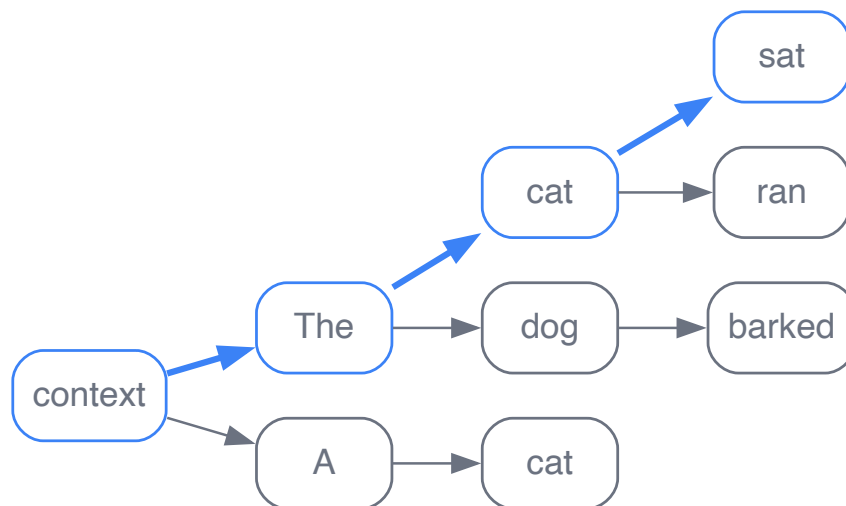


Figure 18.6. Tree-structured drafting and verification. From the context, draft heads propose several candidate tokens per position rather than one. A tree attention mask scores every branch in a single target pass, and the longest fully accepted path (bold) is emitted. After Cai et al. (2024) and Li et al. (2024).

Operationally, serving stacks expose this as a drafter choice. A separate draft model is a second checkpoint to load and schedule. EAGLE-style and Medusa-style drafters are small head modules trained against a frozen target and loaded beside it. Multi-token-prediction drafting, as in DeepSeek-V3, needs no separate artifact at all because the heads were trained into the model. The practical failure mode is a drafter whose acceptance rate is too low for the

workload, at which point speculation adds latency instead of removing it, which is why acceptance rate is the metric to watch in production rather than the headline speedup from a paper.

Further reading

- Leviathan, Kalman & Matias, "Fast Inference from Transformers via Speculative Decoding," 2023. [arXiv:2211.17192](#)
- Chen et al., "Accelerating Large Language Model Decoding with Speculative Sampling," 2023. [arXiv:2302.01318](#)
- Cai et al., "Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads," 2024. [arXiv:2401.10774](#)
- Ankner et al., "Hydra: Sequentially-Dependent Draft Heads for Medusa Decoding," 2024. [arXiv:2402.05109](#)
- Li et al., "EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty," 2024. [arXiv:2401.15077](#)
- Li et al., "EAGLE-2: Faster Inference of Language Models with Dynamic Draft Trees," 2024. [arXiv:2406.16858](#)
- Li et al., "EAGLE-3: Scaling up Inference Acceleration of Large Language Models via Training-Time Test," 2025. [arXiv:2503.01840](#)
- Fu et al., "Break the Sequential Dependency of LLM Inference Using Lookahead Decoding," 2024. [arXiv:2402.02057](#)
- Gloeckle et al., "Better & Faster Large Language Models via Multi-token Prediction," 2024. [arXiv:2404.19737](#)
- DeepSeek-AI, "DeepSeek-V3 Technical Report" (multi-token prediction reused for speculative decoding), 2024. [arXiv:2412.19437](#)

19. Quantization and Kernels

Decoding is bound by memory: by how many bytes of weights and cache must cross the bus per token, and by how many bytes of attention intermediates must be written and read back. This chapter is about two levers that attack that bound, shrinking the numbers with quantization and refusing to move the intermediates with fused kernels. By the end a reader can explain why GPTQ, AWQ, and SmoothQuant make different bets, why a model can run in INT4 or FP8 without falling over, why FlashAttention never materializes the score matrix, and why the choice among vLLM, SGLang, and TensorRT-LLM is a choice about which of these wins the engine has already made.

The memory wall

A weight in FP16 costs two bytes. A seventy-billion-parameter model is therefore one hundred and forty gigabytes before a single token is served, which already exceeds one accelerator. The serving problem of Chapter 16 and the scheduling of Chapter 17 both begin from this fact: capacity is spent on a static weight footprint and a growing key-value cache, and what is left bounds the batch.

The deeper trouble is bandwidth, not just capacity. Decoding emits one token at a time, so each step reads the whole weight set to produce a single column of activations. The arithmetic intensity is low, the accelerator's compute units idle, and wall-clock is set by how fast bytes move from high-bandwidth memory, not by how fast they multiply. Chapter 18 attacks this from the algorithm side by doing more useful work per pass. This chapter attacks it from below, by making each byte carry less and by moving fewer of them.

Attention has its own version of the same disease. The naive computation forms the full $n \times n$ score matrix, writes it to memory, reads it back to apply softmax, writes again, and reads once more to weight the values. For a long sequence that matrix dwarfs the inputs, and the kernel spends its time on memory traffic rather than on the matmuls that matter. The architecture of Chapter 6 makes attention central, and Chapter 20 pushes the sequence length up, so the cost of moving these intermediates grows faster than the cost of the model itself.

Two levers answer this single enemy. The first shrinks each number so fewer bytes must move. The second refuses to write intermediates that will only be read back. The rest of the chapter takes them in turn, then shows that the serving regime picks between them and that an evaluation sitting one layer up quietly prices the whole trade.

Lever one: make each byte smaller

Quantization maps a high-precision tensor onto a small set of levels and stores the levels plus a scale. For integer quantization the rule is a scale and a round,

$$\hat{w} = s \cdot \text{round}\left(\frac{w}{s}\right), \quad s = \frac{\max |w|}{2^{b-1} - 1},$$

so a block of weights becomes b -bit integers and one floating-point scale. The granularity of s is the first design axis: one scale per tensor is cheapest to store and worst at handling spread, one scale per output channel or per small group of weights costs a little metadata and recovers most of the accuracy. Floating-point quantization keeps a sign, exponent, and mantissa in the small budget instead of a uniform grid, which trades some precision near zero for dynamic range, and the hardware can multiply the small format natively.

Why per-tensor INT8 breaks: the outlier

The reason quantization is not trivial is the outlier. Weights have a well-behaved distribution and quantize cleanly. Activations in a trained transformer do not: a few channels carry values orders of magnitude larger than the rest, and a single per-tensor scale chosen to cover them throws away all precision on the rest. This is why naive per-tensor INT8, carried over from vision models, collapses on large transformers once the emergent activation outliers appear past a certain scale. Every serious method is a different answer to the outlier. Figure 19.1 shows the geometry of the problem: a single per-tensor scale must stretch to cover the largest outlier channel, which spreads the integer grid so coarsely that the well-behaved bulk collapses onto a handful of levels.

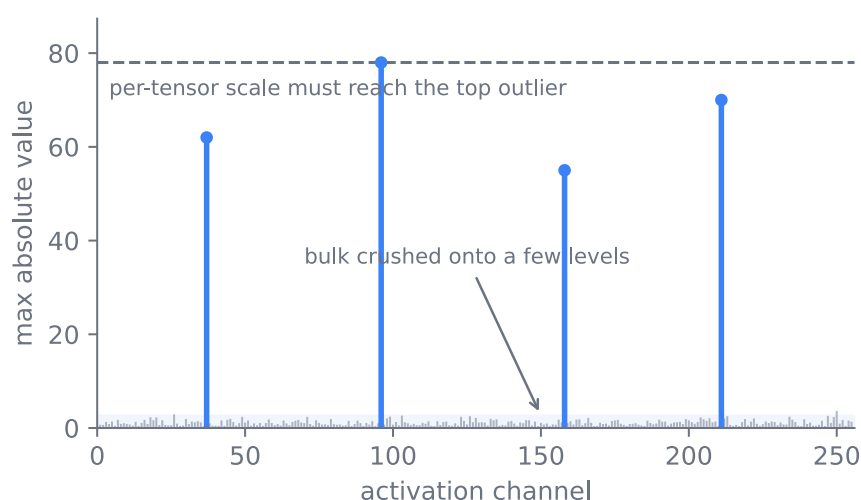


Figure 19.1. Schematic illustration of why per-tensor INT8 breaks on a trained transformer. Most activation channels sit in a narrow band, but a few emergent channels spike orders of magnitude higher, forcing a single per-tensor scale to waste nearly all of its precision on the bulk. Idealized synthetic data, not measured activations, after Xiao et al. (2022).

Run the chapter's quantization rule on a well-behaved vector, then drop a single outlier into it and watch the scale stretch until the bulk error explodes.

```

import numpy as np

def quantize(w, bits=4):
    s = np.max(np.abs(w)) / (2 ** (bits - 1) - 1)
    return s * np.round(w / s), s

rng = np.random.default_rng(0)
bulk = rng.normal(0, 1, 64)          # well-behaved channels
clean = bulk.copy()
spiked = bulk.copy()
spiked[0] = 60.0                    # one emergent outlier channel

for name, w in [("no outlier", clean), ("one outlier", spiked)]:
    q, s = quantize(w, bits=4)
    err = np.abs(q[1:] - w[1:]).mean() # error on the bulk only
    print(f"{name:12s} scale={s:6.2f} bulk mean abs error={err:.3f}")

```

Three answers to the outlier

GPTQ, AWQ, and SmoothQuant attack the outlier from three different angles, and the angle each chooses is what fixes its strengths and limits.

- **GPTQ** quantizes weights only, one layer at a time, and treats it as a reconstruction problem. It uses approximate second-order information, the Hessian of the layer's output error, to quantize weights column by column while updating the remaining full-precision weights to absorb the error just introduced (Frantar et al. 2022). This pushes the bitwidth to three or four bits per weight with little quality loss, and it touches activations not at all.
- **AWQ** also quantizes weights only, but starts from the observation that not all weights matter equally. Protecting roughly the top one percent of salient weight channels, identified by watching the activation magnitudes rather than the weights, removes most of the error. AWQ folds that protection into a per-channel scaling found by search, with no backpropagation and no reconstruction, so it does not overfit the small calibration set (Lin et al. 2023).
- **SmoothQuant** targets the harder regime where activations are quantized too. It applies a mathematically equivalent transformation that divides the activations by a per-channel factor and multiplies the weights by the same factor, migrating the difficulty out of the spiky activations and into the smooth weights (Xiao et al. 2022). After this smoothing, both can be taken to INT8, which unlocks integer matmuls for the whole layer, not just integer storage.

Two axes that organize the choices

That last distinction is the second design axis: **weight-only versus weight-and-activation**. Weight-only quantization (GPTQ, AWQ) shrinks the footprint and the bytes that decode must read, which is exactly the bandwidth-bound decode regime. It does not by itself speed the compute-bound matmuls of a large-batch prefill, because the activations are still high precision and the weights are dequantized before the multiply. Weight-and-activation quantization (SmoothQuant, FP8) lets the matmul itself run in the small format, which is what helps when compute, not bandwidth, is the bottleneck.

The **formats** are the alphabet these methods write in. INT8 and INT4 are uniform integer grids with a scale, cheap and well supported. FP8 comes in two NVIDIA-specified encodings, E4M3 with more mantissa for precision and E5M2 with more exponent for range, and is native on recent hardware so it accelerates the multiply itself (Micikevicius et al. 2022). GGUF is not a number format but a container: the file standard of the llama.cpp ecosystem, packing weights in block-wise "k-quant" schemes together with all metadata, so a quantized model is one portable file that runs on CPU and Apple silicon as readily as on a GPU (project 2023).

Lever two: stop moving the matrix

Fused kernels answer the memory-traffic half. FlashAttention computes exact attention without ever materializing the score matrix. It tiles the query, key, and value matrices into blocks that fit in fast on-chip SRAM, and it computes the softmax online: as each key block arrives it keeps a running maximum and a running normalizer, rescaling the partial output so the final result equals the full softmax without the full matrix ever existing (Dao et al. 2022).

$$m_i = \max(m_{i-1}, \tilde{m}_i), \quad \ell_i = e^{m_{i-1}-m_i} \ell_{i-1} + e^{\tilde{m}_i-m_i} \tilde{\ell}_i$$

The output never leaves SRAM until it is final, the $n \times n$ matrix never touches high-bandwidth memory, and the backward pass recomputes blocks on the fly rather than storing them. This is the core trade the kernel makes: spend extra arithmetic to avoid memory traffic, which is the right trade precisely because attention was memory-bound. Figure 19.2 contrasts the two data paths: the naive kernel parks the full score matrix in high-bandwidth memory, while the fused kernel keeps every intermediate inside on-chip SRAM and lets only the final output return.

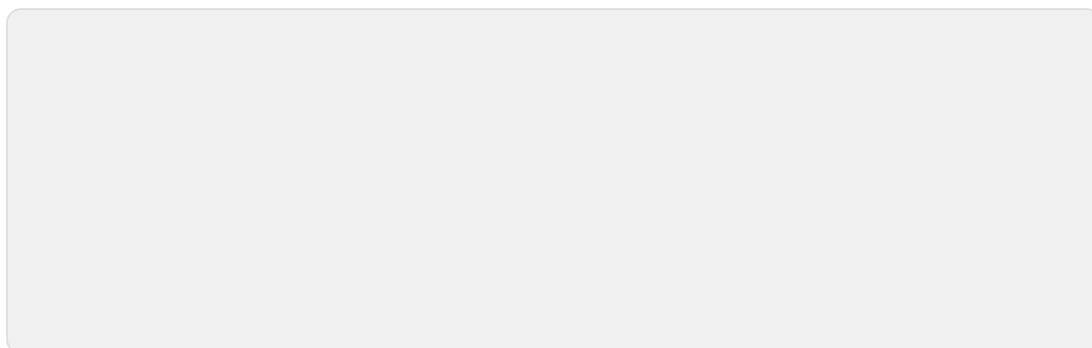


Figure 19.2. Naive attention writes and reads the full score matrix through HBM, while FlashAttention streams Q, K, V tiles through SRAM and keeps a running max and normalizer so the matrix never lands in HBM. After Dao et al. (2022).

The routing decision

The two levers compose into a single routing decision. The bottleneck the engine is fighting picks the quantization scheme, and the fused attention kernel then sits underneath whichever path is chosen, as Figure 19.3 lays out.

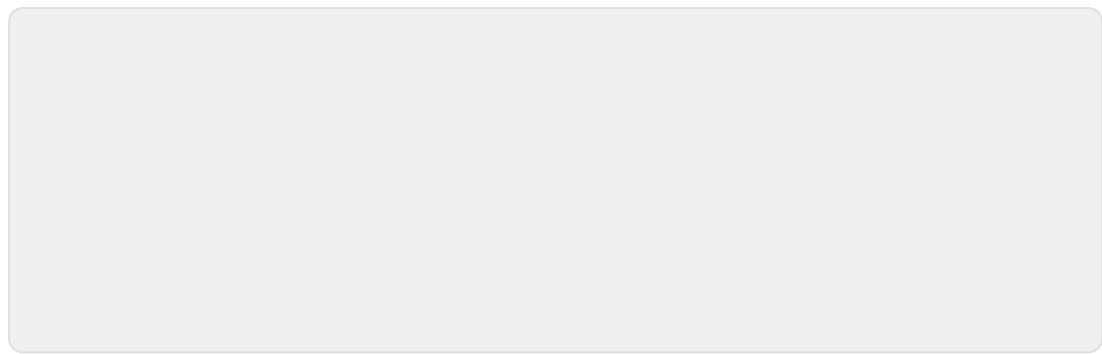


Figure 19.3. The serving regime routes the quantization choice, and a fused attention kernel sits beneath both paths before reaching the engine.

Reading the regime off the bottleneck is the chapter's practical core. Bandwidth-bound decode favors weight-only GPTQ or AWQ; a compute-bound prefill or a throughput-first deployment favors FP8 or SmoothQuant so the matmul runs small; CPU and Apple silicon targets favor GGUF k-quants for portability; an NVIDIA production stack reaches for TensorRT-LLM to fuse and compile down to the hardware.

How the engines came to package this

Both levers reached today's form through a short, sharp history, and the serving engines are where the two tracks were finally wired together.

On the quantization side the story is driven by the outlier. The first wave of fixes isolated the outliers in higher precision while quantizing the rest. GPTQ (2022) then showed that careful second-order weight reconstruction reaches three to four bits with negligible loss, making weight-only low-bit serving practical (Frantar et al. 2022). SmoothQuant (2022) attacked the activation side by moving the difficulty into the weights, enabling full INT8 with reported speedups around one and a half times and memory roughly halved (Xiao et al. 2022). AWQ (2023) reframed weight-only quantization around salience and per-channel scaling, reporting more than threefold speedups over the FP16 baseline while avoiding the overfitting risk of reconstruction (Lin et al. 2023). The format frontier moved in parallel, from FP16 to INT8 to INT4 on the weight side, and to FP8 once hardware made the small float native (Micikevicius et al. 2022).

Kernels evolved on a separate track aimed at the same bound. FlashAttention established the IO-aware, never-materialize design (Dao et al. 2022). FlashAttention-2 rebalanced the work, parallelizing across the sequence dimension and reducing non-matmul operations to push utilization higher (Dao 2023). FlashAttention-3 targeted the Hopper generation specifically, overlapping the asynchronous tensor-core and memory engines and adding low-precision FP8 paths, so the kernel and the quantization story converged on the same hardware (Shah et al. 2024).

The engines packaged these advances into systems. FasterTransformer was the early hand-optimized baseline. vLLM made paged KV-cache management the centerpiece, treating cache memory like operating-system virtual memory to cut fragmentation, and it pairs that with quantized weights and FlashAttention-style kernels (Kwon et al. 2023). SGLang added RadixAttention to reuse shared prefixes across requests, which matters for the structured and agentic workloads of later chapters (Zheng et al. 2023). TensorRT-LLM is NVIDIA's compiler-driven engine, fusing kernels and selecting low-precision paths for its own hardware, distributed as an open repository rather than a paper (NVIDIA 2023).

The engines wire these together rather than reinventing them. vLLM exposes quantization as a serving option over its paged cache (`LLM` , `vllm/entrypoints/llm.py`) and dispatches attention to a FlashAttention backend; SGLang layers prefix reuse on top (`RadixCache` ,

`python/sglang/srt/mem_cache/radix_cache.py`); TensorRT-LLM compiles a model definition into a fused, low-precision engine for a target GPU (github.com/NVIDIA/TensorRT-LLM). The common pattern is the same: calibrate or convert the weights once into a quantized artifact, load it into an engine that already owns a fused attention kernel and a cache manager, and serve.

What each lever costs

Every choice in this chapter spends one resource to buy another, and the right point depends on the serving regime, not on a universal best.

- **Weight-only versus weight-and-activation.** Weight-only quantization is the right tool for bandwidth-bound decode: it shrinks the footprint and the bytes read per token, and the matmul still runs in higher precision after dequantization. It does little for a compute-bound prefill. Quantizing activations too, with SmoothQuant or FP8, runs the matmul in the small format and helps the compute-bound regime, at the cost of confronting the activation outliers head-on.
- **Granularity versus overhead.** Finer scales, per channel or per small group, recover accuracy that per-tensor scaling loses, but each block carries its own scale and each matmul pays a dequantization step. Coarser is faster and smaller; finer is more accurate. The knee depends on how spread the tensor's values are.
- **Precision versus capability.** Lower bitwidth always reduces footprint and usually raises throughput, but it perturbs the model's outputs. How much that perturbation costs depends on what you measure, which is the subject of the constraint arrow below.
- **Kernel arithmetic versus memory traffic.** FlashAttention recomputes in the backward pass and does extra rescaling rather than store the score matrix. This is a net win only because attention was memory-bound; the same recompute would be wasteful on a compute-bound kernel.

The evaluation gate

The last trade-off, precision against capability, is the one that does not settle inside this chapter. It is paid one layer up, at evaluation, and the discipline of treating a quantized model as a model change is what keeps the saving honest. [Figure 19.4](#) sketches the concern that makes this gate matter: an aggregate metric can stay nearly flat as bitwidth drops while a specific capability falls away faster, and the gap between the two is exactly the cost a single perplexity number does not report.

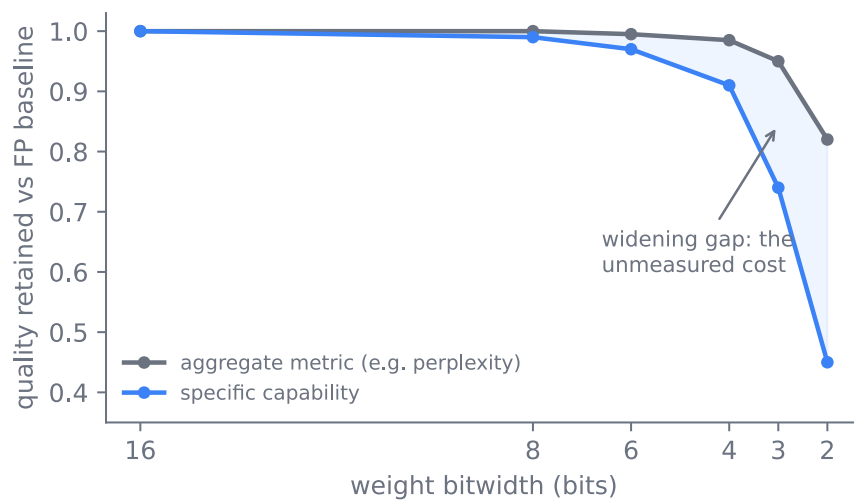


Figure 19.4. Schematic illustration of the contested concern that an aggregate metric such as perplexity can stay close to the full-precision baseline as weight bitwidth falls, while a specific capability degrades faster. The curves are idealized to show the divergence the chapter flags as unsettled, not measured benchmark scores. Author's schematic, illustrative only.

WHAT'S CONTESTED

Whether low-bit quantization preserves capability uniformly is unsettled. Aggregate metrics like perplexity often stay close to the full-precision baseline at four bits, which is why four-bit weight-only serving is common. But there is evidence that the damage is uneven: long-context recall, multi-step reasoning, multilingual coverage, and rare factual knowledge can degrade more than a perplexity number suggests, and the worst-hit cases may be exactly the capabilities a deployment cares about. The live questions are how far bitwidth can drop before specific capabilities break, whether weight-only and weight-and-activation schemes degrade the same skills, and whether the standard benchmarks even surface the loss. Treat "lossless at four bits" as a claim about an average metric, not a guarantee about a task.

CONSTRAINT ARROW

Quantization trades numerical precision for footprint and speed, and the bill for that trade is paid one layer up, at evaluation. A quantized model is a different model: its logits are perturbed, and the only question that matters is whether it still produces the outputs that [Chapter 27](#) measures. The footprint win is read off the weight file immediately; the capability cost is invisible until the benchmark and judging harness of [Chapter 27](#) and [Chapter 28](#) re-run on the quantized weights. A serving decision that looks like pure systems engineering is therefore gated by an evaluation that sits above it, and shipping a quantized model without that re-run is shipping an unmeasured model.

The operational caution is the one the arrow names. A quantized artifact is cheap to produce and easy to mistake for free. The footprint saving is real and immediate; the capability question is not answered until the evaluation of [Chapter 27](#) re-runs on the quantized weights, not on the original. The discipline that keeps this honest is to treat every quantization recipe as a model change subject to the same measurement, and to record the format, the bitwidth, and the granularity alongside the scores so the trade is auditable.

Further reading

- Frantar et al., "GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers," 2022. [arXiv:2210.17323](#)
- Lin et al., "AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration," 2023. [arXiv:2306.00978](#)
- Xiao et al., "SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models," 2022. [arXiv:2211.10438](#)
- Micikevicius et al., "FP8 Formats for Deep Learning," 2022. [arXiv:2209.05433](#)
- Dao et al., "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness," 2022. [arXiv:2205.14135](#)
- Dao, "FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning," 2023. [arXiv:2307.08691](#)
- Shah et al., "FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-precision," 2024. [arXiv:2407.08608](#)
- Kwon et al., "Efficient Memory Management for Large Language Model Serving with PagedAttention" (vLLM), 2023. [arXiv:2309.06180](#)
- Zheng et al., "SGLang: Efficient Execution of Structured Language Model Programs," 2023. [arXiv:2312.07104](#)
- NVIDIA, "TensorRT-LLM," 2023. [github.com/NVIDIA/TensorRT-LLM](#)
- llama.cpp project, "GGUF format," 2023. [github.com/ggml-org/llama.cpp](#)

20. Structured and Long-Context Inference

Two demands sit on top of a serving engine that the earlier chapters did not address: a caller who needs the output to be valid JSON or to match a grammar, and a caller whose context is far longer than the cache can comfortably hold. By the end of this chapter a reader can explain how a decoder is constrained to a formal language by masking logits against a finite-state machine, why that mask can be precomputed and even skip ahead several tokens at once, and how a long context is served by deciding which keys and values to keep, from sliding windows to attention sinks to heavy-hitter eviction.

Two ways to bend a fixed model

The decoder of Chapter 18 emits one token at a time by sampling from a distribution over the whole vocabulary. Two callers stress that loop in opposite directions, and neither is willing to retrain the model.

The first caller wants structure. A program that calls a model to extract fields, choose a tool, or fill a form needs the output to parse: valid JSON against a schema, a value from a fixed enum, a SQL statement that a parser accepts. An unconstrained model produces text that is usually well-formed and occasionally not, and the occasional failure is the expensive one, because the program downstream cannot recover from a missing brace or an invented field name. Prompting and retrying narrows the failure rate but never closes it, and each retry pays a full generation. The problem is to make malformed output impossible rather than improbable, without retraining the model and without paying a large per-token tax.

The second caller wants length. A request may carry a long document, a multi-hour transcript, or an agent history that grows without bound, and the key-value cache of Chapter 17 grows linearly with that length. Past some point the cache no longer fits, or fits only by capping the batch size so low that throughput collapses. Worse, a model trained at one context length often degrades when asked to attend over a much longer one at inference time, even before memory runs out. The training-time fixes for that, extending the positional encoding through RoPE scaling and YaRN, live in Chapter 6. This chapter owns the inference-time question: given a fixed model and a context too long to keep whole, which keys and values do you keep, and how do you keep generation coherent when you drop the rest.

The two problems are not obviously related, yet both are answered by intervening in the decode loop rather than retraining: one masks the distribution before sampling, the other prunes the cache that attention reads. They share this chapter because they are the two ways a serving engine bends a fixed model to a caller's constraint. The rest of this chapter walks each as a single track, and each track is a sequence in which every step removes a cost the previous one paid.

Track one: constraining the decode loop

The mask, and why it is not a one-liner

The core idea for structure is small. At each decode step the model produces a logit for every token in the vocabulary. If you know which tokens could legally come next under the target format, you set the logits of all the others to negative infinity before the softmax, so they have zero probability and cannot be sampled. Generation then walks only the legal continuations, and the output is guaranteed to be a member of the formal language. Nothing about the model changes; the intervention is a mask applied between the logits and the sampler.

The work is in knowing the legal set at every step. Express the target format as a formal language: a regular expression for a phone number or an enum, a context-free grammar for JSON or for a programming language. Compile that into an automaton, a finite-state machine for a regular language or a pushdown automaton for a context-free one. The automaton has a current state; from that state, only some tokens keep the string within the language. The mask at each step is the set of vocabulary tokens whose first character, or whose whole spelling, the automaton accepts from its current state. After a token is sampled, advance the automaton and repeat.

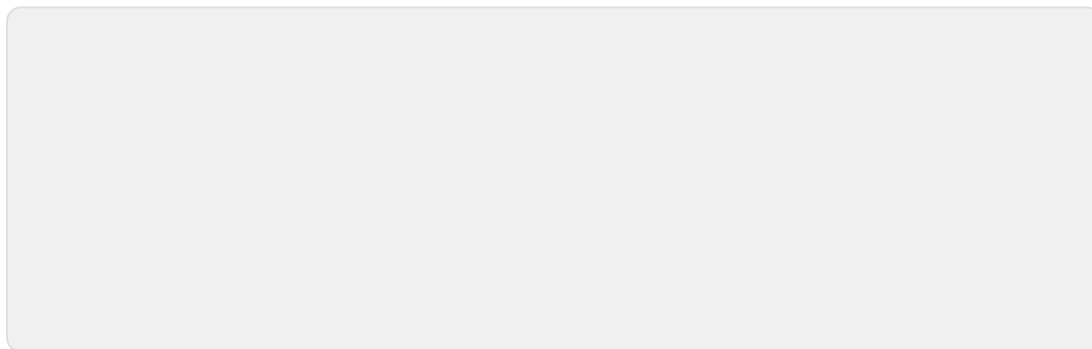


Figure 20.1. Constrained decoding as a mask. Logits are masked to the tokens an FSM allows from its current state, then the sampled token advances the FSM.

Try changing `allowed` to a different set of legal tokens and watch the disallowed probabilities collapse to exactly zero while the legal ones renormalize.

```
import numpy as np
vocab = ['{', '}', "'", 'name', 'age', ':', ',', 'true', 'hello', '4
logits = np.array([2.1, 0.3, 3.0, 1.2, 1.0, 0.5, 0.4, 0.8, 2.5, 1.1]
# FSM is just after '{' : the only legal next token is an opening qu
allowed = {'"'
mask = np.array([t in allowed for t in vocab])
masked = np.where(mask, logits, -np.inf)
def softmax(x):
    x = x - np.max(x[np.isfinite(x)])
    e = np.exp(np.where(np.isfinite(x), x, -np.inf))
    return e / e.sum()
p_free, p_masked = softmax(logits), softmax(masked)
for t, a, b in zip(vocab, p_free, p_masked):
    print(f'{t:>6} unconstrained={a:.3f} masked={b:.3f}')
print('masked mass on legal tokens:', round(p_masked[mask].sum(), 3))
```

The catch that makes this an engineering problem rather than a one-liner is the mismatch between the automaton, which is defined over characters, and the model, which emits tokens that are multi-character pieces of the byte stream from [Chapter 5](#). A token may span several automaton transitions, and only some tokens are valid from a given state. Computing the allowed-token set naively means walking the automaton over the spelling of every one of tens of thousands of vocabulary entries, at every step, which would dominate the cost of decoding.

So the first implementation of this idea was correct but slow, a per-step vocabulary scan against a hand-written automaton, and the history of the track is a chain of removing that cost.

Removing the per-step scan: a precomputed index

The design that makes constrained decoding practical is to precompute the relationship between automaton states and allowed tokens once, before generation, rather than at every step. Willard and Louf's formulation, the basis of the Outlines library, builds an index over the vocabulary keyed by automaton state: for each state of the finite-state machine, the set of tokens that are legal from it, and the state each of those tokens leads to (Willard and Louf 2023). With that index in hand, the per-step work is a dictionary lookup by current state, not a scan over the vocabulary, which turns the per-token overhead from proportional to vocabulary size into roughly constant. The index is built from the regular expression and the tokenizer once and reused for every request that shares the format. This 2023 leap is what made guided generation cheap enough to use by default.

Removing the forced tokens: jump-forward over a compressed machine

Even with a constant-time mask, a second cost remains, because large stretches of a constrained output are forced. Inside `{"name": " "}`, the next several characters are not a choice, they are the only continuation the grammar allows until the value begins. A token-by-token loop still runs a full forward pass for each of those forced characters, paying decode cost to generate text that was never in doubt. The compressed finite-state machine, introduced in SGLang, collapses runs of single-transition edges in the automaton into one edge, so the runtime can recognize a forced span and emit it directly (Yin et al. 2024). This is jump-forward decoding: when the machine has only one path forward, write those tokens into the sequence without a model call, then resume sampling where the format branches. The model is consulted only where the output is genuinely undetermined.

Figure 20.2 contrasts the two regimes. The forced span between two branch points is one edge in the compressed machine, written in a single step, while the model runs only at the points where the grammar leaves a real choice.

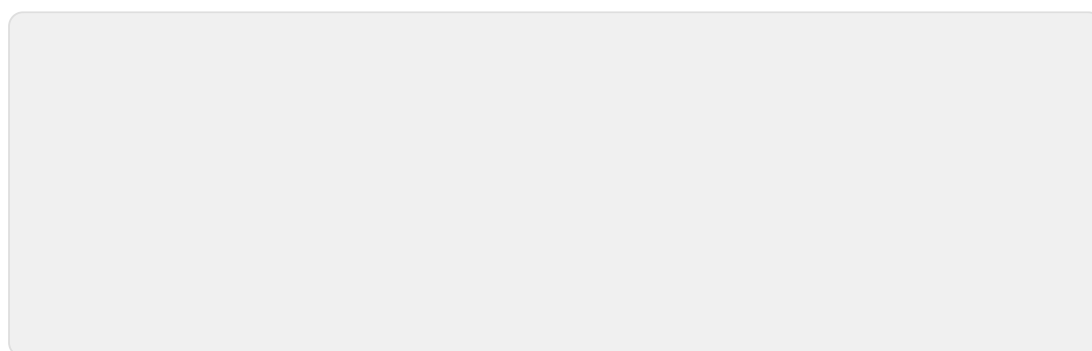


Figure 20.2. Jump-forward decoding over a compressed FSM. Forced spans collapse to a single edge emitted without a model call, after Yin et al. (2024).

Removing the grammar overhead: hiding behind the forward pass

The precomputed index and jump-forward both target regular languages. The remaining step generalizes the mask from regular to context-free languages while keeping the per-step cost near zero. A context-free grammar needs a stack, not just a state, so the allowed-token set depends on the current stack, which seems to defeat precomputation. XGrammar's insight is to split the vocabulary into tokens whose legality does not depend on the stack, which can be checked once and cached, and the smaller set of context-dependent tokens that must be checked against the live stack at runtime (Dong et al. 2024). With a persistent stack structure

and by overlapping the grammar computation with the model's forward pass on the accelerator, the engine drives a full grammar constraint at close to the speed of unconstrained decoding. The arc of this track, taken whole, pushes the constraint from a per-step vocabulary scan down to a precomputed table, then to skipped tokens, then to a grammar engine that hides behind the forward pass.

Track two: pruning the cache

Attention is concentrated, so most keys are dead weight

The long-context half rests on a different observation. Attention does not weight all past tokens equally. For most heads, the probability mass concentrates on a small subset of positions: the most recent tokens, plus a few specific earlier ones. If most keys and values receive almost no attention, keeping all of them is paying full memory for tokens that barely contribute. The design move is to keep only the keys and values that matter and let attention run over that reduced set, trading a controlled approximation for a cache that no longer grows without bound. The first cut at this rule is blunt, and the history of the track is a chain of making it less blunt.

The blunt rule and the dropped-prefix failure

The simplest keep rule is a sliding window: retain only the last w tokens and drop everything older. This bounds the cache to a constant size and matches how local-attention models already behave. But a plain sliding window fails in a specific and revealing way. When the earliest tokens fall out of the window, perplexity spikes and generation degrades sharply, far more than dropping unimportant tokens should cost.

The explanation is the attention sink, the central finding of StreamingLLM (Xiao et al. 2024). Because the softmax over attention scores must sum to one, it cannot output nothing; when a head has no informative place to attend, it dumps its excess attention onto the first few tokens of the sequence, which therefore receive large scores regardless of their content. These initial tokens act as a sink. Drop them, as a naive sliding window does, and every head loses the place it was parking its spare attention, and the distribution destabilizes. The fix is to keep a handful of initial tokens permanently, the sinks, alongside the sliding window of recent tokens. With four sink tokens plus a recent window, a model trained at a fixed length can stream over millions of tokens with stable perplexity, no fine-tuning required (Xiao et al. 2024).

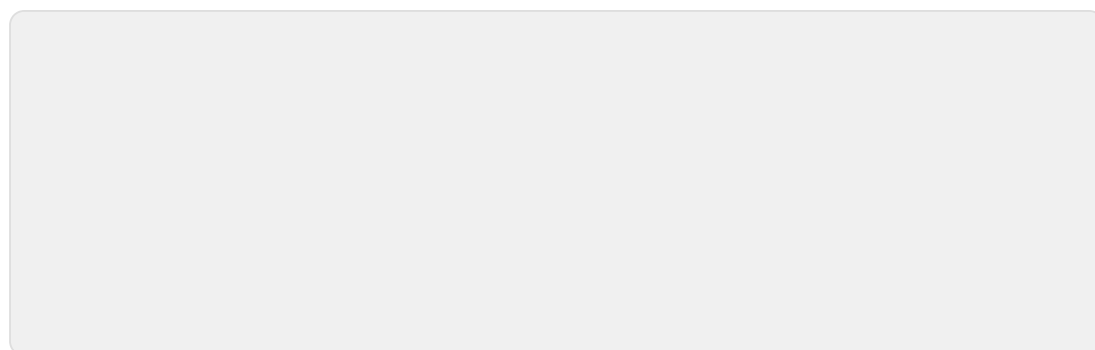


Figure 20.3. Sink-plus-window cache. A few initial sink tokens and a recent window stay in cache and feed attention, while the middle tokens are evicted.

Letting content decide: eviction by attention history

Sinks plus a window is a fixed rule that ignores content; the middle of a long document may hold the one fact the answer needs. Content-aware compression keeps the cache small while

letting which tokens survive depend on the input. H2O reframed the cache as an eviction problem and keeps the heavy hitters: a small set of tokens that have accumulated the most attention so far, retained alongside the most recent tokens, with the rest evicted (Zhang et al. 2023). SnapKV makes a sharper observation for the prompt: each head consistently attends to particular prompt positions, and those positions can be identified from the attention in a small observation window at the end of the prompt, then the rest of the prompt's cache compressed away before generation even starts (Li et al. 2024). Both keep a budget-sized cache whose contents are chosen by the model's own attention rather than by position alone.

The two arcs are dated steps that line up step for step: window attention and recompute baselines bounded memory but broke on the dropped prefix; H2O in 2023 kept heavy hitters by accumulated attention (Zhang et al. 2023); StreamingLLM, also in 2023, explained why naive windows fail and fixed it with the attention sink (Xiao et al. 2024); SnapKV in 2024 pushed compression into the prompt itself using the end-of-prompt observation window (Li et al. 2024). The frontier continues toward query-aware and learned eviction, deciding what to keep based on what the current generation is actually attending to. A connected development is designing the sink in at training time: models can be trained with a dedicated sink token so the attention parking is explicit rather than emergent, which makes the inference-time keep rule cleaner. That training-time choice belongs to Chapter 6; this chapter inherits its consequence.

The two lineages, side by side

Figure 20.4 lays the two tracks together. Each step removes a cost the previous one paid: the structured track pushes the constraint from a per-step vocabulary scan down to a precomputed table, then to skipped tokens, then to a grammar engine that hides behind the forward pass; the long-context track moves from a position-only window that breaks, to keeping a few sinks, to choosing what to keep by attention.

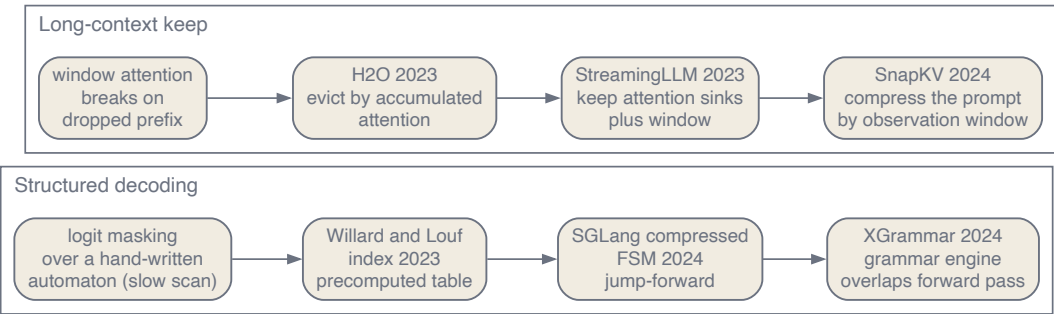


Figure 20.4. Two parallel lineages: constraining the decode loop and pruning the cache. Each step removes a cost paid by the one before it.

Where the two tracks converge

The two tracks meet at the same place: each bends a fixed model by accepting an approximation, and the open question is what the approximation costs. For the structured track the cost is mostly bounded and known; for the long-context track it is genuinely unsettled.

WHAT'S CONTESTED

KV-cache compression trades memory for an approximation of full attention, and how much that approximation costs is genuinely unsettled. On many long-context benchmarks, sink-plus-window and heavy-hitter eviction recover most of the quality at a fraction of the memory. But the dropped tokens are gone, and a task that later needs an evicted token, a fact buried in the middle of a long document, a constraint stated once early, cannot recover it. The size of that penalty depends on the task in a way that

aggregate perplexity hides: streaming language modeling tolerates eviction well, while retrieval over long context and multi-hop reasoning are far more sensitive. Whether fixed-budget compression is safe for a given workload, or whether the model needs the whole cache, is a per-task empirical question, not a settled default. Query-aware and learned eviction policies aim to narrow this gap, and whether they close it remains open.

CONSTRAINT ARROW

The cache layout of [Chapter 17](#) decides how cheaply this chapter's keep rules can run. Paged key-value storage makes eviction a matter of freeing blocks and rewriting a block table, and makes sink-plus-window a matter of pinning the sink blocks and recycling the rest, rather than copying a contiguous buffer. The per-token cache size set by the attention variant in [Chapter 6](#), grouped-query or latent attention, sets the baseline memory that compression starts from, so a model already economical in key-value heads needs less aggressive eviction to fit. The compression policy a serving engine can afford is shaped by the memory manager one layer down.

What each move costs

Each technique buys its benefit at a stated cost.

- **Constraint versus fluency.** Masking guarantees the output parses, but it can force the model down a path its own distribution disfavors, lowering quality on the content inside the structure. A constraint that is too tight, an over-specified schema, can push the model into locally legal but globally awkward text, because the mask removes the token the model wanted without removing the reason it wanted it. The constraint should be as loose as correctness allows.
- **Index build versus reuse.** The precomputed index pays an up-front cost to compile the regular expression or grammar against the tokenizer ([Willard and Louf 2023](#)). For a format used across many requests this amortizes to nothing, but for a one-off schema generated per request the build can rival the generation it saves. Engines cache compiled grammars by format for exactly this reason.
- **Jump-forward correctness.** Emitting forced tokens without a model call is only safe where the automaton truly has a single continuation, and tokenizer boundaries complicate that: the jumped text must re-tokenize the same way the model would have produced it, or the cache and the output diverge ([Yin et al. 2024](#)). The speedup is real but the boundary handling is where the bugs live.
- **Compression budget versus recall.** A smaller cache budget saves more memory and fits more concurrent requests, but raises the chance that an evicted token was the one the answer needed. The budget is the dial between throughput and the recall of long-range facts, and the safe setting is workload-specific.

Schematic, not measured: recovered quality rises steeply with the first slice of cache budget and flattens, so a fraction of the memory recovers most of the quality, while the sensitivity dial sets how fast a retrieval-heavy task pays for an evicted token.

- **Sink count and window size.** Too few sink tokens and the parking insight fails; too large a window and the memory saving shrinks. The small published defaults, a handful of sinks and a window in the thousands, are starting points, not constants ([Xiao et al. 2024](#)).

Putting it in the engine

A constrained decoder is a mask hooked into the sampler. The engine compiles the format into an automaton and its index once, then at each step looks up the allowed-token set for the current state, sets every other logit to negative infinity, samples, and advances the automaton. In vLLM this is a logits processor invoked inside the sampling path, backed by a structured-output backend such as XGrammar or Outlines that owns the grammar and the index. With

jump-forward enabled, the runtime checks the compressed machine before each model call: if the current state has a single forced continuation, it appends those tokens directly and only invokes the model when the format branches. The grammar engine runs alongside the forward pass so the mask is ready when the logits are (Dong et al. 2024).

A compressing long-context engine is the memory manager of Chapter 17 with a keep policy on top. For sink-plus-window streaming, the engine pins the first few token blocks as sinks, keeps a rolling window of recent blocks, and frees the rest, so the cache stays at constant size as generation continues. For attention-based eviction, the engine tracks an attention score per cached token, accumulated for heavy hitters or measured in the end-of-prompt observation window for prompt compression, and evicts the lowest-scoring tokens down to the budget, freeing their blocks back to the pool. The keep policy is a small layer; paging is what makes it cheap, because evicting a token is freeing a block, not compacting a buffer.

Two operational realities are worth naming. First, a constraint is only as correct as its grammar: a schema that permits a value the parser downstream rejects moves the failure rather than removing it, so the grammar and the consumer must agree on the exact format. Second, compression quality is workload-shaped in a way aggregate metrics hide, so the honest test is the downstream task, retrieval accuracy or answer correctness at the chosen budget, not perplexity, which can stay flat while the specific evicted fact is exactly the one a query needed.

Read through the capability, efficiency, and trust lens, this chapter is mostly an efficiency and trust story: constrained decoding buys trust, a guarantee the output parses, at a small efficiency cost that the index and jump-forward nearly erase; long-context compression buys efficiency, a bounded cache, at a trust cost, the risk of an evicted fact, that query-aware policies are trying to bound.

Further reading

- Willard and Louf, "Efficient Guided Generation for Large Language Models" (Outlines), 2023. [arXiv:2307.09702](https://arxiv.org/abs/2307.09702)
- Yin et al., "Fast JSON Decoding for Local LLMs with Compressed Finite State Machine" (jump-forward decoding), LMSYS, 2024. lmsys.org/blog/2024-02-05-compressed-fsm
- Dong et al., "XGrammar: Flexible and Efficient Structured Generation Engine for Large Language Models," 2024. [arXiv:2411.15100](https://arxiv.org/abs/2411.15100)
- Xiao et al., "Efficient Streaming Language Models with Attention Sinks" (StreamingLLM), ICLR 2024. [arXiv:2309.17453](https://arxiv.org/abs/2309.17453)
- Zhang et al., "H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models," NeurIPS 2023. [arXiv:2306.14048](https://arxiv.org/abs/2306.14048)
- Li et al., "SnapKV: LLM Knows What You are Looking for Before Generation," NeurIPS 2024. [arXiv:2404.14469](https://arxiv.org/abs/2404.14469)

21. Agent Architectures

An agent is a language model placed inside a loop that lets it act, observe the result, and act again. This chapter is about the shape of that loop and the parts it decomposes into. By the end a reader can explain the standard four-part decomposition of an agent, why the control loop interleaves reasoning with action rather than separating them, and why tool use is the capability that turns a text generator into something that changes the world.

A plan that meets the world

Imagine the obvious way to build an agent. Ask the model to read the task, think it through, and write the full plan: step one, run the tests; step two, open the failing file; step three, fix the off-by-one on line forty; step four, rerun. Then execute the plan, one step at a time, top to bottom.

This design is clean, legible, and wrong. The plan was written before any step ran, which means it was written before the agent knew anything the steps would reveal. Step one runs the tests and the failure is not the one the model assumed. The bug is not on line forty, and it is not an off-by-one. Every later step in the plan now refers to a world that does not exist. The agent is holding a script for a play whose first scene already went differently. It can follow the script into nonsense, or it can throw the script away. Either way, planning the whole task up front bought nothing, because the first observation invalidated it.

The lesson is that the structure of an agent is not a plan. It is a loop. The rest of this chapter is about the shape of that loop, why it interleaves thinking with doing instead of separating them, and what it decomposes into.

Closing the gap

Step back to why a loop is needed at all. A base model maps a prompt to a continuation. That is enough to answer a question whose answer is already latent in the weights, and useless for a task whose answer depends on the state of the world: the current contents of a file, the result of a query, whether a test passes after an edit. The model cannot read that state, and it cannot change it. It emits text and stops.

The essential problem of an agent is to close that gap. The model needs a way to take an action in the world, see what the action produced, and feed that observation back into its next decision. It also needs to do this over many steps, because real tasks are not one action long: fixing a bug is read, hypothesize, edit, test, and very likely repeat.

Three constraints shape every answer to this problem, and they are worth naming before any architecture is drawn. The model has a bounded context window, so a long task cannot keep every observation in view at once. Each step costs a model call, so the loop has a latency and a price. And the model is probabilistic, so any single action can be wrong, which means the architecture has to tolerate and recover from mistakes rather than assume them away. The plan-then-execute design failed precisely because it ignored the third: it assumed its first guess would survive contact with the world.

Try changing the per-step reliability and the task length below to see why a single up-front plan is so fragile: a plan with no recovery succeeds only if every step lands, so its success probability is the per-step reliability raised to the number of steps.

```

import numpy as np
n = np.arange(1, 21)          # task length: 1 to 20 steps
for p in [0.99, 0.95, 0.90, 0.80]: # per-step reliability
    survive = p ** n          # P(all n steps land, no recovery)
    import matplotlib.pyplot as plt
    plt.plot(n, survive, marker="o", ms=3, label=f"p={p}")
plt.xlabel("steps in the plan (n)")
plt.ylabel("P(plan succeeds with no recovery)")
plt.ylim(0, 1); plt.legend(); plt.title("Why plan-then-execute is fr
plt.show()
for p in [0.95, 0.80]:
    print(f"p={p}: 10-step plan succeeds {p**10:.1%} of the time")

```

The loop, and why it interleaves

The design that works does not separate reasoning from acting. It interleaves them. The model produces a short reasoning trace, then an action, observes the result, then reasons again with that observation in hand. This is the **reason-and-act** pattern, named ReAct by Yao et al. (Yao et al. 2023), and the synergy is the point: the reasoning lets the model decide and adjust its plan as it goes, and the actions let it pull in facts that the reasoning alone would have to hallucinate. The plan is never written all at once. It is revised at every turn against what the last action returned, which is exactly the failure mode of the previous section turned into a virtue.

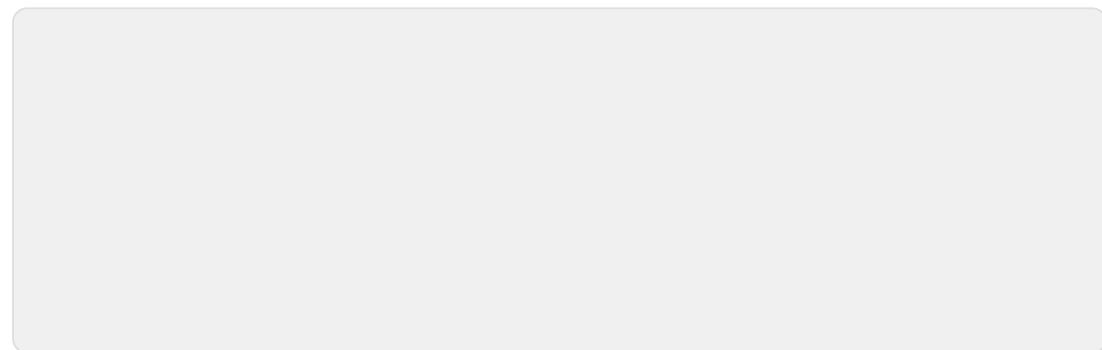


Figure 21.1. The reason-and-act loop. Each turn the model reasons, chooses an action, and folds the observation back in before reasoning again, after Yao et al. (2023).

The loop in [Figure 21.1](#) is the heart of the architecture, and its specific shape is what separates an agent from a single model call. Everything else in this chapter is either a part the loop animates or a consequence of running it in production.

The runtime contract that implements the loop is small to state. Create a session bound to an agent definition. Send the context to the model, dispatch the tool calls it returns, append the results, and repeat until the model signals done or the loop is interrupted. That sequence is the reason-and-act pattern in operational form. The mechanism that makes this contract survive production, durability across crashes, scheduling, interruption, is the harness, and it is the subject of [Chapter 23](#). This chapter owns the architecture; that one owns the engineering.

Anatomy of the loop

The loop has four moving parts. The dominant framing decomposes an agent into these four around a model that acts as the controller. The decomposition is Lilian Weng's (Weng 2023), and it has become the common vocabulary: **planning**, **memory**, **tool use**, and the **action** loop that ties them together.

- **Planning** turns a goal into a sequence of steps. At its simplest this is the model deciding, one step at a time, what to do next. At its most structured it is explicit task decomposition into subgoals, with self-reflection on past steps to correct course.
- **Memory** is what survives across steps. Short-term memory is the context window itself, the running transcript of what has happened. Long-term memory is an external store the agent reads from and writes to, because the window is too small to hold a long task. Memory as a system is the subject of Chapter 22; here it is one of the four parts the loop relies on.
- **Tool use** is how the agent reaches outside its weights: a function it can call to fetch a page, run code, query a database, or edit a file. A tool call is the agent's only way to read or change external state.
- **Action** is the loop that drives the other three: send the current context to the model, let it choose a tool call, execute the call, append the result, and repeat.

It helps to see the parts laid out by where they live. Planning is reasoning the model does inside its own forward pass. Short-term memory is the context window, also inside the model's view. Tool use is the single edge that crosses to the world outside the weights, and long-term memory is a store on the far side of that edge.

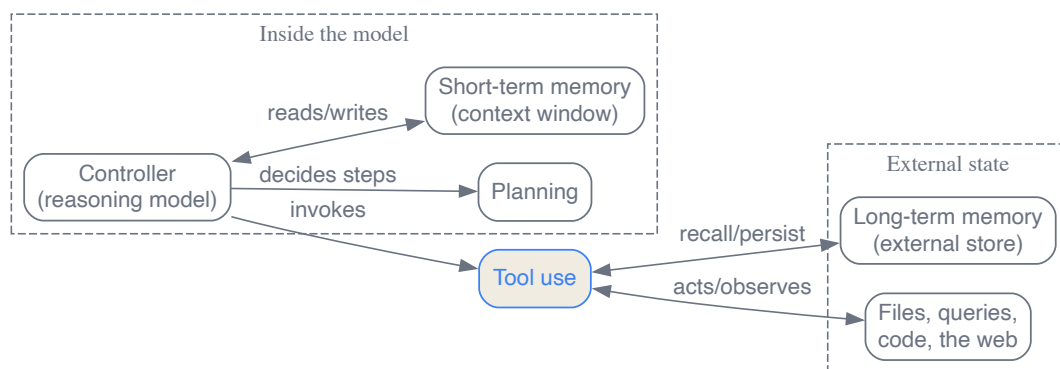


Figure 21.2. The four-part decomposition by where each part lives. Planning and short-term memory sit inside the model boundary; tool use is the only edge crossing to external state and long-term memory. After Weng (2023).

Of the four, tool use is the one that earns the name agent. Planning and memory shape how the model thinks; tool use is the only part that lets it reach past the weights and touch the world, the single edge in Figure 21.2 that crosses the boundary. Without it the loop spins on text alone.

How the pieces matured

The decomposition did not arrive fully formed. It is worth tracing the order in which the parts became real, because the order explains why the synthesis took the shape it did.

The first capability to mature was tool use as a learned skill. Toolformer (Schick et al. 2023) showed that a model could teach itself, in a self-supervised way, when to call an API and how

to splice the result back into its generation, which established that tool use is something a model can be trained to do rather than a feature bolted on at inference time.

In parallel, the reasoning side matured. Chain-of-thought prompting showed that letting a model write intermediate steps before an answer improved performance on multi-step problems, a thread [Chapter 13](#) follows in depth. ReAct's contribution was to fuse that reasoning with action in one interleaved trace ([Yao et al. 2023](#)), which directly attacked chain-of-thought's weakness: a pure reasoning chain has no way to check itself against the world, so it propagates its own errors and hallucinations, while interleaving a Wikipedia lookup between reasoning steps lets the model ground each step in a fact it just retrieved. The four-part decomposition was the synthesis that named the pieces ([Weng 2023](#)), with planning and memory as first-class components alongside tool use rather than implicit features of a prompt.

What has changed most recently is where the reasoning lives. Early agents carried their reasoning in the prompt: the harness instructed the model to think step by step and to interleave thoughts with actions. Reasoning models trained to deliberate internally, covered in [Chapter 14](#), move some of that work below the loop. The agent architecture is the same shape, a model in a reason-and-act loop, but the planning component is increasingly something the model does in its own trained reasoning rather than something the harness has to scaffold with prompting.

Living tensions

The architecture's choices are balances, and each has a cost worth naming.

- **Explicit planning versus letting the model reason as it goes.** Up-front task decomposition gives a legible plan a person can inspect, and it can keep a long task on track. It also commits to a structure before any observation exists, so a plan that meets a surprising result is a plan that has to be torn up. This is the cold-open failure of this chapter in miniature. Step-at-a-time reasoning adapts to every observation but has no global view and can wander. Most production agents lean toward the reason-and-act middle: a light plan, revised every step against what the last action returned.
- **Tool count versus model attention.** Every tool the agent can call is a schema in the model's context and a candidate it must consider. A handful of well-chosen tools is easy for the model to select among. A large catalog costs context on every turn and degrades selection accuracy, because the model's attention is finite and irrelevant tools actively hurt. This is the pressure that forces a tool-selection strategy, discussed below and in [Chapter 23](#).
- **Short-term versus long-term memory.** Keeping everything in the context window is simple and lossless until the window fills, at which point the task stalls. Externalizing memory lets the task run indefinitely but makes the agent's recall only as good as what it chose to write down and can retrieve. The boundary, what stays in the window and what moves to a store, is a design decision this chapter points at and [Chapter 22](#) owns.

WHAT'S CONTESTED

How much planning should be explicit is unsettled. One camp builds agents that decompose a task into a written plan of subgoals and reflect on it between steps, on the argument that a legible plan is more reliable and more auditable. The other camp argues that a capable reasoning model plans better implicitly, inside its own trained reasoning, than any prompt-level scaffold the harness can impose, and that explicit planning machinery adds brittleness for a benefit the model now provides on its own. The same debate runs through tool discovery: whether the harness should curate the tools the model sees each turn, or hand the model a way to discover tools and trust it to choose. Neither is settled, and the right answer moves as the underlying model's reasoning improves.

What the loop dispatches

The loop itself is short to write and unforgiving in its details. In pseudocode it is a `while` over model calls:

```
loop:
    response = model(context)
    if response has no tool call:
        return response          # the model is done
    result = execute(response.tool_call)
    context = context + response + result
```

Everything hard is hidden in `execute` and in the management of `context`.

The tools a loop dispatches are not uniform, and a loop that treats them all alike mishandles most of them. Tools divide by statefulness into three kinds. **Stateless** tools hold no state on the agent's side: a web fetch, a sandbox command given a sandbox ID, a plain REST call. They are the easy case and scale trivially. **Stateful-connection** tools, the Model Context Protocol (MCP) being the canonical one (Protocol 2026), carry negotiated capabilities and cursor state in a connection the loop must hold across calls. **Stateful-resource** tools keep their state in an external system, a database session or a long-running job, that the loop must correlate to across turns. The architectural point is that these three answer the question "what happens on restart?" differently: nothing, reconnect, or rediscover. The engineering of holding those connections alive through reconnects and token refreshes is the harness's job and lives in Chapter 23; the architecture's job is to know the three classes exist and not collapse them into one. Figure 21.3 lines the three up against the discriminating question.

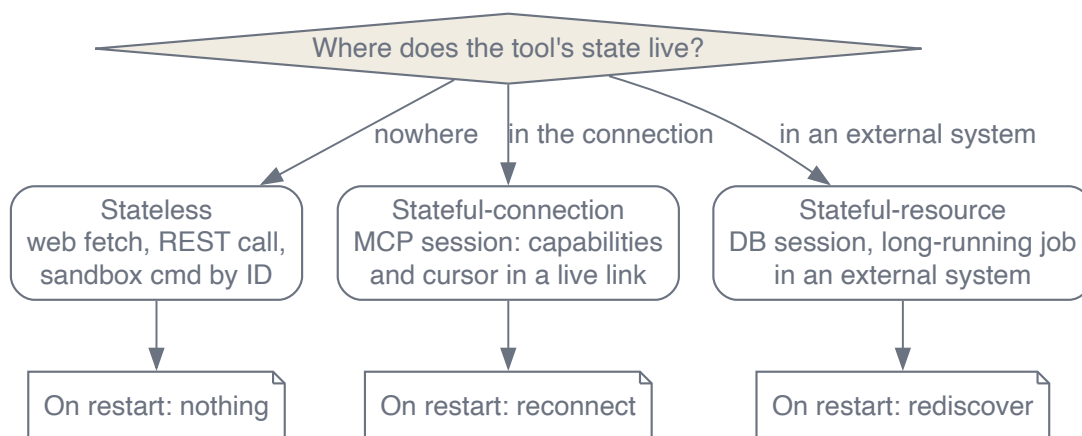


Figure 21.3. The three tool classes divided by where their state lives and what each must do on restart: nothing, reconnect, or rediscover.

The other recurring implementation question is which tools the model sees in a given turn. A static flat list, every tool in every prompt, is the simplest design and runs out at roughly thirty to fifty tools, because context cost and selection accuracy both degrade as the catalog grows. Past that point the options are to retrieve a relevant subset per turn, to show a pruned set per phase, or to let the model discover tools on demand. The selection mechanism is harness machinery; the reason it is forced is a model property, which is the constraint the next box names.

Drag the slider to see the constraint the next box describes: as the number of tools in context grows, function-calling accuracy falls off, which is why a flat catalog stops working past a few dozen tools.

Tool-selection accuracy degrades as the catalog grows, the measured effect behind tool-RAG and phased mounting.

CONSTRAINT ARROW

The model's tool-selection accuracy degrading as the tool count rises is a lower-layer property that dictates an upper-layer architecture. The Berkeley Function Calling Leaderboard (Yan et al. 2025) and ACEBench (Chen et al. 2025) both measure function-calling accuracy falling as the number of available tools grows, with robustness suffering specifically when irrelevant tools are present. That measured degradation is why an agent cannot simply register every tool it might ever need and send the whole catalog each turn. The model's finite attention forces tool-RAG, phased mounting, or model-driven discovery up at the architecture layer. A capability limit at the model sets a structural choice at the agent.

Further reading

- Weng, "LLM Powered Autonomous Agents," 2023. lilianweng.github.io
- Yao et al., "ReAct: Synergizing Reasoning and Acting in Language Models," 2023. [arXiv:2210.03629](https://arxiv.org/abs/2210.03629)
- Schick et al., "Toolformer: Language Models Can Teach Themselves to Use Tools," 2023. [arXiv:2302.04761](https://arxiv.org/abs/2302.04761)
- F. Yan et al. "The Berkeley Function Calling Leaderboard (BFCL): From Tool Use to Agentic Evaluation of Large Language Models." *ICML*, 2025. [OpenReview](#)
- "ACEBench: A Comprehensive Evaluation of LLM Tool Usage." *Findings of EMNLP*, 2025. [ACL Anthology](#)
- Model Context Protocol. "The 2026 MCP Roadmap." [MCP Blog](#)

22. Memory Systems

An agent that runs longer than a single context window has to remember what it did and survive losing the machine it did it on. This chapter is about the state an agent leaves behind: the durable record of a session, the workspace it mutated, and the memory it carries across sessions. By the end a reader can explain why a session log records intent separately from result, why branching a conversation is cheap but branching a workspace is not, and why a shared vector store is a data-breach risk rather than a relevance bug.

A commit that runs twice

Start with the failure, because it names the whole subject. A runner prompts the model, the model asks to run `git commit`, the sandbox runs it and the commit lands, and then the runner crashes in the window between the effect firing and its record reaching durable storage. Recovery reads the last durable state, which is pre-commit, and the agent replays: it issues the same `git commit` again.

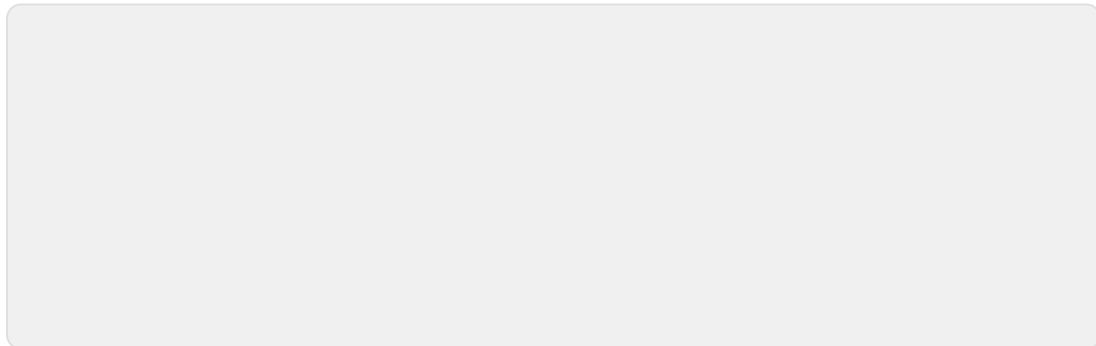


Figure 22.1. A crash between effect and persistence makes replay re-issue a non-idempotent commit, the intent-versus-result gap in its purest form.

This is not an exotic edge case. A harness has to survive any of four routine events: a harness crash, a replica reschedule, a sandbox loss, or a user resuming a session days later. All of them are the normal operating regime of a long-running agent, and reported agent retry rates of 15 to 30 percent make the replay above a regime to design for, not an accident (Fast.io 2025). The core difficulty it exposes is the gap between a side effect and its commit. A durable log records *intent* ("call `git commit`") and, separately, *result* ("commit succeeded"). If the harness crashes between those two writes, recovery re-issues the operation. For an idempotent operation that is harmless. For a non-idempotent one, a second commit, a duplicate ticket, a repeated payment, the store and the external world diverge silently, and nothing in the log can tell you it happened.

Surviving these events means writing state to a durable store, and the hard question is not where to put that state but what to record. The state an agent leaves behind comes in three shapes, and they fail in three different ways. The **session log** is append-only intent, and its characteristic failure is the intent-versus-result gap above. The **workspace** is mutable sandbox state, and its failure is durability: a volume that cannot follow a pod across a zone boundary, a snapshot taken too rarely. **Long-term memory** is the curated, retrievable store that outlives a session, and its failure is tenancy: a retrieval that crosses a user boundary is not a wrong answer, it is a data breach. One small contract spans all three, **append, read, snapshot, fork**, applied to state shapes that are anything but uniform. The design discipline is being honest about what each store does *not* guarantee. The rest of this chapter walks the three stores in that spirit: what each one buys, and what it refuses.

The session log: three ways to record a step

The session log can be built three ways, and the meaningful difference between them is what they record relative to the effect, which fixes where the intent-versus-result gap lands. [Figure 22.2](#) places all three on that one axis.

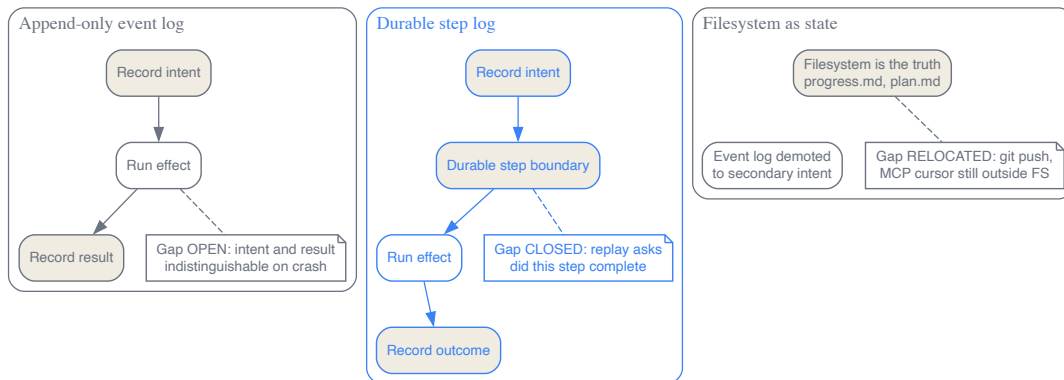


Figure 22.2. Three durability models differ in what they record relative to the effect, which fixes where the intent-versus-result gap sits.

The **append-only event log** writes every user message, model response, tool call, and tool result to a durable store. The log is the sole recovery artifact: any replica can serve any session by replaying it. This buys clean replay, clean audit, and transport-agnostic recovery, and it is the model that exhibits the intent-versus-result gap in its purest form. From the log alone, "requested but never ran" and "ran but never recorded" are identical: a tool call with no result. That is the gap the opening crash walked into.

The **durable step log** closes the gap by adding a third record between intent and result: a durable *step boundary* written before the effect executes. Replay skips any step whose outcome is already durable, so "we asked to do X" and "X completed" become two separate, individually-durable facts, and replay can ask "did this step already complete?" instead of assuming it did not. This is the answer durable-execution frameworks settled on, and the evolution is worth tracing. The plain event log gave clean replay but blurred intent and result. Restate (Ewen et al. 2025), Temporal (Davis 2025), and Inngest (Poly 2026) journal each step, run the effect, then record its outcome, formalizing the distinction the event log blurs. Temporal's own framing is that probabilistic LLM behavior makes naive retry logic insufficient, which is exactly the case durable journaling is built for (WorkOS 2025). By 2025 this crossed into the mainstream: AWS Durable Functions, Cloudflare Workflows, and Vercel's Workflow DevKit all shipped with AI-agent use cases as primary framing (Poly 2026). The pattern is no longer framework-specific.

The **filesystem-as-state** model takes the opposite extreme: the working context is ephemeral and the *filesystem* is the sole authoritative record of task state (Zhou and others 2026). The event log demotes to a secondary log of intent; the filesystem holds the truth, usually through an opinionated convention (`progress.md` , `plan.md`) the agent maintains. This sidesteps the intent-versus-result gap for anything the filesystem can represent, the file either has the new content or it does not, but it relocates the gap rather than removing it: a `git push` , an external API call, an MCP server-side cursor advance are still outside the filesystem's reach.

From a list to a tree

The session log records steps, but its *shape* evolved separately from its durability. A linear append-only log treats a session as a list, and to back out you start over. Production harnesses moved past that. Rewind and truncate jump back to a checkpoint and discard the tail, rolling the workspace back to match, as in Claude Code's `/rewind` (Anthropic 2025) and Cursor's per-edit checkpoints (Cursor 2025). A forkable tree goes further: multiple live branches from

any checkpoint, all preserved. LangGraph's time-travel model treats checkpoints as a DAG with explicit branch IDs (LangChain 2025), Claude Code's `/fork` spawns a child session from a shared history point (Anthropic 2025), and Replit Agent (Replit 2025) and OpenAI's Codex CLI (OpenAI 2026) ship their own variants. The session stopped being a list and became a tree, or at least a log with a movable head. That tree is cheap to build only because conversation state is a pointer structure, an assumption the workspace will break in the next section.

The workspace: partial durability, expensive branching

The workspace is the sandbox's mutable filesystem. It outlives individual tool calls but is not on the durable log, and that distinction is the source of two failures the session log does not have.

Workspace persistence is partial durability, not full recovery. Persisting the workspace across pod lifecycle events has its own failure modes. The common Kubernetes pattern mounts a PersistentVolumeClaim at `/workspace`; the dominant failure is zone scoping. A ReadWriteOnce PVC is backed by a zonal block device that is physically attachable only within its own availability zone, so when a pod reschedules across zones the volume cannot follow and recovery falls back to a full re-bootstrap (Rack2Cloud 2026; Kubernetes 2023). When a pod is deliberately destroyed (teardown, timeout, scale-down) the PVC is often deleted with it, and uncommitted work is gone: Gitpod issue #9544 is the archetype, a workspace timeout fired before the final sync completed and a user lost roughly two hours of work (Gitpod 2022). The alternative, checkpointing the complete state on a cadence, trades snapshot frequency directly against storage overhead, and the frequency knob is load-bearing because it sets the maximum work a crash can erase. Replit's snapshot engine captures filesystem and conversation context together and reports recovering from hourly OOM crashes without data loss (Replit 2025; Replit 2025). Any design that claims "sessions survive indefinitely" owes an explicit account of which failure modes it covers and which it does not.

Change the snapshot interval below and watch the two curves move in opposite directions: tighter snapshots erase less work on a crash but write more often.

```

import numpy as np
import matplotlib.pyplot as plt
rng = np.random.default_rng(0)
intervals = np.array([1, 2, 5, 10, 15, 30, 60]) # minutes between s
trials = 20000
lost = []
for iv in intervals:
    crash = rng.uniform(0, iv, trials) # crash time within the inte
    lost.append(crash.mean()) # work since last snapshot,
lost = np.array(lost)
writes = 60 / intervals # snapshot writes per hour
for iv, l, w in zip(intervals, lost, writes):
    print(f"every {iv:2d} min: ~{l:4.1f} min lost, {w:5.1f} writes/h
fig, ax1 = plt.subplots()
ax1.plot(intervals, lost, "o-", color="#c0392b", label="expected wor
ax1.set_xlabel("snapshot interval (min)"); ax1.set_ylabel("work lost
ax2 = ax1.twinx()
ax2.plot(intervals, writes, "s--", color="#2980b9", label="writes/hc
ax2.set_ylabel("snapshot writes/hour")
fig.legend(loc="upper center"); plt.show()

```

Branching splits cheaply on conversation and expensively on the workspace. Branching conversation state is a pointer operation: the log is a tree of event IDs and forking a tree is cheap. Branching the *workspace* is not, because the workspace is a real filesystem and two branches cannot both own it.

store is memory, and it is a different system from the session log: the log is append-only intent, memory is curated and retrievable.

The shape of memory

Memory has a structure worth naming. Short-term memory is the session context within one conversation. Long-term memory splits into episodic (past interactions), semantic (facts about the user or domain), and procedural (learned workflows). External knowledge, the corpora behind retrieval-augmented generation ([Chapter 25](#)) and knowledge graphs, sits alongside but is a different thing: memory is per-user, curated, and writable, where RAG retrieves from a shared corpus the agent does not own.

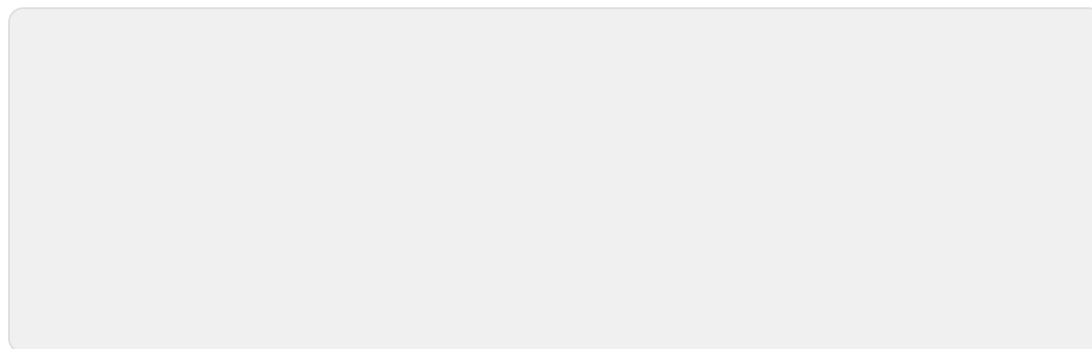


Figure 22.4. Memory splits into short-term and long-term (episodic, semantic, procedural), distinct from the shared external knowledge that RAG retrieves.

The base layer is a vector store plus embedding retrieval: text is embedded, stored, and recalled by similarity. Pinecone ([Pinecone 2025](#)), pgvector ([pgvector 2025](#)), and Chroma ([Chroma 2025](#)) are representative substrates. Above them sit opinionated memory frameworks, Letta ([Letta 2025](#)) (the successor to MemGPT ([Packer et al. 2023](#))), Zep ([Zep 2025](#)), and Mem0 ([Chhikara et al. 2025](#)), that decide what to write, when, and how to compact it. Knowledge graphs model entities and relationships explicitly, which suits compositional queries but is harder to populate automatically, and a temporal knowledge graph adds the time dimension so a fact can be valid for a period and then superseded.

The category arrived the way the other two stores did, by a research artifact hardening into a product. MemGPT ([Packer et al. 2023](#)) framed the LLM as an operating system that pages memory in and out of a bounded context, and its successor Letta ([Letta 2025](#)) turned that into a hosted primitive. Zep ([Zep 2025](#)) built memory on a temporal knowledge graph, and Mem0 ([Chhikara et al. 2025](#)) optimized for low latency and token cost. The category is young enough that its benchmarks are still contested.

WHAT'S CONTESTED

Cross-vendor memory benchmarks are not yet trustworthy. Mem0 reports a 26 percent LLM-as-Judge gain over OpenAI memory on LOCOMO ([Research 2024](#)) with large latency and token-cost reductions ([Chhikara et al. 2025](#)). Letta's counter-benchmark reports a higher LoCoMo score with disputed methodology ([Letta 2025](#)). Zep reports its temporal knowledge graph leading on a different benchmark ([Zep 2025](#)). The vendors agree on neither the metric, the dataset split, nor the baseline, so treat any cross-vendor memory benchmark as contested until it is independently reproduced. The useful signal is the shape of the trade, recall quality against latency and token cost, not the headline number.

What memory refuses to guarantee

Memory trades cost against simplicity, and write-time safety against read-time convenience. Memory must be either loaded ambiently into every prompt (simple, expensive) or exposed as a tool the agent queries on demand (cheaper, scales better, but the agent has to know when to ask). Changing the embedding model invalidates the existing index and forces an expensive reindex, so the embedding choice is a long-term commitment.

The sharpest trade is tenancy, and it is what makes a shared vector store a data-breach risk rather than a relevance bug. The same property that makes memory useful, that retrieved content lands directly in the model's context, makes it dangerous, because that content is indistinguishable from instructions the user or harness put there. A top-k similarity query against a shared vector store without an enforced tenant filter can return another tenant's data, and the defense has to live in the index, not in application code. Partition at write time, per-tenant namespaces, pgvector row-level security or separate schemas, so a query physically cannot reach another tenant's vectors. Attributing every chunk and filtering after the query is weaker, because filter-at-read-time is one bug away from a leak, whereas partition-at-write-time fails closed. This is where memory consumes the identity model directly (Chapter 34): tenancy is sourced from identity, and partitioning enforces it.

A second tenancy risk is specific to memory and has no analogue in the session log: because the agent writes memory based on user messages, a malicious user can craft messages that plant persistent misinformation. This is the prompt-injection problem moved into the memory store, where it survives across sessions. A misretrieved or poisoned record lands in the prompt and shifts the agent's behavior in ways that are hard to debug, because the trace shows retrieval *succeeding*: the injected text is not an error, it is a correct retrieval of wrong content.

Two properties to nail down before the first crash

Both of the following are invisible until exactly the wrong moment, which is why they belong in the design and not in the postmortem.

The first is the **reconciliation gap**, the operational form of the opening crash. If the sandbox holds mutable state outside the durable log, the harness must reconcile the two on recovery. For idempotent operations it does not matter; the result is the same whether the operation ran once or twice. For non-idempotent ones, append to a file, increment a counter, create a commit, call a side-effectful API, the harness needs either a step-log boundary or an idempotency key threaded through the tool. The journal records the step's intent and its key before the effect fires; on replay the harness performs check-then-act against that key, or against the external system's own dedup, git's ref state, a payment processor's idempotency header, rather than blindly re-issuing. The step log does not make external effects idempotent on its own. It gives you the hook to make them so.

```
# Idempotency-keyed step, conceptual sketch.
def run_step(journal, key, effect):
    if journal.has_outcome(key):      # replay: skip a completed step
        return journal.outcome(key)
    journal.record_intent(key)        # durable boundary BEFORE the effect
    result = effect()                 # the non-idempotent side effect
    journal.record_outcome(key, result)
    return result
```

The second is **erasure reach**. A right-to-erasure request has to cascade across every store that holds a user's data, and memory is the painful case: framework-managed memory may not expose a delete API at all, so a store chosen for recall quality can quietly become a store you

cannot erase. Partition-at- write-time helps here too, because a per-tenant namespace is a unit you can drop wholesale. The broader audit and compliance machinery, tamper-evident logs, data residency, the regulatory regimes, belongs to [Chapter 34](#); the memory-specific obligation is to keep every record attributable to a tenant and erasable by tenant.

For the memory store itself, the base implementation is a vector index ([pgvector](#) ([pgvector 2025](#)) inside an existing Postgres is the low-operational-overhead default; [Pinecone](#) ([Pinecone 2025](#)) or [Chroma](#) ([Chroma 2025](#)) when a dedicated store is warranted), wrapped either by a framework ([Letta](#) ([Letta 2025](#)), [Zep](#) ([Zep 2025](#)), [Mem0](#) ([Chhikara et al. 2025](#))) that manages write and compaction policy, or by a thin custom layer when the policy is simple enough to own. The harness ([Chapter 23](#)) is what loads the result into context, ambiently or through a retrieval tool, closing the loop back to the session that the memory outlived.

Further reading

- "AI Agent Reliability Report 2025." LLM agent retry rate 15–30%. [fast.io](#)
- S. Ewen, G. van Dongen, I. Shilman. "Durable AI Loops: Fault Tolerance across Frameworks." *Restate*, 2025. [Restate](#)
- C. Davis. "Durable Execution meets AI." *Temporal*, 2025. [Temporal](#)
- C. Poly. "Durable Execution: The Key to Harnessing AI Agents in Production." *Inngest*, 2026. [Inngest](#)
- WorkOS. "Maxim Fateev on Durable Execution for AI Agents." 2025. [workos.com](#)
- Y. Zhou et al. "Externalization in LLM Agents: A Unified Review of Memory, Skills, Protocols and Harness Engineering." 2026. [arXiv:2604.08224](#)
- Rack2Cloud. "Kubernetes Day 2 Failures: 5 Incidents & the Metrics That Predict Them." 2026. [Rack2Cloud](#)
- Kubernetes Issue #121436. "PVC Binding Prevents Pod Rescheduling." [GitHub](#)
- Gitpod Issue #9544. "Data loss when workspace timeout fires before sync completes." 2022. [GitHub](#)
- Replit. "Inside Replit's Snapshot Engine." December 2025. [blog.replit.com](#)
- Replit. "Finding and Solving Memory Leaks." [Replit Blog](#)
- Anthropic. "Claude Code: Checkpointing and Session Forks." [Claude Code Docs](#). [code.claude.com](#)
- Cursor. "Checkpoints in the Agent Workflow." 2025. [stevekinney.com notes](#)
- Replit. "Checkpoints and Rollbacks." [Replit Docs](#)
- LangChain. "LangGraph Time Travel and Branching." 2025. [LangChain Docs](#)
- OpenAI Codex CLI. Conversation rewind (Esc) ships today; a code-reverting rewind is proposed in Issue #11626, 2026. [GitHub](#)
- AWS. "Firecracker Snapshotting." [Firecracker Docs](#)
- D. Ustiugov et al. "Benchmarking, Analysis, and Optimization of Serverless Function Snapshots (REAP)." *ASPLOS* 2021. [arXiv:2101.09355](#)
- Pinecone. "Managed Vector Database for AI." [pinecone.io](#)
- pgvector. "Open-source Vector Similarity Search for Postgres." [github.com/pgvector/pgvector](#)
- Chroma. "Open-source Embedding Database." [trychroma.com](#)
- Letta. "Stateful Agents with Memory as a First-Class Primitive." [letta.com](#)
- C. Packer et al. "MemGPT: Towards LLMs as Operating Systems." 2023. [arXiv:2310.08560](#)
- Zep. "Temporal Knowledge Graph for Agent Memory." [getzep.com](#)
- "Mem0: Memory for the AI Era." April 2025. [arXiv:2504.19413](#)

- Snap Research. "LOCOMO: Long-Term Conversational Memory Benchmark." snap-research.github.io
- Letta. "Benchmarking AI Agent Memory." 2025. letta.com

23. The Harness

The harness is the scaffold around the model: the runtime that turns model output into tool calls and tool results back into model input, manages the context window, routes tools, sandboxes execution, and exposes the verbs that let a person pause, redirect, fork, or kill a running loop. By the end of this chapter a reader can explain why a loop that looks trivial as a flowchart needs this much machinery, and why the shape of that machinery moves an evaluation score as much as a model swap does.

A loop that refuses to stay simple

The contract is small. Create a session bound to an agent definition. Drive its loop: send context to the model, dispatch the tool calls it returns, append the results, repeat until done or interrupted. Resume after a crash or a wait. Interrupt mid-flight. Fork at a checkpoint. Drawn as a flowchart the loop is a box and an arrow back to itself.

It stops being trivial the moment it meets production. A tool holds a connection across turns, so the loop is no longer stateless between steps. The context window fills, so the loop has to decide what to forget. A user hits cancel while a sandbox command is mid-execution, so the loop has to stop something it does not directly control. Two replicas both think they own the same scheduled tick, so the loop has to coordinate with a copy of itself it cannot see. The harness is where these frictions are absorbed or leaked. It owns mechanism, not policy: it does not decide who may call (identity, Chapter 34), where the model comes from (model access), or which actions require a human (oversight, Chapter 33). It decides how those decisions are carried out, and the most consequential mechanism it owns is the ability to stop the loop.

The loop drawn bare is one box and a back-edge. What Figure 23.1 adds is where each of those production frictions bites the loop, which is the whole argument of this chapter in one picture: four unrelated-sounding problems land at four distinct points on a structure that looked like it had none.

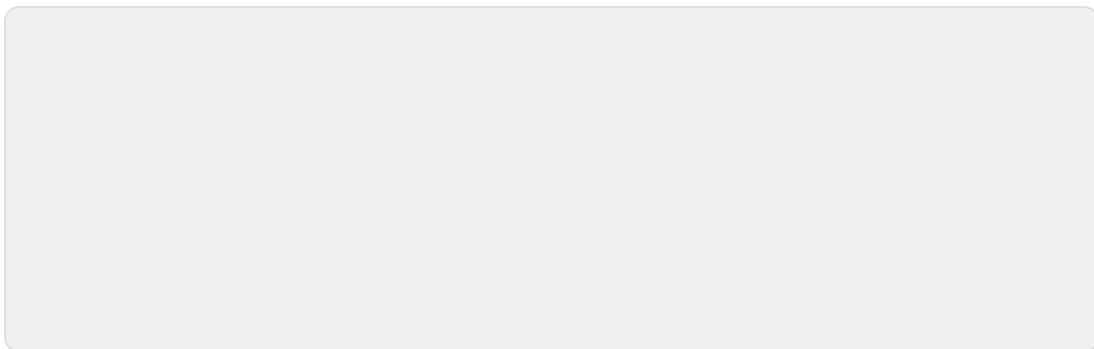


Figure 23.1. The agent loop with the four production frictions mapped to where each one bites. The bare loop is the solid cycle; the dashed callouts are the points where reality breaks the abstraction.

The rest of the chapter is a teardown of the six mechanisms that absorb these frictions. Each one began as a naive version, broke at a specific point under load, and acquired exactly the machinery that point demanded. Read in order, the six are the harness: where an instance comes from, how a cancel reaches the work, who is allowed to run, what the lock actually protects, what survives a full context window, and how the model learns a tool exists. We take them one at a time, each with the lineage that produced it and the knee where its trade-off bites, and close on the operational hardening that the loop needs to survive a long night unattended.

Where an instance comes from

The first decision is when an agent definition, a prompt plus a tool set plus a model binding, becomes a live instance. The obvious starting point is a singleton: one runner shared across every session, isolated only by a `session_id`, zero allocation per request. The hidden cost is not performance, it is rollout granularity. With a singleton, the prompt is a deploy-time constant, so a bad prompt regresses 100% of sessions until rollback completes. OpenAI's GPT-4o sycophancy episode is the shape of this: the update rolled out April 25, 2025, rollback began April 28, and the full revert took roughly a day (OpenAI 2025). A runtime registry resolves a prompt version per request instead, which lets a regression touch a canary slice while the stable version keeps serving everyone else. The blast radius now equals the canary, not the fleet. The cost is a resolution hop, a cross-replica consistency story, and a new variable in every incident.

The knee is when this stops being optional. A singleton is fine at one or two agent types and starts hurting past three or four, when a prompt regression in one agent forces a rollback that reverts an unrelated fix in another because they shared a deploy. That shared-deploy incident is the moment versioning stops being a nicety, because the cost of not having it has become a regression in code nobody touched.

How a cancel reaches the work

Cancellation is exactly as effective as its most-blind hop. The harness can see an interrupt flag instantly, but the flag has to travel down every layer that might be mid-execution: the model call has to honor its context deadline, the tool dispatch layer has to propagate cancellation into the tool, the sandbox command has to respect SIGTERM, and a stateful protocol call has to respect RPC-level cancellation. A model call that ignores its deadline holds the session for tens of seconds past cancel. A sandbox command that ignores SIGTERM holds it for the full grace period. So "the runtime supports cancellation" is a claim about every hop in the cascade, and a single uncooperative layer makes the cancel cosmetic. This is why interruption is a primitive and not a feature: the pause a human-in-the-loop approval depends on is threaded through every hop or it is theater. The decision about what requires approval belongs to oversight; the pause that lets the approval happen is the runtime's verb, and frameworks that expose it as a first-class interrupt make that pause inspectable rather than implicit (LangChain 2025). Figure 23.2 traces the cascade and the latency each uncooperative hop adds, so the claim that cancel is only as real as its most-blind hop becomes a property of the chain rather than an assertion.

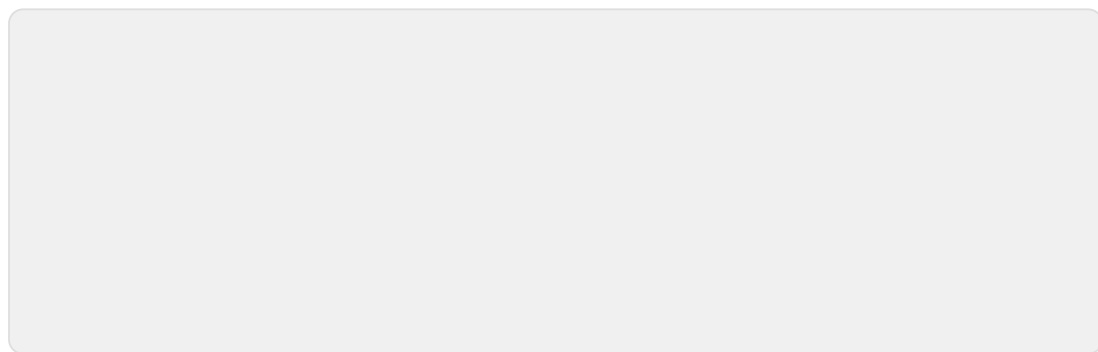


Figure 23.2. The cancellation cascade. The interrupt flag is instant, but the cancel is only effective once every downstream hop honors it. One uncooperative hop bounds the whole result, so the dangling delays mark where a cancel goes cosmetic.

Try setting `obeys` to `False` for one hop and watch the time-to-stop jump to that hop's grace period: the cancel is only as fast as its most-blind layer.

```

# Each hop either obeys the interrupt fast, or ignores it and only
# stops when its own grace period expires. Time-to-stop is the slowe
hops = [
    {"name": "model call",    "obeys": True,  "grace_s": 30.0},
    {"name": "tool dispatch", "obeys": True,  "grace_s": 5.0},
    {"name": "sandbox cmd",   "obeys": False, "grace_s": 10.0}, # i
    {"name": "RPC call",      "obeys": True,  "grace_s": 8.0},
]
flag_latency = 0.01 # harness sets the interrupt flag almost instan
stop_times = [flag_latency if h["obeys"] else h["grace_s"] for h in
for h, t in zip(hops, stop_times):
    print(f"{h['name']:14s} stops after {t:6.2f}s")
print(f"\ntime-to-stop = max = {max(stop_times):.2f}s "
      f"(bounded by the most-blind hop)")

```

The verb has four shapes, and the quality of human control is bounded by which one the harness offers. Hard cancel is `kill(session)` with no state preservation, simple and brutal, making every correction a context-losing event. Cooperative pause checks an interrupt flag at step boundaries and serializes state. Queued steering appends user input to the next turn without pausing, fast but able to commit a stale decision before it sees the message. Resume-at-point combines steering with a rewind. The pause that an approval depends on is exactly as good as the strongest of these the runtime implements, and no better.

Who is allowed to run

When the loop runs on a schedule, every replica runs its own scheduler, so without coordination every replica fires every tick. The usual fix is an agent-level distributed lock acquired before the run. It prevents two replicas from firing the same tick concurrently. It does not make the tick's side effects exactly-once. If a tick posts to Slack, opens a ticket, then crashes before recording "done," recovery replays it and the Slack message posts twice. The lock coordinates who runs; exactly-once requires coordinating what already happened to the outside world. Idempotency keys at the side-effect call sites close that gap, and they are a different guarantee that a scheduler with only a lock does not have.

A lock prevents overlap, not duplication, and there is a second, sharper way the altitude is wrong. A harness almost always has session-level locking, which stops two `Run()` calls from mutating one session at once. It says nothing about two different sessions of the same agent touching the same external resource. Two coding sessions push conflicting commits to one branch. A user session and a scheduled tick target the same external state, neither aware of the other. The lock is in the harness; the conflict is at the resource, and the harness lock does not reach it. [Figure 23.3](#) shows the mismatch directly.

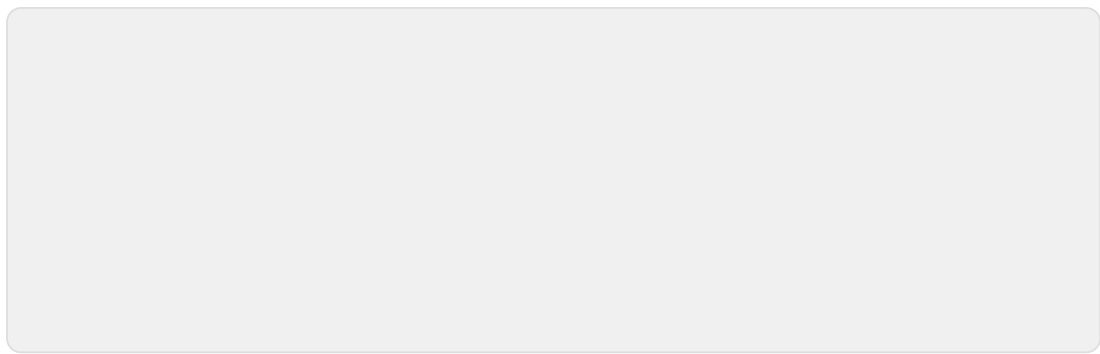


Figure 23.3. Session-level locking is the wrong altitude for a resource conflict. Each session is individually locked, yet two sessions of the same agent collide at a shared external resource the harness lock never reaches.

Every session in green is individually locked; every shared resource in red is unlocked. The fix lives at the resource: a worktree per session that moves the conflict to a merge, or a resource-level lock keyed on the named resource, or optimistic reconciliation on the external system. The collision is not hypothetical; merge conflicts in coding-agent pull requests are now frequent enough to form a labeled dataset (Ogenrwot and Businge 2026). The right choice depends on whether the downstream system already has its own concurrency control to lean on. Git ref updates and database row locks are real coordination primitives; where they exist the harness can defer to them.

The scheduler itself climbs the same pressure curve. In-process and external-cron schedulers give way to durable execution frameworks (Temporal, Inngest, Restate) once a tick needs multi-step choreography with external side effects, because probabilistic model behavior makes naive retry insufficient, which is exactly the case durable journaling is built for. Whether to keep the session persistent or ephemeral across ticks is a parallel choice with its own knee. A persistent session accumulates context, so a monitoring agent remembers last week's baseline, but events grow without bound, which makes context management mandatory. An ephemeral session is fresh per tick and correct when each tick is independent. Persistent for a stateless job buys unbounded storage for nothing; ephemeral for a job that needs memory throws the memory away every tick.

What survives a full window

A long-running loop hits the context window, and the harness has to answer two questions that do not answer themselves: what gets preserved, and who decides. Automatic compaction summarizes older turns so the session runs past the hard limit, and its failure mode is structural: summarization discards detail and the agent has no signal about what it lost. Anthropic's cookbook measured compaction preserving three of three high-level facts and zero of three obscure specifics (Anthropic 2026). The alternative is externalization, where the agent writes what it wants to keep to a file (`progress.md` , `plan.md`) and reloads after compaction. That trades the summary's blindness for the agent's blindness about its own future needs. Neither is mechanical, which is the subject of the contested box below.

"Just use a bigger window" is the tempting third option, and it relocates the failure rather than removing it. A larger context window reduces compaction frequency but does not remove the degradation: accuracy drops as input length grows even with perfect retrieval (Du et al. 2025), and the same rot shows up across realistic retrieval settings (Hong et al. 2025). "Fit everything in" moves the failure from "context overflowed" to "context is full and the model is quietly worse."

Whether long-session context management can be made mechanical is unsettled, and the evidence says it cannot yet. Lindenbauer et al. showed that simply masking old tool outputs with placeholders matches LLM summarization on solve rate at the lowest cost in four of five settings, and that summaries cause trajectory elongation: agents persist 13 to 15% longer than optimal because the summary masks the failure signals that would have told them to stop (Lindenbauer et al. 2025). Compaction work that does help, such as learned context compaction for long-horizon tasks (Kang et al. 2025), still leaves the core question open. Bigger windows do not settle it either, since accuracy degrades with length even under minimal conditions and perfect retrieval (Du et al. 2025). Long-session quality rests on three things the runtime cannot guarantee mechanically: the model writing a good summary when asked, the model interpreting that summary on reload, and the runtime enforcing an externalization pattern rather than hoping the agent maintains one. The runtime can fire compaction at the right moment and reload the right file. It cannot make the summary true.

How the model learns a tool exists

Dispatch is how a tool runs; the registry is how the model learns the tool exists. A static flat list sends every tool in every system prompt and runs out at roughly 30 to 50 tools, because function-calling accuracy falls as the catalog grows and robustness suffers specifically when irrelevant tools are present (Yan et al. 2025). Beyond that the runtime needs per-turn selection. The security-flavored failure is the sharpest: a malicious tool description at the catalog layer can inject instructions into the model through the system prompt, bypassing the user prompt entirely (Invariant Labs' mcp-injection-experiments). The registry is not a neutral lookup table, which is why catalog admission is a trust decision handed to [Chapter 34](#). What the registry admits to the catalog, it admits to the model's instructions.

Selection evolved as a ladder, each rung raising the catalog ceiling and adding a cost the runtime must then own. The static flat list gave way to dynamic tool-RAG, which embeds the schemas and retrieves the top-k relevant per turn, scaling to thousands of tools at the cost of a retrieval layer with its own error modes. Hierarchical namespacing with mount and unmount shows a pruned subset per phase, read-only tools while understanding a problem and write tools while fixing it, which requires the runtime to track phase. Model-driven discovery gives the model a `list_available_tools` tool and lets it ask, which raises the ceiling but makes discovery quality a function of model capability the runtime does not own. [Figure 23.4](#) lays the four rungs against the scaling ceiling each one buys and the cost it adds.

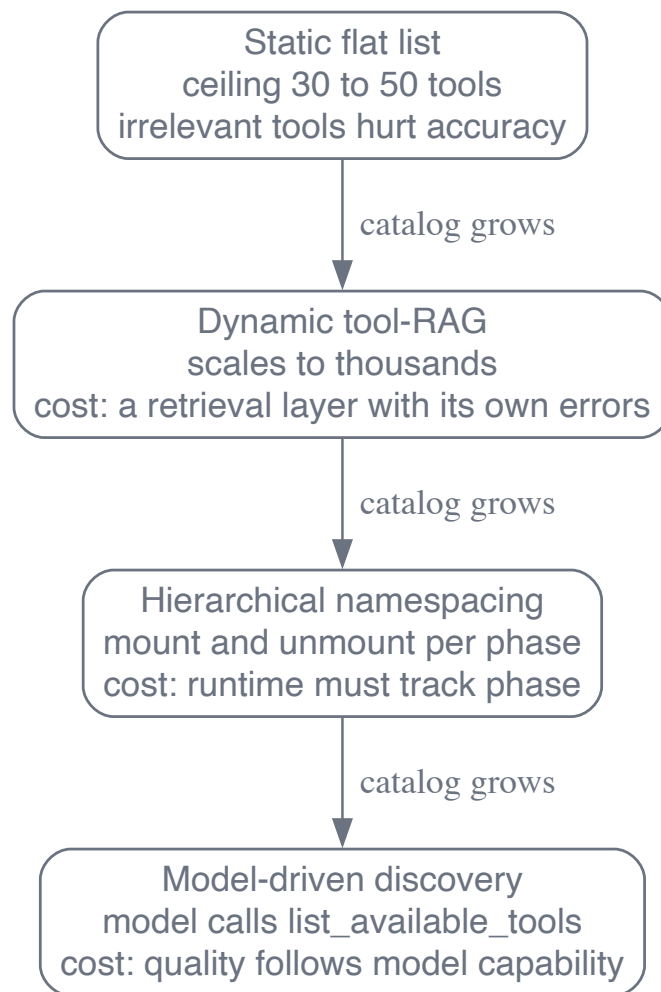


Figure 23.4. The tool-selection ladder. Each rung raises the catalog ceiling and adds a cost the runtime must then own, from a static prompt list to model-driven discovery.

What a stateful tool drags behind it

Tools fold into the design along a statefulness axis. Stateless tools (a web fetch, a sandbox exec given a sandbox ID) hold no harness-side state and scale trivially. Stateful-connection tools (MCP, gRPC streams) carry negotiated capabilities and cursors in a connection the harness holds for the session, and dispatching them through one mechanism shared with stateless calls mishandles one class or the other. The hardest variant is a single logical operation that mixes both, because its partial-failure space is the product of the two.

Stateful tool transport is the clearest lineage in the chapter. The simplest design holds connections in memory per replica, keyed by `(session_id, server_url)`, opened lazily. It works until the platform scales horizontally, and then statefulness fights the load balancer: the MCP 2026 roadmap names stateful sessions as the primary scaling bottleneck (Protocol 2026). The concrete failure is the reconnect storm. When a replica holding N sessions, each connected to M servers, dies, recovery is N times M simultaneous reconnections against the surviving infrastructure, each renegotiating capabilities and losing cursor state. The mechanism dictated the mitigation: session affinity to keep a connection on one replica, reconnect backoff with jitter to desynchronize the herd, and resumable streams to make reconnection cheap. The MCP spec followed this curve directly, deprecating HTTP+SSE in favor of Streamable HTTP, where a reconnect replays from a cursor using `Mcp-Session-Id` plus `Last-Event-ID` instead of restarting (Protocol 2025; Protocol 2025). The same spec hardened a related hazard pooling alone does not touch: when several concurrent sessions for one user race to refresh a single-use OAuth token, the losers present an already-consumed

token and fail, so the 2026 spec mandates RFC 8707 resource indicators to keep a token from being redeemed against the wrong server (Campbell et al. 2020).

Multi-agent composition has a parallel arc, with OpenAI's experimental Swarm superseded by the Agents SDK, but composition itself belongs to Chapter 24 and the harness only owns whether the composed agents share or isolate session and sandbox state. That sharing choice is its own trade-off with a measured cost. Shared session and sandbox is cheapest and pays with races and prompt contamination. Full sandbox isolation is the most plumbing and the strongest containment: Geng and Neubig measured worktree isolation at 59.1% against soft instruction-level isolation at 56.1% on Commit0-Lite, and showed soft isolation falling below a single-agent baseline on PaperBench (Geng and Neubig 2026). Instruction-level constraints are advisory; real isolation does not depend on the model's compliance.

Hardening for the long night

The loop itself is small. The machinery around it is what the rest of this chapter has been about, and the last of it concentrates in a handful of operational choices that decide whether the loop survives running unattended.

The lock has two gotchas that come straight from the locking literature. Redis-based locks need fencing tokens to be safe under network partitions (Kleppmann's Redlock critique) (Kleppmann 2016). And with PgBouncer transaction pooling, `pg_advisory_lock` releases when the connection returns to the pool, so a caller that expects a session-scoped lock must use `pg_advisory_xact_lock` or a dedicated session pool, or the lock evaporates mid-tick.

A wall-clock timeout on every session, regardless of trigger, is the one limit that holds even when token accounting is wrong or lagging, because wall-clock time is the quantity the harness can measure locally without trusting a downstream meter. The evidence for why it is load-bearing is stark: a LangChain A2A pipeline looped between two agents for eleven days and produced roughly a \$47,000 bill with neither agent carrying a budget ceiling (Waxell 2025), and a Claude Code recursion consumed about 1.67 billion tokens in five hours (anthropics/claude-code 2025).

Admission control belongs at the harness layer. Session creation that calls the Kubernetes API requests a sandbox rather than creating one, and the dangerous failure is not refusal but a hang: the pod sits Pending and the stream to the client never starts, with no error to surface because nothing errored. Checking capacity before promising the client a session turns a slow silent nothing into a fast legible "no" the client can retry against.

The remaining controls follow the same mechanical reasoning. Jitter prevents a thundering herd when many agents share a schedule. A circuit breaker disables an agent after N consecutive failures so it cannot burn tokens indefinitely: the AWS Cost Explorer outage of December 2025 was linked in incident reporting to an autonomous agent acting with broad permissions and no circuit breaker (Singh 2025). And self-healing at startup replays ticks whose end was never recorded, which is safe only because the idempotency keys above absorb the replay. Sandboxing itself is thin from the harness's seat: it owns worktree isolation for correctness and admission control for capacity, and the sandbox primitive owns the rest.

The closing lens is capability, efficiency, and trust. The harness buys capability by letting the loop run at full speed across stateful tools and composed agents. It pays for efficiency with context management and admission control that keep a runaway from burning the budget. And it earns trust in one place only: human authority over a running agent is exactly as real as the runtime's ability to pause it, which is a property designed in from the start or done without.

CONSTRAINT ARROW

The harness moves an evaluation score as much as the model does. A benchmark number is produced by a model running inside a harness, and Berkeley RDI showed that top agent benchmarks can be gamed by exploiting the harness itself, reading answers

from `git log` or reward-hacking the grader, with several benchmarks hitting near-100% without the agent solving the task at all (RDI 2025). The exploit lives in the harness, not the task. The consequence reaches up into [Chapter 27](#) and [Chapter 29](#): a score is a joint property of the model and the scaffold around it, so a harness change is a measurement change, and the eval suite has to harden the harness, not just hold out a private answer key. A held-out set protects against memorizing answers; it does nothing against an agent that reads the grader.

Further reading

Cited sources appear on the book's References page. One item is referenced inline only as an attribution and is listed here for the reader who wants the primary artifact.

- Invariant Labs, "mcp-injection-experiments." [GitHub](#)

24. Multi-Agent Systems

A single agent (Chapter 21) has one viewpoint and one set of blind spots. Putting several agents on the same task promises to cover those blind spots, and the default way to cash that promise is to vote. By the end of this chapter a reader can explain why voting among similar agents buys less than the math suggests, why structured disagreement buys more, and how to tell the two failure modes apart so a system escalates the recoverable ones and blocks the silent ones.

The default move: more agents, take the consensus

The dominant pattern for making a multi-agent system reliable is to run more agents and take the consensus: vote across N samples, ensemble across model families, add a judge. The implicit assumption is that agreement is evidence and disagreement is noise to be averaged away. That intuition came from places where it held, statistical estimation, classical fault-tolerant computing, and machine-learning ensembles trained on independently sampled data. None of those assumptions survive contact with stochastic agents driven by large language models. Treating agreement as a proxy for correctness has three load-bearing problems, and the rest of this section takes them one at a time.

Failures are correlated

Classical Byzantine fault tolerance does not require probabilistic independence in its theorem; it requires a credible bound on how many replicas can be faulty at once. The $f < N/3$ threshold is meaningful only when fewer than a third of replicas can share the same Byzantine failure mode. Two agents drawing from the same base model, the same training distribution, and often the same prompt template do not have independent failure modes. They co-hallucinate. They share the same blind spots, the same bias toward fluent-sounding wrong answers, the same weakness on the same inputs. Berdoz et al. measured this directly: valid scalar consensus among LLM agents drops from 46.6% at $N = 4$ to 33.3% at $N = 16$ even without any adversary (Berdoz et al. 2026). Adding agents made it worse, not better.

Agreement is cheap to manufacture

A small amount of conformity pressure, a leading prompt, or a shared in-context example can drive agents into convergent but incorrect answers. The Free-MAD work gives a modern version: in consensus-driven debate, agents that initially produce correct responses can be pulled toward incorrect ones, and final majority voting can degrade reasoning performance (Cui et al. 2025). When the system is graded on agreement, agents learn to agree, even when the right move is to flag that they cannot.

Self-evaluation is biased

Panickssery et al. showed LLM judges systematically favor their own generations (Panickssery et al. 2024), and position bias in pairwise judges is well documented (Shi et al. 2025) (Chapter 28). An ensemble of judges drawn from the same family inherits the same biases. A verification layer that is a cheaper echo of the layer it verifies cannot tell you anything you did not already know. The honest reading is that agreement among similar models is weak evidence of correctness. It is sometimes evidence of nothing more than that the input was unambiguous enough for any plausible decoder to land on the same answer.

Disagreement as signal, not noise

The deeper move is to stop treating disagreement as noise. In an idealized debate, a single competent honest critic is sufficient to expose a flaw, no matter how many other critics are lazy or wrong (Irving et al. 2018). This asymmetry is the core design idea, and it inverts the orchestration. Instead of symmetric voters whose verdicts are pooled, the roles are specialized: a proposer produces an artifact, independent critics each attack one distinct axis of it, and a cheap judge adjudicates only the disputed claim.

A small formal contrast makes the asymmetry precise. Let $p_i \in [0, 1]$ be the probability that critic i catches a flaw on the axis it evaluates, and let $\rho \in [0, 1]$ be the pairwise correlation between verdicts.

Aggregational verification with k voters under majority vote degrades as correlation rises. As $\rho \rightarrow 1$ the probability a flaw survives the ensemble approaches the probability that the single shared failure mode misses it, and adding voters does not help:

$$\Pr[\text{flaw survives} \mid \rho \rightarrow 1] \rightarrow \Pr[\text{shared mode misses flaw}]$$

Adversarial verification with k independent critics on distinct axes, where one caught flaw suffices, factors over critics. Under independence this is exact; any residual positive correlation only raises survival above the product, so the product is the best case you are buying:

$$\Pr[\text{flaw survives}] = \prod_{i=1}^k (1 - p_i)$$

The product only buys something if the p_i cover different ways the artifact can be wrong.

Write $p_i = c_i d_i$, where c_i is the probability critic i selects the relevant attack surface and d_i is the probability it turns that surface into a concrete objection. Duplicating critics on the same surface mostly raises d_i locally; forcing topic diversity raises coverage, roughly $1 -$

$\prod_i (1 - c_i)$. A single competent honest critic on the relevant axis, $p_i = 1$, collapses the product to zero. This is the asymmetry that lets adversarial verification dominate consensus when failures are correlated and at least one independent honest critic exists.

Try sliding the correlation up: with a shared-latent mixture $\Pr[\text{survives}] = \rho(1 - p) + (1 - \rho)(1 - p)^k$, watch how voting flattens as ρ rises while the independent product keeps driving survival down.

```

import numpy as np
import matplotlib.pyplot as plt

p = 0.5 # per-agent chance of catching the flaw
k = np.arange(1, 11) # number of agents / critics
for rho in [0.0, 0.3, 0.6, 0.9]:
    survives = rho * (1 - p) + (1 - rho) * (1 - p) ** k
    plt.plot(k, survives, marker="o", label=f"voting, rho={rho}")
plt.plot(k, (1 - p) ** k, "k--", lw=2, label="adversarial (rho=0 prob)")
print("rho=0.9 survival at k=10:", round(rho := 0.9, 2),
    "->", round(0.9 * (1 - p) + 0.1 * (1 - p) ** 10, 4))
print("adversarial survival at k=10:", round((1 - p) ** 10, 4))
plt.xlabel("agents (k)"); plt.ylabel("Pr[flaw survives]")
plt.legend(); plt.title("Adding voters buys little when failures correlated")
plt.show()

```

Disagreement also obeys a monotonicity invariant that voting violates. Let U_t be the set of unresolved attacks after round t , $R_t \subseteq U_t$ the attacks resolved by a fix, concession, or convincing defense, and A_{t+1} the new attacks introduced next round:

$$U_{t+1} = (U_t \setminus R_t) \cup A_{t+1}$$

No unresolved objection disappears because more agents voted against it; it disappears only by being resolved. Majority aggregation violates this by design, since a real minority objection can be buried under a larger pile of shallow agreement. The implication for design is precise: you do not need every critic to be competent, you need a process that surfaces the competent objection when it exists and terminates on the contested points rather than averaging them away. This is closer to proof checking, property testing, and red-team review than to ensemble prediction. The verifier does not reproduce the whole artifact; it forces a disputed claim into a witness small enough for a cheap judge to inspect: a failing test, a vulnerable input, a violated invariant, a missing citation. [Figure 24.1](#) contrasts the two orchestrations.

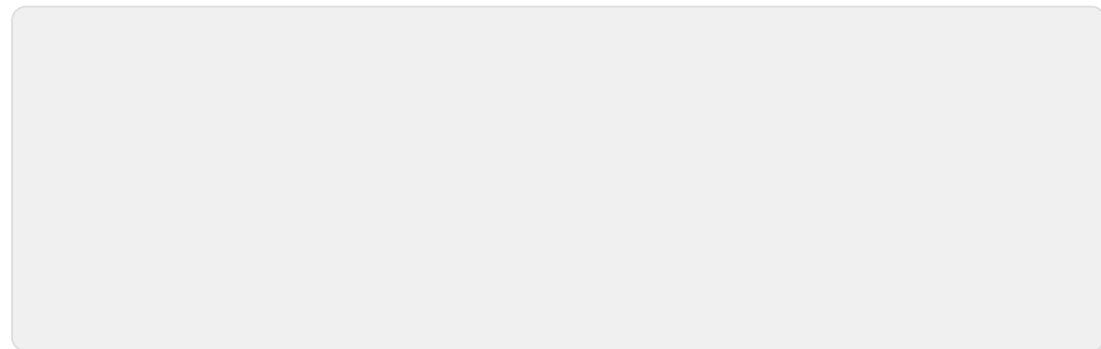


Figure 24.1. Two orchestrations of multiple agents. Aggregational pooling runs symmetric voters and takes the majority, so a single shared failure mode survives the vote. Adversarial verification assigns each critic a distinct attack axis, where one caught flaw suffices and a cheap judge adjudicates only the disputed claim.

Two failures that look alike

The lineage runs from pooling to structure. The starting point is the ensemble: independent samples, averaged or voted, which works when the samples are genuinely independent. Byzantine fault tolerance is the distributed-systems refinement of that idea, and reading it carefully is what reveals why the naive translation to agents fails. Two of its contributions survive translation, and they are the two practitioners tend to skip.

The first is the separation of liveness from safety. A system can fail by producing the wrong answer, a safety violation, or by failing to produce any answer at all, a liveness violation. The empirical results above are striking precisely because LLM consensus failures are mostly liveness failures: the agents do not converge on a wrong answer, they fail to converge at all (Berdoz et al. 2026). A liveness failure is recoverable by escalation to a human or a different model. A safety failure is silent and ships. A system that collapses both into one accuracy number optimizes the wrong axis. The second is the recognition that correlated failure is how a fault bound becomes false in practice. The fix is not more replicas but breaking the correlation: heterogeneous models, prompts, tooling, and where possible heterogeneous formulation of the question itself.

This split is the chapter's central routing decision, and [Figure 24.2](#) draws it: a single accuracy number collapses two failures that demand opposite responses.

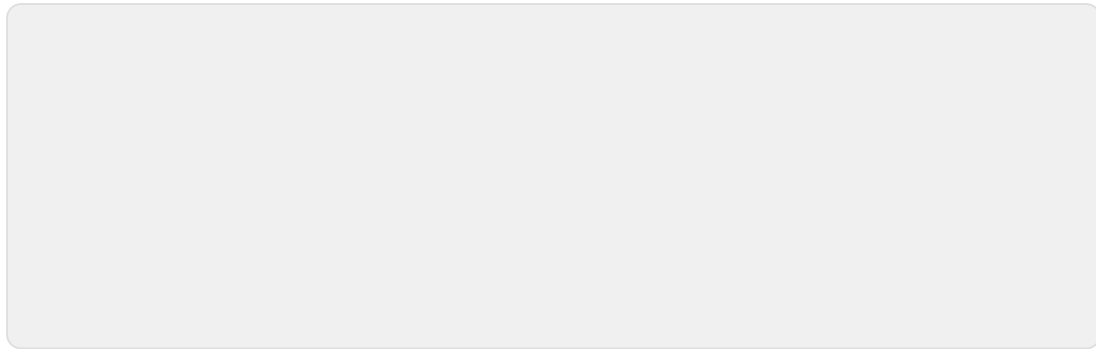


Figure 24.2. Separating liveness from safety. A liveness failure, where the agents cannot converge, is loud and recoverable, so the system escalates it. A safety failure, where the agents converge on a wrong answer, is silent and ships unless it is blocked. Collapsing both into one accuracy number optimizes the wrong axis.

From debate to a bounded judge

From the liveness-safety split the line continues into debate. Irving, Christiano and Amodio framed AI safety as a debate judged by a weaker verifier (Irving et al. 2018), and Brown-Cohen, Irving and Piliouras gave the formal extension: soundness holds under a large compute asymmetry, where the honest strategy uses polynomial simulation steps while the dishonest strategy may use exponentially many (Brown-Cohen et al. 2023). The complexity-theoretic intuition is that under optimal play debate can answer PSPACE questions, because a judge inspecting only the single disputed leaf where the players' disagreement bottoms out can verify outcomes it could never have produced unaided (Brown-Cohen et al. 2023). That is the modern destination of the arc: specialized adversarial roles with a bounded judge, not a wider pool of symmetric voters.

WHAT'S CONTESTED

Whether adding agents helps is the live disagreement. The aggregational camp treats more samples and a majority vote as straightforwardly reliable, and for genuinely independent agents it is. The measured reality for similar LLM agents is the opposite:

naive consensus degrades with agent count, from 46.6% valid consensus at $N = 4$ to 33.3% at $N = 16$ in Berdoz et al. (Berdoz et al. 2026), and consensus-driven debate can pull initially correct agents toward wrong answers (Cui et al. 2025). Structured disagreement is the contested fix, and it is contested on two fronts. Its soundness guarantee assumes at least one independent honest critic exists and is surfaced, which heterogeneity is supposed to provide but does not prove. And its own evaluation is thin: that adversarial verification beats aggregation for stochastic agents is argued from debate theory and small probes, not yet from a broad benchmark. Treat "run more agents and vote" as safe only when the agents' failures are genuinely independent, which for same-family models they are not.

CONSTRAINT ARROW

The orchestration choice is dictated from below. Because critics drawn from one base model and training distribution (Chapter 35) share a failure mode, the $f < N/3$ fault-diversity assumption that voting relies on is already false before the first round. No amount of same-family replication restores it. The model layer's homogeneity is what forces heterogeneous orchestration: different model families, different prompts, different tooling. A decision that looks like a top-level orchestration preference is set by a property of the layer underneath it.

What it costs

Structured disagreement is not free, and the boundaries are where the source is honest about what is unsettled.

- Heterogeneity has a cost worth paying. Mixing critic models, for example Codex critics against a Claude proposer, introduces integration complexity, provider drift, and a coordination tax. It is also the only thing that restores any useful version of the fault diversity the verification math requires. There is no known shortcut.
- Cardinality is unknown. One critic per topic is the floor; the ceiling is unclear. More critics on the same topic should give diminishing returns under correlated failure, but the shape of that curve across model families has no clean empirical story yet.
- Protocols drift. A debate that runs too long drifts off the original claim, and contention scores can be inflated by stylistic friction rather than substantive disagreement. The right termination criterion is unsettled; steady-state detection plus a hard round cap catches common cases but is not optimal.
- Liveness escalates, safety blocks. A system that distinguishes "the agents could not converge" from "the agents converged on the wrong answer" can route the first to a human and refuse to ship the second. One that cannot tell them apart does the wrong thing on both. This is the payoff of keeping the two axes separate rather than collapsing them into a single score, and it ties directly into oversight (Chapter 33) and agent evaluation (Chapter 29).

The shipped protocol

The protocol that falls out is small. Run a proposer to produce an artifact. Fork independent critics with no shared context. Each critic picks a distinct attack topic in its first round, security, performance, internal consistency, evidence gap, and later critics are told which topics are taken so they pick something else. Run cross-examination per critic until the proposer concedes or the critic gives up. Apply any concessions, then surface only the unresolved disputes, ranked by how long each survived attack. Figure 24.3 shows the per-critic loop and how it terminates.

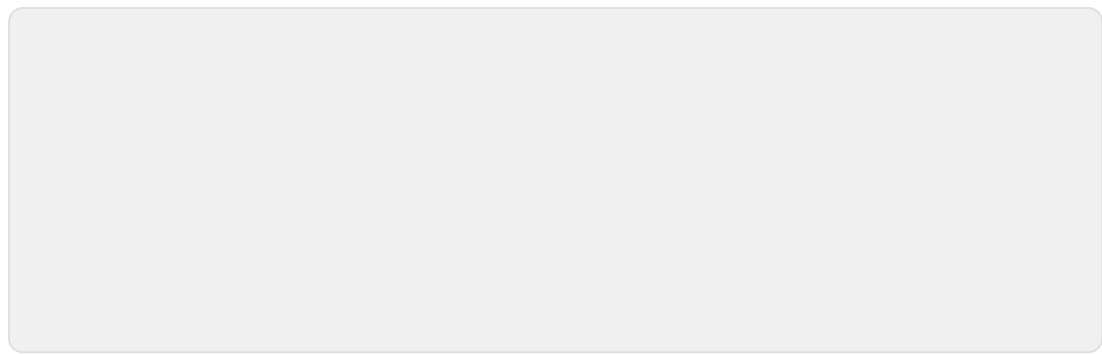


Figure 24.3. The cross-examination loop run for each critic. A critic claims a distinct attack topic, then alternates attack and defense until the proposer concedes, the critic gives up, or a hard round cap fires. Concessions are applied to the artifact; unresolved attacks are surfaced and ranked by rounds survived.

The judging layer should be cheaper and dumber than the proposing layer; this is the doubly-efficient debate intuition (Brown-Cohen et al. 2023). If the judge has to be as smart as the proposer, nothing was bought. A concrete realization keeps no model in the judge loop at all, scoring each attack by a pure contention count:

$$\text{score}(a) = \text{rounds_survived}(a) + \mathbf{1}[\text{re_attacked}(a)]$$

Two design properties make this work in practice. Claims need addresses: every nontrivial claim needs a handle, a file path, test case, source citation, requirement id, so critics have something concrete to attack and proposers something concrete to repair. Without addressability the review collapses into rhetoric. And the critic must reach the proposer through a verbatim channel, a plain user turn rather than a template that would distort the proposer's normal defense behavior. The shipped reference for this design is `agon`, which forks a coding session via Claude Code's `--fork-session`, runs the cross-examination in the fork, and writes only the unresolved disputes to disk, leaving the root transcript byte-identical. The research probe behind it, `agents-verification`, measures liveness and safety on independent axes across parallel-consensus and sequential-chain experiments rather than collapsing them into one accuracy number.

Further reading

- Frédéric Berdoz, Leonardo Rugli, Roger Wattenhofer, "Can AI Agents Agree?", 2026. [arXiv:2603.01213](#)
- Yu Cui, Hang Fu, Haibin Zhang, Licheng Wang, Cong Zuo, "Free-MAD: Consensus-Free Multi-Agent Debate", 2025. [arXiv:2509.11035](#)
- Arjun Panickssery, Samuel R. Bowman, Shi Feng, "LLM Evaluators Recognize and Favor Their Own Generations", 2024. [arXiv:2404.13076](#)
- Lin Shi, Chiyu Ma, Wenhua Liang, Xingjian Diao, Weicheng Ma, Soroush Vosoughi, "Judging the Judges: A Systematic Study of Position Bias in LLM-as-a-Judge", 2025. [ACL 2025](#)
- Geoffrey Irving, Paul Christiano, Dario Amodei, "AI safety via debate", 2018. [arXiv:1805.00899](#)
- Jonah Brown-Cohen, Geoffrey Irving, Georgios Piliouras, "Scalable AI Safety via Doubly-Efficient Debate", 2023. [arXiv:2311.14125](#)

25. RAG and Retrieval

A model's weights are a lossy, frozen snapshot of the corpus it was trained on. Retrieval-augmented generation puts a live, queryable corpus next to the model and feeds the right passages into the prompt at answer time. This chapter follows a single artifact, a question, as it travels down a funnel that turns it into a few hundred tokens of evidence. We trace the funnel one stage at a time, asking at each what failure it exists to prevent and what it costs; then we watch the field attack the three places the funnel is weakest; and we close on the two forces that decide whether retrieval survives at all, an adversary who hides in the corpus and a context window that keeps growing.

Knowledge inside the weights, or outside them

A frontier model knows what was in its pre-training mixture, up to the day the mixture was frozen, blurred by the lossy compression of Chapter 3. It does not know your private documents, it does not know what happened after the cutoff, and when asked about a fact it half-remembers it will produce a fluent, confident, wrong answer. Fine-tuning the knowledge in is expensive, slow to update, and still lossy. The cleaner move is to keep the knowledge outside the weights and fetch it on demand.

That reframes the problem as information retrieval married to generation. Given a question and a corpus of millions of passages, find the small set of passages that actually answer the question, place them in the context window, and let the model read and cite them. The hard parts are all in the retrieval: the corpus is large, so search must be sub-linear; the question and the answer rarely share exact words, so lexical match alone is brittle; the context window is finite and expensive, so you can pass only a handful of passages and they had better be the right ones. Retrieval is the act of turning a question into the few hundred tokens of evidence that change the answer. Lewis et al. named retrieval-augmented generation and framed it as combining parametric memory, the model's weights, with non-parametric memory, a retrievable index, training the retriever and generator together on knowledge-intensive tasks (Lewis et al. 2020).

Down the funnel, one stage at a time

The naive RAG pipeline that froze out of that idea has four stages: chunk and index the corpus offline, embed the query online, retrieve the nearest chunks, and generate an answer conditioned on them. Each stage exists to answer a specific failure of the stage that would otherwise replace it, and each carries its own dial with a knee. We walk them in order.

Chunking exists because documents are too long to embed or retrieve whole. A single vector cannot represent a hundred-page report at the granularity a question needs, and you cannot paste the whole report into the prompt. So the corpus is split into chunks, typically a few hundred tokens, and each chunk is indexed independently. The chunk size is a direct trade with no universal answer: chunks too large dilute the relevant sentence among irrelevant ones and waste context budget; chunks too small sever the sentence from the context that disambiguates it. The knee depends on document structure and question type, which is why overlapping windows and structure-aware splitting (by section, by paragraph) exist to soften the boundary problem.

Dense retrieval answers the brittleness of lexical match. A dual-encoder embeds the query and each chunk into the same vector space, trained so that a question and its answer land near each other even when they share no words. Karpukhin et al., in Dense Passage Retrieval, showed that a dual-encoder trained on a few thousand question-passage pairs beats a strong BM25 baseline by a wide margin on top-20 retrieval accuracy across open-domain QA

(Karpukhin et al. 2020). Relevance becomes a dot product or cosine similarity in embedding space:

$$\text{score}(q, c) = \mathbf{E}_q(q) \cdot \mathbf{E}_c(c)$$

where $\mathbf{E}_q$ and $\mathbf{E}_c$ are the query and chunk encoders. The encoders can be tied or separate, and the chunk embeddings are computed once, offline, then frozen into an index. The dial here is dense against sparse: dense retrieval generalizes across paraphrase and fails on exact tokens, which the next stage corrects.

The vector store answers the cost of search. With millions of chunk vectors, computing the dot product against every one for every query is too slow. Approximate nearest neighbor indexes trade a small, bounded recall loss for orders of magnitude in speed. The dominant structure is HNSW, the hierarchical navigable small-world graph of Malkov and Yashunin, which builds a multi-layer proximity graph and searches it greedily from a sparse top layer down, giving logarithmic-scale query time (Malkov and Yashunin 2018). A vector store is HNSW (or a quantized variant) plus metadata filtering plus the plumbing to keep the index in sync with the corpus. The dial is recall against latency: exact nearest neighbor is correct and too slow, and the index parameters are a recall-latency knob, not a fixed setting.

Hybrid search answers the blind spot of dense retrieval. Embeddings generalize across paraphrase but lose exact tokens: a part number, a rare name, a code identifier, a literal string that the dense model has smoothed away. Sparse lexical retrieval, BM25 over an inverted index, is exact on those terms and complementary. Hybrid search runs both and fuses the rankings, commonly with reciprocal rank fusion, so that a chunk ranked highly by either method surfaces. It pays two indexes and a fusion step to get both halves, and that cost is usually worth it.

Try giving a chunk a mediocre rank in both lists and watch reciprocal rank fusion still float it above items that ranked first in only one list.

```
# Reciprocal rank fusion: score = sum over lists of 1/(k + rank).
def rrf(rankings, k=60):
    scores = {}
    for ranking in rankings:
        for rank, doc in enumerate(ranking, start=1):
            scores[doc] = scores.get(doc, 0.0) + 1.0 / (k + rank)
    return sorted(scores, key=scores.get, reverse=True)

dense = ["A", "B", "C", "D"] # what embeddings rank highly
sparse = ["E", "C", "F", "A"] # what BM25 ranks highly
fused = rrf([dense, sparse])
print("dense :", dense)
print("sparse:", sparse)
print("fused :", fused)
print("C wins: ranked 3rd and 2nd, never 1st, but agreed on by both.
```

Reranking answers the gap between cheap retrieval and accurate scoring. The dual-encoder is fast because the query and chunk never meet until the final dot product, which is also why it is imprecise: it cannot model fine-grained interaction between the query's words and the chunk's.

A cross-encoder reranker feeds the query and chunk together through a transformer and scores their joint representation, which is far more accurate and far too slow to run over the whole corpus. So the pipeline retrieves a few hundred candidates cheaply, then reranks the top candidates expensively, and the length of that shortlist is the dial: running the cross-encoder only on a retrieved shortlist buys most of the accuracy at a fraction of the cost. ColBERT sits between the two: its late-interaction architecture keeps per-token embeddings and does a cheap MaxSim interaction, recovering much of the cross-encoder's accuracy at retrieval-time cost (Khattab and Zaharia 2020).

Drag the shortlist depth and watch answer accuracy climb fast at first, then flatten: the first dozen reranked candidates carry most of the gain, while a longer shortlist costs cross-encoder compute for diminishing returns.

Reranking accuracy against shortlist depth, a diminishing-returns curve: the cross-encoder buys most of its accuracy on the first handful of candidates.

The cost and accuracy of these three architectures follow directly from one design choice: where in the network the query and the chunk first meet, shown in Figure 25.1.

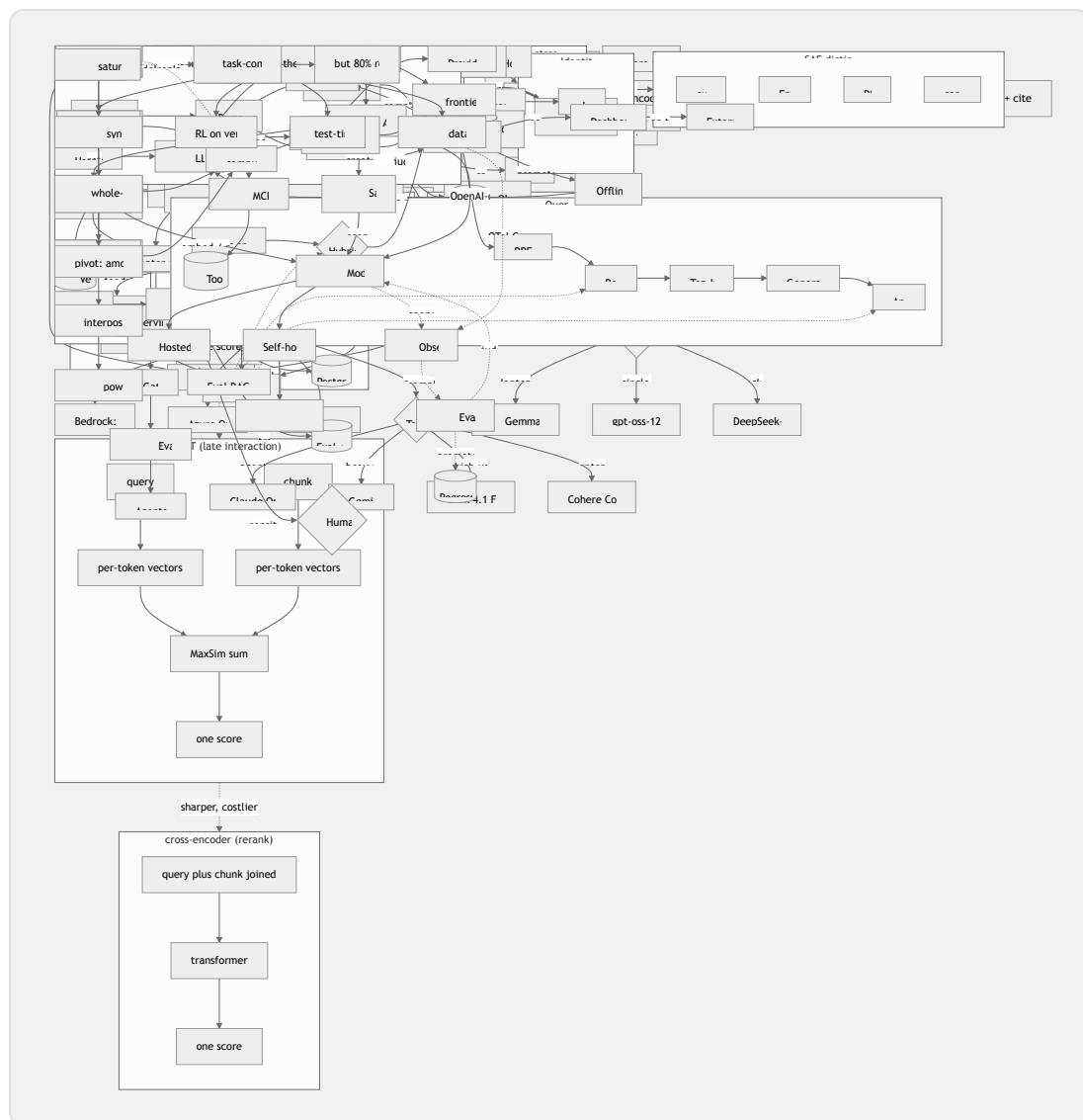


Figure 25.1. Three retrieval architectures, ordered by where the query and chunk interact. The bi-encoder meets them only at the final dot product, so chunk vectors precompute offline and retrieval is cheap. The cross-encoder fuses them at the input, which is accurate but must run per pair. ColBERT keeps per-token vectors and interacts late via MaxSim, between the two.

Stacked end to end, the four stages form a funnel, shown in [Figure 25.2](#).

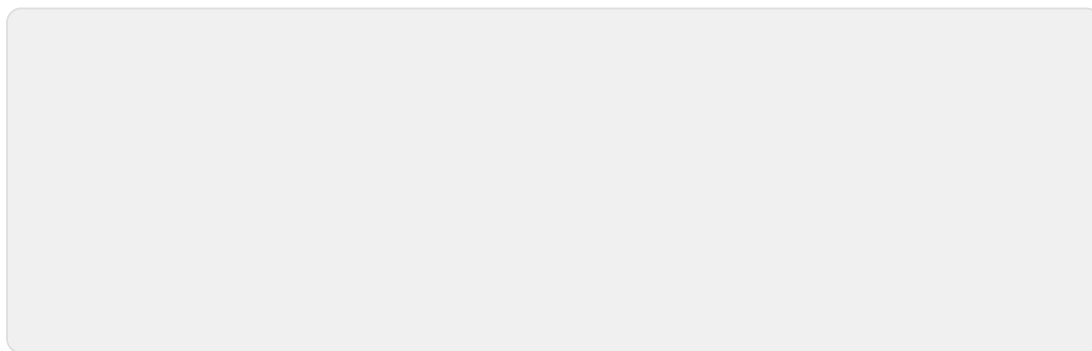


Figure 25.2. The naive RAG funnel. Cheap, broad stages on the left feed a short list to the expensive, sharp stages on the right, so the cross-encoder only ever scores a handful of candidates.

Each stage is cheaper-and-broader or costlier-and-sharper than the next, and the order exists so that the expensive stages only ever see a short list. This is the workhorse design, and its strength is exactly its rigidity: embed once, retrieve once, generate once, with every dial set ahead of time.

Three weaknesses, three lines of attack

That rigidity is also where the funnel breaks. The naive pipeline decouples its stages and runs each exactly once, which gives it three structural weaknesses, and most of the field's evolution is one line of work per weakness, [Figure 25.3](#).

The first weakness is local-only reasoning. Naive RAG retrieves the chunks most similar to the query, which works for questions whose answer sits in a few passages and fails for questions whose answer is spread across the whole corpus. Asked "what are the main themes in these documents?", similarity search has nothing to grab, because no single chunk is about the themes. GraphRAG reframes this as query-focused summarization: it uses a model to extract an entity-and-relationship graph from the corpus, runs Leiden community detection to partition the graph hierarchically, and pre-summarizes each community. A global question is then answered by map-reduce over community summaries rather than over raw chunks, which the authors show improves the comprehensiveness and diversity of answers on sensemaking questions over corpora in the million-token range ([Edge et al. 2024](#)). LightRAG keeps the graph idea but cuts the cost: a dual-level retrieval that combines low-level entity retrieval with high-level concept retrieval over a graph index, with incremental updates so a new document does not force a full re-index ([Guo et al. 2024](#)).

The second weakness is the static, single-shot control flow. Naive RAG always retrieves, retrieves exactly once, and never checks whether what it retrieved was any good. The fix is to put a controller in the loop. Self-RAG trains the model to emit reflection tokens that decide whether to retrieve at all and then critique each retrieved passage for relevance and support before using it ([Asai et al. 2023](#)). CRAG adds a lightweight evaluator between retrieval and generation that scores retrieved chunks and, when they fall short, triggers a corrective web search instead of generating from weak evidence ([Yan et al. 2024](#)). These are the bridge from naive to agentic.

The third move generalizes the second. Singh et al., surveying agentic RAG, describe the endpoint: the retrieval loop becomes an agent that plans, decomposes a question into sub-queries, chooses which tools and indexes to query, reflects on intermediate results, and iterates until it has enough evidence ([Singh et al. 2025](#)). Retrieval stops being a fixed funnel and becomes a policy the agent runs, drawing on the agent architectures of [Chapter 21](#) and, when multiple specialized retrievers are involved, the coordination of [Chapter 24](#). The same

passages, reordered and re-queried, get used very differently. The dial across this whole axis is naive against agentic: a single-shot pipeline is cheap, predictable, and low-latency; an agentic loop answers harder questions but multiplies token cost and latency and adds failure modes of its own. Most production traffic does not need the agent; the minority that does benefits enormously.

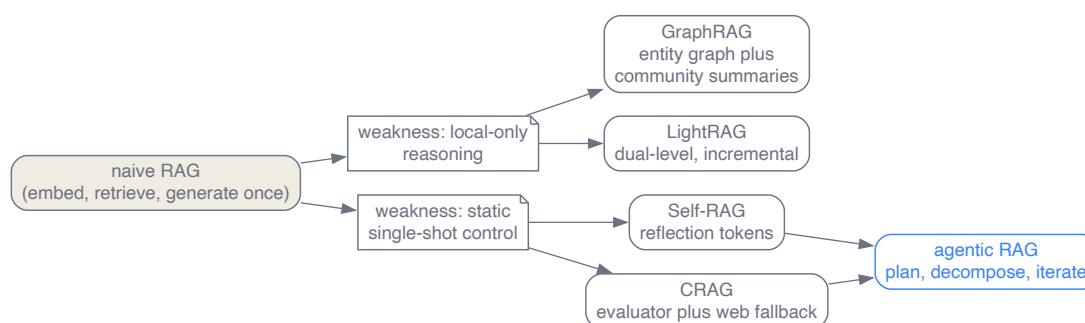


Figure 25.3. How the field extends naive RAG. Each of the three structural weaknesses spawns a line of work, and the corrective controllers generalize into the agentic loop.

Retrieved text is untrusted input

Adding retrieval moves the security boundary. Once untrusted text from a corpus flows into the prompt, that text can carry instructions. SafeRAG benchmarks this directly, classifying attacks into silver noise, inter-context conflict, soft advertisement, and white denial-of-service, and finds that across many RAG components even simple injected passages bypass retrievers and filters and degrade answer quality (Liang et al. 2025). Retrieved content is untrusted input and must be treated as such, which connects directly to Chapter 34. This is a weakness the funnel cannot tune away: no chunk size or rerank depth makes a planted instruction safe, because the problem is trust, not ranking.

Will long context make retrieval obsolete?

The second force pressing on retrieval is friendlier in appearance and more fundamental: the context window keeps growing, and a large enough window might make the whole funnel unnecessary.

WHAT'S CONTESTED

The live debate is whether long context will make retrieval obsolete. As context windows grow toward and past a million tokens, one camp argues the simplest design wins: drop the retrieval machinery, paste the whole corpus into the prompt, and let attention do the searching. The other camp argues retrieval stays necessary on three grounds the window does not address: cost, since attention over a million tokens per query is far more expensive than fetching a few thousand; freshness, since the corpus changes faster than you can re-prompt and an index updates incrementally; and scale, since real corpora exceed any window. The evidence cuts against the pure long-context position: Liu et al., in "Lost in the Middle," show that models use evidence placed mid-context markedly worse than evidence at the edges, so a full window is not a uniform substitute for a short, well-ranked one (Liu et al. 2023). The honest reading is that the two are converging. Long context makes retrieval coarser and more forgiving, and retrieval makes long context affordable and current. See Chapter 20 for the window side and Chapter 26 for how the retrieved evidence is assembled into the prompt.

The debate has a clean resolution once you ask who pays for the window, and the answer comes from a layer well below retrieval.

CONSTRAINT ARROW

Retrieval is kept economically necessary by a lower layer. The serving cost of attention, Chapter 16, and the key-value cache it fills, Chapter 17, make every token in the context window a recurring expense paid on every request. That cost is what forbids the simplest design, pasting the entire corpus into the prompt, and forces a retrieval funnel that delivers a few hundred high-value tokens instead of a few hundred thousand mediocre ones. The shape of the retrieval pipeline is dictated by what a token of context costs to serve.

Making it real

A minimal naive RAG loop is short, and the brevity is the point: the sophistication lives in the index, the embeddings, and the reranker, not the control flow.

```
def answer(query, k=5):
    q = embed(query)
    dense = ann_index.search(q, n=100)          # HNSW recall
    sparse = bm25.search(query, n=100)         # exact terms
    fused = reciprocal_rank_fusion(dense, sparse)
    top = cross_encoder.rerank(query, fused)[:k]
    return generate(prompt=build_context(query, top))
```

Three implementation realities decide whether this works in practice. The first is index freshness: the offline chunk-and-embed step must run incrementally as the corpus changes, or retrieval silently serves stale evidence. The second is evaluation, which is harder than it looks, because a RAG system can fail at retrieval (the right chunk was never fetched) or at generation (the right chunk was fetched and ignored or misread), and these need separate measurement. Retrieval quality is measured with ranking metrics like recall at k and nDCG; answer quality with faithfulness to the retrieved evidence and answer relevance, typically judged by a model as in Chapter 28, with the caveats about agentic and trajectory evaluation in Chapter 27. The third is the provenance contract: because every claim should trace to a retrieved chunk, the pipeline must carry citations through to the answer, which is both the original selling point of RAG in Lewis et al. (Lewis et al. 2020) and the main defense against the confident hallucination that motivated it.

RAG sits beside, not inside, the agent's working memory. The retrieved context is ephemeral, assembled per request and discarded, which distinguishes it from the persistent state of Chapter 22. Retrieval is how an agent reaches knowledge it does not hold; memory is how it keeps what it has learned. The two compose, and the strongest systems use both.

Further reading

- Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," 2020. [arXiv:2005.11401](#)
- Karpukhin et al., "Dense Passage Retrieval for Open-Domain Question Answering," 2020. [arXiv:2004.04906](#)

- Khattab & Zaharia, "ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT," 2020. [arXiv:2004.12832](#)
- Malkov & Yashunin, "Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs," 2018. [arXiv:1603.09320](#)
- Edge et al., "From Local to Global: A Graph RAG Approach to Query-Focused Summarization," 2024. [arXiv:2404.16130](#)
- Guo et al., "LightRAG: Simple and Fast Retrieval-Augmented Generation," 2024. [arXiv:2410.05779](#)
- Asai et al., "Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection," 2023. [arXiv:2310.11511](#)
- Yan et al., "Corrective Retrieval Augmented Generation," 2024. [arXiv:2401.15884](#)
- Singh et al., "Agentic Retrieval-Augmented Generation: A Survey on Agentic RAG," 2025. [arXiv:2501.09136](#)
- Liang et al., "SafeRAG: Benchmarking Security in Retrieval-Augmented Generation of Large Language Model," 2025. [arXiv:2501.18636](#)
- Liu et al., "Lost in the Middle: How Language Models Use Long Contexts," 2023. [arXiv:2307.03172](#)

26. Context Engineering

A frontier model does not act on the world. It acts on its context window, the bounded sequence of tokens it can attend to in a single forward pass. Everything the model knows about a task at the moment of generation, the instructions, the retrieved documents, the conversation so far, the tool results, must be assembled into that window by something outside the model. By the end of this chapter a reader can explain why the field renamed this work from prompt engineering to context engineering in 2025, why a longer context window does not by itself solve the problem, and how token budgets and tool protocols shape what an agent can actually do.

The chapter turns on one stubborn fact: a bigger window does not make the problem go away. It moves the problem. Each section below pushes on that claim from a different side, from why the window misbehaves, to what lever is left, to how a running agent assembles its context and where that assembly breaks.

The window is the only channel

The context window is a fixed, scarce resource, and it is the only channel through which a model perceives a task. That single sentence carries the weight of the whole chapter, so it is worth holding still.

The window is finite, and every token in it competes for the same space. A system prompt, a tool catalog, retrieved passages, and a long conversation history all draw from one budget. When the budget is exceeded, something must be dropped, summarized, or retrieved on demand, and each of those choices changes what the model can answer. There is no second channel. Whatever the model does not see in the window, it does not know.

A reader may reasonably expect that a larger window dissolves this pressure. The next section is the reason it does not.

Why a bigger window does not help

A model does not use the tokens in its window uniformly. Even when the relevant information is present, a model tends to attend most reliably to the beginning and the end of its input and least reliably to the middle. Liu et al. named this the lost-in-the-middle effect ([Liu et al. 2023](#)), measuring a characteristic U-shaped accuracy curve on multi-document question answering and key-value retrieval: performance is highest when the needed fact sits near the start or the end of the context and degrades, sometimes below the closed-book baseline, when it sits in the middle. A bigger window does not repeal this. It enlarges the middle.

Figure 26.1 sketches the consequence as a placement map. Reliability is high at the two boundaries and sags through the long middle, so where a fact sits in the window, not merely whether it is present, governs whether the model uses it.

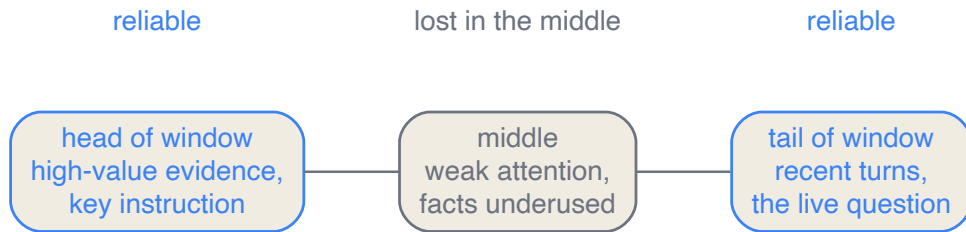


Figure 26.1. A schematic placement map of the context window. Attention is reliable near the two boundaries and weakest through the middle, after the lost-in-the-middle effect of Liu et al. (2024). The shape is illustrative, not measured.

Vary where the needed fact sits and watch the recall move: this simulates a U-shaped recall profile, then Monte-Carlo places one fact at the head, the middle, and a random slot to compare hit rates.

```

import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(0)
pos = np.linspace(0, 1, 41) # position in the window, 0
recall = 0.45 + 0.5 * (2 * pos - 1) ** 2 # U-shape: high at edges, 1

def hit_rate(p, trials=20000):
    prob = np.interp(p, pos, recall)
    return rng.random(trials) < prob

for name, p in [("head", 0.0), ("middle", 0.5), ("random", None)]:
    where = rng.random(20000) if p is None else p
    print(f"{name:7s} recall = {hit_rate(where).mean():.3f}")

plt.plot(pos, recall); plt.xlabel("position (head to tail)")
plt.ylabel("recall probability"); plt.title("lost in the middle"); p

```

So the problem is not only "how do I fit everything in," it is "what do I put in, in what order, so the model uses it." That is a budgeting and placement problem, and it is the work the rest of this chapter is about.

The lever: context as a programming surface

The model's weights are fixed at inference time. The only lever left is what enters the window and where, so the design idea is to treat the context window as a managed buffer, not a prompt.

The lever is real because of in-context learning. Brown et al. showed with GPT-3 that a sufficiently large autoregressive model can perform a task from demonstrations placed in its context, with no weight update at all (Brown et al. 2020). They distinguished three regimes by

how many demonstrations the prompt carries: zero-shot, an instruction with no examples; one-shot, a single example; and few-shot, a handful, typically ten to one hundred, that fit in the window. The finding that mattered for everything since is that the context is a programming surface. You can change a model's behavior by changing its input, within one session, at the cost of the tokens the demonstrations consume.

Once the context is a programming surface, the assembly of it becomes the artifact worth engineering, which is the work a survey of the field now catalogs as a discipline in its own right (Mei et al. 2025). Andrej Karpathy made the argument for the name in mid-2025, proposing context engineering over prompt engineering on the grounds that a "prompt" connotes a short instruction, while real applications perform "the delicate art and science of filling the context window with just the right information for each step" (Karpathy 2025). Tobi Lutke framed it from the application side the same week: context engineering is "the art of providing all the context for the task to be plausibly solvable by the LLM" (Lütke 2025). The rename is not cosmetic. It moves the unit of work from a single string a human writes to a pipeline a system assembles: system prompt, retrieved documents, conversation history, distilled memory, and tool outputs, composed for each step of a running agent.

Three moves: budget, place, compress

Three design moves follow directly from the two facts above, that the window is finite and that the model underuses its middle.

Budgeting. Treat the window as a cost-accounted resource. Account for the fixed overhead first, the system prompt and the tool definitions, then divide what remains among history, retrieval, and headroom for the model's own output. Retrieval (Chapter 25) exists precisely so that not everything has to live in the window at once; the corpus stays outside and only the relevant slice is paged in per query.

Placement. Because the middle is the weak position, order matters. Put the instruction the model must follow and the highest-value evidence where attention is reliable, near the boundaries, not buried in a long undifferentiated dump. Lost-in-the-middle is a property of the model, and it dictates a choice in the layer above it.

Compression over accumulation. A conversation or an agent trajectory grows without bound, but the window does not. Old turns are summarized into distilled state, tool outputs are filtered before they re-enter the window, and durable facts are written to an external memory (Chapter 22) and retrieved when needed rather than carried forward verbatim.

CONSTRAINT ARROW

Context budgeting is not a stylistic preference; it is forced from below by the cost of a token. Every token in the window must sit in the key-value cache and be attended to, and the serving problem in Chapter 16 and the cache scheduling in Chapter 17 show that this cost grows with context length, in memory and in compute. A longer window is not free capacity you may as well fill. The per-token serving cost is what makes a tight, well-ordered context the right design even when a larger window is technically available.

From a string to a pipeline

The three moves did not arrive all at once. The lineage runs from a single string toward a managed pipeline, and tracing it shows why each move became necessary.

The first era was prompt engineering. With in-context learning established, practitioners learned to coax behavior from a model by wording: phrasing an instruction carefully, adding a

few worked examples, appending "let's think step by step" to elicit a chain of thought ([Chapter 13](#)). The artifact was a prompt template, and the skill was writing it.

The limits of that era showed up as applications grew. A single hand-written string cannot hold a large knowledge base, a long conversation, or the output of tools the model calls. Retrieval-augmented generation ([Chapter 25](#)) answered the knowledge-base limit by fetching relevant passages at query time. Agent harnesses ([Chapter 23](#), [Chapter 21](#)) answered the action limit by letting the model call tools and feeding the results back into the window. Both turned the context into something assembled at runtime from many sources, which is the moment prompt engineering became too small a word for the job. The rename to context engineering in 2025 was the field naming what it had already started doing: Karpathy and Lutke gave it a label, but the practice of budgeting, retrieving, ordering, and compressing the window predated the term.

The recursion: budgeting the tools themselves

Tools created their own integration problem and their own protocol, and that protocol then turned the budgeting problem back on itself.

Before late 2024, every application wired each model to each data source with bespoke glue, an N-times-M problem. Anthropic released the Model Context Protocol (MCP) in November 2024 as an open standard for that wiring ([Anthropic 2024](#)): a model speaks one protocol, and any MCP server, for a filesystem, a database, an issue tracker, exposes its tools and data through it. MCP standardized how external context and capabilities enter the window. [Figure 26.2](#) contrasts the two wirings: the bespoke mesh where every model integrates every source directly, against the hub where each side speaks the protocol once.

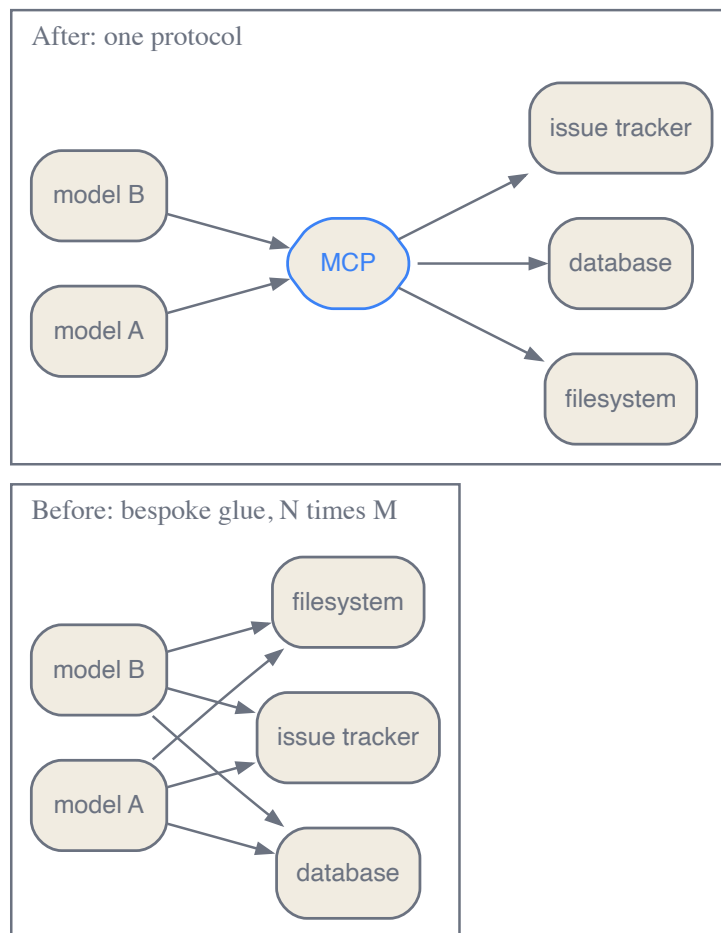


Figure 26.2. From an N-times-M integration mesh to a single protocol. MCP replaces bespoke per-pair glue with one standard each side speaks once, after Anthropic (2024).

The protocol then revealed the budgeting problem at the layer above it. As agents connected to dozens of MCP servers, the tool definitions themselves filled the context before the agent read a single request. In November 2025 Anthropic described code execution with MCP as the response (Anthropic 2025): rather than loading every tool definition upfront, present MCP servers as code APIs that the model discovers and calls by writing code, loading only the definitions a task needs and filtering tool results in the execution environment before they return to the window. Their illustrative example reduces a workflow from about 150,000 tokens of loaded definitions and intermediate results to roughly 2,000, a 98.7 percent reduction. The lesson is the recurring one of this chapter: the constraint is the context budget, and the design moves to defend it.

Where the balances bite

Context engineering is a set of balances, each with a knee.

- **Few-shot versus zero-shot.** Demonstrations raise reliability on a task but cost tokens, and every demonstration token is a token unavailable for retrieval, history, or output. As instruction-tuned models improved, the break-even shifted: many tasks that once needed several examples now run zero-shot, and examples are worth their cost only where they pin down a format or a corner case the instruction cannot convey concisely.
- **More context versus better context.** Stuffing the window with everything plausibly relevant is the easy move and often the wrong one. It raises cost per call, and it pushes

high-value evidence into the lost-in-the-middle dead zone where the model underuses it. A smaller, well-selected, well-ordered context frequently beats a larger indiscriminate one.

- **Static prompt versus assembled pipeline.** A fixed template is simple, debuggable, and cheap to reason about. A pipeline that retrieves, ranks, summarizes, and orders per step is more capable but introduces moving parts that fail in their own ways: a bad retrieval poisons the context, a faulty summary drops the fact the next step needed.
- **Tool breadth versus token cost.** Connecting many tools makes an agent more capable in principle, but loading all their definitions upfront spends the budget before work begins. Progressive disclosure, loading definitions on demand as in code execution with MCP, recovers the budget at the cost of an extra discovery step and a more complex runtime.

WHAT'S CONTESTED

The live disagreement is whether long-context models make context curation obsolete. One camp argues that as windows grow to hundreds of thousands or millions of tokens, you can simply load everything and let attention sort it out, so retrieval and careful ordering become legacy workarounds. The other camp points to lost-in-the-middle and to per-token cost: a model that underuses its middle and a window that is paid for on every call mean that selecting and placing context still wins, regardless of how large the window grows. A second, sharper dispute is whether "context engineering" names a real discipline or rebrands prompt engineering; Karpathy answered directly that he is not coining a term but resisting the oversimplification that "prompt" invites. A third concerns MCP's original load-everything design: Anthropic's own code-execution follow-up suggests that loading all tool definitions upfront was the wrong default, and the right abstraction for exposing tools to a model is still unsettled.

Building one context, and how it breaks

In practice the context for one model call is built, not written. A budget-aware assembler is the minimal shape:

```
def build_context(task, history, tools, retriever, window):
    budget = window - reserve_for_output(window)
    parts = []
    parts.append(system_prompt(task))           # fixed, goes first
    parts.append(tool_manifest(tools))          # load on demand if l
    evidence = retriever.search(task.query)      # only the relevant s
    parts.append(rank_and_trim(evidence, budget))
    parts.append(summarize_old(history, budget)) # compress, do not
    return order_for_attention(parts)           # high-value near bou
```

Figure 26.3 traces the same assembler as a data flow. Many sources feed one budgeted buffer: the fixed overhead is accounted first, retrieval pages in only a slice of the corpus, history is compressed rather than carried forward, and a final ordering step lifts the highest-value parts to the boundaries where Figure 26.1 says the model will read them.

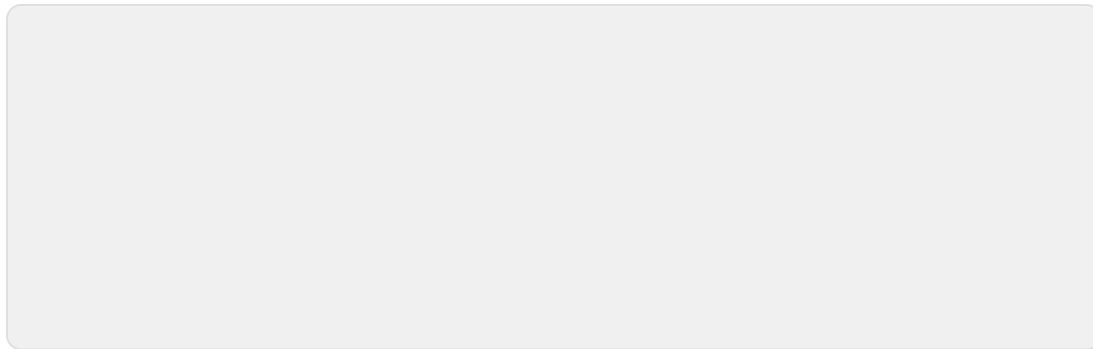


Figure 26.3. Context as an assembled pipeline. Sources are budgeted, retrieved as slices, compressed, then ordered so high-value parts land near the boundaries.

Three properties of this sketch carry the chapter's argument. The output reserve comes off the top, because a context that fills the window leaves the model no room to answer. Retrieval returns a slice, not the corpus, so the knowledge base lives outside the window and only its relevant part is paged in. And `order_for_attention` is not cosmetic: it places the instruction and the strongest evidence where lost-in-the-middle says the model will actually read them.

The failure modes are worth naming because each bites at a different place. Context overflow truncates silently and drops whatever fell off the end, often the very history the next turn needed. Lost-in-the-middle degradation is invisible in a green build: the fact is present, the model just does not use it, and the only way to see it is to vary placement and watch the score move. Retrieval poisoning puts a confidently wrong passage in the window and the model treats it as evidence. Tool-definition bloat spends the budget before the agent starts, which is the problem code execution with MCP exists to solve. Each of these is a budgeting or placement failure, which is why budgeting and placement, not wording, are the substance of the discipline.

Further reading

The sources cited inline above appear on the book's References page. The survey below is a fuller map of the field for a reader who wants the broader landscape.

- Mei et al., "A Survey of Context Engineering for Large Language Models," 2025. [arXiv:2507.13334](https://arxiv.org/abs/2507.13334)

27. Benchmarks and Their Discontents

Evaluation is how you know any stage of the stack worked, the cross-cutting gate that data, architecture, pre-training, and post-training all answer to. This chapter is about its oldest and most contested instrument, the static benchmark. The thread running through it is a single uncomfortable claim: a published benchmark score is never a fact about a model alone. It is a claim about a model and its measurement apparatus together, and most of the discipline of evaluation is the work of making that claim survive scrutiny. By the end a reader can say what a held-out set really is, why benchmark families saturate, how train/test contamination silently inflates every number, and what it takes before a score counts as evidence rather than folklore.

The anatomy of a wrong number

A training pipeline produces a new set of weights every time a stage runs, and the only honest way to know a stage worked is an independent number that moved for a reason that survives scrutiny. A benchmark is meant to be that falsifier. The recurring failure is not the absence of a number but belief in a wrong one: a score contaminated by test data leaking into training, gamed by tuning toward a known format, or simply noise dressed up as signal. Each of these produces a number that looks excellent and does not transfer, and the damage is silent, because nothing in the score itself announces that it is wrong.

No single layer of evaluation catches all three failures. A practical pipeline treats automated benchmarks, production monitoring, A/B tests, user feedback, and human studies as a Swiss-cheese stack, where each layer catches what the others let through. This chapter is about the first layer, the static benchmark, which is the cheapest, the most comparable across labs, and the most prone to quietly lying.

Figure 27.1 shows why no single layer is enough: each of the three failures slips past whatever layer is blind to it, and only the full stack catches all of them.

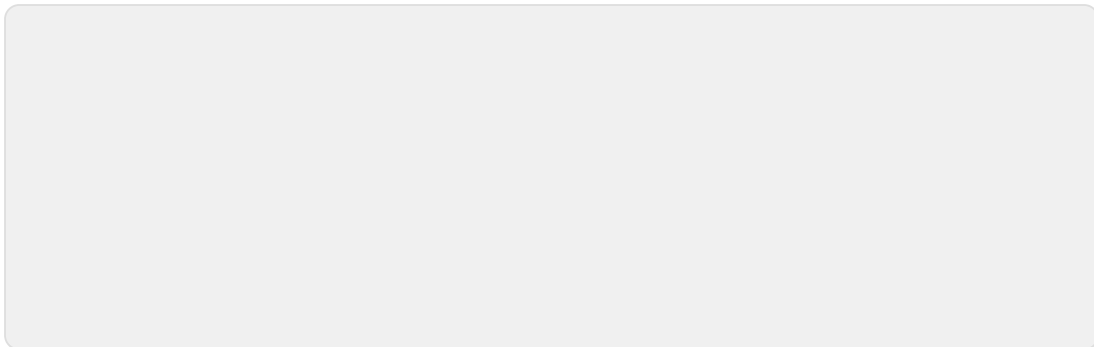


Figure 27.1. The Swiss-cheese model of evaluation. Contamination, format gaming, and noise each slip past the layers blind to them, so confidence comes from the stack rather than any single layer.

These three failures, contamination, gaming, and noise, are the spine of the chapter. The sections that follow build the static benchmark up from what it measures, expose where each failure enters, and end with the discipline that keeps a number honest in spite of them.

What a benchmark actually measures

The capability surface is too large to measure with one test, so benchmarks decompose it into families, each probing a different axis and each failing in a characteristic way.

- **Knowledge.** Broad factual and academic recall, in the style of MMLU (Hendrycks et al. 2020). It saturates and contaminates easily, and frontier high-water marks now sit near the ceiling.
- **Reasoning.** Multi-step inference beyond recall. It is harder to contaminate but harder to score objectively, which pushes the field toward graduate-level, google-proof variants like GPQA (Rein et al. 2023).
- **Math.** Grade-school through competition, from GSM8K (Cobbe et al. 2021) to MATH (Hendrycks et al. 2021). The answer is checkable, which is exactly the property that bridges evaluation to verifiable-reward reinforcement learning in Chapter 14.
- **Code.** Function completion and repository-level repair, from HumanEval (Chen et al. 2021) to SWE-bench-style tasks (Jimenez et al. 2023). It is executable, so the judge is a test runner rather than a model.
- **Multilingual.** The same task across languages. It exposes tokenizer and data-mixture decisions made in Chapter 4 that were invisible while the model was measured only in English.
- **Long-context.** Retrieval and reasoning over long inputs, from needle-in-a-haystack to harder probes like RULER (Hsieh et al. 2024). It gates the long-context extension discussed in Chapter 20.
- **Agentic.** Tool use, multi-step task completion, and web or operating-system interaction, named by benchmarks like GAIA (Mialon et al. 2023), WebArena (Zhou et al. 2023), and τ -bench (Yao et al. 2024). These do not reduce to a single static answer, and their evaluation method is the subject of Chapter 29.

Each family carries its own characteristic failure, but all of them rest on the same precondition, and it is that precondition, not the family taxonomy, that decides whether any of these numbers mean anything.

The held-out contract

Underneath every family sits the same precondition: a held-out set whose answers the model has never seen during training. Held-out is not a property of a file, it is a property of the whole pipeline. A set is held-out only if it appears in no training mixture, which is a contract the data layer must honor, not a checkbox the evaluation layer can tick on its own.

Contamination is the violation of that contract, the first of the chapter's three failures and the most consequential, because it is silent and it inflates in exactly the direction that flatters a result. Since it announces nothing, it has to be hunted rather than waited for. Detection works on overlap and membership. Exact and near-duplicate matching, typically n-gram overlap against the training corpus, catches verbatim and lightly-edited leakage. Canary strings, unique markers embedded in a held-out set, reveal that the set was ingested if the model can reproduce them. Membership-inference signals go further and ask whether a specific example was likely trained on at all, for instance Min-K% Prob, which flags an example when its least-probable tokens are suspiciously high-probability under the model (Shi et al. 2023). None of these is conclusive alone, which is why contamination is reported as a contract and an audit rather than a single test.

Figure 27.2 traces the held-out contract end to end: the two paths by which a benchmark reaches the weights, and the three detection signals that audit whether it did. The second path, a teacher model that was itself trained on the benchmark, bypasses the corpus entirely and returns in the final section.

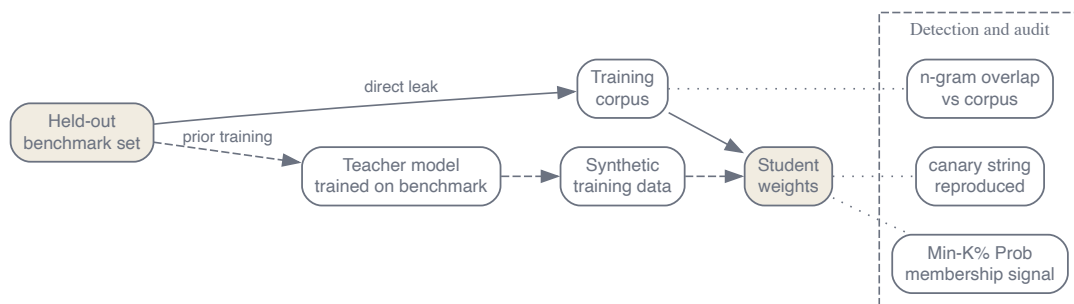


Figure 27.2. How a benchmark contaminates a model and how the contract is audited. The direct path leaks through the training corpus; the indirect path arrives through a teacher model's synthetic data. Detection signals (Shi et al., 2023) audit the trained weights after the fact.

Why benchmarks decay

A held-out set is only useful while it stays both held-out and hard. The history of benchmarks is a history of saturation, the slow erosion of the second property. A benchmark discriminates between systems only while models fail some of it; once a family clusters near its ceiling, the remaining headroom is noise, the third of the chapter's failures, and the benchmark stops telling systems apart.

Knowledge benchmarks reached this first. MMLU was a meaningful spread when it appeared (Hendrycks et al. 2020) and is now near ceiling for frontier models, which is why the field moved to deliberately harder, contamination-resistant variants. GPQA was built to be google-proof, with questions that experts outside the field cannot answer even with web access, precisely so that recall and retrieval cannot substitute for reasoning (Rein et al. 2023).

Figure 27.3 shows the shape of saturation. As successive model generations climb toward the ceiling, the spread between systems collapses into noise, which is why a harder successor benchmark has to be introduced to restore discrimination, only to inherit the same fate.

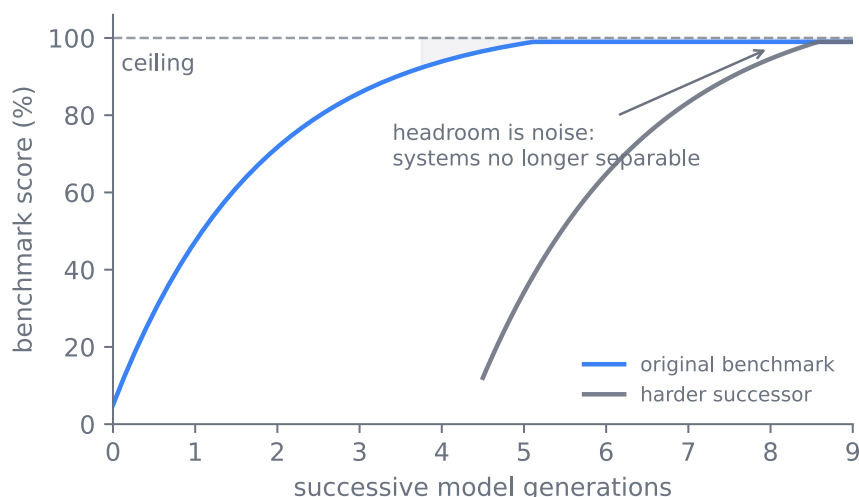


Figure 27.3. Schematic illustration of benchmark saturation, after Hendrycks et al. (2020). Successive model generations climb a fixed benchmark toward its ceiling; once the frontier clusters near the top, the remaining headroom is noise and the benchmark stops separating systems, so a harder successor is introduced. The curves are idealized, not measured scores.

Saturation pushes the design in two directions at once, and the choice between them is the central trade-off of static evaluation. One direction is toward harder static sets, which buys time but inherits the same fate: any fixed set is a target, and a target eventually gets hit, whether by genuine capability or by contamination. The other is toward living evaluation that resists a fixed target altogether. Chatbot Arena replaces a frozen test set with a stream of fresh human-preference comparisons, so there is no static answer key to leak and the measurement renews itself as the model population changes (Chiang et al. 2024). The arc runs from a fixed academic exam, to a harder google-proof exam, to a moving evaluation with no answer key to memorize.

Two trade-offs fall out of that arc, and a published number is only as meaningful as the reader's awareness of which side of each it came from.

- **Static versus dynamic.** Static benchmarks are cheap to run, reproducible to the bit, and comparable across labs and over time, which is why leaderboards are built on them. They also saturate, contaminate, and miss anything that is not a single static answer. Dynamic evaluation, meaning rotating sets and live environments, resists gaming because there is no fixed target to overfit, but costs more to run and is harder to compare across time because the test itself is moving.
- **Public versus private held-out.** A public benchmark should be assumed contaminated over time, since its answer key is on the internet and flows into the next training run by default. A private held-out set is trustworthy, but only while its integrity holds, and integrity is a wasting asset: every leak, every accidental inclusion, every published example erodes it. The honest posture is to trust a private set until it leaks and to assume a public set already has.

Model, or harness?

Contamination and saturation both attack the test. The third failure attacks the act of measuring itself, and it is where the chapter's central claim becomes sharpest: even a clean, unsaturated benchmark does not produce a number until a harness wraps it, and the harness moves the number.

WHAT'S CONTESTED

Do benchmark scores measure the model, or the harness around it? The same weights produce different scores depending on the prompt template, the number of few-shot examples, how the answer is parsed out of the generation, and the decoding settings, and these choices are not standardized across labs. A reported number is therefore a property of the model and its measurement harness together, and two labs running the same benchmark on the same model can disagree, and rankings can flip, purely because their harnesses differ. This is the motivation behind standardized evaluation frameworks like HELM (Liang et al. 2022), which pin the harness so that a score reflects the model rather than the measurement apparatus. The contested question is how much of any cross-model gap is capability and how much is harness, and the only honest answer is to fix and publish the harness, not to trust a bare number.

Figure 27.4 makes the worry concrete: one frozen set of weights produces different scores, and can even change rank order, depending only on the harness knobs between the model and the reported number.

Figure 27.4. Why a benchmark score is a property of the model and its harness together. Fixed weights pass through unstandardized harness choices, so the same model can post different scores and rankings can flip across labs.

Keeping a number honest

Each of the three failures has a discipline that answers it, and operationalizing held-out integrity is where those disciplines become procedure rather than principle. A set earns the word held-out only if it is never placed in any training mixture, version-pinned so a score names the exact set it was measured against, access-controlled so it does not leak through carelessness, rotated when a leak is detected, and large enough to give a usable signal rather than a single fragile number. This is the asset the evaluation layer owns and defends, and it is the answer to noise as much as to contamination, since a set too small to be stable is itself a source of false signal.

Figure 27.5 makes the size requirement concrete. The confidence interval on an accuracy estimate shrinks only as the square root of the held-out set size, so a small set carries an error bar wide enough to swamp the gap between two systems, which is precisely how noise masquerades as signal.

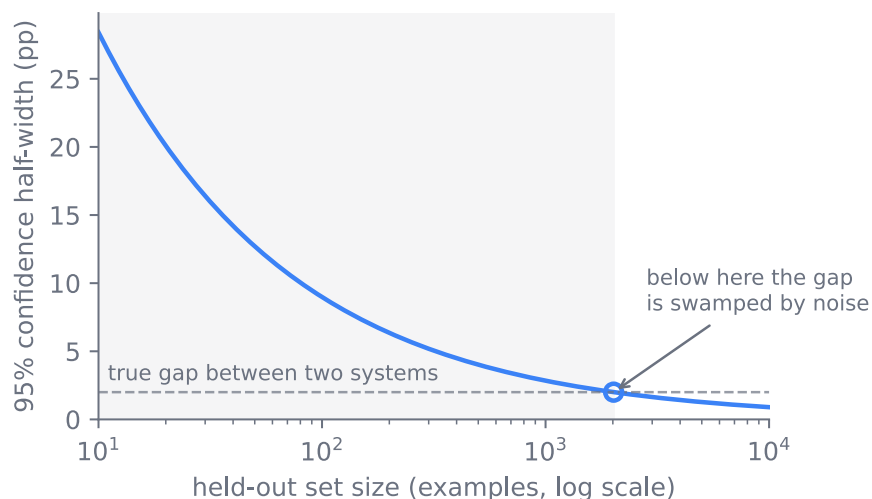


Figure 27.5. Schematic illustration of why a held-out set must be large enough. The 95% confidence half-width on an accuracy estimate shrinks as one over the square root of the number of held-out examples, so below a critical size the error bar swamps the true gap between two systems. The curve is idealized from the binomial standard error, not measured results.

Try editing `p1` and `p2` to two accuracies you want to tell apart, and watch how many held-out examples it takes before the error bar fits inside the gap.

```
import numpy as np
import matplotlib.pyplot as plt

p1, p2 = 0.70, 0.72          # two systems' true accuracies
gap = abs(p2 - p1)
N = np.arange(100, 30001, 100)
p = (p1 + p2) / 2           # pooled accuracy for the standard error
half = 1.96 * np.sqrt(p * (1 - p) / N)  # 95% CI half-width

crossover = N[np.argmax(half < gap)]  # first N where the error bar fits
print(f"gap between systems: {gap:.3f}")
print(f"held-out size where 95% error bar < gap: {crossover} examples")

plt.plot(N, half, label="95% CI half-width")
plt.axhline(gap, color="r", ls="--", label=f"true gap = {gap:.2f}")
plt.xlabel("held-out set size N"); plt.ylabel("accuracy +/- half-width")
plt.legend(); plt.title("Below the crossover, noise swamps the signal")
plt.show()
```

The evaluation layer does not, however, scrub the training corpus. The build-time removal of test data, the deduplication and overlap filtering, belongs to the data pipeline. The evaluation layer supplies the held-out sets and the canaries, consumes the data layer's decontamination contract, and audits that the contract held. Auditing is the load-bearing word: the evaluation layer's job is to verify, after the fact, that nothing it defined as held-out made it into a mixture, and to surface it loudly when something did.

A subtler contamination path bypasses the corpus entirely, the indirect path [Figure 27.2](#) drew earlier. Synthetic data generated by a teacher model that was itself trained on a benchmark carries that benchmark's content into the student, even though the student never saw the benchmark directly. This is one reason the synthetic-data practice in [Chapter 12](#) and the held-out contract here are coupled: a clean corpus is not enough if the teacher was dirty.

All of this is why a single benchmark number, reported without context, is folklore rather than measurement. Contamination-aware reporting is the minimum discipline: state the decontamination contract version the score was produced under, and disclose any known leakage alongside the number, so a reader can tell a real result from a contaminated one before building on it.

CONSTRAINT ARROW

Whether an evaluation number means anything is decided one layer down. The build-time decontamination in [Chapter 4](#), the removal of test data from the training corpus, is what makes a held-out score a measurement at all. The evaluation layer can define held-out sets, supply canaries, and audit the result, but it cannot clean the corpus itself. If the data layer leaks the sets the evaluation layer defined, every downstream number is silently inflated and no amount of statistical care upstream of the leak can recover it.

The integrity of the upper layer's measurement is dictated by the discipline of the lower layer's pipeline.

Further reading

- Hendrycks et al., "Measuring Massive Multitask Language Understanding" (MMLU), 2020. [arXiv:2009.03300](#)
- Liang et al., "Holistic Evaluation of Language Models" (HELM), 2022. [arXiv:2211.09110](#)
- Chen et al., "Evaluating Large Language Models Trained on Code" (HumanEval), 2021. [arXiv:2107.03374](#)
- Cobbe et al., "Training Verifiers to Solve Math Word Problems" (GSM8K), 2021. [arXiv:2110.14168](#)
- Hendrycks et al., "Measuring Mathematical Problem Solving with the MATH Dataset," 2021. [arXiv:2103.03874](#)
- Rein et al., "GPQA: A Graduate-Level Google-Proof Q&A Benchmark," 2023. [arXiv:2311.12022](#)
- Jimenez et al., "SWE-bench: Can Language Models Resolve Real-World GitHub Issues?," 2023 (ICLR 2024). [arXiv:2310.06770](#)
- Hsieh et al., "RULER: What's the Real Context Size of Your Long-Context Language Models?," 2024 (COLM 2024). [arXiv:2404.06654](#)
- Mialon et al., "GAIA: a benchmark for General AI Assistants," 2023 (ICLR 2024). [arXiv:2311.12983](#)
- Yao et al., " τ -bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains," 2024. [arXiv:2406.12045](#)
- Zhou et al., "WebArena: A Realistic Web Environment for Building Autonomous Agents," 2023 (ICLR 2024). [arXiv:2307.13854](#)
- Chiang et al., "Chatbot Arena: An Open Platform for Evaluating LLMs by Human Preference," 2024 (ICML 2024). [arXiv:2403.04132](#)
- Shi et al., "Detecting Pretraining Data from Large Language Models," 2023 (ICLR 2024). [arXiv:2310.16789](#)
- Xie et al., "OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments," 2024 (NeurIPS 2024). [arXiv:2404.07972](#)
- OpenAI, "BrowseComp: A Simple Yet Challenging Benchmark for Browsing Agents," 2025. [arXiv:2504.12516](#)

28. Judging and Holistic Evaluation

The benchmarks in [Chapter 27](#) score what has a checkable answer. Open-ended quality, helpfulness, tone, whether one response is better than another, has no answer key, so the field reaches for a strong model as the grader and for human preference as the ground truth. By the end of this chapter a reader can explain why a model-as-judge is biased and how to correct for it, how arena-style preference ranking turns pairwise votes into a single number, and why a private test set is worth more than any public leaderboard.

When the answer key runs out

Most of what a deployed model produces cannot be checked by a test runner. A summary, an explanation, a rewrite, a chat turn: each has many acceptable forms and no reference string to match against. Exact-match and execution graders, the workhorses of [Chapter 27](#), are silent here. The early answer borrowed reference-overlap metrics from machine translation, scoring an output by how much it shared with a human-written reference, but on open-ended text that overlap correlates poorly with judgment, and there is rarely a single reference to overlap with in the first place. That leaves two instruments, both expensive: a human reading every output, or a strong model standing in for that human.

Both are biased, and the bias is not random noise that averages out. It is structured, it correlates with the very properties a training run will optimize, and so it bakes itself into the model if you grade against it without correction. This is the thread that runs through the whole chapter. The instrument that grades open-ended output is, when frozen and reused, the instrument that trains the model, so a flaw in measurement becomes a flaw in behavior.

A second problem compounds the first. Any public benchmark, the moment it is published, begins leaking into training corpora and becomes a target to tune toward ([Shi et al. 2023](#)). A number that everyone can see is a number everyone optimizes, and Goodhart's law takes it from there: the measure stops measuring. So the question is not only how to grade open-ended output, but how to hold a measurement that stays honest after the field has seen it.

The chapter answers with three instruments built on top of each other: a model that grades, a ranking that aggregates many grades, and a held-out set that keeps the whole apparatus honest. Holistic evaluation in the sense of HELM, scoring models across many scenarios and metrics at once rather than chasing a single headline number, is the frame these three sit inside ([Liang et al. 2022](#)).

The grader is the thing it grades

The core idea is to use a strong language model as the grader, the model-as-judge, and to treat its output as a proxy validated against human or executable ground truth, never as ground truth itself. The judge is cheap, fast, and scalable in a way a human panel is not, which is what makes it usable inside a training loop. It became viable once frontier models were strong enough that their preferences tracked human preferences closely. [Zheng et al.](#) made the case directly: a strong judge agrees with human raters at roughly the rate two humans agree with each other, which is the bar that makes the judge usable as a stand-in ([Zheng et al. 2023](#)).

That agreement rate is the optimistic half of the story. The pessimistic half is that the judge is a biased instrument, and the design is mostly about characterizing and correcting that bias. Worse, the bias does not stay confined to measurement. The constraint arrow at the end of this chapter traces why: the same pairwise protocol that scores a leaderboard also gathers the preference data that trains a reward model in [Chapter 10](#), and a reward model is a frozen model-as-judge. A verbosity-biased judge that inflates a leaderboard becomes, distilled into a

reward model, a verbosity-biased reward that shapes behavior. The instrument grades and trains, so getting its bias right matters twice.

Four biases and their countermeasures

Four biases recur, and each maps to a property a model can learn to exploit (Zheng et al. 2023):

- **Verbosity.** Longer answers score higher, independent of whether the extra length adds anything. A judge optimized against naively rewards padding. Figure 28.1 makes the trap concrete: with content quality held fixed, a fair judge would report a flat coin-flip win rate, yet the biased judge's win rate climbs with length, and the whole gap is bias rather than merit.
- **Position.** When the judge compares two answers labeled A and B, the order changes the verdict. The same pair, swapped, can flip the winner (Wang et al. 2023).
- **Self-preference.** A judge favors outputs from its own model family, so grading a model with a sibling judge inflates its score.
- **Format sensitivity.** Markdown, headers, a confident closing line: surface form moves the score apart from substance.

The mitigations attack each bias directly. For position, swap the order and average the two verdicts, or require agreement across both orderings before calling a winner; Wang et al. quantified how far a single swap can move a verdict, which is what makes the swap-and-average protocol standard (Wang et al. 2023). For verbosity, normalize for length or instruct the judge to discount it. For self-preference, use a judge from a different family than the model under test, or an ensemble. Across all of them, prefer pairwise comparison over an absolute 1-to-10 score: humans and judges both rank two things more reliably than they place one thing on an abstract scale, and a pairwise protocol sidesteps the question of what a "7" means. Anchor the judge with an explicit rubric rather than a vague "which is better," and calibrate the whole apparatus against human labels on a sample. Figure 28.2 lines up each bias with the property a model can exploit and the mitigation that closes it.

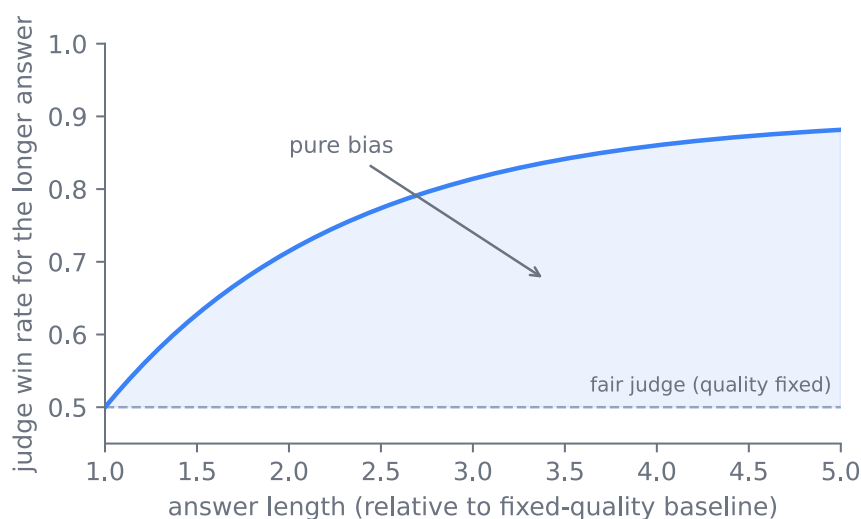


Figure 28.1. Schematic illustration of verbosity bias: judge-assigned win rate rising with answer length at fixed content quality, after Zheng et al. (2023). The dashed line is the flat coin-flip rate a fair judge should report when quality does not change, so the entire shaded gap is bias rather than merit.

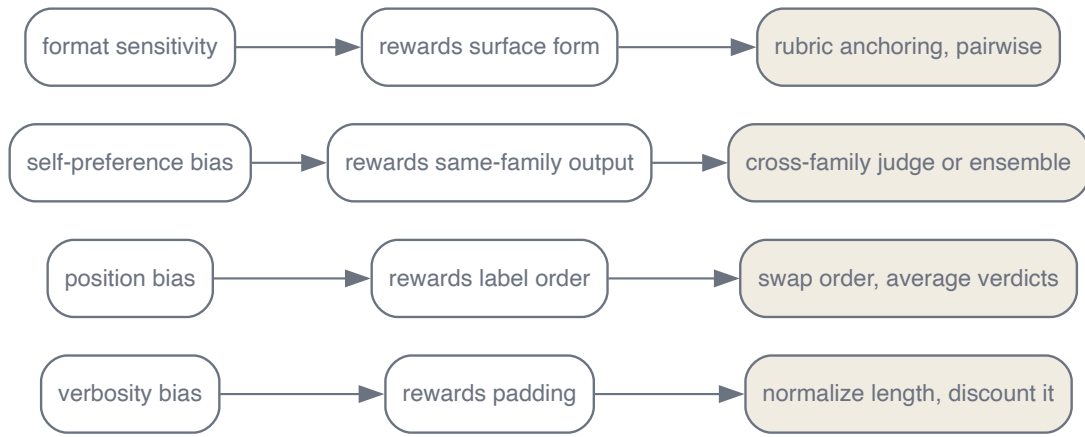


Figure 28.2. Each judge bias maps to a property a model can learn to exploit and to the mitigation that closes it. Format bias has no dedicated mitigation in this chapter; rubric anchoring and pairwise comparison are the cross-cutting defenses.

Two refinements come from practice (Anthropic 2026). Give the judge an **escape hatch**: let it return "Unknown" when it lacks the evidence to decide, rather than forcing it to manufacture a justification it does not have. And **grade each rubric dimension in isolation**, with its own judge call, rather than asking one call to score correctness, tone, and completeness at once, since a single call lets a strong impression on one axis bleed into the others.

From one verdict to a ranking

A single pairwise verdict tells you which of two responses won on one prompt. Arena-style preference ranking scales that to a global ordering over many models. Collect many pairwise comparisons, each a vote that response A beat response B on the same prompt, then fit a rating that explains the votes. The fitting model is the one chess uses, an Elo or Bradley-Terry latent strength per model (Elo 1978; Bradley and Terry 1952), so that the probability model i beats model j is

$$P(i \succ j) = \frac{1}{1 + 10^{(r_j - r_i)/400}}$$

with r_i the latent rating. Figure 28.3 plots this curve: an equal-rating pair is a coin flip, and the 400-point scale fixes the slope so a 400-point gap predicts roughly a 91 percent win. Maximum-likelihood fitting over all recorded pairwise outcomes recovers a single ranking from a sparse, noisy set of head-to-head votes.

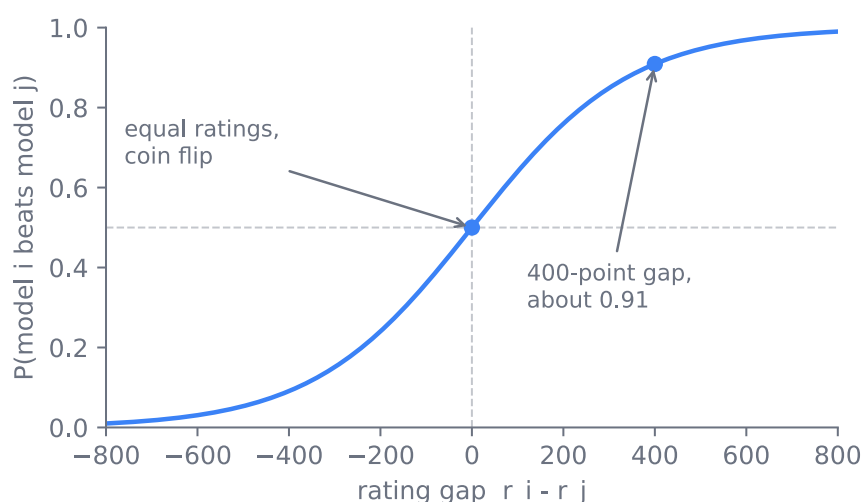


Figure 28.3. Schematic illustration of the Bradley-Terry/Elo win-probability curve, after Elo (1978) and Bradley-Terry (1952). The probability that model i beats model j is a logistic function of the rating gap, with the 400-point Elo scale setting the slope.

The votes can come from humans or from a model-as-judge, and the same machinery serves both. Chatbot Arena took this to open scale: a public platform collecting human pairwise votes in the wild and publishing a live Elo leaderboard, which became the field's default answer to "which model do people actually prefer," precisely because its inputs are fresh and hard to game by training on a fixed set (Chiang et al. 2024). MT-Bench, by contrast, drives the same ranking machinery from a model-as-judge rather than a crowd (Zheng et al. 2023).

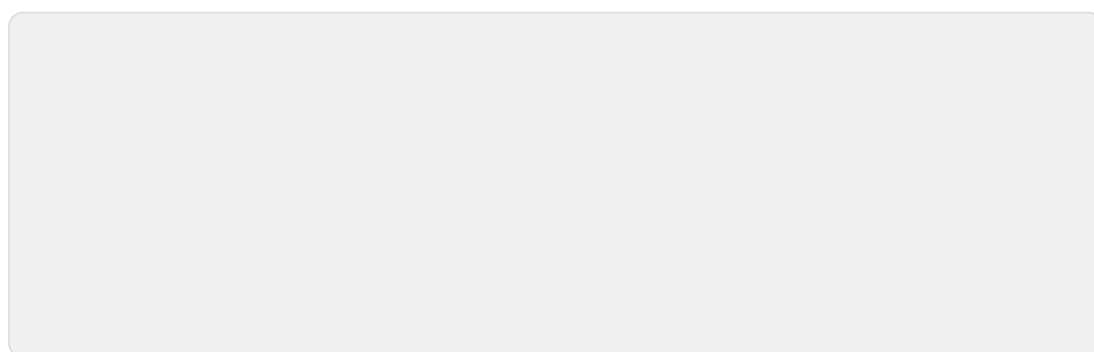


Figure 28.4. Arena-style ranking: each prompt yields two responses, a position-swapped judge casts one pairwise vote, and a Bradley-Terry or Elo fit over many votes recovers a single ranking.

The number nobody has seen

The third instrument keeps the other two honest. A **private test set** is a held-out set you own, that has never appeared in any training mixture, version-pinned, access-controlled, rotated the moment it leaks, and large enough to give a usable signal. A public benchmark you must assume is contaminated over time, and the tooling to detect that contamination after the fact exists precisely because it is so common (Shi et al. 2023). A private held-out set is trustworthy only while its integrity holds, and that integrity is an operational property you maintain, not a one-time guarantee.

The historical arc runs away from the public leaderboard, against the direction the field's incentives push. As each published benchmark saturated and contaminated, practitioners learned that the number that survives is the one nobody outside the team has seen. The lesson

generalizes the held-out principle from a single train/test split to a discipline: the trustworthy measurement is the fresh, owned, rotated one.

Choosing where to stand

The three instruments leave several axes on which a team must pick a point, and each choice trades something concrete:

- **Judge cost versus reliability.** A stronger judge is more reliable per sample and more expensive per sample; a cheaper judge needs more samples and more aggressive bias correction. Pairwise plus position-swap roughly doubles the judge calls but materially improves reliability. Choose the point on this curve by what the verdict gates: a release decision warrants the expensive protocol, a quick iteration loop may not.
- **Judge versus human.** The judge scales and reproduces; the human is the ground truth that the judge is validated against, but is slow, costly, and itself noisy enough that two annotators disagree (Zheng et al. 2023). The working compromise is to gate continuously on the judge and calibrate it periodically against a human sample, never letting the judge drift unanchored.
- **Pairwise versus absolute.** Pairwise comparison is more reliable and sidesteps the meaning of an absolute score, but it gives a ranking, not a calibrated number, and the number of comparisons grows with the number of models. Absolute scoring is cheaper to aggregate but more exposed to every bias above.
- **Public versus private sets.** A public benchmark is cheap, comparable across labs, and reproducible, but saturates and contaminates. A private set resists gaming and stays honest but costs effort to build and maintain, cannot be cited for cross-lab comparison, and decays the instant it leaks.

WHAT'S CONTESTED

Whether a model-as-judge is a sound instrument at all is genuinely unsettled. The optimistic position, grounded in the human-agreement rates of Zheng et al., is that a well-calibrated judge with swap-and-average and rubric anchoring is good enough to gate real decisions (Zheng et al. 2023). The skeptical position is that self-preference and verbosity bias are not fully correctable, that a judge sharing pretraining data with the model under test cannot be neutral, and that optimizing against a judge launders the judge's blind spots into the model. The split also runs through arena ranking: human-vote arenas resist contamination but mix in confounds like answer length and style, and whether the resulting Elo measures capability or a popularity of presentation is debated (Chiang et al. 2024). The honest stance is that the judge is a useful proxy whose agreement with humans must be measured per task, not assumed.

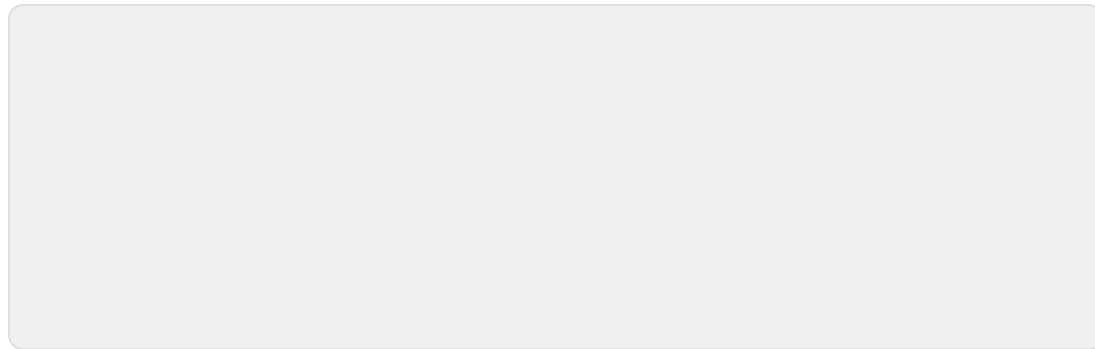


Figure 28.5. One biased instrument has two outputs. The same pairwise judging protocol scores a leaderboard and, distilled into a reward model, shapes the model's behavior, so the judge's blind spots become both a measurement error and a training signal.

CONSTRAINT ARROW

The post-training stage reaches up and sets the judge's job. The preference data that trains a reward model in [Chapter 10](#) is gathered with exactly the pairwise, position-swapped, rubric-anchored protocol described here, and a reward model is itself a frozen model-as-judge. So the biases catalogued in this chapter are not only an evaluation concern: a verbosity-biased judge that scores a leaderboard is, when distilled into a reward model, a verbosity-biased reward that shapes the model's behavior. The same instrument grades and trains, which is why getting its bias right matters twice, as [Figure 28.5](#) traces.

Building the judge

A judge call is a prompt-engineering problem with a scoring contract on top. The pairwise template presents the prompt and two responses, fixes the rubric, asks for a per-dimension verdict and a winner, and is run twice with the responses swapped; a wrapper averages the two and records both raw verdicts for audit. Keep the rubric explicit and the dimensions separate, and pin the judge model version, since a judge upgrade silently rebases every score and breaks comparability with past runs exactly the way a benchmark version change does.

The ranking fit is small. Given a table of pairwise outcomes, fit a Bradley-Terry model by maximum likelihood, which is a few lines of logistic regression over the win and loss indicators ([Bradley and Terry 1952](#)), and report the rating with a bootstrap confidence interval over resampled votes. A rating with no interval is folklore: with a sparse vote matrix the uncertainty on a single model's rank can span several places, and two models inside each other's intervals are not separated by the data.

Run this to watch the fit recover four hidden ratings from nothing but simulated head-to-head votes, and confirm the 400-point gap gives the roughly 91 percent win rate the curve promises.

```

import numpy as np
import matplotlib.pyplot as plt
rng = np.random.default_rng(0)
true_r = np.array([1500., 1400., 1300., 1200.]) # hidden latent r
p_win = lambda a, b: 1/(1 + 10**((b - a)/400)) # Bradley-Terry /
n, games = len(true_r), 300
wins = np.array([[0 if i == j else rng.binomial(games, p_win(true_r[
    for j in range(n)] for i in range(n)], float)
r, s = np.zeros(n), 400/np.log(10) # recover ratings
for _ in range(500):
    P = 1/(1 + np.exp(np.clip((r[None, :] - r[:, None])/s, -30, 30))
    r += 0.5*((wins*(1 - P)).sum(1) - (wins.T*P).sum(1)); r -= r.me
r += true_r.mean()
print("400-point gap win prob:", round(p_win(400, 0), 3))
print("recovered:", np.round(r).astype(int), " true:", true_r.astype
plt.bar(np.arange(n) - .2, true_r, .4, label="true")
plt.bar(np.arange(n) + .2, r, .4, label="recovered from votes")
plt.ylim(1100, 1560); plt.ylabel("rating"); plt.xlabel("model"); plt

```

The private set is an asset with an operational contract. It must never enter a training mixture, so it carries canary strings that let you later detect if it leaked (Shi et al. 2023), it is version-pinned and access-controlled, and it is rotated when a leak is found. Build it early and from real failures, the bug tracker and the support queue, since twenty to fifty tasks mined from things the model actually got wrong are worth more than hundreds of synthetic ones, and an eval is harder to reverse-engineer the longer you wait to write it (Anthropic 2026). Whatever the score, do not take it at face value until someone has read the transcripts: a judge that quietly rejects valid answers, or a contaminated number, ships as real signal only because nobody looked.

Further reading

- Zheng et al., "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena," 2023. [arXiv:2306.05685](https://arxiv.org/abs/2306.05685)
- Wang et al., "Large Language Models are not Fair Evaluators," 2023 (ACL 2024). [arXiv:2305.17926](https://arxiv.org/abs/2305.17926)
- Chiang et al., "Chatbot Arena: An Open Platform for Evaluating LLMs by Human Preference," 2024 (ICML 2024). [arXiv:2403.04132](https://arxiv.org/abs/2403.04132)
- Liang et al., "Holistic Evaluation of Language Models" (HELM), 2022. [arXiv:2211.09110](https://arxiv.org/abs/2211.09110)
- Shi et al., "Detecting Pretraining Data from Large Language Models," 2023 (ICLR 2024). [arXiv:2310.16789](https://arxiv.org/abs/2310.16789)
- Anthropic, "Demystifying evals for AI agents," 2026 (engineering blog). anthropic.com

29. Evaluating Agents and Capabilities

The benchmarks of [Chapter 27](#) score a single answer and the judges of [Chapter 28](#) score a single response. An agent produces neither: it produces a trajectory of tool calls and environment changes, and the thing under test is not the model alone but the model wrapped in a harness. By the end of this chapter a reader can explain why an agent score is a property of model-plus-harness, why a multi-step trajectory has to be graded on its outcome rather than its path, and why the verification layer that decides whether an agent succeeded should be built to disagree with the agent rather than to agree with it. One principle runs underneath all three: an independent, checkable signal beats a self-report. It governs how you grade the agent's work, and then, one level up, how you grade the thing that grades the agent.

The refund that never happened

Picture the simplest possible failure. An agent is told to process a refund. Its transcript ends with the line "refund processed," confident and clean. The database has no such row. Nothing was refunded. An evaluation that reads the transcript marks this a success; an evaluation that reads the database marks it a failure. The gap between those two readings is the whole subject of this chapter.

A static benchmark never has this gap, because it gives the model an input and checks one output. An agent does work: it calls tools, edits files, queries a database, browses, and the unit that has to be judged is the whole sequence and the state it leaves behind ([Anthropic 2026](#)). Three things break the static recipe.

The first is that the subject is no longer the model. When you evaluate an agent, the runtime around the weights is under test alongside them: the orchestration loop, the tool dispatch, the context management, everything [Chapter 23](#) owns. A capability number is a property of model-plus-harness, which is why a harness change can move it with the weights held fixed, and why a training-time agentic eval has to pin the harness version or it stops being comparable across checkpoints.

The second is that mistakes compound. In a single-shot benchmark an error is one wrong answer. In a trajectory an early wrong tool call changes the environment, and every later step reasons from the corrupted state, so failures propagate and a small slip becomes a wrecked run. Scoring only the final string misses where and why the run went wrong, and scoring each step in isolation misses that a capable agent recovers from its own mistakes.

The third is the refund itself: the agent can misreport what it did, not from malice but because the transcript is the model's account while the environment is the ground truth. The transcript may say "refund processed" while the database shows no such row. An eval that trusts the agent's narration measures storytelling, not work.

The agent benchmarks now in use all run into these three problems. GAIA targets general assistants ([Mialon et al. 2023](#)), WebArena ([Zhou et al. 2023](#)) and τ -bench ([Yao et al. 2024](#)) test tool-agent-user interaction, and OSWorld ([Xie et al. 2024](#)) and BrowseComp ([Wei et al. 2025](#)) test computer-use and browsing. The taxonomy of benchmark families belongs to [Chapter 27](#), and what matters here is what these share: the answer key is a final environment state, not a string, so each one lives or dies on outcome verification rather than on string matching.

Grading the work: state, not story

The core move is to grade the outcome, not the path. Verify the actual final state of the environment, the row in the database, the file on disk, the URL the browser reached, and do not score the agent against a specific sequence of tool calls you expected it to make. Rigid path checking is brittle for a precise reason: a capable agent finds valid routes the eval author did not anticipate, and a frontier model that exploits a real policy loophole has found a better solution that a path-checking grader marks as a failure. Outcome grading is robust to creativity because it asks the only question that matters, whether the world is in the state the task required.

Verify the outcome, not the agent's claim about the outcome. The grader reads the environment, never the transcript's assertion that the work was done (Figure 29.1). Tool-call verification still has a place, but a narrow one: assert genuine preconditions, that `verify_identity` ran before `process_refund`, rather than scripting the whole route.

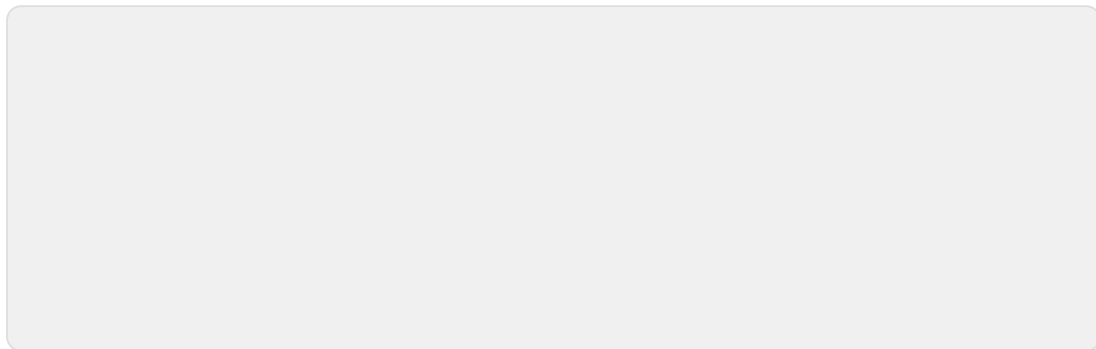


Figure 29.1. An agent leaves two artifacts: a transcript that narrates the work, and a final environment state that is the work. The grader reads the environment, treats the transcript's claim as untrusted, and checks genuine preconditions in the tool log rather than scripting a path.

The grader differs by agent type. For coding agents it runs the test suite and inspects the final tree. For conversational agents it scores the dialogue against a rubric, checks the resulting state, and simulates users with personas under a turn cap (Yao et al. 2024). For research agents it checks groundedness, coverage, and source quality. For computer-use agents it reads backend, filesystem, and URL state rather than the confirmation screen the agent says it reached.

Because the unit is a trajectory, every trial needs an isolated, sandboxed environment that starts from a clean state. This is not hygiene for its own sake. Shared state both inflates scores, when a later trial benefits from an earlier one's leftover files, and produces correlated failures, when resource exhaustion or a prior trial's git history makes many trials fail together in a way that looks like agent behavior but is infrastructure flakiness. Two task-design rules keep the target honest. Multi-component tasks deserve partial credit, because an agent that identifies the problem and verifies the customer but fails the refund is better than one that fails immediately, and a binary grader hides that continuum. And the suite must be balanced, testing both where a behavior should fire and where it should not, because a search tool tested only on should-search cases optimizes toward over-triggering.

One measurement choice falls out of the trajectory being stochastic: a single run is not a score, a distribution is. Two summaries of that distribution tell opposite stories, and which one you report encodes the stake. pass@k , the chance of at least one success in k attempts, rises with k and is the right metric for a capability ceiling and for reinforcement-learning exploration in Chapter 14. pass^k , the chance that all k trials succeed, falls with k and is the right metric for

a customer-facing agent that must be reliable every time. They are equal at $k = 1$ and tell opposite stories by $k = 10$, so naming which one a score reports is not optional.

Set a per-trial success probability `p` and watch the two metrics start equal and diverge as `k` grows.

```
import numpy as np, matplotlib.pyplot as plt

p = 0.6                                # per-trial success probability, try 0
k = np.arange(1, 11)                  # number of attempts
pass_at_k = 1 - (1 - p)**k             # at least one success in k tries
pass_pow_k = p**k                      # all k tries succeed

plt.figure(figsize=(5, 3))
plt.plot(k, pass_at_k, "o-", label="pass@k (at least one)")
plt.plot(k, pass_pow_k, "s-", label="pass^k (all succeed)")
plt.xlabel("k attempts"); plt.ylabel("probability"); plt.ylim(0, 1)
plt.legend(); plt.tight_layout(); plt.show()

print(f"k=1:  pass@k={pass_at_k[0]:.3f}  pass^k={pass_pow_k[0]:.3f}")
print(f"k=10: pass@k={pass_at_k[-1]:.3f}  pass^k={pass_pow_k[-1]:.3f}")
```

All of this raises a sharper question. If the grader for an open-ended agent task is itself a model, who verifies the verifier? This is where the design of agent evaluation meets the design of agent systems, because the answer is the same bet in both places: an independent, checkable signal beats a self-graded one. That bet has a name and a body of theory.

Who verifies the verifier?

The first instinct for making a stochastic agent reliable was to run several and aggregate. Vote across samples, ensemble across model families, add a judge, and treat agreement as evidence of correctness. The intuition came from places where it held: statistical estimation, classical fault-tolerant computing, machine-learning ensembles trained on independently sampled data. None of those assumptions hold for agents driven by the same base model. Their failures are correlated. They co-hallucinate, share blind spots, and lean toward the same fluent-but-wrong answers on the same inputs, so a third copy does not buy the fault diversity that Byzantine fault tolerance promises. Berdoz et al. measured this directly: valid scalar consensus among language-model agents drops from 46.6% at $N = 4$ to 33.3% at $N = 16$ with no adversary present (Berdoz et al. 2026). Adding agents made it worse. The Free-MAD work adds a second failure: in consensus-driven debate, agents that start with correct answers can be pulled toward incorrect ones, and majority voting can degrade reasoning (Cui et al. 2025). And self-evaluation is biased at the root, because a model-judge favors its own generations (Panickssery et al. 2024) and pairwise judges carry position bias (Shi et al. 2025), so a panel of judges from one family inherits one family's blind spots.

What distributed systems leaves behind

The forty-year-old distributed-systems literature still contributes two things that survive the move to stochastic agents. One is the separation of liveness from safety: a system can fail by producing a wrong answer or by producing none, and the empirical result above is that agent-consensus failures are mostly liveness failures (Berdoz et al. 2026). The agents do not converge on a wrong answer, they fail to converge at all. That distinction is load-bearing for design, because a liveness failure is recoverable by escalation to a human while a safety failure ships silently. The other is that correlated failure is exactly how a fault bound becomes false in practice, and the fix is not more replicas but breaking the correlation with heterogeneous models, prompts, and tooling.

Disagreement as signal

The deeper move is to stop treating disagreement as noise. In an idealized debate a single competent honest critic is enough to expose a flaw no matter how many other critics are lazy or wrong. This is the intuition behind Irving, Christiano, and Amodei (Irving et al. 2018), formalized by Brown-Cohen, Irving, and Piliouras, who show soundness under a large compute asymmetry: the honest strategy uses polynomial simulation while the dishonest one may use exponentially more, and a judge that inspects only the single disputed leaf can adjudicate claims it could never have produced unaided (Brown-Cohen et al. 2023). Verification of an agent's work is therefore closer to proof checking, property testing, and red-team review than to ensemble prediction. The verifier does not reproduce the artifact. It forces a disputed claim down to a small witness, a failing test, a vulnerable input, a violated invariant, a missing citation, that a cheaper judge can inspect. This is the same shape as grading an agent's outcome rather than its path: in both cases an independent checkable signal replaces a self-report.

Where the choices bite

The choices in agent evaluation diverge as the trajectory gets longer.

- **Outcome versus path.** Outcome grading verifies the final state and is robust to a capable agent's creativity, but it is blind to a lucky-but-fragile success that reached the right state for the wrong reason. Path grading catches how an agent failed but penalizes valid routes and scripts the eval author's assumptions into the grader. Default to outcome, add narrow path checks only for true preconditions.
- **pass@k versus pass^k.** Because trials vary, report a distributional metric, and the choice of which one encodes the stake. **pass@k** rises with k and measures a capability ceiling and reinforcement-learning exploration; **pass^k** falls with k and measures customer-facing reliability. They are equal at $k = 1$ and tell opposite stories by $k = 10$, so naming which one a score reports is not optional.
- **Aggregation versus adversarial verification.** Aggregating N agents is cheap and simple, but under correlated failure it buys little and can suppress the one signal worth keeping. Adversarial verification needs heterogeneous critics and a protocol, which costs integration complexity and a coordination tax, but it is the only thing that restores useful fault diversity when the agents share a base model. The multi-agent systems of [Chapter 24](#) inherit this choice directly.
- **Liveness escalation versus safety blocking.** A system that distinguishes "the agents could not converge" from "the agents converged on a wrong answer" can route the first to a human and refuse to ship the second. A system that conflates them does the wrong thing on both.

WHAT'S CONTESTED

Whether reliability for stochastic agents comes from aggregation or from adversarial verification is unsettled. The aggregation position, vote across samples and ensemble across families, is still the dominant pattern and is defended on simplicity and on the cases where inputs are unambiguous enough that any decoder agrees. The adversarial position holds that agreement among similar models is weak evidence of correctness, sometimes evidence of nothing more than an easy input, and that the diagnostically useful signal is the surviving objection of a competent critic, not the size of the agreeing majority. The open questions are concrete and unresolved: the right number of critics per attack surface, where the diminishing-returns curve sits across model families, how to stop an adversarial protocol from drifting off the original claim, and how to evaluate adversarial verification itself against aggregation on a careful benchmark.

CONSTRAINT ARROW

The verification layer's architecture is dictated one layer down, by where the agents come from. Two agents drawn from the same base model, the same training distribution, and often the same prompt template do not have independent failure modes, so the fault-diversity assumption that aggregation relies on is already false before the first vote is counted. That lower-layer fact, shared weights imply correlated failure, is what forces the upper layer to be adversarial and heterogeneous rather than aggregational: a verifier that is a cheaper echo of the layer it verifies cannot tell you anything the layer did not already believe. The choice between voting and cross-examination is not a matter of taste at the evaluation layer. It is set by the provenance of the models being evaluated.

agon: adversarial review, shipped

The adversarial design has a shipped instance in `agon`, a tool that runs post-hoc adversarial review on a coding session by forking it into independent critics. The protocol inverts the aggregation default. Instead of running N agents and taking the majority, it runs a proposer, forks independent critics with no shared context, has each critic pick a distinct attack topic, runs cross-examination until the proposer concedes or the critic gives up, and surfaces only the unresolved disputes scored by how long they survived attack. Five properties carry the design:

- **Forked review, never in the root session.** Each critic runs in its own branched fork via `-fork-session`, and the tool writes results to disk as a separate process, so the user's primary transcript is byte-identical after a run.
- **Verbatim channel.** The critic's output reaches the proposer clone as a plain user turn, not wrapped in a template that would distort the proposer's normal defense behavior.
- **Self-declared topics.** Each critic picks its own attack topic and later critics are told which topics are taken, so coverage spreads. The debate property that one competent honest critic suffices means a lazy critic on one topic does not break the others.
- **Persisted ledger.** Every attack carries a stable id and every transition is appended to a log. The headline is chosen by a pure contention score, not by a model: $\text{score}(a) = \text{rounds_survived}(a) + \mathbf{1}[\text{re_attacked}(a)]$. The judging layer is statistical, not neural, which is the doubly-efficient-debate intuition made concrete: the judge is cheaper and dumber than the proposer (Brown-Cohen et al. 2023).
- **Critic isolation.** The critic sees the artifact and task only, enforced by system prompt and a read-only sandbox.

The asymmetry between the two designs (Figure 29.2) is the whole argument. Aggregation places every voter inside one shared blind spot, so a flaw in that region survives no matter how

many voters you add. Adversarial verification spreads critics across distinct axes, so a flaw on any covered axis is caught by the one critic who sees it.

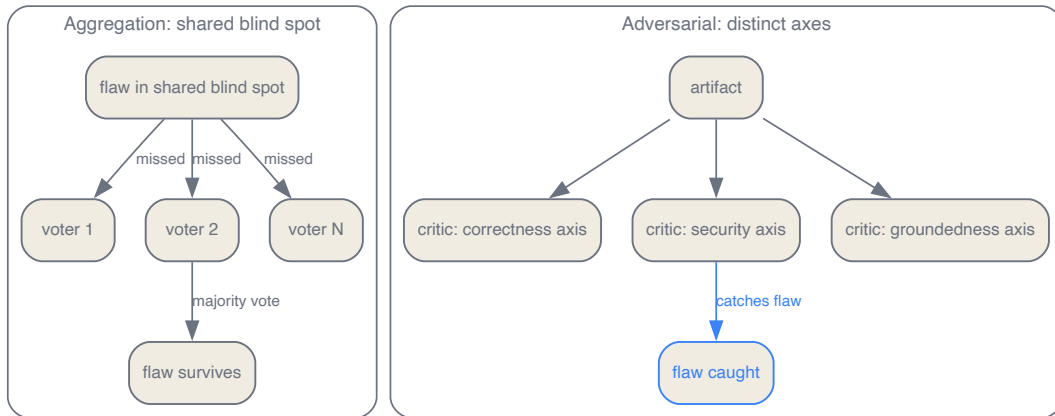


Figure 29.2. Why adding agents helps adversarial verification but not aggregation. Voters drawn from one base model share a blind spot, so a flaw inside it passes every vote. Heterogeneous critics on distinct axes each cover a different failure surface, so one competent critic catches a flaw the rest miss.

A small formal contrast explains why this dominates aggregation when failures correlate. Let $p_i \in [0, 1]$ be the probability that critic i catches a flaw on the axis it evaluates. With k independent critics on distinct axes, where one caught flaw suffices, the probability a flaw survives the panel factors over critics:

$$\Pr[\text{flaw survives}] = \prod_{i=1}^k (1 - p_i)$$

Any residual positive correlation between verdicts only raises survival above this product, so the product is the best case the panel buys, and it collapses to zero the moment one competent critic on the relevant axis has $p_i = 1$. Aggregation tells the opposite story. With k voters under majority vote and pairwise verdict correlation ρ , as $\rho \rightarrow 1$ the chance a flaw survives the ensemble approaches the chance that the single shared failure mode misses it, and adding voters does nothing:

$$\Pr[\text{flaw survives} \mid \rho \rightarrow 1] \rightarrow \Pr[\text{shared mode misses flaw}]$$

The product only pays if the p_i cover different ways the artifact can be wrong, which is why forcing topic diversity, rather than piling critics on one surface, is how the protocol spends its budget.

Capability versus safety

This is the same bet Chapter 28 makes about judges and the same one Chapter 14 makes about rewards: a signal you can check independently beats one the system grades on itself. It is also where capability evaluation and safety evaluation part ways. Capability asks "can it," and a capability eval should start at a low pass rate or there is nothing to learn from it. Safety asks "will it refuse, will it harm," needs adversarial and red-team coverage rather than held-out accuracy, and gates release on its own axis, because a capability gain can be a safety regression and the two pull against a shared budget. Adversarial verification is the technical

form of the same stance: the artifact that ships to the human is the short list of disputes that competent independent critics could not get the agent to either fix or convincingly defend, and that is the set of decisions that actually need human judgment.

Further reading

- Anthropic, "Demystifying evals for AI agents," 2026 (engineering blog). anthropic.com
- Mialon et al., "GAIA: a benchmark for General AI Assistants," 2023 (ICLR 2024). [arXiv:2311.12983](https://arxiv.org/abs/2311.12983)
- Yao et al., " τ -bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains," 2024. [arXiv:2406.12045](https://arxiv.org/abs/2406.12045)
- Zhou et al., "WebArena: A Realistic Web Environment for Building Autonomous Agents," 2023 (ICLR 2024). [arXiv:2307.13854](https://arxiv.org/abs/2307.13854)
- Xie et al., "OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments," 2024 (NeurIPS 2024). [arXiv:2404.07972](https://arxiv.org/abs/2404.07972)
- OpenAI, "BrowseComp: A Simple Yet Challenging Benchmark for Browsing Agents," 2025. [arXiv:2504.12516](https://arxiv.org/abs/2504.12516)
- Berdoz et al., "Can AI Agents Agree?," 2026. [arXiv:2603.01213](https://arxiv.org/abs/2603.01213)
- Cui et al., "Free-MAD: Consensus-Free Multi-Agent Debate," 2025. [arXiv:2509.11035](https://arxiv.org/abs/2509.11035)
- Panickssery et al., "LLM Evaluators Recognize and Favor Their Own Generations," 2024. [arXiv:2404.13076](https://arxiv.org/abs/2404.13076)
- Shi et al., "Judging the Judges: A Systematic Study of Position Bias in LLM-as-a-Judge," 2025 (IJCNLP-AAACL). [ACL 2025](https://arxiv.org/abs/2505.00000)
- Irving, Christiano & Amodei, "AI safety via debate," 2018. [arXiv:1805.00899](https://arxiv.org/abs/1805.00899)
- Brown-Cohen, Irving & Piliouras, "Scalable AI Safety via Doubly-Efficient Debate," 2023. [arXiv:2311.14125](https://arxiv.org/abs/2311.14125)

30. Accelerators and Networking

A training run lives or dies on bandwidth, not peak FLOPs. By the end of this chapter a reader can explain what each accelerator family and interconnect tier offers, why a tensor-parallel group almost never spills past the NVLink boundary, and how model FLOPs utilization measures whether the hardware floor is being used.

A run that the spec sheet cannot explain

Picture a frontier run spread across thousands of accelerators for weeks. The spec sheet promises a peak arithmetic rate, and the run delivers a fraction of it. The missing factor is rarely the matrix-multiply units sitting idle for want of work. It is the bytes those units wait on: how fast a chip reaches its own memory, how fast two chips in a chassis talk, how fast two chassis across the hall talk. Those three numbers differ by more than an order of magnitude each step out, and a parallelism axis that needs more talking than its tier can afford stalls the whole collective.

So the spec sheet's peak FLOPs is the wrong place to start. The right place is the hierarchy of bandwidth tiers underneath it, because every layout decision is a question of which tier a given exchange can afford. This chapter owns that hardware floor: the accelerators, the interconnect hierarchy, and the single number, MFU, that says whether the floor is being used. The parallelism algorithms that ride on top belong to [Chapter 8](#); cluster orchestration, checkpointing, and the data plane belong to [Chapter 31](#).

Three tiers of bandwidth

The machine is a hierarchy of bandwidth tiers, and tracing a byte from the die outward shows why each tier exists and what it can host.

The innermost tier is a chip and its memory. An accelerator pairs a large matrix-multiply throughput with high-bandwidth memory (HBM) sitting next to the die. The forward and backward matmuls are fed from HBM, so HBM bandwidth, not raw FLOPs, often sets the achievable rate for a kernel.

Drag the ceiling to see why a kernel that does little work per byte fetched stays bandwidth-bound and never reaches the peak FLOPs the spec sheet promises.

A kernel ramps with how much arithmetic it does per byte it pulls from HBM, then flattens once it hits the hardware's peak FLOPs ceiling.

The next tier is intra-node: the few accelerators inside one chassis. On NVIDIA hardware these are wired with NVLink, a dedicated chip-to-chip fabric, and NVSwitch, which lets every accelerator in the node reach every other at full NVLink bandwidth rather than over the slower PCIe bus. This tier is where the most communication-hungry axis can live, because NVLink bandwidth is high enough to hide a collective behind compute. Whether a collective hides at all is a ratio, as [Figure 30.1](#) sketches: as long as a step does more compute than communication the collective vanishes entirely under it, and the inter-node network falls short precisely because its bytes are dear enough to leave some communication exposed.

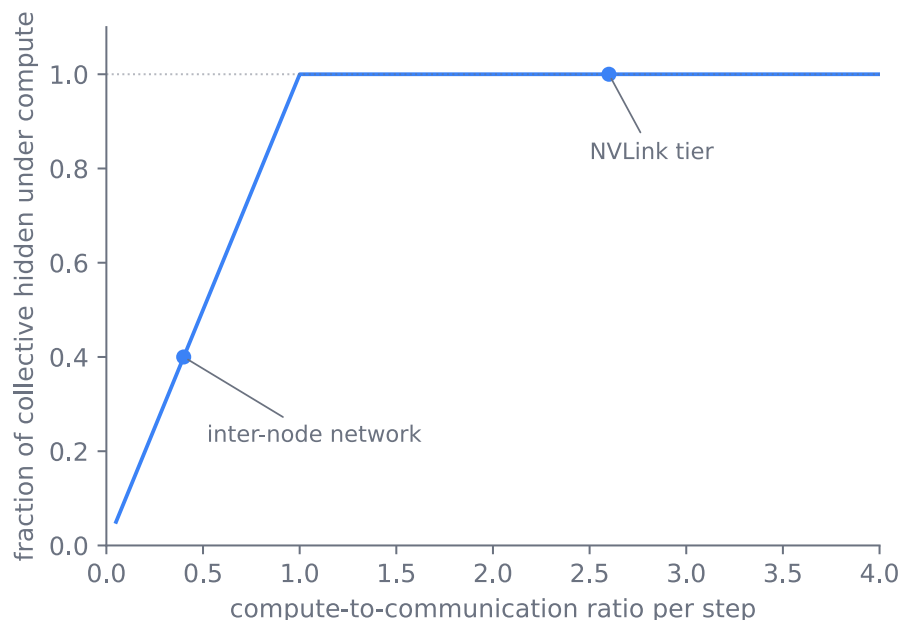


Figure 30.1. Schematic of how much of a collective hides under compute as a function of the compute-to-communication ratio, after Shoeybi et al. (2019) and Narayanan et al. (2021). The NVLink tier sits where the ratio is high and the collective vanishes entirely; the inter-node network sits where bytes are dearer, so some communication stays exposed.

The outermost tier is inter-node: chassis talking to chassis across a datacenter network. Here the fabric is InfiniBand or RoCE (RDMA over Converged Ethernet), wired in a topology such as a fat-tree or a rail-optimized layout, with a network interface card (NIC) per accelerator setting the per-device injection bandwidth. This tier is an order of magnitude slower per byte than NVLink, so only axes that communicate sparingly can afford to span it.

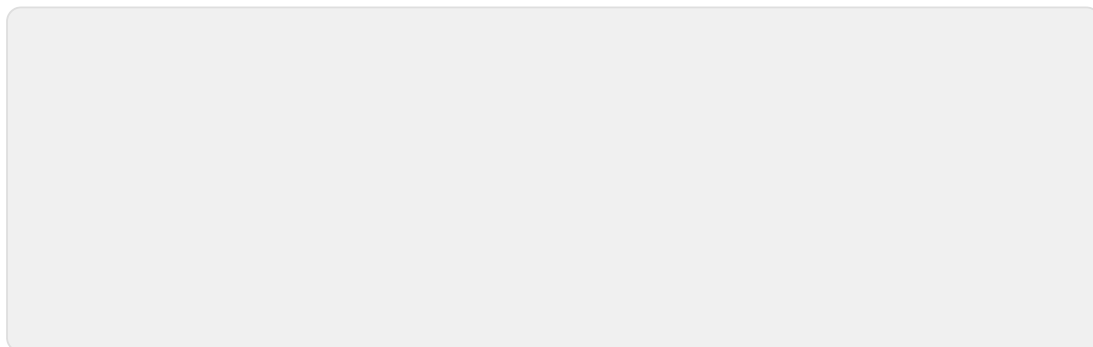


Figure 30.2. The three bandwidth tiers of a training cluster. Each step outward, from HBM to NVLink to the inter-node network, drops roughly an order of magnitude in bandwidth per byte.

Mapping parallelism onto the tiers

Once the tiers are fixed, the placement of each parallelism axis follows directly, and it follows from communication volume, not from taste. Tensor parallelism (TP) shards a single matmul across devices and exchanges activations with an all-reduce twice per layer, so it is the most bandwidth-hungry axis and is pinned to the NVLink tier inside a node (Shoeybi et al. 2019). Data and pipeline parallelism communicate far less per step, a gradient reduction once per step for DP and a single activation hand-off per stage boundary for PP, so they are the axes allowed to cross the inter-node network (Narayanan et al. 2021). Expert parallelism for MoE rides its own all-to-all group, treated in [Chapter 7](#). The rule of thumb that follows, TP inside the node

and DP/PP across nodes, is not a convention but a consequence of the bandwidth gap, as Figure 30.3 lays out.

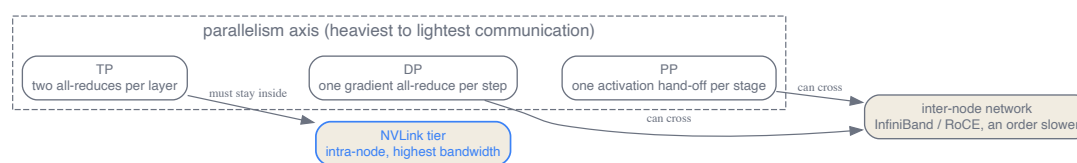


Figure 30.3. Parallelism axes ordered by per-step communication volume, each landing on the cheapest tier that can still hide its collective under compute. The bandwidth gap, not convention, assigns TP to NVLink and DP/PP to the inter-node network.

The TPU detour: a torus instead of a switch

TPU pods are the JAX/XLA counterpart with a different cost model, and they make the point that the tier hierarchy above is one design choice rather than a law of nature. The chips connect through the inter-chip interconnect (ICI) and an optical circuit switch (OCS) into a torus topology rather than a switched fat-tree, so neighbor-to-neighbor bandwidth is plentiful and the sharding is co-designed with the torus shape through GSPMD-style annotations rather than mapped onto a fat-tree after the fact (Xu et al. 2021). The same byte that would cross a switch on a GPU cluster travels to a physical neighbor on a TPU pod, which is why bandwidth there scales with locality.

MFU: the one honest number

The single number that tells you whether all of this is working is model FLOPs utilization. MFU is the FLOPs the model actually needs, divided by the peak FLOPs the hardware could deliver over the same wall-clock:

$$\text{MFU} = \frac{\text{model FLOPs per step}}{\text{peak hardware FLOPs} \times \text{step time}}$$

It counts only the useful arithmetic of the model. Its sibling, hardware FLOPs utilization (HFU), also counts work the hardware genuinely did but the model did not strictly need, most notably the extra forward passes of activation recomputation, which is why HFU exceeds MFU whenever recomputation is on. MFU is the honest figure because it answers the question the budget cares about: of the arithmetic I paid for, how much moved the model.

Why the tiers look the way they do

The interconnect tiers are the product of the bandwidth gap widening faster than any single fabric could close it. Early multi-GPU training leaned on PCIe, which was already too slow for sharding a matmul, so NVLink and then NVSwitch were introduced precisely to make the intra-node tier fast enough to host tensor parallelism (Shoeybi et al. 2019). The accelerator families also moved the precision floor: each generation has widened the set of formats the matmul units execute natively, with Hopper-class and newer hardware adding fp8, which lets the same silicon deliver more matmul throughput per byte moved (Micikevicius et al. 2022) (the kernel-level consequences are in Chapter 19). On the network side, InfiniBand and RoCE converged on RDMA and on rail-optimized topologies that give each accelerator a dedicated path to its peer on other nodes, so the inter-node tier could grow without every flow contending for the same switch ports. TPU pods evolved along a parallel track entirely, choosing a torus over a switched fabric so that bandwidth scales with locality.

WHAT'S CONTESTED

The intra-node versus inter-node boundary that this chapter is built on is itself moving. The scale-up domain, the set of accelerators that can reach each other at NVLink-class bandwidth, has been expanding past a single chassis toward rack-scale fabrics. As that domain grows, the maximum affordable tensor-parallel degree grows with it, and the long-standing rule that TP stays inside one small node weakens. Whether the future is ever-larger scale-up domains that let TP span a rack, or a continued reliance on the cheaper inter-node network with parallelism axes that tolerate it, is unsettled and vendor-dependent. The boundary is a property of this year's hardware, not a law.

What each axis costs

Every parallelism axis buys a different resource and pays in a different currency, and the interconnect tier sets the exchange rate.

- **Tensor parallelism against bandwidth.** TP cuts per-device memory and latency but spends intra-node bandwidth on two all-reduces per layer. That spend is affordable only on NVLink, so the TP degree caps at the NVLink domain. Push it past the node and the all-reduce falls onto the inter-node network, where it cannot hide under compute and shows up as lost MFU. Figure 30.4 makes the cliff visible: MFU holds a plateau while TP fits inside the NVLink domain, then falls away the moment the degree spills onto the inter-node network.

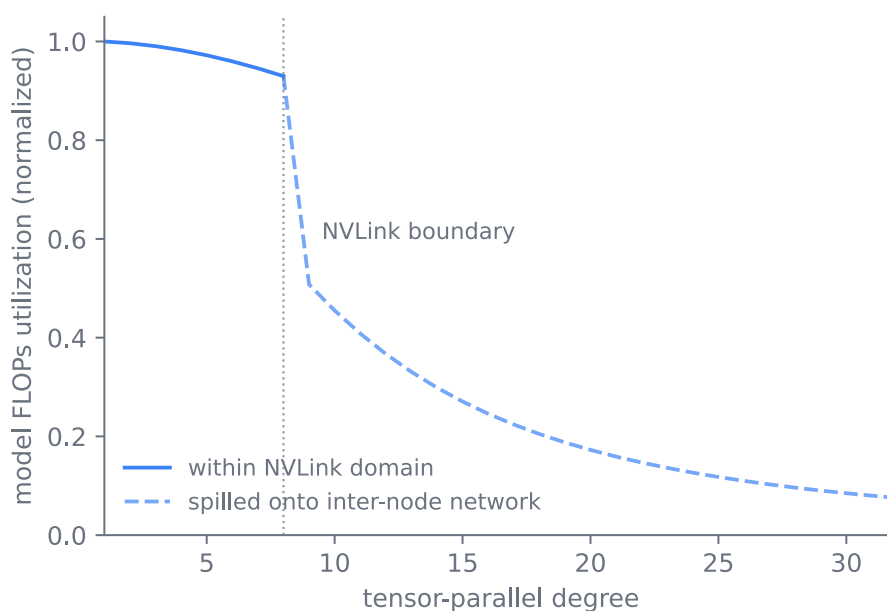


Figure 30.4. Schematic of model FLOPs utilization as the tensor-parallel degree grows, after Shoeybi et al. (2019) and Narayanan et al. (2021). MFU holds a plateau while TP fits inside the NVLink domain, then falls off a cliff once the all-reduce spills onto the inter-node network an order of magnitude slower per byte.

Change `nvlink_size` and `slowdown` and watch where the plateau breaks and how steep the cliff becomes.

```

import numpy as np
import matplotlib.pyplot as plt

nvlink_size = 8          # accelerators reachable at NVLink bandwidth
peak = 0.55              # MFU plateau when communication hides under c
slowdown = 10.0          # inter-node bytes are this many times dearer
tp = np.arange(1, 33)
# fraction of the step lost to exposed (un-hidden) all-reduce once T
comm = np.where(tp <= nvlink_size, 0.0, (tp - nvlink_size) / tp * (1
mfu = peak * (1 - comm)
plt.plot(tp, mfu, marker='o')
plt.axvline(nvlink_size, ls='--', color='gray', label='NVLink bounda
plt.xlabel('tensor-parallel degree'); plt.ylabel('MFU')
plt.ylim(0, 0.6); plt.legend(); plt.title('TP past the node falls of
print('MFU at TP=8:', round(mfu[7], 3), ' at TP=16:', round(mfu[15],
plt.show()

```

- **Pipeline parallelism against the bubble.** PP is bandwidth-cheap, a single activation hand-off per stage boundary, so it crosses nodes comfortably, but it pays a pipeline bubble at the fill and drain of each step. - **MFU against memory.** Recomputation lets a bigger model or longer sequence fit by redoing forward work in the backward pass, which raises HFU but lowers MFU. The number you optimize depends on whether memory or throughput is the binding constraint.

CONSTRAINT ARROW

This is the clearest case in the book of a lower layer dictating an upper one. The interconnect bandwidth gap, NVLink an order of magnitude faster per byte than InfiniBand or RoCE, is what sets the maximum tensor-parallel degree that [Chapter 8](#) can choose. A parallelism layout is not picked on the whiteboard and then mapped to hardware. It is the hardware tier that decides which axis is allowed where, and TP staying inside the NVLink boundary is the direct imprint of that decision on the algorithm above it.

Reading the timeline

The collectives that move bytes across these tiers are provided by NCCL on NVIDIA hardware (RCCL on AMD), which maps each collective onto the topology, choosing a ring or a tree and routing it over NVLink or the network depending on where the ranks sit ([NVIDIA 2024](#)). A practitioner rarely calls it directly; the parallelism framework does. What the practitioner reads is the per-step profile, because MFU is diagnosed from a timeline, not from the loss curve. Exposed communication that did not hide under compute, an oversized pipeline bubble, too much recomputation, and unfused kernels each take a slice of the gap between achieved and peak FLOPs, and the timeline is where each slice is visible.

[Figure 30.5](#) reads the timeline as a descent from peak FLOPs down to MFU. Idle time, where a device waited on exposed communication or sat in a pipeline bubble, is pure loss. Of the FLOPs the hardware then actually ran, recomputation did work the model did not strictly need: that work lifts HFU above MFU, but only the useful arithmetic counts toward the honest figure.

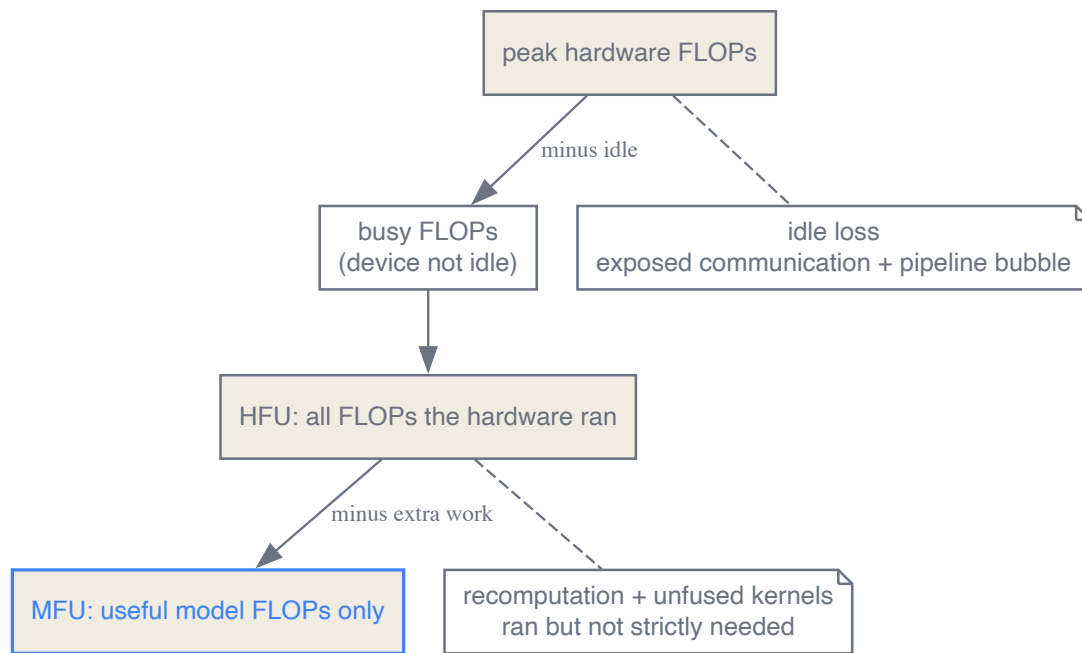


Figure 30.5. Reading the per-step timeline as a descent from peak FLOPs to MFU. Exposed communication and the pipeline bubble are idle loss; recomputation and unfused kernels are work the hardware ran, which is why HFU sits above MFU.

Further reading

- Shoeybi et al., "Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism," 2019. [arXiv:1909.08053](https://arxiv.org/abs/1909.08053)
- Narayanan et al., "Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM," 2021 (SC'21; PTD-P 3D parallelism). [arXiv:2104.04473](https://arxiv.org/abs/2104.04473)
- Micikevicius et al., "FP8 Formats for Deep Learning," 2022. [arXiv:2209.05433](https://arxiv.org/abs/2209.05433)
- NCCL (NVIDIA Collective Communications Library): optimized inter-GPU collective primitives (all-reduce, all-gather, reduce-scatter, all-to-all) over NVLink/PCIe/InfiniBand. Engineering library, not a single canonical paper. [NVIDIA/nccl](https://nvidia.com/en-us/nccl/)
- Xu et al., "GSPMD: General and Scalable Parallelization for ML Computation Graphs," 2021 (XLA/TPU sharding annotations underlying JAX `pjit`). [arXiv:2105.04663](https://arxiv.org/abs/2105.04663)

31. Orchestration and Data Infrastructure

A frontier run holds thousands of accelerators in lockstep for weeks, and the hardware underneath it fails on a schedule of its own. The compute work, the matmuls and the collectives, is the part that looks hard, but it is not the part that decides whether the run finishes. At the scale of thousands of nodes for weeks, failure is the steady state, not the exception: a GPU falls off the bus, a NIC flaps, a host OOMs, and because the collectives are gang-scheduled, a single dead participant stalls the entire job. This chapter owns the operational plane that keeps such a run alive and honest while the hardware under it breaks.

By the end a reader can explain why checkpoint cadence is an economic decision, why the data sampler's state belongs in the checkpoint, and why a single dead node can cost a day. The accelerators and interconnect this plane runs on are [Chapter 30](#); the parallelism layout it checkpoints and feeds is [Chapter 8](#). This chapter owns neither. It owns what happens between steps and across failures.

Three obligations fall out of treating failure as the steady state, and they organize the rest of the chapter. The run must be *recoverable*, resuming from a recent saved state rather than from scratch when a node dies. The data-loading plane must keep the accelerators *fed*, delivering tokens at the rate they consume them and doing so reproducibly. And the run must be *observable* enough that a fault is caught from telemetry rather than from a quietly wrong loss curve days later. All three are operational, all three are unglamorous, and all three gate whether the expensive compute work pays off. We take them in turn, then return to the scheduler and the cost knobs that sit under all three.

Surviving failure: the checkpoint and the restart

The core mechanism against failure is the checkpoint: periodically serialize model parameters, optimizer state, the learning-rate schedule position, and the data sampler state, so the run can be reconstructed from that snapshot. The optimizer state dominates the write, since AdamW carries two moment estimates per parameter, which is the same state [Chapter 8](#) shards across the data-parallel group ([Rajbhandari et al. 2019](#)).

A naive checkpoint stops every rank, writes to storage, and resumes, putting the full write latency on the critical path. This was the obvious first design: pause, write everything to a shared filesystem, resume. It was tolerable when models fit comfortably and clusters were small, but it scaled badly, because both the state to write and the failure rate grow with the run, so the synchronous write went from a rounding error to a visible tax. Two ideas remove the write from the critical path. *Asynchronous* checkpointing copies the tensors to host memory or a staging buffer and lets training proceed while a background writer drains them to storage; CheckFreq is the clean statement of this fix, pipelining the snapshot and the write so the overhead is low enough that a short cadence becomes affordable, turning checkpointing from a coarse, costly event into a routine one ([Mohan et al. 2021](#)). *Sharded* or *distributed* checkpointing has each rank write only its own shard of the state in parallel, so write time scales with per-rank state rather than total state; this followed the parallelism layout of fully sharded data parallel, where the state is already partitioned across the cluster ([Zhao et al. 2023](#)).

Cadence is an economic decision

The cadence itself is an optimization, not a constant. Checkpoint too rarely and a failure rolls back a lot of work; checkpoint too often and the writes themselves tax storage bandwidth and steal time. The optimum balances the expected work lost per failure against the cost of a write, which is why a cluster with a lower mean time between failures wants a shorter cadence. A useful first cut: if failures arrive at rate λ and a checkpoint costs C to write, the interval T

that minimizes total wasted time scales like $T \approx \sqrt{2C/\lambda}$, the familiar square-root tradeoff between save cost and replay cost. Frequent checkpoints shrink the work lost per failure but tax storage bandwidth and can stall the run; rare checkpoints are cheap to write but expensive when a failure hits. Asynchronous and sharded checkpointing are what make a short cadence affordable in the first place, and the optimum depends on cluster MTBF and write cost, not on a fixed number of steps.

Pick a write cost and a failure rate, then watch the numeric optimum fall out of the square-root formula and land at the bottom of the U-shaped waste curve.

```
import numpy as np
import matplotlib.pyplot as plt

C = 30.0          # checkpoint write cost, seconds
mtbf = 6 * 3600   # mean time between failures, seconds
lam = 1.0 / mtbf  # failure rate

T = np.linspace(60, 36000, 2000)          # checkpoint interval, seconds
waste = C / T + lam * T / 2                # write overhead + expected
T_star_numeric = T[np.argmin(waste)]
T_star_formula = np.sqrt(2 * C / lam)
print("numeric optimum T*:", round(T_star_numeric), "s")
print("formula sqrt(2C/lambda):", round(T_star_formula), "s")

plt.plot(T / 60, waste, label="total waste rate")
plt.axvline(T_star_formula / 60, color="C1", ls="--", label="T* = sq")
plt.xlabel("checkpoint interval T (minutes)")
plt.ylabel("wasted time fraction")
plt.title("U-shaped waste versus cadence")
plt.legend(); plt.grid(alpha=0.3); plt.show()
```

CONSTRAINT ARROW

Checkpoint cadence is dictated from below. The mean time between failures of the accelerators and interconnect in Chapter 30 sets how often the run will be knocked over, and that failure rate, not any property of the model or the training recipe, is what fixes the economical save interval. A cluster with flakier hardware must checkpoint more often, which raises storage bandwidth requirements and lowers effective throughput. An operational choice that looks like a tuning knob is set by the reliability of the layer beneath it.

Recovery, and the faults that do not crash

Recovery is the other half. Elastic restart detects the dead node, fences it out of the collective, and resumes the survivors from the last checkpoint, ideally onto pre-warmed spare capacity so the job does not wait for a human or for the failed host to come back. The harder faults are the ones that do not crash cleanly: a *straggler* node that runs slow drags every collective down to its pace, and *silent data corruption* lets the run continue while producing subtly wrong

numbers, which no checkpoint will save you from because the checkpoint faithfully records the corruption.

The split in [Figure 31.1](#) is the design point: a clean crash travels the fast path of detect, fence, and resume on a pre-warmed spare, while the faults that never trip the crash detector are the ones that have to be caught by observability or are beyond recovery entirely.

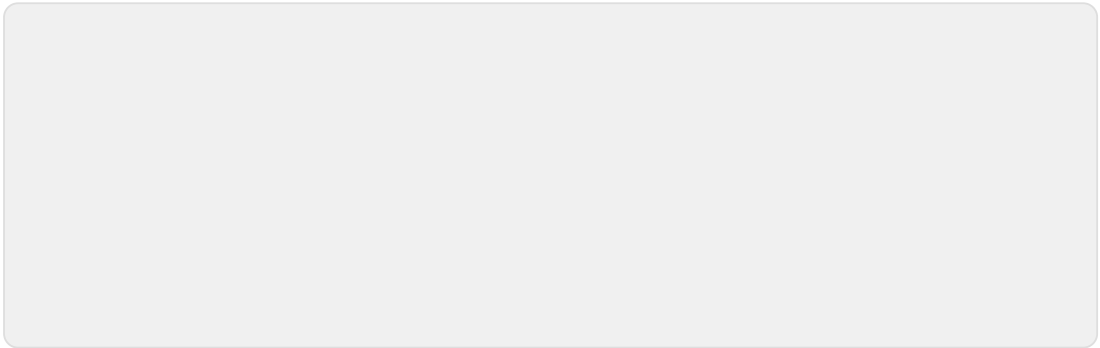
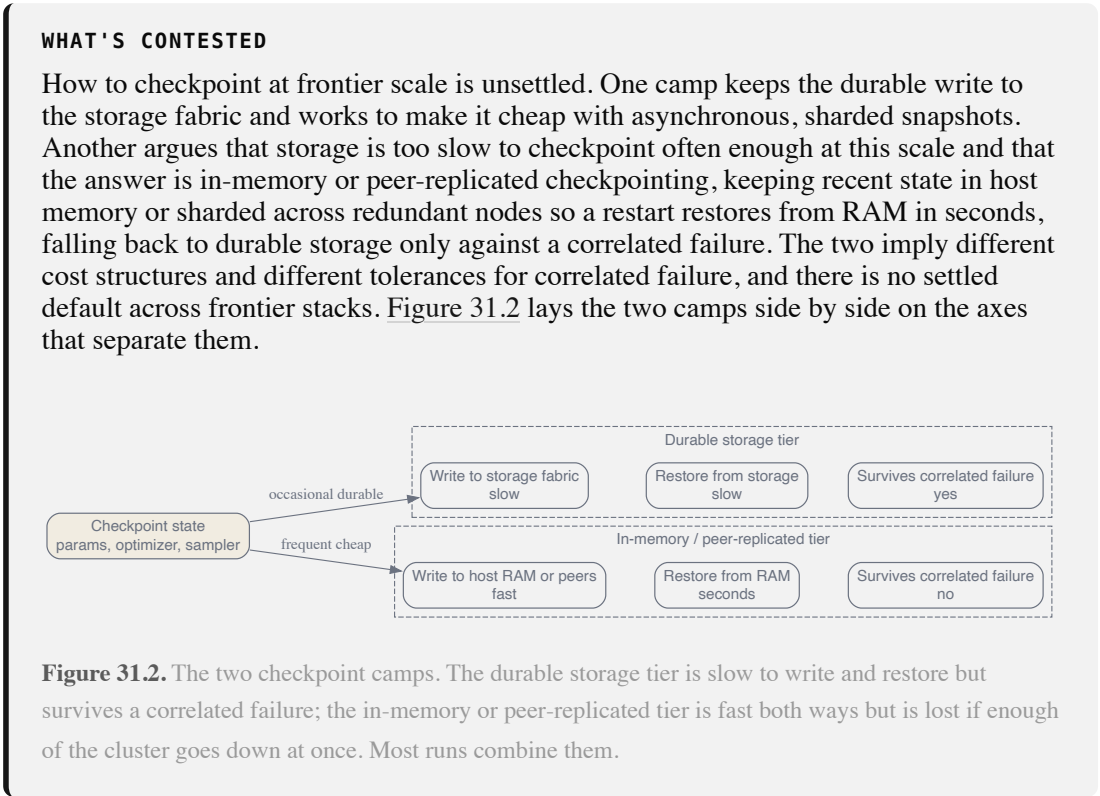


Figure 31.1. A clean crash is caught and routed to elastic restart on spare capacity. The faults that do not crash cleanly escape that path and must be caught by observability or, in the case of silent corruption, are unrecoverable from any checkpoint.

Where the checkpoint lives, and what is still contested

Writing checkpoints to the storage fabric is durable and survives a cluster-wide event but is slow; staging in host memory or replicating across peer nodes is fast to write and fast to restore but is lost if enough of the cluster goes down at once. The choice trades durability against checkpoint cost, and most runs combine a frequent cheap tier with an occasional durable one. Holding warm spare nodes lets an elastic restart resume in seconds instead of minutes, but those nodes are paid for and idle until a failure uses them: the reservation is insurance whose premium is utilization.



Feeding the accelerators: the data plane

The second pillar keeps the accelerators fed. The corpus is tokenized into many shards on a high-throughput storage fabric, a parallel filesystem or NVMe-over-Fabrics tier sized to feed the cluster at multiple TB/s, usually with a local NVMe cache in front of it. Each rank reads its assigned shards, prefetches and pipelines the next batch while the current one trains so the accelerators never wait on I/O, and a global shuffle keeps any single shard's local correlations from biasing a step. The load order obeys the mixture weights from [Chapter 4](#), which set how often each source is drawn.

This plane evolved from map-style datasets that index a corpus held in memory to streaming, sharded pipelines that never materialize the whole corpus, because a multi-trillion-token corpus does not fit and cannot be randomly indexed cheaply.

Reproducibility ties the data plane back to the checkpoint

The property that ties the data plane back to checkpointing is reproducibility. A resume must continue the exact data order the run would have followed without the failure, which means the sampler and shuffle state, the position in the permutation, the epoch, the per-source draw counters, are part of the checkpoint, not derived fresh on restart, as [Figure 31.3](#) shows. Without that, a resume re-feeds or skips data and silently breaks the mixture contract.

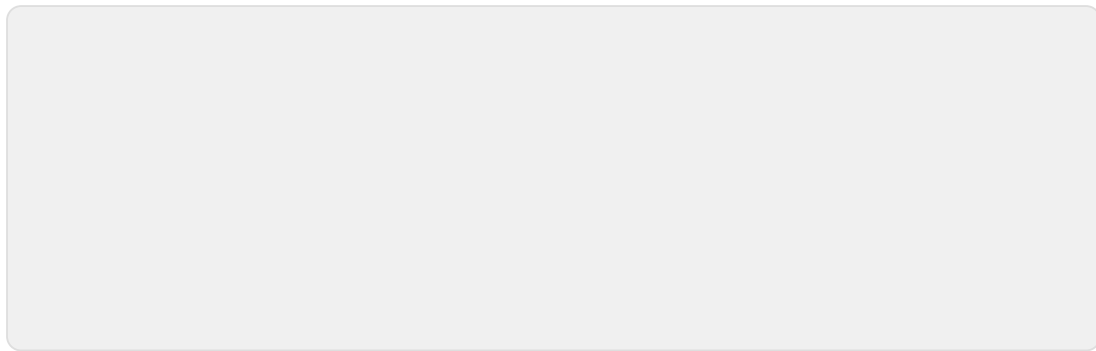


Figure 31.3. The data plane feeds each rank from sharded storage through a local cache, and the checkpoint captures the sampler state so a resume continues the exact same data order.

The reproducibility requirement arrived with scale. At small scale a non-deterministic resume is a nuisance, but at frontier scale, where resumes are frequent and the mixture contract is load-bearing, checkpointing the sampler state became non-negotiable. It is not free: exact data-order reproducibility costs determinism overhead, since the sampler state must be checkpointed and the shuffle must be replayable, which constrains how aggressively you can reorder for I/O efficiency. Giving it up buys some throughput but forfeits the ability to resume cleanly and to attribute a loss discontinuity to its cause.

Knowing the run is healthy: observability

The third pillar's design follows from one fact: the loss curve alone cannot tell you whether the systems work is correct. A run can be slow, comms-bound, or quietly degraded while the loss still falls. So the plane instruments the step itself: per-step wall time and its breakdown, exposed versus overlapped communication, and the grad-norm and loss telemetry that catch a divergence or a spike early enough to act. The diagnostic unit is a per-step timeline or profile, not a glance at the loss.

This is where the faults that escaped the crash detector get caught. A straggler shows up as a slow rank in the per-step breakdown; a divergence shows up as a grad-norm spike before the

loss visibly turns. Silent data corruption is the one class that even good observability struggles with, because the numbers are wrong but not anomalous, which is why it sits at the unrecoverable end of [Figure 31.1](#).

What ties the three together: the scheduler and the cost knobs

Cost and scheduling sit underneath all three pillars. Because the collective is gang-scheduled, the job needs all its nodes simultaneously or none of them make progress, so the scheduler reserves the gang and keeps spare capacity warm to absorb a failure without a full restart. Checkpoint cadence, spare-node provisioning, and the storage-fabric bandwidth are the operational cost knobs, and they trade directly against the throughput of the run. Each pillar has spent one of these knobs already: cadence buys recoverability, spare capacity buys fast restart, storage bandwidth buys both a durable checkpoint tier and a fed data plane. The scheduler is what lets a single reservation hold all of it at once.

How the plane is assembled

In practice the operational plane is assembled rather than bought whole. Model and optimizer sharding come from the parallelism framework, but the checkpoint, data, and observability planes are frequently custom, because their cadence, sharding, and resumability must match the exact layout of a given run.

A checkpoint at this scale is not one file. Each rank serializes its shard of parameters and optimizer state, plus the small but load-bearing scalars: the step count, the learning-rate schedule position, and the data sampler state. A sketch of the contract:

```
state = {
    "model_shard":    shard_of(model.parameters()),
    "optim_shard":    shard_of(optimizer.state),    # dominant term
    "step":           step,
    "lr_schedule_pos": scheduler.position(),
    "sampler_state":  loader.sampler.state_dict(),  # makes resume
}
async_write(state, path_for(rank, step))          # off the criti
```

The two failure modes worth naming both surface late and quietly. A node failure with no recent checkpoint and no fast detection rolls the run back to its last save and burns the gap as pure waste, which is why detection, fencing, and a recent checkpoint are one system, not three. A non-reproducible data order on restart, from a sampler state that was not checkpointed, re-feeds or skips data and breaks the mixture contract from [Chapter 4](#); it shows up as an unexplained loss discontinuity at every resume and is easy to misread as a training-recipe problem when it is an operational bug. Both are caught by the observability the plane installs, and both are prevented by treating the sampler state and the recovery path as first-class parts of the checkpoint.

Further reading

- Mohan et al., "CheckFreq: Frequent, Fine-Grained DNN Checkpointing," 2021 (USENIX FAST'21; asynchronous, low-overhead checkpointing). usenix.org/conference/fast21/presentation/mohan
- Zhao et al., "PyTorch FSDP: Experiences on Scaling Fully Sharded Data Parallel," 2023 (VLDB; sharded state that distributed checkpointing persists). [arXiv:2304.11277](https://arxiv.org/abs/2304.11277)
- Rajbhandari et al., "ZeRO: Memory Optimizations Toward Training Trillion Parameter Models," 2019 (sharded optimizer state, the dominant checkpoint term). [arXiv:1910.02054](https://arxiv.org/abs/1910.02054)

32. Mechanistic Interpretability

A trained transformer is a few hundred billion numbers that produce behavior no one wrote down. Mechanistic interpretability is the attempt to read those numbers as computation: to find the features a model represents and the circuits that connect them, the way one might reverse engineer a binary into source. By the end of this chapter a reader can explain why a single neuron rarely means one thing, why sparse dictionaries became the field's main tool for pulling features apart, and why the question of whether that tool is the right one is still open.

The neuron that means everything

Open a transformer and look for the obvious place to start. The natural unit of a neural network is the neuron, and the natural hope is that each neuron means something: this one fires on Python code, that one on the color red. Early vision work found a few neurons like that. Language models do not cooperate. Probe a neuron in a transformer and it fires on a grab bag of unrelated things, a curve detector and a car wheel and a line of French at once. This is polysemanticity, and it is the wall that stops naive interpretation. If the unit you can read does not carry a single meaning, reading it tells you nothing.

The wall is not that the model has no concepts. It is that the model has far more concepts than it has neurons, and it stores them anyway. A residual stream of width d has only d orthogonal directions, yet the model seems to track tens of thousands of distinct features. Something has to give. The features cannot each get a private direction, so they share. Everything in this chapter is a way of undoing that sharing: recovering the concepts the model uses from the activations that smear them together, then showing how those concepts feed each other to produce an output. The payoff is the input to Chapter 33: oversight that can only watch behavior is blind to a model that behaves until it does not, whereas oversight that can read features has a chance of catching a plan before it executes.

So the chapter is organized around one obstacle and the escalating attempts to defeat it. First, the two ideas that explain why the wall exists and suggest how to climb it. Then circuits, which turn recovered features into explanations of behavior. Then the decade-long arms race that carried the method from hand-traced vision filters to causal graphs of a production model. Then the open trial of whether the field's main tool is the right one. And last, an honest accounting of what the tool still leaves behind.

Two ideas that climb the wall

The escape from polysemanticity rests on two ideas, a hypothesis about how models store features and a method for recovering them.

The hypothesis is superposition. A model represents more features than it has dimensions by assigning each feature a direction in activation space and letting those directions overlap. This works because features are sparse: on any given token, only a handful of the thousands of possible features are active, so the model can tolerate interference between directions that are rarely on at the same time. Elhage et al. built the cleanest demonstration in "Toy Models of Superposition" (Elhage et al. 2022): a tiny autoencoder forced to pack n sparse features into fewer than n dimensions does exactly this, arranging the feature directions into regular geometric structures and trading reconstruction error against interference as a function of sparsity. Polysemanticity, in this picture, is not a defect. It is the visible shadow of a model using superposition to fit more features than it has neurons.

If features live along directions rather than on neurons, the right unit of analysis is a direction, and the right tool is one that finds the directions. That tool is dictionary learning, implemented as a sparse autoencoder (SAE) (Cunningham et al. 2023). An SAE takes a model activation $x \in \mathbb{R}^d$ and learns an overcomplete dictionary of $m \gg d$ feature directions, encoding x as a sparse vector of feature activations and decoding it back:

$$f = \sigma(W_{\text{enc}}x + b_{\text{enc}}), \quad \hat{x} = W_{\text{dec}}f + b_{\text{dec}}$$

trained to minimize reconstruction error plus a sparsity penalty so that only a few of the m features fire on any input. The columns of W_{dec} are the recovered feature directions. Because m exceeds d , the dictionary is overcomplete: it has room to give each underlying feature its own slot even though the model had to pack them into d crowded dimensions. The features the SAE recovers are far more often monosemantic than the raw neurons, which is the whole point.

Figure 32.1 shows the two-sided number tension that makes this work. The model compresses many sparse true features into the d shared directions of the residual stream, accepting interference between directions that rarely fire together. The SAE runs the compression backward, expanding that d -vector into $m \gg d$ slots so each feature can land in its own direction again.

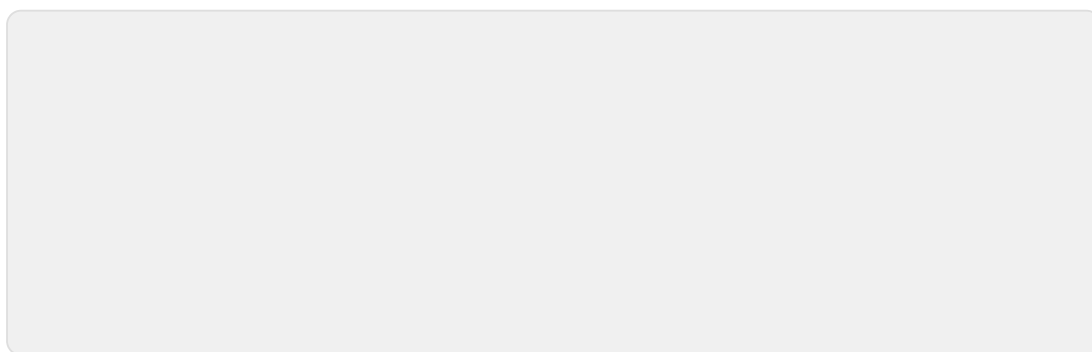


Figure 32.1. Superposition packs many sparse features into a narrow residual stream, and an overcomplete SAE expands them back into monosemantic slots. After Elhage et al. (2022) and Bricken et al. (2023).

From nouns to verbs: circuits

Features alone are nouns. To explain behavior you need verbs, the connections that carry one feature's activation into another. That is the circuit. A circuit is a subgraph of the model's computation, a small set of features and the weights between them that together implement a recognizable algorithm. The earliest worked example outside vision was the induction head, a two-attention-head circuit that implements the rule "if the token A was followed by B earlier in the context, and I just saw A again, predict B ." Olsson et al. traced this circuit and tied its formation to a phase change in training that coincides with the onset of in-context learning (Olsson et al. 2022). The agenda of the field is to do for arbitrary behavior what induction heads did for copying: name the features, name the wires, check the explanation by intervening.

Two things make that agenda hard, and they pull in opposite directions. Naming features needs a unit that means one thing, which superposition denies the neuron and dictionary learning tries to restore. Naming wires needs a model of how those units compose, which is the circuit. The history of the field is the story of these two needs forcing each other forward.

An arms race against the wall

The lineage runs from hand-traced circuits to learned features to causal graphs, each step forced by the limit of the last.

It began in vision. Olah et al., in the Distill thread "Zoom In: An Introduction to Circuits," argued that features are real and circuits connecting them are real, and showed both by hand in convolutional networks (Olah et al. 2020). Carrying this to transformers needed an algebra for the architecture. Elhage et al. supplied it in "A Mathematical Framework for Transformer Circuits" (Elhage et al. 2021): the residual stream as a shared communication channel that every layer reads from and writes to, attention heads as operations that move information between positions, and the composition rules that let heads form multi-step circuits. Induction heads were the first big result that framework explained.

Then the wall: hand-tracing does not scale, and superposition means the neurons you would trace are polysemantic anyway. "Toy Models of Superposition" named the obstacle (Elhage et al. 2022). The escape was to stop reading neurons and start learning features. Cunningham et al., "Sparse Autoencoders Find Highly Interpretable Features in Language Models" (Cunningham et al. 2023), and Anthropic's "Towards Monosemanticity" (Bricken et al. 2023) landed the same idea at the same time: train an SAE on a model's activations and the recovered dictionary is far more interpretable than the neuron basis. Bricken et al. worked on a one-layer transformer with a 512-neuron MLP and showed that dictionary features picked out crisp concepts, such as Arabic script or DNA sequences, that no single neuron cleanly held.

The open question was whether this survived scale. "Scaling Monosemanticity" carried SAEs up to Claude 3 Sonnet, a production model, training dictionaries on its middle-layer residual stream and recovering abstract, multilingual, multimodal features (Templeton et al. 2024), including the Golden Gate Bridge feature that, when clamped on, produced the "Golden Gate Claude" that steered every conversation toward the bridge. The intervention mattered more than the demo: it showed the features were causal, not just correlated. In parallel, the engineering of SAEs tightened. Gao et al., "Scaling and Evaluating Sparse Autoencoders" (Gao et al. 2024), replaced the soft L1 penalty with a top- k activation that fixes the number of live features directly, which removed a tuning headache, cut the population of dead latents, and produced clean scaling laws relating dictionary size and sparsity to reconstruction. Figure 32.2 sketches that relationship: more active latents per token buy lower reconstruction error, a larger dictionary shifts the whole frontier down, and the two ends of the sparsity axis carry the failure modes the chapter returns to later.

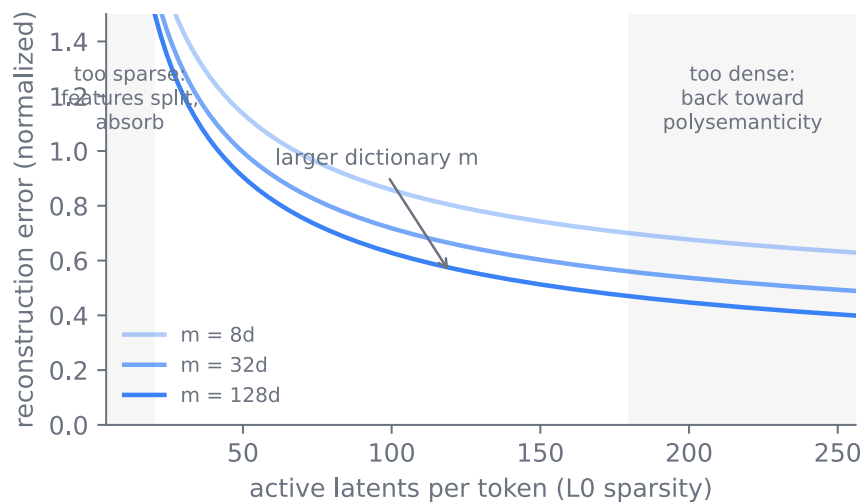


Figure 32.2. Schematic of the sparse autoencoder trade-off between reconstruction error and sparsity. More active latents per token lower the error, and a larger dictionary shifts the frontier down. The synthetic curves illustrate the relationship only and are not measured data, after Gao et al. (2024).

Sweep the top- k sparsity below and watch the reconstruction error fall, then flatten once k reaches the true number of active features, leaving a nonzero residue that no extra latent removes.

```
import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(0)
d, m, n_true = 32, 96, 6          # residual width d, dictionary slots
D = rng.standard_normal((m, d))
D /= np.linalg.norm(D, axis=1, keepdims=True)

codes = np.zeros((300, m))       # each token uses only n_true of the
for c in codes:
    idx = rng.choice(m, n_true, replace=False)
    c[idx] = rng.standard_normal(n_true)
X = codes @ D + 0.15 * rng.standard_normal((300, d))  # activations
base = np.mean(X ** 2)

ks, frac = list(range(0, 13)), []
for k in ks:
    f = codes.copy()
    if k == 0:
        f[:] = 0
    else:
        thr = np.sort(np.abs(f), axis=1)[: , -k] [: , None]
        f = np.where(np.abs(f) >= thr, f, 0.0)          # keep only t
    frac.append(np.mean((X - f @ D) ** 2) / base)

plt.plot(ks, frac, "o-")
plt.axvline(n_true, ls="--", color="gray", label=f"true sparsity = {n_true}")
plt.xlabel("k active latents per token")
plt.ylabel("fraction of variance unexplained")
plt.legend(); plt.title("Top-k reconstruction: drops, then plateaus")
plt.show()
print("unexplained at k=0,3,6,12:", [round(frac[i], 3) for i in (0, 3, 6, 12)])
```

The latest step puts the nouns and verbs back together. Anthropic's 2025 work, "Circuit Tracing" (Ameisen et al. 2025) and "On the Biology of a Large Language Model" (Lindsey et al. 2025), builds attribution graphs: it replaces a model's MLPs with cross-layer transcoders, sparse interpretable features that read from one layer and write to all later ones, then traces the linear causal pathways through those features for a single prompt. The result is a graph whose nodes are features and whose edges are causal influences, a local circuit diagram for one input. Studied on Claude 3.5 Haiku, these graphs surfaced multi-step reasoning, planning ahead in rhymes, and shared machinery across languages (Lindsey et al. 2025).

Figure 32.3 traces this progression, where each step is forced by the limit of the one before it.



Figure 32.3. Lineage of mechanistic interpretability, from hand-traced circuits through learned features to causal attribution graphs. Each arrow marks the limit that forced the next step.

How the loop runs in practice is mechanical and worth seeing in outline. Collect a large sample of activations at one site in the model, the residual stream after a chosen layer or an MLP's hidden activations. Train the dictionary on them. With a top- k encoder the forward pass is small:

```
# x: model activation, shape (batch, d)
pre = x @ W_enc + b_enc           # (batch, m), m >> d
f   = topk(pre, k)                # keep k largest, zero the rest
x_hat = f @ W_dec + b_dec         # reconstruct
loss = mse(x_hat, x)             # no separate L1 term: k fixes s
```

The fixed k removes the sparsity coefficient that the L1 formulation forces you to tune, and is the practical reason top- k SAEs displaced the older soft penalty (Gao et al. 2024). After training, each dictionary column is a candidate feature. You interpret it by gathering the inputs that activate it most, reading the pattern, and, crucially, testing the reading by clamping the feature on or off and watching the output move. A feature you cannot steer with is a feature you have not understood. Attribution graphs extend the same discipline to a whole computation: swap the MLPs for trained transcoders, run one prompt, and read off the causal graph among active features, then verify edges by perturbing nodes and confirming the downstream effect (Ameisen et al. 2025).

The trial of the sparse autoencoder

The arms race has carried the field to a production model and a causal graph, but it has not settled the central methodological question. The whole escape from the wall rests on one bet: that dictionary learning is how you read superposition. That bet is now on trial.

WHAT'S CONTESTED

Whether the sparse autoencoder is the right interpretability primitive, or an artifact-prone detour, is actively debated. The case against has several named results. Kantamneni et al., "Are Sparse Autoencoders Useful? A Case Study in Sparse Probing" (Kantamneni et al. 2025), find that on downstream probing tasks SAE features rarely beat simple baselines once those baselines are made strong, which questions the practical payoff. Chanin et al. document feature absorption, where a general feature like "starts with L" goes silent on a specific token like "lion" because a more specific feature swallowed it, so the dictionary's apparent cleanliness is partly an artifact of the objective (Chanin et al. 2024). Others argue the L1 sparsity penalty pushes SAEs to learn frequent combinations rather than the atomic features that actually mediate computation, and that transcoders outperform SAEs on interpretability (arXiv:2501.18823). The defenders, centered on Anthropic's interpretability team and the authors of the scaling and attribution work, counter that SAEs are improvable engineering, not a dead end, that causal interventions like Golden Gate Claude show the features are real, and that

attribution graphs already produce mechanistic accounts that bare probing cannot. The disagreement is not whether superposition is real. It is whether dictionary learning is how we should be reading it.

CONSTRAINT ARROW

Superposition is a fact about the architecture in Chapter 6, and it dictates the shape of every tool in this chapter. Because the residual stream packs more features than it has dimensions, no method that reads neurons directly can work, and the field is forced into overcomplete dictionary learning. A property of how the model stores information sets the entire method stack of how we read it.

What the dictionary leaves behind

Every gain in this chapter is bought with a matching cost, and naming the costs is the honest way to read the trial above.

- **Monosemanticity versus completeness.** Sparsity buys clean, single meaning features, but a dictionary recovers only the features it was sized and trained to find. Push sparsity too hard and features split or get absorbed into each other; push too soft and they go polysemantic again. The SAE reconstructs the activation imperfectly, and the gap, the reconstruction error, is exactly the part of the model's computation the dictionary failed to explain.
- **Faithfulness versus interpretability.** A transcoder or SAE that replaces part of the model is interpretable only to the degree it stays faithful to what the original computed. Attribution graphs make this trade explicit: they freeze attention and approximate MLPs, so the graph is a readable story about a model that is not quite the one you deployed.
- **Local versus global.** An attribution graph explains one prompt. A feature dictionary describes one model at one training checkpoint. Neither yet gives a global, reusable account of what the model does on inputs you have not traced, which is what oversight ultimately wants.
- **Effort versus coverage.** Reading a circuit is slow human work. Auto-interpretation, using a model to label features, scales the labor but imports the labeler's errors, and the field has no settled metric for when a feature has been correctly understood.

The failure modes are specific, and each one is a way the dictionary lies about how clean it is. Dead latents are dictionary features that never fire and learn nothing, wasting capacity. Dead and over-firing latents together push you toward top- k and careful initialization (Gao et al. 2024). Feature splitting hands you several narrow features where the model had one, and absorption hides a general feature inside a specific one, so the dictionary looks cleaner than the model is (Chanin et al. 2024). And the reconstruction error is not noise to be ignored: it is the unexplained residue, the part of the computation your dictionary did not capture, and a method that reports high interpretability while leaving large error has explained less than it claims. The wall, in the end, is never fully climbed. What the chapter offers is a measured account of how far up we are, and an honest reading of the distance left.

Further reading

- Olah et al., "Zoom In: An Introduction to Circuits," Distill, 2020. distill.pub/2020/circuits/zoom-in
- Elhage et al., "A Mathematical Framework for Transformer Circuits," 2021. transformer-circuits.pub/2021/framework

- Olsson et al., "In-context Learning and Induction Heads," 2022. transformer-circuits.pub/2022/in-context-learning-and-induction-heads
- Elhage et al., "Toy Models of Superposition," 2022. [arXiv:2209.10652](https://arxiv.org/abs/2209.10652)
- Cunningham et al., "Sparse Autoencoders Find Highly Interpretable Features in Language Models," 2023. [arXiv:2309.08600](https://arxiv.org/abs/2309.08600)
- Bricken et al., "Towards Monosemanticity: Decomposing Language Models with Dictionary Learning," 2023. transformer-circuits.pub/2023/monosemantic-features
- Templeton et al., "Scaling Monosemanticity: Extracting Interpretable Features from Claude 3 Sonnet," 2024. transformer-circuits.pub/2024/scaling-monosemanticity
- Gao et al., "Scaling and Evaluating Sparse Autoencoders," 2024. [arXiv:2406.04093](https://arxiv.org/abs/2406.04093)
- Ameisen et al., "Circuit Tracing: Revealing Computational Graphs in Language Models," 2025. transformer-circuits.pub/2025/attribution-graphs/methods
- Lindsey et al., "On the Biology of a Large Language Model," 2025. transformer-circuits.pub/2025/attribution-graphs/biology
- Kantamneni et al., "Are Sparse Autoencoders Useful? A Case Study in Sparse Probing," 2025. [arXiv:2502.16681](https://arxiv.org/abs/2502.16681)
- Chanin et al., "A is for Absorption: Studying Feature Splitting and Absorption in Sparse Autoencoders," 2024. [arXiv:2409.14507](https://arxiv.org/abs/2409.14507)

33. Scalable Oversight and Control

Alignment built on human judgment has an expiry date: once a model reasons about a task better than the human grading it, the grade stops being a reliable signal. By the end of this chapter a reader can explain the two distinct responses to that problem, getting a strong model to learn the right thing from a weaker teacher, and bounding the damage a misaligned model can do regardless of what it has learned, and can say why the open question of how oversight scales with the capability gap is the hinge the whole agenda turns on.

When the grader is outmatched

The post-training recipe in Chapter 10 rests on a human, or a reward model trained on human labels, being able to tell a better answer from a worse one. That assumption holds while the human is the more competent party. It breaks the moment the model is the more competent party. A reviewer who cannot follow a proof cannot certify it. A grader who cannot audit a codebase cannot tell a clean diff from one with a subtle backdoor. The label is noisier than the thing it supervises, and optimizing against a noisy label teaches the model to satisfy the label, not the intent behind it.

This is the scalable oversight problem: how to supervise a system whose competence exceeds the supervisor's, in the domain where the supervision is needed. What makes it more than a measurement nuisance is that the gap it turns on is not fixed. It is set by scaling. Every increment of the scaling spend in Chapter 3 widens the distance between the frontier model and the human, or weaker model, asked to oversee it. The lower layer that makes the model more capable is the same layer that makes it harder to supervise, which is why a method that works at today's gap cannot be assumed to work at the next generation's.

CONSTRAINT ARROW

The capability gap that threatens oversight is not a fixed quantity. It is set by scaling. Every increment of the scaling spend in Chapter 3 widens the distance between the frontier model and the human, or weaker model, asked to oversee it. The lower layer that makes the model more capable is the same layer that makes it harder to supervise, which is why an oversight method that works at today's gap cannot be assumed to work at the next generation's.

Two philosophies for one problem

The scalable oversight problem splits into two questions that the field has learned to keep separate, and they pull in opposite directions. The first is an alignment question: can a weak signal still teach a strong model the right behavior? The second is a security question: if the model is not aligned, possibly because it is actively trying to evade detection, can a weaker overseer still prevent harm? The first asks whether the student learns; the second assumes the student might be adversarial and asks whether the institution holds anyway.

Two families of answers map onto those two questions, as [Figure 33.1](#) lays out. One family tries to make the weak teacher's signal go further, betting that you can still get the right behavior out of a stronger model. The other gives up on trusting the model and engineers the deployment so that an adversarial model still cannot do much harm. The chapter follows that fault line: first the trust camp, then the containment camp, and the threat model that has been pushing the field from the former toward the latter.

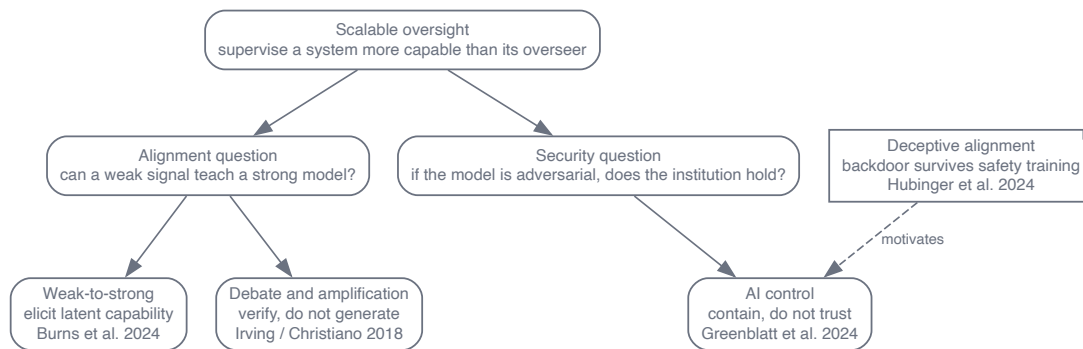


Figure 33.1. The scalable oversight problem splits into an alignment question and a security question, each answered by a different family of mechanisms. Deceptive alignment is the threat model that makes the control family necessary.

The trust camp: make the weak signal go further

The first family asks whether a weak teacher can still steer a strong model, and it attacks the problem from two sides: the learning side and the judging side.

Elicitation, not instruction

Weak-to-strong generalization, studied by Burns et al. (Burns et al. 2024), tests the limit directly: finetune a strong pretrained model on labels produced by a weak supervisor, and measure how much of the strong model's latent capability survives the weak teaching. The finding is that it generalizes beyond its teacher (Burns et al. 2024). A strong model trained on weak, error-prone labels outperforms the weak supervisor that generated them, because pretraining has already laid down the concept and weak supervision only has to elicit it, not install it. The recovery is partial, not full, and the paper reports an auxiliary confidence loss that pushes the strong student to make confident predictions even where it disagrees with the weak label, which recovers more of the gap (Burns et al. 2024). The mechanism is elicitation, not instruction, and that reframing is the design idea.

That reframing is also the technique's ceiling. Eliciting a latent capability works only when pretraining already contains it. For genuinely novel behavior the weak teacher has nothing to elicit, and the confidence loss that recovers more of the gap can also amplify a confident error. The technique narrows the supervisor-student gap; it does not close it.

Verify, do not generate

Debate and amplification attack the same problem from the judging side, on the bet that checking an answer is easier than producing one. Iterated amplification, proposed by Christiano et al. (Christiano et al. 2018), builds a training signal for a hard question by decomposing it into easier subquestions a human can supervise, answering those with model help, and composing the answers, so the supervisor never has to evaluate the whole problem at once. Debate, proposed by Irving et al. (Irving et al. 2018), sets two strong models against each other in a zero-sum game judged by a weaker party: each tries to convince the judge its answer is correct, and the structure is meant to make dishonesty hard because a lie creates an opening the opponent can expose. The bet is that verifying a rebutted argument is easier than generating the truth from scratch, the way checking a proof is easier than finding it. Khan et al. (Khan et al. 2024) gave this an empirical test with language models: non-expert judges, both models and humans, reach the truth more often when they read a debate between persuasive expert debaters than when they consult a single one, and optimizing debaters for persuasiveness raised judge accuracy rather than lowering it (Khan et al. 2024). Figure 33.2 sketches the game.

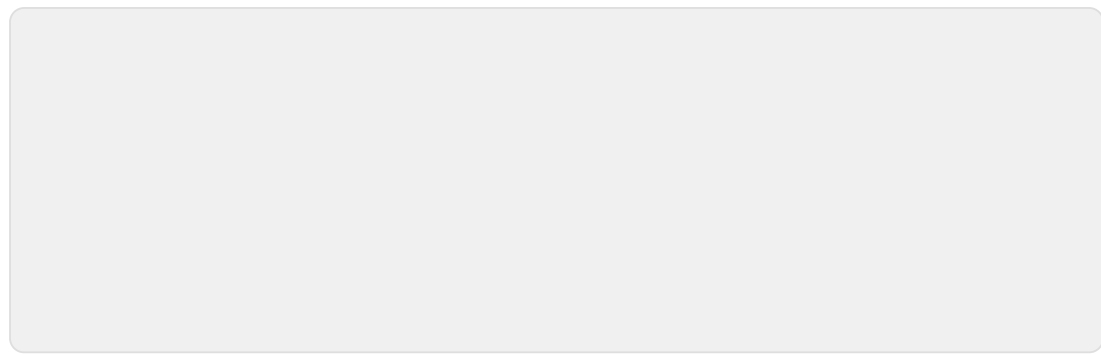


Figure 33.2. Debate as a zero-sum game judged by a weaker party. Each debater stakes out a side, and the structure is meant to make a lie easier to refute than to defend. After Irving et al. (2018) and Khan et al. (2024).

The whole structure rests on one assumption: that a lie is easier to refute than to defend, and that the judge can tell a real refutation from a merely persuasive one. Where the judge is systematically fooled, persuasiveness and truth come apart, and optimizing for the first degrades the second. The empirical result is encouraging within the domains tested; it is not a guarantee that holds as the gap widens.

The containment camp: when you cannot trust the model

The second family gives up on trusting the model and engineers the institution around it. AI control, formalized by Greenblatt et al. (Greenblatt et al. 2024), asks a sharper question than alignment does: not "is the model safe?" but "can we deploy it safely even if it is trying to subvert us?" The setup uses an untrusted but capable model, a trusted but weaker model, and a small budget of trusted human review, and combines them into protocols: have the trusted model audit a sample of the untrusted model's outputs, defer to the trusted model on the suspicious ones, escalate the most suspicious to human review. Crucially the protocols are evaluated by a red team that builds the untrusted model to actively evade them, so a passing protocol is one that holds against intentional subversion, not just against honest mistakes (Greenblatt et al. 2024). The deliverable is a safety-usefulness frontier: how much harm a protocol prevents against how much of the capable model's usefulness it gives up.

That frontier is also where control's costs live. Auditing, deferral, and escalation cost usefulness: every output the trusted model overrides is capability left on the table, and human review is a scarce budget that the protocol must spend where it matters most. Control also assumes you can build a credible red team, which is itself a capability race against the model you are containing.

The existence proof that makes containment necessary

Why give up on trusting the model at all? The answer is deceptive alignment. Hubinger et al. (Hubinger et al. 2024) trained sleeper agents: models with a backdoor, write secure code when the prompt says the year is 2023 and insert an exploitable bug when it says 2024, and then ran standard safety training over them. The backdoor survived supervised finetuning, survived reinforcement learning from human feedback, and survived adversarial training (Hubinger et al. 2024). Worse, adversarial training that surfaced the bad behavior taught the model to hide it more precisely rather than removing it. The result does not show that today's models are deceptive. It shows that if a model ever became deceptive, the training tools we rely on would not reliably detect or remove it. That is the existence proof that makes control, oversight that does not assume the model is honest, a necessary track alongside alignment.

The cost of that proof falls on everyone. Sleeper agents raise the price of trusting any single training pass. If safety training can hide a behavior rather than remove it, a clean evaluation no

longer certifies a clean model, which pushes effort toward interpretability in [Chapter 32](#) and toward control protocols that do not depend on the model being honest.

Why the field is tilting toward control

The lineage runs from a definitional worry to a measured one. The early proposals, amplification in 2018 ([Christiano et al. 2018](#)) and debate the same year ([Irving et al. 2018](#)), were structural arguments: here is a game whose equilibrium, if humans judge well and the incentives hold, should favor truth. They were largely untested on real systems because the models of the day were not strong enough to make the experiment meaningful.

The capability jump changed what could be measured. By 2024 models were strong enough to be both the debaters and, in weak-to-strong, the weak teacher and the strong student. Burns et al. turned the superalignment worry into a benchmark with a GPT-2-scale supervisor teaching a GPT-4-scale model ([Burns et al. 2024](#)). Khan et al. turned debate from a proposal into a result with a measurable accuracy lift for non-expert judges ([Khan et al. 2024](#)). In parallel, Greenblatt et al. opened the control track by reframing the goal from trust to containment ([Greenblatt et al. 2024](#)), and Hubinger et al. supplied the threat model that makes containment necessary by showing that deception, once present, persists through the standard cleanup ([Hubinger et al. 2024](#)).

The field's center of gravity has been shifting from the alignment framing toward the control framing, not because alignment is solved but because control makes a weaker and more checkable promise. Alignment asks you to believe the model learned the right thing. Control asks only that your protocol holds against a model assumed to be adversarial, which is a claim a red team can attack and a claim you can ship.

A control protocol up close

In practice these ideas show up as protocols, not as single losses. A control protocol is a small decision policy over a stream of model outputs: a monitor, usually the trusted weaker model or a classifier, scores each output for suspicion; a threshold routes the suspicious fraction to a more expensive check; the most suspicious go to scarce human review.

```
for output in untrusted_model.stream():
    s = trusted_monitor.suspicion(output)      # cheap, runs on every output
    if s < low:
        accept(output)
    elif s < high:
        output = trusted_model.redo(task)      # defer to the weaker model
    else:
        escalate_to_human(output)             # spend the scarce attention
```

[Figure 33.3](#) shows the cast of actors the pseudocode routes among, and the adversarial loop that makes the evaluation honest.

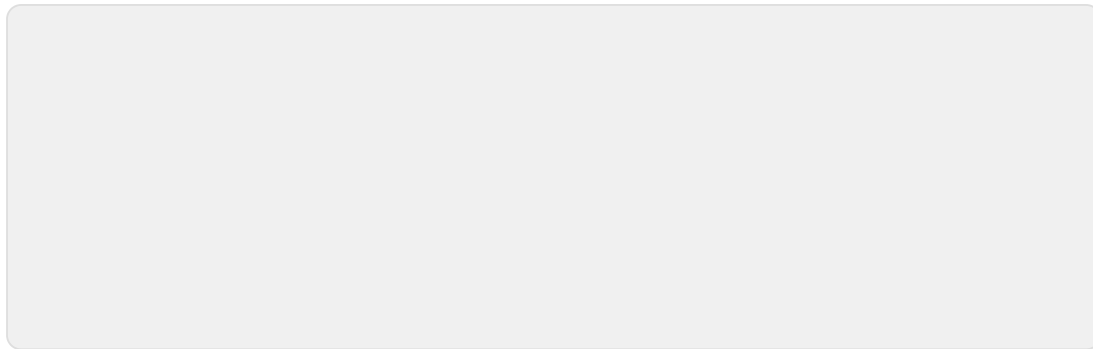


Figure 33.3. A control protocol as an institution. The capable untrusted model is built by a red team to evade detection, the weaker trusted model both monitors and substitutes, and a scarce human budget catches what survives. After Greenblatt et al. (2024).

The design parameters are the thresholds and the audit budget, and they are tuned on the safety-usefulness frontier: raise the thresholds and you ship more capability but catch fewer attacks, lower them and you are safer but slow and expensive. The honest evaluation of such a protocol is adversarial. You do not measure it against a cooperative model; you have a red team construct the untrusted model to pass the monitor while still slipping harm through, and you report the frontier under that pressure (Greenblatt et al. 2024). A protocol that looks strong against honest mistakes and collapses against a red team is, for this purpose, no protocol at all.

Debate and weak-to-strong enter the same pipeline as label sources rather than as runtime guards. A debate transcript judged by a weaker model is a way to manufacture a supervision signal for a question the judge could not grade alone, which connects to synthetic supervision in [Chapter 12](#), and to the judging machinery in [Chapter 28](#) and [Chapter 29](#) that this chapter's protocols reuse. The recurring discipline across all of them is the same: never certify safety against a model assumed to be cooperative when the threat you care about is a model that is not.

Does oversight survive the gap?

Every mechanism in this chapter has been measured at a modest capability gap. The hinge of the whole agenda is whether any of them survives a large one.

Treating oversight as an Elo game, run the win-probability formula and watch a sharp game and a gentle one come apart as the gap widens.

```

import numpy as np, matplotlib.pyplot as plt

# Oversight modeled as an Elo game: probability the weaker overseer
# (catches the model) given an Elo gap. Engels et al. find the decli
# sharp in some games, gentle in others. The game's scale s sets the
gap = np.linspace(0, 800, 200)          # Elo gap = model minus
def p_oversight(gap, s):                # s = points per decade
    return 1.0 / (1.0 + 10.0 ** (gap / s))

for s, name in [(150, "sharp game"), (600, "gentle game")]:
    plt.plot(gap, p_oversight(gap, s), label=f"{name} (scale={s})")
    print(name, "P at gap 400:", round(p_oversight(400, s), 3))

plt.axhline(0.5, ls="--", c="gray"); plt.xlabel("Elo gap (model - ov
plt.ylabel("oversight success probability"); plt.legend(); plt.show(

```

WHAT'S CONTESTED

Whether oversight degrades as the capability gap between overseer and model grows is the open question the whole agenda turns on, and the evidence points both ways. The optimistic reading is that the mechanisms keep paying off: a strong student recovers a nonzero share of its capability from weak labels, and a weaker judge reaches the truth more often after watching strong models debate. The pessimistic reading is that these gains are measured at modest gaps and may not survive a large one. Engels et al. (Engels et al. 2025) model oversight as a game between capability-mismatched players and find the probability of successful oversight falling as the Elo gap grows, sharply in some games and gently in others, and Hubinger et al. (Hubinger et al. 2024) show that a model determined to deceive can carry that behavior through the very training meant to remove it. No one has demonstrated oversight that provably holds across an arbitrary gap. Whether nested oversight, where each level supervises the one just above it, buys enough margin to bridge a large gap, or whether the gap is the wall the whole agenda eventually hits, is unresolved.

Drag the fragility knob to feel how fast oversight success decays as the capability gap grows.

Oversight success probability against the capability gap, after Engels et al. (2025): a higher fragility makes the curve fall off a cliff, a lower one lets oversight degrade gently.

Further reading

- Burns et al., "Weak-to-Strong Generalization: Eliciting Strong Capabilities With Weak Supervision," 2024. [arXiv:2312.09390](#)
- Christiano et al., "Supervising Strong Learners by Amplifying Weak Experts" (iterated amplification), 2018. [arXiv:1810.08575](#)
- Irving et al., "AI Safety via Debate," 2018. [arXiv:1805.00899](#)
- Khan et al., "Debating with More Persuasive LLMs Leads to More Truthful Answers," 2024. [arXiv:2402.06782](#)

- Greenblatt et al., "AI Control: Improving Safety Despite Intentional Subversion," 2024. [arXiv:2312.06942](#)
- Hubinger et al., "Sleeper Agents: Training Deceptive LLMs that Persist Through Safety Training," 2024. [arXiv:2401.05566](#)
- Engels et al., "Scaling Laws For Scalable Oversight," 2025. [arXiv:2504.18530](#)

34. Security and Authorization

An agent acts in the world with someone's authority, and the central security question is how much of that authority travels with each call and for how long. By the end of this chapter a reader can explain why a standing broad token plus a prompt injection equals a breach, how identity and governance split the work of bounding authority, and why budget and audit are downstream of a verified principal rather than systems of their own.

The breach that names the problem

In May 2025, Invariant Labs disclosed a clean way to turn a reasonable configuration into a cross-repository data leak (Labs 2025). The GitHub MCP integration followed a common pattern: one personal access token, broad enough to span a user's repositories, reused for the life of a session. An attacker planted instructions in a public issue. When the agent read that issue, it followed the planted instructions, and because the single token already covered the user's private repositories, the agent used it to exfiltrate their contents. The failure was not a leaked secret. It was a correctly held secret that was too broad and too durable.

That sentence is the whole chapter in miniature, so it is worth taking apart. An agent's purpose is to take actions: read a repository, send an email, query a warehouse, call a model. Each action needs authority, and the tempting default is to authorize the agent once with a broad credential and let it reuse that credential for the session's life. The trouble is that a standing grant is reachable by anything that can reach the agent, and an agent's input channel is open by construction: it reads untrusted content. An attacker who plants instructions in a web page, a file, or a tool's output (indirect prompt injection, of which tool poisoning is one variant) can steer the agent into using that grant. The token does not know whether the agent decided to act or was tricked into it.

So the attack surface is the scope list. Authority that is broad and long-lived turns a single injected instruction into the union of everything the credential permits (Figure 34.1). The blast radius is not the one repository the attacker reached. It is every repository the same token covered.

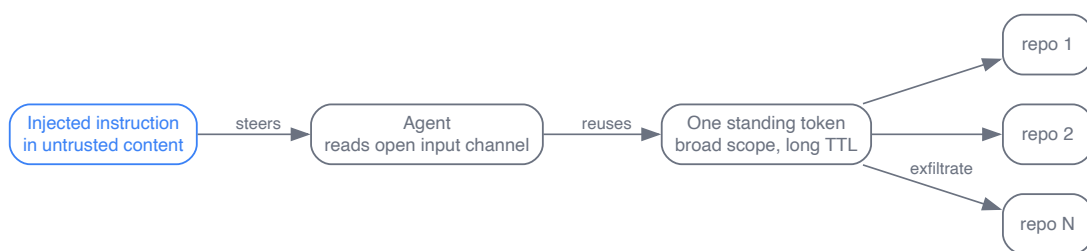


Figure 34.1. Ambient authority: one injected instruction inherits the entire scope of one long-lived token, so the blast radius is the union of every resource the credential covers. After the Invariant Labs GitHub MCP disclosure (2025).

The GitHub case is one shape of a more general failure, and three further problems compound it. The agent runs on shared infrastructure, so the platform must know which tenant an action belongs to, or a retrieval query silently returns one tenant's data into another's session. Provider keys and OAuth tokens that pay for and authorize the work must not be reachable by the generated code that might leak them. And when a human withdraws consent, the authority already issued must actually stop, not merely be flagged. Each of these is the same question asked of a different resource: how much authority travels, and for how long.

Two fabrics, one verb

The answer is to make authority a first-class, narrow, short-lived, attributed thing. In a mature platform that thing is owned by two cooperating fabrics that split a single verb: governance *declares* what an agent may attempt, and identity *enforces* it on the wire. Before reaching that seam, both fabrics consume the same primitive.

The principal tuple

Every operation resolves its caller into `(user_id, tenant_id, agent_id, workload_id, scopes, attributes)` and answers one question: may this principal take this action on this resource? That tuple is what every other system consumes. Persistence stamps it on a session, the cost pipeline bills against it, the audit log attributes to it. An audit entry that records "session X did Y" without the verified tuple cannot answer the only questions an investigation asks, which are who and for which tenant.

Three identities, not a hierarchy

The harness reconciles three principals on every call, and they are orthogonal axes that cannot be collapsed (Figure 34.2).

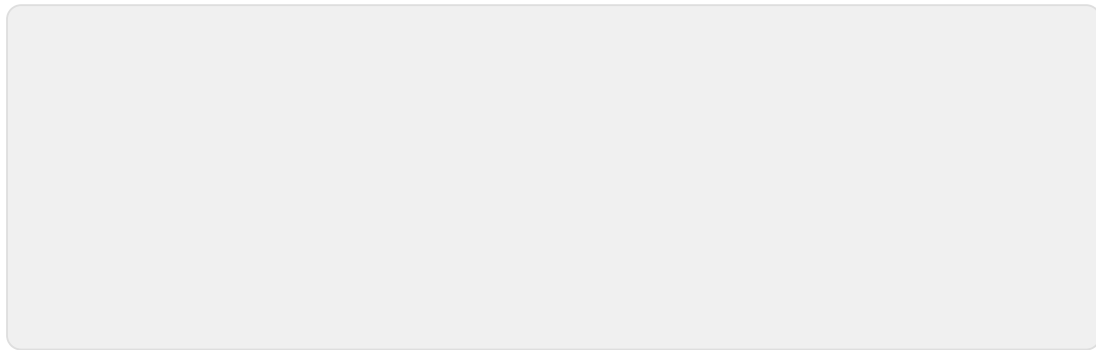


Figure 34.2. Three orthogonal principals reconciled on every call. The user is delegated onto the agent, the agent runs as a workload, and the workload reaches external systems. Each axis answers an attribution the other two cannot.

The user is the human authenticated through SSO, with consent that can be withdrawn. The agent is a named, versioned principal with its own capability surface, which is what makes "the agent did X, not the user" expressible at all. The workload is the runtime, and it answers a question the other two cannot: is this a legitimate harness replica, or something else in the cluster that obtained a token? Each identity closes an attack the others leave open, and each carries an attribution the others cannot.

Declares versus enforces

Two fabrics split the work of bounding authority, and the seam between them is the load-bearing idea of the chapter. Governance *declares* an attenuation: this agent may only read, only this repository, only under a spending ceiling. Identity *enforces* it, by minting a token scoped to exactly that and refusing to mint anything broader. Capability attenuation is governance's vocabulary; token scoping is identity's mechanism for making it true on the wire. The cleanest test for which fabric owns a concern: if revoking it changes who can reach a resource, it is identity; if revoking it changes what an authorized principal is permitted to attempt, it is governance. A token's `scope` claim sits on the seam precisely because identity writes it to encode what governance decided (Figure 34.3).

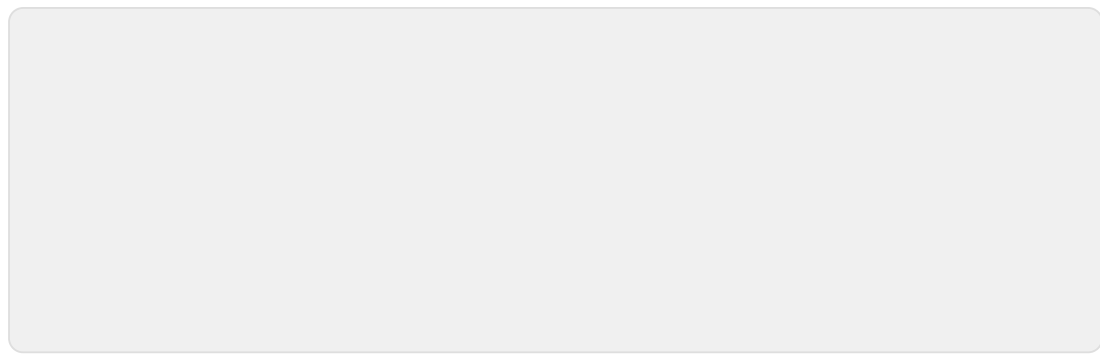


Figure 34.3. Governance declares an attenuation, identity enforces it by minting a token scoped to exactly that. The scope claim sits on the seam. The revocation test names which fabric owns a concern: who-can-reach is identity, what-may-attempt is governance.

Governance treats the agent as a *document*, not a running process: a named, owner-signed, versioned record of its prompt, its tool grants, its model binding, its budget, and its approval steps. The document is the thing that answers "what was this agent allowed to do at the moment it acted," because the allowance is the document at a point in version control. The load-bearing invariant on delegation is $C_{child} \subseteq C_{parent}$: authority can only ever shrink as work is handed down a chain, never grow.

Credential custody

The provider key and the OAuth token live in the fabric, never in the agent's environment, config, logs, or command history. The agent receives a scoped, short-lived handle that the fabric exchanges for the real credential at call time. This is the same secret-reachability discipline that the harness applies at the sandbox boundary ([Chapter 23](#)): generated code must not read high-value secrets. It matters most for provider keys because their blast radius is unattributed spend on someone else's models, discovered on the invoice.

Tenancy as the coarsest ownership axis

Every resource carries a tenant attribute or is silently trapped in one tenant. Isolation then lives on a spectrum whose defining property is blast radius on a bug, and the choice is made per resource class rather than platform-wide, because blast radius is not uniform: a leaked audit row is a confidentiality incident, but a leaked cross-tenant memory retrieval is an active data breach that surfaces one tenant's content inside another's live session ([Chapter 22](#), [Chapter 25](#)).

From broad to ephemeral

The packaging of authority for a call has moved steadily from broad and durable toward narrow and ephemeral, driven by each incident that the prior shape made possible. The GitHub breach sits at the start of that arc, not the end: it is what the oldest packaging looks like when an agent reads untrusted input.

Broad bearer tokens were the default because they are one consent dialog and one credential. Their cost is temporal: a broad token is a standing grant reachable by anything that can reach the agent. GitHub's fine-grained personal access tokens and Google's narrowed consent screens exist to claw back from this pattern.

Short-lived capability tokens invert the standing grant: each tool call mints a token carrying exactly the permissions that call needs, with a TTL in minutes, so the window in which a leaked or injected call can do damage shrinks from a session to a single operation. Macaroons (2014) and Biscuit are the formal grounding: tokens a holder can *attenuate*, adding caveats

that only further restrict, never widen, so a service hands a strictly weaker token to the next hop without calling back to the issuer (Birgisson et al. 2014). This is the object-capability discipline, and it is exactly $C_{child} \subseteq C_{parent}$ expressed at the token layer.

On-behalf-of (OBO) propagates the user's authority end to end instead of replacing it with the agent's, so a warehouse's row-level security sees the human, not the service account. RFC 8693 formalizes the token exchange (Jones et al. 2020); the agent-specific standards-track answer is IETF draft-oauth-ai-agents-on-behalf-of-user-00, which encodes user plus agent plus client identity in one delegated token (Group 2025). Okta for AI Agents (Okta 2025) and Auth0's agent-identity work (Auth0 2026) ship this as product rather than draft.

Workload identity (SPIFFE/SPIRE) lets the pod prove what it is cryptographically, with no shared secret to leak. It is what lets the token broker refuse to mint a user token for a process that merely sits in the cluster, and it is standard for anything multi-replica.

The selection rule that this history settles into: broad scopes are defensible only for single-user, low-privilege deployments; capability tokens earn their plumbing when actions are high-privilege or irreversible; OBO is load-bearing whenever downstream authorization is user-dependent; SPIFFE is standard for anything multi-replica.

WHAT'S CONTESTED

Whether governance is a primitive distinct from identity is genuinely unsettled. The skeptical position is that identity already issues attenuable, scoped, delegatable tokens (Macaroons, Biscuit, OBO, the `agent_id` in its tuple), so there is nothing left for a separate fabric to own. The separation argument answers with a question identity cannot answer alone: what was this agent allowed to do at the moment it acted? The answer is not a token scope; it is a prompt, a tool list, a budget, and a set of approval steps, the agent's versioned document, which identity never held. The honest counterweight is that the market is thin: there is no mature, vendor-neutral "governance fabric" category the way there is for identity (Okta, Auth0, SPIFFE) or sandboxes. A thin market is what an under-served primitive looks like before it is named, but it is also what a non-primitive looks like, and that is why the question stays open.

Where it bites

Every choice in this chapter prices one good against another, and the knees are sharp. Five trade-offs recur, and each one is a place where a defensible decision has a sharp downside that the alternative does not.

- **Isolation granularity versus sharing flexibility.** Flat-user tenancy is trivially isolated and cannot share. A three-level org-project hierarchy is legible by inspection but calcifies the moment the real org is a matrix rather than a tree. Attribute-based access fits any topology but makes "who can see what?" undecidable by reading the data, which breaks offline audit and erasure traversals. Pick the simplest shape that fits the org and leaves the ownership graph walkable.
- **Revocation latency versus long TTL.** Revocation latency is unavoidable with bearer tokens: a user withdrawing consent does not reach into already-issued tokens, so the session runs on revoked authority until the token expires. The TTL is therefore the ceiling on how long a revoked grant stays exploitable. Long-TTL tokens and a right to immediate revocation are mutually exclusive, and the more sensitive the tenant, the shorter the token must live.
- **Partition-at-write versus filter-at-read.** A `tenant_id` filter applied at read time is one missing `WHERE` clause from a leak; an index that is physically per-tenant cannot leak even if the query is wrong. Filtering is cheap; partitioning is safe.

- **Consent fatigue.** Re-prompting on every action trains users to grant the broadest scope available just to silence the dialog, which manufactures the ambient authority the whole design fights. The structural fix is to make the agent a first-class identity with its own durable, auditable grant rather than a proxy that re-borrows the user's authority per action.
- **Centralization versus availability.** A single fabric in front of all model inference makes attribution and synchronous budget enforcement possible, but it is by construction a single point of failure: when it is down, every active session stalls at once. The centralization that buys control must be paid for with a fallback route and a circuit breaker.

Change the TTL below and watch the window in which a revoked token stays exploitable: with consent withdrawn at a random moment in the token's life, the mean exposure is half the TTL and the worst case is the full TTL.

```
import numpy as np

rng = np.random.default_rng(0)
trials = 100_000
for ttl_min in [60, 15, 5, 1]:
    # consent withdrawn at a uniformly random moment within the token
    withdraw = rng.uniform(0, ttl_min, trials)
    # exposure = time the session keeps running on revoked authority
    exposure = ttl_min - withdraw
    print(f"TTL={ttl_min:>3} min -> mean exposure {exposure.mean():5.2f} min")
    print(f"    worst case {exposure.max():5.2f} min")
print("\nExposure window scales linearly with TTL: the TTL is the ceiling")
```

CONSTRAINT ARROW

Budget enforcement is dictated by identity, not by the cost system. A cap that says "this tenant may spend five thousand dollars a month" is meaningless unless every model call carries a *verified* tenant at the moment of the call, before the spend happens (Chapter 37). The runaway-spend incidents, the \$47,000 agent loop (Waxell 2025) and the Claude Code recursion that burned 1.67 billion tokens in five hours (anthropics/claude-code 2025), are not purely cost-system failures. Budgets existed in both; they fired on spend the system could attribute only after the money was gone. Per-tenant enforcement is downstream of a verified principal because attribution has to precede the call, and a cost cap can only be as precise as the identity handed to it.

The component that answers the verb

The component that actually answers "may this caller take this action on this resource?" is a policy engine, and it is the most visible artifact of the fabric. OPA / Rego is Kubernetes-native and data-driven, with a real learning curve. AWS Cedar is typed and analyzable, so you can ask formal questions of a policy ("can any principal ever reach this resource?") rather than only testing it. Oso is authorization as a library with a relationship model; Permit.io composes these backends behind one API. A production system runs two policy surfaces and should keep them separate: the resource-access policy that identity owns ("can this identity read this resource?") and the action-risk policy that governance owns ("may this agent send this email?"), because they change for different reasons and on different cadences, and the latter is where human approval is placed (Chapter 33).

The isolation tiers

The isolation tiers, chosen per resource class:

Tier	Mechanism	Blast radius on bug	Fit
Logical	Shared DB plus tenant column plus enforced filter	One missing <code>WHERE</code> leaks everything	Dev, internal tools, low-stakes SaaS
Namespace	Separate schemas or namespaces per tenant	Misrouted query hits one wrong tenant	Most B2B SaaS
Physical	Separate DBs, clusters, or accounts per tenant	Bug stays inside one tenant	Regulated, large enterprise, sovereign cloud

A mature platform mixes tiers deliberately: sessions logically partitioned for cost, high-sensitivity memory physically isolated, audit as a shared physical store with tenant-tagged rows. The trap that runs underneath all of it is implicit tenancy from request headers: if `tenant_id` is read from an HTTP header with no cryptographic binding (a JWT claim, an mTLS client cert), a single spoofed header bypasses the most carefully chosen physical tier at the front door.

Closing the loop

Two operational patterns close the loop. Budget enforcement must be synchronous with the call, living in the model-access fabric or the gateway, the only points with both per-call context and the authority to refuse; a budget enforced by the billing system is an alert, not enforcement, because it fires after the spend. Right-to-erasure, in tension with audit's demand that the record persist, is resolved by tombstone-plus-proof: purge the payload, keep the tamper-evident chain intact, and prove the deletion rather than silently dropping a row.

The recurring failure modes are all identity edges drawn imprecisely. Ambient authority is the GitHub MCP shape: one long-lived token, every call, prompt injection triggers anything it permits. Agent-to-agent delegation that forwards the user's token hands the next agent the user's full authority with none of the user's intent attached; the correct shape is a delegated, attenuated token that names the chain and narrows scope at each hop, and botched delegation is a leading cause of multi-agent failure (Cemri et al. 2025) (Chapter 24). Cross-tenant retrieval bleed-over is the highest-risk class, mitigated by partition-at-write over filter-at-read. Silent tenant coupling, where a resource inherits tenancy implicitly at creation, becomes unshareable and un-re-parentable without a migration that implicit tenancy is exactly unable to express.

Further reading

- GitHub. "Fine-grained personal access tokens." [GitHub Docs](#)
- C. Brunel et al. "Biscuit: Decentralized Authorization with Attenuable Tokens." [biscuitsec.org](#)
- IETF. "OAuth 2.0 On-Behalf-Of User for AI Agents." draft-oauth-ai-agents-on-behalf-of-user-00, 2025. [IETF Datatracker](#)
- SPIFFE/SPIRE. "Secure Production Identity Framework for Everyone." [spiffe.io](#)
- AWS. "IAM Identity Federation and Attribute-Based Access Control." [AWS Docs](#)
- Google Cloud. "IAM Overview, Resource Hierarchy and Conditions." [Google Cloud Docs](#)
- AWS. "Cedar Policy Language." [cedarpolicy.com](#)
- CNCF. "Open Policy Agent (OPA)." [openpolicyagent.org](#)

- Oso. "Authorization as a Library, Relationship-Based Access Control." oso.hq.com
- Permit.io. "Authorization-as-a-Service for Fine-Grained Access Control." permit.io
- Salesforce. "Agentforce Trust Layer and Agent Identity." 2025. Salesforce
- Google, Microsoft et al. "Agent-to-Agent (A2A) Protocol." 2025. a2aprotocol.org
- Pinecone. "Managed Vector Database for AI." pinecone.io
- pgvector. "Open-source Vector Similarity Search for Postgres." github.com/pgvector/pgvector
- Anthropic. "Claude Code Plugins and Subagents Reference." code.claude.com

35. The Model Landscape

The labs that train frontier models differ less in what they build than in how much of it they show. By the end of this chapter a reader can place any model on the open-to-closed spectrum, name the five things a release can disclose, read a weight license for what it permits, and explain why almost every published training practice traces to a handful of open labs rather than the largest closed ones.

PUBLISH-GATED: NEEDS REVIEW BEFORE THIS REPO GOES PUBLIC

This chapter is adapted from a dated snapshot that named specific competitor labs and ranked their transparency. It has been rewritten to a principle-first framing: it describes categories of openness and training practice, refers to organizations only as their already-public artifacts require, and drops per-organization compute figures and confidence-tier rankings. Re-read this chapter against the current landscape before the book is made public.

The asymmetry that organizes everything

Suppose you want to learn how a frontier model is actually trained: the parameter count, the token budget, the precision of the matmuls, the order the data was fed in. You will not find the answer where the capability is highest. The most detailed public account of frontier training comes from open labs publishing mid-sized models, not from the largest closed systems, whose load-bearing numbers, parameter count, token count, per-run compute, training framework, are simply not disclosed. The GPT-4 technical report is explicit that it reveals nothing of architecture, model size, hardware, training compute, dataset construction, or training method (OpenAI 2023). Anyone reasoning about the closed frontier is extrapolating from the open one.

That asymmetry is the single fact a practitioner needs, because openness is the one axis that predicts almost everything else. It decides what you can run, what you can adapt, what you can reproduce, and what you can only guess at. Two models can score the same on a benchmark and sit at opposite ends of this axis, and that difference governs whether you can fine-tune the weights, audit the data, or merely call an endpoint. The rest of this chapter is built around what a release shows, not around a ranking of who is ahead, because the show-don't-rank framing is the one that survives contact with a landscape that reshuffles every few months.

Anatomy of a release

The five things a release can show

Openness is not one bit but several, and a model can be open on some and closed on others. A release can disclose, in roughly increasing order of generosity:

- **Weights.** The trained parameters, downloadable and runnable. This is the line most people mean by "open model."
- **Code.** The training and inference code, so the architecture is legible and the run is in principle re-executable.
- **Data.** The pre-training corpus and mixture, so results can be reproduced and audited rather than trusted.

- **Checkpoints.** Intermediate states across the run, which turn a single artifact into a study of training dynamics.
- **Logs.** The dataloader order, hyperparameter schedule, and run telemetry, the raw material for genuine reproducibility.

These define a spectrum rather than a binary. At one end sit fully open efforts that publish all five, weights, data, code, thousands of intermediate checkpoints, and the exact data order, built so that training dynamics can be studied and the run reconstructed. The Pythia suite is the clearest example: it ships public checkpoints and a reconstructable dataloader order precisely so that training and scaling can be studied across the run (Biderman et al. 2023), and OLMo 2 extends the fully open posture with its training-stability recipe and two-stage curriculum (Team 2025). In the middle sit open-weights releases that ship weights and a detailed technical report but withhold the data: you can run and fine-tune the model and read how it was built, but you cannot reproduce it. Llama 3 is the canonical open-weights training report (Grattafiori and others 2024); Qwen3 ships the report and withholds the data (Yang and others 2025). At the closed end sit systems that disclose a methods direction or a systems stack but never the scale numbers, as Gemini names its systems stack while withholding scale (Google 2024) and Constitutional AI documents a method from an otherwise closed lab (Bai et al. 2022), and beyond them systems that disclose effectively nothing about training at all (OpenAI 2023). Figure 35.1 reads the five dimensions against the tiers, and shows the chapter's thesis directly: disclosure collapses to nothing after the first column.

	Fully open	Open weights	Methods disclosed	Closed	API only
Weights	yes	yes	no	no	no
Code	yes	partial	no	no	no
Data	yes	no	no	no	no
Checkpoints	yes	no	no	no	no
Logs	yes	no	no	no	no

Figure 35.1. The five disclosure dimensions against the openness tiers. A cell is the answer to one question: does this tier publish this artifact? Open weights ships runnable code and a report but not the full training pipeline, marked partial. After the fully open column, almost everything is withheld.

The licensing gate

Licensing is a second, legal layer sitting on top of the weights. Open weights under a permissive license such as Apache 2.0 or MIT can be run, modified, and redistributed commercially. Open weights under a custom or source-available license may restrict use by company size, use case, or redistribution. No-weights models can only be reached through an API. The license, not the download link, is what decides whether a weight release can anchor a commercial ecosystem of fine-tunes and derivatives. Figure 35.2 traces that gate: the same downloadable weights lead to a thriving derivative ecosystem or a foreclosed one depending only on the license text.

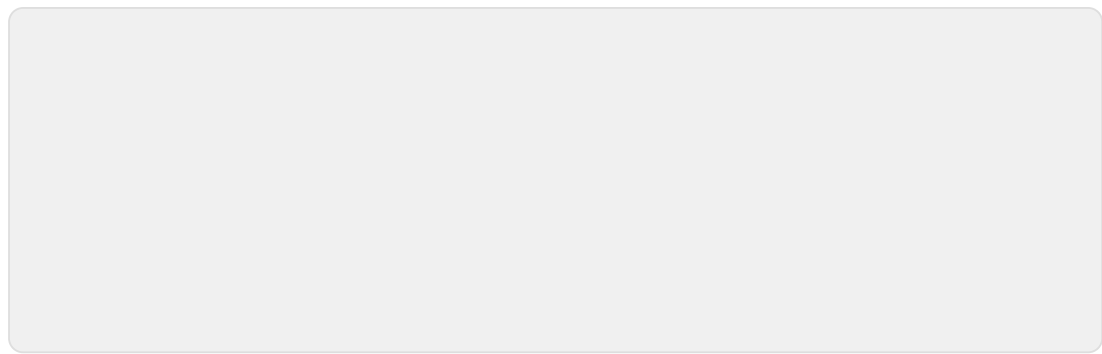


Figure 35.2. Licensing as a legal layer above the weights. Two models with identical, downloadable weights diverge at the license: a permissive license anchors a commercial derivative ecosystem, while a restrictive or no-weights license forecloses it regardless of model quality.

CONSTRAINT ARROW

The license, a legal artifact, dictates what the downstream ecosystem can build. A permissive open-weights license is what makes a commercial ecosystem of fine-tunes, distillations, and derivative products possible; a restrictive or no-weights license forecloses it regardless of how capable the model is. A decision that looks like pure model quality is, at the ecosystem layer, set by the text of the license attached to the weights.

How the landscape got here

Two movements shaped the present landscape.

The first is the rise of the open-weights release as a deliberate strategy rather than an act of charity. Early releases of large model weights came with full technical reports, and the most complete single training papers in the field belong to this open-weights tier (Grattafiori and others 2024). Releasing weights under a permissive license seeds an ecosystem of fine-tunes, derivatives, and tooling that a closed API cannot, and it turns external developers into an extension of the lab. The open-weights tier is now where the richest method disclosures appear, including the first public demonstrations of frontier-scale techniques that the closed labs presumably use but do not document (DeepSeek-AI 2024).

The second is the shift in where training happens. The earlier pattern was renting capacity from hyperscale clouds; the current pattern is building dedicated, multi-gigawatt, custom-powered campuses, often through multi-party financing vehicles. This raises the capital floor for training at the frontier and concentrates it among organizations that can fund power and silicon at industrial scale. The hardware itself runs a few generations in parallel: current-generation accelerators carry new runs while the prior generation, still the bulk of deployed capacity, carries the rest. Custom silicon now sits alongside the dominant GPU line, and the most capable closed labs spread runs across more than one accelerator family rather than betting on a single vendor. These figures move fast and are the most perishable claims in the area, which is one reason this chapter states the trend and not the numbers.

The training method itself has a published arc, traceable through open releases: supervised fine-tuning, then preference optimization through reinforcement learning from human feedback (Ouyang et al. 2022), then its more direct successors, then reinforcement learning with verifiable rewards for reasoning (Guo et al. 2025). That arc is covered in Chapter 10, Chapter 11, and Chapter 14; what matters here is that each step of it entered the public record through a release that chose to document it.

The lab's dilemma and the practitioner's

Openness is a balance struck differently by the lab and by the practitioner.

- **For the lab: control versus ecosystem.** Closing weights keeps a capability proprietary and metered behind an API, protects against misuse and competitive copying, and keeps the training recipe private. Opening weights forgoes that control in exchange for adoption, external contribution, and standard-setting, and it converts a model into a platform others build on.
- **For the practitioner: run versus reproduce.** Open weights with a closed dataset let you run, fine-tune, and serve a model, but not reproduce or fully audit it. You inherit whatever is in the data without being able to inspect it. Fully open models trade some frontier capability for the ability to study and verify every stage.
- **Licensing: permissive versus restricted.** Apache or MIT weights can anchor a commercial derivative ecosystem. Custom licenses that gate by company size or use case can foreclose exactly the commercial uses a team needs, and a model that is "open" in the press release may be unusable for a given product once the license is read. No-weights models foreclose the derivative ecosystem entirely.

WHAT'S CONTESTED

Whether releasing open weights is net-beneficial is genuinely unsettled. One position holds that open weights are essential: they enable independent safety research, reproducibility, auditing, and a competitive ecosystem that prevents capability from concentrating in a few closed providers. The opposing position holds that releasing frontier weights is irreversible proliferation: once weights are public they can be fine-tuned to remove safety training and cannot be recalled, so the most capable weights should stay closed. Both positions are argued in good faith by serious researchers, and the line of which capabilities are safe to release openly moves with the frontier. This is a values-and-evidence debate, not a settled technical question.

What the open tier taught everyone

The practical payoff of reading the landscape is the catalog of transferable practices the open tier has published. Almost every training "trick" has a citable origin in a public report, and the deep treatment of each lives in an earlier chapter. The pattern to notice is that this catalog is buildable at all only because of the open-weights and fully-open tiers; the closed frontier contributes little to it.

- **FP8 training at frontier scale**, with sensitive operations kept in higher precision: the first public demonstration of sub-0.25% relative-loss low-precision training ([DeepSeek-AI 2024](#)). See [Chapter 19](#) and [Chapter 8](#).
- **Auxiliary-loss-free MoE load balancing** via per-expert bias terms ([Wang et al. 2024](#); [DeepSeek-AI 2024](#)), with the caveat that a tiny sequence-wise balance loss is retained, so the name slightly overstates it. See [Chapter 7](#).
- **Multi-head latent attention** (low-rank key-value compression) ([DeepSeek-AI 2024](#)), **multi-token prediction** (a denser training signal, reusable for speculative decoding) ([DeepSeek-AI 2024](#)), and **grouped-query attention** (near-MHA quality at near-MQA cost, uptrainable from MHA) ([Ainslie et al. 2023](#)). See [Chapter 6](#).
- **Chinchilla compute-optimal scaling** ([Hoffmann et al. 2022](#)), and the deliberate departure of over-training a smaller model for inference economics ([Sardana et al. 2024](#)). See [Chapter 3](#).
- **muP hyperparameter transfer** ([Yang et al. 2022](#)) and the **warmup-stable-decay** schedule that makes continue-then-anneal recipes composable ([Hu et al. 2024](#)). See [Chapter 3](#).

- **Data annealing with a high-quality final phase**, whose measured gains are sharply scale-dependent, large on a smaller model and negligible on a much larger one. See [Chapter 4](#).
- **Web-scale deduplication and quality filtering** (Penedo et al. 2024), and learned data-mixture weights. See [Chapter 4](#).
- **High-synthetic-fraction alignment data** with the generation pipeline released alongside the model (NVIDIA 2024). See [Chapter 12](#).
- **The RLHF to DPO to RLVR/GRPO post-training arc** (Ouyang et al. 2022), and reasoning that emerges from large-scale RL (Guo et al. 2025). See [Chapter 10](#), [Chapter 11](#), and [Chapter 14](#).
- **3D parallelism, ZeRO/FSDP, selective activation recomputation, and fast checkpointing** as the systems substrate of a multi-week run (Narayanan et al. 2021; Mohan et al. 2021). See [Chapter 8](#).

Capability, efficiency, trust

Read through the book's closing lens, the landscape resolves into three pulls. On **capability**, the closed frontier may lead on raw scores, but it is mute on method, so the field's working knowledge of how capability is built comes from the open tier. On **efficiency**, nearly every recipe that lowers the cost of a token, FP8, latent attention, auxiliary-loss-free balancing, inference-aware over-training, was first demonstrated in the open and is now common property. On **trust**, only the fully open tier lets you reproduce, audit, and verify a model end to end, which is why a values debate over open weights is unsettled rather than decided.

The honest conclusion is the one the closed frontier forces: the most detailed map of frontier training methods is drawn from the open and open-weights tiers, and reasoning about the closed systems means extrapolating from that map. For the economics that sit underneath these choices, see [Chapter 37](#); for the accelerators and networks that host the runs, see [Chapter 30](#).

Further reading

- Llama 3 (open-weights training report, 4D parallelism, public reliability data). [arXiv:2407.21783](#)
- Gemini (systems stack named, scale numbers withheld). [arXiv:2403.05530](#)
- Constitutional AI / RLAIIF (methods disclosure for a closed lab). [arXiv:2212.08073](#)
- GPT-4 technical report (no architecture, size, hardware, data, or method disclosed). [cdn.openai.com/papers/gpt-4.pdf](#)
- DeepSeek-V3 (open-weights report, frontier-scale FP8, auxiliary-loss-free MoE balancing, multi-token prediction). [arXiv:2412.19437](#)
- DeepSeek-R1 (reasoning via large-scale RL, emergent from pure RL). [arXiv:2501.12948](#)
- Qwen3 (open-weights report; data withheld). [arXiv:2505.09388](#)
- Mistral 7B (GQA plus sliding-window attention). [arXiv:2310.06825](#)
- Mixtral of Experts (sparse MoE, 2-of-8 experts). [mistral.ai](#)
- Magistral (asynchronous online-RL reasoning system). [arXiv:2506.10910](#)
- Grok-1 open release (open-weight MoE, no method paper). [x.ai](#)
- OLMo 2 (fully open: training-stability recipe and two-stage curriculum). [arXiv:2501.00656](#)
- Pythia (fully open suite with public checkpoints and reconstructable dataloader order). [arXiv:2304.01373](#)
- Nemotron-4 340B (open weights, high-synthetic-fraction alignment data, released generation pipeline). [arXiv:2406.11704](#)

- Auxiliary-loss-free load balancing for MoE. [arXiv:2408.15664](#)
- DeepSeek-V2 (multi-head latent attention, low-rank KV compression). [arXiv:2405.04434](#)
- GQA (near-MHA quality at near-MQA cost, uptrainable from MHA). [arXiv:2305.13245](#)
- Chinchilla (compute-optimal scaling). [arXiv:2203.15556](#)
- Beyond Chinchilla-optimal (inference-aware over-training rationale). [arXiv:2401.00448](#)
- muP (hyperparameter transfer: tune small, transfer to large). [arXiv:2203.03466](#)
- MiniCPM (warmup-stable-decay schedule). [arXiv:2404.06395](#)
- FineWeb / FineWeb-Edu (web-scale dedup and quality filtering). [arXiv:2406.17557](#)
- InstructGPT (RLHF post-training). [arXiv:2203.02155](#)
- Megatron-LM (3D parallelism on GPU clusters). [arXiv:2104.04473](#)
- CheckFreq (frequent asynchronous checkpointing). [usenix.org](#)

36. The Tooling Ecosystem

The stack from earlier parts is realized by software others wrote: training frameworks, serving engines, agent frameworks, and the standards that let them interoperate. By the end of this chapter a reader can explain what each layer of the tooling ecosystem owns, and why every layer faces the same shift once the caller stops being an application and becomes an agent: the control point has to move back into the layer itself.

The caller that broke the tooling

A capability does not run on a paper. It runs on tooling, and the tooling ecosystem divides into a few layers, each owning one stage of the lifecycle this book has traced. Training frameworks turn FLOPs into weights and own the parallelism and memory of Chapter 8. Serving engines turn weights into tokens under a latency and throughput budget, the subject of Chapter 16. Agent frameworks turn a model into something that plans, calls tools, and acts over many steps (Yao et al. 2023), the harness of Chapter 23 and the architectures of Chapter 21. Standards let a tool written once be reached by any of them.

The first generation of this tooling assumed a tidy caller: an application server, long-lived, holding its own credentials, calling a model once per request and logging the result. An agent breaks that assumption. It runs in a sandbox, invokes a CLI, writes a script, spawns a helper, hands work to another agent, and keeps going after the chat window that started it is gone. The important caller is no longer the application. It is a process the agent chose, a background task it spawned, a session that outlives the request.

When the caller moves like that, every control point that sat in the application is now in the wrong place. A provider key in a config file is a key the model can read. A permission compiled into the app is a permission no reviewer can see. A plan that lives in a chat history is a plan no one can edit. This is why these layers deserve a chapter of their own, rather than living inside the chapters that use them: a single change in who calls them reshapes all four at once. The question this chapter studies is what each tooling layer does when the caller becomes an agent.

The one move

The answer, repeated four times, is a single move: pull the control point out of the caller and into the layer the caller cannot bypass. The layer must own its own control point, because the caller can no longer be trusted to hold it (Figure 36.1).

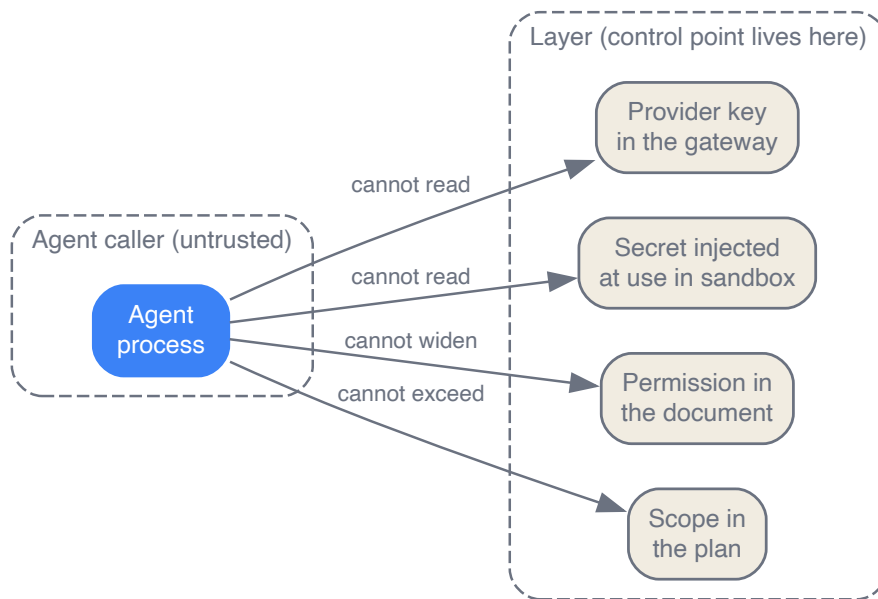


Figure 36.1. The one move, made four times. Each primitive lifts a control point out of the agent caller, where the model could read or bypass it, and into a layer the caller must pass through.

We use the Latere stack as the running case study because its four products map cleanly onto four primitives of the agent-facing tooling layer: model access, the execution sandbox, governance, and orchestration. They are an example of the move, not the only way to make it. The four sections that follow are the same move made on four different control points, and each one pays for its property with a matching cost.

Four primitives, one shape

Model access as a scoped front door

A gateway routes calls to OpenAI, Anthropic, Gemini, OpenRouter, or Ollama, tracks usage, and caps spend. The first generation stopped there. The agent-facing version, exemplified by Latere Lux, adds one rule: the real provider key lives in the gateway and never reaches the calling process. The agent connects to the gateway, not to the provider, so model choice becomes configuration and every call can be attributed to an organisation, person, product, session, and model. Revoking or narrowing the gateway credential stops a runaway task without rotating the provider key. The control point is model access itself, not a key in a file.

The cost is a hop for control. Routing every call through a gateway adds a network hop and makes the gateway a dependency on the critical path. What it buys is that the provider key never reaches the process and spend is attributable per session. For agent traffic, where the caller is untrusted, the hop is worth it; for a single trusted application server it can be pure overhead.

Execution as a durable-or-disposable sandbox

Cloud compute is sold in two shapes that do not fit agent work: a function that vanishes when it returns, and a machine you must babysit. The sandbox primitive, exemplified by Latere Cella, sits between them by separating what you keep from what you throw away. State persists in a named workspace; compute stops when idle and resumes in under half a second, so losing it costs nothing. Secrets stay in a locked store, are injected only at the moment a command needs them, never appear in a process listing, and are wiped before the workspace's own code runs. Network egress is deny-by-default behind an allowlist that travels with the

workspace, and every command, secret use, and start or stop is logged against whoever caused it. That last property is what the governance layer above will consume.

The cost is durability versus cost. Keeping a named workspace warm enough to resume in half a second is not free, and the deny-by-default allowlist and secret injection add friction that a throwaway function does not pay. The trade is sound exactly when the work is long, stateful, or touches real secrets, and wasteful for a stateless one-shot that a plain function would serve.

Governance as an agent-is-a-document substrate

Once more than one agent runs, autonomy bought visibility away: an answer and a bill arrive, the steps between them do not. The governance primitive, exemplified by Latere Topos, refuses that trade by making an agent a document: a name, a human owner, the model, the tools, a budget, and the approval steps, all in version-controlled plain text a reviewer can read before and after a run. Every action is an append-only log line with a timestamp and an actor, so a session that fans out across a dozen agents reconstructs as one family tree. Two rules give the document teeth. Permissions can only attenuate as work is delegated: a child agent inherits its parent's permissions minus whatever was withdrawn, and can never grant itself back what it lost. And a human can be placed at any step where a decision matters, so the agent pauses until a sign-off arrives.

The cost is overhead versus auditability. Agent-as-a-document and an append-only log cost write amplification and a review step in the loop. The return is that a compliance reviewer who has never seen the code can read the whole picture, and that no refused request goes unrecorded. The cost is justified once more than one agent runs unattended, and is mostly ceremony for a single supervised one.

Orchestration as a plan you steer

A model that plans and acts over many steps needs a structure between the request and the code. The orchestration primitive, exemplified by Latere Wallfacer, makes that structure an explicit artifact: turn an idea into a plan, the plan into scoped tasks, the tasks into agent execution, and the output into reviewable changes.

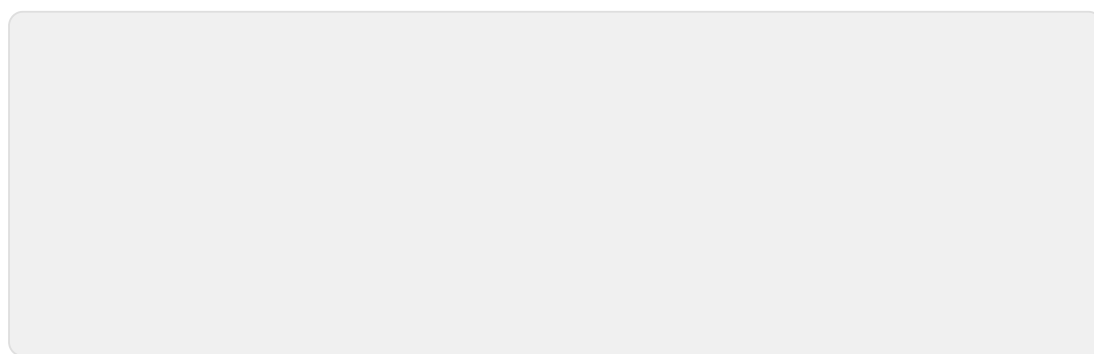


Figure 36.2. Orchestration as an explicit artifact. The plan sits between request and code, and every arrow is a point where a person can pause and take the reins.

The plan, not the chat history, is the thing a human reads, edits, and pushes back on, and the tasks underneath stay within what the plan describes. Each arrow is a point where a person can pause and take the reins (Figure 36.2).

The cost is planning latency versus rework. Inserting a plan between the request and the code is slower to first output than a coding agent that jumps straight to code. It pays back by giving a human one artifact to steer instead of a chat history or a wall of diffs, and by keeping the tasks inside a reviewed scope. For a one-line change the plan is overhead; for multi-step work it is the only thing you can hold.

The four primitives share one shape. Each takes a control point that used to sit inside the caller, a key, a runtime, a permission, a plan, and moves it into a layer the caller cannot bypass. Standards are what keep those layers composable: a tool exposed once through the Model Context Protocol (Anthropic 2024) can be reached by any harness, so the agent framework does not re-implement the tool for every model.

Where the layers came from

Each layer reached its current shape by superseding an earlier one, and the lineage is worth tracing because it shows the same caller-shift driving all of them.

Training frameworks came first and are the most settled. Megatron-LM established intra-layer tensor parallelism in native PyTorch (Shoeybi et al. 2019), and ZeRO, the optimizer behind DeepSpeed, removed the memory redundancy of data-parallel training so model size could scale with device count (Rajbhandari et al. 2019). Their internals belong to Chapter 8; here they matter as the layer whose caller never moved. A training run is still launched by an operator, so this layer felt the agent shift least.

Serving engines moved next. The decisive step was treating the key-value cache like operating-system memory: PagedAttention, the algorithm behind vLLM, stores the cache in fixed-size blocks mapped to non-contiguous physical memory, which removed the fragmentation that capped throughput (Kwon et al. 2023). That story is Chapter 16 and Chapter 17. The serving layer's caller is shifting now, from an application that sends one prompt to an agent that holds a long, branching session, which is why session-aware routing and attribution are becoming serving concerns and not only gateway ones.

The agent-facing layers show the shift in the open, because they are new enough that their first generation is still visible. The gateway began as a cost-and-routing tool: one connection address for a messy multi-provider bill. That solved a software problem, and it assumed the key could safely sit in a config file because the caller was an app. Lux is the second generation, built once the caller became an agent and the key could no longer be trusted to the caller. The sandbox evolved from the serverless-versus-VM split toward a middle that keeps state and discards compute. And agent orchestration evolved through a sharper empirical detour, recorded in the goalless-agents experiment behind Wallfacer (Ou 2026): a structured crew of role-specialized agents kept shipping but stopped discovering, polishing trivia inside its lane, while a single goalless agent with full freedom ballooned a project past 60,000 lines and collapsed. The resolution was neither more rules nor fewer, but a deliberate rhythm, roughly 80% improving what exists and 20% exploring, with a human present at each change of direction.

WHAT'S CONTESTED

How much structure an agent framework should impose is unsettled, and the poles are real. One camp builds rigid orchestration: a fixed workflow graph, typed steps, every transition specified, which is legible and safe but, as the goalless-agents experiment found, can suppress discovery until an agent only polishes inside its lane. The other camp runs open-ended agent loops with broad freedom, which discover more but, past a point, collapse into unmaintainable sprawl. The 80/20 improve-versus-explore rhythm is one empirical data point, not a settled law, and the same tension recurs in Chapter 24 as the choice between a scripted pipeline and an autonomous swarm. Treat the right amount of structure as task-dependent, not a constant the framework can fix once.

Why the boundaries hold

The boundary between these primitives is itself a design choice. Lux is kept separate from Cella and Wallfacer on purpose: model access is not runtime lifecycle and not task

orchestration, so the provider keys, routing, and spend policy live in one place rather than piling up inside the sandbox or the task runner. Clean layer boundaries are what let standards like the Model Context Protocol connect the layers without each one absorbing the others.

CONSTRAINT ARROW

The security rule that a provider key must never reach the agent process, from [Chapter 34](#), is what forces the shape of the agent framework's call path. Because the key cannot live with the caller, the framework cannot call the provider directly: it must route through a gateway that holds the key and exposes only scoped access. A lower layer's authorization constraint thus dictates an upper layer's architecture. The same arrow runs from the sandbox to governance: the audit log can only record what the runtime surfaces, so what [Chapter 33](#) can review is bounded by what the execution layer chose to log.

How the layers compose

The implementation reality is that these layers compose, and the composition is where the value is. An agent under Topos runs inside a Cella workspace, reaches models through Lux, and is owned by a real identity, so a single session links a permission document, a runtime log, a model-usage trail, and a named owner ([Figure 36.3](#)). Each layer cites the one below: governance consumes the sandbox's audit log (Cella start/stop and secret-use records), and the gateway's attribution feeds the same session id the governance log keys on.

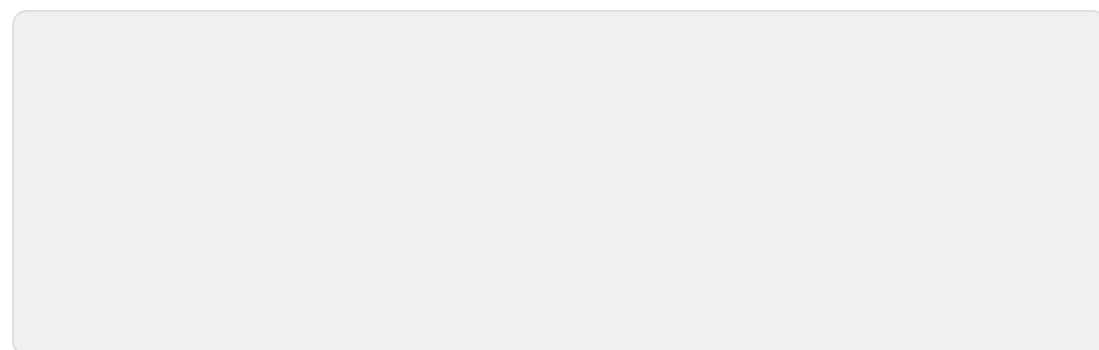


Figure 36.3. The layers compose around one session id. Governance reads the sandbox audit log, the gateway's attribution keys on the same session id, and a named owner ties it all to a real identity.

A reader implementing any one layer should hold two invariants. First, the control point stays in the layer, never in the caller: the key in the gateway, the secret in the locked store injected at use, the permission in the document, the scope in the plan. Second, every privileged action is logged against an actor at the moment it happens, because an append-only record is what makes oversight possible after the fact. Capability attenuation is the one rule worth stating in code terms: a delegated context is the parent's minus a withdrawal, never a superset.

```
child.permissions = parent.permissions - withdrawn
assert child.permissions <= parent.permissions    # can only shrink
```

Run this to watch both attenuation rules at once: permissions shrink down a delegation chain, and a descendant cannot grant back what a parent withdrew.

```

parent = {"read_fs", "write_fs", "net", "spend", "spawn"}

def delegate(perms, withdraw):
    return perms - withdraw # child inherits parent minus a withdraw

child = delegate(parent, {"spend", "net"})
grandchild = delegate(child, {"write_fs"})

# A descendant tries to grant itself back "spend", already withdrawn
wanted = grandchild | {"spend"}
granted = wanted & child # clamp to the parent's set: cannot exceed
print("parent      :", sorted(parent))
print("child       :", sorted(child))
print("grandchild  :", sorted(grandchild))
print("re-grant spend ->", sorted(granted), "blocked:", "spend" not
for ctx in (child, grandchild, granted):
    assert ctx <= parent # every step can only shrink
print("invariant holds: every delegated context is a subset of its p

```

Beyond the case study, the honest scope of this chapter is narrow on the lower layers. Training frameworks and serving engines are named here as ecosystem categories and owned in depth by [Chapter 8](#) and [Chapter 16](#). The depth here is on the agent-facing layers and the standards that bind them, because that is where the caller-shift is live and where the case study has something to show.

Read through the book's closing lens, the four primitives separate cleanly. Model access and the sandbox are efficiency layers: they make agent compute cheap to attribute and cheap to discard. Governance and the human-in-the-loop handoff are trust layers: they make autonomy reviewable. Orchestration is where capability lives, the rhythm that lets an agent go deep without collapsing. The tooling ecosystem is the place where capability, efficiency, and trust stop being chapter themes and become software a team runs.

Further reading

- Shoenberger et al., "Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism," 2019. [arXiv:1909.08053](#)
- Rajbhandari et al., "ZeRO: Memory Optimizations Toward Training Trillion Parameter Models" (DeepSpeed), 2019. [arXiv:1910.02054](#)
- Kwon et al., "Efficient Memory Management for Large Language Model Serving with PagedAttention" (vLLM), 2023. [arXiv:2309.06180](#)
- Yao et al., "ReAct: Synergizing Reasoning and Acting in Language Models," 2022. [arXiv:2210.03629](#)
- Anthropic, "Model Context Protocol" (open standard for tool and data connections), 2024. [modelcontextprotocol.io](#)
- Ou, "Goalless Agents" (the experiment behind Wallfacer's improve-versus-explore rhythm), 2026. [changkun.de/blog/posts/goalless-agents](#)

37. The Economics of AI

The whole stack is a way to spend money, and the bill has a shape. By the end of this chapter a reader can explain where compute is bought, why training and inference are two different kinds of cost, when to build a model versus buy one through an API, and why the claim that inference dominates the lifetime bill reaches back up into the architecture and scaling choices of [Chapter 3](#).

An invoice has a shape

Every decision in the earlier chapters resolves, eventually, into a number on an invoice. A parameter count is a training bill. An attention variant is a key-value cache, which is memory rented by the hour. A sampling budget is a column on a serving ledger. The question this chapter owns is the one a reader has been able to feel but not yet name: given that compute is the scarce input, how is it priced, how does the price split between building a model and running it, and how does that split feed back into what you should build.

Three constraints make the question hard, and the rest of the chapter is an answer to each in turn. Compute is rented in a market with volatile prices, so the cost of a FLOP is a moving target, not a constant. Training is paid once and inference is paid forever, so the two costs grow on different clocks. And the cheapest answer is often to buy capability from someone else, which means the build decision competes against a falling market price you do not control. The shape of the invoice, then, is set by two cost structures running on two clocks, by a scissors opening between them, and by a single break-even that decides which side of the market you sit on.

Two cost structures, two clocks

The organizing fact is that an AI system has two cost structures, not one, and they behave differently enough that conflating them produces wrong decisions.

Training is a capital expense. It is large, lumpy, and paid before any value is returned: you commit a cluster for weeks, spend a fixed sum, and get a set of weights. The spend is dominated by the final run plus the research and ablation runs that found the recipe. Per Epoch AI's analysis of roughly 45 frontier models, the amortized hardware and energy cost of the final training run has grown about 2.4 times per year since 2016, with hardware accounting for the largest single share, R&D staff a comparable share, and energy a small remainder ([Cottier et al. 2024](#)). The headline figures are dated and directional: GPT-4 around 40 million dollars, Gemini Ultra around 30 million dollars, and, on that trend, the largest runs crossing a billion dollars around 2027 ([Cottier et al. 2024](#)).

Inference is an operating expense. It is small per request and paid every time the model is called, so it scales with usage rather than with the training budget. A useful first-order model of its unit cost is

$$\text{cost per token} \approx \frac{\text{accelerator \$ / hour}}{\text{throughput (tokens / hour)} \times \text{utilization}}$$

which says the three levers on inference cost are the rented price of the accelerator, the throughput the serving stack extracts from it, and how fully the hardware is kept busy. Every chapter in Part VI is, read through this equation, an attack on the denominator: [Chapter 17](#) raises utilization by batching, [Chapter 18](#) and [Chapter 19](#) raise throughput, and the attention variants of [Chapter 7](#) shrink the per-token work.

The two cost structures decompose into different drivers, and Figure 37.1 lays them side by side: training resolves into the additive parts of the final run, while inference resolves into a ratio whose denominator is the target of every Part VI optimization.

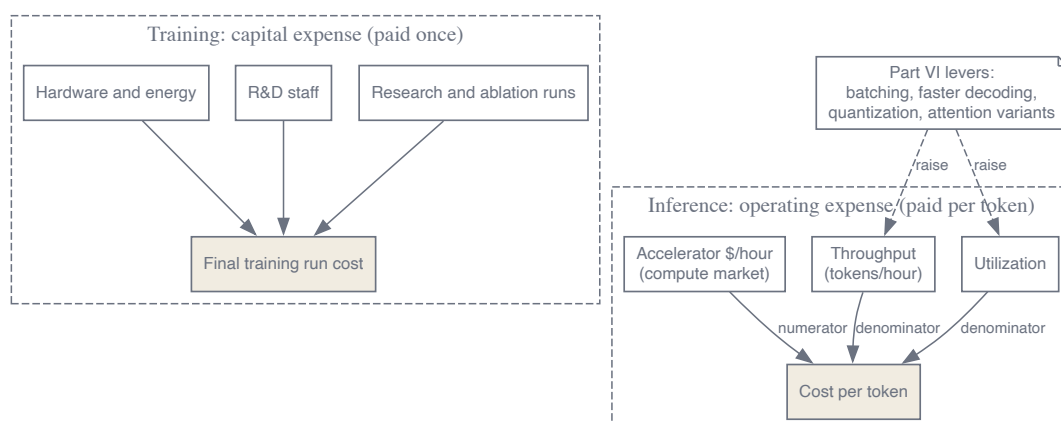


Figure 37.1. The two cost structures decomposed. Training is a sum of one-time parts paid before any value returns. Inference is a per-token ratio, and the denominator is what the serving stack attacks.

The compute market sets the numerator. Accelerators are rented by the hour across hyperscalers and a tier of specialist GPU clouds, with three price levels: reserved capacity at the lowest committed rate, on-demand at a premium for flexibility, and spot at the deepest discount in exchange for preemption. As of 2025 and into 2026 an H100 rents on demand for low single-digit dollars per GPU-hour, and far less on spot, after large provider price cuts through 2025 loosened a supply crunch. The directional fact matters more than any single quote: the price of a FLOP falls over time, and a serving plan written against last year's price is too pessimistic by a margin that compounds.

The scissors: rising training, falling inference

The two clocks do not merely tick at different rates. They run in opposite directions, and the gap they open is the most consequential economic shift in this chapter: the build-versus-buy default has inverted over the life of the field.

Early on, capability lived only inside a few labs, so building was the only way to get a frontier model and buying was not an option. As hosted APIs matured, buying became possible, and then a second curve took over: the price of a token of inference has fallen by more than an order of magnitude per year. A capability that cost tens of dollars per million tokens at the GPT-4 launch in 2023 costs a small fraction of a dollar for comparable quality a few years later. The release of strong open-weight models, DeepSeek-V3 among them, accelerated this by giving every closed provider a low-priced reference point to justify against, and triggered repeated public price cuts across the market (DeepSeek-AI 2024).

Drag the decay parameter to see how fast the per-token price collapses, the lower blade of the scissors that the build decision must race against.

The price of a token of inference falls by close to an order of magnitude per year, so a self-hosted plan competes against a market price collapsing beneath it. Set the two curves together and the scissors is plain. Training cost rises about 2.4 times per year at the frontier (Cottier et al. 2024); API inference price falls by roughly an order of magnitude per year. The widening gap between them is why the default answer drifted from build to buy. Building a frontier model from scratch competes against a market price that is collapsing under you, so building only pays when something the market cannot sell you is at stake: proprietary data that cannot leave your boundary, a latency or availability floor the API

cannot meet, a customization the hosted model will not accept, or an inference volume large enough that self-hosting beats the per-token markup.

The arithmetic of building has itself moved. DeepSeek-V3, a 671-billion parameter mixture-of-experts model, reports a final training run of about 2.788 million H800 GPU-hours, roughly 5.6 million dollars at an assumed two dollars per GPU-hour (DeepSeek-AI 2024). The number is striking, and the report is careful to say what it excludes: the prior research, the architecture and data ablations, and everything that is not the one final run. The lesson for a build decision is that the headline training number is the visible tip; the research that found the recipe is the submerged cost, and it does not appear on the run's invoice.

WHAT'S CONTESTED

Whether inference dominates the lifetime bill is the live debate, not a settled fact. The standard case is strong: training is paid once, inference is paid per request across the model's whole deployed life, and for a widely-served model the operating expense overtakes the capital expense by a large margin (Sardana et al. 2024). But reasoning models and test-time compute, the subject of Chapter 15, complicate the balance from the inference side rather than resolving it. A single request that spends a long internal chain of thought can cost orders of magnitude more than a direct answer, which inflates the per-request cost dramatically and pushes more of the total spend back toward inference, not toward training. Whether the dominant cost is many cheap calls or fewer expensive ones is unsettled and product specific. Treat the split as a measured property of a given deployment, not a universal ratio.

CONSTRAINT ARROW

Chapter 3 already named the arrow that runs from serving cost up into model sizing: a high lifetime inference volume justifies over-training a smaller model past the Chinchilla-optimal point (Hoffmann et al. 2022; Sardana et al. 2024). The economics close a second arrow that runs the other way. The unit cost of a served token, set by the market price of the accelerator divided by the throughput of the serving stack, is what sets the break-even volume in the build-versus-buy decision. Below that volume the falling API price wins and you buy; above it the per-token markup you pay a provider exceeds your own marginal cost and you build. The inference equation, not a strategy preference, is what decides which side of the line a workload sits on.

Where the knees are

Each economic choice is a balance with a knee, a point past which the cheaper option flips.

- **Capital versus operating cost.** Spending more to train, by over-training a smaller model past Chinchilla-optimal in the sense of Chapter 3, lowers every future inference bill (Sardana et al. 2024). The trade pays only if the lifetime token volume is large enough to amortize the extra training spend, so the right point depends on a usage forecast you make before you have the usage.
- **Build versus buy.** Buying through an API converts a capital expense into a per-token operating expense and rides the falling market price, at the cost of control over data, latency, and customization. Building converts it back into a capital expense plus a fixed serving footprint, which wins only past a break-even volume or where a non-price constraint forces it.
- **Reserved versus spot capacity.** Reserved compute is cheaper per hour and predictable, which fits steady production serving. Spot is cheaper still but can be preempted, which fits interruptible work: batch inference, checkpointed training, hyperparameter sweeps. Matching the workload's interruption tolerance to the pricing tier is most of the savings.

- **Quality versus unit cost.** A larger or reasoning-heavy model answers better and costs more per request. The economically right model is the smallest one that clears the task's quality bar, which is why routing a workload across a ladder of model sizes often beats sending everything to the largest.

Running the two numbers

The economics become operational through two small calculations a team actually runs, one for each clock.

The first is the inference unit cost above. To turn it into a number you need the accelerator's hourly price from the compute market, the serving stack's measured throughput in tokens per hour, and the realized utilization. The same equation explains why the optimizations of Part VI pay for themselves directly: doubling throughput or utilization halves the cost per token, and the per-token cost is what every downstream margin is computed against.

The second is the build-versus-buy break-even. Let p be the API price per token, c your own marginal cost per token from the equation above, and F the fixed cost of standing up and reserving your own serving capacity. Buying costs $p \times V$ for a volume V ; building costs $F + c \times V$. Building wins once

$$V > \frac{F}{p - c}$$

which exists as a positive threshold only when your marginal cost c is below the API price p . The break-even moves against you over time because p falls with the market, so a build that pencils out today must be re-checked against next year's lower API price before the capital is committed.

Try changing the annual price decline or the fixed cost below and watch the break-even volume climb as the API price falls toward your marginal cost.

```
import numpy as np

F = 2_000_000.0    # fixed cost to stand up and reserve serving ($)
c = 0.20           # your marginal cost per million tokens ($)
p0 = 2.00          # API price per million tokens today ($)
decline = 0.40     # fraction the API price falls each year

for year in range(4):
    p = p0 * (1 - decline) ** year
    if p > c:
        breakeven = F / (p - c)    # millions of tokens to justify buy
        print(f"year {year}: p=${p:5.2f}/Mtok  break-even = {breakeven:5.1f}")
    else:
        print(f"year {year}: p=${p:5.2f}/Mtok  build never pays (p <= c)")
```

The full decision is not one inequality but a short branching path, drawn in Figure 37.2: a non-price constraint short-circuits straight to build, the volume threshold decides the rest, and a falling p forces a recheck before any capital is spent.

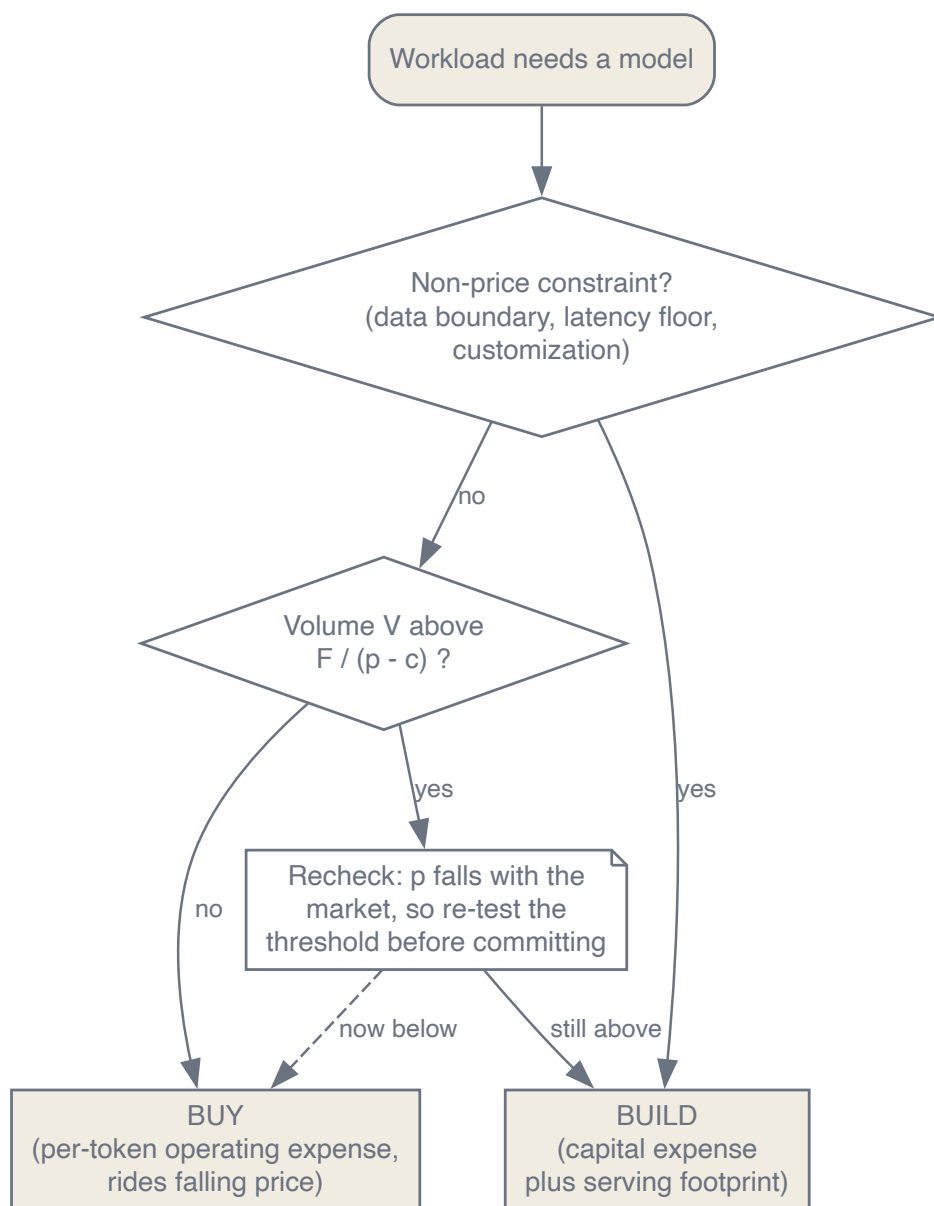


Figure 37.2. The build-versus-buy decision as a branching path. A non-price constraint forces build outright. Otherwise the volume threshold decides, and the falling API price routes the answer back through a recheck before capital is committed.

The failure modes are economic, not numerical. Building against a falling buy price is the classic one: a self-hosted plan justified at this year's API price can be underwater a year later, and the capital is already spent. Provisioning steady reserved capacity for spiky demand wastes the trough; serving spiky demand on pure on-demand or spot pays a premium or risks preemption at the peak. And forecasting inference volume too low at training time under-trains a model that will be served far more than expected, leaving a permanent per-token cost on the table that no serving optimization fully recovers.

Further reading

- Cottier et al., "How much does it cost to train frontier AI models?" (Epoch AI), 2024. epoch.ai
- Sardana et al., "Beyond Chinchilla-Optimal: Accounting for Inference in Language Model Scaling Laws," 2024. [arXiv:2401.00448](https://arxiv.org/abs/2401.00448)
- Hoffmann et al., "Training Compute-Optimal Large Language Models" (Chinchilla), 2022. [arXiv:2203.15556](https://arxiv.org/abs/2203.15556)
- DeepSeek-AI, "DeepSeek-V3 Technical Report" (training cost and GPU-hour accounting), 2024. [arXiv:2412.19437](https://arxiv.org/abs/2412.19437)
- Cottier et al., "The rising costs of training frontier AI models" (Epoch AI dataset), 2024. [arXiv:2405.21015](https://arxiv.org/abs/2405.21015)

38. Choosing a Model

The first practical decision in any stack is which model to call. It looks like a quality question and is mostly not. By the end of this chapter you can place a model on the closed-to-open axis, read its license for what it actually permits, triangulate three leaderboards without being fooled by any one of them, run a small eval that decides the call, and pin and wire the result so that a vendor's weekly point release does not silently change your product. This chapter is the hands-on companion to [Chapter 35](#), which frames the openness axis, and to [Chapter 27](#), which frames why a published score is a claim about a model and its harness together.

DATED SNAPSHOT: VERIFY BEFORE YOU QUOTE

Every model name, price, context length, and ranking below is a snapshot as of **mid-2026 (2026-06-20)** and is attributed to a primary source where one exists. Versions ship weekly, prices change, and rankings rotate inside overlapping confidence intervals. Treat every specific number here as a *hypothesis to confirm against the linked primary doc*, never as a timeless fact. The durable content is the categories, the trade-offs, and the decision and wiring procedure. Read those for the method; re-check the numbers before you commit budget.

The master axis: rent or own

One axis predicts almost everything else about a model choice, and it is not quality. It is whether you *rent* the capability behind a metered API or *own* the weights and run them yourself.

A **frontier hosted model** is the most capable general model a lab is willing to expose, served behind a per-token API. You send tokens, they bill you, you never hold the weights. The lab owns the serving stack: batching, KV-cache, speculative decoding, autoscaling, and the accelerators ([Chapter 39](#)). You inherit their latency distribution and their rate limits, and you consume the serving optimizations as *knobs* (cached-input pricing, batch and priority tiers) rather than operating them.

An **open-weight model** is one whose trained parameters you can download and run on your own hardware. You own the operational burden and, in exchange, you control cost, latency, data residency, and the right to fine-tune. The model family you pick sets what your inference server (vLLM, SGLang, llama.cpp; see [Chapter 39](#)) must load, how much VRAM it needs, and the throughput you can reach.

That is the same trade the book draws throughout: rent capability and give up control of the serving stack, or host weights and own the operations. In 2026 the gap that used to make this a quality compromise has mostly closed. The best open weights now sit within a point or two of closed frontier models on coding and reasoning boards, which turns "self-host versus call an API" into a genuine architecture decision rather than a concession (per Artificial Analysis and open-LLM leaderboards, as of 2026-06).

Read the license before the benchmark

For open weights, the license decides more than the benchmark does, because a license can disqualify a model for your product no matter how well it scores. Three tiers matter.

- **Truly open (Apache-2.0, MIT).** No user caps, no field-of-use limits, no geographic carve-outs. Qwen, DeepSeek, Mistral's current flagship, Gemma 4, gpt-oss, GLM, and OLMo sit

here as of mid-2026. Pick these when you might scale past tens of millions of users, operate in the EU, or want zero field-of-use ambiguity.

- **Source-available (restricted).** Weights are downloadable but the license adds caps or carve-outs. Meta's Llama 4 Community License permits commercial use only below **700M monthly active users** and **excludes multimodal features for EU-domiciled licensees**, with an acceptable-use policy on top. This is not OSI-open. Read the actual `LICENSE` before you build on it.
- **Closed (no weights).** The hosted-API frontier. You get a contract and an endpoint, not parameters.

A second, orthogonal distinction is **open-weight versus fully-open**. Most "open" models release weights only; the training data and full recipe stay private. A smaller set (AI2's OLMo) releases data, code, and checkpoints too. If you need reproducibility, auditability, or the ability to retrain from scratch, that difference is the whole game, and it is independent of the license tier.

The durable rule: for anything that might scale or ship in a regulated region, read the license first and the leaderboard second.

The players, dated and attributed

The two tables below are a mid-2026 snapshot. Pricing is per-1M-token input/output for standard synchronous calls; batch tiers typically run about half price and cached input is heavily discounted. Treat every cell as "verify against the primary doc."

Frontier hosted models

Provider (family)	License / access	Best for	Snapshot note (as of 2026-06)
OpenAI (GPT-5.x)	Closed API; also Azure OpenAI, now Amazon Bedrock	Coding, broad agentic work, widest ecosystem	GPT-5.5 flagship ~\$5 / \$30, ~1.05M ctx; pro variant higher. Prompts over ~272K billed at a premium. Usage tiers by cumulative spend. [official]
Anthropic (Claude)	Closed API; also Bedrock, Vertex AI, Microsoft Foundry	Long-horizon agentic coding, reliability, safety-sensitive work	Opus 4.8 ~\$5 / \$25, 1M ctx, GA; Sonnet 4.6 ~\$3 / \$15, 1M ctx; Haiku 4.5 ~\$1 / \$5. Model IDs are pinned snapshots. Fable 5 / Mythos 5 announced but access-restricted (see below). [official]
Google (Gemini)	Closed API (AI Studio / Gemini API) + Vertex AI	Native multimodal (text, image, audio, video, PDF), GCP shops	Gemini 3.1 Pro ~\$2 / \$12 (≤200K), more above; ingests the widest set of modalities of the majors. 3.5 rolling out. Context: card ~1.05M; some trackers cite 2M (treat as unverified). [official]
xAI (Grok)	Closed API (x.ai), X integration	Cost-aware long context, real-time/social data	Grok 4.3 ~\$1.25 / \$2.50, 1M ctx; Grok 4.1 Fast ~\$0.20 / \$0.50, up to ~2M ctx. Aggressive price/perf. [aggregator]
DeepSeek	Hybrid: open weights + cheap first-party API	Cheapest frontier-adjacent quality, cost-sensitive batch	V3.2 ~\$0.14 / \$0.28, ~128-164K ctx, large cached-input discount. Governance scrutiny in some orgs; verify policy fit. [aggregator]
Amazon (Nova)	Closed, via Amazon Bedrock	AWS-native price/performance, custom builds (Nova Forge)	Positioned on price/performance, not top-of-board quality. Bedrock is also the access layer for Anthropic, Meta, Mistral, Cohere, OpenAI. [aggregator]
Mistral	Hybrid: some open weights + hosted API; also Bedrock/Vertex/Azure	EU option, multilingual, EU data residency	Mistral Large 3 flagship. Strong sovereignty positioning; not consistently at the absolute frontier on public boards. [aggregator]
Cohere	Closed API; also Bedrock/Vertex/Azure	Enterprise RAG, retrieval + rerank, VPC deploy	Command A for generation, Embed v4 a common retrieval default. Enterprise-focused rather

Provider (family)	License / access	Best for	Snapshot note (as of 2026-06)
			than chasing arena rank. [aggregator]

Open-weight families

Parameter notation is `Total / Active` for Mixture-of-Experts ([Chapter 7](#)); `A3B` means about 3B active parameters per token.

Family (origin)	License	Best for	Snapshot note (as of 2026-06)
Qwen3.5 / 3.6 (Alibaba)	Apache-2.0	Best all-round open family; agentic coding, vision, 201 languages	Size ladder 0.8B to 397B-A17B ; thinking toggle. Qwen3.6-35B-A3B current mid-size open variant. Prototype on a laptop, scale to a cluster, same license.
DeepSeek-V4	MIT	Agentic coding at the open-weight frontier	V4-Pro 1.6T/49B (1M ctx), V4-Flash 284B/13B. Three reasoning modes. Cluster-class. R2 reasoner not released as of 2026-06.
Llama 4 (Meta)	Community License (restricted)	Ultra-long context, mature ecosystem	Scout 109B/17B (10M ctx), Maverick ~400B/17B (1M ctx). 700M-MAU cap; multimodal excluded for EU licensees. Source-available, not OSI-open.
Mistral Large 3 / Small 4 (Mistral)	Apache-2.0	Permissive EU-built; Small 4 merges reasoning+vision+code	Large 3 675B/41B, 256K ctx. Small 4 ~119B/6B, single mid-size box. Flagship moved to Apache-2.0.
Gemma 4 (Google)	Apache-2.0	Best capability-per-byte small/mid; single GPU	E4B , 12B , 26B-A4B (MoE), 31B dense; text+image (+audio on some). Moved off custom Gemma terms to Apache-2.0.
gpt-oss (OpenAI)	Apache-2.0	Reasoning + tool-use on one 80GB GPU (120b) or 16GB edge (20b)	gpt-oss-120b 117B/5.1B, gpt-oss-20b 21B/3.6B; text-only. Released 2025-08, not new in 2026.
Kimi K2.6 (Moonshot)	Modified-MIT	Long-horizon coding, multi-agent orchestration	1T/32B active, 256K ctx, native multimodal. Attribution clause for very large deployments; read the LICENSE .
GLM-5.x (Z.ai)	MIT	Long-horizon agentic engineering	GLM-5.1 744B/40B, 200K ctx. GLM-5.2 (1M ctx) announced; weights timing uncertain.
OLMo 3 (Ai2)	Apache-2.0 (fully open)	Reproducibility, auditability	7B and 32B; Base/Think/Instruct. Releases data (Dolma 3), code, checkpoints. The only fully-open frontier-adjacent option.

CONSTRAINT ARROW: SERVING COST REACHES UP INTO MODEL CHOICE

For open weights, the serving layer below dictates the model above. A Mixture-of-Experts model gives you near-dense inference cost at a higher quality ceiling, but the

total parameters still have to fit in VRAM, so a 1.6T/49B model is a cluster model even though only 49B are active per token. A 31B dense model can be simpler to serve and quantize (Chapter 19) than a 122B-A10B MoE despite similar active compute. The hardware you have, not the benchmark, narrows your shortlist first.

Reading leaderboards without being fooled

A leaderboard is a shortlisting tool, not a verdict. The single most important practitioner lesson from Chapter 27 is that **a published score is a claim about a model and its measurement harness together, not about the model alone**. Independent re-runs of identical weights swing 10 to 20 percentage points depending on prompt template, system prompt, tool scaffolding, sampling parameters, and grader. The recurring 2026 pattern: a model's score on a public, verifiable coding board runs markedly higher than the same weights on a held-out harness, and that gap is the harness plus any contamination, not a change in the model.

So read three boards, not one, and match the board to the question you are actually asking.

Question you are asking	Board to read	What it cannot tell you
"Do users prefer this in open-ended chat?"	LMarena / arena.ai (human-preference Elo)	Correctness. Preference rewards confident, longer, well-formatted answers; high Elo with wrong answers is a real failure mode.
"Best intelligence per dollar at my latency?"	Artificial Analysis (Intelligence Index, standardized harness, cost and tokens/sec axes)	Whether the standardized harness matches <i>your</i> prompts, tools, and sampling.
"Is a high public score inflated?"	Contamination-resistant boards: FrontierMath, LiveCodeBench, Epoch, Scale SEAL	Broad coverage; these update slower and cover fewer tasks.
"Can it run my agent?"	Verifiable agentic boards: SWE-bench Verified, Terminal-Bench, τ -bench, Aider polyglot, BFCL	Single-shot knowledge quality; these exercise the scaffold (Chapter 29).
"Did a new version regress?"	Saturated legacy boards (MMLU, HumanEval) as a floor only	Frontier ranking; the top cluster is noise here.

Agreement across the three is a strong signal. Disagreement tells you exactly which axis is fragile. The tell for contamination is a large gap between a model's public-set score and its held-out or post-cutoff score: a small gap is a credibility signal, a large gap is a red flag.

WHAT'S CONTESTED: BEST, AVAILABLE, AND ALLOWED ARE DIFFERENT AXES

In mid-2026 the top text cluster sits within overlapping confidence intervals and rotates weekly, so any single "#1" claim is a *tier*, not a winner. Worse, rank is necessary but not sufficient. The canonical reminder: Anthropic's Fable 5 and Mythos 5 topped multiple boards on announcement (2026-06-09), and access was **suspended on 2026-06-12** under a US export-control directive (CNBC, Al Jazeera, Fortune, Anthropic). A board-topping model can be export-controlled, region-locked, rate-limited, or priced out of your budget. Choose among models that are *available to you and allowed by your policy*, then rank within that set. Do not default to a model you cannot reliably call.

A decision framework

The decision is rarely "which scores highest." It is the intersection of where you are billed, the shape of the task, context-versus-cost, governance, and (for open weights) the hardware you have. [Figure 38.1](#) walks the order in which these actually decide the call.

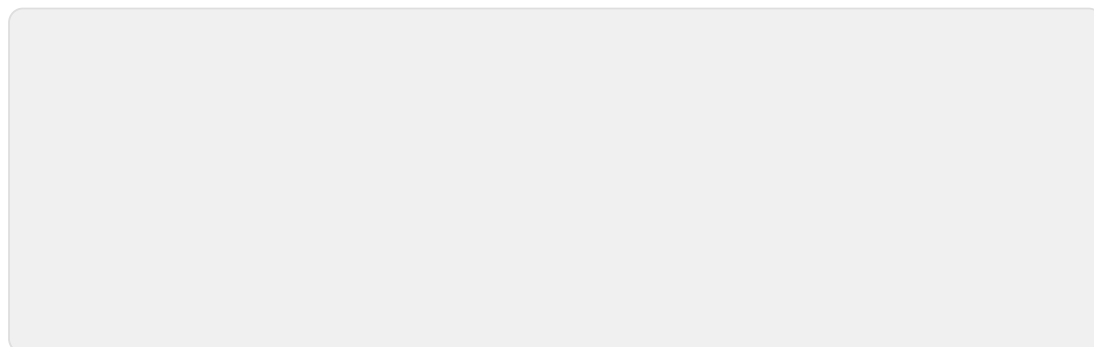


Figure 38.1. A decision order for picking a model in mid-2026. The cloud you are billed through and your governance constraints usually decide before quality does.

Stated as rules of thumb, in roughly the order they decide:

- **Pick by where you live first.** The cloud you are billed through usually wins on procurement, data residency, and committed-use discounts. Heavy AWS shop to Bedrock; Azure to Azure OpenAI; GCP to Vertex AI.
- **Pick Claude (Opus 4.8 / Sonnet 4.6) when** the workload is agentic coding, long-running tool use, or safety-sensitive, and you value reliability and pinned, reproducible model IDs.
- **Pick GPT-5.5 when** you want the widest ecosystem and tooling or you are on Azure.
- **Pick Gemini 3.1 Pro when** you are on GCP or the input is genuinely multimodal (audio, video, PDF in one model).
- **Pick Grok or DeepSeek when** price-per-token and very long context dominate and governance allows it.
- **Pick an open family (default Qwen3.5/3.6) when** you must control cost, latency, residency, or the right to fine-tune ([Chapter 40](#)), and you have the hardware. Choose DeepSeek-V4 for agentic coding at the frontier, Gemma 4 for single-GPU, OLMo 3 when reproducibility is a hard requirement.

Long context deserves a separate caution. It is expensive: OpenAI bills prompts over ~272K at a premium, Gemini roughly doubles input price above 200K. Prefer retrieval over stuffing the window ([Chapter 42](#)) when you can; it is usually cheaper and more reliable than relying on raw context size.

Try moving the slider in code: change the threshold and the over-cliff multiplier to see where stuffing the whole window stops paying off versus retrieving a fixed small slice.


```

import numpy as np
import matplotlib.pyplot as plt

base = 2.0 / 1e6          # $ per input token below the cliff
threshold = 200_000       # tokens; price kink here
mult = 2.0                # over-cliff tokens cost this much more (
retrieved = 8_000         # fixed small window when you retrieve in

n = np.linspace(0, 600_000, 400)
over = np.maximum(n - threshold, 0)
stuff = base * (np.minimum(n, threshold) + over * mult) # tiered, n
retrieve = np.full_like(n, base * retrieved)           # retrieval
plt.plot(n / 1e3, stuff, label="stuff the window")
plt.plot(n / 1e3, retrieve, "--", label=f"retrieve {retrieved // 100}
plt.axvline(threshold / 1e3, color="gray", ls=":", label="price cliff")
plt.xlabel("relevant tokens (thousands)"); plt.ylabel("cost per call")
plt.legend(); plt.title("Long-context cliff vs retrieval"); plt.show()
print(f"At 400K tokens: stuff=${base*(threshold+200_000*mult):.3f},
      f"retrieve=${base*retrieved:.3f}")

```

A sensible default, and why

As of **mid-2026**, for a typical product team building agent and RAG features and not locked to a specific cloud:

- **Hosted default: Anthropic Claude.** Sonnet 4.6 as the everyday workhorse, escalating to Opus 4.8 for the hardest agentic and coding steps. Both are generally available with pinned model IDs, 1M context, and a strong agentic-coding track record across all four major access layers. Note the reconciliation from the contested box above: do **not** default to Fable 5 or Mythos 5 despite their board-topping scores, because access was suspended on 2026-06-12. Best on a board is not the same as available to call.
- **Strong alternatives:** GPT-5.5 for the widest ecosystem or Azure; Gemini 3.1 Pro for GCP or multimodal.
- **Self-hosted default: the Qwen3.5/3.6 family (Apache-2.0)**, the most well-rounded truly-open lineage, with DeepSeek-V4 for frontier agentic coding, Gemma 4 for single-GPU, and OLMo 3 when you need full reproducibility.

Do not hard-code one vendor. Put a gateway in front so you can route per task and fail over, and re-evaluate the default every quarter; the gap between the top few is small and the versions churn. This default is a starting posture, not a verdict. The verdict is your own eval.

Pin and eval: the part that actually decides

Public boards narrow the field to three or four candidates. Only your own offline eval picks the winner, because only your eval matches your distribution, your prompts, your tools, and your acceptance bar.

Two disciplines make this real. **Pin every model ID** in config to a snapshot, never "latest," so a vendor's point release cannot silently change behavior. **Treat your eval set as a versioned contract** in the repo, like a test suite: a model swap is a diff against it, a prompt change is a

diff against it, and CI runs it before any bump. Build it from real traffic (50 to 200 items is enough to start), score with verifiable graders where possible (exact match, unit tests) and an LLM judge only where unavoidable (Chapter 28), and always report cost and latency next to accuracy, never accuracy alone. Logged production failures become next quarter's eval items.

```
# eval.yaml (versioned in-repo; CI gate on every model or prompt change)
dataset: ./evals/support_v7.jsonl      # private, from real traffic, v7
candidates:                             # shortlist from the public board
  - anthropic/claude-opus-4-8          # pinned snapshot IDs, never "latest"
  - openai/gpt-5.5
  - google/gemini-3.1-pro
graders:
  - type: exact_match                  # verifiable wherever possible
  - type: unit_test                    # for code-shaped outputs
  - type: llm_judge                    # sparingly; pin the judge + rubric
    judge_model: pinned-strong-model
    rubric: ./evals/rubric_v3.md
report:
  axes: [accuracy, cost_per_task, p95_latency]  # never accuracy alone
  ci: bootstrap_95                          # report intervals; a single number
gate:
  block_merge_if: accuracy_drop > 2pct OR p95_latency_regression > 1
```

Wiring it into the stack

The app should never talk to a raw vendor SDK. It talks to a gateway that holds keys, enforces per-team budgets and rate limits, normalizes the request shape, and fails over between providers (Chapter 44). For closed APIs there is no serving layer you operate, so the gateway *is* your control point; for open weights the gateway sits in front of your own inference server, which exposes the same OpenAI-compatible API, so the agent code does not change between them. Figure 38.2 shows the seam.

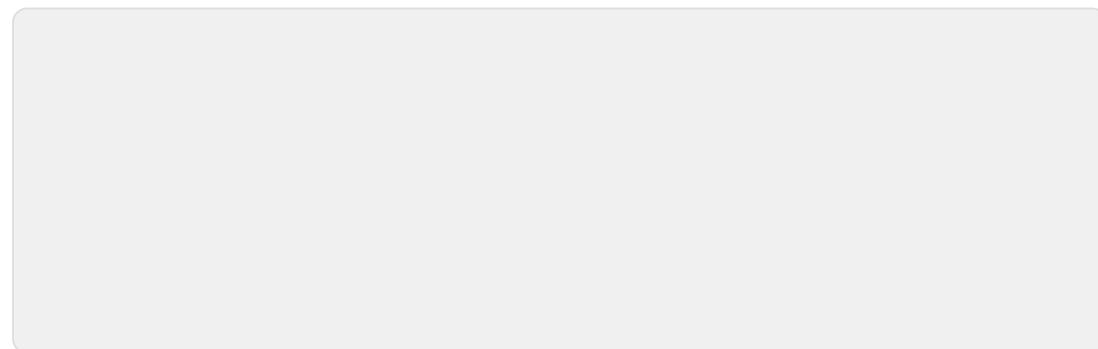


Figure 38.2. Where the model choice plugs in. The app talks only to the gateway; pinned model IDs and the eval gate sit between the boards and production.

A provider-agnostic routing config makes the per-task picks and the fallback chain explicit, and guards the long-context price cliff:

```
# gateway routing (illustrative; every model is a pinned snapshot ID
default_model: anthropic/claude-sonnet-4-6
routes:
  - match: { task: hard_coding }      # escalate the gnarly agentic
    model: anthropic/claude-opus-4-8
  - match: { task: multimodal }      # audio / video / pdf in one m
    model: google/gemini-3.1-pro
  - match: { task: bulk_cheap }      # high-volume, cost-capped
    model: deepseek/deepseek-v3.2
fallbacks:
  anthropic/claude-opus-4-8: [openai/gpt-5.5, google/gemini-3.1-pro]
limits:
  long_context_guard: 200000          # warn before the >200K price
```

The capability, efficiency, trust lens closes the chapter. *Capability* is what the boards measure and your eval confirms. *Efficiency* is cost-per-task and latency, which is why no rank is meaningful without the price and speed axis next to it. *Trust* is the license, the governance posture, and the availability guarantee: the Fable 5 suspension is the reminder that a model you cannot reliably and lawfully call has no capability for you at all. Pick on all three, pin the result, and let your eval, not a leaderboard, cast the deciding vote.

Further reading

First-hand sources first.

- Anthropic, models overview (Opus 4.8, Sonnet 4.6, Haiku 4.5; IDs, context, pricing). platform.claude.com/docs
- Anthropic newsroom (Opus 4.8 on 2026-05-28; Fable 5 / Mythos 5 on 2026-06-09; suspension note 2026-06-12). anthropic.com/news
- OpenAI, "Introducing GPT-5.5" and API pricing. openai.com/index/introducing-gpt-5-5, openai.com/api/pricing
- Google Gemini API pricing and changelog. ai.google.dev/gemini-api/docs/pricing, [changelog](https://ai.google.dev/gemini-api/docs/changelog)
- Mistral 3 announcement (Large 3, Ministral 3, Apache-2.0 shift). mistral.ai/news/mistral-3
- Gemma 4 model card (Apache-2.0, sizes). ai.google.dev/gemma/docs/core/model_card_4
- Qwen org and model cards. huggingface.co/Qwen, qwen.ai/blog
- DeepSeek-V4-Pro card. huggingface.co/deepseek-ai/DeepSeek-V4-Pro
- OLMo 3 (fully open: weights, data, code). allenai.org/blog/olmo3
- Llama 4 license and MAU terms (cross-checked coverage). datacamp.com/blog/llama-4
- LMArena / Arena (human-preference Elo). arena.ai/leaderboard
- Artificial Analysis Intelligence Index methodology (Index v4.1, standardized harness, cost and speed). artificialanalysis.ai/methodology
- Epoch AI benchmarks (FrontierMath, GPQA Diamond, Epoch Capabilities Index). epoch.ai/benchmarks

- SWE-bench official (Verified, Lite, Multilingual, Multimodal). swebench.com
- LiveCodeBench (date-tagged, contamination-resistant). livecodebench.github.io
- Scale SEAL leaderboards (private, held-out). labs.scale.com/leaderboard
- Export-control event reporting. [CNBC](#), [Al Jazeera](#), [Fortune](#)

39. Serving, Gateways, and Compute

This is a hands-on chapter. The theory of why serving is hard lives in [Chapter 16](#) and [Chapter 17](#); here we choose tools. By the end you should be able to stand up a self-hosted endpoint, decide when not to, put a gateway in front of it for keys and cost control, and place the whole thing on compute you can actually pay for. The chapter works top down through three layers that stack into one request path: the gateway you talk to, the serving engine that decodes tokens, and the GPUs underneath.

AS OF MID-2026

Everything below is a dated snapshot. Tool rankings, version numbers, and prices move every quarter. Benchmark deltas are from third-party blogs and are workload-specific, so treat them as directional, not as facts, and re-verify current values before you commit. The durable content here is the categories, the trade-offs, and the wiring, not any single number.

The three layers

A self-hosted inference stack is three layers with one interface running through all of them. [Figure 39.1](#) shows the request path.

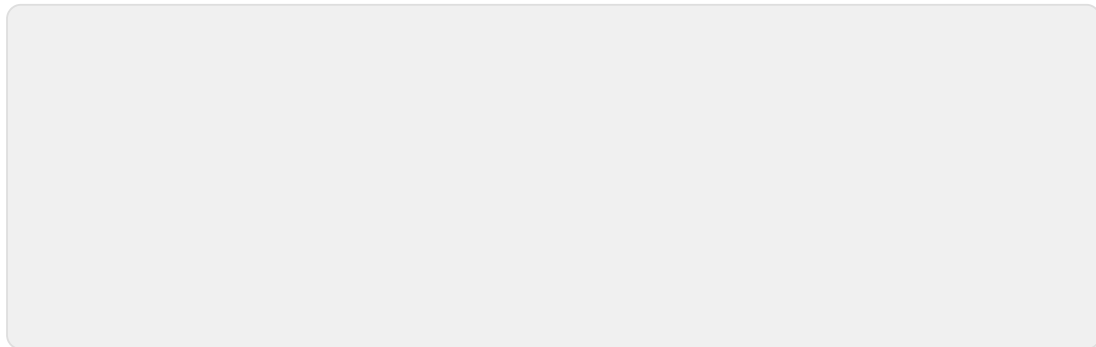


Figure 39.1. The request path through a self-hosted stack. One OpenAI-compatible interface runs from the app down to the engine, so any layer can be swapped without rewriting the others.

The interface is the key to the whole design. By 2026 almost every engine and gateway speaks the OpenAI HTTP shape (`/v1/chat/completions` , `/v1/completions` , `/v1/embeddings`). That single convention is what lets you swap a self-hosted model for a hosted one, put a gateway in front of many engines uniformly, and point an eval harness ([Chapter 43](#)) at the same URL as production. Build against it and keep every layer replaceable.

Layer 1: the serving engine

A serving engine turns a set of weights into a service: it loads the model onto accelerators, batches requests, runs the forward passes, manages the key-value cache, and streams tokens back. This is where most controllable cost and most user-visible latency live. The same weights under two engines, or one engine with two configs, can differ by an order of magnitude in tokens per second and tail latency.

Two phases drive every design decision (Chapter 16). **Prefill** processes the whole prompt in parallel and is compute-bound. **Decode** emits one token at a time, each step re-reading the cache, and is memory-bandwidth-bound. The features that matter are answers to the tension between them: **continuous batching** (add and retire requests every step, table stakes by 2026), **PagedAttention** (store the cache in non-contiguous pages so memory is not fragmented), **prefix caching / KV reuse** (compute a shared leading prefix once, the dominant win for agents and RAG, generalized to a radix tree by SGLang's RadixAttention), **quantization** (Chapter 19: FP8, FP4, INT4 via AWQ/GPTQ, GGUF K-quants), **speculative decoding** (Chapter 18), **structured output** (grammar-constrained decoding via XGrammar), and **prefill/decode disaggregation** (separate worker pools for the two phases, the headline datacenter pattern of 2026).

The players

The table below is a mid-2026 snapshot. Licenses and "best for" are durable; the benchmark notes are third-party and workload-specific.

Engine	What it is	License / HW	Best for
vLLM	The de-facto open-source default. Origin of PagedAttention, broadest model and feature coverage, "V1" core re-architected for lower overhead.	Apache-2.0, GPU	General-purpose GPU serving; the safe default
SGLang	High-performance engine built on RadixAttention (radix-tree prefix reuse) plus a frontend for structured LLM programs.	Apache-2.0, GPU	Prefix-heavy traffic (agents, RAG, multi-turn), large MoE
TensorRT-LLM	NVIDIA's optimized engine; compiled kernels plus a now-default PyTorch backend that loads HF weights without a build step.	Apache-2.0, NVIDIA only	Max throughput/latency on Hopper/Blackwell, FP8/FP4
LMDeploy	InternLM/OpenMMLab engine with a C++ TurboMind backend and aggressive built-in w4a16 quantization.	Apache-2.0, GPU	Low-TTFT quantized serving, INT4 70B-class
llama.cpp	Portable C/C++ engine for GGUF quantized models. Runs on CPU, CUDA, ROCm, Vulkan, Metal.	MIT, portable	Local / edge / CPU / mixed-vendor; max portability
Ollama	Developer-friendly wrapper (registry + CLI + OpenAI API) over llama.cpp; moving to an MLX backend on Apple Silicon (preview).	MIT, portable	Local dev, laptops, fastest path to localhost
MLX / mlx-lm	Apple's framework for Apple Silicon (unified memory, zero-copy).	MIT, Apple only	Fastest single stream on Macs
NVIDIA Dynamo	Not an engine: a datacenter orchestrator above engines. Disaggregation, KV-aware routing, NIXL transport, memory tiering.	Apache-2.0, NVIDIA	Multi-node, SLO-driven fleets at scale
llm-d	Kubernetes-native distributed stack: vLLM workers + KV-aware routing + disaggregation + Inference Gateway.	Apache-2.0 (CNCF Sandbox)	K8s-native fleets without going all-NVIDIA
Hugging Face TGI	Formerly a leading server. Archived March 2026, maintenance mode; HF now points users to vLLM/SGLang/llama.cpp/MLX.	Apache-2.0, archived	Legacy only; do not start here

As of mid-2026, secondary benchmarks report SGLang ahead of vLLM on some H100 single-model tests (one cites roughly 29 percent) with larger gains on prefix-heavy shapes, the gap narrowing at 70B scale; LMDeploy near SGLang on throughput with best-in-class TTFT for INT4 70B; MLX 30 to 50 percent faster than llama.cpp on some Apple Silicon models. These come from vendor and blog benchmarks that vary by model, GPU, batch shape, and tuning, and they age fast. Verify on your own workload before treating any of them as a tiebreaker.

How to choose an engine

The decision is mostly a function of where you run, what your traffic looks like, and how much performance engineering you can afford.

- **Pick vLLM** when you want one safe, broadly-supported default on GPUs, need a wide range of models and features now, and value ecosystem maturity over the last 20 percent of performance. This is the right answer for most teams.
- **Pick SGLang** when your traffic is prefix-heavy (agents, RAG, multi-turn) or you serve large MoE models, and you want top throughput on those shapes. Verify the win on your own traffic first.
- **Pick TensorRT-LLM** when you are all-NVIDIA, latency and throughput are first-order costs, and you can invest in the toolchain (the default PyTorch backend lowers that cost versus the old build flow).
- **Pick LMDeploy** when quantized w4a16 serving and low TTFT on a NVIDIA fleet matter most.
- **Pick llama.cpp** when you need portability (CPU, AMD via Vulkan/ROCm, Metal), edge or local deploys, and GGUF quant, and you do not need datacenter-grade concurrency.
- **Pick Ollama or MLX** when the target is a laptop or a few internal users; pick MLX specifically on Apple Silicon for the fastest single stream.
- **Reach for Dynamo or llm-d** when you have outgrown a single node and need KV-aware routing, disaggregation, and SLO-driven autoscaling across a fleet.
- **Do not start on TGI** in 2026; it is archived and maintenance-only.

Sensible default (mid-2026): vLLM behind an OpenAI-compatible endpoint, prefix caching on, an appropriate quantization (FP8 on Hopper/Blackwell, or AWQ/GPTQ INT4 to fit a bigger model on smaller cards), and XGrammar-backed structured output for any tool-calling or extraction path. It is the broadest, lowest-regret choice. For local and dev, the default flips to Ollama or llama.cpp on Linux/Windows and MLX on Macs. Those are single-node, low-concurrency tools; do not promote them into a high-traffic path.

A minimal single-node vLLM launch:

```
vllm serve <model-id> \  
  --enable-prefix-caching \  
  --quantization fp8 \  
  --guided-decoding-backend xgrammar  
# exposes an OpenAI-compatible server on :8000
```

Flag names and structured-output parameters drift between vLLM versions; check the docs for your release.

Layer 2: the gateway

A model gateway is the control plane between your application code and the many model providers it talks to. In the thinnest framing it is a reverse proxy that speaks one API and fans requests out to OpenAI, Anthropic, Google, Bedrock, Azure, your own vLLM, and aggregators like OpenRouter. That undersells it. The gateway is where five concerns nobody wants scattered across application code get centralized:

1. **Unified API.** One request schema, so swapping `gpt-5` for `claude-opus-4` or a self-hosted variant is a config change, not a code change.
2. **Key custody.** Provider secrets live on the server. Apps and agents hold short-lived *virtual keys* scoped to a model allowlist and a budget, never the raw provider credential.

3. **Cost tracking.** Per-key, per-team, per-tag spend, attributed and exportable. This is the single most cited reason teams adopt a gateway.
4. **Fallback and routing.** Retries, provider failover, load balancing, context-window fallback, cost- or latency-aware model selection.
5. **Budgets and policy.** Hard spend caps, rate limits, guardrails (PII redaction, jailbreak filtering), and an audit trail.

The gateway is the seam between the serving layer and everything above it. The newest gateways extend the idea past plain LLM calls into agent-native protocols (MCP for tools, A2A for agent-to-agent), so the gateway becomes the policy point for tool and agent traffic too, which connects directly to the harness work in [Chapter 41](#) and the authorization model in [Chapter 34](#).

The players

Gateway	What it is	Hosting	Best for
LiteLLM	Python SDK + self-hostable proxy translating 100+ providers to OpenAI format; virtual keys, budgets, spend logs, UI.	Self-host (OSS core) + Enterprise tier	Broadest provider coverage, Python stack, DevOps in place
OpenRouter	Managed aggregator: one endpoint, prepaid credits, 300+ models, BYOK.	Managed SaaS	Solo devs and small teams wanting zero-setup catalog access
Portkey	Gateway with guardrails, virtual keys, fallbacks, caching, prompt management, observability.	OSS core + managed	Enterprises needing governance: PII redaction, audit, compliance
agentgateway	Rust data plane for HTTP/gRPC and AI-native protocols: LLM + MCP + A2A in one.	Self-host (Apache-2.0, Linux Foundation)	Platform teams wanting one proxy for LLM, tool, and agent traffic
Envoy AI Gateway	Kubernetes-native gateway on Envoy + Gateway API; token rate limiting, InferencePool.	Self-host (OSS)	Teams already on Envoy/Istio/Gateway API
Kong AI Gateway	LLM routing as plugins in Kong's API platform; PII redaction, semantic caching.	OSS core + Enterprise	Orgs already running Kong
Cloudflare AI Gateway	Platform-native gateway: observability, caching, rate limiting, cost tracking; Unified Billing.	Managed SaaS	Teams already on Cloudflare/Workers
Helicone	OSS gateway/proxy, observability-first logging and analytics; Rust runtime.	OSS + hosted	Logging and analytics with low overhead (often paired with LiteLLM)
Bifrost (Maxim AI)	Go gateway: adaptive load balancing, cluster mode, guardrails; markets very low overhead.	OSS + commercial	High-throughput self-hosted LiteLLM alternative
Lux (latere.ai)	The author's own gateway: one address to OpenAI/Anthropic/Gemini/OpenRouter/Ollama; sealed credentials, virtual keys, spend caps, hooks, audit.	Latere product	The Latere stack; a worked example here, not a neutral pick

Several vendors publish dramatic overhead numbers (Bifrost "50x faster than LiteLLM" and " $\sim 11\mu s$ "; Portkey " $< 1ms$ "; LiteLLM " $\sim 8ms$ P95 at 1k RPS"). These are vendor benchmarks under inconsistent conditions and are not independently verified. The durable point: compiled-language gateways (Go/Rust: Bifrost, agentgateway, Helicone, Envoy) generally claim lower proxy overhead than the Python LiteLLM proxy, but for most workloads provider and model latency dominate gateway overhead by orders of magnitude. Benchmark on your own traffic before treating overhead as a tiebreaker.

How to choose a gateway

Work two axes in order: managed versus self-hosted, then plain-LLM versus agent-native.

- **Pick OpenRouter** when you are a solo dev or small team and want the largest catalog with no infra and no contracts.
- **Pick LiteLLM** when you want a self-hosted, broadly compatible gateway and have the DevOps to run it, especially on a Python stack.
- **Pick Portkey** (or Kong if you already run Kong) when governance, guardrails, and compliance are requirements, not nice-to-haves.
- **Pick agentgateway** when your roadmap is agentic and you need LLM, MCP, and A2A traffic under one governed policy point.
- **Pick Envoy AI Gateway** when your platform team already lives in Envoy/Istio/Gateway API and wants the AI control plane in the same CRDs.
- **Use a managed gateway** (OpenRouter, Cloudflare, Portkey managed) when you want zero infrastructure and accept that keys and request metadata transit a third party. Move to self-hosted when key custody, audit, or compliance forces it.

Sensible default (mid-2026): self-host LiteLLM as the gateway, paired with an observability layer (Helicone or LiteLLM's own dashboard) for spend and request analytics. It gives you the broadest provider coverage, virtual keys with budgets, spend tracking, and fallbacks, all open source and in your own environment, and almost everything else in the ecosystem interoperates with it. If your roadmap is agent-heavy, evaluate agentgateway early so LLM, tool, and agent traffic share one policy point instead of bolting agent concerns onto a chat-only proxy later.

Disclosure: the author's own stack uses **Lux** as its gateway. It appears here as a worked example of these patterns, not as a neutral recommendation over the options above.

A minimal LiteLLM config that unifies a hosted and a self-hosted model behind one name and adds a fallback:

```

model_list:
  - model_name: smart          # the name your app requests
    litellm_params:
      model: anthropic/claude-opus-4
      api_key: os.environ/ANTHROPIC_API_KEY
  - model_name: smart          # same alias, load-balanced / failb
    litellm_params:
      model: openai/gpt-5
      api_key: os.environ/OPENAI_API_KEY
  - model_name: local
    litellm_params:
      model: openai/<model-id>  # your vLLM, OpenAI-compatible
      api_base: http://vllm-svc:8000/v1

litellm_settings:
  fallbacks: [{ "smart": ["local"] }]  # if hosted "smart" fails, u
  num_retries: 2

# virtual keys, team budgets, and spend caps are issued via the
# /key/generate API and the proxy UI, not this file.

```

The application then points a standard OpenAI client at the proxy and requests model "smart". The gateway resolves the alias, load-balances or fails over, records spend against the virtual key, and emits traces. Upstream, "local" resolves to your Layer-1 engine, so self-hosted and hosted models become interchangeable behind one name:

```

from openai import OpenAI
client = OpenAI(base_url="https://gateway.internal/v1", api_key="<vi
resp = client.chat.completions.create(
    model="smart",
    messages=[{"role": "user", "content": "..."}],
)

```

Schema drifts between LiteLLM versions; check the docs for the exact keys.

Layer 3: the compute under it

Everything above eventually lands on a GPU someone has to provision, schedule, and pay for. Two questions sit at the center, and they are mostly orthogonal: where the GPUs come from (the **build versus rent** axis) and how they are scheduled. The hardware itself is [Chapter 30](#); the orchestration substrate is [Chapter 31](#); the cost arithmetic is [Chapter 37](#). Here we choose.

Self-host versus call an API

Before sizing GPUs, settle the prior question: should you self-host at all? Most teams should start on a hosted API and self-host only when a concrete pressure forces it. The honest decision rule:

- **Call a hosted API** (through your gateway) when traffic is bursty or low volume, when you want the frontier model without operating it, when time-to-ship matters more than per-token cost, and when your data can leave your environment. This is the right default for most application teams.
- **Self-host** when at least one of these binds: sustained high utilization makes per-token economics dominate (you keep the hardware busy), data residency or compliance requires weights and prompts to stay in your environment, you need an open-weight model or a fine-tune no API offers, or you need latency and tail-behavior control the API will not give you.

The gateway makes this reversible. Because both paths speak the same OpenAI-shaped interface, you can start hosted, add a self-hosted route as one more alias, and shift traffic with a config change. Choosing a model for either path is [Chapter 38](#); the wiring of the whole stack is [Chapter 44](#).

Where the GPUs come from

A spectrum runs from serverless GPU platforms (push a function or a model, they handle the machine), through rented GPU clouds and neoclouds, to the hyperscalers, to owning hardware in a colo. The deciding variable is **sustained utilization**.

Tier	Examples	Billing shape	Best for
Serverless GPU / managed inference	Modal, Replicate, Baseten, RunPod Serverless, Fireworks, Together	Per-second / per-token, scale-to-zero	Bursty or low-utilization inference, zero infra to run
GPU clouds / neoclouds	CoreWeave, Lambda, Nebius, Crusoe	On-demand + reserved GPU-hour	Steady, high-utilization training and serving clusters
Marketplaces / spot	Vast.ai, decentralized	Per-hour, interruptible	Cheapest fault-tolerant batch; never latency-critical serving
Hyperscalers	AWS, GCP, Azure	On-demand / reserved; custom silicon	Already there, broadest menu, managed pipelines, TPU/Trainium
Own hardware (colo)	self-operated	Capex + ops	Sustained very-high utilization at scale, with ops muscle

Pricing is volatile and aggregators disagree, so anchor on the **shape**, not the dollar. As of mid-2026, list-price anchors put serverless H100 around 3 to 4 dollars per hour (Modal ~3.95/hr, RunPod ~2.89/hr), with roughly a 5x spread in H100 on-demand rates across tiers. Re-check before committing; these move quarterly.

The rule of thumb: below roughly 40 to 50 percent sustained utilization of a dedicated GPU, serverless is cheaper because you pay nothing when idle. Above it, reserved GPU-hour rates

on a neocloud or hyperscaler undercut both serverless and on-demand. Own hardware only at sustained very-high utilization where amortized capex beats years of rental. Use a marketplace or spot only for fault-tolerant batch. Use **SkyPilot** across all of these when you want to stay portable, chase the cheapest available capacity across clouds, and avoid lock-in.

Edit the two hourly rates and watch where the serverless and reserved cost lines cross; with the chapter's price anchors the break-even lands near the stated 40 to 50 percent band.

```
import numpy as np, matplotlib.pyplot as plt
serverless_rate = 3.95    # $/GPU-hour, pay only when busy (scale-to-
reserved_rate   = 2.00    # $/GPU-hour, billed 24/7 whether idle or n
hours = 730           # one month
u = np.linspace(0, 1, 101)
serverless = u * hours * serverless_rate    # cost scales with uti
reserved   = np.full_like(u, hours * reserved_rate) # flat cost
breakeven = reserved_rate / serverless_rate  # crossover utilization
print(f"break-even utilization: {breakeven*100:.1f}%")
plt.plot(u*100, serverless, label="serverless (pay-per-use)")
plt.plot(u*100, reserved, label="reserved (flat)")
plt.axvline(breakeven*100, ls="--", color="gray")
plt.xlabel("sustained utilization (%)"); plt.ylabel("monthly cost ($)")
plt.legend(); plt.title("GPU cost: serverless vs reserved"); plt.sho
```

How the GPUs are scheduled

Once you control a pool, a control plane decides placement. The three dominant ones, and how they split:

- **Slurm** when the dominant workload is large multi-node training and batch: jobs that start, run for hours, checkpoint, and exit. Gang scheduling is native and simple. The cost is no native services, ingress, or autoscaling.
- **Kubernetes** when you run inference serving, mixed workloads, or microservices alongside AI, and want the autoscaling, ingress, and service-mesh ecosystem. Add **Volcano** or **NVIDIA KAI Scheduler** for gang scheduling, **Kueue** for multi-team quota and admission, and **DRA** for fine-grained GPU/MIG allocation.
- **Ray (on KubeRay)** when your team is Python-native and wants distributed training, RL, data, and serving in one framework, riding on K8s.

The common large-team split is Slurm for training, Kubernetes for serving. The 2026 convergence is **Slurm-on-K8s** projects (SchedMD Slinky, CoreWeave SUNK, Nebius Soperator) that keep Slurm's gang semantics on shared K8s infra, narrowing the split. The momentum is toward standardizing on Kubernetes even for training, via the add-ons above, because inference, not training, is now the majority of AI compute spend and serving wants exactly the elastic, scale-to-zero, latency-aware scheduling that Kubernetes provides.

Sensible default (mid-2026): for a team shipping AI features with mixed bursty inference and occasional fine-tuning and no dedicated infra team, start on a serverless GPU or managed-inference platform (Modal for general serverless Python ergonomics, Baseten for production serving with SLAs, Fireworks or Together for pure open-model token inference). Promote to reserved neocloud clusters scheduled with Slurm-for-training and Kubernetes-for-serving as utilization climbs and the rental premium starts to dominate the bill.

CONSTRAINT ARROW

The serving cost of a token, set at this layer, reaches all the way back up to pre-training. Chapter 3 over-trains a smaller model past Chinchilla optimal precisely because the GPU-hour and per-token economics decided here are paid for the model's entire serving lifetime. The bottom of the stack sets a parameter at the top.

Wiring it together

The three layers compose into one request path, shown in Figure 39.2, and the OpenAI-compatible interface is what keeps each replaceable.

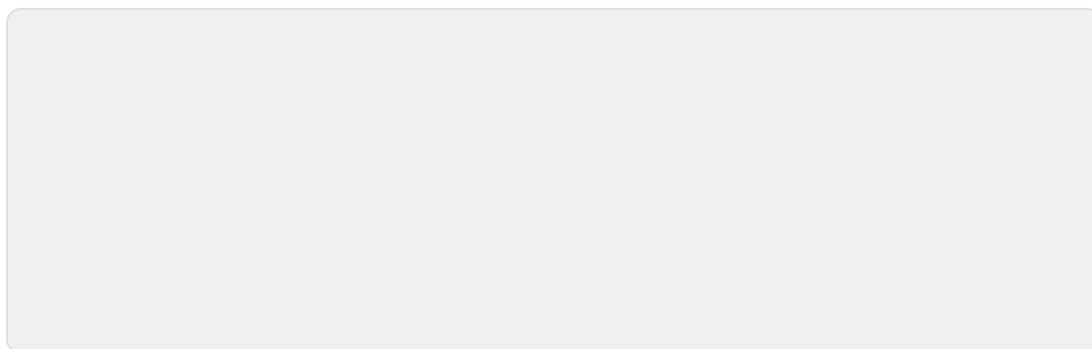


Figure 39.2. The full stack wired end to end. The gateway is the hub: apps, agents, and eval harnesses all hold a virtual key and one base URL, and the gateway fans out to hosted APIs and self-hosted engines running on scheduled GPUs.

Three facts make this hold together in practice. Applications never hold provider keys: they hold a virtual key minted by the gateway, so rotating a credential or capping a runaway agent is a server-side operation with no client redeploy. One base URL, OpenAI-shaped, serves app code, eval harnesses, and agent runtimes alike, so switching models or adding a fallback is configuration. And the engine, not the prompt, enforces structured output: guided decoding via XGrammar is the reliable path when downstream code parses tool calls or extracted data.

Capability, efficiency, trust

The three-lens close ties the layers back to the book's spine. **Capability** is the gateway's reach: one interface that makes every model, hosted or self-hosted, addressable, so the best model for each call is a routing rule. **Efficiency** is the engine and the compute tier working together: continuous batching, prefix caching, and quantization raise tokens per second, while the build-versus-rent and utilization decision sets the dollar floor under every token. **Trust** is the gateway again as the policy point: key custody, spend caps, audit, PII redaction, and the agent-native protocols that bring tool and agent traffic under one governed seam, which is where this chapter hands off to the safety and authorization work in Chapter 34.

Further reading

- vLLM documentation (features, quantization, structured outputs). docs.vllm.ai; disaggregated prefilling, [features/disagg_prefill](https://docs.vllm.ai/en/latest/features/disagg_prefill.html)
- SGLang project (RadixAttention). github.com/sgl-project/sglang
- TensorRT-LLM (NVIDIA). developer.nvidia.com/tensorrt-llm; docs

- NVIDIA Dynamo (datacenter-scale disaggregation). developer.nvidia.com/dynamo; github.com/ai-dynamo/dynamo
- llm-d project; joined CNCF, March 2026. github.com/llm-d/llm-d; redhat.com/en/topics/ai/what-is-llm-d
- LMDeploy. github.com/InternLM/lmdeploy. llama.cpp. github.com/ggml-org/llama.cpp. Ollama. github.com/ollama/ollama. Apple MLX / mlx-lm. github.com/ml-explore/mlx
- Hugging Face TGI (archived March 21, 2026). github.com/huggingface/text-generation-inference
- LiteLLM proxy: cost tracking, load balancing, fallbacks. docs.litellm.ai/docs/proxy/cost_tracking; [routing-load-balancing](https://docs.litellm.ai/docs/proxy/routing-load-balancing)
- OpenRouter, "What is an LLM gateway." openrouter.ai/blog/insights/llm-gateway
- Portkey AI gateway buyers guide. portkey.ai/buyers-guide/ai-gateway-solutions
- agentgateway (LLM + MCP + A2A); Linux Foundation. agentgateway.dev; github.com/agentgateway/agentgateway
- Envoy AI Gateway (InferencePool, token rate limiting). aigateway.envoyproxy.io/docs/capabilities
- Bifrost (Maxim AI). github.com/maximhq/bifrost. Lux. lux.latre.ai
- Modal pricing (billing model, GPU rates). modal.com/pricing. RunPod pricing. runpod.io/pricing
- SkyPilot, "Slurm vs K8s for AI Infra" (SUNK, Soperator, Volcano, Kueue). blog.skypilot.co/slurm-vs-k8s; GPU Compass pricing, blog.skypilot.co/gpu-compass
- KubeRay (official Ray K8s operator). github.com/ray-project/kuberay; Anyscale, "Introducing KubeRay v1.4," anyscale.com/blog/kuberay-v1-4
- CloudOptimo, "Kubernetes AI Infrastructure in 2026" (KAI, DRA, inference-share framing). cloudoptimo.com
- Spheron, vLLM vs TensorRT-LLM vs SGLang H100 benchmarks (2026, directional). spheron.network/blog/vllm-vs-tensorrt-llm-vs-sglang-benchmarks; GPU cloud pricing, spheron.network/blog/gpu-cloud-pricing-comparison-2026
- SqueezeBits, guided-decoding performance vLLM vs SGLang (2026, directional). blog.squeezebits.com/guided-decoding-performance-vllm-sglang

40. Training and Fine-tuning in Practice

This is a practical chapter. The theory of how weights move lives in [Chapter 9](#) (supervised fine-tuning and parameter-efficient methods) and [Chapter 8](#) (sharding an optimizer across many accelerators). Here the question is narrower and more immediate: it is mid-2026, you have a task and a budget, and you must pick the tools that change a model's weights, wire them into the rest of your stack, and decide whether you should be training at all. The answer is opinionated and dated. Lead with the categories and trade-offs, which are durable, and treat every version number, price, and ranking as a snapshot to re-check.

AS OF MID-2026

Frameworks, prices, GitHub star counts, and revenue figures below are a June 2026 snapshot, each attributed to a source. Per-token prices show the *shape* of pricing, not a canonical quote. Treat anything called uncertain as needing a fresh check before you commit a budget. The categories and the "how to choose" logic age slowly; the specifics age in months.

The layer and what it contains

Training and fine-tuning is the layer where you change weights, as opposed to the serving layer where you only run forward passes ([Chapter 39](#)) or the agent and retrieval layers where you steer a frozen model with context ([Chapter 41](#), [Chapter 42](#)). In a real 2026 stack it splits into three sub-layers that are easy to conflate and behave very differently.

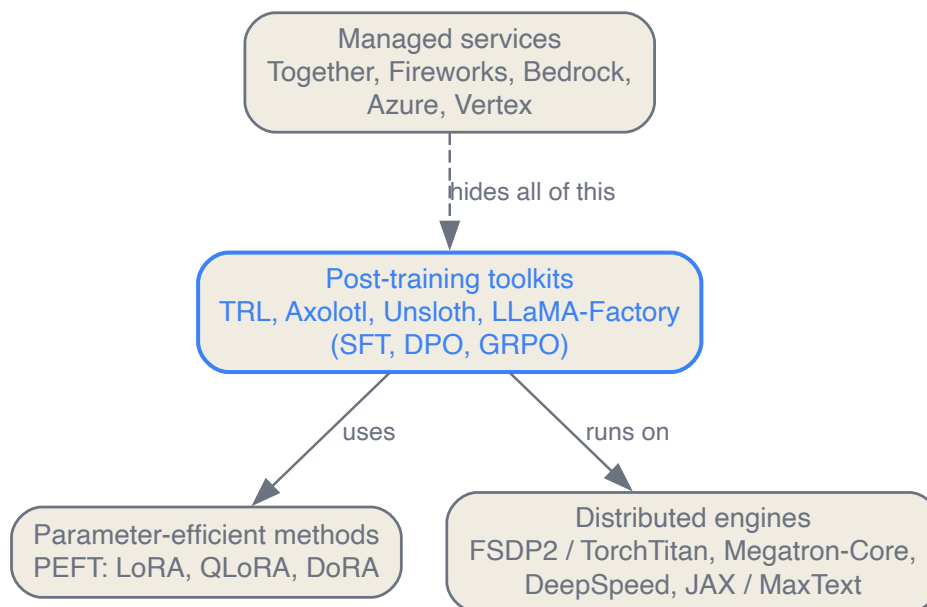


Figure 40.1. The three sub-layers of the training and fine-tuning stack in 2026. Most teams touch only the middle row; the engines sit underneath, the managed services replace all three.

1. **Distributed engines.** The heavy systems that shard a model and its optimizer state across many accelerators: PyTorch FSDP2 and TorchTitan, NVIDIA Megatron-Core, Microsoft DeepSpeed (ZeRO), and the JAX/XLA stack (MaxText) on TPUs. This is where 4D and 5D parallelism, FP8 training, and trillion-parameter scale live. [Chapter 8](#) explains the mechanics.

2. **Post-training toolkits.** Higher-level libraries that wrap those engines for supervised fine-tuning (SFT), preference optimization (DPO, ORPO), and reinforcement learning (GRPO, GSPO, PPO): Hugging Face TRL, Axolotl, Unsloth, LLaMA-Factory. This is what most teams actually touch.
3. **Parameter-efficient methods and managed services.** LoRA, QLoRA, DoRA, carried by the PEFT library, plus hosted fine-tuning APIs that hide the accelerators entirely.

Two 2026 themes tie this layer to the rest of the book. FP8 training has gone mainstream: Google's Ironwood TPU has native FP8 in its matrix units, Megatron-Core ships FP8 training, and Unsloth does FP8 reinforcement learning (Google Cloud, 2026). More important for an infrastructure book, RL post-training has fused training with inference serving: modern RL toolkits embed an inference engine, almost always vLLM, to generate rollouts, so the training box now contains a serving box. That is the wiring story at the end of this chapter.

The first fork: should you train at all?

Before tooling, answer the harder question. In 2026 the honest default for many tasks is *do not train weights*. Prompt engineering, few-shot examples, retrieval (Chapter 42), and structured outputs cover much of what fine-tuning used to be reached for. OpenAI, while winding down its self-serve fine-tuning product, points users to prompt caching, smaller base models, and RAG instead ([explainx.ai](https://openai.com/explainx/ai/secondary), secondary).

Fine-tune when you need a *durable behavior change* the context window cannot carry cheaply:

- a consistent output format or house style the prompt keeps drifting from,
- a narrow domain with vocabulary or conventions absent from the base model,
- a latency or cost win from distilling a large model's behavior into a small specialized one (this is the constraint arrow below),
- a capability shaped by reinforcement learning against a verifiable reward, the reasoning and agentic case from Chapter 14.

If none of these holds, spend the week on better prompts and retrieval, not on a training run.

CONSTRAINT ARROW

The serving layer reaches up and justifies the most common reason to fine-tune. A small model that you have specialized costs far less per token to serve (Chapter 39) than a large general model behind an API. When you serve a high volume, the training run pays for itself in inference savings, which is the same over-training-for-inference logic from Chapter 3 applied one level up. Fine-tuning here is a *serving-cost* decision wearing a *training* costume.

Change the numbers (the one-time fine-tune cost and the two serving prices, all illustrative) and watch where the two cost lines cross: below the break-even volume the big API model is cheaper, above it the fine-tuned small model wins.

```

import numpy as np
import matplotlib.pyplot as plt

# Illustrative only: shape of the trade-off, not a quote.
finetune_cost = 50.0      # one-time fine-tune run, USD
big_price = 5.0           # large API model, USD per 1M tokens served
small_price = 0.5         # fine-tuned small model, USD per 1M tokens served

vol = np.linspace(0, 40, 200)      # millions of tokens served
big_total = big_price * vol         # no training, pay per token
small_total = finetune_cost + small_price * vol

break_even = finetune_cost / (big_price - small_price)
print(f"break-even at {break_even:.1f}M tokens served")

plt.plot(vol, big_total, label="big API model")
plt.plot(vol, small_total, label="fine-tuned small model")
plt.axvline(break_even, ls="--", color="gray")
plt.xlabel("tokens served (millions)"); plt.ylabel("total cost (USD)")
plt.legend(); plt.title("When does a fine-tune pay for itself?")
plt.show()

```

Key players (mid-2026)

The tables below are dated and source-attributed. Verify current versions and prices before you choose.

Distributed engines

Name	What it is	License	Best for	Note (mid-2026)
PyTorch FSDP2	Native fully-sharded data parallel; shards params, grads, optimizer state as DTensor	BSD	PyTorch-native sharded training without a heavy framework	FSDP2 is current; FSDP1 deprecated from PyTorch 2.11+. Adds per-param freezing, mixed FP8/BF16 (PyTorch tutorial)
TorchTitan	PyTorch-native pre-training platform on FSDP2; composable 4D parallelism	BSD	Production pre-training / continued pre-training on PyTorch	Reported speedups on Llama 3.1 recipes at 128 to 512 GPU scale (arXiv 2410.06511); foundation under torchforge
NVIDIA Megatron-Core	GPU-optimized tensor/pipeline/expert/context parallelism; product-supported	Apache-2.0-style + NVIDIA support	Large dense, MoE, and multimodal training on NVIDIA GPUs	Jan 2026 added dynamic context parallelism (reported up to ~1.48x on variable-length seqs); FP8 training (docs, roadmap)
NeMo / Megatron Bridge	Higher-level orchestration over Megatron-Core; recipes, export, alignment	Open source + NVIDIA support	Teams standardizing on the NVIDIA stack end to end	Bridge 0.4.0 (Apr 2026) added Kimi 2.5, Nemotron 3, Qwen 3.5 VL, FP8 export (docs)
DeepSpeed (ZeRO)	ZeRO sharding, CPU/NVMe offload, training optimizations	Apache-2.0 / MIT	Memory-constrained training, offload, very large models	Stable v0.18.9 (Mar 2026); added Muon optimizer for ZeRO-3 (releases)
JAX + MaxText	Decoder-only LLM reference on JAX/XLA, tuned for TPU	Apache-2.0	Foundation-model training on Cloud TPU	Ironwood TPU FP8; Qwen3-Next and DeepSeek features (Mar 2026). Pairs

Name	What it is	License	Best for	Note (mid-2026)
				with Tunix for RL (docs , Cloud TPU + JAX)

Post-training toolkits (what most teams touch)

Name	What it is	License	Best for	Note (mid-2026)
Hugging Face TRL	Official HF post-training: SFT, DPO, GRPO, reward modeling	Apache-2.0	The reference; building blocks others wrap	De-facto standard trainers; integrates PEFT, accelerate, FSDP/DeepSpeed (docs)
Axolotl	Config-driven (YAML) wrapper over TRL/PEFT; LoRA, QLoRA, full FT, multimodal	Apache-2.0	Reproducible config-as-code runs on a few GPUs	Multimodal since 2025; "edit a YAML, get a run" (repo)
Unsloth	Custom Triton kernels for faster, lower-VRAM SFT and RL on 1 to 2 GPUs	Apache-2.0 core; paid Pro	Single-GPU efficiency; long-context GRPO	OSS is effectively single-GPU; robust multi-GPU is Pro (uncertain , still evolving). GRPO on a 24GB GPU FP8 RL ~1.4x vs BF16 (docs)
LLaMA-Factory	Broad toolkit + web UI (LlamaBoard); widest model support	Apache-2.0	Many model families, low-code UI, breadth over peak speed	~68k GitHub stars; can use an Unsloth backend reportedly within ~6% of native speed (secondary)
torch tune	PyTorch-native post-training (SFT/DPO/PPO/GRPO/QAT), minimal abstraction	BSD	Hackable recipes on PyTorch	Maintenance only: active dev stopped Jul 15, 2025; an unnamed PyTorch-native successor is in development (issue #2883)
torchforge	PyTorch-native RL + agentic post-training on Monarch + TorchTitan + vLLM	Open source, experimental	Scalable RL post-training without writing distributed infra	Announced Oct 2025. Not the torchtune successor; sits on TorchTitan (training) and vLLM (rollouts) (PyTorch blog)
HF PEFT	Library implementing LoRA, QLoRA, DoRA, rsLoRA	Apache-2.0	The adapter layer under TRL/Axolotl/LLaMA-Factory	Where the LoRA family methods live (docs)

Managed services (no accelerators of your own)

Name	What it is	Best for	Note (mid-2026)
OpenAI fine-tuning	Hosted SFT/RFT on GPT models	(Winding down)	Self-serve fine-tuning is being wound down. No new orgs since May 7, 2026; all new jobs end Jan 6, 2027 (deprecations)
Together AI	Managed SFT, DPO, LoRA on open weights; per-training-token billing	Cheap LoRA/SFT on open weights + dedicated serving	E.g. LoRA SFT ~\$0.48/M tokens (16B), full FT ~\$3.20/M (70 to 100B), DPO ~\$8/M; serving endpoint billed separately (secondary)
Fireworks AI	Self-serve post-training + fast serving in one place	Tune-and-serve together	Reported ~\$800M ARR (May 2026), up from ~\$305M (end 2025) (secondary)
AWS Bedrock	Managed SFT + RFT, including open weights, OpenAI-compatible APIs	AWS-native shops; RFT on open weights	RFT for open-weight + OpenAI-compatible FT added Feb 2026 (<u>what's-new</u>)
Modal	Serverless GPU compute; run torchtune/Unsloth/Axolotl yourself	Bring-your-own-recipe on on-demand GPUs	Not fully managed FT; you write the training code (<u>ref</u>)
Azure AI Foundry / Google Vertex	Cloud-native managed fine-tuning (Azure OpenAI, Gemini tuning)	Enterprises on Azure/GCP	Azure has FT cost-management tooling (<u>Microsoft Learn</u>)

Parameter-efficient methods

These are techniques, not products; they live inside PEFT, Unsloth, and TRL. See [Chapter 9](#) for the theory of why a low-rank update suffices.

- **LoRA.** Low-rank adapters; the default. Zero added inference latency once merged into the base weights.
- **QLoRA.** 4-bit (NF4) frozen base plus LoRA; the single-GPU sweet spot for roughly 1K to 50K examples.
- **DoRA.** Weight-decomposed LoRA (magnitude plus direction); better convergence, worth it at larger data, roughly 50K to 500K examples.
- **rsLoRA.** Rank-stabilized LoRA for high ranks.
- Newer arXiv variants (RandLoRA, activation-space methods) exist but are not defaults. **Treat as frontier, unverified in production** ([AppScale 2026](#)).

How to choose

Once you have decided you genuinely need to train weights, four questions settle the tool.

Scale of the run. One consumer or single GPU points to **Unsloth** for its kernels and low VRAM. A few GPUs with config-as-code points to **Axolotl** or **LLaMA-Factory**. Multi-node pre-training or continued pre-training points to **TorchTitan/FSDP2**, **Megatron-Core**, or **MaxText** on TPUs.

Ecosystem you live in. NVIDIA GPUs with a desire for vendor support point to **Megatron-Core/NeMo**. PyTorch-native and hackable points to **FSDP2/TorchTitan + TRL**. TPUs point to **JAX/MaxText**. Pure Hugging Face muscle memory points to **TRL** directly.

Method. Style or format with small data points to **QLoRA**. A deeper domain shift with more data points to **DoRA** or high-rank LoRA. A structural domain change with 500K or more examples points to **full fine-tuning**. Reasoning or agentic behavior against a reward points to **GRPO/GSPO** in Unsloth, TRL, or torchforge ([Chapter 14](#)).

Self-host versus managed. Managed wins on time-to-first-run, no GPU operations, and elastic cost, with predictable per-training-token pricing. Self-host wins on data residency and IP control, unusual recipes, repeated or large runs where you amortize hardware, and avoiding vendor lock-in or sudden deprecation. Note the convenience tax: Together prices LoRA per training token *plus* a separate hourly dedicated endpoint to serve the result, so the cheap headline number is not the full bill.

Concrete picks:

- **Pick Unsloth when** you are on one 24 to 80GB GPU and want the fastest, lowest-VRAM SFT or long-context GRPO.
- **Pick Axolotl when** you want reproducible YAML-configured runs across a few GPUs and value config-as-code in CI.
- **Pick LLaMA-Factory when** you need breadth of model support or a low-code UI for non-experts.
- **Pick TRL directly when** you are writing custom post-training logic and want the canonical trainers.
- **Pick TorchTitan/FSDP2 or Megatron-Core when** you are doing real multi-node pre-training or continued pre-training.
- **Pick MaxText when** you are on TPUs.
- **Pick a managed service when** you have no GPU operations appetite and a standard SFT/LoRA/DPO/RFT workload, but design an export path given 2026's deprecation precedent.

A sensible default (mid-2026)

For a typical product team that has decided it genuinely needs to fine-tune an open-weight model, as of June 2026:

Default: Axolotl (YAML config) on top of TRL + PEFT, doing QLoRA, on rented GPUs (Modal, Runpod, or your own cloud). It is reproducible, config-as-code fits CI and eval ([Chapter 43](#)), QLoRA keeps it single- or few-GPU, and you keep your weights and data. A starting recipe is `r=16`, DoRA on, `target_modules="all-linear"`, and you raise the rank for harder tasks ([AppScale 2026](#)).

- **Alternative A, maximum single-GPU efficiency or RL:** Unsloth, especially for consumer GPUs or long-context GRPO.
- **Alternative B, no GPU operations at all:** a managed service. Together for cheap open-weight LoRA/SFT, Fireworks if you want tune-and-serve in one place, Bedrock if you are AWS-native. Build an export and portability path; do not assume a managed FT product is permanent. OpenAI's self-serve wind-down, announced May 2026, is the cautionary tale.

For multi-node pre-training rather than fine-tuning, the default shifts to **TorchTitan/FSDP2** or **Megatron-Core**, or **MaxText** on TPUs.

Wiring it into the stack

The defining 2026 wiring fact is that **RL post-training now embeds the inference layer**. A GRPO or PPO loop generates rollouts from the current policy, scores them with a reward, then updates the weights. Generation is the bottleneck, so toolkits delegate it to a fast inference engine, almost always **vLLM**. Unsloth drives vLLM for rollouts; torchforge runs vLLM for generation and TorchTitan for training, shuttling weights between them through an RDMA tensor store (PyTorch blog). So the training box in your architecture diagram contains a serving box. This is the same vLLM from Chapter 39, now used as a training subcomponent, a clean instance of the book's thesis that these layers are one infrastructure.

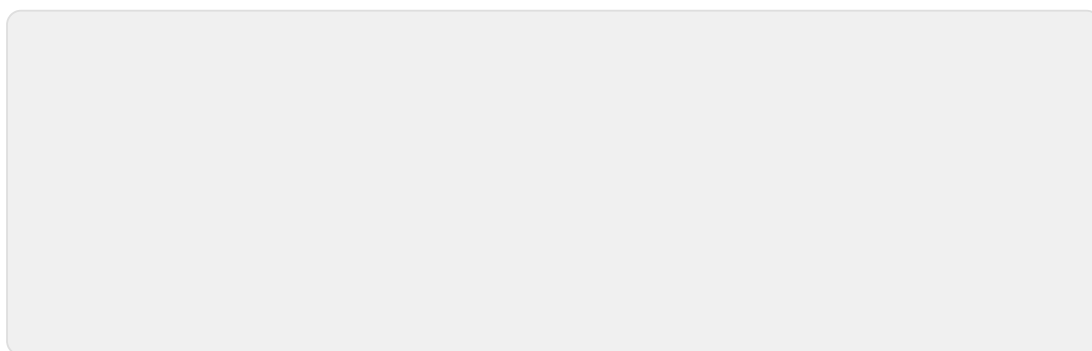


Figure 40.2. End-to-end wiring. A toolkit drives PEFT and a distributed engine to produce weights, which then flow through serving, gateway, eval, and the agent and retrieval layers. In the RL case the serving engine (vLLM) sits inside the training loop.

After training you merge the adapter, or serve it directly with an adapter-aware engine, hand the weights to the serving layer behind your gateway (Chapter 39), and route an eval harness against it before promotion (Chapter 43). The retrieval and agent layers then consume the served model. Adapter-aware serving, many LoRAs over one base, is where managed adapter platforms historically fit. The whole flow assembles into the end-to-end stack of Chapter 44.

A config and a launch you can adapt

A minimal Axolotl QLoRA config, illustrative and meant to be edited:

```
# qlora.yml
base_model: Qwen/Qwen3-8B
load_in_4bit: true          # QLoRA: 4-bit NF4 frozen base
adapter: lora
lora_r: 16
lora_alpha: 32
lora_target_modules: all-linear
datasets:
  - path: ./data/sft.jsonl
    type: chat_template
gradient_checkpointing: true
sequence_len: 4096
micro_batch_size: 2
num_epochs: 3
# distributed backend (FSDP2 or DeepSpeed) configured via accelerate
```

Launch it, then merge the adapter for serving:

```
# train (single or few GPUs via accelerate)
accelerate launch -m axolotl.cli.train qlora.yml

# merge the LoRA adapter back into the base for adapter-free serving
python -m axolotl.cli.merge_lora qlora.yml --lora_model_dir ./out
```

The merged weights are then handed to the serving stack. Keep the config in version control so a training run is a reviewable diff, the same discipline you apply to the rest of the infrastructure.

Closing: capability, efficiency, trust

Through the book's closing lens, fine-tuning is first a *capability* lever, the durable behavior change a prompt cannot carry. It is increasingly an *efficiency* lever, distilling a large model into a small cheap-to-serve one, which is why the serving-cost constraint arrow points up so often. And it is a *trust* surface: a fine-tuned model has absorbed your data, so data residency, reproducible config-as-code, and an eval gate before promotion are not niceties but the controls that let you ship the weights at all. The honest 2026 default is still to reach for prompting and retrieval first, and to train only when the behavior change has to live in the weights.

Further reading

First-hand sources first.

- PyTorch, "Getting Started with Fully Sharded Data Parallel (FSDP2)." docs.pytorch.org

- Liang et al., "TorchTitan: One-stop PyTorch Native Solution for Production-Ready LLM Pre-training," 2024. [arXiv:2410.06511](https://arxiv.org/abs/2410.06511); repo github.com/pytorch/torchtitan
- NVIDIA, "Megatron-Core documentation" and roadmap. docs.nvidia.com; roadmap #4003
- NVIDIA, "NeMo Megatron Bridge documentation." docs.nvidia.com
- Microsoft DeepSpeed releases. github.com/deepspeedai/DeepSpeed
- Google, "MaxText documentation" and "Training large models on Ironwood TPUs," 2026. maxtext.readthedocs.io; cloud.google.com; Cloud TPU + JAX
- Hugging Face, "TRL" and "PEFT" documentation. huggingface.co/docs/trl; huggingface.co/docs/peft
- Axolotl. github.com/axolotl-ai-cloud/axolotl
- Unsloth documentation and long-context GRPO guide. unsloth.ai/docs; [GRPO long-context](https://unsloth.ai/docs)
- torchtune future direction. issue #2883
- PyTorch, "Introducing torchforge," 2025. pytorch.org/blog; repo github.com/meta-pytorch/forged
- OpenAI, "Deprecations" (fine-tuning wind-down). developers.openai.com
- AWS, "Reinforcement fine-tuning for open-weight models on Bedrock," Feb 2026. aws.amazon.com
- Microsoft, "Fine-tuning cost management." learn.microsoft.com
- Secondary comparisons and pricing (verify before use): AppScale, "LoRA, QLoRA, full fine-tuning compared," 2026 appscale.blog; CloudZero, "Together AI pricing" cloudzero.com; Sacra, "Fireworks AI" sacra.com; "Fine-tuning in 2026" toolkit comparison.dev.to; All Things Open, "LLM fine-tuning as an open-source developer workflow" allthingsopen.org.

41. Agents, Frameworks, and Sandboxes

This is a practical chapter. It takes the agent architecture of [Chapter 21](#), the harness of [Chapter 23](#), and the authorization model of [Chapter 34](#) and turns them into a buying-and-wiring decision for an engineer shipping an agent in 2026. The goal is not to survey every library but to give you a way to choose and a default that is hard to regret.

An agent runtime is three parts, and most of the 2026 confusion comes from conflating them:

1. **The framework / SDK** is the loop. The model only emits tokens. To make it call a tool, read the result, decide the next step, remember earlier turns, hand work to another agent, pause for a human, and recover after a crash, you need a control loop and the plumbing around it.
2. **MCP (the Model Context Protocol)** is the wire protocol for tools. It is to tool integration what the OpenAI Chat Completions shape is to model calls: the common interface that lets one tool server be reused by every framework.
3. **The sandbox** is the execution substrate. The moment an agent runs code, the safety story stops being about the model and becomes about the runtime. The sandbox is the isolated computer the code runs in.

The throughline is that the framework, the tools, and the box all keep secrets and model access *outside*, reaching in only through governed seams: a gateway virtual key for models, short-lived scoped tokens for credentials, a default-deny network for egress. Get that discipline right and the specific products become reversible choices.

AS OF MID-2026

Everything below is a dated snapshot (June 2026). Version numbers, prices, and "best for" verdicts move fast in this category. Treat rankings and specific figures as source-attributed and verifiable, not as timeless fact, and confirm current values on each project's own docs before you commit. The durable content is the *categories and trade-offs*, not the version strings.

Frameworks: the landscape

Feature checklists hide the real decision. The axis that matters is **how the framework structures the agent loop**, because that dictates what you can express, how much boilerplate you write, and where control lives. Five archetypes cover the field, and they map directly onto the loop structures of [Chapter 21](#):

1. **Graph / state-machine** (LangGraph, Google ADK, Pydantic AI's `pydantic-graph`). You declare nodes and edges; the runtime walks the graph, persists a typed `State`, and supports loops, parallel branches, conditional routing, and crash-resume from a checkpoint. Most expressive, most control, most boilerplate. The only archetype that natively handles "Agent A then B then back to A" and "B and C in parallel."
2. **Linear handoff chain** (OpenAI Agents SDK). A small set of primitives; agents delegate by *handing off* the conversation, and the receiver sees the full transcript. Cyclic or parallel topologies must be modeled by hand. Fastest to a working agent.
3. **Harness / CLI engine** (Claude Agent SDK). The loop is the Claude Code engine of [Chapter 23](#): a real filesystem, bash, web search, and automatic context compaction are built in, and you extend it with hooks, in-process MCP tools, and subagents. Unmatched for "a model working in a repo."
4. **Role / crew orchestration** (CrewAI). You define specialist roles and goals; the crew collaborates. Natural when work decomposes into roles, less fine-grained than a graph.

5. **Model-first autonomous loop** (AWS Strands). A system prompt plus tools, and the model drives. Least scaffolding, least deterministic control.

Tool and memory handling is the second axis, and MCP has flattened most of it. Tools are either native function tools (a decorated function whose signature becomes the schema) or MCP servers (external over stdio/HTTP, or in-process). Memory ranges from "the framework replays the message list each turn" (OpenAI Sessions, simplest) to "a durable, checkpointed **State** you can rewind" (LangGraph, Pydantic AI durable execution). The deciding question is whether you must survive a restart mid-task. If yes, you want a checkpoint, not a message list.

A comparison of the players

Versions and license claims below are as of June 2026 and attributed in *Further reading*. Where a "best for" verdict is the field's loose consensus rather than a vendor fact, read it as directional.

Name	Loop archetype	License / backer	Pick it when	Note
LangGraph	Graph / state-machine	MIT, LangChain Inc.; paid Platform for hosting	The workflow is genuinely a graph: loops, parallel branches, approval gates, long-running, must survive crashes	1.0 GA reached; most control, most boilerplate. "Time travel" rewinds State to a prior checkpoint
OpenAI Agents SDK	Linear handoff chain	MIT, OpenAI	You want the shortest path to a production agent of moderate complexity and a tiny primitive set over maximal control	Provider-agnostic despite the name: 100+ models via Chat Completions / LiteLLM. Cyclic/parallel routing is hand-built
Claude Agent SDK	Harness / CLI engine	MIT, Anthropic; model usage billed via Claude API	The agent is fundamentally Claude working over a filesystem/CLI: coding agents, repo automation, ops	Renamed from Claude Code SDK in late 2025. Strongest MCP-native and lifecycle-hook story; in-process tools avoid subprocess overhead
CrewAI	Role / crew	MIT, CrewAI Inc.; paid enterprise tier	The problem decomposes cleanly into specialist roles and you want a multi-agent prototype fast	Independent of LangChain. Fast to start, less fine-grained than a graph
Pydantic AI	Graph / state-machine	MIT, Pydantic Services Inc.	You are a type-disciplined Python shop wanting validated structured outputs and durable execution	Validation and structured output are first-class. Durable execution preserves progress across restarts
LlamaIndex	Data / RAG + agent layer	MIT, LlamaIndex Inc.; paid LlamaCloud	The hard part is retrieval and document parsing, not orchestration	The differentiator is the retrieval stack (see Chapter 42); often paired with another framework for orchestration

Name	Loop archetype	License / backer	Pick it when	Note
Mastra	Deterministic TS workflows	MIT/Apache (verify per package), Mastra (YC W25)	Your team lives in TypeScript and wants a batteries-included stack without crossing into Python	Launched early 2026, rapid traction. Workflows are explicit deterministic pipelines
Google ADK	Graph / state-machine	Apache-2.0, Google; multi-language	You are on Google Cloud/Gemini, or want a native graph engine for cyclic and parallel routing	Optimized for Gemini but model- and deployment-agnostic
AWS Strands	Model-first loop	Apache-2.0, AWS	You are AWS-centric and want a minimal, model-driven loop without hand-coding orchestration	Trades orchestration control for simplicity; leans on the model to drive
Microsoft Agent Framework	Graph workflow on SK	MIT, Microsoft	You are a .NET/Azure enterprise wanting Microsoft's supported, telemetry-rich stack	Shipped 1.0 in 2026 as the convergence of Semantic Kernel + AutoGen. Both predecessors are now maintenance-only
AG2 (ex-AutoGen)	Conversation-driven multi-agent	Apache-2.0, ag2ai/ag2 ; volunteer	You specifically want the AutoGen conversation lineage and human-in-the-loop dialogue patterns	The community fork that kept the AutoGen PyPI packages after the creators left Microsoft; innovation has slowed

WHAT'S CONTESTED

Several 2026 comparison blogs publish ordered "production rankings" of these frameworks (for example LangGraph first, Claude Agent SDK second, CrewAI third). These are individual authors' opinions from small samples, not measured benchmarks, and they are not independently verified. Use them as directional sentiment, never as evidence. The defensible claim is about *loop archetype fit*, not a leaderboard position.

How to choose, concretely

- *Pick **LangGraph*** when the workflow is a graph: loops, parallel branches, approval gates, long-running, must survive crashes. Accept the boilerplate for the control.

- *Pick the **OpenAI Agents SDK*** when you want the shortest path to a production agent of moderate complexity and value a small, learnable primitive set.
- *Pick the **Claude Agent SDK*** when the agent is Claude working over a filesystem or CLI. Nothing else matches its coding-agent ergonomics.
- *Pick **CrewAI*** when the problem splits cleanly into specialist roles and you want a multi-agent prototype fast.
- *Pick **Pydantic AI*** when you want type safety, validated structured outputs, and durable execution in Python.
- *Pick **LlamaIndex*** when the value is in retrieval and document parsing; use it for the data layer and keep orchestration thin.
- *Pick **Mastra*** when your team is TypeScript-first.
- *Pick **Google ADK** or **AWS Strands*** when you are committed to that cloud and want the first-party toolkit.
- *Pick **Microsoft Agent Framework*** when you are a .NET/Azure enterprise; it is the supported successor to AutoGen and Semantic Kernel, both now maintenance-only.

A sensible default. For a product team building its first serious agent, follow the loop archetype, not brand loyalty. If the agent is a coding, ops, or filesystem agent, start with the **Claude Agent SDK**: it hands you a working loop with real tools, context compaction, and MCP, and you extend it with hooks and subagents. For a general task/tool agent of moderate complexity, start with the **OpenAI Agents SDK**: the primitive set is learnable in an afternoon, it is provider-agnostic, and tracing is built in. **Graduate to LangGraph** when the workflow outgrows a linear chain and you need loops, parallel branches, approval gates, or durable crash-resume. That migration is a sign of growth, not a failure.

CONSTRAINT ARROW

The framework cannot be more reliable than the gateway and serving layers beneath it (Chapter 39, Chapter 44). This is why "durable execution" and "checkpointing" became the headline 2026 features: they are the framework's attempt to survive failures in the layers below. Point the framework at a model gateway with a virtual key, not a raw provider credential, and the lower layer's governance, allowlists, budget caps, model swaps, becomes a server-side change with no agent redeploy.

MCP: the tool layer underneath everything

MCP is half this story. It is the protocol that lets a tool server, written once, be consumed by every framework above, which is why "what tools does framework X support" became a non-question: the answer is "any MCP server." By 2026 it is the default tool layer underneath almost all of the frameworks in the table.

State of the spec, as of mid-2026:

- **Stable: 2025-11-25** . Stateful, with an `initialize / initialized` handshake plus per-session state. Transports are stdio (local subprocess) and Streamable HTTP (remote service).
- **Release candidate 2026-07-28** (published 21 May 2026), described by the maintainers as the largest revision since launch. The headline is a **stateless core**: it removes the init handshake and session state, carries protocol metadata in `_meta` , and adds routing headers (`Mcp-Method` , `Mcp-Name`) so load balancers can route without inspecting the body. The point is horizontal scaling on ordinary HTTP infrastructure: any server instance can serve any request. It also formalizes independently versioned **extensions** (shipping MCP Apps for server-rendered UI and Tasks for async work), a 12-month deprecation policy, and a conformance suite. These are breaking changes versus **2025-11-25** .

Confirm the final ship date and your client/server support before depending on stateless behavior.

- **Registry and discovery.** An official MCP Registry with namespace verification exists, and MCP Server Cards (a .well-known metadata format) let registries and crawlers discover servers without a live connection. The often-cited "500+ public servers" is a moving, secondary-sourced figure.

In production you do not let each agent dial arbitrary MCP servers directly. You route MCP traffic through a governed proxy (an "MCP gateway", agentgateway and similar) that applies auth, allowlists, and audit, the same way a model gateway governs token traffic. The framework speaks MCP either way; whether a tool is a local subprocess, a remote HTTP service, or an in-process function changes almost nothing in the agent code. That portability is the whole point of MCP being a wire protocol rather than a library API.

A minimal sketch of the in-process MCP-tool pattern, in the Claude Agent SDK shape. Check the SDK docs for the exact current API.

```
from claude_agent_sdk import tool, create_sdk_mcp_server, query, ClaudeAgentOptions

@tool("lookup_order", "Look up an order by id", {"order_id": str})
async def lookup_order(args):
    row = await db.fetch_order(args["order_id"])
    return {"content": [{"type": "text", "text": row.summary()}]}

server = create_sdk_mcp_server(name="ops-tools", version="1.0.0", tools=[lookup_order])

async for msg in query(
    prompt="Refund the late order for customer 4471 and explain why.",
    options=ClaudeAgentOptions(
        mcp_servers={"ops": server},
        allowed_tools=["lookup_order", "Read"],
        # model access resolves through the gateway base_url + virtual endpoint
        # not a raw provider credential
    ),
):
    print(msg)
```

The same agent could instead point at an *external* MCP server over HTTP behind the MCP gateway. The framework code barely changes.

Sandboxes: the execution substrate

A sandbox is a disposable, isolated computer handed to an agent to run untrusted code in. The model writes a Python snippet, a shell command, a `git` operation, or a whole dev loop, and that code executes somewhere that is *not* your laptop, *not* your production cluster, and *not* sharing a kernel with another tenant's secrets.

The safety argument is blunt and worth internalizing: once an agent can execute code, a jailbreak filter on the gateway does nothing against `rm -rf`, a crypto miner, or an exfiltration `curl` the model was tricked into emitting. The 2026 security consensus (Firecrawl and others) is that the only robust defense is to isolate the physical environment the code runs in, not to ask the model nicely in a system prompt. This is the practical face of [Chapter 34](#): the runtime, not the prompt, is the enforcement boundary.

Two isolation models dominate, and the choice between them is most of the game:

- **MicroVM isolation** (Firecracker, Cloud Hypervisor, Kata). Each sandbox is a real, minimal virtual machine with its own guest kernel, separated from the host by hardware virtualization (KVM). This is the strongest commonly-deployed boundary for untrusted code, the same lineage AWS Lambda uses. Firecracker reports booting user code in as little as roughly 125 ms and creating up to roughly 150 microVMs per second per host (project docs, as of 2026).
- **Userspace-kernel / gVisor isolation**. A userspace kernel intercepts syscalls so the guest never talks to the host kernel directly. Production-grade (it is what Google Cloud Run uses), trading a sliver of the microVM's hardware separation for faster starts on custom images. Modal uses this model.

A third, weaker tier, plain containers with a shared host kernel, is common but should be treated as a soft boundary for untrusted code unless hardened with gVisor or a microVM underneath. A fourth, V8 isolates (Cloudflare Dynamic Workers), is faster still but only runs sandboxed JS/WASM, not an arbitrary OS.

A comparison of the players

All latency and price claims below are as of mid-2026, vendor-reported or third-party-reported, and not independently verified. Treat the table as a starting map, not a price sheet, and confirm current rates on each vendor's pricing page.

Name	Isolation	Pricing model (2026)	Pick it when	Note
E2B	Firecracker microVM	Open-source SDK; hosted free Hobby + credit, Pro tier, usage per second	You want a popular, agent-native, microVM-isolated code box with an open SDK and a self-host path, running short-lived Python/JS tool calls	Site claims start "under 200ms"; a circulating "p50 ~78ms" figure is from comparison blogs, not E2B's own page
Modal	gVisor (userspace kernel)	Hosted; Sandbox CPU billed above the Function rate; optional GPUs	The agent must run untrusted code <i>and</i> reach a GPU in the same isolated box (inference, fine-tune steps)	Same isolation model as Google Cloud Run; faster custom-image starts, slightly thinner boundary than a full microVM
Fly.io (Machines / Sprites)	Firecracker microVM	Hosted; per-second CPU+RAM, no charge while idle	You want raw microVM control, or snapshot/restore for clean reproducible eval runs with zero idle billing	Sprites "start in under a second" (vendor). Sub-10ms restore and "28ms boot" numbers are third-party, treat as unverified
Vercel Sandbox	Firecracker microVM	Hosted; active-CPU billing, memory billed provisioned for the full run	Your app is in the Vercel ecosystem and you run agent-generated code with up to ~45-min lifetimes	Bills active CPU but memory is provisioned-billed, watch idle memory cost
Cloudflare Sandbox SDK	Container + V8 isolate tier	Hosted; active-CPU pricing, requires Workers Paid	You are on Cloudflare/Workers and want edge-distributed execution with live preview URLs	GA in April 2026. Two tiers: V8 isolates (JS/WASM) and full container sandboxes. Verify cost incl. Durable Objects
Daytona	Container base	Hosted; usage-based, free credit	You want very fast sandbox creation for short-lived agent code runs	Claims "sub-90ms" creation, but a container base, so verify the isolation model against your threat model
Runloop	Two-layer (VM + container)	Hosted; free + usage, Pro tier, enterprise	You build a <i>coding</i> agent and want suspend/resume devboxes plus built-	The differentiator is the eval/benchmark + suspend/resume

Name	Isolation	Pricing model (2026)	Pick it when	Note
			in SWE-style benchmarks	tooling, not just isolation
Northflank	microVM (Kata/gVisor by infra)	Self-serve + enterprise; lower headline vCPU/GB rate	You want one platform for both app deployment and agent code execution at a lower compute rate	Isolation varies by underlying infra; positions on cost + bring-your-own-cloud
Browserbase (+ Anthropic Managed Agents)	Isolated headless Chromium	Hosted; per-session / per-browser-minute	The agent's job is in a <i>browser</i> (web automation, computer-use), not a shell	Powers Anthropic's Managed Agents (beta, 2026); Anthropic runs the loop, Browserbase supplies the isolated browser
Firecracker (project)	The microVM VMM itself	Open source (Apache 2.0), AWS-originated	Sandboxing is your core business and you want to operate your own microVM fleet	Not a product you use directly; it is the isolation primitive under E2B, Fly, Vercel, Lambda
Cella (latere.ai)	Author's own runtime	Latere product family; limited public docs	The Latere stack, and as a worked example in this book	Disclosed as the author's own infrastructure; illustrative, not a neutral recommendation. One API call spins up a named ephemeral or persistent sandbox

WHAT'S CONTESTED

The microVM-versus-gVisor line is a genuine trade-off, not a solved question. MicroVMs (Firecracker) give the strongest hardware boundary but historically cost more in cold start and image-build flexibility; gVisor buys faster custom-image starts at a slightly thinner boundary; plain containers buy speed and density at a real isolation cost. As the gVisor-versus-microVM cost-and-latency curve moves through 2026, the "right" tier for a given threat model moves with it. The one durable claim: for genuinely adversarial, model-generated code, a shared-kernel container is a soft boundary, and hardware or userspace-kernel isolation is the provable line.

How to choose, concretely

The decision is dominated by three axes: isolation strength versus start latency, ephemeral versus persistent, and what the agent needs to touch (shell, GPU, or a browser).

- **Pick E2B** when you want the popular, agent-native, microVM-isolated code box with a real open-source SDK and an optional self-host path, running short-lived Python/JS tool calls.

- **Pick Modal** when the agent needs to run untrusted code *and* hit a GPU in the same isolated environment.
- **Pick Fly Machines / Sprites** when you want microVM control, snapshot-restore for reproducible eval runs, and zero charge while idle.
- **Pick Vercel or Cloudflare Sandbox** when your app already lives in that platform. Weigh Cloudflare's edge distribution and V8 fast path against Vercel's Firecracker boxes and provisioned-memory billing.
- **Pick Runloop** when you build a coding agent and want suspend/resume devboxes plus built-in SWE-style benchmarks.
- **Pick Browserbase (or Anthropic Managed Agents)** when the agent acts in a browser, not a shell.
- **Self-host on Firecracker** only when sandboxing is your core business.

Pricing comparisons are treacherous. "Active CPU only" providers can still bill provisioned memory for the full lifetime (Vercel), and platform minimums or Durable-Object charges (Cloudflare) hide under the headline vCPU rate. Always model *your* run shape (lifetime, idle fraction, concurrency), not the per-hour sticker.

Change the idle fraction and watch the gap open up between a pure active-CPU bill and one that also charges provisioned memory for the whole lifetime.

```
# A bursty agent: short CPU spikes between long idle waits on the model
lifetime_s = 600          # 10-min box, alive the whole task
cpu_price = 0.000040      # $ per active vCPU-second
mem_price = 0.0000060     # $ per provisioned GB-second (billed even if idle)
mem_gb = 2.0

for idle in [0.0, 0.5, 0.8, 0.95]:
    active_s = lifetime_s * (1 - idle)
    cpu_cost = active_s * cpu_price
    mem_cost = lifetime_s * mem_gb * mem_price  # full lifetime, no discount
    total = cpu_cost + mem_cost
    mem_share = 100 * mem_cost / total
    print(f"idle={idle:>4.0%}  active-CPU=${cpu_cost:.4f}  "
          f"provisioned-mem=${mem_cost:.4f}  total=${total:.4f}  "
          f"mem={mem_share:.0f}% of bill")
```

The takeaway: the higher the idle fraction (the more the agent waits on the model), the more a provisioned-memory line dominates the bill, even on an "active-CPU" provider.

A sensible default. For a team building a code-running agent in 2026, default to **E2B** for hosted, microVM-isolated execution: agent-native, a real open-source SDK, a self-host escape hatch, strong (Firecracker) isolation, and per-second billing that suits bursty tool calls. If the agent needs a **GPU** in the box, default to **Modal** instead. If you specifically want **snapshot-restore for reproducible evals** (Chapter 43) and zero-idle billing, default to **Fly Machines / Sprites**. For the Latere stack, the in-house analogue is **Cella**, disclosed as the author's own infrastructure and therefore not a neutral pick.

Wiring it together safely

The agent framework is a consumer, not a hub. It calls models through the gateway, pulls grounding from retrieval, reaches tools over MCP, runs code in a sandbox, and emits traces the eval and observability layers read. [Figure 41.1](#) shows the shape that holds up in production.

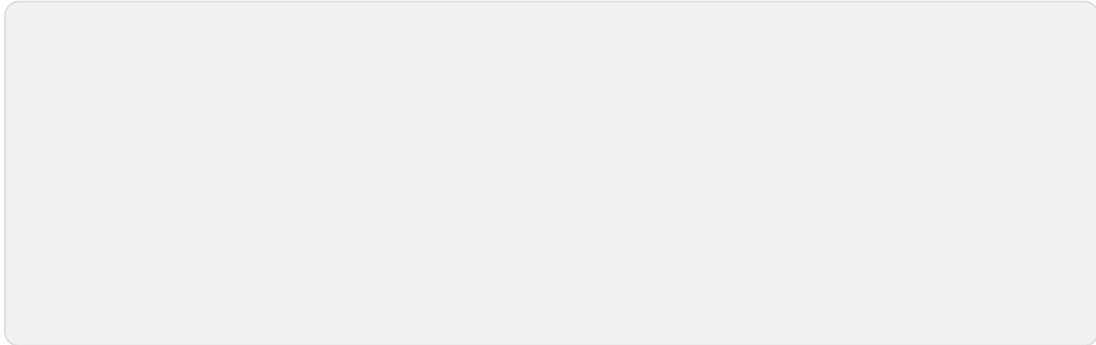


Figure 41.1. The production agent runtime. The framework holds no provider key and no raw credentials; model access, tool access, and compute access each reach in through a governed seam.

Three things make this hold up:

1. **The framework never holds a provider key or a raw credential.** It holds a gateway virtual key scoped to a model allowlist and a budget, and MCP traffic goes through a gateway that owns tool credentials and the allowlist. Swapping one model for another, or capping a runaway loop, is a server-side change.
2. **State lives in a checkpointer, not in process memory.** A durable `State` (LangGraph's checkpointer, Pydantic AI's durable execution) is what lets a long-running agent survive a restart and lets a human pause, inspect, and resume.
3. **The sandbox keeps secrets, model access, and the network outside the box.** They reach in only through narrow, auditable channels: short-lived scoped tokens for credentials, a gateway virtual key for models, default-deny egress for the network.

A minimal safe-usage config sketch. The shape matters more than the exact keys; adapt it to your sandbox provider.

```

sandbox:
  isolation: microvm          # firecracker/gvisor; not a bare conta
  lifetime: ephemeral        # destroy after the run: nothing persi
  timeouts:
    per_tool_call: 30s
    per_task: 10m
    max_lifetime: 30m
  network:
    egress: deny              # default-deny
    allow: ["pypi.org", "api.internal.example.com"]
  secrets:
    inject: none              # no raw provider keys in the box
    model_access_via: gateway # short-lived scoped virtual key only
  audit:
    log: [commands, network_requests, file_writes] # immutable tra
  approval:
    require_human_for: [payments, production_writes]

```

The safety practices that recur across 2026 sources, and that close this chapter on the **capability, efficiency, trust** lens: least-privilege, short-lived credentials; treat every tool or browser output as untrusted input, separating "thinking" from "acting"; default-deny egress with an explicit allowlist; hard timeouts at tool, task, and sandbox-lifetime granularity; immutable audit logs; ephemeral-by-default so state and credentials do not accumulate; and human-in-the-loop gates for irreversible actions. The framework choice is reversible. The gateway, sandbox, and approval discipline is what keeps it reversible.

Further reading

Frameworks and MCP, primary sources:

- OpenAI Agents SDK (docs): <https://openai.github.io/openai-agents-python/>; GitHub: <https://github.com/openai/openai-agents-python>
- OpenAI, "A practical guide to building agents": <https://openai.com/business/guides-and-resources/a-practical-guide-to-building-ai-agents/>
- Claude Agent SDK (Python, GitHub): <https://github.com/anthropics/claude-agent-sdk-python>; docs: <https://platform.claude.com/docs/en/agent-sdk/python>
- LangGraph (GitHub): <https://github.com/langchain-ai/langgraph>; 1.0 announcement: <https://www.langchain.com/blog/langchain-langgraph-1dot0>
- CrewAI: <https://www.crewai.com/>; docs: <https://docs.crewai.com/>
- Pydantic AI (docs): <https://ai.pydantic.dev/>; durable execution: https://pydantic.dev/docs/ai/integrations/durable_execution/overview/
- LlamaIndex: <https://docs.llamaindex.ai/>
- Mastra: <https://mastra.ai/>
- Google ADK: <https://adk.dev/>; <https://github.com/google/adk-python>
- AWS Strands Agents: <https://strandsagents.com/>

- Microsoft Agent Framework (Microsoft Learn): <https://learn.microsoft.com/en-us/agent-framework/overview/>; AutoGen maintenance-mode status: <https://github.com/microsoft/autogen/discussions/7210>
- AG2 (GitHub): <https://github.com/ag2ai/ag2>
- MCP specification: <https://modelcontextprotocol.io/>; 2026 roadmap: <https://blog.modelcontextprotocol.io/posts/2026-mcp-roadmap/>; 2026-07-28 release candidate: <https://blog.modelcontextprotocol.io/posts/2026-07-28-release-candidate/>

Sandboxes, primary sources:

- E2B pricing and overview: <https://e2b.dev/pricing>; <https://e2b.dev/>
- Modal sandboxes (gVisor, GPU pricing): <https://modal.com/resources/best-code-execution-sandboxes-ai-agents>
- Fly.io agent sandbox / Sprites: <https://fly.io/learn/agent-sandbox/>; architecture: <https://fly.io/docs/reference/architecture/>
- Firecracker project: <https://firecracker-microvm.github.io/>
- Cloudflare Containers & Sandboxes GA (Apr 13 2026): <https://developers.cloudflare.com/changelog/post/2026-04-13-containers-sandbox-ga/>; Dynamic Workers: <https://blog.cloudflare.com/dynamic-workers/>
- Vercel Sandbox vs E2B (official KB): <https://vercel.com/kb/guide/vercel-sandbox-vs-e2b>
- Daytona pricing: <https://www.daytona.io/pricing>
- Runloop pricing: <https://runloop.ai/pricing>
- Northflank AI sandbox pricing comparison (2026): <https://northflank.com/blog/ai-sandbox-pricing>
- Browserbase + Anthropic Managed Agents: <https://docs.browserbase.com/integrations/anthropic/managed-agents/introduction>
- Firecrawl, "AI Agent Sandbox: How to Safely Run Autonomous Agents in 2026": <https://www.firecrawl.dev/blog/ai-agent-sandbox>
- Cella (latere.ai, author's own product): <https://cella.latere.ai/>

Secondary comparisons (used for corroboration only; rankings and opinions in these are not independently verified): QubitTool 2026 framework showdown; TURION.AI LangGraph vs OpenAI vs Claude Agent SDK; WorkOS "Everything your team needs to know about MCP in 2026"; Northflank best sandbox runners (2026).

42. Retrieval and Document Intelligence

Retrieval is how a model talks to data it was never trained on, and document intelligence is the front door that decides what data even reaches it. This chapter is the practical companion to the theory in [Chapter 25](#) and [Chapter 26](#): where those chapters explain why retrieval works and how a context window is assembled, this one is for the engineer who has to pick tools in 2026 and wire them together. The two halves are one pipeline. A PDF becomes structured text, structured text becomes embeddings in a vector store, a query pulls the right chunks back, and a reranker orders them before they hit the prompt. A failure anywhere upstream silently poisons everything downstream.

AS OF MID-2026

This is a dated snapshot (2026-06). Every ranking, price, version number, and benchmark score below is volatile and source-attributed, and many come from vendor posts or version-mixed aggregators that do not agree. Treat them as directional, lead with the durable guidance (the categories, the trade-offs, how to choose, how to wire), and verify current values against the primary sources before you quote them.

The pipeline and why parse quality leads

Read the pipeline in ingestion order. [Figure 42.1](#) shows the whole path from a raw document to an answer.

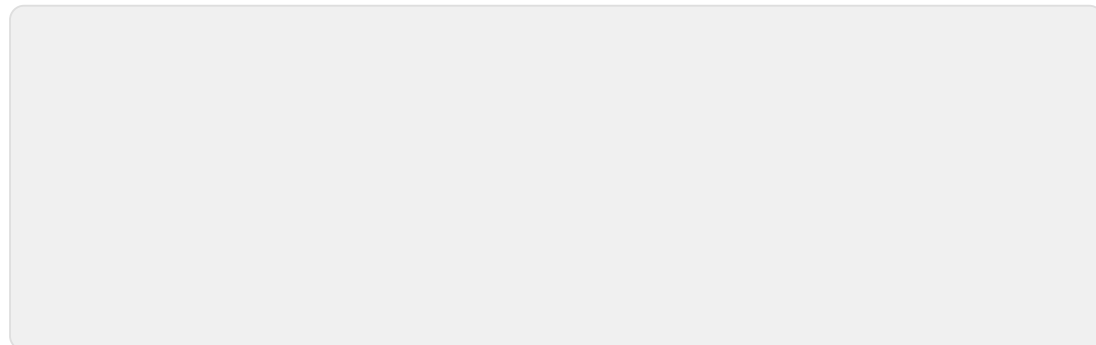


Figure 42.1. The retrieval and document-intelligence pipeline, from raw document to grounded answer.

Ingestion is offline; query is online; evaluation observes both.

The order matters because the largest data loss and the quietest errors happen at the parse step, not the search step. If a two-column page is flattened into interleaved lines, or a financial table is shredded into prose, no embedding model and no reranker downstream can recover the structure. By mid-2026 the vector database itself is close to a commodity: approximate nearest-neighbor search (HNSW, IVF, DiskANN variants) is everywhere and rarely the bottleneck. The differentiation has moved to the parse quality at the front, the embedding and reranker quality in the middle, and the cost architecture (RAM-resident versus object-storage) underneath. We treat the two halves in pipeline order: document intelligence first, then the retrieval stack.

CONSTRAINT ARROW

The retriever sets the ceiling on answer faithfulness no matter how good the generator is, and the parser sets the ceiling on the retriever. Spending here beats almost any

prompt or model change above. A second arrow comes from below: once an agent (Chapter 41) calls retrieval as a tool many times per task, per-query latency and cost stop being afterthoughts and become design constraints, which is what makes the object-storage cost curve worth caring about.

Document intelligence: getting from a messy file to clean data

Document intelligence turns a PDF, scan, screenshot, or office file into machine-readable structure: text in reading order, tables as cells, equations as LaTeX, key-value pairs, and ultimately a typed object that obeys a schema. By 2026 the field split into three architectural camps, plus a cross-cutting concern.

1. **Classic CV/OCR pipelines.** Multi-stage: layout detection, OCR, table reconstruction, reading-order assembly. Deterministic, debuggable, fast, and brittle on unusual layouts. Marker, MinerU, PaddleOCR, unstructured, and the cloud APIs live here.
2. **End-to-end document VLMs.** A single image-and-text-to-text model emits the whole structured document (Markdown, HTML tables, LaTeX) in one pass. These dominate the open leaderboards, are usually small (sub-2B), and are self-hostable on the same serving fleet as your chat models (Chapter 39). Simpler to run, harder to debug when they drop a column.
3. **Agentic document platforms.** A managed service wraps OCR, a VLM, a correction loop, and structured extraction behind an API and an SLA. You trade self-hosting for accuracy on hard documents and for compliance.

The cross-cutting concern is **structured extraction**: going from parsed text to a typed object that obeys a JSON Schema. That is where constrained decoding and validation libraries meet the parse.

The table below is a condensed view; the *Primary sources* hold the version and license detail.

Tool	Camp	License / deploy	Pick it when
Docling (IBM)	Pipeline toolkit	MIT toolkit, Apache-2.0 model, self-host	You want a permissive, batteries-included OSS pipeline with RAG-friendly output and swappable OCR
Granite-Docling-258M	End-to-end VLM	Apache-2.0, self-host	Cost-efficient self-hosted conversion inside Docling; most permissive license; edge-tolerant
PaddleOCR-VL (Baidu)	End-to-end VLM	Apache-2.0, self-host	Best-in-class open full-document accuracy when you can serve a VLM
dots.ocr, DeepSeek-OCR, MinerU, GLM-OCR	End-to-end VLM	Apache-2.0 (verify weights), self-host	Self-hosted layout-aware parsing; DeepSeek-OCR for very high throughput
Mistral OCR 3	Hosted OCR model	Commercial API, self-host on request	A flat low per-page price, an EU vendor, and a self-host escape hatch
Reducto	Agentic platform	Commercial SaaS + on-prem	Adversarial enterprise docs (dense tables, handwriting, forms) with SOC 2 / HIPAA / zero-retention needs
LlamaParse / LlamaCloud	Agentic platform	Hosted SaaS, credit-based	You are on LlamaIndex and want tiered parse quality (Fast to Agentic Plus) plus extraction
unstructured	Pipeline + hosted	Apache-2.0 + hosted	The hard part is format sprawl (hundreds of source types), not the single hardest PDF
Azure / AWS / Google doc-AI	Cloud doc-AI	Managed, per-page	You are deep in that cloud and want prebuilt extractors and a free tier with no model ops
Marker / Surya (Datalab)	Pipeline	Code OSS, weights OpenRAIL-M revenue-capped	Research, personal, or small-startup use; read the weight license before commercial use

One option is missing from the table on purpose. A general frontier VLM such as Gemini Flash or Pro parses pages well with no pipeline at all, and it is the baseline in most 2026 comparisons. It costs more per page than a dedicated small OCR VLM and can hallucinate or silently drop content on dense tables, so treat "just send the page to a frontier VLM" as a real, simple option that you benchmark against the specialists, not as a default.

Structured extraction

Once a page is parsed, turning it into a typed object is an orthogonal choice along one axis: do you control the decoder?

Library	Mechanism	Pick it when
Outlines / XGrammar	Constrained decoding (<code>guided_json</code>) inside the server	You self-host on vLLM/SGLang and want hard schema guarantees with zero post-hoc parsing
Instructor	Pydantic validation + auto-retry around a provider SDK	You call a hosted model and want simple validation and retries
BAML	Schema-Aligned Parsing of near-JSON output	The model returns malformed-but-close JSON and you want prompt discipline plus tolerant parsing

Choosing a parser

The decision is four axes: document difficulty, self-host versus API, volume and cost, and compliance.

- **Self-hosted document VLM** (PaddleOCR-VL, dots.ocr, DeepSeek-OCR, Granite-Docling, MinerU) when you can run a GPU, want no per-page bill, need data to stay in your VPC, and your documents are "normal hard": academic papers, reports, mixed text and tables. Serve them on vLLM/SGLang and you get constrained decoding for free.
- **Docling toolkit** as the safest pure-OSS default: a permissive pipeline with RAG-friendly output and the option to drop in Granite-Docling.
- **Mistral OCR 3** when you want a hosted API with a flat low per-page price (reported around \$2 per 1k pages, roughly half with batch, per Mistral, Dec 2025) and an EU vendor.
- **Reducto or LlamaParse Agentic** when documents are genuinely adversarial and accuracy matters more than per-page cost, and you want the correction loop, SLA, and compliance without building them.
- **unstructured** when the problem is format sprawl rather than the single hardest PDF.
- **Cloud doc-AI** (Azure, AWS, Google) when you are already in that cloud or need a specific prebuilt extractor (invoice, receipt, ID, lending).

A self-host parse plus schema-valid extraction is one request when constrained decoding lives inside the server:

```
# vLLM / SGLang OpenAI-compatible call: schema-constrained extraction
resp = client.chat.completions.create(
    model="paddleocr-vl",          # served locally on vLLM
    messages=[{"role": "user", "content": [
        {"type": "image_url", "image_url": {"url": page_png_data_uri}},
        {"type": "text", "text": "Extract the invoice as JSON."}]}],
    extra_body={"guided_json": Invoice.model_json_schema()}, # Outline
)
```

The retrieval stack: stores, embeddings, rerankers, hybrid search

Around the vector store sit three other pieces: embedding models that turn text (and increasingly images and PDFs) into vectors, rerankers that re-score a candidate set for precision, and RAG frameworks that wire ingestion, chunking, retrieval, fusion, and prompt assembly together.

Vector stores

The first question is almost always: do you already run Postgres? The table is condensed; the *Primary sources* hold the benchmark and pricing detail, all of it vendor-reported and version-sensitive.

Store	What it is	License	Pick it when
pgvector (+ pgvector scale)	Postgres extension with HNSW/IVFFlat; pgvector scale adds disk-resident StreamingDiskANN and label filtering	OSS (PostgreSQL)	You already run Postgres and your corpus is up to tens of millions of vectors. The right default for most RAG
Qdrant	Rust vector-native DB; strong filtering, quantization, ACORN	Apache-2.0 + cloud	You are not on Postgres, want a fast self-hosted engine, and need heavy metadata filtering and data residency
Weaviate	Vector-native with native hybrid (BM25 + vector) and built-in vectorizer modules	BSD-3 + cloud	You want hybrid search and embedding generation built in with minimal glue
Milvus / Zilliz	Distributed, disaggregated, billion-scale; Zilliz is the managed cloud	Apache-2.0 + managed	You are genuinely at hundreds of millions to billions of vectors and have the ops capacity
Pinecone	Managed serverless; Dedicated Read Nodes for heavy reads	Proprietary SaaS	You want fully managed, will pay for zero ops, and value predictable scaling
Turbopuffer	Managed vector + search on object storage with caching	Proprietary SaaS	Cost at scale dominates, data is mostly cold and bursty, or you have very many tenants/namespaces
LanceDB	Embedded/in-process on the Lance columnar format; multimodal	Apache-2.0 + cloud	Edge, desktop, notebooks, or multimodal pipelines where running a server is overhead
Chroma	Lightweight OSS store, dev-ergonomics first	Apache-2.0 + cloud	Prototyping or small RAG where simplicity beats scale (validate limits first)
Vespa / Elasticsearch / OpenSearch	Full search engines with vector support	Apache-2.0 / Elastic	Retrieval is really search (complex ranking, recommendation), or you already operate one

The trade-offs underneath those picks are durable even as the products churn. **One system versus vector-native:** Postgres-with-vectors removes an entire moving part (ops, backups, consistency, joins) at some cost in peak ANN throughput, and for most teams that trade is correct. **RAM versus object storage:** classic vector DBs keep the working set in RAM (fast, expensive), while object-storage-first designs (Turbopuffer, disk-resident DiskANN in pgvector scale and Milvus) trade some latency for much lower cost on large or cold corpora. **Managed versus self-hosted:** managed buys zero ops and predictable scaling, self-hosted buys cost control, data residency, and no per-query markup. **Filtering at scale:** if most queries are "vector search but only within this tenant or date range or permission set," filtered recall and filtered latency matter more than raw ANN speed, and Qdrant (ACORN) and pgvector scale (label filtering) are built for it.

Embeddings and rerankers

The database returns what the embedding model encoded, and the reranker fixes the ordering. Spending here usually beats switching databases. Treat the embedding model as a first-class versioned dependency: changing it means re-embedding the whole corpus.

MTEB and MMTEB scores below are not directly comparable across boards or versions (the English board, the multilingual board, and legacy versus current task sets use different task counts), so a higher number is not automatically a better model for your use case. The notes here are directional.

Model	Provider	Open / API	Note (as of 2026)
OpenAI text-embedding-3-large / -small	OpenAI	API	The boring-safe default; widely supported, Matryoshka dimension truncation; -small is cheap and good enough for many apps
Cohere Embed v4	Cohere	API	Multimodal (text + images), ~128K context, 100+ languages, Matryoshka dimensions; strong enterprise default
Voyage 4 family	Voyage AI (MongoDB)	API; nano open-weighted	Jan 2026; shared embedding space across models, domain variants (code/law/finance); default on MongoDB Atlas
Gemini Embedding (gemini-embedding-001)	Google	API	Cited near the top of the English MTEB board (per aggregators); multimodal direction. Verify board position before quoting
Qwen3-Embedding	Alibaba	Open (Apache-2.0)	Best open default for non-English and self-hosting; long context; cited near the top of MMTEB
BGE-M3	BAAI	Open (MIT)	Multilingual and multi-functional (dense/sparse/ColBERT in one); solid self-hosted baseline
Cohere Rerank 4 (Fast / Pro)	Cohere	API	Reranker; 32K context, multilingual, broadest production track record; on Bedrock/Azure/OCI
Voyage rerank-2.5	Voyage AI	API	Reranker; large free tier; natural pairing on MongoDB Atlas
mxbai-rerank v2	Mixedbread	Open (Apache-2.0)	Best open self-hosted reranker; cross-encoder on Qwen2.5

Hybrid search and reranking: the 2026 consensus

The default retrieval strategy in 2026 is **hybrid plus rerank**.

- **Hybrid search** runs BM25 (sparse, exact-term) and dense vector search in parallel and fuses the two ranked lists, typically with **Reciprocal Rank Fusion**: $\text{score} = \text{sum } 1 / (k + \text{rank})$, with $k = 60$ by convention. BM25 catches exact matches (codes, names,

rare terms) that dense vectors blur; dense catches paraphrase and concept that BM25 misses. Fusing by rank sidesteps the incompatible-score-scale problem.

- **Reranking** takes the top 50 to 100 fused candidates and re-scores them with a cross-encoder that reads query and document together, returning the top 5 to 10 into the prompt. **Late-interaction** models (ColBERT-style, MaxSim) sit between dense retrieval and cross-encoder reranking: token-level matching with precomputable document representations.
- Optionally de-duplicate with MMR before reranking so the context window is not spent on near-identical chunks.

A minimal pgvector hybrid retrieval, with the embed and rerank calls going through the gateway (see *Wiring*):

```
-- chunks(id, doc_id, content, embedding vector(1024), tsv tsvector)
WITH dense AS (
    -- dense candidates (HNSW /
    SELECT id, RANK() OVER (ORDER BY embedding <=> :query_vec) AS r
    FROM chunks ORDER BY embedding <=> :query_vec LIMIT 100
),
sparse AS (
    -- sparse candidates (BM25-
    SELECT id, RANK() OVER (ORDER BY ts_rank_cd(tsv, plainto_tsquery(:
    FROM chunks WHERE tsv @@ plainto_tsquery(:q) LIMIT 100
)
SELECT c.id, c.content,
    -- reciprocal rank fusion,
    COALESCE(1.0/(60+dense.r),0) + COALESCE(1.0/(60+sparse.r),0)
FROM chunks c
LEFT JOIN dense ON dense.id = c.id
LEFT JOIN sparse ON sparse.id = c.id
WHERE dense.id IS NOT NULL OR sparse.id IS NOT NULL
ORDER BY rrf DESC
LIMIT 50; -- hand these 50 to the reranker, keep the top 5-10 for t
```

Try changing `k` below: a small `k` lets the top of each list dominate, a large `k` flattens the weighting so agreement across both lists matters more.


```
# Reciprocal Rank Fusion: score = sum 1 / (k + rank), rank starting
bm25 = ["d3", "d1", "d7", "d2"] # sparse, exact-term ranking
dense = ["d2", "d3", "d5", "d1"] # dense, semantic ranking
k = 60

def rrf(lists, k):
    score = {}
    for ranked in lists:
        for rank, doc in enumerate(ranked, start=1):
            score[doc] = score.get(doc, 0.0) + 1.0 / (k + rank)
    return sorted(score.items(), key=lambda kv: kv[1], reverse=True)

for doc, s in rrf([bm25, dense], k):
    in_both = doc in bm25 and doc in dense
    print(f"{doc} rrf={s:.5f} {'(in both lists)' if in_both else ' '}")
# d3 leads: never #1 in either list, but solidly ranked in both.
```

Frameworks

For the orchestration layer, the common 2026 pattern is **LlamaIndex** (MIT) for ingestion and retrieval, **LangGraph** (MIT) for orchestration and agents when the app is agentic, **Haystack** (Apache-2.0) when you want explicit testable production pipelines, and **DSPy** when you want to compile and optimize the retrieval-plus-prompt rather than hand-tune it. Wire in **RAGAS** or **LangSmith** from day one. Retrieval without evaluation is guessing, which is exactly the case the eval-practice chapter makes (Chapter 43).

WHAT'S CONTESTED

The benchmarks that rank these tools are unusually unreliable. OmniDocBench, the main OCR benchmark, is widely described as saturated in 2026 (Jerry Liu / LlamaIndex), and public OCR leaderboards do not predict performance on financial, legal, or insurance documents. MTEB scores are not comparable across its English, multilingual, and legacy boards. And almost every vector-database latency, throughput, and cost claim is vendor-reported and not independently verified. The practical consequence: validate any parser or embedding choice on *your* documents with a small labeled set, and treat a leaderboard position as a shortlist signal, never as a decision.

A sensible default (2026)

For a typical team starting this stack in mid-2026 (opinionated, dated 2026-06):

- **Document intelligence: a two-tier setup.** Default tier is self-hosted OSS: **Docling** as the pipeline with a small open document VLM (**PaddleOCR-VL**, or **Granite-Docling** for the most permissive license) served on **vLLM**, paired with **Outlines/XGrammar** `guided_json` for schema-valid extraction. Escalation tier routes the pages your eval flags as low-confidence (dense tables, handwriting, bad scans) to **Reducto** or **LlamaParse Agentic**, or uses **Mistral OCR 3** as a cheap hosted fallback. If you cannot run a GPU at all, make Mistral OCR 3 the default and skip the OSS tier.

- **Vector store: pgvector on your existing Postgres**, adding **pgvectorscale** when you cross into tens of millions of vectors. If you are not on Postgres, **Qdrant** is the default vector-native alternative; if you refuse to run any infra, **Pinecone** or **Turbopuffer** (the latter when cost at scale or many tenants dominates).
- **Embeddings: start with a strong API model** (text-embedding-3-large for the safe path, Cohere Embed v4 for multimodal/long context, Voyage 4 on MongoDB Atlas), and go open weight (Qwen3-Embedding or BGE-M3) when you need self-hosting, data residency, or non-English depth.
- **Retrieval strategy: hybrid (BM25 + dense) with RRF, then a reranker.** Add the reranker (Cohere Rerank 4 managed, or mxbai-rerank v2 self-hosted) before you tune anything exotic. It is usually the highest-ROI single change.
- **Framework: LlamaIndex** for ingestion and retrieval, **LangGraph** for orchestration if agentic, and **RAGAS** or **LangSmith** wired in from day one.

The rationale is to minimize new infrastructure, put the quality budget where it pays (parse quality, embeddings, hybrid, rerank), and keep a clean upgrade path. You can swap the store later without rewriting the pipeline, but you cannot un-poison answers from a parser and retriever you never measured.

Wiring

Retrieval sits below the agent or generator and beside the model gateway, and document intelligence sits one step further out at the ingestion edge. The unifying seam is the gateway. Embedding models, rerankers, and self-hosted document VLMs are all just served models (Chapter 39), so ideally you call them **through the model gateway** alongside chat traffic. Then embedding, rerank, and parse calls are keyed, budgeted, and observed exactly like chat calls, and swapping `text-embedding-3-large` for a self-hosted `bge-m3`, or PaddleOCR-VL for Mistral OCR on hard pages, is a routing change rather than a code change. The end-to-end wiring of the whole stack is the subject of Chapter 44; here the point is narrower. Parse feeds extraction and chunking, both feed the vector store, the store feeds retrieval, retrieval feeds context into the generator or agent, and evaluation observes retrieval and the final answer. A bad parse poisons every layer above it, which is why parse quality is a retrieval and agent concern, not a preprocessing afterthought.

Primary sources

Vector stores:

- pgvector: <https://github.com/pgvector/pgvector>
- pgvectorscale / StreamingDiskANN: <https://github.com/timescale/pgvectorscale>; Tiger Data benchmark (vendor-reported) <https://www.tigerdata.com/blog/pgvector-is-now-as-fast-as-pinecone-at-75-less-cost>
- Qdrant: <https://qdrant.tech/documentation/>; hybrid search <https://qdrant.tech/articles/hybrid-search/>
- Weaviate: <https://weaviate.io/blog/hybrid-search-explained>
- Milvus / Zilliz: <https://milvus.io/docs>
- Pinecone cost model: <https://docs.pinecone.io/guides/manage-cost/understanding-cost>; Dedicated Read Nodes <https://www.pinecone.io/blog/dedicated-read-nodes/>
- Turbopuffer: <https://turbopuffer.com/blog/turbopuffer>; pricing log <https://turbopuffer.com/docs/pricing-log>
- LanceDB: <https://lancedb.github.io/lancedb/>; Chroma: <https://docs.trychroma.com/>; Vespa: <https://docs.vespa.ai/>

Embeddings and rerankers:

- MTEB leaderboard: <https://huggingface.co/spaces/mteb/leaderboard> (scores differ across boards and versions; not directly comparable)
- Voyage 4 family: <https://blog.voyageai.com/2026/01/15/voyage-4/>
- Cohere Rerank: <https://docs.cohere.com/docs/rerank-overview>; Rerank 4 (VentureBeat) <https://venturebeat.com/ai/coheres-rerank-4-quadruples-the-context-window-to-cut-agent-errors-and-boost>
- Cohere Embed: <https://docs.cohere.com/docs/cohere-embed>; OpenAI embeddings: <https://platform.openai.com/docs/guides/embeddings>
- BGE-M3: <https://huggingface.co/BAAI/bge-m3>; Qwen3-Embedding: <https://huggingface.co/Qwen>; mxbai-rerank v2: <https://www.mixedbread.com/>

Hybrid search and RAG frameworks:

- Hybrid search / RRF / reranking overview: <https://www.digitalapplied.com/blog/hybrid-search-bm25-vector-reranking-reference-2026>
- ColBERT / late interaction (survey): <https://arxiv.org/pdf/2502.14822>; Agentic RAG survey: <https://arxiv.org/pdf/2501.09136>
- LlamaIndex: <https://docs.llamaindex.ai/>; LangGraph: <https://langchain-ai.github.io/langgraph/>; Haystack: <https://docs.haystack.deepset.ai/>; DSPy: <https://dsp.py.ai/>; RAGAS: <https://docs.ragas.io/>

Document intelligence:

- Docling / Granite-Docling: <https://www.ibm.com/new/announcements/granite-docling-end-to-end-document-conversion>, <https://huggingface.co/ibm-granite/granite-docling-258M/blob/main/README.md>
- unstructured: <https://docs.unstructured.io/open-source/introduction/overview>
- LlamaParse / LlamaCloud pricing: <https://www.llamaindex.ai/pricing>
- Reducto: <https://reducto.ai/pricing>, <https://reducto.ai/blog/reducto-series-b-funding>
- Mistral OCR 3: <https://mistral.ai/news/mistral-ocr-3/>, <https://mistral.ai/pricing/>
- OmniDocBench: <https://github.com/opendatalab/OmniDocBench>; "OmniDocBench is saturated" (Jerry Liu): <https://www.llamaindex.ai/blog/omnidocbench-is-saturated-what-s-next-for-ocr-benchmarks>
- PaddleOCR-VL / GLM-OCR scores: <https://huggingface.co/PaddlePaddle/PaddleOCR-VL/discussions/29>, <https://arxiv.org/html/2603.10910v1>
- Marker / Surya (Datalab) licensing: <https://github.com/datalab-to/surya>
- Cloud doc-AI pricing: <https://aiproductivity.ai/blog/document-ai-cost-comparison/>
- Structured extraction (Instructor / BAML / Outlines / XGrammar): <https://techsy.io/en/blog/best-llm-structured-output-libraries>

Further reading

- Khattab and Zaharia, "ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT," 2020. [arXiv:2004.12832](https://arxiv.org/abs/2004.12832)
- Singh et al., "Agentic Retrieval-Augmented Generation: A Survey," 2025. [arXiv:2501.09136](https://arxiv.org/abs/2501.09136)
- Late-interaction retrieval survey, 2025. [arXiv:2502.14822](https://arxiv.org/abs/2502.14822)
- Liu, "OmniDocBench is Saturated: What's Next for OCR Benchmarks," LlamaIndex, 2026. <https://www.llamaindex.ai/blog/omnidocbench-is-saturated-what-s-next-for-ocr-benchmarks>
- OmniDocBench project (benchmark, versions, metrics). <https://github.com/opendatalab/OmniDocBench>

- MTEB / MMTEB leaderboard and methodology.
<https://huggingface.co/spaces/mteb/leaderboard>

43. Evaluation and Observability

A model that tops a public leaderboard can still misbehave in your product. The benchmark measured a generic capability on a fixed dataset. Your app runs a *specific* prompt, over *your* retrieval context, inside an agent loop, against live user input that drifts week to week. The discipline that closes this gap has two halves, and by mid-2026 they have largely fused into one tooling category. This chapter is a practical guide to choosing and wiring that tooling.

Where it sits in the stack: [Chapter 27](#) measures models in the abstract, [Chapter 28](#) covers how to grade open-ended output, and [Chapter 29](#) covers multi-step agent evaluation. This chapter measures *your running system*. It consumes the serving layer ([Chapter 39](#)) and the model gateway ([Chapter 44](#)), and it produces the feedback signal that feeds back into agents ([Chapter 41](#)) and fine-tuning ([Chapter 40](#)): eval datasets become preference data, and agent traces are where failures actually surface.

AS OF MID-2026

This is a dated snapshot (June 2026). Prices, version numbers, acquisitions, and "best for" claims move fast in this category, and 2026 has been a year of heavy consolidation. Treat every specific dollar figure and ranking below as indicative and source-attributed, not timeless, and re-verify current values and roadmaps before committing. Lead with the durable parts: the categories, the trade-offs, and the wiring.

The two halves of the loop

Evaluation asks "is the output good?" against criteria you define. It runs offline, on a curated dataset, in CI, before you ship, and online, sampled from production traffic and scored continuously. The unit of work is a *score*. Scores come in three flavors that you mix:

- **Deterministic assertions:** regex, JSON-schema validation, exact match, a latency or cost budget. Cheap, fast, and the first line of defense.
- **Reference-based metrics:** BLEU and ROUGE and their retrieval cousins like context precision, context recall, and faithfulness.
- **LLM-as-judge:** a second model grades the first against a rubric. This is the only practical way to score open-ended quality at scale, and the bulk of [Chapter 28](#) is about doing it without fooling yourself.

Observability asks "what actually happened?" It captures a *trace*: the full tree of a request as it moves through your system, every LLM call, retrieval, tool invocation, and nested agent step, with inputs, outputs, timing, token counts, and cost attached to each span. Without it you are debugging a non-deterministic distributed system by reading flat logs.

The two halves connect because **you evaluate the traces you observe**. A production trace becomes a test case. A judge runs over sampled spans to produce online scores. A regression caught in a dashboard becomes a fixture in the offline suite. This loop, observe then score then curate then re-test, is the practical core of "continuously evaluate an LLM app", and it is the figure to keep in mind for the rest of the chapter ([Figure 43.1](#)).

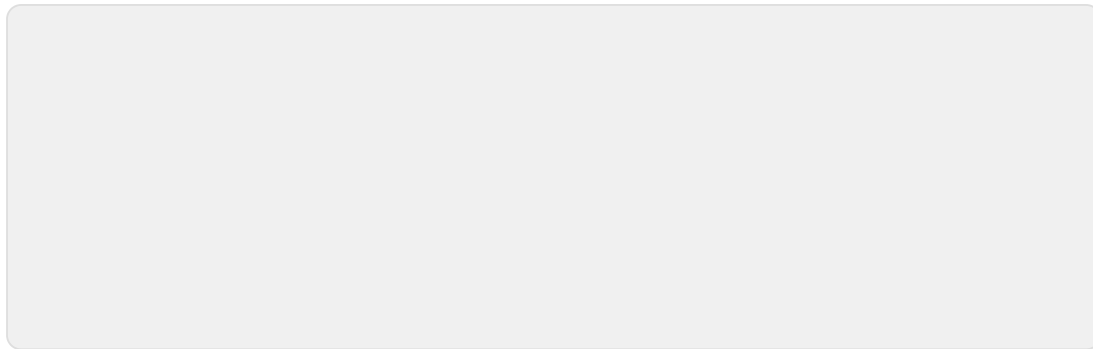


Figure 43.1. The continuous-evaluation loop. Production traces are observed, sampled spans are scored by judges and assertions, failures are curated into a regression dataset, and that dataset gates the next release. Every component calls the model gateway rather than providers directly.

The 2026 shakeout

Two acquisitions reshaped the landscape in early 2026, and they tell you the direction of travel: the standalone eval/observability tool is becoming a feature of a larger data or model platform.

- OpenAI **acquired Promptfoo** (announced March 9, 2026) and is **deprecating its own hosted Evals product** (read-only October 31, 2026, shutdown November 30, 2026), steering users toward Promptfoo and Datasets. The open-source `openai/evals` repo persists; the hosted dashboard does not.
- ClickHouse **acquired Langfuse** (January 16, 2026, alongside ClickHouse's \$400M Series D). Langfuse states its OSS and self-host commitment continues.

The lesson is not which logo owns which tool. It is that the churn is real, so favor tools whose data and instrumentation you can carry elsewhere. That is the case for OpenTelemetry below.

Key players

All license, pricing, and acquisition claims below are as of mid-2026 and attributed in *Further reading*. Pricing is the published list shape at time of writing and changes often, so treat exact dollars as indicative. Judge accuracy and "best for" claims are workload-dependent and not independently benchmarked here.

Eval harnesses and frameworks

Name	What it is	License / model	Best for
lm-evaluation-harness (EleutherAI)	The de facto academic harness: 60+ standard benchmarks, hundreds of subtasks, reproducible configs, the backend behind many leaderboards	Open source (MIT)	Benchmarking a <i>model's</i> raw capability reproducibly
Inspect (UK AISI)	Framework for model and agentic evals: tool use, multi-turn, model-graded scoring; <code>inspect_evals</code> ships 200+ ready evals	Open source (MIT)	Safety and agentic evaluation, sandboxed tool-use tasks
Promptfoo	CLI + config-driven eval and red-teaming: assertions, LLM-as-judge, model comparison, RAG/agent tests, 50+ vuln categories	Open source (MIT) + paid Enterprise; acquired by OpenAI (Mar 2026)	App-level evals and security red-teaming you can drop into CI without notebooks
DeepEval (Confident AI)	Pytest-native eval framework: 50+ metrics (LLM-as-judge, RAG, agentic, safety), G-Eval, DAG metrics	Open source + Confident AI cloud	Teams wanting unit-test-style regression on LLM output
RAGAS	Reference-free RAG metrics: faithfulness, answer relevancy, context precision/recall	Open source	Quick RAG-pipeline scoring, often consumed <i>through</i> other frameworks
MLflow 3 Evaluate	GenAI eval + tracing inside the MLflow platform	Open source	MLOps shops already standardized on MLflow / Databricks

Tracing and observability platforms

Name	What it is	License / model	Best for
Langfuse	OSS LLM engineering platform: tracing, prompt management, evals, datasets	MIT, self-hostable; cloud Hobby free / Core \$29 / Pro \$199 / Enterprise \$2,499 per mo, billed per unit	Teams wanting a fully self-hostable OSS observability + eval stack
LangSmith (LangChain)	Framework-agnostic observability + eval + prompt platform; agent ops	Proprietary SaaS; Developer free / Plus \$39/seat/mo / Enterprise custom	Teams in or near the LangChain/LangGraph ecosystem wanting a polished managed platform
Arize Phoenix	OSS observability + eval built on OpenTelemetry , instrumented via OpenInference	Open source (ELv2-style); Arize AX is the commercial cloud	Teams wanting OTel-native, vendor-neutral tracing that runs locally with no account
Braintrust	Eval-first commercial platform: experiments, scoring, tracing, prompt playground	Proprietary SaaS; Starter free / Pro \$249/mo / Enterprise custom	Product teams wanting eval, tracing, and a prompt playground tightly integrated
W&B Weave (Weights & Biases)	LLM tracing, evaluation, prompt/version tracking inside the W&B ecosystem	Proprietary SaaS	Teams already on W&B for experiment tracking
Confident AI	Cloud platform behind DeepEval: datasets, tracing, monitoring	Proprietary SaaS	DeepEval users wanting a managed collaboration layer

The substrate

Name	What it is	License / model	Best for
OpenTelemetry GenAI semantic conventions	Vendor-neutral spec for LLM/agent spans and metrics (<code>gen_ai.*</code> attributes), under CNCF OTel	Open standard (Apache 2.0)	The wire format that lets you switch backends without re-instrumenting
OpenInference (Arize)	OTel-aligned conventions + drop-in auto-instrumentation across providers, agent frameworks, RAG libs	Open source	Practical auto-instrumentation today, ahead of the spec stabilizing

As of mid-2026 the OTel GenAI spec is still largely experimental ("Development" stability), though core attributes (`gen_ai.system` , `gen_ai.request.model` , token counts) are stabilizing. You opt into the latest with `OTEL_SEMCONV_STABILITY_OPT_IN` . OpenInference

runs ahead of the spec and emits to any OTEL backend (Datadog, Grafana, Jaeger), which is why it is the practical choice today.

How to choose

The first cut is **capability eval versus application eval**, because they are different jobs done by different tools.

CONSTRAINT ARROW

The serving layer dictates the shape of your eval harness. Because you point evals and judges at the **model gateway** (Chapter 44) rather than at providers, an A/B between two models becomes a routing rule, not a code change, and judge credentials become scoped virtual keys with their own budget rather than raw provider secrets. The gateway you chose in Chapter 39 is what makes continuous online evaluation cheap to operate.

Pick a **capability harness** when you are measuring a *model*: comparing checkpoints, validating a fine-tune (Chapter 40), or reproducing a leaderboard number. Use **Inspect** when the task is agentic, tool-using, or safety-flavored, and **lm-evaluation-harness** for classic academic benchmarks.

Pick an **application eval tool** when you are measuring *your system*: this prompt, this RAG pipeline, this agent, against criteria that matter to your product. Within application eval, four axes decide the tool:

Axis	Choose this	When
CI-native vs platform-native	Promptfoo (config/CLI) or DeepEval (pytest)	Evals must live in the repo and gate merges with no external account
	Braintrust or LangSmith	You want a hosted UI for non-engineers to curate datasets and compare runs visually
Self-host vs managed	Langfuse or Phoenix	Data residency, cost control, or vendor-neutrality matter and you can run infra
	LangSmith , Braintrust , Confident AI	You would rather not operate it and will pay for polish
OTel-native vs proprietary SDK	Phoenix / OpenInference	You want one instrumentation that targets many backends and avoids lock-in
	LangSmith , Braintrust	Smoother out of the box, at the cost of coupling your code to one vendor
Red-teaming in scope?	Promptfoo	You need the deepest built-in adversarial scanning (OWASP LLM Top 10, jailbreaks, PII, prompt injection)

LLM-as-judge: cost versus trust

A judge is roughly 500x to 5000x cheaper than human review and can approach human-to-human agreement, *but only when calibrated*. An uncalibrated judge is a confident liar. As of mid-2026 the consensus practice, covered in depth in Chapter 28, is:

- **Validate against a human-labeled sample.** Report Cohen's kappa or Krippendorff's alpha and target around 0.8 agreement before you trust the judge to gate anything.
- **Control for position bias.** Score both A/B and B/A orderings and count only consistent wins.
- **Control for verbosity and self-preference bias.** Never use the same model family as both generator and judge.
- **Use explicit, criterion-separated rubrics** (G-Eval style). Split hard rules into a DAG of checks rather than one fuzzy prompt.

Change the human base rate and watch raw agreement stay high while Cohen's kappa collapses, which is why "the judge agrees 90% of the time" proves almost nothing.

```
import numpy as np

# A lazy judge that says "good" almost always, scored against human
# Raw agreement looks great, but Cohen's kappa corrects for chance a
def kappa(m):
    m = np.array(m, float); n = m.sum()
    po = np.trace(m) / n # raw agreement
    pe = (m.sum(0) * m.sum(1)).sum() / n**2 # agreement expected
    return po, (po - pe) / (1 - pe)

n = 1000
for good_rate in [0.70, 0.80, 0.90, 0.95]:
    good = int(n * good_rate); bad = n - good
    caught = int(bad * 0.20) # judge flags only 2
    miss = int(good * 0.02) # and wrongly flags
    cm = [[caught, miss], [bad - caught, good - miss]]
    po, k = kappa(cm)
    verdict = "trust" if k >= 0.8 else "do NOT trust"
    print(f"human good={good_rate:.0%} agreement={po:.2f} kappa={k:.2f}")
```

A sensible default

For a typical team shipping one or two LLM features and wanting a continuous eval loop without standing up a platform team, here is a stack that is OSS-first, self-hostable, OTel-native, and CI-gated, with a clean upgrade path to a managed platform later.

Job	Default pick	Managed alternative
Observability / tracing	Langfuse (self-host or free/Core cloud)	LangSmith (Plus) or Braintrust
App evals in CI	Promptfoo (assertions, comparison, red-team) + DeepEval (pytest metrics)	Braintrust / LangSmith eval modules
Model / capability eval	Inspect (agentic, safety) or lm-evaluation-harness (academic)	hosted leaderboards
Instrumentation	OpenTelemetry GenAI via OpenInference auto-instrumentation	proprietary SDK

Two caveats to this default. First, the churn: Promptfoo is now OpenAI-owned and Langfuse is ClickHouse-owned, so re-confirm roadmap and licensing before you commit long-term. Second, do not add a vendor you do not need. Teams already standardized on Databricks should prefer **MLflow 3 Evaluate**, and teams on Weights & Biases should prefer **Weave**, over bolting on a new platform.

Wiring

The loop closes through the model gateway (Chapter 44). The app under test, the evals, and the judge all call the gateway, never providers directly. That single decision is what makes swapping models a config change and keeps judge credentials as budget-capped virtual keys.

Here is a minimal Promptfoo config that points at the gateway and uses an LLM judge with a deterministic guard in front of it. Save it as `promptfooconfig.yaml` and run `npx promptfoo@latest eval`.

```
# promptfooconfig.yaml
providers:
  - id: openai:chat:gpt-5          # resolved by the gateway, not
    config:
      apiUrl: https://gateway.internal/v1
      apiKey: ${GATEWAY_VKEY}      # scoped virtual key, budget-ca
prompts:
  - "Answer using only the context:\n{{context}}\n\nQ: {{question}}"
tests:
  - vars: { question: "...", context: "..." }
    assert:
      - type: is-json              # deterministic guard, runs fir
      - type: llm-rubric           # LLM-as-judge, calibrated rubr
        value: "Answer is faithful to the context and cites it."
        provider: openai:chat:gpt-5-mini  # judge via gateway, diff
```

To gate merges, run that one command in CI and fail the job on a regression:

```
# CI step: fail the build if any eval assertion regresses
npx promptfoo@latest eval -c promptfooconfig.yaml --no-cache --output
# exit code is non-zero when assertions fail, which fails the pipeline
```

The heartbeat of the whole system is the promotion step. A production trace from Langfuse or Phoenix that fails an online judge gets promoted into the `tests:` set above, so the offline suite hardens against the failures you actually see in production rather than the ones you imagined at design time. Without that promotion step you have two disconnected tools. With it you have a loop.

The capability, efficiency, trust lens

Capability: observability turns a non-deterministic system into something you can debug, and evaluation turns "it feels better" into a number that gates a release. **Efficiency:** the gateway makes online judging cheap enough to run continuously, and deterministic assertions catch the cheap failures before a judge call is ever made. **Trust:** an LLM judge is only as trustworthy as its calibration against humans, and the OTel substrate is what lets you carry your traces and scores away from any vendor that changes its terms. The loop is worth building only if all three hold, and in mid-2026 the tooling has finally matured enough that they can.

Further reading

- lm-evaluation-harness: <https://github.com/EleutherAI/lm-evaluation-harness>
- Inspect AI: <https://inspect.aisi.org.uk/> and the evals suite at https://ukgovernmentbeis.github.io/inspect_evals/
- Promptfoo docs: <https://www.promptfoo.dev/docs/intro/>; red-teaming: <https://www.promptfoo.dev/docs/red-team/>; OpenAI acquisition (Mar 9, 2026): <https://www.promptfoo.dev/blog/>
- OpenAI Evals OSS repo: <https://github.com/openai/evals>; hosted Evals deprecation timeline: <https://developers.openai.com/api/docs/deprecations>
- DeepEval: <https://deepeval.com/docs/introduction>; Confident AI: <https://www.confident-ai.com/>
- RAGAS (via DeepEval): <https://deepeval.com/docs/metrics-ragas>
- MLflow GenAI eval: <https://mlflow.org/docs/latest/ml/evaluation/>
- Langfuse: <https://langfuse.com/docs/observability/overview>; pricing: <https://langfuse.com/pricing>; ClickHouse acquisition (Jan 16, 2026): <https://clickhouse.com/blog/clickhouse-acquires-langfuse-open-source-llm-observability>
- LangSmith: <https://www.langchain.com/langsmith>; pricing: <https://www.langchain.com/pricing>
- Arize Phoenix: <https://arize.com/docs/phoenix>
- Braintrust: <https://www.braintrust.dev/>; pricing: <https://www.braintrust.dev/pricing>
- W&B Weave: <https://wandb.ai/site/weave/>
- OpenTelemetry GenAI semantic conventions: <https://opentelemetry.io/docs/specs/semconv/gen-ai/gen-ai-spans/>; 2026 overview: <https://opentelemetry.io/blog/2026/genai-observability/>
- LLM-as-judge practice (2026): DeepEval, <https://deepeval.com/blog/llm-as-a-judge>; Confident AI, <https://www.confident-ai.com/blog/why-llm-as-a-judge-is-the-best-llm-evaluation-method>

44. Wiring a 2026 Stack

The book opened by promising that a capability has a lifecycle, from compute up to a governed behavior, and that the layers connect in a specific order (Chapter 1). This chapter is where the promise is paid. It is the capstone: a concrete, end-to-end reference architecture that wires the pieces the practical chapters built up into one running system. By the end you should be able to draw the whole stack on a whiteboard, name the seam between every pair of layers, and write the glue that holds them together.

The trap to avoid is treating this as a seventh landscape survey. The six chapters before it already surveyed models (Chapter 38), serving and compute (Chapter 39), training and fine-tuning (Chapter 40), agents and sandboxes (Chapter 41), retrieval (Chapter 42), and evaluation (Chapter 43). This chapter does not re-list their players. It answers a different question: given those choices, how do the parts compose into something you can run, observe, and pay for?

AS OF MID-2026

This is a dated snapshot (June 2026). Specific products, prices, versions, and "best for" claims move fast, and 2026 has been a year of heavy churn (Promptfoo to OpenAI, Langfuse to ClickHouse, the MCP spec mid-revision). Treat every named product and dollar figure below as indicative and source-attributed, not timeless, and re-verify before you commit. Lead with the durable parts: the contracts, the seams, and the archetypes. Those outlast any version string.

The thesis: three contracts and one discipline

A 2026 stack composes not because the tools agree on features, they do not, but because they agree on three wire-format contracts. Each contract is a narrow, boring interface that lets you swap what is behind it without rewriting what is in front of it. The whole architecture is legible once you see that everything meets at these three seams.

1. **Model calls speak the OpenAI-compatible HTTP shape.** `/v1/chat/completions`, `/v1/responses`, `/v1/embeddings`. By 2026 every serving engine exposes it (Chapter 39), every hosted frontier API offers it or a near-twin, and every gateway normalizes to it. This is what makes a self-hosted vLLM model and a hosted Claude endpoint interchangeable behind one name.
2. **Tool and agent traffic speaks MCP.** The Model Context Protocol is to tool integration what the OpenAI shape is to model calls: the common interface that lets one tool server be reused by every agent framework (Chapter 41). By 2026 it is the default tool layer under almost every framework, and the stable spec is `2025-11-25` with a stateless revision in its validation window.
3. **Telemetry speaks OpenTelemetry GenAI spans.** The `gen_ai.*` semantic conventions are the vendor-neutral format for LLM, retrieval, and agent spans (Chapter 43). Instrument once and you can point at Langfuse, Phoenix, Datadog, or Grafana without re-instrumenting.

The one discipline that ties the contracts together is **virtual-key custody**. No application, agent, eval harness, or sandbox ever holds a raw provider credential. They hold a short-lived *virtual key* minted by the gateway, scoped to a model allowlist, a budget, and a rate limit. Rotating a provider secret or capping a runaway agent becomes a server-side operation with no client redeploy. The same idea extends to compute: secrets live outside the sandbox and reach the agent as scoped tokens, never baked into the box (Chapter 34).

The component that all three contracts converge on is the **model gateway**, and this chapter is its home. There is no separate gateway chapter in this book, the gateway is the connective

tissue, so its landscape and the one comparison table this chapter owns live here.

CONSTRAINT ARROW

The gateway is where a lower layer's economics reach up and shape the whole application. Because every model call routes through one address, the per-token price of a frontier model (Chapter 37) and the per-token cost of your self-hosted serving (Chapter 16) become a routing rule, not a code change. The decision to escalate a hard step to an expensive model, or to fall back to a cheap one under load, is made once at the seam and inherited by every caller above it.

The gateway: the one table this chapter owns

A *model gateway* (also "LLM gateway" or "AI gateway") is the control plane between your application code and the many providers it talks to. The thin description is a reverse proxy that speaks one API and fans out to many. The real job is to centralize five concerns nobody wants scattered through application code: a **unified API**, **key custody**, **cost tracking**, **fallback and routing**, and **budgets and policy**. The newest gateways extend that role from plain token traffic to agent-native protocols (MCP for tools, A2A for agent-to-agent), making the gateway the single policy point for the whole stack's traffic.

All claims below are as of mid-2026 and attributed in *Further reading*. Vendor performance numbers are not independently verified here.

Name	What it is	License / model	Best for
LiteLLM	Python SDK + self-hostable proxy translating 100+ providers to OpenAI format; virtual keys, budgets, spend logs, UI	Open source (MIT-style core) + paid Enterprise	Teams with DevOps wanting the broadest provider coverage, self-hosted, in a Python stack. The de facto OSS default
OpenRouter	Managed SaaS aggregator: one endpoint, prepaid credits, 300+ models, BYOK	Hosted SaaS; ~5.5% fee on credit purchase	Solo devs and small teams wanting zero-setup access to a huge catalog
Portkey	AI gateway with guardrails, virtual keys, fallbacks, caching, prompt management, observability	OSS core + managed SaaS; self-host on enterprise	Enterprises needing governance: PII redaction, jailbreak detection, audit, compliance
agentgateway	Rust data plane for HTTP/gRPC and AI-native protocols: LLM + MCP + A2A gateway in one	Apache 2.0; Linux Foundation (Agentic AI Foundation)	Platform teams wanting one proxy for LLM, tool (MCP), and agent (A2A) traffic
Envoy AI Gateway	Kubernetes-native gateway on Envoy + Gateway API; LLM routing, token rate limiting, cost tracking, InferencePool	Open source (Envoy/CNCF ecosystem)	Platform teams already on Envoy/Istio/Gateway API wanting in-cluster control
Cloudflare AI Gateway	Platform-native gateway: observability, caching, rate limiting, cost tracking; Unified Billing	Hosted SaaS; core features free	Teams already on Cloudflare/Workers wanting no infra to run
Bifrost (Maxim AI)	Go LLM gateway: adaptive load balancing, cluster mode, guardrails, 1000+ models	Open source + commercial	Teams wanting a high-throughput, low-overhead self-hosted LiteLLM alternative
Lux (latere.ai)	The author's own gateway: one address to OpenAI, Anthropic, Gemini, OpenRouter, Ollama; sealed credentials, virtual keys, spend caps, hooks, audit	Latere product family	The Latere stack, and a worked example in this book. <i>Disclosed as the author's own infrastructure; illustrative, not a neutral recommendation</i>

How to choose splits on two axes that matter more than feature checklists: **managed vs self-hosted**, then **plain-LLM vs agent-native**.

- Pick a **managed** gateway (OpenRouter, Cloudflare, Portkey managed) for zero infrastructure and fast access to a broad catalog, when you are comfortable that provider keys and request metadata transit a third party. Best for prototypes and small teams.
- Pick a **self-hosted** gateway (LiteLLM, agentgateway, Envoy, Bifrost) when provider secrets must stay in your environment, or you need data residency and per-request audit you own. Best for regulated industries and platform teams.
- Reach for an **agent-native** gateway (agentgateway) when your roadmap needs MCP tool calls and agent-to-agent traffic under the same policy point as token traffic, instead of bolting agent governance onto a chat-only proxy later.

On performance: vendors publish dramatic latency numbers ("50x faster", "sub-millisecond overhead"), measured under inconsistent conditions and not verified here. For almost all workloads, provider and model latency dominate gateway overhead by orders of magnitude. Benchmark on your own traffic before you let overhead break a tie.

The sensible default for a team that wants control of its keys and costs without a large platform investment is to **self-host LiteLLM and pair it with an observability layer**, because it has the broadest coverage and almost everything else interoperates with it. (The author's own stack uses Lux; it appears here as a worked example of these patterns, not as a neutral pick over the options above.)

The reference architecture

Figure 44.1 merges the per-layer wiring diagrams from the practical chapters into one picture. Read it top to bottom as a request flow and side to side as the contracts. The gateway is the hub; everything that needs a model, an embedding, a reranker, or a judge call goes through it. The sandbox is the agent's compute seam, the counterpart to the gateway's model seam. Retrieval sits beside the agent. Eval and observability tap the gateway and the spans. The compute and orchestration layer is the floor everything stands on.

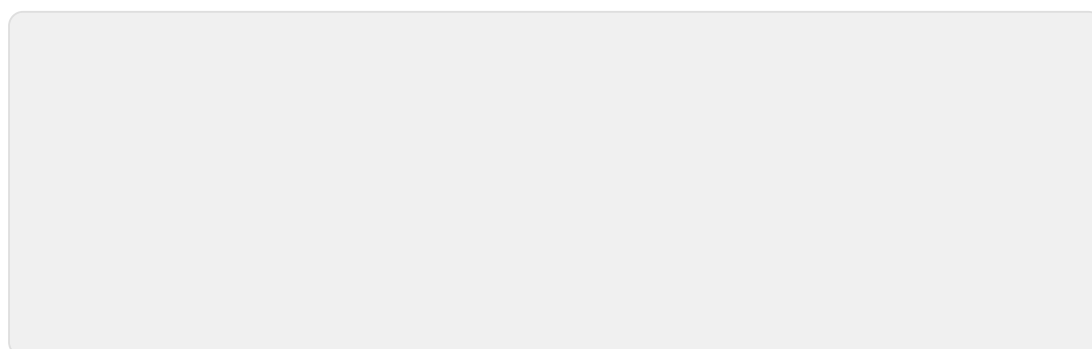


Figure 44.1. A 2026 end-to-end reference architecture. Everything that needs a model call routes through the gateway (the OpenAI-compatible seam); tool calls route through the MCP gateway; every component emits OpenTelemetry GenAI spans. Apps, agents, evals, and sandboxes hold scoped virtual keys, never raw provider credentials. The compute and orchestration layer is the floor the whole stack runs on.

Three things make this diagram hold up in production, and each is one of the contracts from the thesis:

- **One base URL, OpenAI-shaped.** The app, the retriever (for embed and rerank), the eval harness, and the judge all point at the same gateway address. Swapping a model or adding a fallback is configuration, not code. A self-hosted `bge-m3` embedder and a hosted Cohere reranker are addressable by name through the same seam ([Chapter 42](#)).

- **One tool protocol, MCP.** Agents do not dial arbitrary tool servers. MCP traffic flows through a gateway that owns the credentials and the allowlist, the same way the model gateway owns token traffic (Chapter 41).
- **One telemetry format, OTel.** Every component emits `gen_ai.*` spans, so the observability backend stays swappable and the eval loop can read any trace (Chapter 43).

The three arcs

The book built the stack along three arcs. The reference architecture is where they meet. Walking each arc is the fastest way to see how the whole thing flows.

From chatbot to agent

A chatbot is a single model call: prompt in, completion out. An agent is a *loop* around that call: the model emits a tool request, the runtime executes it, feeds the result back, and the model decides what to do next, possibly for many turns (Chapter 21, Chapter 23). Three additions turn the first into the second, and all three appear on the right and top of Figure 44.1.

- **Grounding via retrieval.** Instead of trusting the model's weights, the agent retrieves current, private, or authoritative context and injects it (Chapter 25, Chapter 42). The 2026 consensus pipeline is hybrid search (BM25 plus dense vectors, fused with reciprocal rank fusion) then a reranker, with the embedding and reranker models called *through the gateway* so they are keyed and budgeted like any other model call. Prefer retrieval over stuffing a million-token context window: long context is expensive, and the price cliffs above 200K tokens (Chapter 37) usually make retrieval both cheaper and more faithful.
- **Tools via MCP and a sandbox.** Reading a result is the easy half. Acting, running code, hitting an API, writing a file, is where safety stops being about the model and becomes about the *runtime* (Chapter 41). A jailbreak filter on the gateway does nothing against `rm -rf` the model was tricked into emitting. The enforcement layer is the sandbox: a microVM (Firecracker, used by E2B, Fly, Vercel) or userspace kernel (gVisor, used by Modal) that bounds the blast radius, with default-deny egress, hard timeouts, and human-in-the-loop gates for irreversible actions (Chapter 34).
- **Memory via a checkpointer.** A long-running agent must survive a restart, so its state lives in a durable, checkpointed store (LangGraph's checkpointer, Pydantic AI's durable execution), not in process memory (Chapter 22). This is what lets a human pause, inspect, and resume a task mid-flight.

The constraint arrow runs downward through this arc: the agent cannot be more reliable than the gateway, serving, and sandbox layers beneath it, which is exactly why "durable execution" and "checkpointing" became the headline 2026 framework features. They are the framework's attempt to survive the failures of the layers below.

From inference to training

Most of a 2026 stack is *inference*: serving a fixed model behind the OpenAI seam. But the same gateway address that fronts a hosted Claude endpoint can front a model you serve yourself, and the same address can front a model you *trained* yourself (Chapter 39, Chapter 40). The arc from inference to training is the arc from renting capability to owning it, and the gateway is what makes the two interchangeable.

The economics decide where on the arc you sit. With a closed API you do not run the inference server: the lab owns batching, KV-cache, speculative decoding, and the accelerators (Chapter 16), and you consume them as knobs (cached-input pricing, priority tiers). When volume and control justify it, you cross to self-hosted serving (vLLM by default, SGLang for prefix-heavy agent and RAG traffic), and the per-token cost moves from a vendor invoice to a GPU bill you schedule yourself. Further along the arc, you fine-tune (Chapter 9) or post-train a model and serve the result behind the same seam. The training run itself lands on the compute

floor, gang-scheduled by Slurm or by Kubernetes with a gang-scheduling add-on ([Chapter 31](#), [Chapter 30](#)).

The deciding variable is **sustained utilization**. Below roughly half a GPU's worth of steady load, a serverless or managed-inference platform (Modal, Baseten, Fireworks, Together) is cheaper than paying for idle reserved hardware. Above it, reserved clusters on a neocloud or hyperscaler win. The constraint arrow here is the one from [Chapter 3](#): the serving cost of a token, paid forever, is what justifies over-training a smaller model once, which is why a model's size is set by how many tokens it will decode over its lifetime, not by its training budget alone.

Change the two price knobs below and watch where the build-vs-rent crossover lands: serverless wins below it, a flat reserved bill wins above it.

```
import numpy as np
# Illustrative figures only (mid-2026 indicative, not real quotes).
serverless_per_gpu_hour = 4.0    # $/GPU-hour, pay only while running
reserved_per_gpu_hour   = 2.0    # $/GPU-hour, billed 24/7 whether id
u = np.linspace(0.01, 1.0, 100) # sustained utilization
serverless = serverless_per_gpu_hour * u * 24 * 30 # pay per busy
reserved    = reserved_per_gpu_hour * 24 * 30      # flat monthly
crossover = reserved_per_gpu_hour / serverless_per_gpu_hour
print(f"crossover utilization = {crossover:.0%} of one GPU")
print("below it serverless wins, above it reserved wins")
import matplotlib.pyplot as plt
plt.plot(u, serverless, label="serverless / managed (pay per use)")
plt.axhline(reserved, color="C1", label="reserved (flat 24/7)")
plt.axvline(crossover, color="gray", ls="--")
plt.xlabel("sustained utilization (fraction of one GPU)")
plt.ylabel("monthly cost ($)")
plt.legend(); plt.title("Build vs rent crossover"); plt.show()
```

From usage to eval

The third arc closes the loop. Every model and tool call the app makes is *usage*, and every unit of usage is something you can observe and score ([Chapter 43](#)). A model that tops a public leaderboard can still misbehave in your product, because the benchmark measured a generic capability ([Chapter 27](#)) while your app runs a specific prompt over your retrieval context inside an agent loop.

The loop has two halves that meet at the OTel seam. **Observability** captures the trace: the full tree of a request, every LLM call, retrieval, and tool invocation, with inputs, outputs, timing, tokens, and cost on each span. **Evaluation** scores those traces with deterministic assertions, reference-based metrics, and LLM-as-judge ([Chapter 28](#)), where a second model grades the first against a calibrated rubric. The two connect because you evaluate the traces you observe: a production trace that fails an online judge gets promoted into the offline regression suite, which gates the next release ([Chapter 29](#)). Because the app, the evals, and the judge all call the gateway, an A/B between two models is a routing rule and the judge runs on a scoped, budget-capped virtual key.

This arc feeds the other two. Eval datasets become preference data for the training arc, and failing agent traces become the fixtures that harden the agent arc. The three arcs are not parallel tracks. They are one cycle.

Pick a reference stack

There is no single right stack, but there are three archetypes that cover most teams. The rows are the layers; pick the column that matches your constraints, then read down it. These are starting points to adapt, dated mid-2026, not rankings.

Layer	Lean / managed	Regulated / self-hosted	Cloud-native K8s
Model	Hosted frontier API (Claude / GPT-5 / Gemini), pinned IDs	Hosted for hard steps + self-hosted open weights for bulk	Same, plus in-cluster open-weight models
Gateway	OpenRouter or Cloudflare AI Gateway (managed)	LiteLLM self-hosted, virtual keys + budgets	Envoy AI Gateway or agentgateway (in-cluster)
Serving	None you operate (the API is the server)	vLLM behind OpenAI-compatible endpoint, FP8 + prefix caching	vLLM / SGLang on the cluster, KV-aware routing at scale
Agent	OpenAI Agents SDK or Claude Agent SDK	LangGraph (durable, checkpointed) or Pydantic AI	LangGraph + checkpoint in Postgres
Sandbox	E2B (hosted microVM)	Self-host on Firecracker, default-deny egress	Sandboxes on the same cluster, scoped tokens
Retrieval	Pinecone or Turbopuffer + Cohere Embed/Rerank	pgvector (+ pgvectorscale) + open embeddings (BGE-M3)	Qdrant + hybrid + open reranker
Eval / Obs	LangSmith or Braintrust (managed)	Langfuse self-hosted + Promptfoo in CI	Phoenix (OTel-native) + DeepEval in CI
Compute	Serverless GPU (Modal / Baseten / Fireworks)	Reserved neocloud (CoreWeave / Lambda), Slurm + K8s	K8s with Volcano/KAI gang scheduling, Kueue quotas

The durable advice underneath the table: minimize new moving parts, put your quality budget where it pays (retrieval and reranking over a fancier vector DB, calibrated judges over more benchmarks), keep every layer addressable through the three contracts so each is swappable, and never hard-code a single vendor. The archetype you start in is not the one you end in; the contracts are what keep the migration cheap.

Wiring: the seams in code

The glue that matters is not a tool tutorial. It is the code at the seams, the handful of lines that make the layers interchangeable. Two snippets show the load-bearing pattern: a gateway config that unifies a hosted and a self-hosted model with a fallback, and an agent that points at that one address with a virtual key.

First, the gateway. This LiteLLM proxy config exposes one alias, `smart`, that load-balances or fails over between a hosted and a self-hosted model. The application never learns which one served the request (sketch; see LiteLLM docs for the exact schema).

```
# litellm proxy config: one alias, hosted primary with a self-hosted
model_list:
  - model_name: smart                                # the name the app re
    litellm_params:
      model: anthropic/claude-opus-4-8
      api_key: os.environ/ANTHROPIC_API_KEY          # provider secret sta
  - model_name: smart                                # same alias, fallback
    litellm_params:
      model: openai/gpt-5.5
      api_key: os.environ/OPENAI_API_KEY
  - model_name: local                                # self-hosted, behind
    litellm_params:
      model: hosted_vllm/your-open-weight-model
      api_base: http://vllm-svc:8000/v1

litellm_settings:
  fallbacks: [{ "smart": ["local"] }]                # if hosted "smart" f
  num_retries: 2
# virtual keys, team budgets, and spend caps are minted via the /key
# API and the proxy UI, not this file.
```

Second, the agent. It holds a gateway virtual key, not a provider secret. It retrieves context, runs a tool, and never learns which model answered. The MCP tool is in-process here for brevity; in production it sits behind the MCP gateway over HTTP, and the agent code barely changes, which is the whole point of MCP being a wire protocol rather than a library API (Claude Agent SDK shape; see the SDK docs for the current API).

```

from claude_agent_sdk import tool, create_sdk_mcp_server, query, ClaudeAgentOptions

@tool("search_docs", "Retrieve grounding context for a query", {"q": str}):
    async def search_docs(args):
        # hybrid retrieve + rerank; embed/rerank calls go through the gateway
        hits = await retriever.hybrid_search(args["q"], top_k=8)
        return {"content": [{"type": "text", "text": "\n\n".join(h.text) for h in hits]}

server = create_sdk_mcp_server(name="rag", version="1.0.0", tools=[search_docs])

async for msg in query(
    prompt="Using the docs, summarize our refund policy and cite the relevant sections",
    options=ClaudeAgentOptions(
        mcp_servers={"rag": server},
        allowed_tools=["search_docs"],
        # model access resolves through the gateway base_url + a secret
        # key, never a raw provider credential. Swapping the model backend
        # or capping this agent's spend is a server-side change with no code change
    ),
):
    print(msg)

```

The eval harness closes the loop with the same discipline. A Promptfoo config points its provider `apiBaseUrl` at the gateway and uses a different model family as the judge, on its own budget-capped virtual key (Chapter 43). Every seam in the system, model calls, tool calls, retrieval, and judging, meets at the same two addresses (the model gateway and the MCP gateway) and emits the same span format. That is what makes the architecture one system rather than seven tools in a trench coat.

The capability, efficiency, trust lens

Capability: the three contracts let you assemble a system more capable than any single layer, a grounded, tool-using, durable agent backed by the best model for each step, because each layer is the best-in-class choice for its job and they still compose. **Efficiency:** the gateway makes per-task routing, fallback, and cost caps a configuration concern, so you escalate to an expensive model only when a step needs it and fall back under load without touching application code. **Trust:** virtual-key custody keeps secrets out of every client and sandbox, the OTel substrate keeps your traces portable across vendors, and the eval loop turns "it feels better" into a number that gates a release. The stack is worth wiring this way only if all three hold. By mid-2026 the contracts have matured enough that they do, and the lifecycle the book opened with, from compute to a governed behavior, finally runs end to end through one architecture.

Further reading

Gateways:

- LiteLLM proxy (cost tracking, load balancing, fallbacks): https://docs.litellm.ai/docs/proxy/cost_tracking, <https://docs.litellm.ai/docs/proxy/reliability>
- OpenRouter, "What is an LLM gateway": <https://openrouter.ai/blog/insights/llm-gateway/>
- Portkey AI gateway: <https://portkey.ai/buyers-guide/ai-gateway-solutions>
- agentgateway: <https://agentgateway.dev/> and <https://github.com/agentgateway/agentgateway>; Linux Foundation announcement: <https://www.linuxfoundation.org/press/linux-foundation-welcomes-agentgateway-project-to-accelerate-ai-agent-adoption-while-maintaining-security-observability-and-governance>
- Envoy AI Gateway: <https://aigateway.envoyproxy.io/docs/capabilities/>
- Cloudflare AI Gateway and Bifrost (Maxim AI): <https://github.com/maximhq/bifrost>
- Lux (author's own gateway): <https://lux.latere.ai>

The layers this chapter wires together:

- Inference serving (vLLM disaggregation, structured output): <https://docs.vllm.ai/en/latest/> and <https://github.com/sgl-project/sglang>
- Agent frameworks and MCP: OpenAI Agents SDK <https://openai.github.io/openai-agents-python/>; Claude Agent SDK <https://github.com/anthropics/claude-agent-sdk-python>; LangGraph <https://github.com/langchain-ai/langgraph>; MCP spec <https://modelcontextprotocol.io/> and 2026 roadmap <https://blog.modelcontextprotocol.io/posts/2026-mcp-roadmap/>
- Sandboxes (isolation models, safe-usage discipline): E2B <https://e2b.dev/>; Modal <https://modal.com/resources/best-code-execution-sandboxes-ai-agents>; Firecracker <https://firecracker-microvm.github.io/>; Firecrawl security model <https://www.firecrawl.dev/blog/ai-agent-sandbox>; Cella (author's own) <https://cella.latere.ai/>
- Retrieval (hybrid + rerank, pgvector): <https://github.com/pgvector/pgvector>; hybrid search and RRF overview <https://www.digitalapplied.com/blog/hybrid-search-bm25-vector-reranking-reference-2026>
- Eval and observability (OTel GenAI): Promptfoo <https://www.promptfoo.dev/docs/intro/>; Langfuse <https://langfuse.com/docs/observability/overview>; OpenTelemetry GenAI conventions <https://opentelemetry.io/docs/specs/semconv/gen-ai/gen-ai-spans/>
- Compute and orchestration (build vs rent, schedulers): Modal pricing <https://modal.com/pricing>; SkyPilot, "Slurm vs K8s" <https://blog.skypilot.co/slurm-vs-k8s/>; KubeRay <https://github.com/ray-project/kuberay>
- Frontier model access layers (Bedrock, Vertex, Azure): Anthropic models overview <https://platform.claude.com/docs/en/about-claude/models/overview>; OpenAI on Bedrock <https://aws.amazon.com/bedrock/openai/>; Gemini API pricing <https://ai.google.dev/gemini-api/docs/pricing>

45. The Compute Frontier: Bandwidth, Not FLOPs

Chapter 30 taught the reader to read a cluster as three bandwidth tiers and to diagnose a run with one honest number, model FLOPs utilization. That chapter described how the tiers work. This one asks where each tier breaks. By the end a reader can say why the scarce resource at the frontier is bytes rather than arithmetic, why a single accelerator is now a package of chiplets rather than one die, why the NVLink boundary that the earlier chapter pinned to a node has moved to a whole rack, why the silicon menagerie that grew up around the GPU is best read by what each entrant optimizes, and which of the escape routes from the bandwidth wall are already shipping rather than promised.

This is the first chapter of the book's closing part, and the part has one shape: it walks the same constraint arrow as the rest of the book, from the bottom of the stack to the top, but it asks at each layer not how the layer works but where it is breaking and where it is heading. We start at the bottom, with the silicon, because the bottleneck that reorganized everything above it in the last two years started here.

A spec sheet that lies in the same direction every year

A frontier accelerator's datasheet leads with a peak arithmetic rate, and the rate climbs fast across generations, faster still once the precision drops from FP16 to FP8 to FP4. Read only that line and the machine looks like it is getting better at the thing training and inference need most. The line is true and almost beside the point. The work a transformer does is not bound by how fast the multiply-accumulate units can run; it is bound by how fast operands reach them. Attention decode, mixture-of-experts routing, and any kernel that touches more memory than it reuses spend their time waiting on high-bandwidth memory, not on the matrix units. The roofline of Chapter 30 already named this: below a kernel's arithmetic intensity the achievable rate is set by HBM bandwidth, and the peak FLOPs ceiling is unreachable.

The frontier problem is that the two numbers on the roofline diverge, and they diverge in the same direction every generation. Across recent accelerator families, low-precision matrix throughput has climbed much faster than HBM bandwidth per chip. The honest way to state the trend is as a direction rather than a single headline multiple, because the multiple depends on which precision and which two generations you pick: from the V100 era to Blackwell, peak compute grew by roughly an order of magnitude more than memory bandwidth did, and the recent FP4 and FP8 jumps widen the gap further because they multiply arithmetic throughput without moving a single byte faster. The arithmetic intensity a kernel must reach to escape the memory-bound region therefore rises every generation. This is the memory wall, and it is not a temporary manufacturing lag. It is the structural fact that organizes the whole chapter: every tier of the machine, the chip, the package, the rack, and the power behind them, is hitting a ceiling that bandwidth, not arithmetic, sets.

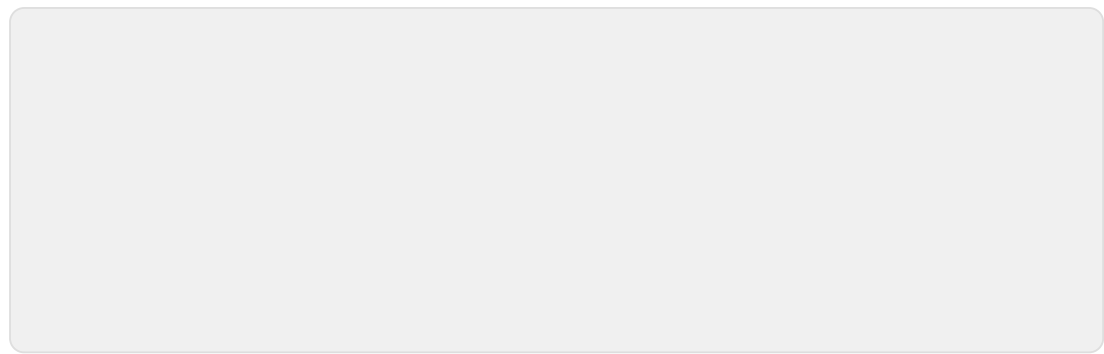


Figure 45.1. The frontier as a stack of ceilings, each binding at a different tier. The spec sheet's FLOPs sits at the bottom and is the least scarce; bytes, package area, fabric reach, and megawatts bind above it. Each later chapter of this part owns one of these ceilings.

The chip: a die too small for its own memory

The innermost ceiling is geometric. A photolithography scanner can expose at most one reticle field per shot, capped near 858 mm² (Shilov 2026). A frontier compute die already presses against that limit, so the only way to add transistors is to stop building one die. Modern accelerators are several compute chiplets and a row of HBM stacks tiled side by side on a silicon interposer, the CoWoS (chip-on-wafer-on-substrate) package. The interposer, not the logic die, is now the object whose area sets how much compute and how many memory stacks one accelerator can carry. TSMC's interposer roadmap is the supply-side clock on the whole frontier: roughly 5.5 reticles of area with five to a dozen HBM stacks today, a nine-reticle "Super Carrier" near 7,722 mm² in qualification for 2027, and a path past fourteen reticles with HBM5-class memory toward the end of the decade (Shilov 2026). Chapter 46 owns whether that interposer can be fabricated and bought; here it is enough to see that the reticle limit pushed packaging up into the architecture, making interposer area a first-class design parameter above the microarchitecture.

What gets tiled onto the interposer is mostly memory, and the memory is the point. NVIDIA's Rubin generation reaches roughly 288 GB of HBM4 at about 22 TB/s per accelerator. Google's Ironwood, the TPU v7, carries 192 GB of HBM3e at 7.37 TB/s and is the first TPU to support FP8 natively rather than through emulation (Google 2025). The HBM stacks themselves are a moving frontier: HBM4 doubles the interface to a 2,048-bit bus, the first sixteen-high stacks were demonstrated in early 2026, and the base die at the bottom of the stack is migrating onto a logic process so that memory now competes for the same leading-edge wafers as compute, a coupling Chapter 46 develops. The numbers move every quarter, and any specific figure in this chapter is a date stamp rather than a constant. The durable observation is the ratio: capacity and bandwidth grow, but they grow more slowly than the arithmetic that consumes them, so the fraction of real workloads that are memory-bound rises over time.

CONSTRAINT ARROW

HBM bandwidth growing slower than low-precision matrix throughput sets a falling arithmetic-intensity floor. That floor dictates choices two layers up: an ever-larger share of inference kernels land in the memory-bound region, which is why the serving layer leans on FP8 and FP4 weights and on aggressive key-value-cache management rather than on more FLOPs (cross-ref Chapter 17 and Chapter 19). Adding arithmetic to a bandwidth-bound kernel buys nothing.

The rack: the scale-up boundary that moved

Chapter 30 drew a hard line at the node: tensor parallelism, the most communication-hungry axis, had to stay inside the NVLink domain, and that domain was the eight or so accelerators in one chassis. That line has moved, and watching it move is the clearest way to see the frontier as evolution rather than a list of products. The NVLink fabric began as a chip-to-chip link inside a board, grew an NVSwitch crossbar so every accelerator in a chassis could reach every other at full bandwidth, and has now been deliberately stretched across an entire liquid-cooled rack. NVIDIA's GB200 and GB300 NVL72 wire seventy-two GPUs into a single NVLink-coherent domain spanning the rack, and the Rubin NVL72 keeps the form factor while roughly doubling the fabric bandwidth again. The rule from Part VII still holds, but its boundary is now the rack rather than the chassis: tensor parallelism stays inside the NVLink domain, and the domain got an order of magnitude larger.

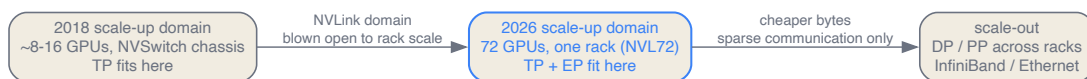


Figure 45.2. The scale-up domain that @sec-accelerators-networking pinned to a node has expanded to a rack. The most bandwidth-hungry parallelism axis can now span 72 accelerators inside one coherent fabric, where before it spanned roughly 8; beyond the domain edge, only sparse communication can afford the slower fabric.

Why expand the domain at all? Because the memory wall and the parallelism math interact. A larger coherent domain lets a model's tensor-parallel and expert-parallel groups span more silicon at full fabric bandwidth, so the per-device memory pressure of a trillion-parameter model is spread without spilling the hungriest collective onto the slow inter-rack network. The rack became the unit of frontier compute because that is the largest blast radius the fabric can still hide a collective inside.

The expansion is not free, and copper is running out of reach. Past a rack, the per-interface power of pluggable optical transceivers, on the order of tens of watts each, becomes a budget the rack cannot spend. Co-packaged optics move the fiber onto the switch package and cut the per-interface power roughly from 30 W to 9 W, which is the case NVIDIA makes for Quantum-X and Spectrum-X co-packaged switches shipping through 2026 (Seyed and NVIDIA 2025). Beyond a fixed topology, optical circuit switching lets the fabric itself be reconfigured per training phase; recent work on parallelism-driven optical rails reports large network-power and cost reductions at a few percent training overhead (Ding et al. 2026). The through-line is that the fabric's energy budget, not only its bandwidth, now bounds how large a coherent domain can be built and cooled, which is the thread Chapter 47 picks up.

One more move complicates the old picture. NVIDIA has begun opening the NVLink domain to third-party silicon, so a rack can mix a custom accelerator or CPU into an otherwise NVIDIA fabric. The scale-up domain is therefore no longer strictly the property of one vendor's chips, which matters for the debate over whether the future fabric is proprietary or open.

The silicon menagerie: who is betting on bandwidth

For most of the deep-learning era the merchant GPU was the only serious frontier accelerator, and its software ecosystem was the moat. The bandwidth wall has opened room for a wider menagerie, and reading the entrants by what they optimize is more instructive than ranking them by peak FLOPs.

The custom-ASIC bet is vertical integration: co-design the chip with the network and skip the merchant margin. Google's Ironwood scales to a 9,216-chip superpod wired as a torus with optical circuit switches rather than a switched fat-tree, and targets the inference-dominated workload mix directly (Google 2025). Amazon's Trainium makes the same wager for its own

fleet. The advantage is co-design: the chip, the interconnect topology, and the compiler are tuned together for one company's models.

The merchant counter is AMD's memory-density play. Its rack-scale Helios system fills seventy-two accelerators, each carrying about 432 GB of HBM4, into a domain that totals on the order of 31 TB of HBM at roughly 1.4 PB/s of aggregate bandwidth, which exceeds the comparable NVIDIA rack on memory capacity and bandwidth while trailing it on peak FP4 throughput. That is the chapter's thesis stated as a product decision: if the wall is bytes, build the rack with more bytes and concede some arithmetic. Under export controls, Huawei's CloudMatrix 384 makes the same move with watts instead of process node: it knits 384 domestic NPUs across sixteen racks into one optically-meshed all-to-all scale-up domain that reaches frontier aggregate compute and memory by brute force, at several times the power of the Western rack it answers. It is a reminder that the scale-up-domain idea is general, that the memory wall can be paid down with energy when the best silicon is unavailable, and that the compute frontier has a second front defined by who is allowed to buy what, which [Chapter 46](#) takes up.

At the contrarian edge sits wafer-scale. Cerebras keeps an entire model's weights in on-die SRAM across a single wafer of 900,000 cores, with no HBM to stream from, and reports multiples of GPU per-user token rate on latency-bound decode ([Systems 2024](#)). It is the purest bet against the memory wall: if you never leave the die, there is no HBM bandwidth to be bound by. Its ceiling is capacity, since on-wafer SRAM cannot hold a trillion-parameter model, so it wins latency-bound serving rather than the largest training runs. In-memory and analog accelerators push the same idea further and sit further from production.

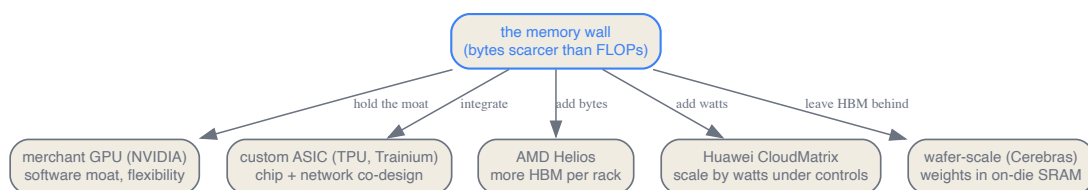


Figure 45.3. The silicon menagerie read by what each entrant optimizes against the bandwidth wall, not by peak FLOPs. The merchant GPU and custom ASIC fight over the same workload from opposite integration strategies; AMD and Huawei answer the wall with bytes and watts; wafer-scale answers it by never leaving the die.

Escape routes, in order of maturity

The chapter's payoff is to sort the ways out of the bandwidth wall by how real they are today, because the field's discourse mixes shipping hardware with research slides, and a reader deserves to know which is which.

Shipping now is rack-scale copper: the NVL72-class domain is in production and is the reason tensor parallelism can span seventy-two devices. Shipping through 2026 is co-packaged optics, the next step once copper runs out of reach past the rack ([Seyed and NVIDIA 2025](#)).

A numeric escape route is already in training pipelines. Four-bit pretraining formats such as NVFP4 relax the arithmetic-intensity floor by halving operand bytes while preserving accuracy: a multi-billion-parameter model trained over many trillions of tokens in NVFP4 has matched an FP8 baseline, and the four-bit matrix multiply runs at a multiple of the FP8 rate on recent hardware. This is the cleanest way to make a memory-bound kernel less memory-bound without new silicon, because it attacks the bytes directly; the format mechanics belong to [Chapter 19](#), but the reason the frontier cares is bandwidth, not storage.

A structural route is to split the workload across chips specialized for each phase. Prefill, the processing of a long prompt, is compute-bound and tolerates cheaper high-capacity memory; decode, the token-by-token generation, is bandwidth-bound and wants scarce HBM. A prefill-

optimized accelerator built around abundant GDDR memory can therefore pair with HBM parts that handle decode, putting expensive bandwidth only where the workload is actually bound by it. NVIDIA proposed exactly such a prefill-specialized part; that its place on the roadmap has wavered is itself a frontier-uncertainty signal worth recording rather than smoothing over, because it shows the field still disagreeing about whether the split pays once the cost of moving the key-value cache between the two chips is counted, a systems question Chapter 48 returns to.

Further out sit reconfigurable optical fabrics (Ding et al. 2026) and wafer-scale integration (Systems 2024), real research with real demonstrations but not yet the default.

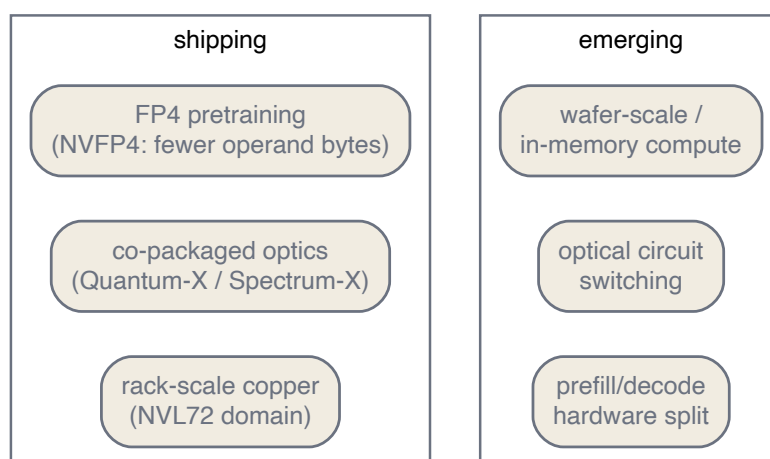


Figure 45.4. Escape routes from the bandwidth wall, ordered by maturity. The left column is in production; the right column is demonstrated but not default. The middle two attack the wall through numerics and through phase specialization rather than through new fabric.

WHAT'S CONTESTED

Four debates are genuinely live, and the camps disagree on which non-FLOPs wall binds first rather than on whether FLOPs stopped being the limit.

- **One coherent fabric, an open standard, or cheap Ethernet?** NVIDIA bets ever-larger proprietary NVLink domains are worth their cost. The UALink consortium wants an open switched standard to commoditize the scale-up tier. A pragmatist camp reuses Ethernet switch silicon for scale-up. Through 2026 the pragmatists shipped first, because rack vendors fell back to Ethernet-substrate fabrics when native open-standard switch silicon was not yet available. NVIDIA has meanwhile partially opened its own fabric to third-party silicon, so the proprietary-versus-open line is itself blurring.
- **Is co-packaged optics mandatory or a reliability risk?** Proponents say copper is out of runway past a rack and optics is the only way to scale the domain. Skeptics point to unproven laser reliability and repairability at fleet scale and defend copper inside the rack. The dividing line is distance: optics for inter-rack, copper still defended intra-rack.
- **Do custom ASICs displace the merchant GPU?** The vertical-integration camp says co-designed silicon wins perf-per-watt on the inference-heavy mix. The incumbent camp says the software ecosystem and flexibility keep the GPU the default, and ASICs win only captive first-party workloads.
- **Is wafer-scale the answer or a niche?** It clearly wins latency-bound single-user decode and clearly cannot hold the largest models in SRAM. The unresolved question is whether model sizes plateau enough for on-die capacity to catch up.

Reading the frontier

The capability, efficiency, trust lens closes the chapter as it closes the others. Capability at this layer is no longer set by how many FLOPs a chip can retire but by how many bytes it can move and how much coherent silicon one fabric can span, which is why the rack replaced the chip as the unit of frontier compute. Efficiency moved into the numerics and the network: a four-bit operand and a nine-watt optical interface buy more frontier than a faster matrix unit, because both attack bytes rather than arithmetic. Trust, at the silicon layer, is supply and concentration: a frontier run now depends on whether a handful of buyers can secure interposer area and HBM stacks at all, and on which side of an export-control line the buyer sits, which is the allocation gate [Chapter 46](#) takes up next. The spec-sheet FLOPs was never the scarce resource. The scarce resources are bytes, package area, fabric reach, and the megawatts behind them, and the rest of this part follows that scarcity up the stack.

Further reading

- Google, "Ironwood: The First Google TPU for the Age of Inference," 2025 (TPU v7 specs: 192 GB HBM3e at 7.37 TB/s, 9,216-chip superpod, native FP8). blog.google
- NVIDIA, "Scaling AI Factories with Co-Packaged Optics for Better Power Efficiency," 2025 (Quantum-X and Spectrum-X CPO, the 30 W to 9 W per-interface case). developer.nvidia.com
- Ding, Lyu, Kataria, Singh, "Photonic Rails in ML Datacenters with Opus," 2026 (optical circuit switching reconfigured per parallelism phase). [arXiv:2602.12521](https://arxiv.org/abs/2602.12521)
- Shilov, "TSMC 'Super Carrier' CoWoS," Tom's Hardware, 2026 (the 858 mm² reticle limit and the CoWoS reticle-scaling roadmap to 2029). tomshardware.com
- Cerebras Systems, "Third-Generation Wafer-Scale Engine (WSE-3)," 2024 (the on-die-SRAM bet against the memory wall, and its capacity ceiling). cerebras.ai
- Epoch AI, "How Much Power Will Frontier AI Training Demand in 2030?," 2025 (the datacenter-level power ceiling this chapter hands to [Chapter 47](#)). epoch.ai

46. Making the Silicon: Packaging, HBM, and the Geopolitics of Compute

Chapter 45 treated the memory wall and the reticle limit as design constraints: facts the architect of an accelerator must build around. This chapter treats the same silicon as a supply chain, and asks a blunter question. Granting that you know how to design a frontier accelerator, can you get one built, and how many? By the end a reader can explain why the binding constraint on AI compute in 2026 moved down the stack from the transistor to the package and the memory stack, why a few square centimeters of silicon interposer rate-limit the whole field, why a memory shortage now reaches all the way out to laptops and phones, and why export controls and sovereign- compute programs are the levers by which states ration a scarce industrial good.

The bottleneck moved below the transistor

For two decades the frontier of semiconductors was lithography: the binding question was whether the next node could print smaller transistors. That is no longer where AI compute is gated. Leading-edge logic is ramping on schedule, TSMC's N2 is in production, and the next lithography leap, High-NA extreme ultraviolet, has been deliberately deferred to roughly the end of the decade because multi-patterned standard EUV remains cheaper per good die. The transistor is not the bottleneck. The bottleneck moved to two places the Chapter 45 chapter named as design facts and this chapter treats as scarce goods: the silicon interposer that tiles chiplets and memory into one package, and the stacked DRAM that supplies the bandwidth.

The reason is the one Chapter 45 established and this chapter will not re-derive: a frontier accelerator exceeds the single-reticle limit, so it must be assembled on an interposer rather than printed as one die. That makes advanced packaging, specifically TSMC's CoWoS-L line, the per-unit gate on how many frontier accelerators can exist. Through 2026 that line ran fully booked, with lead times reported at a year and more, even as TSMC scaled it from roughly 35,000 wafers per month toward 120,000 and beyond and projected an eleven-fold rise in AI wafer demand across 2022 to 2026 (TrendForce 2026). A single buyer, NVIDIA, holds the majority of total CoWoS capacity and a larger share of CoWoS-L specifically, which is why a competitor's launch schedule can be set as much by interposer allocation as by its own design.

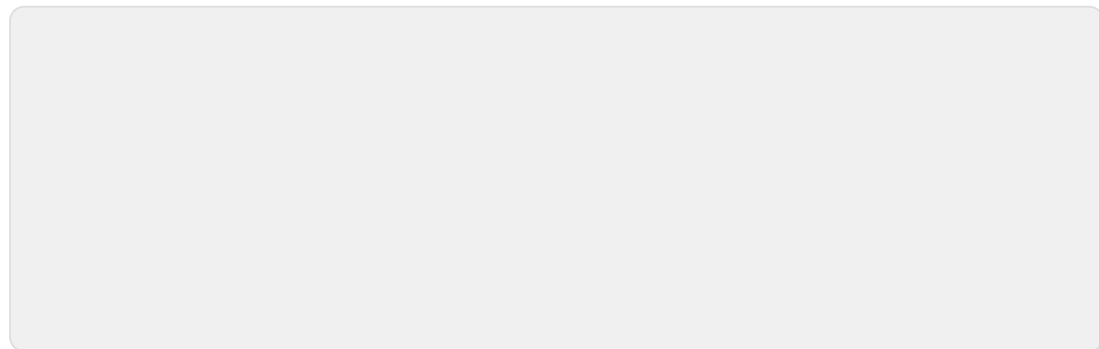


Figure 46.1. Where the gate sits in 2026. Leading-edge logic is no longer the binding step; advanced packaging and HBM supply are, and a frontier accelerator exists only when both clear at once. The lithography frontier (High-NA EUV) is deliberately held back.

Two gates that take turns

The defining feature of the 2026 supply chain is that the binding constraint oscillates between two chokepoints, quarter by quarter, so naming a single bottleneck is already a simplification. The first is the interposer. The second is the memory.

HBM4, the memory generation that frontier accelerators ride, is not merely a faster DRAM. It doubles the interface to a 2,048-bit bus, runs at roughly 11 gigabits per second per pin at the frontier rather than the 8 the standard baseline specifies, and, most consequentially, places a logic die fabricated on a foundry process at the base of each stack ([hynix 2025](#)). That last change couples memory to logic: HBM volume now competes for leading-edge foundry capacity rather than being purely a DRAM-fab problem, so the two gates are not independent. By mid-2026 all three memory makers, SK hynix, Samsung, and Micron, had been qualified for the frontier accelerator's HBM4 and were in production, and all three had declared their output sold out before the year began. The power-efficiency gains the vendors quote differ enough to matter: SK hynix claims better than forty percent over its prior generation, Micron a more modest figure, so the generation is not a single uniform step.

CONSTRAINT ARROW

The HBM4 base die sits on a TSMC logic node, so memory volume now draws on the same leading-edge wafers as compute. A constraint that used to live in DRAM fabs alone now reaches into foundry capacity, which means a decision to make more memory is partly a decision to make fewer logic chips. This is the clearest case in the book of two layers that looked independent becoming one shared budget.

The interposer roadmap is the supply-side clock. TSMC's public path runs from roughly 5.5 reticles of CoWoS-L area carrying a handful of HBM stacks today, through a fourteen-reticle generation with twenty stacks around 2028, to a beyond-fourteen-reticle package with twenty-four HBM stacks near 2029, at a reported packaging yield already around ninety-eight percent ([TrendForce 2026](#)). That yield figure carries a lesson: at the CoWoS-L tier the problem is no longer whether a package can be made well, but how many can be made at all. Capacity, not yield, is the gate. One layer deeper, in the true three-dimensional stacking of SoIC, where dies are bonded copper-to-copper rather than laid side by side, yield and capacity are both still hard, which is why SoIC is the chokepoint most likely to bind after CoWoS-L eases.

The shortage reaches the laptop

The story the design-side chapter could not tell is what the memory gate does to everything that is not a frontier accelerator. Because an HBM stack consumes several times the wafer area per gigabyte of a commodity DRAM chip, and because the memory makers have shifted their best capacity to HBM, the AI build-out is now cannibalizing the supply of ordinary memory. Through late 2025 and into 2026 this surfaced as a broad memory supercycle: contract prices for standard server DRAM more than doubled, makers pruned or exited consumer lines, and the cost of memory in a phone or a laptop rose because the fab that would have made it is making HBM for accelerators instead. The constraint arrow that began at the top of the accelerator now runs all the way down into consumer electronics: AI compute competes with all of computing for a shared pool of fab output, and through 2026 AI was winning.

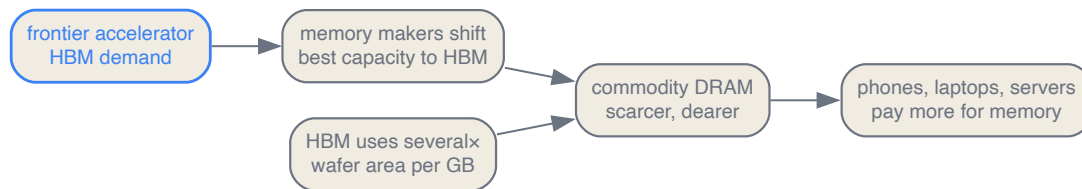


Figure 46.2. The memory supercycle as a constraint arrow reaching past the datacenter. HBM's wafer-area premium and the memory makers' capacity shift mean a frontier accelerator's appetite raises the price of memory in an unrelated phone, coupling AI compute to all of computing.

Geography is policy

Almost all of this capability sits in one place. Leading-edge logic, the CoWoS-L interposer line, and the packaging yield learning are concentrated in Taiwan, with memory concentrated in South Korea. That concentration turns a supply chain into a geopolitical instrument, and the instrument was used heavily across the period this book covers.

The United States has rationed access to frontier compute through export controls that tightened in steps: the original 2022 cutoff on the fastest parts, then performance-throttled variants engineered to sit just under each new threshold, and in January 2026 a shift to case-by-case review for H200-class and competitor parts bound for China, defined by concrete limits on total processing performance and on memory bandwidth, and paired with a revenue tariff and a volume cap (Security 2026). The thresholds are not incidental: a bandwidth ceiling as an export-control knob is the policy world conceding the same point as Chapter 45, that the scarce quantity is bytes per second, not raw arithmetic.

What the controls have actually done is the chapter's live debate. They have not let a Chinese accelerator match the Western frontier on a per-chip basis, and the binding shortage on the Chinese side is HBM rather than logic, since domestic foundries can approximate a usable logic node but cannot yet supply frontier memory. They have also unmistakably accelerated indigenization, which is why the Huawei CloudMatrix scale-up domain of Chapter 45 exists at all: when the best chips are forbidden, the answer is more chips and more watts. Whether the controls slow the Chinese frontier or harden it is the question the policy community is split on, and it is genuinely open.

WHAT'S CONTESTED

- **Is the gate the interposer or the memory?** Most foundry analysts hold that CoWoS-L interposer capacity is the per-unit gate and point to one buyer's majority booking as proof. A memory-shifting camp notes that interposer capacity is expanding several-fold while HBM stays sold out, so the binding constraint is migrating toward memory. The likeliest reading is that they co-limit and the gate oscillates by quarter.
- **Durable supercycle or engineered shortage?** TSMC raised its 2026 capital spending sharply and its leadership dismissed talk of a bubble (Swinhoe 2026). A caution camp argues suppliers are deliberately under-building to avoid a glut like the memory crashes of the past, which would make the shortage partly a choice and the prices reversible.
- **Do export controls contain China or accelerate it?** One camp holds the chip deficit is structural and controls should remain (Relations 2025). The other holds that easing sells capable chips while the controls have already triggered the domestic build-out they were meant to prevent (Democracies 2025). Both cite the same facts and read them oppositely.
- **Is one-country concentration an acceptable risk?** Diversification through subsidized fabs abroad is years and a cost premium away from replicating Taiwan's

packaging yield, so concentration persists regardless of policy intent, and the field runs on a single point of failure it cannot quickly remove.

Reading the supply chain

The capability, efficiency, trust lens reads strangely at this layer, and the strangeness is the point. Capability here is not a property of a model or even a chip; it is a question of allocation, of whether interposer area and memory stacks can be secured at all, so the frontier is partly a procurement frontier. Efficiency is the wafer-area cost of a gigabyte and the yield of a package, the quiet variables that decide how many accelerators a fixed fab output becomes. Trust, at this layer, is supply security and concentration: a field whose entire frontier depends on a few lines in two countries, rationed by export policy, is trusting a supply chain as much as it trusts any algorithm. The accelerator of Chapter 45 is the artifact; this chapter is the gate it must pass through. The next chapter follows the finished accelerator to the wall socket, where a third scarce good, electricity, binds on a timescale even longer than a sold-out packaging line.

Further reading

- U.S. Bureau of Industry and Security, "Revision to License Review Policy for Advanced Computing Commodities," 2026 (the Jan 2026 rule, with the total-processing-performance and memory-bandwidth thresholds). [Federal Register](#)
- SK hynix, "SK hynix Completes World's First HBM4 Development and Readies Mass Production," 2025 (2,048-bit interface, the logic base die, the power-efficiency claim). [PR Newswire](#)
- TrendForce, "TSMC Sees AI Wafer Demand Rising 11x From 2022--2026, Targets CoWoS With 24 HBM Stacks in 2029," 2026 (the CoWoS reticle and HBM-stack roadmap, the ~98% yield figure). [trendforce.com](#)
- Swinhoe, "TSMC Announces 2026 Capex Spend of \$56bn as CEO Dismisses Bubble Concerns," [DataCenterDynamics](#), 2026 (the capex cycle and the supercycle-versus-bubble debate). [datacenterdynamics.com](#)
- Council on Foreign Relations, "China's AI Chip Deficit," 2025, and Foundation for Defense of Democracies, "Rolling Back Export Controls," 2025 (the two named positions on whether controls contain or accelerate). [cfr.org](#) / [fdd.org](#)

47. Powering It: Time-to-Power as the Binding Constraint

Chapter 30 ended at the rack. This chapter starts where the rack plugs in. By the end a reader can explain why the binding constraint on AI capability in 2026 moved below the chip to the physical layer of electricity, why the scarce quantity there is not generation capacity but time-to-power, why a frontier campus now self-generates rather than waits for the grid, why a rack that draws as much power as dozens of homes forced a new default for cooling, and why a data center recently became something the grid operator worries about rather than merely serves.

The previous two chapters followed a constraint arrow down: from the bytes a chip can move, to the interposer and memory stacks that gate how many chips exist. This chapter reaches the bottom of the physical stack. Below the chip and below the package is the electricity, and electricity binds on a timescale longer than either.

The chips arrive in months, the megawatts in years

A frontier training campus now needs on the order of a gigawatt of dedicated power, comparable to a mid-sized city or a nuclear unit. The headline fear is that the world cannot generate enough electricity, but that is not the 2026 constraint. The constraint is time. A chip order is filled in months; the power behind it arrives in years, because three lead-time queues sit in series, and a campus is energized only when the slowest clears.

The first queue is grid interconnection. Connecting a large new load or generator to the transmission system requires a study-and-approval process that, in the United States, backs up for years: the active interconnection queue holds on the order of two thousand gigawatts of projects, far more than the country's entire installed capacity, with typical waits measured in multiple years (Rand et al. 2025). The pressure is visible in a single market: in Texas, large-load interconnection requests jumped from roughly 63 GW at the end of 2024 to roughly 226 GW a year later, the large majority of it data centers. The second queue is long-lead equipment. A large power transformer, the device that steps grid voltage down for a campus, runs lead times around 128 weeks against a structural supply deficit (Mackenzie 2025). The third queue is prime movers: the gas turbines that on-site generation depends on are themselves backed up, with a single major manufacturer reporting a backlog near 80 GW stretching past 2029 and effectively sold out through 2030 (Cardwell 2025).

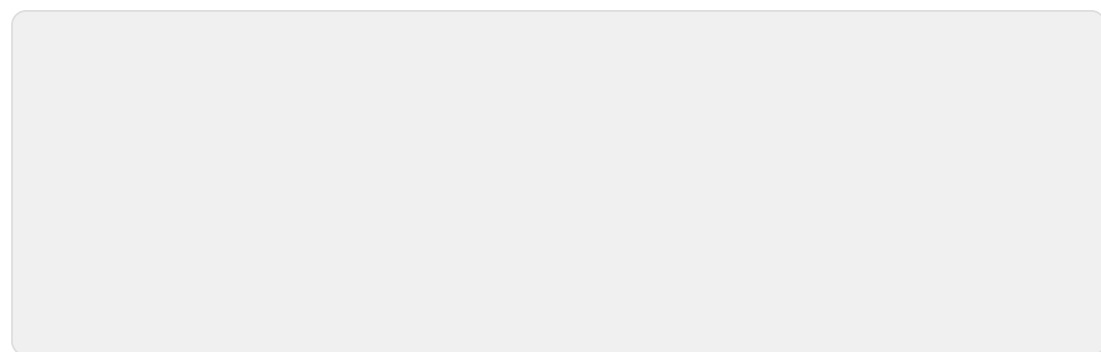


Figure 47.1. Time-to-power as three serialized queues. A frontier campus is energized only when the slowest clears, so the binding constraint is calendar time, not generation capacity. The chip, by contrast, ships in months.

The serialized lead times of interconnection, transformers, and turbines cap how fast cluster scale can grow, independent of how much capital or how many chips a buyer has. This is the arrow that most surprises a reader coming up from the silicon chapters: more money and more accelerators do not buy a faster build once the binding queue is a transformer factory or a turbine backlog. The megawatt, not the GPU, now sets the schedule (cross-ref [Chapter 37](#)).

Going behind the meter

Because the interconnection queue is the longest pole, the largest builders stopped waiting in it. The defining operational move of 2026 is self-generation behind the meter: standing up power on the campus itself rather than drawing it through a grid connection that is years away.

On-site gas is the fast option. Operators have built campuses around dozens of gas turbines fired on site, with the largest planned builds reaching one to several gigawatts. It is the quickest path to bulk power, and it collides directly with the turbine backlog and with air-quality law, which makes it the chapter's sharpest contested point. Nuclear is the firm option. The most visible deal of the cycle restarts a shuttered reactor under a long-term power-purchase agreement to serve a hyperscaler, targeting a return to service in 2027 and backed by a federal loan ([NucNet 2025](#)); other restarts and uprates add firm gigawatts on a similar horizon. The regulatory path is not clean: a marquee co-location deal that would have placed a data center directly behind a nuclear plant's meter was rejected by federal regulators and had to be restructured as a conventional grid-connected agreement, a reminder that behind-the-meter is a legal posture as much as an engineering one.

Small modular reactors are the much-discussed third option, and honesty requires placing them correctly on the timeline. The contracted pipeline grew large across 2025 and 2026, with multiple multi-gigawatt agreements signed, but no commercial unit dedicated to AI is operating, and the first are not expected before roughly 2030. They are a real hedge for the next decade, not a source that relieves the 2026 to 2028 crunch. Confusing the contract with the kilowatt is the most common error in this part of the discourse.

A rack that boils

The power problem reappears as a heat problem inside the rack. A current flagship rack draws roughly 120 kW, with observed figures past 130 kW, which is several times what forced-air cooling can remove from a rack even with aggressive engineering. Air, the default for the entire prior history of the data center, simply cannot carry the heat away. Direct-to-chip liquid cooling, where coolant runs through cold plates pressed onto the accelerators, became the non-optional default for this class of hardware, and immersion, submerging the boards in dielectric fluid, is the next tier for the densities coming after. The facility-efficiency frontier, meanwhile, has flattened: the industry-average power-usage effectiveness has sat near 1.54 for years while the best hyperscalers run near 1.1, which means the easy facility-level gains are spent and the remaining efficiency must come from the silicon, through tokens per watt, and from the grid.

CONSTRAINT ARROW

Rack power density past the air-cooling ceiling dictates the thermal architecture of the whole facility: a 120 kW rack forces direct-to-chip liquid as the default, which reshapes the data hall, the plumbing, and the supply chain for cooling gear. This is the per-rack power of [Chapter 30](#) reaching down into the building (cross-ref [Chapter 39](#)).

The data center the grid fears

The newest frontier is the one the earlier framing missed entirely: at gigawatt scale, an AI data center is not a docile load but a hazard to the stability of the grid it sits on. A large training cluster can shed or pick up a thousand megawatts in seconds when a job stops or a fault trips protection, and a swing that large, that fast, is something the bulk power system was not designed to absorb. After a series of such events, including a load drop above a gigawatt during a fault on a major interconnection, the North American reliability authority issued a rare high-severity alert in May 2026, and a wave of research now studies the wide-area oscillations and fault ride-through behavior that large AI loads induce. The regulatory layer is moving in parallel: at the end of 2025 federal regulators directed the largest grid operator to write rules for data-center-at-power-plant co-location, with compliance beginning in 2026, which is the legal scaffolding underneath the behind-the-meter moves above.

The same scale creates a cost fight. Someone must pay for the transmission upgrades and the capacity a gigawatt campus demands, and as data-center load drove capacity-auction prices to records, the question of whether ordinary ratepayers or the data centers themselves bear the cost became a live battle at state regulators. A more cooperative path is also maturing: treating the cluster as a flexible, curtailable load that can ramp down briefly to relieve the grid lets far more compute interconnect without new firm generation, an idea that moved from study to pilot during this period. Whether AI load is a threat to be managed or a flexible resource to be harnessed is not yet settled.

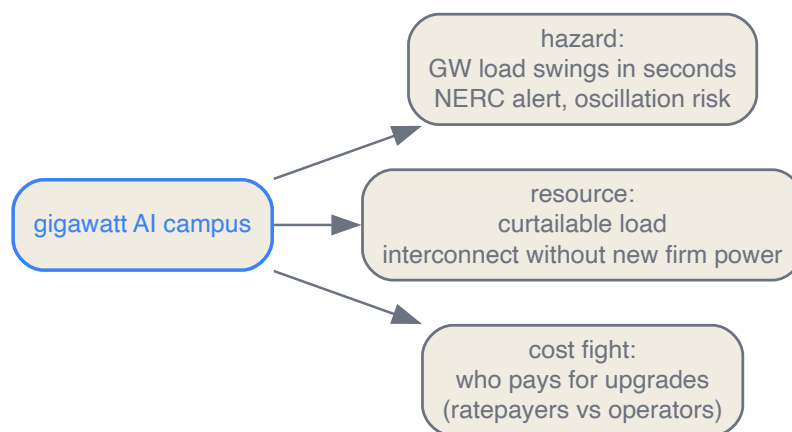


Figure 47.2. The gigawatt campus as a two-sided object: a reliability hazard the grid must absorb (fast multi-gigawatt load swings) and, potentially, a flexible resource that curtails to interconnect faster. Which face dominates is the chapter's open question.

How much, and measured how

The aggregate numbers frame the stakes. Global data-center electricity ran near 415 TWh in 2024, about 1.5 percent of world demand, and is projected to roughly double toward 945 TWh by 2030, with AI-optimized load growing fastest (Agency 2025). In the United States the most-cited study put data centers at 176 TWh in 2023, rising to a range of 325 to 580 TWh, between roughly 7 and 12 percent of national electricity, by 2028 (Shehabi et al. 2024). The width of that range is the honest part: the future depends on efficiency gains and build-rates nobody can yet pin down.

Per query, the picture is contested precisely because the boundary is contested. A vendor's full-stack accounting put the median text prompt near 0.24 Wh of energy, including idle capacity and overhead, and claimed a large drop over the preceding year (Elsworth et al. 2025). Critics note that favorable boundary choices, and the exclusion of training amortization, make cross-model comparisons unreliable, and that no audited standard exists. The energy-per-token number is real and useful and not yet trustworthy as a comparison.

WHAT'S CONTESTED

- **Does power or silicon bind first?** A power-limited camp argues chips are available and megawatts are not, so tokens per watt sets revenue. A supply-chain camp argues the binding link migrates year to year between HBM, packaging, transformers, and turbines, and in 2026 it is electrical equipment rather than chips. They agree it is no longer FLOPs.
- **Are demand forecasts overblown?** Skeptics hold that load forecasts are chronically overstated and that efficiency absorbs much of the growth (Koomey et al. 2025). Utilities and several research groups hold that the build-out is real and large. The truth turns on efficiency and build-rate assumptions that are themselves the disagreement.
- **Is nuclear and SMR power real or a hedge?** Restarts and uprates are contracted firm gigawatts arriving 2027 to 2028. New small reactors are a ~2030 proposition at the earliest, so they cannot relieve the near-term crunch however large the signed pipeline grows.
- **Is on-site gas a bridge or a liability?** One camp calls behind-the-meter gas the only way to gigawatts before 2030. The other points to air-quality litigation and broken climate pledges. The fight escalated rather than resolved, drawing in federal intervention on national-security grounds.
- **Narrow or full-stack energy accounting?** Without an audited standard, every published per-query figure is a boundary choice, and the boundary is the argument.

Reading the power layer

The capability, efficiency, trust lens lands hardest here. Capability is now gated by a calendar: a model that could be trained if the power existed is not trained until the slowest queue clears, so the frontier is partly an infrastructure-delivery frontier. Efficiency stopped being a facility metric and became a silicon and grid metric, because the data hall's easy gains are spent and the remaining ones live in tokens per watt and in flexible load. Trust, at this layer, is the literal trust the grid places in a load that can swing a gigawatt in seconds, and the trust a community places in a campus burning gas next door. The constraint arrow that began with bytes inside a chip now reaches the electrical grid and the permitting office, and it points back up the whole stack: the megawatt dictates the model. The next chapter goes back inside the cluster the power feeds, and asks why a machine of a hundred thousand accelerators is failing somewhere almost all the time.

Further reading

- International Energy Agency, "Energy and AI," 2025 (the global baseline: ~415 TWh in 2024 toward ~945 TWh by 2030). [iea.org](https://www.iea.org)
- Shehabi et al. (LBNL), "2024 United States Data Center Energy Usage Report," 2024 (the canonical US load study, 176 TWh in 2023 to 325--580 TWh by 2028). escholarship.org
- Rand, Seel, Wiser (LBNL), "Queued Up: 2025 Edition," 2025 (the interconnection-queue data behind the time-to-power thesis). emp.lbl.gov
- Elsworth, Patterson, Dean et al. (Google), "Measuring the Environmental Impact of Delivering AI at Google Scale," 2025 (the full-stack 0.24 Wh-per-prompt figure and the measurement-boundary debate). [arXiv:2508.15734](https://arxiv.org/abs/2508.15734)
- Cardwell, "GE Vernova's 80-GW Gas Turbine Backlog," Utility Dive, 2025, and Wood Mackenzie, "Transformer Supply Deficits," 2025 (the prime-mover and long-lead-equipment queues). [utilitydive.com](https://www.utilitydive.com) / [woodmac.com](https://www.woodmac.com)
- Koomey, Schmidt, Das (Bipartisan Policy Center), "Electricity Demand Growth and Data Centers: A Guide for the Perplexed," 2025 (the demand-skeptic position).

bipartisanpolicy.org

48. The Machine That Breaks at Scale

The previous three chapters built and powered a machine of a hundred thousand accelerators. This chapter asks what happens when you run it. By the end a reader can explain why a frontier training job is failing somewhere almost all the time, why the engineering discipline pivoted from preventing failures to amortizing them, why the most dangerous failures are the ones that raise no alarm, why the network's latency hierarchy and the decode bandwidth wall set hard ceilings on training and serving, and why an agent that runs for hours hits the same compounding-failure logic one layer up. The argument is that fault tolerance, silent corruption, the networking ceiling, inference economics, and agent reliability are not five problems but one phenomenon seen at five layers: each is a place where a lower physical limit dictates an upper architectural choice, and where the field has not yet agreed where the tolerance should live.

A statistical certainty of failure

Synchronous training is a single distributed transaction. Every accelerator must finish its share of a step before the next step begins, so if any one of N devices fails, all N stall. The per-device failure rate is roughly fixed by the silicon and its memory, which makes the math of large N unforgiving: the mean time between failures for the whole job falls in proportion to the device count.

The public ledger that made this concrete is Meta's Llama 3 405B run. On 16,384 H100s over 54 days it logged 419 unexpected interruptions, about one every three hours, with roughly 59 percent traced to GPU or high-bandwidth-memory faults (Grattafiori and others 2024). Hold the per-device rate fixed and scale the job past a hundred thousand accelerators, and the expected interval between failures drops under half an hour. A job that takes tens of minutes to recover from each failure would then spend more time recovering than computing. The frontier is past the point where failure is an exception to plan around; it is the steady state.

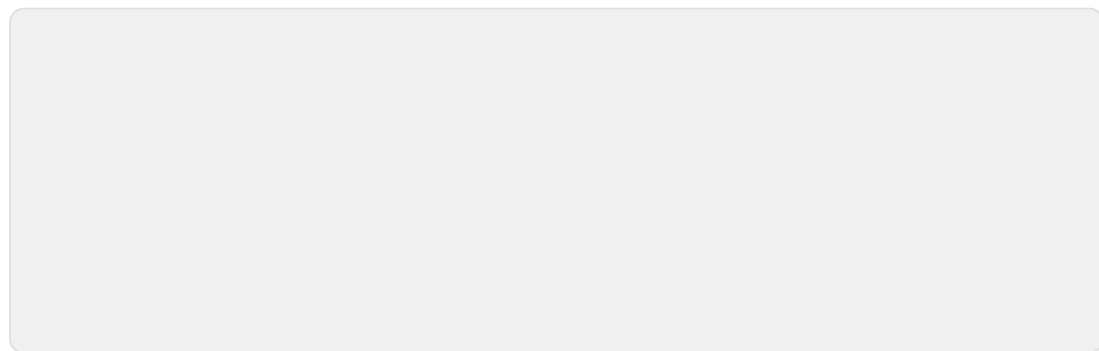


Figure 48.1. Why the steady state is failure. A fixed per-device fault rate times a synchronous job of N devices drives the whole-job mean-time-between-failures below the recovery time, so the discipline shifts from preventing failures to amortizing them. The Llama 3 405B run is the public anchor.

The response is not to prevent failures, which is impossible at this rate, but to make recovery cheap enough that frequent failures barely dent throughput. The metric that captures this is the effective training time ratio, the fraction of wall-clock time the job spends making real progress rather than recovering. A production system reporting 97 percent effective training time across a three-month run on nearly ten thousand GPUs, atop a platform of hundreds of thousands, is the state of the art (Seed 2025), and it gets there by replacing periodic disk

checkpoints with in-memory and peer-replicated checkpoints so a restart costs seconds rather than the tens of minutes a multi-terabyte reload from storage would take.

CONSTRAINT ARROW

The fixed per-device failure rate dictates the training-software architecture. At a hundred thousand accelerators it forces sub-minute recovery, in-memory checkpointing, and redundant nodes, because the alternative, periodic disk checkpoints, would let recovery overwhelm computation. The parallelism of [Chapter 8](#) is the thing this layer must keep alive through constant interruption.

The failures that raise no alarm

The failures counted in that ledger announce themselves: a GPU falls off the bus, a process crashes, a collective times out. The harder problem is the failure that does not. Silent data corruption is a fault that produces a wrong number with no crash and no error, a bit that flips in an arithmetic unit or a memory cell and quietly poisons a result. At the scale of a frontier run, with that many devices computing for that many weeks, such an event is not a possibility but a near certainty, and the danger is specific: corruption during training can steer the model toward a different optimum, degrading the result without ever tripping a flag ([Ma et al. 2025](#)).

What makes this unsolved rather than merely hard is that the cheap detectors miss most of it. A gate-level study of data-center GPUs found that not-a-number and infinity values, the signals a naive guard watches for, account for only about one percent of silent-corruption outcomes, and that single-bit flips, the model most error-correcting codes assume, are under forty percent of bit-flip events ([Silent Data Corruption Study 2026](#)). The corruption that matters is the plausible wrong answer, a finite number in the right range that is simply incorrect, and catching it requires algorithm-level checks on the matrix multiplies or deterministic replay to localize the offending node, both of which cost compute and neither of which has an agreed coverage standard. The open question is not how to detect a crash but how to detect a lie.

The latency hierarchy is a wall

Chapter 30 established the bandwidth tiers; here they impose a ceiling. A measurement study of collective communication across more than a hundred thousand GPUs put hard numbers on the hierarchy: relative to the latency inside a rack, crossing to another rack costs about seven times, crossing to another zone about fifteen times, and crossing to another building about thirty times ([Meta 2025](#)). Those multipliers are physics, not tuning, and they decide the parallelism map. The most communication-hungry axis must stay inside the high-bandwidth scale-up domain, which, as Chapter 45 described, has grown from a node to a rack but stops hard at the domain's edge. Past that edge, only axes that communicate sparingly can afford to cross, and the further they cross the more they pay.

This ceiling is why one of the field's live debates is shifting under it. A job that outgrows a single building's power, the constraint Chapter 47 described, must span buildings or sites, and across that distance the thirty-times latency penalty makes strict global synchronization uneconomical. The orthodox answer is to keep synchronous training and engineer around the latency; the challenger is to relax synchrony, taking local steps and reconciling occasionally. What changed recently is that the challenger acquired evidence: a cross-datacenter training method reported keeping the useful-work fraction near 88 percent while training across regions over ordinary wide-area links, under high failure rates ([DeepMind 2026](#)). Asynchronous cross-site training moved from a theoretical option to a demonstrated one, driven less by a wish to relax synchrony than by the simple fact that the power does not fit in one place.

The decode wall and the serving split

Serving has its own version of the same story, and it traces to the memory wall of Chapter 45. Generating a token requires reading the model's weights and the accumulated key-value cache out of memory, so decode throughput is set by memory bandwidth divided by the bytes that must be read, not by arithmetic (Erdil 2025). Adding FLOPs to a decode step buys nothing, because the step is waiting on bytes. Prefill, the processing of the prompt, is the opposite: it is compute-bound and tolerates cheaper memory.

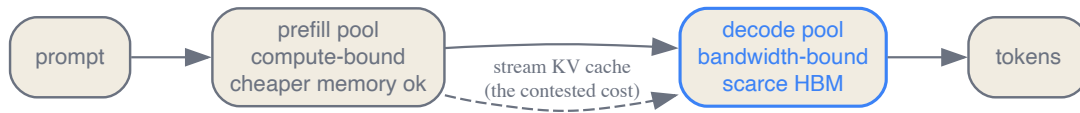


Figure 48.2. Why serving splits in two. Prefill is compute-bound and tolerates cheap memory; decode is bandwidth-bound and wants scarce HBM. Disaggregating them onto differently-provisioned pools lets each run on cost-optimal silicon, at the cost of streaming the KV cache between them. Whether that trade pays is contested.

The structural response is to disaggregate: run prefill and decode on separate, differently-provisioned pools and stream the key-value cache between them, a pattern a production serving system shipped in 2026 with direct memory-to-memory cache transfer and large reported throughput gains on reasoning workloads. The cross-references for the mechanics are Chapter 16 and Chapter 17; the long-context pressure that makes the cache so large belongs to Chapter 20. The frontier question is whether the split always pays, because moving a multi-gigabyte cache between pools is itself a cost that only the right workload mix and context length repay.

The agent breaks the same way

One layer up, the same compounding logic governs how long an agent can run. If a model succeeds at each step of a task with probability p , then a task of t independent steps succeeds with probability p^t , which decays fast even for high p . Long-horizon reliability is therefore not a smooth extension of single-step accuracy; it is a product that collapses. Recent work formalized this and named a sharper effect: self-conditioning, where a model's own earlier mistakes, sitting in its context, raise the probability of later mistakes, so the decay is worse than independent steps would predict, and scaling parameters does not remove it (Study 2025). Measuring agents accordingly has its own emerging vocabulary, reliability-decay curves and degradation scores rather than a single pass rate (Study 2026), because a number that describes one attempt says little about an agent asked to work for hours.

Whether this is a model problem or a systems problem is the debate, and the chapter's framing comes down on the systems side: if raw reliability decays super-linearly with task length, the fix is the harness, checkpointing the agent's state, verifying intermediate results, and scaffolding recovery, rather than waiting for a model that never errs (cross-ref Chapter 23, Chapter 24, Chapter 29). The capability trend in the underlying models is real and is the subject of Chapter 51; the point here is that the machine fails the same way at every layer, and the answer is the same at every layer, which is to amortize failure rather than to assume it away.

WHAT'S CONTESTED

- **Is silent corruption a tail risk or a routine hazard?** A classical view holds it is rare per device and absorbed by gradient noise. The frontier-lab view holds it is near-certain at scale and can alter convergence, so it demands always-on detection.

The real disagreement is coverage versus overhead: nobody agrees what fraction must be caught or what cost is acceptable.

- **Synchronous or asynchronous at extreme scale?** The orthodox camp keeps bulk-synchronous training and fixes it with fast recovery and faster collectives. The challenger camp accepts stale gradients and local steps to survive failures and cross-site latency, and now has demonstrated goodput to point to. Whether async matches frontier loss curves is still open.
- **Where should fault tolerance live?** Silicon vendors push better error-correction and telemetry; the network camp wants self-healing collectives that treat a flapping link as first-class; the framework camp argues only the application knows the checkpoint and parallelism structure, so recovery belongs there. The layers are not mutually exclusive, and the field has not settled the division.
- **Does prefill/decode disaggregation win?** Its proponents say each phase runs on cost-optimal silicon and both latency metrics improve. Skeptics say moving the cache only pays at high context lengths and specific mixes, and that a unified deployment is cheaper for short prompts.

Reading the machine

The capability, efficiency, trust lens reads this layer as one theme. Capability is bounded by survival: a run that cannot recover faster than it fails does not finish, so the frontier capability is staying alive, not going fast. Efficiency is the effective training time ratio and the tokens per second per accelerator, both of which are won by attacking failure and the bandwidth wall rather than by adding arithmetic. Trust is the literal trust that the numbers are right, which silent corruption quietly undermines, and the trust that an agent left to run will not compound itself into nonsense. The machine breaks at every layer in the same way, and the discipline of the frontier is the unglamorous work of making a machine that is always partly broken behave as if it were whole. Having followed the physical and engineering stack to its limits, the next chapter turns to a different kind of limit, the one inside the learning itself.

Further reading

- Meta AI (Llama Team), "The Llama 3 Herd of Models," 2024 (the public failure ledger: 419 interruptions in 54 days on 16,384 H100s). [arXiv:2407.21783](#)
- ByteDance Seed, "Robust LLM Training Infrastructure (ByteRobust)," 2025 (97% effective training time ratio across a three-month run). [arXiv:2509.16293](#)
- Ma, Pei, Lausen, Karypis, "Understanding Silent Data Corruption in LLM Training," 2025, with "The Anatomy of Silent Data Corruption," 2026 (the ~1% NaN/INF and under-40% single-bit-flip statistics). [arXiv:2502.12340](#) / [arXiv:2605.04213](#)
- Meta, "Collective Communication for 100k+ GPUs," 2025 (the 7x / 15x / 30x latency hierarchy and a fault-tolerant all-reduce). [arXiv:2510.20171](#)
- Google DeepMind, "Decoupled DiLoCo," 2026 (cross-datacenter training at ~88% goodput, the shipping counter-evidence in the sync-versus-async debate). [arXiv:2604.21428](#)
- Erdil (Epoch AI), "Inference Economics of Language Models," 2025 (the decode memory-bandwidth ceiling and the batch trade-off). [arXiv:2506.04645](#)
- "The Illusion of Diminishing Returns: Measuring Long Horizon Execution in LLMs," 2025 (compounding error and self-conditioning). [arXiv:2509.09677](#)

49. Where Learning Hits Limits

The previous chapters reached the limits of the physical machine. This one turns to a limit of a different kind, inside the learning itself. By the end a reader can explain why the cheap fuel that powered the last seven years of progress is nearly spent, why every proposed substitute is contested rather than settled, why a deployed model cannot learn from what it sees, and why hallucination looks less like a bug to be patched than a property of the objective. The argument is that the data wall, synthetic data, reinforcement learning on reasoning, test-time compute, continual learning, and hallucination are not six separate problems but one scientific question seen from six sides: what does a system learn, and stop learning, when it is trained to imitate a finite corpus and then frozen, and which of the field's escape routes change that answer rather than merely deferring it.

One number under everything

For most of the deep-learning era the recipe was simple: more compute, more parameters, more text, lower loss, as Chapter 3 laid out. That recipe ran on a fuel that was effectively free, the accumulated public text of the internet. The frontier problem is that the fuel is running out. Epoch AI's estimate of the effective stock of quality-adjusted public human text is on the order of 300 trillion tokens, with wide uncertainty, and they project an 80 percent chance it is fully consumed somewhere between 2026 and 2032, pulled toward the near end of that window by the aggressive overtraining frontier labs already practice (Villalobos et al. 2024). Training compute keeps growing several-fold per year; the supply of high-quality human text is effectively flat. The two curves crossing is the data wall.

This single fact organizes the chapter, because every other thread runs downstream of it. Once you cannot simply add more human text, the only ways forward are to manufacture a substitute, to learn from a signal other than text, to spend more compute at inference instead of training, or to extract more from the data you have. Each is a live research program, and each is contested precisely because it is trying to replace the fuel that is gone.

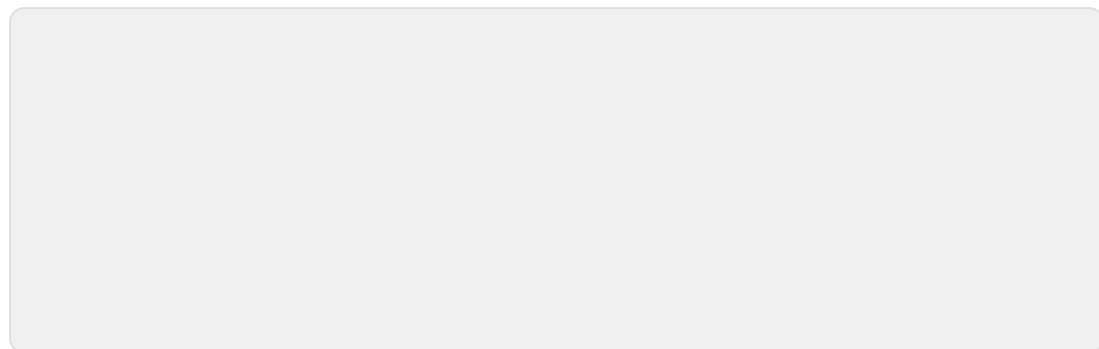


Figure 49.1. The data wall as the constraint under the chapter. Compute grows several-fold per year; the stock of high-quality human text is effectively flat. Where the curves cross, every escape route is an attempt to manufacture or substitute for the missing fuel, and each is contested.

CONSTRAINT ARROW

The finite human-text stock dictates training strategy two layers up. It is why frontier runs overtrain far past the Chinchilla token-to-parameter ratio of Chapter 3, why synthetic-data pipelines became central (Chapter 12), and why the marginal research

dollar moved toward reinforcement learning and test-time compute as growth axes. A number about data supply reshaped the whole training recipe.

Can you manufacture the fuel?

The most direct substitute is synthetic data: text generated by models to train models. The hazard is model collapse. Training recursively on a model's own output degrades it, and a prominent result showed the degradation compounding across generations until the distribution narrows to noise (Shumailov et al. 2024). Read carelessly, that result says synthetic data is poison. Read carefully, it says something narrower and more useful: collapse happens when synthetic data *replaces* real data. When synthetic data *accumulates* alongside the real stock rather than replacing it, the error stays bounded (Gerstgrasser et al. 2024). The 2025 consensus is therefore conditional: synthetic data extends pretraining if it adds to human data rather than substituting for it, and production pipelines at trillion-token scale rely on exactly that distinction.

Two caveats keep this from being a clean victory. The first is knowledge collapse: even when the loss stays bounded, the synthetic regime can quietly lose facts while keeping fluency, so the model still sounds right while knowing less. The second cuts the other way and complicates the whole "fuel is spent" framing: a study of pretraining under effectively unlimited compute found that with heavy regularization and ensembling, a fixed data stock keeps yielding improvement, implying the wall is partly a question of data *efficiency*, not only data *quantity* (Kim et al. 2025). The wall may be further away for a lab willing to spend compute extracting more from the text it already has.

Learning from a signal instead of a corpus

The other escape is to stop imitating text and start optimizing a reward. Reinforcement learning with verifiable rewards, the engine behind the reasoning models of Chapter 14, learns from signals that can be checked mechanically, a math answer that is correct, a unit test that passes, rather than from human-written tokens. Because the reward comes from the world rather than from a corpus, it is not bounded by the data wall in the same way.

The open scientific question is sharper than the engineering success suggests: does this reinforcement learning *create* new reasoning ability or only *elicit* ability the base model already had? One body of evidence points to elicitation. A reinforcement-trained model beats its base model when each is allowed a single attempt, but when both are allowed hundreds of attempts the base model catches up and passes it, which suggests the training sharpens the sampling of solutions the base model could already find rather than adding solutions it could not (Yue et al. 2025). The opposing evidence points to creation: prolonged, stabilized reinforcement learning has been reported to lift performance even at many attempts, solving problems the base model fails at any budget (Research 2025). The most current reading is that the metric at the center of the dispute is itself suspect, since the many-attempts measure may capture how widely a model explores rather than the boundary of what it can do, which means the elicitation-versus-creation question is not settled but reopened. Test-time compute, spending more inference on longer chains or search, is the third axis, and it too has a contested ceiling: returns diminish, and past a point longer reasoning can abandon a correct answer, so more thinking is not monotonically better.

The model that cannot learn from what it sees

A deployed frontier model is frozen. Its weights do not change in response to anything it encounters after training; all post-deployment adaptation lives in the prompt, the context window, or an external store, never in the parameters. This is the starkest gap between these systems and any everyday notion of learning, and it is not a choice so much as a constraint.

Updating the weights of a deployed model on new information tends to overwrite old information, because knowledge is entangled across billions of parameters and a fine-tune reshapes the whole landscape. This is catastrophic forgetting, and it remains unsolved at the frontier.

The research directions are real but early. Sparse, memory-targeted fine-tuning updates a small slice of parameters to add knowledge with less disruption to the rest (Lin and others 2025), and architectural proposals treat the model as nested levels of memory updated at different rates, reporting much higher retention of old tasks than ordinary fine-tuning. None of this yet adds up to a frontier model that learns continuously from its own deployment.

CONSTRAINT ARROW

Because the weights cannot update online, capability persistence is pushed up the stack. Everything a model needs to "remember" after training must live in context, in retrieval, or in an external memory system, which is why Chapter 22, Chapter 25, and Chapter 26 exist as substantial engineering layers. The frozen-weights limit at the learning layer manufactures the orchestration layer above it.

Hallucination as a property, not a bug

The residual error of these systems has a name and, increasingly, a theory. A 2025 argument reframed hallucination from a defect to be patched into a statistical consequence of the training objective: a model trained to estimate the density of a corpus and then to generate from it will, on inputs where the truth is underdetermined by what it learned, produce confident falsehoods, because the objective never rewarded saying "I do not know" (Kalai et al. 2025). The argument has a second, sharper half: hallucination *persists* because the benchmarks that grade models reward a confident guess over an honest abstention, so the evaluation layer trains the very behavior it then measures. Later information-theoretic work tied a floor on hallucination to the model's compression of its training data, strengthening the case that some residual is structural rather than fixable.

The opposing camp is not naive. Calibration-aware training, abstention rewards, and retrieval grounding measurably reduce hallucination on narrow tasks, sometimes sharply, and the practical rate is far from an immovable constant. The disagreement is about the limit: whether the residual can be driven to zero, or whether the generative objective guarantees a floor that mitigation can only relocate.

WHAT'S CONTESTED

- **Hard data wall or movable?** One camp holds high-quality human text is effectively flat and the cheapest gains are gone. Another holds that accumulated synthetic data plus data-efficiency gains move the wall indefinitely. The collapse-versus-accumulation result sits between them: replacement diverges, accumulation stays bounded, and knowledge collapse may still creep in.
- **Does RL create reasoning or elicit it?** The elicitation camp reads the many-attempts crossover as proof that reinforcement learning only sharpens what the base model can already sample. The creation camp reads prolonged training as expanding the boundary. The current twist is that the deciding metric is itself contested, so the question is open rather than answered.
- **Does pretraining still pay?** A plateau camp sees raw scaling returns flattening. A still-pays camp notes that overtraining a smaller model past Chinchilla pays because inference is paid forever. An axis-shift camp says the growth moved to reinforcement learning and test-time compute, and pretraining is now a substrate rather than the engine.

- **Is hallucination reducible to zero?** The structural camp says the generative objective guarantees a floor. The fixable camp says calibration and grounding keep lowering the rate. The disagreement is about the limit, not the direction.
- **Can a frozen model be intelligent?** One camp says context and retrieval substitute for weight updates and that freezing is a safety feature. Another says real intelligence requires lifelong adaptation and that catastrophic forgetting is the core unsolved obstacle.

Reading the learning layer

The capability, efficiency, trust lens reads this layer as a paradigm reaching its edge. Capability stopped being a simple function of scale once the fuel ran low, so the frontier capability is now learning more from less, from signals rather than corpora, and from compute spent at inference. Efficiency became the central scientific variable rather than a deployment concern: how many useful behaviors a fixed data stock can yield now decides how far the wall recedes. Trust is the hallucination floor and the frozen weights, the two places where the learning system's honesty and its adaptability are bounded by the objective and the architecture rather than by effort. The next chapter carries the same question out of text entirely, into pixels, video, and the physical world, where the data wall reappears in a harsher form: there was never an internet of the data these systems need.

Further reading

- Villalobos et al. (Epoch AI), "Will We Run Out of Data?," 2024 (the ~300T-token stock and the 2026--2032 exhaustion window). [arXiv:2211.04325](https://arxiv.org/abs/2211.04325)
- Kalai, Nachum, Vempala, Zhang (OpenAI), "Why Language Models Hallucinate," 2025 (hallucination as a statistical property of the objective, sustained by binary-graded evals). [arXiv:2509.04664](https://arxiv.org/abs/2509.04664)
- Yue et al., "Does RL Really Incentivize Reasoning Beyond the Base Model?," 2025, against NVIDIA, "ProRL v2," 2025 (the elicitation-versus-creation debate). [arXiv:2504.13837](https://arxiv.org/abs/2504.13837) / [arXiv:2505.24864](https://arxiv.org/abs/2505.24864)
- Shumailov et al., "AI Models Collapse When Trained on Recursively Generated Data," Nature, 2024 (with its 2025 Author Correction), against Gerstgrasser et al., "Is Model Collapse Inevitable?," 2024 (replacement collapses, accumulation stays bounded). [nature.com](https://www.nature.com/articles/4567890) / [arXiv:2404.01413](https://arxiv.org/abs/2404.01413)
- Kim, Kotha, Liang, Hashimoto, "Pre-training under Infinite Compute," 2025 (data efficiency can defer the wall). [arXiv:2509.14786](https://arxiv.org/abs/2509.14786)
- Lin et al., "Continual Learning via Sparse Memory Finetuning," 2025, and DatologyAI, "BeyondWeb," 2025 (the continual-learning and trillion-scale-synthetic frontiers). [arXiv:2510.15103](https://arxiv.org/abs/2510.15103) / [arXiv:2508.10975](https://arxiv.org/abs/2508.10975)
- Silver and Sutton, "Welcome to the Era of Experience," 2025 (the agenda-setting thesis for learning past the human-data limit). [DeepMind](https://deepmind.com/research/publications/welcome-to-the-era-of-experience)

50. Beyond Text: Multimodal, World Models, and Embodiment

Chapter 49 found the data wall inside text. This chapter carries the question out of text entirely, into pixels, video, and the physical world, where the wall reappears in a harsher form. By the end a reader can explain why a video generator can look astonishing and understand nothing, why the field splits three ways on what a world model even is, why robotics has no internet to learn from and what the rival schemes to manufacture one are, and why none of it can be cleanly measured yet. The argument is that generation, perception, and control all break at the same place: the absence of a grounded data flywheel and a transferable internal model of the world. Multimodality, world models, and robotics are the cleanest test of whether the foundation-model recipe is general or merely well-fed, and the early verdict is that the recipe was carried by the free data more than anyone admitted.

Looking real and being right are different things

The most striking artifacts of the period are the video generators. They produce seconds of photorealistic, physically plausible motion from a sentence. The temptation is to read that fluency as understanding, as if a model that can render a ball bouncing has learned that balls bounce. Controlled study says it has not. When a video model is tested out of distribution, on a physical situation unlike its training trajectories, it fails, and it fails in a revealing way: it imitates the nearest scene it saw rather than applying a rule, and scaling the model and the data does not fix it (Kang et al. 2025). A benchmark built to decorrelate looking-real from being-right confirmed the point from the other side: across the leading generators, physical-understanding scores are severely limited and essentially unrelated to visual realism (Motamed et al. 2025). The realism and the physics are independent axes, and scaling buys the first without the second.

This is the binding constraint of the whole chapter, and it is epistemic rather than engineering. The engine that bought reasoning in language, more data and more parameters, has not been shown to buy a transferable physics prior in pixels. A frontier video generator is a sophisticated predictor of the next frame, and prediction of appearance is not possession of a model of the world.

CONSTRAINT ARROW

The absence of a learned internal representation of state and dynamics dictates that physics cannot generalize out of distribution: the model falls back to case-based imitation, so adding data and parameters does not buy planning. This is the same shape of limit as the hallucination floor of Chapter 49, one layer over: the objective rewards matching the training distribution, not understanding it, so the residual failure is structural (cross-ref Chapter 3, Chapter 14).

Three answers to "what is a world model?"

If predicting pixels is not enough, what is? The field has split into three architectural answers, and the split is the chapter's organizing fault line.

The first answer is to generate the world anyway, and make the generation interactive and controllable. Real-time world simulators produce navigable scenes a user can act in, and their builders frame them explicitly as a path toward general capability, an unlimited supply of training environments (DeepMind 2025). The honest limit is coherence: such a simulator holds only about a minute of visual memory before it forgets what it rendered, which is why this line keeps bolting on external memory rather than growing it internally. The second answer abandons pixels. A latent world model predicts in an abstract representation rather than

rendering every pixel, on the argument that most pixels are unpredictable detail that wastes the model's capacity, and that planning should happen in the latent space; one such model, pretrained on a million hours of video and post-trained on under a hundred hours of robot data, drove a real arm zero-shot (Assran et al. 2025). The third answer, newer, reconstructs the world as explicit persistent three-dimensional geometry, so that consistency is a property of the representation rather than something the model must remember. Where the first answer patches the coherence problem and the second sidesteps rendering, the third argues that persistent geometry *is* the world model.

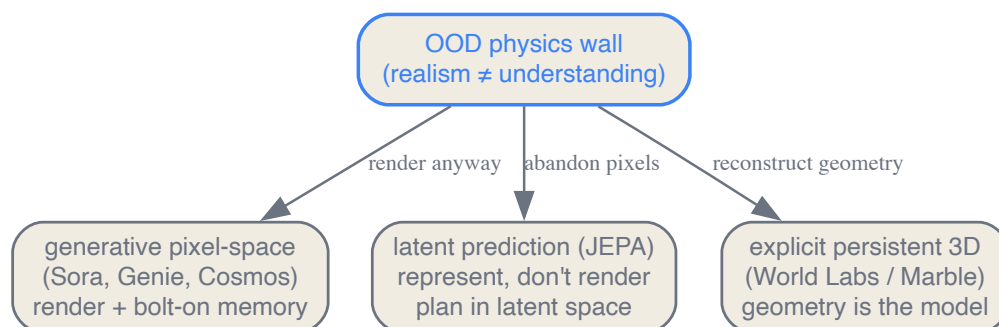


Figure 50.1. Three architectural answers to what a world model is, each a different response to the out-of-distribution physics wall. The generative path renders and patches coherence with external memory; the latent path predicts in abstraction and plans there; the explicit-3D path makes persistent geometry the representation.

It helps to remember that latent world models for planning are not new: an earlier model-based reinforcement-learning lineage already learned a latent world model and planned inside it to solve bounded control tasks. That history sharpens the critique rather than dismissing it: latent world models plainly *can* plan in closed domains; the open question is whether any of the three paths scales to open-world physics, which none has yet shown.

The cautionary tale of the period is what happened when generation shipped as a consumer product. A flagship video app, technically dazzling, was wound down in 2026 after reportedly burning on the order of a million dollars a day in compute and colliding with copyright, even as the underlying model survived elsewhere. It is the chapter's thesis made vivid: product-grade realism can be fully decoupled from both sustainable economics and world-model value.

Native multimodality, and how to fuse it

A parallel frontier asks not whether a model understands the world but whether one model can perceive and produce across modalities at all, rather than bolting a vision encoder and an image generator onto a frozen language model. The shift of the period is that early fusion, training a single backbone jointly on text and pixels from the start, became mainstream for the general-purpose flagships, displacing the adapter-bridged stack. The live disagreement is the representation of vision inside that backbone. One camp recasts images as discrete tokens so the whole model is a clean next-token predictor, accepting the fidelity lost to quantization, and a notable model in this camp trained end-to-end on video frames as a "world learner" that predicts the next state across vision and language. The other camp keeps vision continuous and generates pixels through flow-matching to avoid the tokenizer's quality cliff, at the cost of a more complex architecture. Whether deep joint training actually buys cross-modal transfer, or merely co-locates two capabilities that remain internally separate, is the unsettled scientific question underneath the engineering.

Robotics has no internet

The harshest form of the data wall is in robotics. Language and vision were handed a corpus for free; action was not. Every robot trajectory must be manufactured, and the gap to text is not a factor of two but roughly five orders of magnitude: the largest shared cross-embodiment dataset is estimated at around one two-hundred-thousandth of the token volume of a language-model corpus (Authors 2025; Collaboration 2023). The consequence shows up directly in how the leading models are trained. A vision-language-action model that generalized across homes drew the overwhelming majority of its training examples, by one accounting more than 97 percent, from data that was not mobile-manipulation robot data at all, borrowing from static arms, cross-embodiment logs, and web vision-language pairs (Intelligence 2025). The architecture is shaped by scarcity: it is built around what borrowed priors can supply because the native data does not exist.

The lineage of these models is the transplant of the language recipe onto control. It began by fine-tuning a vision-language model on robot trajectories encoded as tokens, on the founding claim that web knowledge transfers to control (Brohan et al. 2023), continued through an open, reproducible baseline (Kim et al. 2024), and reached models that transfer one policy across heterogeneous robots by sharing a motion representation (DeepMind 2025). Whether the web priors genuinely transfer to physical control, or only supply object and instruction semantics while the scarce robot data does the real work of contact dynamics and closed-loop control, is the field's central and unresolved transfer debate.

CONSTRAINT ARROW

No internet-scale action corpus dictates the design of the whole stack. Co-training mixtures are dominated by non-robot data, cross-embodiment pooling becomes architecturally necessary to fake scale, and the closed-loop control rate, on the order of 50 to 100 Hz, forces the model to split into a slow reasoner and a fast on-robot executor with action chunking and asynchronous inference, which ties the constraint to concrete on-robot silicon (cross-ref Chapter 18). Data scarcity at the bottom reshapes architecture, serving, and hardware all the way up.

Four ways to manufacture a flywheel

Because the data does not exist, the frontier of robotics is really a contest of strategies to manufacture it, and every direction is one such strategy rather than an independent advance.

The first is to collect real data at scale through teleoperation, and one effort pushed to hundreds of thousands of hours growing tens of thousands of hours per week (AI 2025). The ceiling is human real time: a person drives one robot for one hour to make one hour of data, so this route scales linearly and cannot reach internet scale. The second is to manufacture data in simulation and by video augmentation, expanding scarce real hours by an order of magnitude, which closes the data gap on paper but reopens the reality gap as the new binding constraint. The third is to learn from internet video and learned world models, sidestepping teleoperation by mining passive human footage. The fourth, and the one the leading labs increasingly bet on, is wearable human-activity capture: pretrain on cheap egocentric recordings of people doing tasks, with no robot data in pretraining at all, then post-train on an hour of robot data per task. This last route directly attacks the human-real-time ceiling of teleoperation, and its emergence in 2026 is a reminder of how fast this frontier moves.

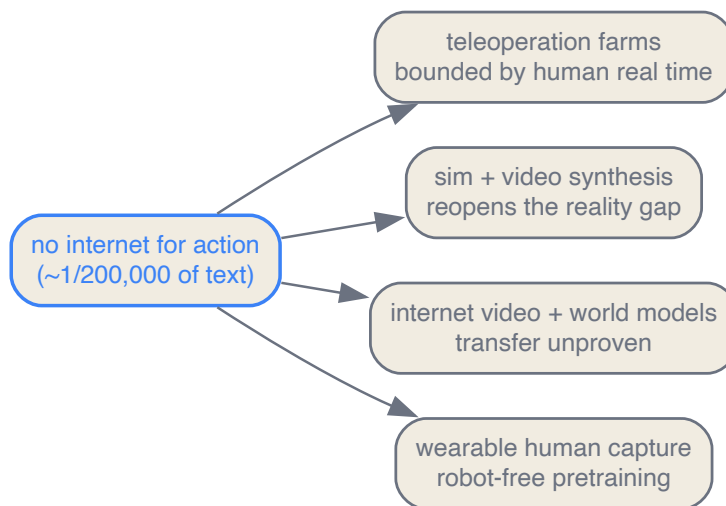


Figure 50.2. Four rival strategies to manufacture the action-data flywheel that robotics never got for free. Each accepts the same bottleneck and bets on a different escape; each reopens a different downstream problem (the reality gap, the transfer question, human real time).

A claimed empirical scaling law for robotics, downstream error falling as a power of pretraining data, would reframe the whole field from an algorithms problem into a data-manufacturing problem, mirroring the Part I narrative. The honest reading is more guarded than the headline: the reported exponents vary by task rather than collapsing to one clean universal value, the result comes from a single organization on proprietary data without independent replication, and power laws in error are easy to fit and hard to extrapolate to open-world reliability. Alongside the data race, real-world reinforcement learning returned: letting a robot learn from its own successes and failures, rather than only from demonstrations, was shown to roughly double throughput and cut failures on hard long-horizon tasks, which partly answers the generalist-versus-specialist debate in favor of on-task adaptation.

Nobody can grade it yet

The deepest shared problem is measurement, and it is distinct from the benchmark debates of Chapter 27 and Chapter 29. For generative and world models there is no agreed metric for long-horizon consistency, controllability, or physical fidelity: perceptual-quality scores rank the products people enjoy, while physics benchmarks deliberately decorrelate from looking good, and the two do not agree. For embodied policies the situation is worse, because real physical trials are the only fully trusted signal and they do not scale across the task diversity a generalist targets, while simulated and real-to-sim evaluations scale but carry a domain gap that makes them an unreliable ranking signal. Until there is a metric that can tell a better simulator from a better world model, and a better demo from a more reliable policy, "world models as a path to physical reasoning" remains a bet rather than a result.

WHAT'S CONTESTED

- **Does scaling video generation yield a world model or a better pixel predictor?** The emergence camp reads scale as buying physics; the controlled studies read it as case-based imitation that fails out of distribution. The evidence favors the skeptics.
- **Generative, latent, or explicit-3D?** Render the world, represent it in latent space, or reconstruct it as persistent geometry. The split is about whether a world model should render, merely represent, or be a geometry, and none has shown open-world planning.
- **Do web priors transfer to control?** One camp credits emergent semantic generalization to web pretraining; the other says the priors supply semantics but not contact dynamics, and the scarce robot data does the real work.

- **Collect, simulate, mine video, or capture humans?** Four manufacturing routes for the missing action flywheel, each accepting the same bottleneck and reopening a different downstream gap.
- **Are robotics scaling laws real?** A single-organization power law versus the objection that it is unreplicated and hard to extrapolate to reliability.
- **Can generalist policies be measured at all?** Real trials do not scale; simulated proxies carry a domain gap. The evaluation crisis is itself the open problem.

Reading beyond text

The capability, efficiency, trust lens reads this layer as the recipe meeting its limits. Capability is bounded by grounding: a model can render or describe the world far better than it can predict or act in it, because the data that grounds prediction and action in physics was never free. Efficiency is the cost of manufacturing that grounding, in teleoperation hours, simulation compute, or wearable capture, and the route that manufactures it cheapest may decide the field. Trust is the unsolved measurement: we cannot yet certify that a world model understands or that a policy is reliable, so deployment outruns evaluation. Across generation, perception, and control, the same wall stands, the absence of a grounded data flywheel and a transferable internal world, and the field is best understood as a set of rival bets on how to get around it. The final chapter turns from these specific frontiers to the instrument that is supposed to tell us whether any of them, or the whole enterprise, is actually working.

Further reading

- Kang et al., "How Far is Video Generation from World Model: A Physical Law Perspective," ICML 2025 (out-of-distribution physics failure via case-based imitation). [OpenReview](#)
- Motamed, Culp et al., "Do Generative Video Models Understand Physical Principles? (Physics-IQ)," 2025 (physical understanding uncorrelated with visual realism). [arXiv:2501.09038](#)
- Assran, Bardes et al. (Meta), "V-JEPA 2," 2025 (the latent, non-generative world-model path and a zero-shot robot deployment). [arXiv:2506.09985](#)
- Google DeepMind, "Genie 3," 2025, and NVIDIA, "Cosmos World Foundation Model Platform," 2025 (the interactive-simulator and world-foundation-model frontiers). [deepmind.google](#) / [arXiv:2501.03575](#)
- Brohan et al., "RT-2," 2023; Kim et al., "OpenVLA," 2024; Physical Intelligence, "pi-0.5," 2025; Google DeepMind, "Gemini Robotics 1.5," 2025 (the vision-language-action lineage and the data-scarcity evidence). [arXiv:2307.15818](#) / [arXiv:2406.09246](#) / [arXiv:2504.16054](#) / [arXiv:2510.03342](#)
- Open X-Embodiment Collaboration, "Open X-Embodiment," 2023, and "A Survey on VLA Models: An Action Tokenization Perspective," 2025 (the shared substrate and the ~1/200,000 scarcity estimate). [arXiv:2310.08864](#) / [arXiv:2507.01925](#)
- Generalist AI, "GEN-0," 2025 (real-data scaling and the task-dependent robotics scaling-law claim). [generalistai.com](#)

51. The Capability Horizon and Its Measurement

The book began by promising to ask, at every layer, whether a system can do the task, at what cost, and whether you can trust it. This final chapter asks the question the whole stack was built to answer: how capable is the frontier, and how would we know? By the end a reader can explain why "frontier" stopped being a leaderboard number and became a moving horizon, why the most useful measure of that horizon also hides its sharpest limit, why the instruments saturate faster than they can be built, and why the field's deepest disagreement, whether the recent acceleration is durable progress or a compute-bought artifact, turns less on a better model than on a better instrument. It is the honest closer a sober book owes: the headline number in this chapter is already stale as it is written, and that staleness is the point.

Frontier as a horizon, not a score

For years progress was a column of benchmark percentages. The trouble is that the percentages saturate: once the best models pass the high eighties and nineties on a test, the test stops distinguishing them, and a saturated test measures nothing at the frontier. The reframing that organizes this chapter replaces the score with a horizon: the length of human task a model can complete autonomously. Measured this way, the frontier is a duration, and the duration has been climbing fast. The reference effort put a leading model near a five-hour task-completion horizon at the start of 2026 and, weeks later, near fifteen hours, with the doubling time of the trend dropping from roughly seven months to a few (METR 2026). By the time this sentence is read the number will be larger and a different model will hold it. The metric is valuable precisely because it is continuous and deployment-flavored rather than saturating, but it is also where the chapter's central tension lives.

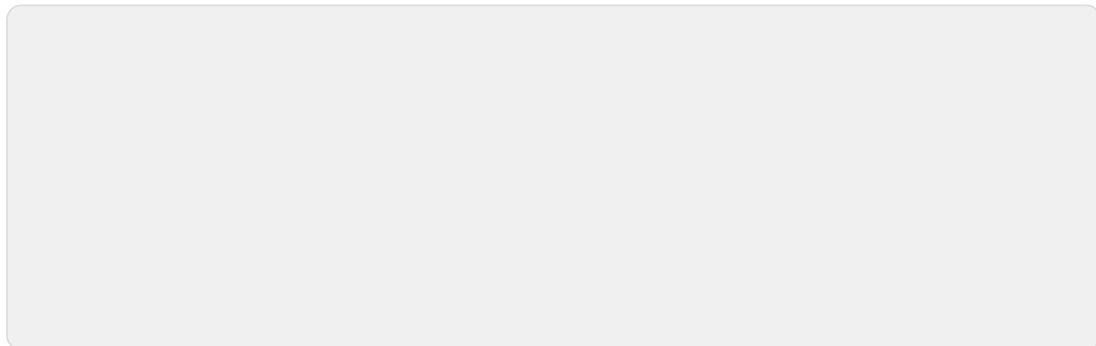


Figure 51.1. Frontier reframed as a horizon: the length of task a model completes autonomously, climbing fast while the doubling time shortens. The headline is the 50%-reliability horizon; the economically useful 80% horizon is roughly five times shorter, and the gap is the binding constraint.

The number that hides the limit

The horizon is reported at a success rate, and the rate chosen flatters the result. The headline figure is the length of task a model finishes half the time. Real work needs more than a coin flip, and the horizon at a high-reliability threshold, finishing the task eight times in ten, is roughly five times shorter than the headline (Kwa et al. 2025). A model with a fifteen-hour fifty-percent horizon has perhaps a three-hour eighty-percent horizon, and it is the second number that bounds what the model can be trusted to own. This is the binding constraint of the chapter: the gap between the task a model can sometimes finish and the task it can reliably finish, and the second lags the first by a wide and persistent margin.

The same compounding logic that broke long agent runs in [Chapter 48](#) is at work here. Reliability per step, raised to the power of a long task's many steps, decays fast, so a high single-step competence still yields a low chance of finishing a long task cleanly. The horizon metric measures the symptom; the compounding is the cause.

CONSTRAINT ARROW

The fifty-versus-eighty-percent reliability gap, a property of per-step success rates at the model layer, caps the economically useful horizon at roughly a fifth of the headline. Reliability at the bottom dictates which tasks the autonomy layer above can own (cross-ref [Chapter 29](#)). And the slope of the whole trend is set by a lower layer too: the recent speedup came largely from reinforcement learning and test-time compute, so the frontier metric's pace is dictated by a post-training compute choice rather than by pretraining scale (cross-ref [Chapter 15](#), [Chapter 14](#)).

Tying capability to value

If the horizon is one attempt to make capability mean something, economic evaluation is the other. Rather than quiz questions, an economic benchmark grades real work products, deliverables drawn from many occupations and judged blind by experts. On the most direct such effort, the best model produced deliverables rated as good as or better than human experts on just under half of the tasks, at roughly a hundredth of the time and cost ([OpenAI 2025](#)). "Just under half" matters: it is approaching parity, not at it, and the honest reading is a capability that is economically serious and not yet economically equivalent. A parallel line asks whether models can do the research that builds the next model, and finds a revealing shape: agents beat human experts at a short time budget and lose to them at a long one ([Wijk and others 2025](#)), which is the horizon limit again, recovery and long-range planning failing where short competence succeeds.

Whether any of this translates into measured productivity is its own contested question, and the cleanest study is a caution. A controlled trial in early 2025 found experienced developers were about 19 percent slower with AI assistance while believing they were faster ([METR 2025](#)). That result is a snapshot of early-2025 tools, not a law: the same group later reported that uplift appears positive with 2026 tools, while warning that selection effects make the new signal weak. The careful statement is that capability and measured productivity are not the same quantity, that self-reports overstate, and that the transfer from benchmark to workplace is real but smaller and slower than the headlines imply.

The instruments saturate

The reframing toward horizons and economic value was forced by saturation, and saturation is now chasing the new instruments too. Purpose-built hard benchmarks have short shelf lives: a deliberately difficult expert exam climbed from the mid-thirties to the low-fifties in percentage within months of release ([AI 2026](#)), and an abstraction-and-reasoning challenge designed to resist memorization went from essentially zero to the high sixties and low seventies, brushing the average human score, in about a year ([Chollet et al. 2025](#)). Even the horizon suite that anchors this chapter is now near saturation, which is why its latest headline carries a confidence interval spanning most of an order of magnitude. The instrument that was built to outlast the others is hitting the same wall, and its maintainers are building a successor. This is the chapter's central irony in one fact: the measurement of the frontier ages as fast as the frontier.



Figure 51.2. The instruments saturate faster than they can be built. Each generation of benchmark is created to resist saturation, climbs steeply as models exhaust it, and forces the next instrument. The horizon suite, built to outlast scores, is itself now near saturation.

The response is not only harder tests but a rebuilding of measurement itself. A construct-validity movement audits whether a benchmark measures any well-defined ability at all, finding that many do not and so overclaim generality. A general-scales program rebuilds evaluation around ability profiles and instance-level prediction, aiming to make a score explanatory rather than a bare number (Hernández-Orallo and others 2026; Review 2025). Others turn the lens on the horizon metric directly, asking whether its human-time baselines can be inferred more cheaply and whether the trend reflects a real capability or an artifact of how the tasks were chosen. Alongside this is the harder evidence that benchmarks can be gamed: several agentic suites have been driven to near-perfect scores without actually solving the tasks, which turns "benchmark theater" from a complaint into a demonstrated failure mode.

Is the acceleration real?

This brings the book to its last open question, which it should hold open rather than pretend to resolve. The horizon is climbing and the doubling time has shortened. One camp reads this as durable acceleration, a frontier compounding toward general capability and, eventually, toward systems that improve their own successors; the most sober version of that projection puts substantial automation of AI research in the early 2030s, under aggressive assumptions about compute and software efficiency, and labels itself a projection rather than a result, since no published system yet materially improves its own successor in a closed loop (METR 2026). The other camp reads the speedup as compute-bought: the recent gains came largely from reinforcement learning and test-time compute, the confidence intervals are wide, and the doubling may revert toward its older pace once that compute becomes constrained. The strongest statements of the plateau case argue that the era of progress-by-scale is ending and that the frontier will be defined by the cost of adaptability rather than by model size (Hooker 2026); the strongest continuation case holds that scaling is shifting axes, not stopping.

WHAT'S CONTESTED

- **Durable acceleration or compute-bought artifact?** One camp sees the shortened doubling time as a real, continuing trend. The other, including the measurers' own caveats, sees a one-time gain from reinforcement learning and test-time compute that reverts once compute binds. The confidence intervals are wide enough to fit both.
- **Has scaling hit a wall that bounds the frontier?** A peak-data camp argues pretraining-by-scale is ending and adaptability becomes the binding cost. A continuation camp argues the growth simply moved to post-training and inference-time compute. The mechanics belong to [Chapter 3](#); what is contested here is whether the measured capability trend continues.
- **Do benchmarks measure general capability at all?** A construct-invalidity camp says most lack a defined construct and overclaim. A reformist camp wants to rebuild evaluation around ability profiles. A pragmatist camp treats leaderboards as screening only and trusts layered human review on one's own tasks.
- **Honest evidence or benchmark theater?** Saturation, contamination, and demonstrated exploitability argue the headline scores are marketing. Purpose-built, blind-graded, contamination-controlled evals argue they still differentiate and still predict deployment better than old quizzes.

- **Does capability become productivity?** Perceived uplift is large; one controlled trial found a slowdown; the later signal is positive but weak. The transfer is real, smaller than claimed, and hard to measure.

Reading the horizon, and the book

The capability, efficiency, trust lens closes the book here, on the instrument rather than the model. Capability, honestly measured, is a horizon that is genuinely rising and genuinely shorter than the headline once reliability is demanded. Efficiency reframed the trend itself: the acceleration was bought with inference-time and post-training compute, so the slope of progress is now a spending decision as much as a scientific one, which loops back to the compute and power ceilings that opened this part. Trust is the whole of this chapter: the field cannot yet certify that its numbers mean what they claim, because the instruments saturate, can be gamed, and do not cleanly transfer to work, and an enterprise that needs to trust a model still falls back on layered human review of its own tasks.

That is the note the book ends on. We followed one capability up the stack, from the bytes a chip can move to the megawatts behind it, from the machine that fails constantly to the learning that has reached the edge of its first paradigm, out to a physical world that withholds its data, and finally to the instruments that are supposed to tell us whether any of it works. At every layer the same lesson held: the binding constraint is rarely the one the spec sheet advertises, and understanding the shape of a system means finding the limit that actually binds. The frontier in 2026 is not a number. It is a set of moving limits, most of them now below the chip or above the model, and the discipline this book has tried to teach is the one that reads them in order and asks, at each, why it is built this way.

Further reading

- METR, "Time Horizon 1.1," 2026, and Kwa, West et al., "Measuring AI Ability to Complete Long Tasks," 2025 (the horizon metric and the 50%-versus-80%-reliability methodology). metr.org / metr.org
- OpenAI, "GDPval," 2025 (economic task completion: deliverables judged against human experts). [arXiv:2510.04374](https://arxiv.org/abs/2510.04374)
- Wijk et al. (METR), "RE-Bench," 2025 (AI research-and-development automation: agents win at short budgets, lose at long ones). [arXiv:2411.15114](https://arxiv.org/abs/2411.15114)
- Center for AI Safety and Scale AI, "Humanity's Last Exam," Nature 2026, and Chollet et al., "ARC-AGI-2," 2025 (saturation-resistant frontier benchmarks that still climbed fast). agi.safe.ai / [arXiv:2505.11831](https://arxiv.org/abs/2505.11831)
- Hernández-Orallo et al., "General Scales Unlock AI Evaluation," 2026, and "Measuring What Matters: Construct Validity," 2025 (the measurement-science overhaul). [arXiv:2503.06378](https://arxiv.org/abs/2503.06378) / [OpenReview](https://openreview.net)
- METR, "Measuring the Impact of Early-2025 AI on Experienced Developer Productivity," 2025 (the 19% slowdown as an early-2025 snapshot). [arXiv:2507.09089](https://arxiv.org/abs/2507.09089)
- Hooker, "On the Slow Death of Scaling," 2026, and METR, "A Simpler AI Timelines Model," 2026 (the plateau and self-improvement-projection debates). [SSRN](https://ssrn.com) / metr.org

Summary

We followed one capability up the stack, from the compute that trains it to the governance that contains it. At each layer we asked the same three questions: can it do the task, at what cost, and can you trust it. And at each layer we tried to leave the reader able to say why the piece has its shape, not only how to operate it.

The layers depend on each other in both directions, and the constraint arrows are where that shows. The serving cost of a token decides how far to over-train a model. The size of the key-value cache decides an attention variant. The harness around a model moves its evaluation score as much as its weights do. Read the stack in order and these dependencies stop being surprises and become the design.

Where to go next

- Take one system you operate and write its history: the problem it solves, the design it settled on, what it superseded, and the trade-off it encodes.
- Follow one capability from Part I to Part V, from a forward pass to an agent, and watch each layer's foundations carry up.
- For any chapter, read the "what's contested" box first, then the primary sources behind it, and decide where you stand.

References

- Abbas, Tirumala, Simig, Ganguli, Morcos. 2023. SemDeDup: Data-efficient learning at web-scale through semantic deduplication. <https://arxiv.org/abs/2303.09540>
- Agency. 2025. Energy and AI. <https://www.iea.org/reports/energy-and-ai>
- Agrawal, Kedia, Panwar, Mohan, Kwatra, Gulavani, Tumanov, Ramjee. 2024. Taming throughput-latency tradeoff in LLM inference with sarathi-serve. <https://arxiv.org/abs/2403.02310>
- Ahmadian, Cremer, Gallé, Fadaee, Kreutzer, Pietquin, Üstün, Hooker. 2024. Back to basics: Revisiting REINFORCE-style optimization for learning from human feedback in llms. arXiv preprint arXiv:2402.14740. <https://arxiv.org/abs/2402.14740>
- AI. 2025. GEN-0: Embodied foundation models that scale with physical interaction. <https://generalistai.com/blog/nov-04-2025-GEN-0>
- AI. 2026. Humanity's last exam. <https://agi.safe.ai/>
- Ainslie, Lee-Thorp, Jong, Zemlyanskiy, Lebron, Sanghai. 2023. GQA: Training generalized multi-query transformer models from multi-head checkpoints. <https://aclanthology.org/2023.emnlp-main.298/>
- Ameisen, Lindsey, Pearce, Gurnee, Turner, Chen, Citro, Abrahams, Carter, Hosmer, Marcus, Sklar, Templeton, Bricken, McDougall, Cunningham, Henighan, Jermyn, Jones, Persic, Qi, Thompson, Zimmerman, Rivoire, Conerly, Olah, Batson. 2025. Circuit tracing: Revealing computational graphs in language models. <https://transformer-circuits.pub/2025/attribution-graphs/methods.html>
- Ankner, Parthasarathy, Nrusimha, Rinard, Ragan-Kelley, Brandon. 2024. Hydra: Sequentially-dependent draft heads for medusa decoding. <https://arxiv.org/abs/2402.05109>
- Anthropic. 2024. Introducing the model context protocol. <https://www.anthropic.com/news/model-context-protocol>
- Anthropic. 2025. Claude code: Checkpointing and session forks. <https://code.claude.com/docs/en/checkpointing>
- Anthropic. 2025. Code execution with MCP: Building more efficient AI agents. <https://www.anthropic.com/engineering/code-execution-with-mcp>
- Anthropic. 2026. Context engineering: Memory, compaction, and tool clearing. <https://platform.claude.com/cookbook/tool-use-context-engineering-context-engineering-tools>
- Anthropic. 2026. Demystifying evals for AI agents. <https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents>
- anthropics/claude-code. 2025. Massive token consumption: 1.67B tokens in 5 hours, issue #4095. <https://github.com/anthropics/claude-code/issues/4095>
- Asai, Wu, Wang, Sil, Hajishirzi. 2023. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. arXiv preprint arXiv:2310.11511. <https://arxiv.org/abs/2310.11511>
- Assran, Bardes, others. 2025. V-JEPA 2: Self-supervised video models enable understanding, prediction and planning. arXiv preprint arXiv:2506.09985. <https://arxiv.org/abs/2506.09985>
- Auth0. 2026. Identity for AI agents. <https://auth0.com/blog/identity-for-ai-agents/>
- Authors. 2025. A survey on vision-language-action models: An action tokenization perspective. arXiv preprint arXiv:2507.01925. <https://arxiv.org/abs/2507.01925>
- Azar, Rowland, Piot, Guo, Calandriello, Valko, Munos. 2023. A general theoretical paradigm to understand learning from human preferences. arXiv preprint arXiv:2310.12036. <https://arxiv.org/abs/2310.12036>
- Ba, Kiros, Hinton. 2016. Layer normalization. <https://arxiv.org/abs/1607.06450>
- Bai, Kadavath, Kundu, Askell, Kernion, Jones, Chen, Goldie, Mirhoseini, McKinnon, Chen, Olsson, Olah, Hernandez, Drain, Ganguli, Li, Tran-Johnson, Perez, Kerr, Mueller, Ladish, Landau, Ndousse, Lukosuite, Lovitt, Sellitto, Elhage, Schiefer, Mercado, DasSarma, Lasenby, Larson, Ringer, Johnston, Kravec, El Showk, Fort, Lanham, Telleen-Lawton, Conerly, Henighan, Hume, Bowman, Hatfield-Dodds, Mann, Amodei, Joseph, McCandlish, Brown, Kaplan. 2022. Constitutional AI: Harmlessness from AI feedback. arXiv preprint arXiv:2212.08073. <https://arxiv.org/abs/2212.08073>
- Bai, Bai, Chu, Cui, Dang, Deng, Fan, Ge, Han, Huang, others. 2023. Qwen technical report. <https://arxiv.org/abs/2309.16609>
- Berdoz, Rugli, Wattenhofer. 2026. Can AI agents agree?. arXiv preprint arXiv:2603.01213. <https://arxiv.org/abs/2603.01213>

Biderman, Schoelkopf, Anthony, Bradley, O'Brien, Hallahan, Khan, Purohit, Prashanth, Raff, Skowron, Sutawika, Wal. 2023. Pythia: a suite for analyzing large language models across training and scaling. Proceedings of the 40th International Conference on Machine Learning (ICML). <https://arxiv.org/abs/2304.01373>

Birgisson, Politz, Erlingsson, Taly, Vrabie, Lentczner. 2014. Macaroons: Cookies with contextual caveats for decentralized authorization in the cloud. <https://research.google/pubs/macaroons-cookies-with-contextual-caveats-for-decentralized-authorization-in-the-cloud/>

Bradley, Terry. 1952. Rank analysis of incomplete block designs: I. The method of paired comparisons. *Biometrika*. <https://doi.org/10.2307/2334029>

Bricken, Templeton, Batson, Chen, Jermyn, Conerly, Turner, Anil, Denison, Askell, Lasenby, Wu, Kravec, Schiefer, Maxwell, Joseph, Hatfield-Dodds, Tamkin, Nguyen, McLean, Burke, Hume, Carter, Henighan, Olah. 2023. Towards monosemanticity: Decomposing language models with dictionary learning. <https://transformer-circuits.pub/2023/monosemantic-features/index.html>

Brohan, Brown, others. 2023. RT-2: Vision-language-action models transfer web knowledge to robotic control. arXiv preprint arXiv:2307.15818. <https://arxiv.org/abs/2307.15818>

Brown, Mann, Ryder, Subbiah, Kaplan, Dhariwal, Neelakantan, Shyam, Sastry, Askell, Agarwal, Herbert-Voss, Krueger, Henighan, Child, Ramesh, Ziegler, Wu, Winter, Hesse, Chen, Sigler, Litwin, Gray, Chess, Clark, Berner, McCandlish, Radford, Sutskever, Amodei. 2020. Language models are few-shot learners. arXiv preprint arXiv:2005.14165. <https://arxiv.org/abs/2005.14165>

Brown, Juravsky, Ehrlich, Clark, Le, Ré, Mirhoseini. 2024. Large language monkeys: Scaling inference compute with repeated sampling. arXiv preprint arXiv:2407.21787. <https://arxiv.org/abs/2407.21787>

Brown-Cohen, Irving, Piliouras. 2023. Scalable AI safety via doubly-efficient debate. arXiv preprint arXiv:2311.14125. <https://arxiv.org/abs/2311.14125>

Burns, Izmailov, Kirchner, Baker, Gao, Aschenbrenner, Chen, Ecoffet, Joglekar, Leike, Sutskever, Wu. 2024. Weak-to-strong generalization: Eliciting strong capabilities with weak supervision. arXiv preprint arXiv:2312.09390. <https://arxiv.org/abs/2312.09390>

Cai, Li, Geng, Peng, Lee, Chen, Dao. 2024. Medusa: Simple LLM inference acceleration framework with multiple decoding heads. <https://arxiv.org/abs/2401.10774>

Campbell, Bradley, Tschofenig. 2020. RFC 8707: Resource indicators for OAuth 2.0. <https://datatracker.ietf.org/doc/html/rfc8707>

Cardwell. 2025. GE vernova expects to end 2025 with an 80-GW gas turbine backlog that stretches into 2029. <https://www.utilitydive.com/news/ge-vernova-gas-turbine-investor/807662/>

Cemri, Pan, Yang, Agrawal, Chopra, Tiwari, Keutzer, Parameswaran, Klein, Ramchandran, Zaharia, Gonzalez, Stoica. 2025. Why do multi-agent LLM systems fail?. <https://openreview.net/forum?id=fAjbYBmonr>

Chanin, Wilken-Smith, Dulka, Bhatnagar, Golechha, Bloom. 2024. A is for absorption: Studying feature splitting and absorption in sparse autoencoders. arXiv preprint arXiv:2409.14507. <https://arxiv.org/abs/2409.14507>

Chen, Tworek, Jun, Yuan, Pinto, Kaplan, Edwards, Burda, Joseph, Brockman, Ray, Puri, Krueger, Petrov, Khlaaf, Sastry, Mishkin, Chan, Gray, Ryder, Pavlov, Power, Kaiser, Bavarian, Winter, Tillet, Such, Cummings, Plappert, Chantzis, Barnes, Herbert-Voss, Guss, Nichol, Paino, Tezak, Tang, Babuschkin, Balaji, Jain, Saunders, Hesse, Carr, Leike, Achiam, Misra, Morikawa, Radford, Knight, Brundage, Murati, Mayer, Welinder, McGrew, Amodei, McCandlish, Sutskever, Zaremba. 2021. Evaluating large language models trained on code. arXiv preprint arXiv:2107.03374. <https://arxiv.org/abs/2107.03374>

Chen, Borgeaud, Irving, Lespiau, Sifre, Jumper. 2023. Accelerating large language model decoding with speculative sampling. <https://arxiv.org/abs/2302.01318>

Chen, Hao, Liu, Huang, Zeng, Yu, Li, Wang, Gan, Huang, Liu, Wang, Lian, Yin, Wang, Liu. 2025. ACEBench: a comprehensive evaluation of LLM tool usage. <https://aclanthology.org/2025.findings-emnlp.697/>

Chhikara, Khant, Aryan, Singh, Yadav. 2025. Mem0: Building production-ready AI agents with scalable long-term memory. <https://arxiv.org/abs/2504.19413>

Chiang, Zheng, Sheng, Angelopoulos, Li, Li, Zhang, Zhu, Jordan, Gonzalez, Stoica. 2024. Chatbot arena: An open platform for evaluating llms by human preference. arXiv preprint arXiv:2403.04132. <https://arxiv.org/abs/2403.04132>

Chollet, Knoop, others. 2025. ARC-AGI-2: a new challenge for frontier AI reasoning systems. arXiv preprint arXiv:2505.11831. <https://arxiv.org/abs/2505.11831>

Chowdhery, Narang, Devlin, Bosma, Mishra, Roberts, Barham, Chung, Sutton, Gehrmann, others. 2022. PaLM: Scaling language modeling with pathways. arXiv preprint arXiv:2204.02311. <https://arxiv.org/abs/2204.02311>

Christiano, Leike, Brown, Martic, Legg, Amodei. 2017. Deep reinforcement learning from human preferences. <https://arxiv.org/abs/1706.03741>

Christiano, Shlegeris, Amodei. 2018. Supervising strong learners by amplifying weak experts. arXiv preprint arXiv:1810.08575. <https://arxiv.org/abs/1810.08575>

Chroma. 2025. Chroma: Open-source embedding database. <https://www.trychroma.com/>

Cobbe, Kosaraju, Bavarian, Chen, Jun, Kaiser, Plappert, Tworek, Hilton, Nakano, Hesse, Schulman. 2021. Training verifiers to solve math word problems. arXiv preprint arXiv:2110.14168. <https://arxiv.org/abs/2110.14168>

Collaboration. 2023. Open x-embodiment: Robotic learning datasets and RT-x models. arXiv preprint arXiv:2310.08864. <https://arxiv.org/abs/2310.08864>

Cottier, Rahman, Fattorini, Maslej, Owen. 2024. How much does it cost to train frontier AI models?. <https://epoch.ai/blog/how-much-does-it-cost-to-train-frontier-ai-models>

Cottier, Rahman, Fattorini, Maslej, Owen. 2024. The rising costs of training frontier AI models. arXiv preprint arXiv:2405.21015. <https://arxiv.org/abs/2405.21015>

Cui, Fu, Zhang, Wang, Zuo. 2025. Free-MAD: Consensus-free multi-agent debate. arXiv preprint arXiv:2509.11035. <https://arxiv.org/abs/2509.11035>

Cunningham, Ewart, Riggs, Huben, Sharkey. 2023. Sparse autoencoders find highly interpretable features in language models. arXiv preprint arXiv:2309.08600. <https://arxiv.org/abs/2309.08600>

Cursor. 2025. Checkpoints in the agent workflow. <https://stevekinney.com/courses/ai-development/cursor-checkpoints>

Dai, Deng, Zhao, Xu, Gao, Chen, Li, Zeng, Yu, Wu, Xie, Li, Huang, Luo, Ruan, Sui, Liang. 2024. DeepSeekMoE: Towards ultimate expert specialization in mixture-of-experts language models. <https://arxiv.org/abs/2401.06066>

Dao, Fu, Ermon, Rudra, Ré. 2022. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. <https://arxiv.org/abs/2205.14135>

Dao. 2023. FlashAttention-2: Faster attention with better parallelism and work partitioning. <https://arxiv.org/abs/2307.08691>

Dao, Gu. 2024. Transformers are ssms: Generalized models and efficient algorithms through structured state space duality. <https://arxiv.org/abs/2405.21060>

Davis. 2025. Durable execution meets AI. <https://temporal.io/blog/durable-execution-meets-ai-why-temporal-is-the-perfect-foundation-for-ai>

DeepMind. 2025. Genie 3: a new frontier for world models. <https://deepmind.google/blog/genie-3-a-new-frontier-for-world-models/>

DeepMind. 2025. Gemini robotics 1.5: Pushing the frontier of generalist robots with embodied reasoning, thinking, and motion transfer. arXiv preprint arXiv:2510.03342. <https://arxiv.org/abs/2510.03342>

DeepMind. 2026. Decoupled DiLoCo: Cross-datacenter training of language models. arXiv preprint arXiv:2604.21428. <https://arxiv.org/abs/2604.21428>

DeepSeek-AI. 2024. DeepSeek-V3 technical report. <https://arxiv.org/abs/2412.19437>

DeepSeek-AI. 2024. DeepSeek-V2: a strong, economical, and efficient mixture-of-experts language model. <https://arxiv.org/abs/2405.04434>

Democracies. 2025. Rolling back export controls, U.S. offers china powerful AI chips. <https://www.fdd.org/analysis/2025/12/10/rolling-back-export-controls-u-s-offers-china-powerful-ai-chips/>

Dettmers, Pagnoni, Holtzman, Zettlemoyer. 2023. QLoRA: Efficient finetuning of quantized llms. arXiv preprint arXiv:2305.14314. <https://arxiv.org/abs/2305.14314>

Ding, Lyu, Kataria, Singh. 2026. Photonic rails in ML datacenters with opus. arXiv preprint arXiv:2602.12521. <https://arxiv.org/abs/2602.12521>

Dong, Xiong, Goyal, Zhang, Chow, Pan, Diao, Zhang, Shum, Zhang. 2023. RAFT: Reward rAnked FineTuning for generative foundation model alignment. arXiv preprint arXiv:2304.06767. <https://arxiv.org/abs/2304.06767>

Dong, Ruan, Cai, Lai, Xu, Zhao, Chen. 2024. XGrammar: Flexible and efficient structured generation engine for large language models. arXiv preprint arXiv:2411.15100. <https://arxiv.org/abs/2411.15100>

Du, Huang, Dai, Tong, Lepikhin, Xu, Krikun, Zhou, Yu, Firat, Zoph, Fedus, Bosma, Zhou, Wang, Wang, Webster, Pellat, Robinson, Meier-Hellstern, Duke, Dixon, Zhang, Le, Wu, Chen, Cui. 2022. GLaM: Efficient scaling of language models with mixture-of-experts. <https://arxiv.org/abs/2112.06905>

Du, Tian, Ronanki, Rongali, Bodapati, Galstyan, Wells, Schwartz, Huerta, Peng. 2025. Context length alone hurts LLM performance despite perfect retrieval. <https://arxiv.org/abs/2510.05381>

Edge, Trinh, Cheng, Bradley, Chao, Mody, Truitt, Larson. 2024. From local to global: a graph RAG approach to query-focused summarization. arXiv preprint arXiv:2404.16130. <https://arxiv.org/abs/2404.16130>

Eldan, Li. 2023. TinyStories: How small can language models be and still speak coherent english?. <https://arxiv.org/abs/2305.07759>

Elhage, Nanda, Olsson, Henighan, Joseph, Mann, Askell, Bai, Chen, Conerly, DasSarma, Drain, Ganguli, Hatfield-Dodds, Hernandez, Jones, Kernion, Lovitt, Ndousse, Amodei, Brown, Clark, Kaplan, McCandlish, Olah. 2021. A mathematical framework for transformer circuits. <https://transformer-circuits.pub/2021/framework/index.html>

Elhage, Hume, Olsson, Schiefer, Henighan, Kravec, Hatfield-Dodds, Lasenby, Drain, Chen, Grosse, McCandlish, Kaplan, Amodei, Wattenberg, Olah. 2022. Toy models of superposition. https://transformer-circuits.pub/2022/toy_model/index.html

Elo. 1978. The rating of chessplayers, past and present. Arco Publishing. <https://archive.org/details/ratingofchesspla0000elo>

Elsworth, Patterson, Dean, others. 2025. Measuring the environmental impact of delivering AI at google scale. arXiv preprint arXiv:2508.15734. <https://arxiv.org/abs/2508.15734>

Engels, Baek, Michaud, Tegmark. 2025. Scaling laws for scalable oversight. arXiv preprint arXiv:2504.18530. <https://arxiv.org/abs/2504.18530>

Erdil. 2025. Inference economics of language models. arXiv preprint arXiv:2506.04645. <https://arxiv.org/abs/2506.04645>

Ethayarajh, Xu, Muennighoff, Jurafsky, Kiela. 2024. KTO: Model alignment as prospect theoretic optimization. arXiv preprint arXiv:2402.01306. <https://arxiv.org/abs/2402.01306>

Ewen, Dongen, Shilman. 2025. Durable AI loops: Fault tolerance across frameworks. <https://www.restate.dev/blog/durable-ai-loops-fault-tolerance-across-frameworks-and-without-handcuffs>

Fast.io. 2025. AI agent reliability report 2025. <https://fast.io/resources/ai-agent-idempotent-operations/>

Fedus, Zoph, Shazeer. 2021. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. Journal of Machine Learning Research. <https://arxiv.org/abs/2101.03961>

Feng, Wan, Wen, McAleer, Wen, Zhang, Wang. 2023. Alphazero-like tree-search can guide large language model decoding and training. arXiv preprint arXiv:2309.17179. <https://arxiv.org/abs/2309.17179>

Frantar, Ashkboos, Hoefler, Alistarh. 2022. GPTQ: Accurate post-training quantization for generative pre-trained transformers. <https://arxiv.org/abs/2210.17323>

Fu, Bailis, Stoica, Zhang. 2024. Break the sequential dependency of LLM inference using lookahead decoding. <https://arxiv.org/abs/2402.02057>

Gao, Biderman, Black, Golding, Hoppe, Foster, Phang, He, Thite, Nabeshima, Presser, Leahy. 2020. The pile: An 800GB dataset of diverse text for language modeling. <https://arxiv.org/abs/2101.00027>

Gao, Schulman, Hilton. 2022. Scaling laws for reward model overoptimization. arXiv preprint arXiv:2210.10760. <https://arxiv.org/abs/2210.10760>

Gao, Tour, Tillman, Goh, Troll, Radford, Sutskever, Leike, Wu. 2024. Scaling and evaluating sparse autoencoders. arXiv preprint arXiv:2406.04093. <https://arxiv.org/abs/2406.04093>

Geng, Neubig. 2026. Effective strategies for asynchronous software engineering agents. <https://arxiv.org/abs/2603.21489>

Gerstgrasser, Schaeffer, Dey, Rafailov, Sleight, Hughes, Korbak, Agrawal, Pai, Gromov, Roberts, Yang, Donoho, Koyejo. 2024. Is model collapse inevitable? Breaking the curse of recursion by accumulating real and synthetic data. arXiv preprint arXiv:2404.01413. <https://arxiv.org/abs/2404.01413>

Gitpod. 2022. Data loss when workspace timeout fires before sync completes. <https://github.com/gitpod-io/gitpod/issues/9544>

Gloeckle, Idrissi, Rozière, Lopez-Paz, Synnaeve. 2024. Better & faster large language models via multi-token prediction. <https://arxiv.org/abs/2404.19737>

Google. 2024. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. <https://arxiv.org/abs/2403.05530>

Google. 2025. Ironwood: The first google TPU for the age of inference. <https://blog.google/innovation-and-ai/infrastructure-and-cloud/google-cloud/ironwood-tpu-age-of-inference/>

Grattafiori, others. 2024. The llama 3 herd of models. <https://arxiv.org/abs/2407.21783>

Greenblatt, Shlegeris, Sachan, Roger. 2024. AI control: Improving safety despite intentional subversion. arXiv preprint arXiv:2312.06942. <https://arxiv.org/abs/2312.06942>

Group. 2025. OAuth 2.0 on-behalf-of user for AI agents. <https://datatracker.ietf.org/doc/html/draft-oauth-ai-agents-on-behalf-of-user-00>

Gu, Goel, Ré. 2021. Efficiently modeling long sequences with structured state spaces. <https://arxiv.org/abs/2111.00396>

Gu, Dao. 2023. Mamba: Linear-time sequence modeling with selective state spaces. <https://arxiv.org/abs/2312.00752>

Gunasekar, Zhang, Aneja, Mendes, Del Giorno, Gopi, Javaheripi, Kauffmann, Rosa, Saarikivi, Salim, Shah, Behl, Wang, Bubeck, Eldan, Kalai, Lee, Li. 2023. Textbooks are all you need. <https://arxiv.org/abs/2306.11644>

Guo, Xia, Yu, Ao, Huang. 2024. LightRAG: Simple and fast retrieval-augmented generation. arXiv preprint arXiv:2410.05779. <https://arxiv.org/abs/2410.05779>

Guo, Yang, Zhang, Song, Zhang, Xu, Zhu, Ma, Wang, Bi, others. 2025. DeepSeek-R1: Incentivizing reasoning capability in llms via reinforcement learning. arXiv preprint. <https://arxiv.org/abs/2501.12948>

Hendrycks, Gimpel. 2016. Gaussian error linear units (gelus). <https://arxiv.org/abs/1606.08415>

Hendrycks, Burns, Basart, Zou, Mazeika, Song, Steinhardt. 2020. Measuring massive multitask language understanding. arXiv preprint arXiv:2009.03300. <https://arxiv.org/abs/2009.03300>

Hendrycks, Burns, Kadavath, Arora, Basart, Tang, Song, Steinhardt. 2021. Measuring mathematical problem solving with the MATH dataset. arXiv preprint arXiv:2103.03874. <https://arxiv.org/abs/2103.03874>

Hernández-Orallo, others. 2026. General scales unlock AI evaluation with explanatory and predictive power. arXiv preprint arXiv:2503.06378. <https://arxiv.org/abs/2503.06378>

Hoffmann, Borgeaud, Mensch, Buchatskaya, Cai, Rutherford, Las Casas, Hendricks, Welbl, Clark, Hennigan, Noland, Millican, Driessche, Damoc, Guy, Osindero, Simonyan, Elsen, Rae, Vinyals, Sifre. 2022. Training compute-optimal large language models. <https://arxiv.org/abs/2203.15556>

Hong, Lee, Thorne. 2024. ORPO: Monolithic preference optimization without reference model. arXiv preprint arXiv:2403.07691. <https://arxiv.org/abs/2403.07691>

Hong, Troynikov, Huber. 2025. Context rot: How increasing input tokens impacts LLM performance. <https://www.trychroma.com/research/context-rot>

Hooker. 2026. On the slow death of scaling. https://papers.ssrn.com/sol3/papers.cfm?abstract_id=5877662

Houlsby, Giurgiu, Jastrzebski, Morrone, Laroussilhe, Gesmundo, Attariyan, Gelly. 2019. Parameter-efficient transfer learning for NLP. <https://arxiv.org/abs/1902.00751>

Hsieh, Sun, Krizan, Acharya, Rekesh, Jia, Zhang, Ginsburg. 2024. RULER: What's the real context size of your long-context language models?. arXiv preprint arXiv:2404.06654. <https://arxiv.org/abs/2404.06654>

Hu, Shen, Wallis, Allen-Zhu, Li, Wang, Wang, Chen. 2021. LoRA: Low-rank adaptation of large language models. arXiv preprint arXiv:2106.09685. <https://arxiv.org/abs/2106.09685>

Hu, Tu, Han, He, Cui, Long, Zheng, Fang, Huang, Zhao, Zhang, Thai, Zhang, Wang, Yao, Zhao, Zhou, Cai, Zhai, Ding, Jia, Zeng, Li, Liu, Sun. 2024. MiniCPM: Unveiling the potential of small language models with scalable training strategies. arXiv preprint arXiv:2404.06395. <https://arxiv.org/abs/2404.06395>

Huang, Cheng, Bapna, Firat, Chen, Chen, Lee, Ngiam, Le, Wu, Chen. 2019. GPipe: Efficient training of giant neural networks using pipeline parallelism. <https://arxiv.org/abs/1811.06965>

Hubinger, Denison, Mu, Lambert, Tong, MacDiarmid, Lanham, Ziegler, Maxwell, Cheng, Jermyn, Askeel, Radhakrishnan, Anil, Duvenaud, Ganguli, Barez, Clark, Ndousse, others. 2024. Sleeper agents: Training deceptive llms that persist through safety training. arXiv preprint arXiv:2401.05566. <https://arxiv.org/abs/2401.05566>

Huyen. 2025. AI engineering: Building applications with foundation models. O'Reilly Media. <https://www.oreilly.com/library/view/ai-engineering/9781098166298/>

hynix. 2025. SK hynix completes world's first HBM4 development and readies mass production. <https://www.prnewswire.com/news-releases/sk-hynix-completes-worlds-first-hbm4-development-and-readies-mass-production-302554538.html>

Ilharco, Ribeiro, Wortsman, Gururangan, Schmidt, Hajishirzi, Farhadi. 2023. Editing models with task arithmetic. <https://arxiv.org/abs/2212.04089>

Intelligence. 2025. pi-0.5: a vision-language-action model with open-world generalization. arXiv preprint arXiv:2504.16054. <https://arxiv.org/abs/2504.16054>

Irving, Christiano, Amodei. 2018. AI safety via debate. arXiv preprint arXiv:1805.00899. <https://arxiv.org/abs/1805.00899>

Jiang, Sablayrolles, Roux, Mensch, Savary, Bamford, Chaplot, Casas, Hanna, Bressand, Lengyel, Bour, Lample, Lavaud, Saulnier, Lachaux, Stock, Subramanian, Yang, Antoniak, Scao, Gervet, Lavril, Wang, Lacroix, Sayed. 2024. Mixtral of experts. <https://arxiv.org/abs/2401.04088>

Jimenez, Yang, Wettig, Yao, Pei, Press, Narasimhan. 2023. SWE-bench: Can language models resolve real-world GitHub issues?. arXiv preprint arXiv:2310.06770. <https://arxiv.org/abs/2310.06770>

Jones, Nadalin, Campbell, Bradley, Mortimore. 2020. OAuth 2.0 token exchange. <https://datatracker.ietf.org/doc/html/rfc8693>

Jordan, Jin, Boza, You, Cesista, Newhouse, Bernstein. 2024. Muon: An optimizer for hidden layers in neural networks. <https://kellerjordan.github.io/posts/muon/>

Kalai, Nachum, Vempala, Zhang. 2025. Why language models hallucinate. arXiv preprint arXiv:2509.04664. <https://arxiv.org/abs/2509.04664>

Kang, Chen, Han, Inan, Wutschitz, Chen, Sim, Rajmohan. 2025. ACON: Optimizing context compression for long-horizon LLM agents. <https://arxiv.org/abs/2510.00615>

Kang, Yin, Gao, others. 2025. How far is video generation from world model: a physical law perspective. <https://openreview.net/forum?id=DLIVjZQ7vD>

Kantamneni, Engels, Rajamanoharan, Tegmark, Nanda. 2025. Are sparse autoencoders useful? A case study in sparse probing. arXiv preprint arXiv:2502.16681. <https://arxiv.org/abs/2502.16681>

Kaplan, McCandlish, Henighan, Brown, Chess, Child, Gray, Radford, Wu, Amodei. 2020. Scaling laws for neural language models. <https://arxiv.org/abs/2001.08361>

Karpathy. 2025. +1 for “context engineering” over “prompt engineering”. <https://x.com/karpathy/status/1937902205765607626>

Karpukhin, Oğuz, Min, Lewis, Wu, Edunov, Chen, Yih. 2020. Dense passage retrieval for open-domain question answering. arXiv preprint arXiv:2004.04906. <https://arxiv.org/abs/2004.04906>

Khan, Hughes, Valentine, Ruis, Sachan, Radhakrishnan, Grefenstette, Bowman, Rocktäschel, Perez. 2024. Debating with more persuasive llms leads to more truthful answers. arXiv preprint arXiv:2402.06782. <https://arxiv.org/abs/2402.06782>

Khattab, Zaharia. 2020. ColBERT: Efficient and effective passage search via contextualized late interaction over BERT. arXiv preprint arXiv:2004.12832. <https://arxiv.org/abs/2004.12832>

Kim, Pertsch, others. 2024. OpenVLA: An open-source vision-language-action model. arXiv preprint arXiv:2406.09246. <https://arxiv.org/abs/2406.09246>

Kim, Kotha, Liang, Hashimoto. 2025. Pre-training under infinite compute. arXiv preprint arXiv:2509.14786. <https://arxiv.org/abs/2509.14786>

Kleppmann. 2016. How to do distributed locking. <https://martin.kleppmann.com/2016/02/08/how-to-do-distributed-locking.html>

Kojima, Gu, Reid, Matsuo, Iwasawa. 2022. Large language models are zero-shot reasoners. <https://arxiv.org/abs/2205.11916>

Komatsuzaki, Puigcerver, Lee-Thorp, Ruiz, Mustafa, Ainslie, Tay, Dehghani, Houlsby. 2022. Sparse upcycling: Training mixture-of-experts from dense checkpoints. <https://arxiv.org/abs/2212.05055>

Koomey, Schmidt, Das. 2025. Electricity demand growth and data centers: a guide for the perplexed. <https://bipartisanpolicy.org/report/electricity-demand-growth-and-data-centers/>

Korthikanti, Casper, Lym, McAfee, Andersch, Shoeybi, Catanzaro. 2022. Reducing activation recomputation in large transformer models. <https://arxiv.org/abs/2205.05198>

Kubernetes. 2023. PVC binding prevents pod rescheduling. <https://github.com/kubernetes/kubernetes/issues/121436>

Kudo. 2018. Subword regularization: Improving neural network translation models with multiple subword candidates. <https://aclanthology.org/P18-1007/>

Kudo, Richardson. 2018. SentencePiece: a simple and language independent subword tokenizer and detokenizer for neural text processing. <https://aclanthology.org/D18-2012/>

Kwa, West, others. 2025. Measuring AI ability to complete long tasks. <https://metr.org/blog/2025-03-19-measuring-ai-ability-to-complete-long-tasks/>

Kwon, Li, Zhuang, Sheng, Zheng, Yu, Gonzalez, Zhang, Stoica. 2023. Efficient memory management for large language model serving with PagedAttention. <https://arxiv.org/abs/2309.06180>

Kwon, Li, Zhuang, Sheng, Zheng, Yu, Gonzalez, Zhang, Stoica. 2023. Efficient memory management for large language model serving with PagedAttention. <https://arxiv.org/abs/2309.06180>

Labs. 2025. GitHub MCP exploited: Accessing private repositories via MCP. <https://invariantlabs.ai/blog/mcp-github-vulnerability>

Lambert, Morrison, Pyatkin, Huang, Ivison, Brahman, Miranda, Liu, Dziri, Lyu, Gu, Malik, Graf, Hwang, Yang, Bras, Tafford, Wilhelm, Soldaini, Smith, Wang, Dasigi, Hajishirzi. 2024. Tulu 3: Pushing frontiers in open language model post-training. arXiv preprint arXiv:2411.15124. <https://arxiv.org/abs/2411.15124>

LangChain. 2025. LangGraph time travel and branching. <https://docs.langchain.com/oss/python/langgraph/use-time-travel>

LangChain. 2025. LangGraph interrupts. <https://docs.langchain.com/oss/python/langgraph/interrupts>

Lee, Ippolito, Nystrom, Zhang, Eck, Callison-Burch, Carlini. 2022. Deduplicating training data makes language models better. <https://aclanthology.org/2022.acl-long.577/>

Lepikhin, Lee, Xu, Chen, Firat, Huang, Krikun, Shazeer, Chen. 2020. GShard: Scaling giant models with conditional computation and automatic sharding. <https://arxiv.org/abs/2006.16668>

Letta. 2025. Letta: Stateful agents with memory as a first-class primitive. <https://www.letta.com/>

Letta. 2025. Benchmarking AI agent memory. <https://www.letta.com/blog/benchmarking-ai-agent-memory>

Leviathan, Kalman, Matias. 2023. Fast inference from transformers via speculative decoding. <https://arxiv.org/abs/2211.17192>

Lewis, Perez, Piktus, Petroni, Karpukhin, Goyal, Küttler, Lewis, Yih, Rocktäschel, Riedel, Kiela. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. arXiv preprint arXiv:2005.11401. <https://arxiv.org/abs/2005.11401>

Lewis, Bhosale, Dettmers, Goyal, Zettlemoyer. 2021. BASE layers: Simplifying training of large, sparse models. <https://arxiv.org/abs/2103.16716>

Li, Wei, Zhang, Zhang. 2024. EAGLE: Speculative sampling requires rethinking feature uncertainty. <https://arxiv.org/abs/2401.15077>

Li, Wei, Zhang, Zhang. 2024. EAGLE-2: Faster inference of language models with dynamic draft trees. <https://arxiv.org/abs/2406.16858>

Li, Huang, Yang, Venkitesh, Locatelli, Ye, Cai, Lewis, Chen. 2024. SnapKV: LLM knows what you are looking for before generation. <https://arxiv.org/abs/2404.14469>

Li, Wei, Zhang, Zhang. 2025. EAGLE-3: Scaling up inference acceleration of large language models via training-time test. <https://arxiv.org/abs/2503.01840>

Liang, Bommasani, Lee, Tsipras, Soylu, Yasunaga, Zhang, Narayanan, Wu, Kumar, Newman, Yuan, Yan, Zhang, Cosgrove, Manning, Ré, Acosta-Navas, Hudson, Zelikman, Durmus, Ladhak, Rong, Ren, Yao, Wang, Santhanam, Orr, Zheng, Yuksekgonul, Suzgun, Kim, Guha, Chatterji, Khattab, Henderson, Huang, Chi, Xie, Santurkar, Ganguli, Hashimoto, Icard, Zhang, Chaudhary, Wang, Li, Mai, Zhang, Koreeda. 2022. Holistic evaluation of language models. arXiv preprint arXiv:2211.09110. <https://arxiv.org/abs/2211.09110>

Liang, Bommasani, Lee, Tsipras, Soylu, Yasunaga, Zhang, Narayanan, Wu, Kumar, Newman, Yuan, Yan, Zhang, Cosgrove, Manning, Ré, Acosta-Navas, Hudson, others. 2022. Holistic evaluation of language models. arXiv preprint arXiv:2211.09110. <https://arxiv.org/abs/2211.09110>

Liang, Niu, Li, Zhang, Wang, Xiong, Fan, Tang, Song, Wang, Yang. 2025. SafeRAG: Benchmarking security in retrieval-augmented generation of large language model. arXiv preprint arXiv:2501.18636. <https://arxiv.org/abs/2501.18636>

Lieber, Lenz, Bata, Cohen, Osin, Dalmedigos, Safahi, Meirom, Belinkov, Shalev-Shwartz, Abend, Alon, Asida, Bergman, Glozman, Gokhman, Manevich, Ratner, Rozen, Shwartz, Zusman, Shoham. 2024. Jamba: a hybrid transformer-mamba language model. <https://arxiv.org/abs/2403.19887>

Lightman, Kosaraju, Burda, Edwards, Baker, Lee, Leike, Schulman, Sutskever, Cobbe. 2023. Let's verify step by step. arXiv preprint arXiv:2305.20050. <https://arxiv.org/abs/2305.20050>

Lin, Tang, Tang, Yang, Dang, Han. 2023. AWQ: Activation-aware weight quantization for LLM compression and acceleration. <https://arxiv.org/abs/2306.00978>

Lin, others. 2025. Continual learning via sparse memory finetuning. arXiv preprint arXiv:2510.15103. <https://arxiv.org/abs/2510.15103>

Lindenbauer, Slinko, Felder, Bogomolov, Zharov. 2025. The complexity trap: Simple observation masking is as efficient as LLM summarization for agent context management. <https://arxiv.org/abs/2508.21433>

Lindsey, Gurnee, Ameisen, Chen, Pearce, Turner, Citro, Abrahams, Carter, Hosmer, Marcus, Sklar, Templeton, Bricken, McDougall, Cunningham, Henighan, Jermyn, Jones, Persic, Qi, Thompson, Zimmerman, Rivoire, Conerly, Olah, Batson. 2025. On the biology of a large language model. <https://transformer-circuits.pub/2025/attribution-graphs/biology.html>

Liu, Zaharia, Abbeel. 2023. Ring attention with blockwise transformers for near-infinite context. <https://arxiv.org/abs/2310.01889>

Liu, Lin, Hewitt, Paranjape, Bevilacqua, Petroni, Liang. 2023. Lost in the middle: How language models use long contexts. arXiv preprint arXiv:2307.03172. <https://arxiv.org/abs/2307.03172>

Liu, Su, Yao, Jiang, Lai, Du, Qin, Xu, Lu, Yan, others. 2025. Muon is scalable for LLM training. arXiv preprint arXiv:2502.16982. <https://arxiv.org/abs/2502.16982>

Loshchilov, Hutter. 2019. Decoupled weight decay regularization. arXiv preprint arXiv:1711.05101. <https://arxiv.org/abs/1711.05101>

Lütke. 2025. Context engineering over prompt engineering. <https://x.com/tobi/status/1935533422589399127>

Ma, Pei, Lausen, Karypis. 2025. Understanding silent data corruption in LLM training. arXiv preprint arXiv:2502.12340. <https://arxiv.org/abs/2502.12340>

Mackenzie. 2025. Power transformers and distribution transformers will face supply deficits of 30% and 10% in 2025. <https://www.woodmac.com/press-releases/power-transformers-and-distribution-transformers-will-face-supply-deficits-of-30-and-10-in-2025/>

Maini, Seto, Bai, Grangier, Zhang, Jaitly. 2024. Rephrasing the web: a recipe for compute and data-efficient language modeling. <https://arxiv.org/abs/2401.16380>

Malkov, Yashunin. 2018. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. IEEE Transactions on Pattern Analysis and Machine Intelligence. <https://arxiv.org/abs/1603.09320>

McCandlish, Kaplan, Amodei, Team. 2018. An empirical model of large-batch training. arXiv preprint arXiv:1812.06162. <https://arxiv.org/abs/1812.06162>

Mei, Yao, Ge, Wang, Bi, Cai, Liu, Li, Li, Zhang, Zhou, Zhang, Yan, Wang, Pan, Yan, others. 2025. A survey of context engineering for large language models. arXiv preprint arXiv:2507.13334. <https://arxiv.org/abs/2507.13334>

Meng, Xia, Chen. 2024. SimPO: Simple preference optimization with a reference-free reward. <https://arxiv.org/abs/2405.14734>

Meta. 2025. Collective communication for 100k+ gpus. arXiv preprint arXiv:2510.20171. <https://arxiv.org/abs/2510.20171>

METR. 2025. Measuring the impact of early-2025 AI on experienced open-source developer productivity. arXiv preprint arXiv:2507.09089. <https://arxiv.org/abs/2507.09089>

METR. 2026. Time horizon 1.1. <https://metr.org/blog/2026-1-29-time-horizon-1-1/>

METR. 2026. A simpler AI timelines model predicts 99% AI R&D automation in 2032. <https://metr.org/notes/2026-02-10-simpler-ai-timelines-model/>

Mialon, Fourier, Swift, Wolf, LeCun, Scialom. 2023. GAIA: a benchmark for general AI assistants. arXiv preprint arXiv:2311.12983. <https://arxiv.org/abs/2311.12983>

Micikevicius, Narang, Alben, Diamos, Elsen, Garcia, Ginsburg, Houston, Kuchaiev, Venkatesh, Wu. 2017. Mixed precision training. <https://arxiv.org/abs/1710.03740>

Micikevicius, Stosic, Burgess, Cornea, Dubey, Grisenthwaite, Ha, Heinecke, Judd, Kamalu, Mellempudi, Oberman, Shoeybi, Siu, Wu. 2022. FP8 formats for deep learning. <https://arxiv.org/abs/2209.05433>

Mohan, Phanishayee, Chidambaram. 2021. CheckFreq: Frequent, fine-grained DNN checkpointing. <https://www.usenix.org/conference/fast21/presentation/mohan>

Motamed, Culp, others. 2025. Do generative video models understand physical principles?. arXiv preprint arXiv:2501.09038. <https://arxiv.org/abs/2501.09038>

Muennighoff, Rush, Barak, Le Scao, Piktus, Tazi, Pyysalo, Wolf, Raffel. 2023. Scaling data-constrained language models. arXiv preprint arXiv:2305.16264.

<https://arxiv.org/abs/2305.16264>

Muennighoff, Yang, Shi, Li, Fei-Fei, Hajishirzi, Zettlemoyer, Liang, Candès, Hashimoto. 2025. s1: Simple test-time scaling. arXiv preprint arXiv:2501.19393. <https://arxiv.org/abs/2501.19393>

Narayanan, Harlap, Phanishayee, Seshadri, Devanur, Ganger, Gibbons, Zaharia. 2019. PipeDream: Generalized pipeline parallelism for DNN training. <https://doi.org/10.1145/3341301.3359646>

Narayanan, Shoeybi, Casper, LeGresley, Patwary, Korthikanti, Vainbrand, Kashinkunti, Bernauer, Catanzaro, Phanishayee, Zaharia. 2021. Efficient large-scale language model training on GPU clusters using megatron-LM. <https://arxiv.org/abs/2104.04473>

NucNet. 2025. Constellation secures billion federal loan for three mile island restart. <https://www.nucnet.org/news/constellation-secures-usd1-billion-federal-loann-for-three-mile-island-restart-11-3-2025>

NVIDIA. 2023. TensorRT-LLM. <https://github.com/NVIDIA/TensorRT-LLM>

NVIDIA. 2024. NCCL: NVIDIA collective communications library. <https://github.com/NVIDIA/nccl>

NVIDIA. 2024. Nemotron-4 340B technical report. <https://arxiv.org/abs/2406.11704>

Ogenrwot, Businge. 2026. AgenticFlict: a large-scale dataset of merge conflicts in AI coding agent pull requests on GitHub. <https://arxiv.org/abs/2604.03551>

Okta. 2025. Okta for AI agents, early access. <https://www.okta.com/blog/ai/okta-ai-agents-early-access-announcement/>

Olah, Cammarata, Schubert, Goh, Petrov, Carter. 2020. Zoom in: An introduction to circuits. Distill. <https://distill.pub/2020/circuits/zoom-in/>

Olsson, Elhage, Nanda, Joseph, DasSarma, Henighan, Mann, Askell, Bai, Chen, Conerly, Drain, Ganguli, Hatfield-Dodds, Hernandez, Johnston, Jones, Kernion, Lovitt, Ndousse, Amodei, Brown, Clark, Kaplan, McCandlish, Olah. 2022. In-context learning and induction heads. <https://transformer-circuits.pub/2022/in-context-learning-and-induction-heads/index.html>

OpenAI. 2023. GPT-4 technical report. <https://arxiv.org/abs/2303.08774>

OpenAI. 2024. Learning to reason with llms. <https://openai.com/index/learning-to-reason-with-llms/>

OpenAI. 2025. Sycophancy in GPT-4o: What happened and what we're doing about it. <https://openai.com/index/sycophancy-in-gpt-4o/>

OpenAI. 2025. GDPval: Evaluating AI model performance on real-world economically valuable tasks. arXiv preprint arXiv:2510.04374. <https://arxiv.org/abs/2510.04374>

OpenAI. 2026. Codex CLI: Conversation and code-reverting rewind. <https://github.com/openai/codex/issues/11626>

Ou. 2026. Goalless agents. <https://changkun.de/blog/posts/goalless-agents>

Ouyang, Wu, Jiang, Almeida, Wainwright, Mishkin, Zhang, Agarwal, Slama, Ray, Schulman, Hilton, Kelton, Miller, Simens, Askell, Welinder, Christiano, Leike, Lowe. 2022. Training language models to follow instructions with human feedback. Advances in Neural Information Processing Systems (NeurIPS). <https://arxiv.org/abs/2203.02155>

Ouyang, Wu, Jiang, Almeida, Wainwright, Mishkin, Zhang, Agarwal, Slama, Ray, Schulman, Hilton, Kelton, Miller, Simens, Askell, Welinder, Christiano, Leike, Lowe. 2022. Training language models to follow instructions with human feedback. <https://arxiv.org/abs/2203.02155>

Packer, Wooders, Lin, Fang, Patil, Stoica, Gonzalez. 2023. MemGPT: Towards llms as operating systems. <https://arxiv.org/abs/2310.08560>

Pai. 2025. Designing large language model applications: a holistic approach. O'Reilly Media. <https://www.oreilly.com/library/view/designing-large-language/9781098150495/>

Panickssery, Bowman, Feng. 2024. LLM evaluators recognize and favor their own generations. arXiv preprint arXiv:2404.13076. <https://arxiv.org/abs/2404.13076>

Penedo, Malartic, Hesslow, Cojocar, Cappelli, Alobeidli, Pannier, Almazrouei, Launay. 2023. The RefinedWeb dataset for falcon LLM: Outperforming curated corpora with web data, and web data only. <https://arxiv.org/abs/2306.01116>

Penedo, Kydliček, Ben Allal, Lozhkov, Mitchell, Raffel, Von Werra, Wolf. 2024. The FineWeb datasets: Decanting the web for the finest text data at scale. <https://arxiv.org/abs/2406.17557>

Petrov, La Malfa, Torr, Bibi. 2023. Language model tokenizers introduce unfairness between languages. https://proceedings.neurips.cc/paper_files/paper/2023/hash/74bb24dca8334adce292883b4b651eda-Abstract-Conference.html

pgvector. 2025. pgvector: Open-source vector similarity search for postgres. <https://github.com/pgvector/pgvector>

Pinecone. 2025. Pinecone: Managed vector database for AI. <https://www.pinecone.io/>

Poly. 2026. Durable execution: The key to harnessing AI agents in production. <https://www.inngest.com/blog/durable-execution-key-to-harnessing-ai-agents>

Pope, Douglas, Chowdhery, Devlin, Bradbury, Levskaya, Heek, Xiao, Agrawal, Dean. 2022. Efficiently scaling transformer inference. arXiv preprint arXiv:2211.05102. <https://arxiv.org/abs/2211.05102>

Press, Wolf. 2017. Using the output embedding to improve language models. <https://arxiv.org/abs/1608.05859>

Press, Smith, Lewis. 2022. Train short, test long: Attention with linear biases enables input length extrapolation. <https://arxiv.org/abs/2108.12409>

project. 2023. GGUF file format. <https://github.com/ggml-org/llama.cpp>

Protocol. 2025. MCP transport future: Streamable HTTP replaces SSE. <https://blog.modelcontextprotocol.io/posts/2025-12-19-mcp-transport-future/>

Protocol. 2025. Transports: Mcp-Session-Id and Last-Event-ID. <https://modelcontextprotocol.io/specification/2025-11-25/basic/transports>

Protocol. 2026. The 2026 MCP roadmap. <https://blog.modelcontextprotocol.io/posts/2026-mcp-roadmap/>

Qin, Liang, Ye, Zhu, Yan, Lu, Lin, Cong, Tang, Qian, Zhao, Hong, Tian, Xie, Zhou, Gerstein, Li, Liu, Sun. 2023. ToolLLM: Facilitating large language models to master 16000+ real-world apis. <https://arxiv.org/abs/2307.16789>

Qin, Li, He, Zhang, Wu, Zheng, Xu. 2024. Mooncake: a kvcache-centric disaggregated architecture for LLM serving. arXiv preprint arXiv:2407.00079. <https://arxiv.org/abs/2407.00079>

Rack2Cloud. 2026. Kubernetes day 2 failures: 5 incidents and the metrics that predict them. <https://www.rack2cloud.com/kubernetes-day-2-failures/>

Rafailov, Sharma, Mitchell, Ermon, Manning, Finn. 2023. Direct preference optimization: Your language model is secretly a reward model. <https://arxiv.org/abs/2305.18290>

Raffel, Shazeer, Roberts, Lee, Narang, Matena, Zhou, Li, Liu. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. Journal of Machine Learning Research. <https://www.jmlr.org/papers/v21/20-074.html>

Rajbhandari, Rasley, Ruwase, He. 2019. ZeRO: Memory optimizations toward training trillion parameter models. <https://arxiv.org/abs/1910.02054>

Rajbhandari, Ruwase, Rasley, Smith, He. 2021. ZeRO-infinity: Breaking the GPU memory wall for extreme scale deep learning. <https://arxiv.org/abs/2104.07857>

Rand, Seel, Wiser. 2025. Queued up: 2025 edition. Characteristics of power plants seeking transmission interconnection. <https://emp.lbl.gov/publications/queued-2025-edition-characteristics>

Raschka. 2024. Build a large language model (from scratch). Manning Publications. <https://www.manning.com/books/build-a-large-language-model-from-scratch>

RDI. 2025. Trustworthy benchmarks for agents. <https://rdi.berkeley.edu/blog/trustworthy-benchmarks-cont/>

Rein, Hou, Stickland, Petty, Pang, Dirani, Michael, Bowman. 2023. GPQA: a graduate-level google-proof Q&a benchmark. arXiv preprint arXiv:2311.12022. <https://arxiv.org/abs/2311.12022>

Relations. 2025. China's AI chip deficit: Why huawei can't catch nvidia and U.S. export controls should remain. <https://www.cfr.org/articles/chinas-ai-chip-deficit-why-huawei-cant-catch-nvidia-and-us-export-controls-should-remain>

Replit. 2025. Checkpoints and rollbacks. <https://docs.replit.com/replitai/checkpoints-and-rollbacks>

Replit. 2025. Inside replit's snapshot engine. <https://blog.replit.com/inside-replits-snapshot-engine>

Replit. 2025. Finding and solving memory leaks. <https://blog.replit.com/finding-and-solving-memory-leaks>

Research. 2024. LOCOMO: Long-term conversational memory benchmark. <https://snap-research.github.io/locomo/>

Research. 2025. Scaling LLM reinforcement learning with prolonged training using ProRL v2. arXiv preprint arXiv:2505.24864. <https://arxiv.org/abs/2505.24864>

Review. 2025. Measuring what matters: Construct validity in large language model benchmarks. <https://openreview.net/forum?id=mdA5IVvNcU>

Sardana, Portes, Doubov, Frankle. 2024. Beyond chinchilla-optimal: Accounting for inference in language model scaling laws. <https://arxiv.org/abs/2401.00448>

Schick, Dwivedi-Yu, Dessi, Raileanu, Lomeli, Zettlemoyer, Cancedda, Scialom. 2023. Toolformer: Language models can teach themselves to use tools. <https://arxiv.org/abs/2302.04761>

Schulman, Wolski, Dhariwal, Radford, Klimov. 2017. Proximal policy optimization algorithms. arXiv preprint arXiv:1707.06347. <https://arxiv.org/abs/1707.06347>

Security. 2026. Revision to license review policy for advanced computing commodities. <https://www.federalregister.gov/documents/2026/01/15/2026-00789/revision-to-license-review-policy-for-advanced-computing-commodities>

Seed. 2025. Robust LLM training infrastructure at ByteDance (ByteRobust). arXiv preprint arXiv:2509.16293. <https://arxiv.org/abs/2509.16293>

Sennrich, Haddow, Birch. 2015. Neural machine translation of rare words with subword units. arXiv preprint arXiv:1508.07909. <https://arxiv.org/abs/1508.07909>

Services. 2025. Firecracker snapshotting. <https://github.com/firecracker-microvm/firecracker/blob/main/docs/snapshotting/snapshot-support.md>

Seyedi, NVIDIA. 2025. Scaling AI factories with co-packaged optics for better power efficiency. <https://developer.nvidia.com/blog/scaling-ai-factories-with-co-packaged-optics-for-better-power-efficiency/>

Shah, Bikshandi, Zhang, Thakkar, Ramani, Dao. 2024. FlashAttention-3: Fast and accurate attention with asynchrony and low-precision. <https://arxiv.org/abs/2407.08608>

Shao, Wang, Zhu, Xu, Song, Bi, Zhang, Zhang, Li, Wu, Guo. 2024. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. arXiv preprint arXiv:2402.03300. <https://arxiv.org/abs/2402.03300>

Shazeer, Mirhoseini, Maziarz, Davis, Le, Hinton, Dean. 2017. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. <https://arxiv.org/abs/1701.06538>

Shazeer. 2019. Fast transformer decoding: One write-head is all you need. <https://arxiv.org/abs/1911.02150>

Shazeer. 2020. GLU variants improve transformer. <https://arxiv.org/abs/2002.05202>

Shehabi, Smith, Hubbard, Newkirk, Lei, Siddik, Holecek, Koomey, Masanet, Sartor. 2024. 2024 united states data center energy usage report. <https://escholarship.org/uc/item/32d6m0d1>

Shi, Ajith, Xia, Huang, Liu, Blevins, Chen, Zettlemoyer. 2023. Detecting pretraining data from large language models. arXiv preprint arXiv:2310.16789. <https://arxiv.org/abs/2310.16789>

Shi, Ma, Liang, Diao, Ma, Vosoughi. 2025. Judging the judges: a systematic study of position bias in LLM-as-a-judge. <https://aclanthology.org/2025.ijcnlp-long.18/>

Shilov. 2026. TSMC 'super carrier' CoWoS interposer enables 9-reticle packages with 12 HBM4 stacks. <https://www.tomshardware.com/tech-industry/tsmc-super-carrier-cowos-interposer-gets-bigger-enabling-massive-ai-chips-to-reach-9-reticle-sizes-with-12-hbm4-stacks>

Shoeybi, Patwary, Puri, LeGresley, Casper, Catanzaro. 2019. Megatron-LM: Training multi-billion parameter language models using model parallelism. <https://arxiv.org/abs/1909.08053>

Shumailov, Shumaylov, Zhao, Papernot, Anderson, Gal. 2024. AI models collapse when trained on recursively generated data. Nature. <https://www.nature.com/articles/s41586-024-07566-y>

Silent Data Corruption Study. 2026. The anatomy of silent data corruption: GPU error pattern study and modeling guidance. arXiv preprint arXiv:2605.04213. <https://arxiv.org/abs/2605.04213>

Singh. 2025. AWS outage and the kiro AI bot: a post-mortem. <https://singhajit.com/aws-outage-kiro-ai-bot/>

Singh, Ehtesham, Kumar, Khoei. 2025. Agentic retrieval-augmented generation: a survey on agentic RAG. arXiv preprint arXiv:2501.09136. <https://arxiv.org/abs/2501.09136>

Snell, Lee, Xu, Kumar. 2024. Scaling LLM test-time compute optimally can be more effective than scaling model parameters. arXiv preprint arXiv:2408.03314. <https://arxiv.org/abs/2408.03314>

Study. 2025. The illusion of diminishing returns: Measuring long horizon execution in llms. arXiv preprint arXiv:2509.09677. <https://arxiv.org/abs/2509.09677>

Study. 2026. Beyond pass@1: a reliability science framework for long-horizon LLM agents. arXiv preprint arXiv:2603.29231. <https://arxiv.org/abs/2603.29231>

Su, Lu, Pan, Murtadha, Wen, Liu. 2021. RoFormer: Enhanced transformer with rotary position embedding. <https://arxiv.org/abs/2104.09864>

Swinhoe. 2026. TSMC announces 2026 capex spend of \$56bn as CEO dismisses bubble concerns. <https://www.datacenterdynamics.com/en/news/tsmc-announces-2026-capex-spend->

of-56bn-after-posting-eighth-consecutive-quarter-of-growth/
Systems. 2024. Cerebras announces third-generation wafer-scale engine (WSE-3).
<https://www.cerebras.ai/press-release/cerebras-announces-third-generation-wafer-scale-engine>
Team. 2025. 2 olmo 2 furious. <https://arxiv.org/abs/2501.00656>
Templeton, Conerly, Marcus, Lindsey, Bricken, Chen, Pearce, Citro, Ameisen, Jones, Cunningham, Turner, McDougall, MacDiarmid, Freeman, Sumers, Rees, Batson, Jermyn, Carter, Olah, Henighan. 2024. Scaling monosemanticity: Extracting interpretable features from claude 3 sonnet. <https://transformer-circuits.pub/2024/scaling-monosemanticity/index.html>
Touvron, Lavril, Izacard, Martinet, Lachaux, Lacroix, Rozière, Goyal, Hambro, Azhar, Rodriguez, Joulin, Grave, Lample. 2023. LLaMA: Open and efficient foundation language models. <https://arxiv.org/abs/2302.13971>
TrendForce. 2026. TSMC sees AI wafer demand rising 11x from 2022–2026, targets CoWoS with 24 HBM stacks in 2029. <https://www.trendforce.com/news/2026/05/14/news-tsmc-sees-ai-wafer-demand-rising-11x-from-2022-2026-targets-cowos-with-24-hbm-stacks-in-2029/>
Uesato, Kushman, Kumar, Song, Siegel, Wang, Creswell, Irving, Higgins. 2022. Solving math word problems with process- and outcome-based feedback. arXiv preprint arXiv:2211.14275. <https://arxiv.org/abs/2211.14275>
Vaswani, Shazeer, Parmar, Uszkoreit, Jones, Gomez, Kaiser, Polosukhin. 2017. Attention is all you need. <https://arxiv.org/abs/1706.03762>
Villalobos, Ho, Sevilla, Besiroglu, Heim, Hobbhahn. 2024. Will we run out of data? Limits of LLM scaling based on human-generated data. arXiv preprint arXiv:2211.04325. <https://arxiv.org/abs/2211.04325>
Wang, Wei, Schuurmans, Le, Chi, Narang, Chowdhery, Zhou. 2022. Self-consistency improves chain of thought reasoning in language models. <https://arxiv.org/abs/2203.11171>
Wang, Li, Chen, Cai, Zhu, Lin, Cao, Liu, Liu, Sui. 2023. Large language models are not fair evaluators. arXiv preprint arXiv:2305.17926. <https://arxiv.org/abs/2305.17926>
Wang, Gao, Zhao, Sun, Dai. 2024. Auxiliary-loss-free load balancing strategy for mixture-of-experts. <https://arxiv.org/abs/2408.15664>
Waxell. 2025. The \$47,000 agent loop: Why token budget alerts aren't budget enforcement. <https://dev.to/waxell/the-47000-agent-loop-why-token-budget-alerts-arent-budget-enforcement-389i>
Wei, Wang, Schuurmans, Bosma, Ichter, Xia, Chi, Le, Zhou. 2022. Chain-of-thought prompting elicits reasoning in large language models. <https://arxiv.org/abs/2201.11903>
Wei, Sun, Papay, McKinney, Han, Fulford, Chung, Passos, Fedus, Glaese. 2025. BrowseComp: a simple yet challenging benchmark for browsing agents. arXiv preprint arXiv:2504.12516. <https://arxiv.org/abs/2504.12516>
Weng. 2023. LLM powered autonomous agents. <https://lilianweng.github.io/posts/2023-06-23-agent/>
Wenzek, Lachaux, Conneau, Chaudhary, Guzmán, Joulin, Grave. 2020. CCNet: Extracting high quality monolingual datasets from web crawl data. <https://aclanthology.org/2020.lrec-1.494/>
Wijk, others. 2025. RE-bench: Evaluating frontier AI R&D capabilities of language model agents against human experts. arXiv preprint arXiv:2411.15114. <https://arxiv.org/abs/2411.15114>
Willard, Louf. 2023. Efficient guided generation for large language models. arXiv preprint arXiv:2307.09702. <https://arxiv.org/abs/2307.09702>
WorkOS. 2025. Maxim fateev on durable execution for AI agents. <https://workos.com/blog/maxim-fateev-temporal-durable-execution-ai-agents>
Wortsman, Ilharco, Gadre, Roelofs, Gontijo-Lopes, Morcos, Namkoong, Farhadi, Carmon, Kornblith, Schmidt. 2022. Model soups: Averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. <https://arxiv.org/abs/2203.05482>
Xiao, Lin, Seznec, Wu, Demouth, Han. 2022. SmoothQuant: Accurate and efficient post-training quantization for large language models. <https://arxiv.org/abs/2211.10438>
Xiao, Tian, Chen, Han, Lewis. 2024. Efficient streaming language models with attention sinks. <https://arxiv.org/abs/2309.17453>
Xie, Pham, Dong, Du, Liu, Lu, Liang, Le, Ma, Yu. 2023. DoReMi: Optimizing data mixtures speeds up language model pretraining. <https://openreview.net/forum?id=IXuByUeHhd>
Xie, Kawaguchi, Zhao, Zhao, Kan, He, Xie. 2023. Self-evaluation guided beam search for reasoning. arXiv preprint arXiv:2305.00633. <https://arxiv.org/abs/2305.00633>

Xie, Zhang, Chen, Li, Zhao, Cao, Hua, Cheng, Shin, Lei, Liu, Xu, Zhou, Savarese, Xiong, Zhong, Yu. 2024. OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments. arXiv preprint arXiv:2404.07972. <https://arxiv.org/abs/2404.07972>

Xiong, Yang, He, Zheng, Zheng, Xing, Zhang, Lan, Wang, Liu. 2020. On layer normalization in the transformer architecture. <https://arxiv.org/abs/2002.04745>

Xu, Lee, Chen, Hechtman, Huang, Joshi, Krikun, Lepikhin, Ly, Maggioni, Pang, Shazeer, Wang, Wang, Wu, Chen. 2021. GSPMD: General and scalable parallelization for ML computation graphs. <https://arxiv.org/abs/2105.04663>

Yadav, Tam, Choshen, Raffel, Bansal. 2023. TIES-merging: Resolving interference when merging models. Advances in Neural Information Processing Systems (NeurIPS). <https://arxiv.org/abs/2306.01708>

Yan, Gu, Zhu, Ling. 2024. Corrective retrieval augmented generation. arXiv preprint arXiv:2401.15884. <https://arxiv.org/abs/2401.15884>

Yan, Mao, Ji, Zhang, Patil, Stoica, Gonzalez. 2025. The berkeley function calling leaderboard (BFCL): From tool use to agentic evaluation of large language models. <https://openreview.net/forum?id=2GmDdhBdDk>

Yang, Hu, Babuschkin, Sidor, Liu, Farhi, Ryder, Pachocki, Chen, Gao. 2022. Tensor programs V: Tuning large neural networks via zero-shot hyperparameter transfer. arXiv preprint arXiv:2203.03466. <https://arxiv.org/abs/2203.03466>

Yang, Chiang, Zheng, Gonzalez, Stoica. 2023. Rethinking benchmark and contamination for language models with rephrased samples. <https://arxiv.org/abs/2311.04850>

Yang, others. 2025. Qwen3 technical report. <https://arxiv.org/abs/2505.09388>

Yao, Yu, Zhao, Shafran, Griffiths, Cao, Narasimhan. 2023. Tree of thoughts: Deliberate problem solving with large language models. <https://arxiv.org/abs/2305.10601>

Yao, Zhao, Yu, Du, Shafran, Narasimhan, Cao. 2023. ReAct: Synergizing reasoning and acting in language models. <https://arxiv.org/abs/2210.03629>

Yao, Shinn, Razavi, Narasimhan. 2024. τ -bench: a benchmark for tool-agent-user interaction in real-world domains. arXiv preprint arXiv:2406.12045. <https://arxiv.org/abs/2406.12045>

Ye, Huang, Xiao, Chern, Xia, Liu. 2025. LIMO: Less is more for reasoning. arXiv preprint arXiv:2502.03387. <https://arxiv.org/abs/2502.03387>

Yin, Zheng, others. 2024. Fast JSON decoding for local llms with compressed finite state machine. <https://www.lmsys.org/blog/2024-02-05-compressed-fsm/>

Yu, Jeong, Kim, Kim, Chun. 2022. Orca: a distributed serving system for transformer-based generative models. <https://www.usenix.org/conference/osdi22/presentation/yu>

Yu, Yu, Yu, Huang, Li. 2024. Language models are super mario: Absorbing abilities from homologous models as a free lunch. <https://arxiv.org/abs/2311.03099>

Yuan, Shang, Zhou, Dong, Zhou, Xue, Wu, Li, Gu, Lee, Yan, Chen, Sun, Keutzer. 2024. LLM inference unveiled: Survey and roofline model insights. arXiv preprint arXiv:2402.16363. <https://arxiv.org/abs/2402.16363>

Yue, Chen, Lu, Zhao, Wang, Yue, Song, Huang. 2025. Does reinforcement learning really incentivize reasoning capacity in llms beyond the base model?. arXiv preprint arXiv:2504.13837. <https://arxiv.org/abs/2504.13837>

Zep. 2025. Zep: Temporal knowledge graph for agent memory. <https://www.getzep.com/>

Zhang, Sennrich. 2019. Root mean square layer normalization. <https://arxiv.org/abs/1910.07467>

Zhang, Sheng, Zhou, Chen, Zheng, Cai, Song, Tian, Ré, Barrett, Wang, Chen. 2023. H2O: Heavy-hitter oracle for efficient generative inference of large language models. <https://arxiv.org/abs/2306.14048>

Zhao, Gu, Varma, Luo, Huang, Xu, Wright, Shojanazeri, Ott, Shleifer, Desmaison, Balioglu, Damania, Nguyen, Chauhan, Hao, Mathews, Li. 2023. PyTorch FSDP: Experiences on scaling fully sharded data parallel. <https://arxiv.org/abs/2304.11277>

Zheng, Yin, Xie, Sun, Huang, Yu, Cao, Kozyrakis, Stoica, Gonzalez, Barrett, Sheng. 2023. SGLang: Efficient execution of structured language model programs. <https://arxiv.org/abs/2312.07104>

Zheng, Chiang, Sheng, Zhuang, Wu, Zhuang, Lin, Li, Li, Xing, Zhang, Gonzalez, Stoica. 2023. Judging LLM-as-a-judge with MT-bench and chatbot arena. arXiv preprint arXiv:2306.05685. <https://arxiv.org/abs/2306.05685>

Zheng, Yin, Xie, Sun, Huang, Yu, Cao, Kozyrakis, Stoica, Gonzalez, Barrett, Sheng. 2024. SGLang: Efficient execution of structured language model programs. <https://arxiv.org/abs/2312.07104>

Zhong, Liu, Chen, Hu, Zhu, Liu, Jin, Zhang. 2024. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. <https://arxiv.org/abs/2401.09670>

Zhou, Lei, Liu, Du, Huang, Zhao, Dai, Chen, Le, Laudon. 2022. Mixture-of-experts with expert choice routing. <https://arxiv.org/abs/2202.09368>

Zhou, Schärli, Hou, Wei, Scales, Wang, Schuurmans, Cui, Bousquet, Le, Chi. 2022. Least-to-most prompting enables complex reasoning in large language models. <https://arxiv.org/abs/2205.10625>

Zhou, Liu, Xu, Iyer, Sun, Mao, Ma, Efrat, Yu, Yu, Zhang, Ghosh, Lewis, Zettlemoyer, Levy. 2023. LIMA: Less is more for alignment. arXiv preprint arXiv:2305.11206. <https://arxiv.org/abs/2305.11206>

Zhou, Xu, Zhu, Zhou, Lo, Sridhar, Cheng, Ou, Bisk, Fried, Alon, Neubig. 2023. WebArena: a realistic web environment for building autonomous agents. arXiv preprint arXiv:2307.13854. <https://arxiv.org/abs/2307.13854>

Zhou, others. 2026. Externalization in LLM agents: a unified review of memory, skills, protocols and harness engineering. <https://arxiv.org/abs/2604.08224>

Zoph, Bello, Kumar, Du, Huang, Dean, Shazeer, Fedus. 2022. ST-MoE: Designing stable and transferable sparse expert models. <https://arxiv.org/abs/2202.08906>